AnovaCraft Reference Manual

Generated 2026-10-24 from the AnovaCraft repository

Contents

Introduction

Getting started

1. Installing and running
2. A tour of the window
3. Your first analysis
4. Using analysis dialogs
5. Sample data
6. Crash reports and recovery

Working with data

7. Documents, files and projects
8. The Open Data dialog
9. Images as tables
10. Editing the data grid
11. Dates and times
12. Copying and pasting
13. Formula columns
14. Table operations
15. Data from databases and the web
16. Missing values and imputation
17. Multiple imputation
18. Row states: select, exclude, hide

Reports and plots

19. Reading a report
20. Working with plots
21. Saving plots and results
22. Response Explorer and 3-D profiler
23. Graph canvas
24. Figure page
25. 3-D scatter, contour and surface plots
26. Image plots
27. Dashboards

Analyses

28. Describe
29. Compare

30. Resampling: bootstrap and permutation tests
31. Model: linear and response-surface models
32. Model: generalized, mixed and penalized models
33. Bayesian analysis
34. Model: decision trees and validation
35. Model: random forests and boosted trees
36. Model: partial least squares
37. Model: support vector machines
38. Model: adaptive regression splines
39. Model: neural networks
40. Model: Gaussian processes and sequential optimization
41. Model: model search
42. Model: comparing models
43. Model: several responses
44. Model: dose–response, method comparison and shelf life
45. Model: treatment effects from observational data
46. Model: nonlinear regression
47. Model: curve templates
48. Multivariate
49. Multivariate: text analysis
50. Design
51. Quality
52. Reliability
53. Series and Signals: time series
54. Series and Signals: signals and spectra
55. Series and Signals: signal processing
56. Series and Signals: image processing
57. Coming soon

Automation

58. Scripting and recording

Appendix

59. Settings
60. Keeping the manuals current
61. Screenshots
62. Automated GUI tests
63. Web build

Introduction

AnovaCraft is an open-source desktop application for statistical analysis, design of experiments and predictive modelling. You open a data table, choose an analysis from the menus, and get a report of collapsible sections with tables and interactive plots. Selecting rows in any plot or in the data grid highlights the same rows everywhere, so you can follow an observation from a residual plot back to the data and into every other report.

This is the **Reference Manual**: it explains everything the application does in detail, for people who want every option and the method behind each result. If you are new to the app or to statistics, start with the **Quick Guide** (Help ▸ Quick guide), which describes every menu item in plain language and links back here for the details. Both manuals are built into the app and open as PDFs from the Help menu.

This manual is organised the way you work:

- **Getting started** installs and runs the app, walks around the window and runs a first analysis on a sample dataset.
- **Working with data** covers opening and saving files, editing the grid, formula columns, table operations such as sorting, joining and reshaping, data from databases and the web, missing values and multiple imputation, and row states (selecting, excluding and hiding rows).
- **Reports and plots** explains the parts every analysis shares: report sections and their options menus, plot navigation, saving plots and results, the Response Explorer, the graph canvas, figure pages, 3-D plots and dashboards.
- **Analyses** has a chapter per menu (Describe, Compare, Model, Multivariate, Design, Quality, Reliability and Series and Signals), with separate chapters for the larger topics such as Bayesian analysis, each kind of predictive model, nonlinear regression, curve templates and text analysis.
- **Automation** is the reference for Rhai scripts, the Script window and recording your actions as a script.

Every method in AnovaCraft follows a published statistical definition, and each analysis is tested against an independent reference (for example the NIST Statistical Reference Datasets). Chapters name the method and its reference so you can read more about it.

AnovaCraft is an independent implementation built from published statistical literature; it contains no code, scripts or designs from any commercial statistics package and is not

affiliated with their makers. It uses general statistical names for its methods (for example, adaptive regression splines and random forests).

Tip: File ▸ Open sample data lists datasets chosen to show each kind of analysis. Each chapter names the sample that goes with it.

Installing and running

Downloading the app

Ready-built versions are on the project's Releases page. For macOS (11 or later, Apple silicon or Intel), download the `macos-universal .zip`, open it and drag **AnovaCraft.app** to your Applications folder. The app is not signed with an Apple Developer ID, so macOS refuses to open it the first time: try to open it once, then choose **Open Anyway** under System Settings ▸ Privacy & Security (on macOS 14 and earlier you can instead right-click the app and choose **Open**). macOS remembers the choice.

For Windows (64-bit), download the `windows-x64 .zip`, unzip it anywhere and run `AnovaCraft.exe`; there is no installer. Windows SmartScreen may warn about an unrecognized app the first time: choose **More info ▸ Run anyway**.

Building from source

AnovaCraft can also be built from source with Rust. You need the Rust toolchain (install it from https://rustup.rs) and a copy of the repository.

```
git clone https://github.com/anovacraft/AnovaCraft.git
cd AnovaCraft
cargo run --release -p app-gui
```

The first build takes several minutes; later runs start in seconds.

A standalone app

To build an application you can keep in your Applications folder or copy to another computer:

- **macOS:** `./scripts/build-macos-app.sh` writes `AnovaCraft.app` to `dist/` (add `universal` for one app that runs on Apple silicon and Intel). It is built with whole-program optimization (the `dist` build profile), which takes longer than `cargo build`

`--release` but makes the analyses run faster. It also writes a `.zip` of the app for sharing (`AnovaCraft-<version>-macos-universal.zip`). The app is not code-signed, so the first time allow it as described under Downloading the app.

- **Windows:** `./scripts/build-windows-exe.sh`, run on macOS or Linux with the MinGW-w64 cross compiler, writes `dist/AnovaCraft-windows-x64/` and a `.zip` of it (`AnovaCraft-<version>-windows-x64.zip`). Copy the folder to a Windows PC and run `AnovaCraft.exe`; no installer is needed.

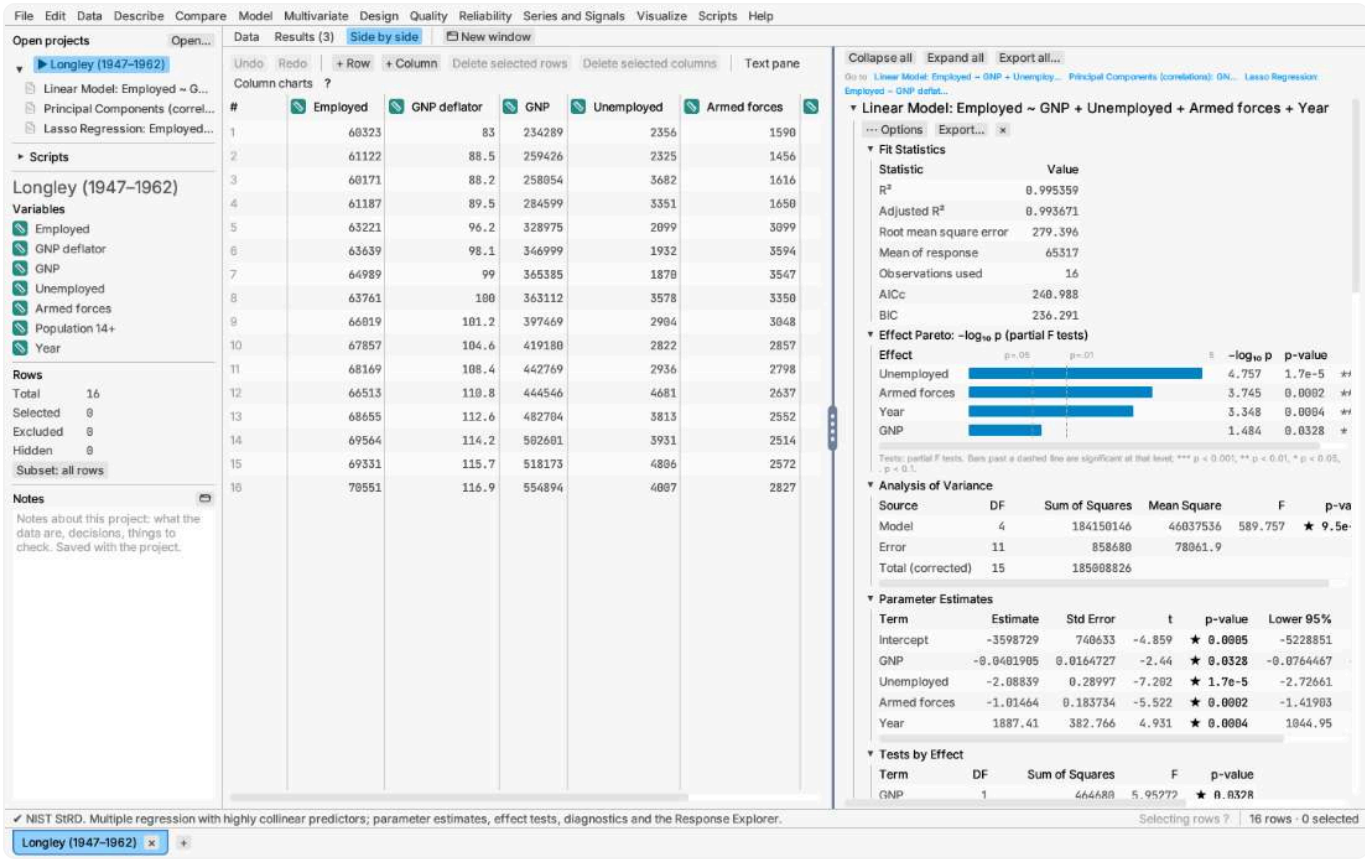
Running scripts without the window

Scripts can also run headless, for batch jobs and scheduled reports:

```
cargo run --release -p stat-script --bin anovacraft-run -- script.rhai
```

See Scripting and recording.

A tour of the window



The window has five parts.

Menu bar. File, Edit, Data, Describe, Compare, Model, Multivariate, Design, Quality, Reliability, Series and Signals, Visualize, Scripts and Help. Every menu is there even before any data is open; menus that need data say "Open a dataset first" until a table is open, and Design works without one, because it creates a new table. **Scripts** holds the Script window, Open script..., Run script over a folder... and action recording (see Scripting and recording). **Help** opens the Quick Guide or this Reference Manual in your browser, lists every keyboard shortcut, and shows the version under About AnovaCraft.

Left sidebar.

- *Open projects* lists every open table, grouped by project when a project holds several. Click one to switch to it; right-click it to save, export or close it. A • after the name means it has unsaved changes.
- *Scripts* (click the heading to open it) lists the scripts in your scripts folder and those kept in the active project; click one to open it, or right-click to run it.

- *Variables* lists the active table's columns. Each shows its measurement level as an icon with a tooltip (a ruler for continuous, a tag for nominal, a numbered list for ordinal) and its data type. Formula columns are marked ƒ. Click a variable to select its column in the grid and scroll to it; right-click it for the same menu as the column's heading; drag it onto a graph canvas shelf. When a table has more than eight columns, a **Filter** box beside the heading shows only the variables whose names contain what you type.
- *Rows* counts the total, selected, excluded and hidden rows.
- *Notes* holds free-text notes about the project: what the data are, decisions, things to check. They are saved in the project file, and the first line shows under Recent projects on the start page. The notes are always those of the project you are working in, so switching to a table of another project switches to its notes. The small window button beside the heading moves the notes into a window of their own (which also follows the project you work in); **Back to sidebar**, or closing that window, puts them back.

Getting-started tips. The first time you open a dataset, a few short tips sit under the workspace tabs. **Got it** hides them for good (File ▸ Settings… can show them again).

Workspace tabs. Above the center area, choose **Data** (the grid), **Results** (the reports for this table, with their count) or **Side by side** (the grid with the reports in a pane beside it, useful for brushing). ⌘1, ⌘2 and ⌘3 (Ctrl+1/2/3 on Windows and Linux) switch between them. **New window**, beside them, moves the grid (or the results) into a window of its own, for example onto a second screen; see Views in windows of their own.

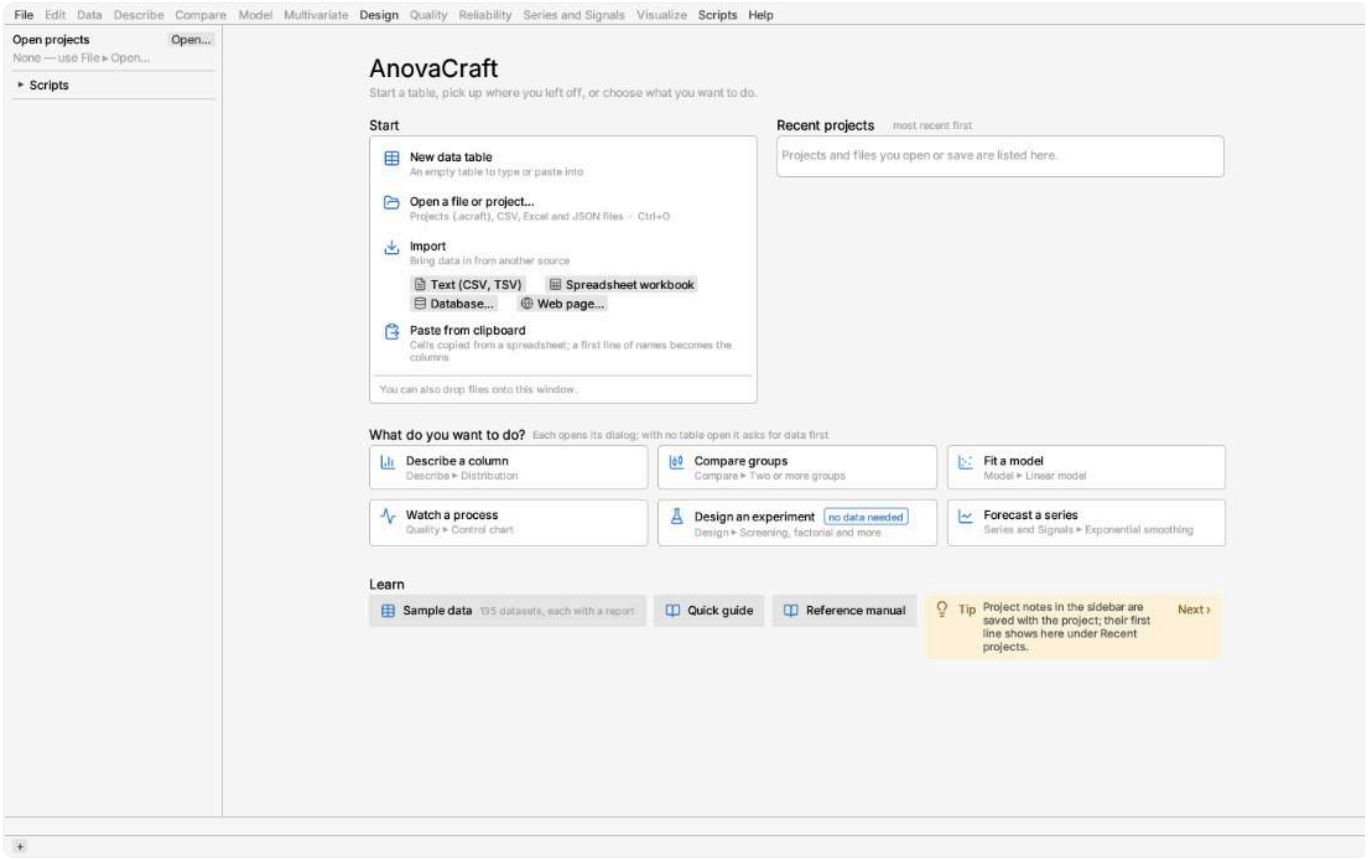
Document tabs. Along the bottom edge, one tab per open document. The × closes it (you are asked first if it has unsaved changes), + opens another file, and right-click offers the same actions as the sidebar. A • after a tab's name, and "(edited)" in the window title, mean the document has unsaved changes.

Status bar. Shows the result of the last action (shortened with "…" when it is longer than the space beside the row counts), ● Recording while actions are being recorded, and **Undo remove** for 30 seconds after a report is removed. Hover "Selecting rows ?" for a reminder of the selection gestures: click or drag in a plot or on the grid's row numbers to select rows; Shift adds (on row numbers, the run of rows between two clicks), ⌘/Ctrl toggles, ⌥/Alt removes. The right end shows the cell at the grid's cursor with its column and value, and how many rows are selected, excluded and hidden.

The start page

The app starts on the start page, and shows it again whenever nothing is open (opening a file from Finder or the command line skips it).

- **Start**: **New data table** (an empty table to type or paste into), **Open a file or project…**, **Import** from delimited text, a spreadsheet workbook, a database or a web address, and **Paste from clipboard** (cells copied from a spreadsheet; a first line of names becomes the column names). You can also drop files onto the window.
- **Recent projects**, most recent first: each project with when you last opened or saved it, its tables and the first line of its notes ("No notes" when it has none); a plain data file shows its rows and columns. Click one to open it. **Show all recent** lists the rest.
- **What do you want to do?**: six common tasks, each naming the menu item it opens (Describe ▸ Distribution, Compare ▸ Two or more groups, Model ▸ Linear model, Quality ▸ Control chart, the Design menu's designs, and Time Series ▸ Exponential smoothing). Designing an experiment needs no data; the others, chosen with no table open, show an **Open data for …** box in front of the start page: **Choose a file…**, **Start an empty table** or one of the samples listed there, after which the task's dialog opens on that table. **Cancel** (or Escape) closes the box and opens nothing.
- **Learn**: **Sample data** lists every sample by topic (see Sample data), **Quick guide** and **Reference manual** open the plain-language guide or this manual as a PDF, and a tip with **Next** for another.



Help ▸ Quick guide and Help ▸ Reference manual open the two manuals. Both are built into the app as PDFs: AnovaCraft writes the one you ask for to the `manuals` folder beside its settings file (in `~/Library/Application Support/Statistical Engine` on macOS, `%APPDATA%\Statistical Engine` on Windows, `~/.config/statistical-engine` on Linux) and opens it in the system's PDF viewer, so they work offline and always describe the version you are running. Each PDF has a linked contents page and a bookmark for every chapter; links into the other manual go to its pages on GitHub. A newer version of the app replaces the old copies the first time you open them. Help ▸ About AnovaCraft also links to both, and to the manuals' pages online. It names AnovaCraft's licence (the Apache License 2.0 or the MIT licence, at your option; `COPYRIGHT`, `LICENSE-APACHE` and `LICENSE-MIT` in the repository), and its **Open-source licences** button shows the third-party notices built into the app: the copyright notices and licence texts of the bundled fonts (Inter, JetBrains Mono, Liberation Sans and Serif, DejaVu Sans and egui's default fonts), the Lucide icons, the VADER sentiment lexicon and every Rust library compiled into the app. The same text is `THIRD-PARTY-NOTICES.txt` in the repository, in the macOS app bundle's Resources folder and beside the Windows program; `anovacraft-run --licences` prints it, and Publish as web app… writes it into the published folder.

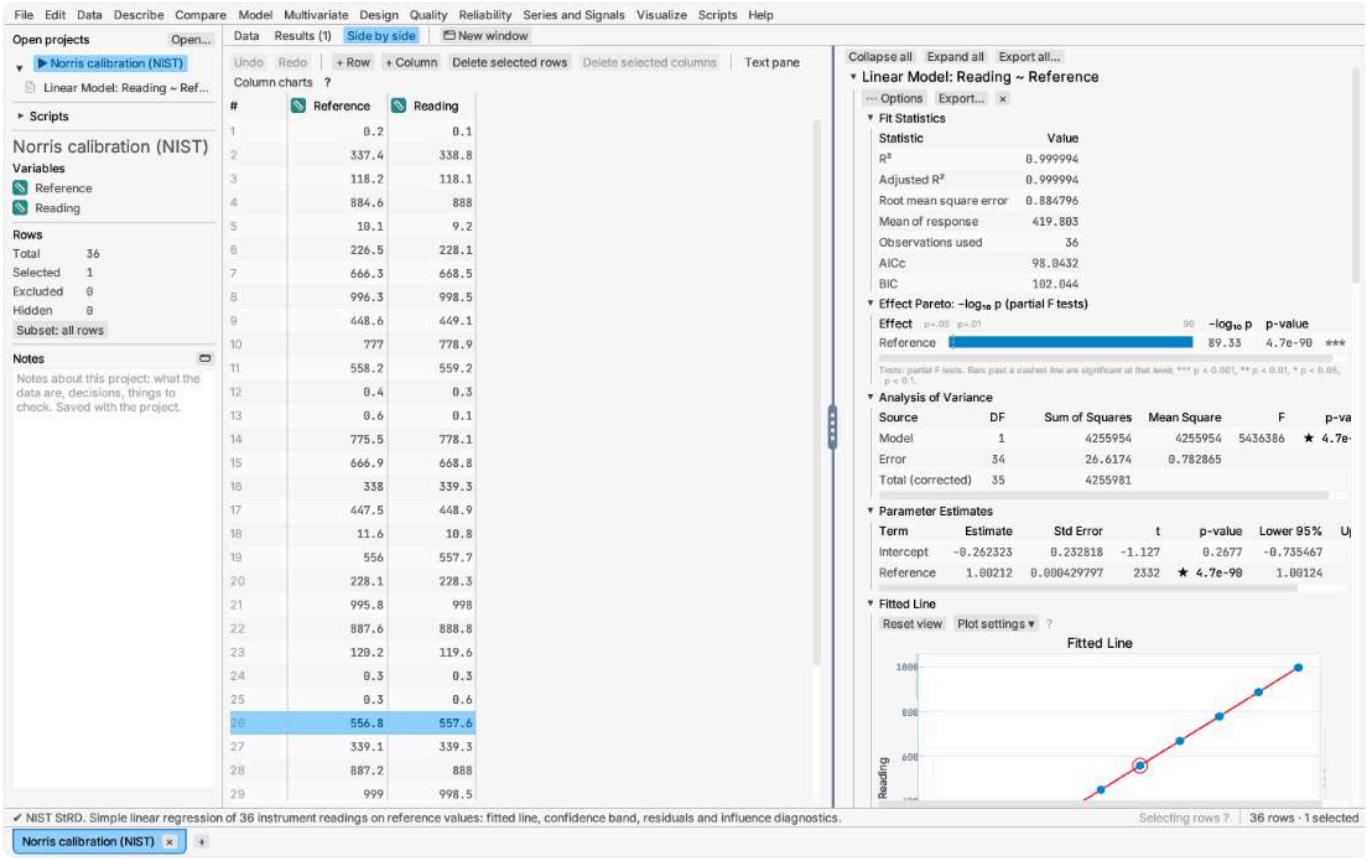
Help ▸ Keyboard shortcuts lists every shortcut. The most used are ⌘O open, ⌘N new table, ⌘S save, ⇧⌘S save as, ⌘W close, ⌘Z undo, ⌘F find, and ⌘1/2/3 for the workspace views

(Ctrl instead of ⌘ on Windows and Linux).

Your first analysis

This walk-through fits a regression to a NIST reference dataset and follows an observation between plots.

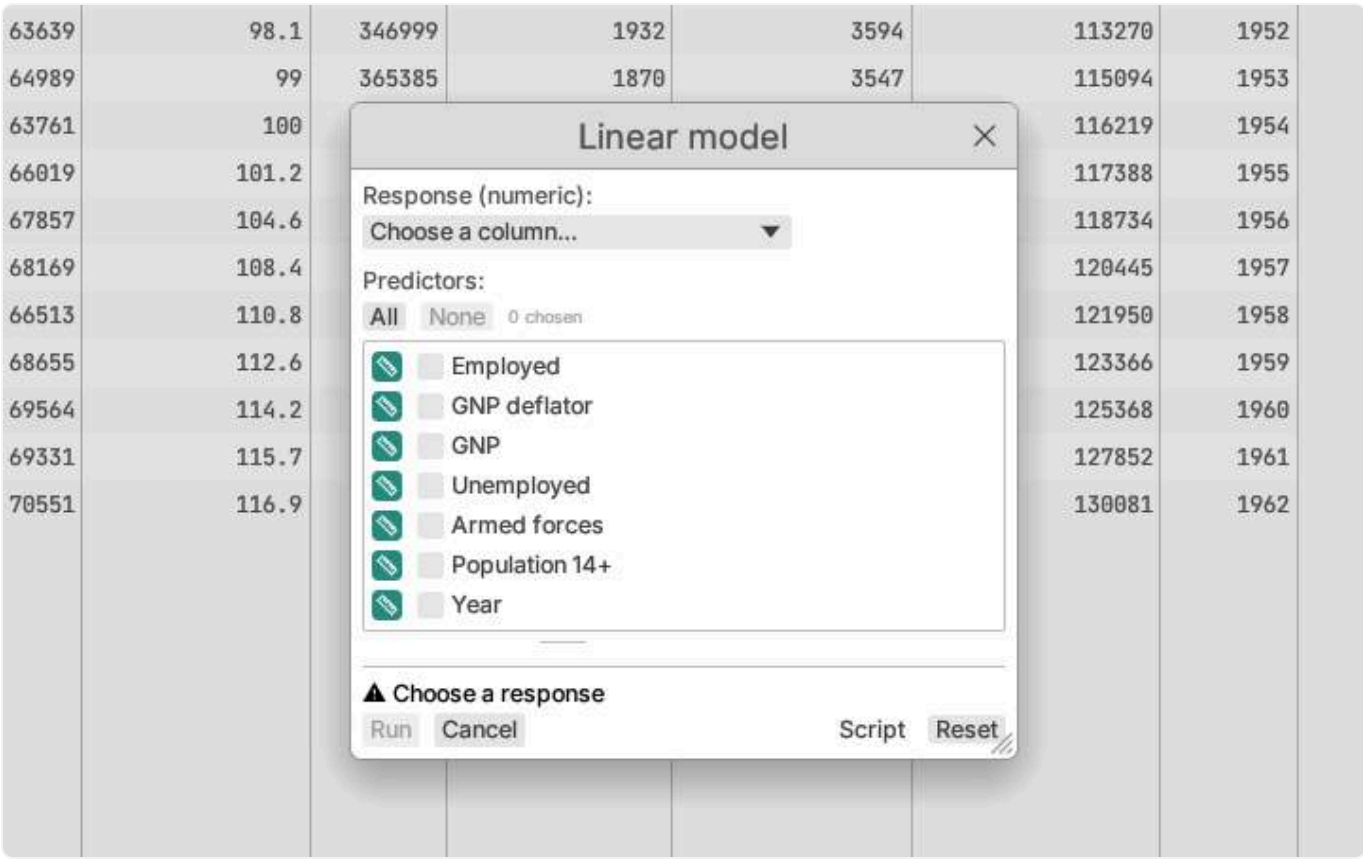
1. **Open the data.** Choose File ▸ Open sample data ▸ Models ▸ *Norris calibration (NIST)*. The table opens in a new document tab.
2. **Look at the columns.** The sidebar lists `Reference` and `Reading`, both continuous (ruler icon). The sample also opens with a ready-made report; you can close it with **Remove report** and fit your own.
3. **Fit a model.** Choose Model ▸ Linear model..., choose `Reading` as the response, tick `Reference` in the predictor list and click **Run**. The Results tab opens with the report.
4. **Read the report.** Fit statistics (R^2, RMSE and others), the Effect Pareto, parameter estimates and effect tests come first, then a fitted line plot. Click a section heading to collapse or expand it.
5. **Add a diagnostic.** Open the report's **⋯ Options** menu (or right-click a section heading) and add the residual plot.
6. **Brush.** Drag a box around the largest residual. The point is ringed in the residual plot, in the fitted line plot and in the data grid (switch to **Side by side** to see the grid next to the report).
7. **Exclude it.** Choose Data ▸ Row states ▸ Exclude selected rows from analyses. The report re-runs without that row; excluded rows are drawn hollow and gray.
8. **Save your work.** Right-click the document in the sidebar and save it as a project (`.acraft`). Reopening the project restores the data, the exclusion and the report exactly.



Next, read Using analysis dialogs, Reading a report and Working with plots, which apply to every analysis.

Using analysis dialogs

Every analysis, design and table operation starts from a dialog. They all work the same way.



Choosing columns

- A drop-down list chooses one column for a role (a response, a grouping column). Each column shows its measurement-level icon; long lists have a filter box at the top. You can also drag a variable from the sidebar onto the list.
- Any list with more entries than it shows (a drop-down list, a check-box list, the script preview) fades out at its cut-off edge under a ▾ (▴ at the top once scrolled down), and its scroll bar is always drawn: scroll or drag the bar to see the rest. Scroll bars in reports, the data grid and the sidebar are always drawn too, wherever there is more to see.
- A check-box list chooses several columns. Type in **Filter columns** to narrow a long list; **All** and **None** select or clear every column listed. Shift-click a box to give every

box between it and the one you clicked before the same state. Drag a variable from the sidebar onto the list to add it. Drag the short bar under the list to make it taller.
- Where two lists sit side by side, the columns being analysed or summarized (responses, measures) are on the left and the columns that group or explain them (grouping columns, factors, classifications) on the right: Summarize by group, Multi-way table, Factorial analysis, Augment design, Signal-to-noise analysis and Declare table as design all follow this order.
- Columns selected in the data grid (click their headings) are filled in when you open a dialog, in its main role: the predictors of a model, the columns of a distribution report, X and Y of a bivariate analysis.

The foot of the dialog

- **Run** (analyses) or **Create** (designs and new tables) does the work. While something is missing, the button is unavailable and the line above it says what to do, marked with ⚠: "Choose a response", "Choose at least one predictor", "Group and data column must differ". Hovering the button shows the same text.
- When the dialog is ready, the line says what it will do and on how many rows, for example "Linear model of Employed on 2 effects · 46 of 48 rows (2 excluded)".
- **Cancel** closes the dialog. **Reset** clears every choice and starts again from the defaults.
- **Script** (analyses) shows the script call the dialog's settings make, updated as you change them, with **Copy script** to put it on the clipboard for the Script window (see Scripting). It stays on for other dialogs until you turn it off.
- **Enter** runs and **Escape** cancels. While you are typing in a field, Enter only finishes the field; press **⌘/Ctrl+Enter** to run from there.

More options

Settings most analyses leave at their defaults (method variants, tuning values, seeds, tolerances and explanatory notes) are in a **More options** section that starts closed. Opening it changes nothing; the defaults are the same either way. A random seed can be any whole number from 0 to $2^{53} - 1$ (9 007 199 254 740 991), the largest a project file stores exactly; a larger seed in a project or script is read as that largest one.

Moving and resizing

Dialogs open in the middle of the window. Drag a dialog's title bar to move it out of the way of the data, and drag its edges to make it larger. It reopens where you left it.

Remembered settings

A dialog remembers what you chose last, for each open document, while the app runs: reopening Linear model after a fit shows the same response and predictors, ready to change one thing. A dialog still open when you switch to another document closes, and its settings are kept for the document it was open on: reopen it there to carry on. Adding columns (a formula or a saved prediction) keeps the remembered settings; inserting, deleting or moving columns forgets them, as does closing the document.

Editing an analysis

Choose **Edit analysis…** at the top of a report's **⋯ Options** menu (or right-click the report's heading). The report's dialog opens with its columns and settings filled in. Change what you need, then:

- **Update** replaces the report in place. It keeps its position, its display options (the sections you turned on or off) and each plot's size and axis settings; plot views re-fit to the new results.
- **Run as new** keeps the report and adds a second one with the new settings.

Graphs are changed on their canvas, so they do not offer Edit analysis…. Reports opened from a project file or a script can be edited too: the dialog is filled from the request the project stores.

Sample data

File ▸ Open sample data lists datasets chosen to demonstrate each kind of analysis, in one submenu per topic: Describe and compare, Models, Curve fitting, Bayesian analysis, Multivariate, Designed experiments, Quality and control charts, Measurement systems, Reliability and survival, Series and signals, and Graphs. The start page's **Sample data** button lists the same groups. Each entry names the dataset and what it shows; hover over it for a longer description. Many samples open with a ready-made report, so you can see the analysis before building your own.

Sets marked NIST come from the NIST Statistical Reference Datasets; sets with a year are classic published datasets; *simulated* sets were generated for the app with a known structure.

Sample	Shows
Longley economy (NIST)	multiple regression, PCA, lasso
Norris calibration (NIST)	simple regression, diagnostics
Plant growth (simulated)	ANCOVA, outliers, mixed models (residual variance by group, robust SE)
Catalyst factorial (simulated)	interactions, quadratic terms, 3-D profiler
Michelson light speed (NIST)	distribution, one-sample tests, tolerance interval
Clinic wait times (simulated)	skewed data, Box–Cox, Welch and rank tests, formulas
Silicon resistivity (NIST)	one-way ANOVA, median test
Customer survey (simulated)	contingency table, correspondence analysis
Fill weights (simulated)	$\bar{X}$/R, phases, EWMA, CUSUM, capability
Beam deflection (NIST)	individuals chart, runs tests
Gauge study (simulated)	gauge R&R, intraclass correlation
Flower measurements (simulated)	PCA, clustering, PLS-DA, correlations, scatterplot matrix, outlier distances, MANOVA

Sample	Shows
Customer churn (simulated)	logistic and Poisson GLM, lasso, profilers, proportions
Leukemia remission (1963)	Kaplan–Meier, log-rank
Bearing life test (simulated)	Weibull and lognormal life fits
Seal inspections (invented)	interval-censored life distributions, two suppliers
Bearing endurance test (invented)	Weibull with a known shape (β = 2): two failures in twelve units
Device-A accelerated life test (Meeker & Escobar 1998)	life–stress model: Arrhenius activation energy, acceleration factors, life at use
Laser degradation (Meeker & Escobar 1998)	degradation paths, pseudo failure times, random-effects paths
Steel fatigue (invented)	fatigue S–N curves with run-outs: Basquin and endurance-limit fits, B10 curve, fatigue strength
Well water arsenic (invented)	censored values (non-detects) in the Distribution report: reversed Kaplan–Meier, censored fits
Adhesive bond strength (simulated)	destructive degradation with an Arrhenius rate
Sleep drug trial (1908)	paired t, two-group tests, power
Reaction yield	response surface, contours, profilers
Coating formulation (simulated)	several responses, stepwise terms per response, desirability, simulation
Yarn elongation	mixture model, mixture profiler, ternary plot
Daily inspection (simulated)	p, u and c attribute charts
Hald cement (1932)	stepwise selection, best subsets, profilers, collinearity
Loan applications (simulated)	decision trees, validation, random forest, boosted trees, model comparison
Fuel spectra (simulated)	partial least squares

Sample	Shows
Pump sensors (simulated)	support vector machines, model comparison
Fuel consumption (simulated)	multivariate adaptive regression splines, model comparison
Polymer batches (simulated)	neural networks, model comparison, model search
Reaction yield (simulated)	Gaussian processes, proposing the next runs (sequential optimization)
Rosner's measurements (1983)	outlier tests
Blood counts (Dunnett 1955)	comparisons with a control
Plastic film (Johnson & Wichern)	two-way MANOVA
Seal strength (simulated)	distribution identification; non-normal process capability
PK profiles (invented)	non-compartmental pharmacokinetics: AUC, half-life, clearance, geometric means by formulation
Coating batches (simulated)	between/within capability
Supplier lots (simulated)	attribute capability, acceptance sampling
Solder defects (simulated)	Pareto charts (by line, with Other), cause-and-effect diagram
Shaft diameters (simulated)	multi-vari charts (report and graph-canvas form)
Clinical visits (invented)	value labels on coded columns, row colours and markers by column, columns shown on hover, column charts
Coil thickness (simulated)	three-way (between/within), moving-average and zone charts
Short-run machining (simulated)	short-run Z and moving-range chart for several part numbers
Ward safety (simulated)	Laney p' and u' charts, g and t rare-event charts

Sample	Shows
Tablet hardness (simulated)	capability summary page, bootstrap intervals for percentile indices
Bottle filling (simulated)	multivariate control charts: Hotelling T^2, generalized variance, MEWMA
Reactor sensors (simulated)	principal-components control chart: T^2, SPE, contributions, change point
Clean-room monitoring (simulated)	environmental monitoring alert and action limits by location
Heat treatment (simulated)	analysis of means: means, two-way, proportions, rates
Course evaluation (simulated)	item analysis with reverse-scored items, rank correlations
Ability tests (simulated)	factor analysis: maximum likelihood with varimax, principal axis with promax
Honey origins (simulated)	linear and quadratic discriminant analysis with validation roles, canonical plot
Gauge linearity (simulated)	gauge bias and linearity
Gauge repeatability (simulated)	single-gauge repeatability, Cg and Cgk
Weld pull test (simulated)	nested (destructive) gauge R&R, gauge run chart
Bore gauges (simulated)	expanded gauge R&R by REML, unbalanced, variance intervals
Moulding shrinkage (simulated)	orthogonal array (inner/outer), signal-to-noise analysis, declared design
Seedling growth (simulated)	nonlinear regression: a three-parameter logistic growth curve
Compound potency (simulated)	global curve fit: two dose–response curves sharing slope and plateaus, potency difference tested
Assay standard curve (simulated)	weighted standard curve, outlier removed (ROUT), unknowns read off by inverse prediction
Substrate depletion (simulated)	nonlinear regression of a differential equation (Michaelis–Menten rate)

Sample	Shows
ELISA plate (simulated)	curve template: 4PL standard curve, back-calculated standards, acceptance, precision profile, diluted samples
Receptor saturation (simulated)	curve template: total and nonspecific binding fitted together, Kd and Bmax
Competition binding (simulated)	curve template: one-site competition, IC50 and Ki (Cheng–Prusoff)
Enzyme inhibition (simulated)	curve template: mixed inhibition as a global fit, inhibition mechanisms compared
Antagonist shift (simulated)	curve template: Gaddum–Schild global fit, pA2 and the Schild plot
Bacterial growth (simulated)	curve template: modified Gompertz growth of three strains, lag and doubling time
Clonogenic survival (simulated)	curve template: linear-quadratic cell survival, alpha/beta, SF2 and D10
Potency assay (simulated)	curve template: relative potency of a test lot, parallelism test
Rotenone (Martin 1942)	probit analysis: effective doses with fiducial limits
Glucose methods (simulated)	method comparison: Deming and Passing–Bablok regression, difference plot
Tablet stability (simulated)	shelf life: batch poolability tests, confidence bound against the limit
Capsule stability, six lots (simulated)	shelf life with random lots: bound for a future lot beside the fixed-lot analysis
Wine bitterness (Randall 1989)	ordinal mixed model: cumulative logit with a random judge effect, thresholds, odds ratios
Service ratings (simulated)	ordinal logistic regression: proportional odds, parallel-lines test, predicted probabilities
Commute modes (simulated)	nominal logistic regression: baseline-category logits against a chosen reference
Bioequivalence crossover (simulated)	crossover trial, average equivalence of a ratio
Reaction times by task and cue (simulated)	factorial repeated measures: two within-subject factors and one between, sphericity corrections, simple effects

Sample	Shows
Dendrite lengths per mouse (simulated)	nested comparison: neurons within mice within treatments, REML tests and nested ANOVA
Horse-kick deaths (1898)	goodness of fit to a Poisson distribution
Paint drying times (simulated)	comparisons with the best
Prototype failures (simulated)	reliability growth (Crow–AMSAA) with two design phases
Pump repairs (simulated)	repairable systems: mean cumulative function, two designs compared, trend tests
Pump failure modes (simulated)	competing risks: cumulative incidence per mode and site, Gray's test, Fine–Gray regression, Weibull per mode combined into the system
Monthly warranty returns (simulated)	warranty returns: shipments and returns by month as censored life data, Weibull fit, forecast of returns with prediction limits
Hotel occupancy (simulated)	exponential smoothing: automatic model selection by AICc (multiplicative seasonality), 12-month forecast with prediction limits, residual checks
Reactor temperature (simulated)	ARIMA: automatic order search (differencing by KPSS tests), coefficients, 24-hour forecast with prediction limits
Kiln fuel feed (simulated)	ARIMA with an input series: delay from the prewhitened cross-correlation, transfer function and steady-state gain, impulse response, forecasts from the planned feed
Leukemia with covariates (Freireich 1963)	proportional hazards: hazard ratios, proportional-hazards check, survival curves by treatment
Capacitor life test (simulated)	parametric survival regression: interval-censored Weibull model with temperature and lot, time ratios
Visual inspection (simulated)	attribute agreement: within and between inspectors and against a standard, kappas, Kendall's W
Go/no-go gauge (simulated)	attribute gauge bias and repeatability (AIAG analytic method)
Multi-centre trial (invented)	multi-way table; stratified 2 × 2 tables: CMH test, common odds ratio, Breslow–Day, Fisher's exact test

Sample	Shows
Repair times (invented)	bootstrap of a report table (percentile and BCa intervals), permutation tests of two groups and of a correlation
Cell populations (simulated)	Gaussian mixture clustering (covariance structure and number of clusters by BIC), k-means for comparison
Symptom checklist (simulated)	latent class analysis of seven categorical items (number of classes by BIC, response probabilities, profile plot)
Expression profiles (simulated)	t-SNE, UMAP and classical scaling maps coloured by cell type
Exotherm profiles (simulated)	functional data: batch temperature profiles smoothed by B-splines with GCV, functional principal components, scores modelled on catalyst, feed rate and agitator, landmark and continuous registration
Town distances (invented)	classical and metric multidimensional scaling from a distance matrix; principal coordinates with negative eigenvalues
Shrub traits (invented)	Gower distances of mixed column types, principal coordinates, hierarchical clustering on distances
Design comparison (planned)	design evaluation: two candidate 15-run designs compared (power, prediction variance, fraction of design space, aliasing, efficiencies)
Filtration rate (Montgomery)	factorial analysis of an unreplicated 2^4: normal and half-normal plots of effects, Lenth's method, cube plot, dispersion tests
Coating thickness (simulated)	factorial analysis of a replicated 2^3: t tests of effects, dispersion effects of ln s
Furnace coatings (simulated)	D-optimal split-plot design (furnace temperature hard to change, 8 heats of 3 runs), design declaration with whole plots, split-plot mixed model by REML with Satterthwaite F tests
Borehole water flow (Worley 1987)	computer experiment on a maximin Latin hypercube (space-filling design), surrogate linear model with the Response Explorer
Air quality (simulated)	missing-value imputation: EM and low-rank SVD, missing-data patterns (a sensor outage, values missing at high humidity)

Sample	Shows
Espresso machine reviews (simulated)	text analysis: word cloud, phrases, semantic dimensions and document clusters, topic model, sentiment scores against star ratings
Monthly electricity demand (simulated)	time series: description (ACF, Ljung–Box, unit-root tests, spectrum), multiplicative decomposition with seasonal indices
Product adoption (simulated)	time series: logistic S-curve trend model with forecasts
Advertising and sales (simulated)	time series: cross-correlation with prewhitening, sales following advertising by two weeks
Test bench signals (simulated)	signal processing: Fourier spectrum with Welch's average and a spectrogram, a low-pass filter, cross-correlation finding a 10 ms delay, wavelet denoising with a scalogram; three runs stacked by run
Hillside survey (invented)	3-D scatter plot with colour, size and shape; contour and surface plots by triangulation and smoothing thin-plate spline
Gene screen (simulated)	many two-group tests at once: Welch t per gene with Benjamini–Hochberg adjusted p-values and a volcano plot
Plasma concentration (simulated)	area under a curve by formulation with Bailer's standard error, peaks, saving curves as % of range
Trial endpoints (invented)	p-values of twelve endpoints adjusted for multiple testing (six methods and q-values), Fisher's and weighted Stouffer combined tests
Response rates by site (simulated)	Bayesian estimation of two proportions with a region of practical equivalence, an adverse-event rate ratio, a Bayesian logistic model with a random intercept per site
Stroop reaction times (simulated)	Bayes factor paired and two-sample t tests, correlation and contingency-table Bayes factors with robustness plots, a Bayesian ANOVA
Eight schools (Rubin 1981)	Bayesian hierarchical normal model with known standard errors: shrinkage, convergence diagnostics, trace and rank plots

The **Projects** submenu at the end of the list opens sample projects of several tables:

Sample project	Shows
Greenhouse trial (simulated)	two tables (plant heights, and each greenhouse's weekly climate) and a dashboard with a heading, a note and four report tiles, filtered by greenhouse (both tables) and fertilizer (the plant table only)
Tensile tests (invented)	reusing reports and plot styles: three batches of stress–strain tests in tables of the same layout; batch A has a formatted graph and a linear model to copy, paste and duplicate onto batch B (same column names) or batch C (named differently), or to save as a template
Clinical trial with dropout (simulated)	multiple imputation: a two-arm trial with patients dropping out after bad visits, its imputation by chained equations with convergence and density diagnostics, and the stacked table of 20 imputations with an analysis of covariance and a comparison of the arms pooled by Rubin's rules

The repository's `examples/data` folder also has small files in each import format (CSV, TSV, semicolon-delimited text, JSON, a two-sheet workbook and projects) for trying the importers.

Crash reports and recovery

AnovaCraft is built so that a fault in one part does not cost you your work.

An analysis that fails

If an analysis stops on an internal error (a bug, rather than data it cannot use), its report says **stopped with an internal error** and shows the message in place of its sections. The rest of the program keeps running, and the report keeps its settings, so it is saved with the project as it was. It computes again whenever its data, row states or settings change.

Crash reports

Each internal error, and anything that makes the program stop, writes a short text file: what happened, where in the program, the version, and a backtrace. The files go to

System	Folder
macOS	`~/Library/Logs/AnovaCraft`
Windows	`%LOCALAPPDATA%\AnovaCraft\Logs`
Linux	`$XDG_STATE_HOME/anovacraft/logs` (normally `~/.local/state/anovacraft/logs`)

Files for errors the program survived are named `analysis-error-…`, those for stops `crash-…`; the newest 20 are kept. **Help ▸ Show crash reports** opens the folder. The files contain no data from your tables.

Recovery copies

Every two minutes, each open project with changes since it was opened or saved is written, on a background thread, as an ordinary project file to the `Recovery` folder beside the settings file (for example `~/Library/Application Support/Statistical Engine/Recovery` on macOS). Row selections and the window layout are left out, as they are when the

program decides whether a project has unsaved changes. Saving a project deletes its copy, and quitting normally deletes all of them.

When AnovaCraft starts after a run that did not end normally (it crashed, was forced to quit, or the computer lost power), it shows **Restore unsaved work**, listing each project kept, the file it came from and when the copy was written:

- **Restore** opens the copies as projects with unsaved changes. Saving one writes to the file it came from; a project that had never been saved asks for a name.
- **Discard** deletes the copies.
- **Decide later** keeps them and asks again at the next start.
- **Show crash report** (when the run ended in a crash) opens its report.

A copy reflects the project as it was at most two minutes before the end. Several windows of the program running at once each keep their own copies and do not offer each other's.

Documents, files and projects

Each open table, together with its row states and reports, is a **document**, and every document belongs to a **project**: one project file holds one or more tables. Several projects can be open at once: they are listed under *Open projects* in the sidebar and as tabs along the bottom of the window. Opening a file or sample while another is open adds a new project.

A project of one table is shown just as a dataset always was: one entry in the list, one tab. A project of several tables is shown as a tree — the project's name (click it, or its triangle, to fold its tables away), then its tables, with each table made from another indented under it. Along the bottom, its tabs (tables, then dashboards) sit together in a framed group headed by the project's name, in the accent colour while it holds the view on screen. Click the header (▾) to fold the group's tabs away behind it (▸, with the number of tabs); the tab on screen stays visible, and a second click shows them all again. Right-click a table (in the sidebar or on its tab) to save its project, export the table or close it; right-click a project's name to save or close the whole project (its tab-bar header too). A project with unsaved changes shows a • after its name (after the table's name for a one-table project), and the window title adds "(edited)". Closing a project with unsaved changes asks once, and so does quitting.

To rename a table, double-click its name in the sidebar or on its tab (or right-click it and choose **Rename…**), type the new name and press Enter; Escape keeps the old one. Renaming a table is an edit like any other, so Edit ▸ Undo puts the old name back. Rename a project the same way from its name in the sidebar or its tab-bar header: the window title follows, the new name is saved in the project file and is the name offered when you choose Save project as… (the file you then save names the project). A blank name goes back to the file's name.

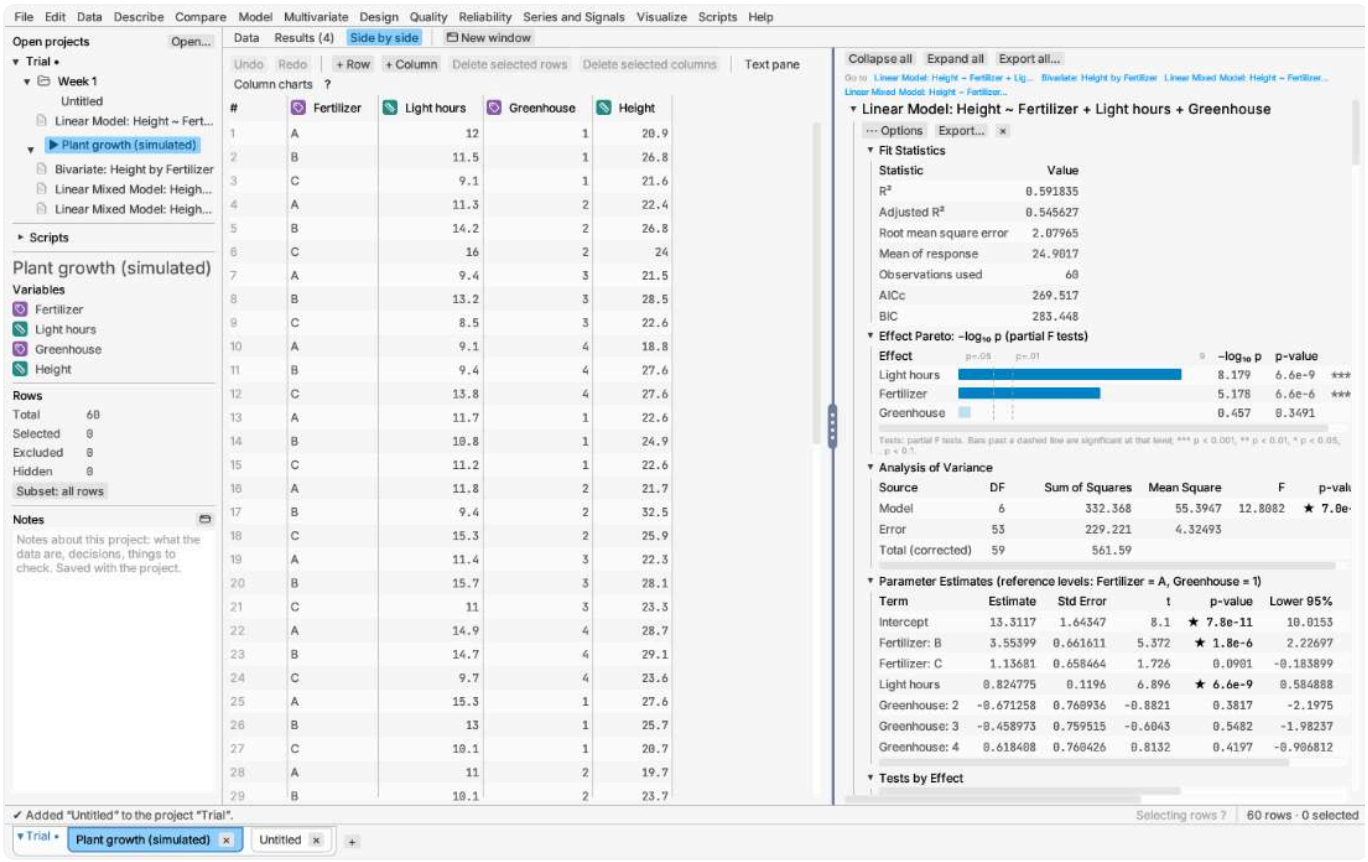
Reports, canvases and folders in the project tree

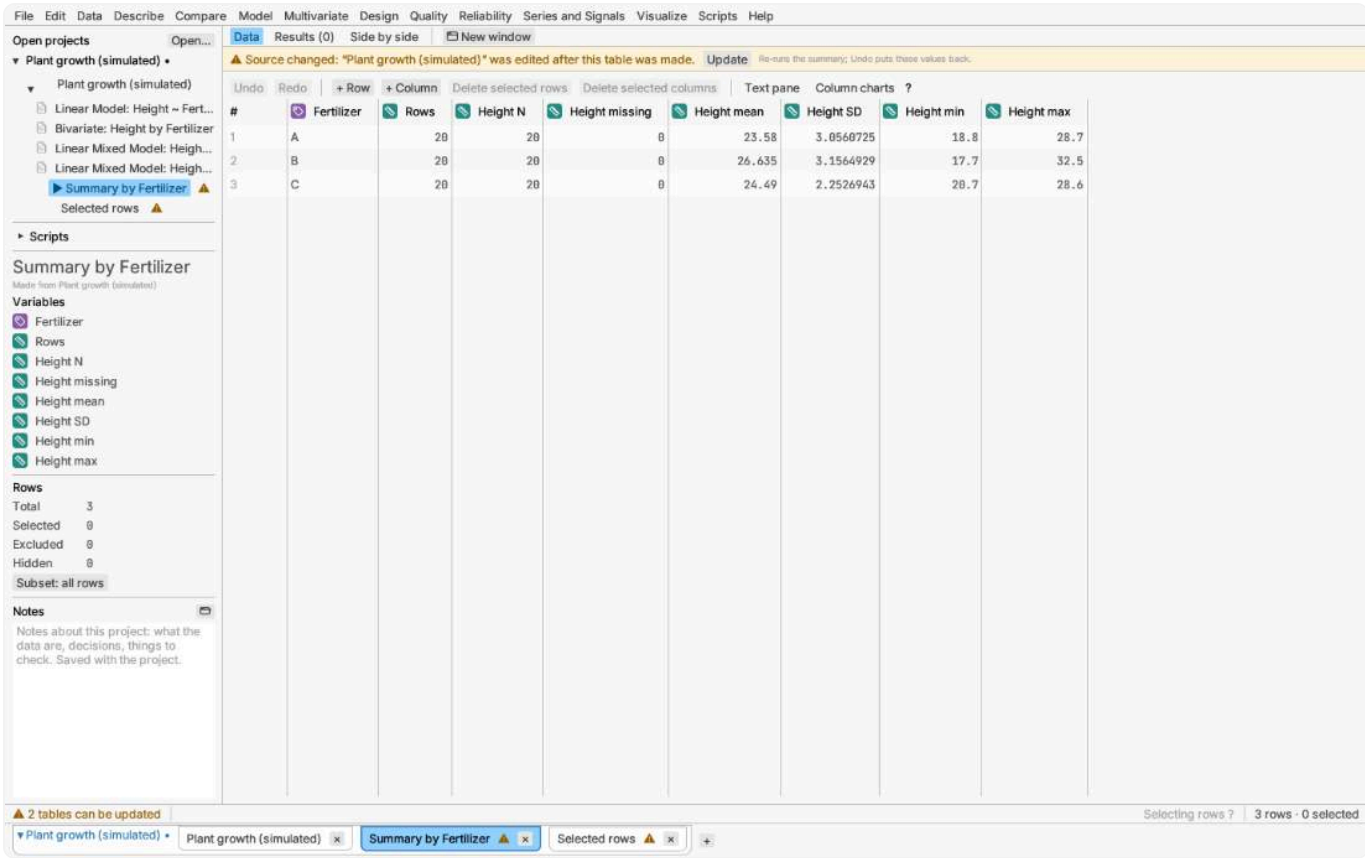
Under each table in the sidebar its reports and graph canvases are listed, newest last, with a page icon for a report and a scatter icon for a canvas. Click one to show it: the table's results come into view, scrolled to the report and opened, or the window holding it comes to the front. Right-click it to open it in a window of its own (or bring it back) or to move it to a folder. The ▾ before a table's name folds its reports away (▸ shows them again). A project's notebooks are listed with its dashboards, with a book icon; click one to open it.

Folders group what belongs together: a week's tables, the plots for a paper, a dashboard and its sources. To make one, right-click the project's name and choose **New folder**, or right-click any table, report, dashboard or notebook and choose **Move to folder ▸ New folder**, which makes the folder with that item in it. Either way the folder's name opens for typing; press Enter to keep it. Then:

- **Drag** a table, report, canvas, dashboard or notebook onto a folder to file it there (the folder is framed while you hold it over it), or use **Move to folder ▸** and pick the folder. Drag it onto the project's name, or choose **Move to folder ▸ Out of folders**, to list it at the top of the project again.
- **Nest** folders by dragging one onto another, or with right-click ▸ **New folder inside**. A folder cannot go into itself or a folder inside it.
- **Rename** a folder by double-clicking its name (or right-click ▸ Rename…), and fold it with its triangle.
- **Click** a folder's name to bring the windows of everything in it to the front; when nothing in it has a window, the main window shows its first table (or dashboard). Right-click ▸ **Open each in a window** gives every table (its data grid), report, dashboard and notebook in it a window of its own.
- **Delete folder** removes only the folder: what it held, folders included, moves up a level and stays in the project.

A folder only groups. A table in a folder lists its reports with it; a report filed in a folder on its own names its table in its tooltip and still belongs to that table, re-running when its data change. Closing a table or a report takes it out of its folder. Folders are saved with the project.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Trial •
Week 1
Untitled
Linear Model: Height ~ Fert...
Plant growth (simulated)
Bivariate: Height by Fertilizer
Linear Mixed Model: Heigh...
Linear Mixed Model: Heigh...
Scripts
Plant growth (simulated)
Variables
Fertilizer
Light hours
Greenhouse
Height
Rows
Total 60
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data Results (4) Side by side New window
Undo Redo + Row + Column Delete selected rows Delete selected columns Text pane
Column charts ?
Fertilizer Light hours Greenhouse Height
1 A 12 1 20.9
2 B 11.5 1 26.8
3 C 9.1 1 21.6
4 A 11.3 2 22.4
5 B 14.2 2 26.8
6 C 16 2 24
7 A 9.4 3 21.5
8 B 13.2 3 28.5
9 C 8.5 3 22.6
10 A 9.1 4 18.8
11 B 9.4 4 27.6
12 C 13.8 4 27.6
13 A 11.7 1 22.6
14 B 10.8 1 24.9
15 C 11.2 1 22.6
16 A 11.8 2 21.7
17 B 9.4 2 32.5
18 C 15.3 2 25.9
19 A 11.4 3 22.3
20 B 15.7 3 28.1
21 C 11 3 23.3
22 A 14.9 4 28.7
23 B 14.7 4 29.1
24 C 9.7 4 23.6
25 A 15.3 1 27.6
26 B 13 1 25.7
27 C 10.1 1 20.7
28 A 11 2 19.7
29 B 10.1 2 23.7
Collapse all Expand all Export all...
Linear Model: Height ~ Fertilizer + Light hours + Greenhouse
Options Export...
Fit Statistics
Statistic Value
R² 0.591835
Adjusted R² 0.545627
Root mean square error 2.07965
Mean of response 24.9017
Observations used 60
AICc 269.517
BIC 283.448
Effect Pareto: −log10 p (partial F tests)
Effect −log10 p p-value
Light hours 8.179 6.6e-9 ***
Fertilizer 5.178 6.6e-6 ***
Greenhouse 0.457 0.3491
Analysis of Variance
Source DF Sum of Squares Mean Square F p-val
Model 6 332.368 55.3947 12.8082 ★ 7.0e
Error 53 229.221 4.32493
Total (corrected) 59 561.59
Parameter Estimates (reference levels: Fertilizer = A, Greenhouse = 1)
Term Estimate Std Error t p-value Lower 95%
Intercept 13.3117 1.64347 8.1 ★ 7.8e-11 10.0153
Fertilizer: B 3.55399 0.661611 5.372 ★ 1.8e-6 2.22697
Fertilizer: C 1.13681 0.658464 1.726 0.0901 -0.183899
Light hours 0.824775 0.1196 6.896 ★ 6.6e-9 0.584888
Greenhouse: 2 -0.671258 0.768936 -0.8821 0.3817 -2.1975
Greenhouse: 3 -0.458973 0.759515 -0.6043 0.5482 -1.98237
Greenhouse: 4 0.618408 0.760426 0.8132 0.4197 -0.906812
Tests by Effect
✔ Added "Untitled" to the project "Trial".
Selecting rows ? 60 rows · 0 selected
▾ Trial • Plant growth (simulated) Untitled +


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Plant growth (simulated) •
Plant growth (simulated)
Linear Model: Height ~ Fert...
Bivariate: Height by Fertilizer
Linear Mixed Model: Heigh...
Linear Mixed Model: Heigh...
Summary by Fertilizer
Selected rows
Scripts
Summary by Fertilizer
Made from Plant growth (simulated)
Variables
Fertilizer
Rows
Height N
Height missing
Height mean
Height SD
Height min
Height max
Rows
Total 3
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data Results (0) Side by side New window
Source changed: "Plant growth (simulated)" was edited after this table was made. Update Re-runs the summary; Undo puts these values back.
Undo Redo + Row + Column Delete selected rows Delete selected columns Text pane Column charts ?
Fertilizer Rows Height N Height missing Height mean Height SD Height min Height max
1 A 20 20 0 23.58 3.0560725 18.8 28.7
2 B 20 20 0 26.635 3.1564929 17.7 32.5
3 C 20 20 0 24.49 2.2526943 20.7 28.6
2 tables can be updated
Selecting rows ? 3 rows · 0 selected
▾ Plant growth (simulated) • Plant growth (simulated) Summary by Fertilizer Selected rows +

Opening data

File ▸ Open… (or the + on the document tabs, or dropping files on the window) reads:

- **CSV, TSV and other delimited text.** The delimiter is detected (for `.csv` files too, so a file saved with semicolons where the decimal mark is a comma opens correctly); `.tsv` and `.tab` files use tabs. Quoted fields may contain the delimiter and line breaks. UTF-8, UTF-16 and Windows-1252 text are read.
- **JSON:** an array of row objects, such as `[{"x": 1, "y": "a"}, …]`.
- **Spreadsheet workbooks:** `.xlsx`, `.xlsm`, `.xlsb`, `.xls` and `.ods`. A workbook opens as one project named after the file, with one table per chosen sheet (or one table of the sheets stacked).

Text files and workbooks first show the Open Data dialog: a preview of the cells where you say which row holds the column names, where the values start and end and which columns to read. It starts from what it detects, so for most files pressing Enter is enough.

A newly opened or imported table (a data file, a workbook sheet, a table from the clipboard, the web or a database) opens on the **Data** tab with no reports: nothing is analysed until you choose an analysis from a menu. Projects and samples open with the reports they hold.

Column names that are blank become `Column N` (N is the column's position) and a repeated name gets `(2)`, `(3)`, …, in every format; the import note lists the names that were changed.

- **Projects** (`.acraft`), described below.
- **Scripts** (`.rhai`) open in the Script window.

File ▸ Open recent lists the last ten files you opened or saved, newest first; the start page shown when nothing is open lists them too, with when each was last used and what it holds. A file that can no longer be opened is dropped from the list.

Column types (number, text, date) are inferred from the values; see Dates and times for how dates are recognised.

Empty cells are missing values. So are the usual missing-value markers that other programs write: `NA`, `N/A`, `#N/A`, `#NA`, `<NA>`, `NaN`, `nan`, `-NaN`, `-nan`, `NULL`, `null`, `None`, `n/a`, `#N/A N/A` and the `1.#IND` / `1.#QNAN` forms (the same list pandas uses; spelled exactly so, apart from surrounding spaces). They count as missing only when deciding that a column holds numbers or dates: a column of numbers with some `NA` cells opens as a number column

with those cells missing, and the import notes say how many were read that way. A text column, such as country codes that include `NA` for North America, keeps them as text. `inf` and `-inf` are not numbers here, so a column containing them opens as text.

Other ways to start a table:

- File ▸ New data table (⌘N / Ctrl+N): an empty table to type into.

The app starts on the start page, with no table open. When a file, sample, design or other table opens, an empty Untitled table nobody has typed into closes by itself, so it does not linger beside your data.

- File ▸ New table from clipboard: cells copied from a spreadsheet or other tab- or comma-separated text, through the Open Data dialog; a first line of names becomes the column names.
- Design menu: a generated experimental design (see Design).
- Table operations, which make new tables from existing ones (see Table operations); the new table joins its source's project unless you choose otherwise (see Tables made from other tables).

Exporting data

Export data as (in the File menu and a document's right-click menu) writes **CSV**, **TSV**, **JSON** or `.xlsx`. Text and JSON export write dates in ISO 8601; `.xlsx` export writes real date cells with number formats. With **Write value labels** ticked in the submenu, a column with value labels is written as its labels (see Value labels).

Export reports…, in the same menus, saves the document's reports as a PDF, Word, PowerPoint, HTML, Markdown or Excel file (see Saving plots and results).

Projects

A project file (`.acraft`) saves everything about its tables:

- each table's data, column types and measurement levels, and formula columns;
- row states (selected, excluded, hidden, colours);

- every report's request and display options; reopening the project re-runs each report, so it reproduces exactly;
- for a table made by a table operation, the operation, which tables of the project it was made from, and a fingerprint of what they held then;
- scripts kept with the project (see below);
- the project's notes (the sidebar's Notes; see A tour of the window);
- its dashboards: tiles, filters and names;
- its folders: their names, how they nest and what each holds;
- the window layout: the table or dashboard shown, each table's view (Data, Results or Side by side) and the views and dashboards shown in windows of their own, with where they were on screen.

Use Save project / Save project as… from the File menu or a table's right-click menu; both save the whole project, every table in it. Project files are documented JSON, so other programs can read and write them. A project of one table with nothing to remember about its layout is written in the one-table form (format version 1 or 2, version 2 when it has date columns) that older versions of AnovaCraft open; a project of several tables, or one whose layout is saved, is written as format version 3. Older files still open.

A report that cannot be restored (for example one naming a column the table no longer has, in a file edited by hand) is left out, and a note at the bottom of the window says which; the table and its other reports open. Formula columns whose formulas cannot be computed (one that refers to itself through other formula columns, or one that does not compile) keep their saved values, also with a note.

To add a table to the project you are working in, rather than start a new project, use File ▸ **Add to this project** ▸ **Empty table**, **Table from clipboard** or **Data file…**. Files opened any other way can join it too: the Open Data dialog has a box, *Add to the open project "…"*, cleared unless you came from Add to this project. (Data from a database or the web always starts a project of its own, since it keeps its source to refresh from.)

Closing a table that belongs to a project of several tables (its × or Close) takes it out of the project; save the project to keep the change. If other tables of the project were made from it, the app asks first: they keep their data but can no longer be updated from it. Right-click the project's name and choose **Close project** to close all its tables.

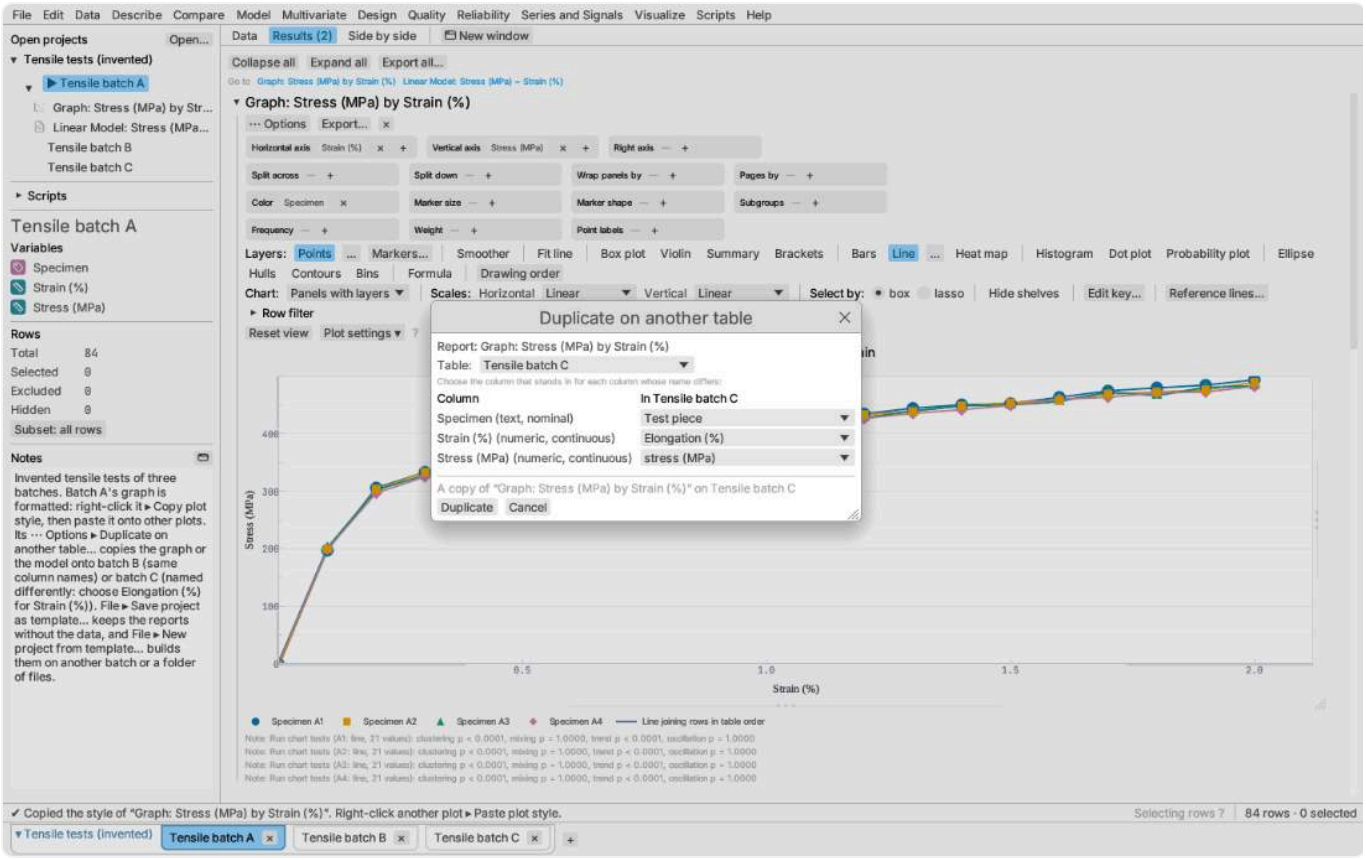
Reusing reports on other data

A report's ⋯ Options ▸ **Duplicate on another table…** copies it, with its display options, row filter, column picker and plot styles, to another open table (in this project or another).

Graph canvases have it too. Choose the table; every column the report reads that has a column of the same name there is matched at once, a name differing only in case, spaces, underscores or dots is offered, and for the rest a list of the table's columns asks which one stands in. A note warns when a chosen column holds another kind of values (text for numbers). Duplicate adds the copy at the top of that table's results.

File ▸ **Save project as template…** writes a `.acrafttemplate` file: the project's reports, graph canvases, dashboards, scripts, notes and plot styles, and for each table only the columns they read (names, kinds and measurement levels) and the formula columns they use, with no data, row states or window layout. Tables made from other tables are left out (the status line says so). File ▸ **New project from template…** asks for a template, then for each of its tables an open table or a **Data file…** (a one-table template starts on the table shown when it fits), with the same column lists as Duplicate for names that differ. For a one-table template, **Each data file in a folder** makes one project with a table per file: the choices made on the first file apply to all, and a file they do not fit is matched by name or left out with a note. The template's formula columns are computed again on the new data, renamed columns are followed in formulas, reports and dashboard filters, and the new project starts unsaved.

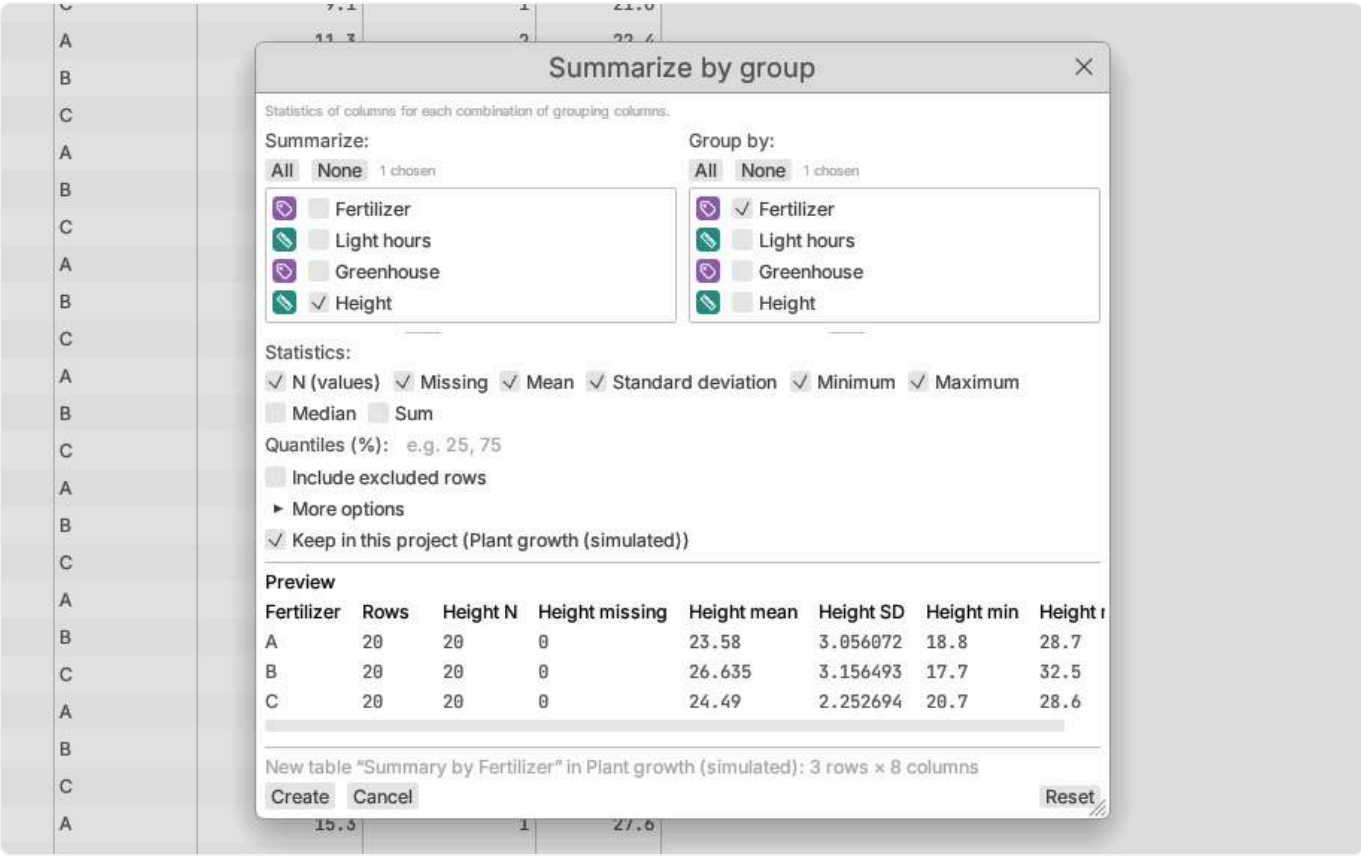
Scripts and notebooks are copied as they are; ones that name columns by their old names need editing. The sample project **Tensile tests (invented)** (File ▸ Open sample data ▸ Projects) has three batches to try this on; batch C names its columns differently.



Tables made from other tables

The Data menu's table operations (Summarize by group, Subset and sample, Stack, Unstack, Transpose, Combine rows, Join, and Sort or Remove duplicates into a new table), Design ▸ Augment design… and **Save as data table** on a report's table make a new table from the current one. Their dialogs have a **Keep in this project (project name)** box, ticked at first: the new table joins the current table's project and is listed under it. Clear the box to open the new table as a separate project, as before. (For a report table, choose **Save as data table** to keep it in the project or **Save as data table in a new tab** for a separate one.)

New tables get short names that say what they hold, such as "Summary by Fertilizer", "Fertilizer A only" or "Stacked Employed, GNP", numbered if the name is taken.



When the source changes. A table made by a table operation and kept in its source's project remembers what its source held. When the source is edited afterwards (its values, its columns, or the rows the operation reads, such as excluded rows for a summary), the made table is marked with ⚠ in the sidebar and on its tab, the status bar says how many tables can be updated, and a banner above its grid says "Source changed: *source* was edited after this table was made." Nothing changes by itself: choose **Update** in the banner to run the operation again on the source's current data. The update is one step in the table's undo history, so Edit ▸ Undo puts the previous values back (and the mark with them). Formula columns you added to the made table are kept and recomputed; a report on the made table that uses a column the update would remove stops the update with a message. Augmented designs and report tables are copies: they join the project but are not updated.

Scripts in a project

A project can carry Rhai scripts (Script window ▸ Keep in project). They are listed under **Scripts** in the File menu and the document's right-click menu, where each can be run, opened in the Script window, set to run when the project opens, or removed. A script set to run on opening never runs silently: the app asks first (Run, Don't run, or Show in Script

window). The sidebar's *Scripts* section lists them too, beside the scripts in your scripts folder; right-click a folder script there and choose **Keep in project** to copy it into the active project.

Notebooks in a project

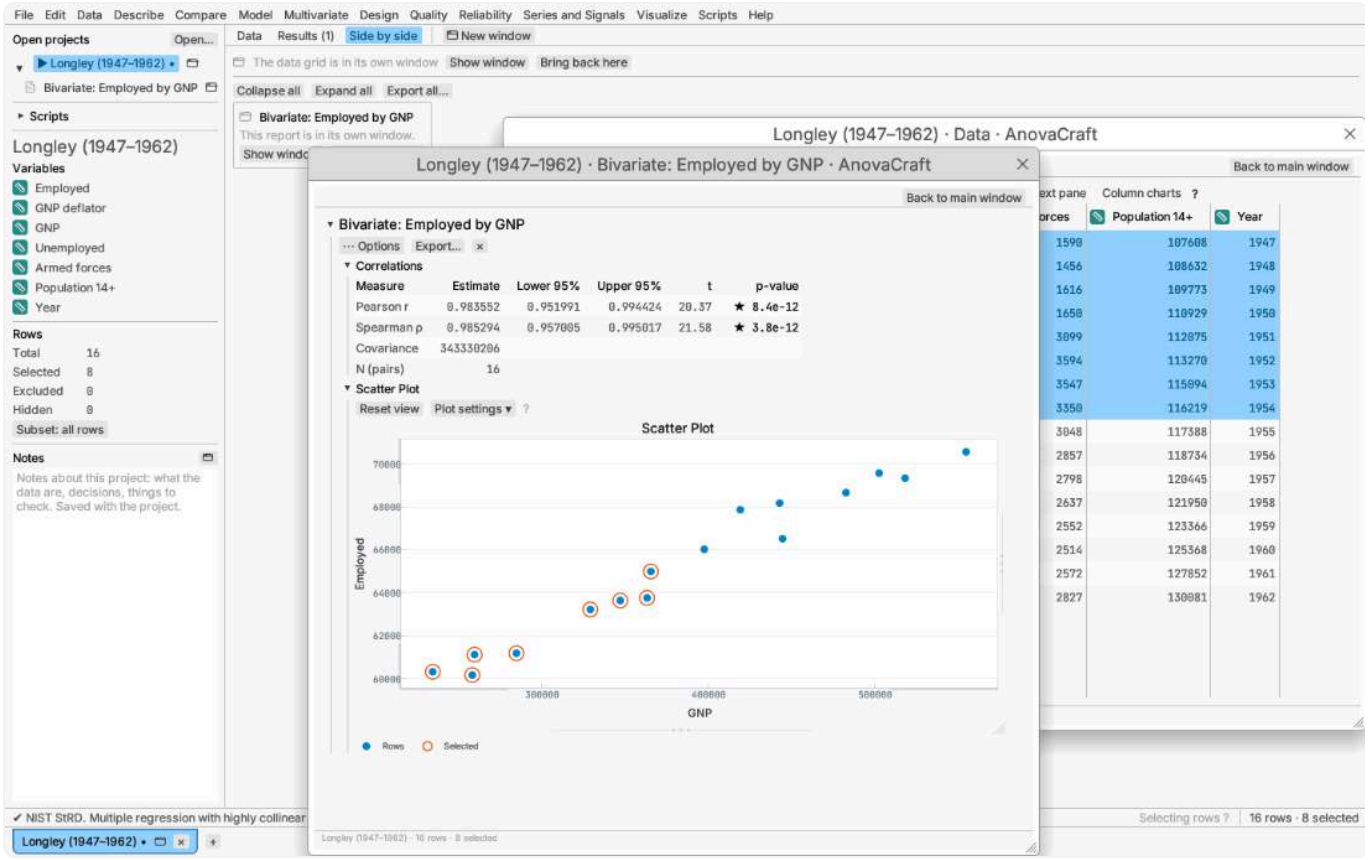
A project can also carry script notebooks (Scripts ▸ New notebook): pages of text and code cells with the outputs of their last run. Scripts ▸ Notebooks lists them; saving the project saves them with their outputs, so reopening the project shows the results without running anything. See Scripting and recording.

Views in windows of their own

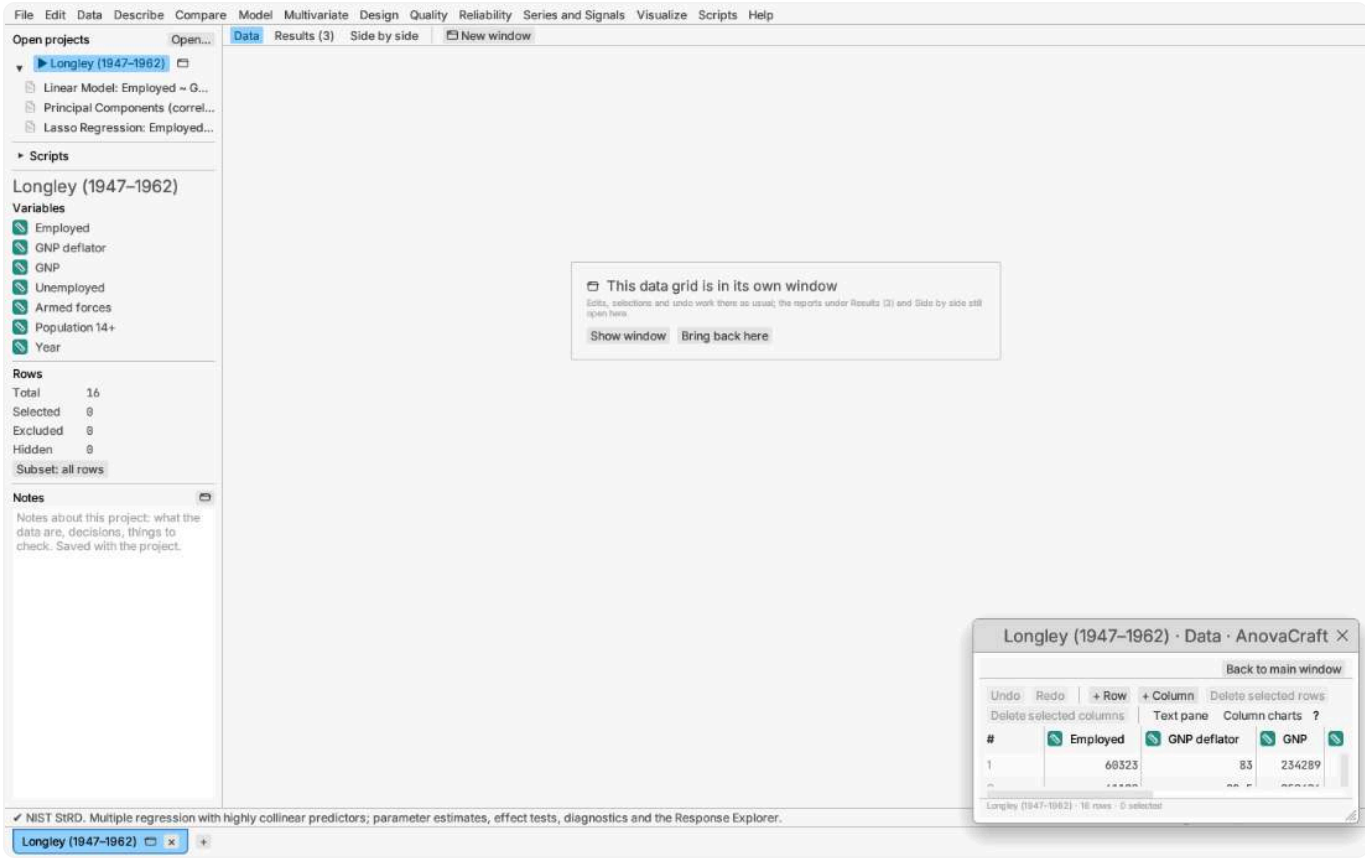
A document's **data grid**, its **results** (every report, with the Go to outline) or a **single report** — a graph canvas, a model with its Response Explorer, a 3-D surface — can each be shown in a window of its own, for example to keep the grid on one screen and the reports on another.

- **New window**, beside the Data / Results / Side by side tabs, moves the data grid into its own window when Data is shown, and the results when Results or Side by side is shown.
- **Open in new window**, in a report's ⋯ Options menu and in the menu you get by right-clicking the report's title, moves that one report.

Each window is titled with the dataset and what it shows, such as "Longley · Data · AnovaCraft" or "Longley · Linear Model: Employed ~ GNP · AnovaCraft", and has a **Back to main window** button at its top right. Its foot gives the dataset's row counts.



A view is in one place at a time. Where it would be in the main window, a panel says that it is in its own window, with **Show window** (bring the window to the front) and **Bring back here**. A report in its own window leaves the same panel in the results. In **Side by side**, a view in its own window leaves the other one across the whole width, under a one-line note ("The results are in their own window", with the same two buttons). The dataset stays in *Open datasets* and on its tab, marked with a small window symbol; point at it to see which views have windows. Clicking a dataset that is already selected brings its windows to the front.



Closing a view's window — its close button, ⌘W / Ctrl+W while it is in front, or Back to main window — puts the view back in the main window, which switches to that dataset; nothing is closed. Closing a dataset closes its windows (after asking about unsaved changes), and quitting closes them all. A table whose source changed shows the Source changed banner with Update at the top of its data grid's window too.

Which dataset commands act on. The menus, dialogs and the sidebar act on the dataset of the window you used last. Clicking in a window of another dataset makes that dataset the one the menus act on without changing what the main window shows; the sidebar then names it ("From the window you used last") and the status bar puts its name before the row counts. Clicking in the main window's grid or results (or choosing a dataset in the list or the tabs) makes the main window's dataset the target again, and so does editing a cell there; using the menu bar does not. Dialogs always open in the main window, which comes to the front.

Shortcuts work in every window and act on that window's dataset: undo and redo, copy, cut and paste, Find, Select all, Save, and ⌘W / Ctrl+W, which closes the window it is pressed in. Quit (⌘Q / Ctrl+Q) works from any window.

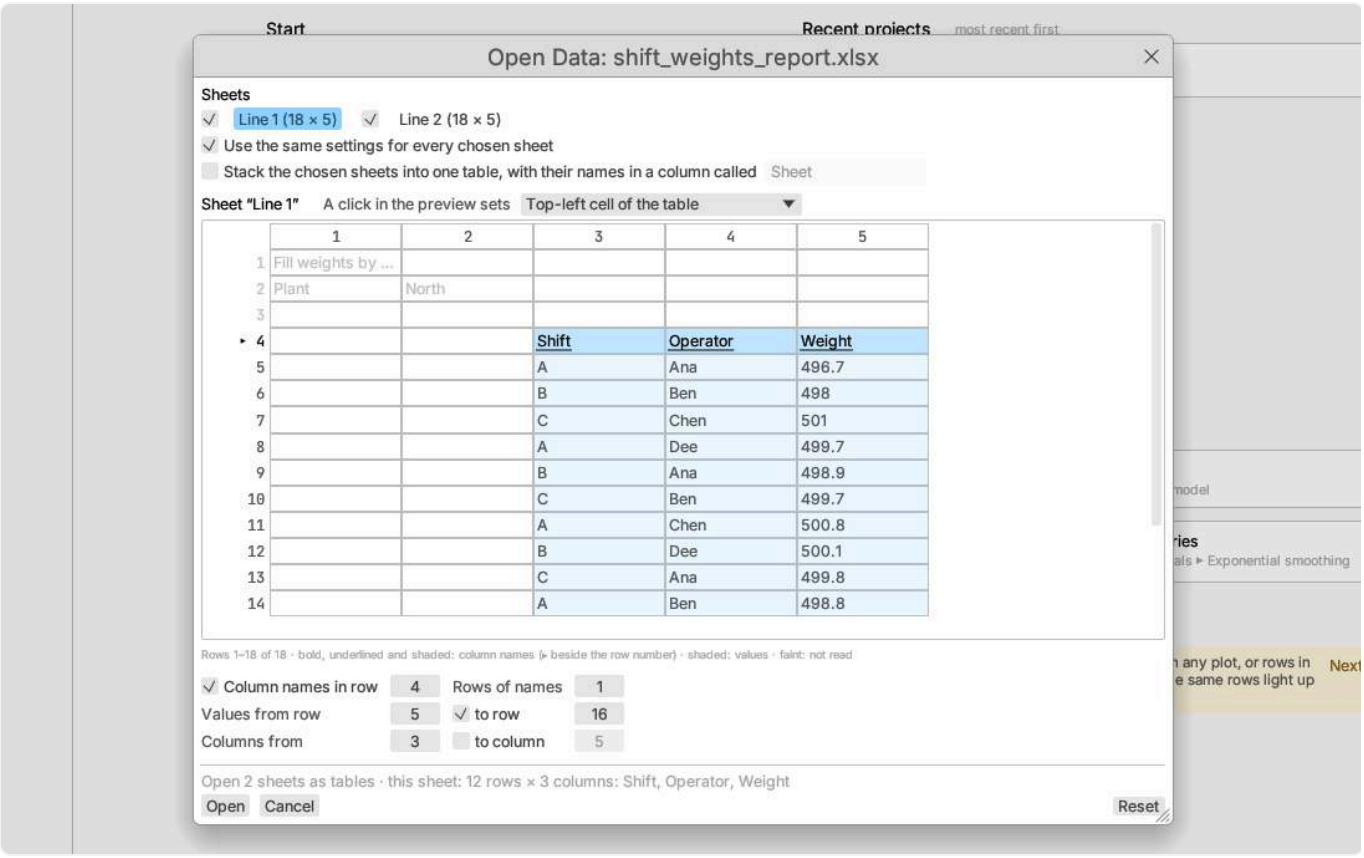
Selections are shared. Selecting points in a plot in any window highlights the same rows in every window at once, as in a single window.

Graph canvas. Columns cannot be dragged from one window to another, so a graph canvas in its own window (or a results window holding one) lists the dataset's columns at its left: drag them from there onto the shelves.

Saved with the project. Saving a project saves which views are in windows of their own and where each window was, with the table shown and each table's view. Reopening the project puts the windows back. A window saved on a screen this computer does not have (a second monitor, say) is moved onto the main window's screen, and made smaller if it would not fit. Changing the layout alone does not count as an unsaved change. Recorded scripts do not record windows, since they change how things are shown, not the analysis.

The Open Data dialog

Text files (`.csv` , `.tsv` , `.tab` , `.txt` , `.dat`), spreadsheet workbooks, tables read from a web page or web text, and **File ▸ New table from clipboard** all pass through the **Open Data** dialog before they become a table. Files often do not start with the column names in their first row: a report saved from another program may have a title, a few lines about the data, a blank row, and only then the table, which may not start in the first column either. The dialog shows the cells as they are and lets you say where the table sits.



The preview

The preview shows the cells with their row numbers down the left and column numbers across the top (the first 200 rows; tick **Show every row** for more). It marks what each choice takes in:

- **column names**: bold, underlined, strongly shaded, and a ▸ beside the row number;
- **values**: lightly shaded;

- cells that are **not read**: faint text, no shading.

Point at a cell to see its full text.

A click in the preview sets what **A click in the preview sets** names:

Choice	A click on a cell…
Top-left cell of the table (the default)	makes its row the row of names and its column the first column; the values start below the names
Row of column names	makes its row the row of names
First row of values / Last row of values	makes its row the first or last row read as values
First column / Last column	makes its column the first or last column read

Where the table sits

The numbers under the preview say the same as the shading and can be typed or dragged:

- **Column names in row** (untick when the file has no names: the columns are then called `Column 1`, `Column 2`, …) and **Rows of names**. With two or more rows of names, each column's name joins its cells with a space; a blank cell in an upper row takes the text to its left, as a heading merged across several columns does, so `Temperature` above `Min` and `Max` gives `Temperature Min` and `Temperature Max`.
- **Values from row**, and **to row** when the table stops before the end of the sheet (for example above a footnote). Blank rows among the values are skipped.
- **Columns from**, and **to column** to stop before the last column. Columns that are blank in every row read are left out.

Blank names become `Column N` and a repeated name gets `(2)`, `(3)`, …, as described in Documents, files and projects; the import note lists any name that was changed.

The line above **Open** says what the choices give for the sheet on show, for example *12 rows × 3 columns: Shift, Operator, Weight*. **Open** (or Enter) opens the table, **Cancel** (or Escape) opens nothing, and **Reset** goes back to the detected settings.

What is detected

The dialog opens with settings worked out from the cells, as a starting point:

- The table is the largest run of rows without a blank row. A run of similar width above or below it, over the same columns, joins it (a table with a blank row in the middle stays whole); a narrow run, such as a title or a footnote, or a heading block off to the side, does not.
- The first row of the table that is at least half as wide as the table holds the names, unless it holds numbers and no column has text in it above numbers further down (then the file has no names).
- The first and last columns are those the table's rows use.

Text files

For text, the dialog also shows:

- **Columns separated by**: *Detect* (the choice shown in brackets, such as *Detect (semicolon)*), or comma, tab, semicolon, vertical bar or space. The separator is detected for every text file, `.csv` included, except that `.tsv` and `.tab` files use tabs. Pasted text with a tab in it is split at tabs.
- **Decimal mark**: *Point 1.5* or *Comma 1,5*; the one found in the file is marked *(detected)*. With a decimal comma, a column whose every value reads that way becomes numbers: `1,5` is 1.5 and `1.234,5` is 1234.5 (points may only separate groups of three digits). A column with any other value, such as a date `03.04.2026`, is left as it is.
- **Also read as missing**: texts, separated by commas, that stand for a missing value in this file (such as `-`, `n.a.` or `999`), on top of the standard markers listed under Opening data.

Files that are not UTF-8 are still read: UTF-16 text (what spreadsheets save as "Unicode text") is recognised by its byte-order mark or by the zero bytes between its letters, and anything else is read as Windows-1252 (Western European), with a note saying so.

If rows have fewer fields than the widest row, the missing fields are left blank and a note says how many rows were short.

Workbooks

A workbook lists its sheets at the top, each with its size. Tick the sheets to open (empty sheets cannot be ticked) and click a sheet's name to show it in the preview.

- **Use the same settings for every chosen sheet** (on when the sheets look alike): a change applies to every ticked sheet. Untick it to set each sheet on its own.
- **Stack the chosen sheets into one table**: the sheets' rows are put one under another, with a first column (called *Sheet* unless you name it) holding each row's sheet name. Columns are matched by name; a sheet without a column leaves it blank. Values are typed after stacking, so a column is numeric only when it holds numbers in every sheet.

Without stacking, each ticked sheet becomes a table, the tables of one workbook together in one project named after the file.

Opening without the dialog

File ▸ Settings… ▸ Open files directly when the table starts in the top-left cell (off by default) skips the dialog for files whose detected table has its names in row 1, its values from row 2 to the end, every column, and no decimal comma. Other files still show the dialog.

Scripts open files without the dialog: `open_table(path)` uses the detected settings, and `open_table(path, #{…})` takes the dialog's choices as a map (see Scripting and recording). A recorded script writes the map when the choices differ from the detected ones.

A table already open

Data ▸ Rows ▸ Use row as column names fixes a table that was opened with its names as a row of values: the row of the cell cursor (or the one selected row) becomes the column names and is removed, and text columns whose remaining values are all numbers or all dates become number or date columns. It is one step that **Undo** reverses.

Web tables

File ▸ Import from the web… shows the dialog for a table on a web page and for delimited text (JSON needs no choices). The choices are kept with the table, in its project too, so **Data ▸ Refresh from source** reads the same rows and columns again; when the choices were left as detected, a refresh detects them again.

Images as tables

An image file opens as an ordinary table with **one row per pixel**, so every analysis, formula column and plot can use it. Pixels are listed row by row from the top of the picture, left to right.

Column	What it holds
`x`	The pixel's column: the position of its centre, counted from the centre of the top-left pixel, times the pixel size.
`y`	The pixel's row, counted the same way, **downwards** as in image files.
`intensity`	For a grey image, the grey level in the file's own units (0–255 for 8-bit files, 0–65535 for 16-bit ones, the stored number for floating-point TIFF).
`red` , `green` , `blue` , `luminance`	For a colour image read as colour: the three channels, and the luminance 0.2126 R + 0.7152 G + 0.0722 B (the Rec. 709 weights, applied to the stored values with no gamma correction).

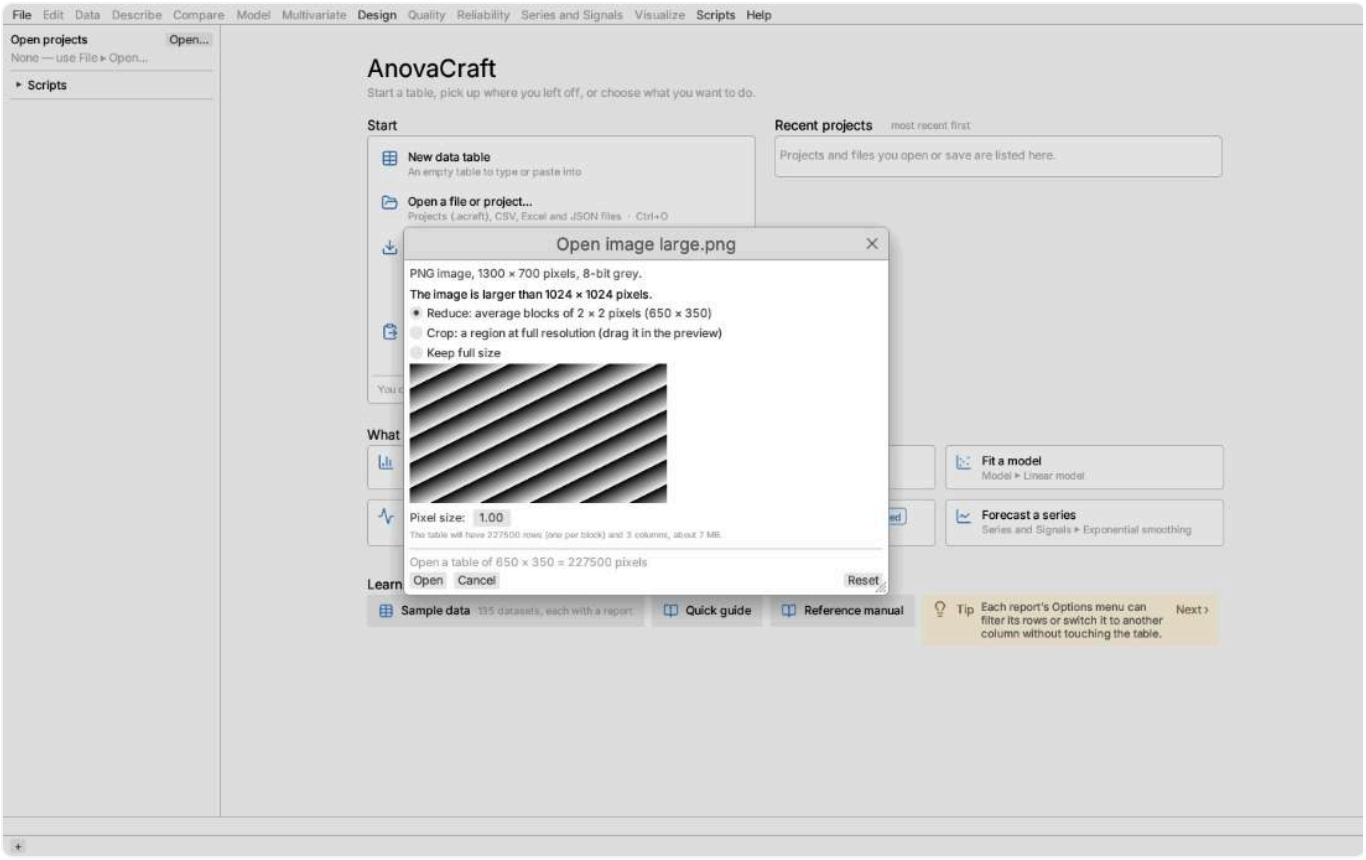
PNG (8- or 16-bit grey or colour, and palette images), JPEG, TIFF (8- or 16-bit, or 32-bit floating point) and BMP files are read. Transparency is not read; the dialog and the note say so when a file has it.

The Open image dialog

File ▸ Open… (or dropping a file on the window) shows the **Open image** dialog for an image: the file's format, size and pixel type, a preview, and these choices.

- **Columns** (colour images only): **Grey (intensity)** gives one `intensity` column, the luminance; **Red, green, blue and luminance** gives all four.
- **Pixel size**: the length of one pixel in the units `x` and `y` should be in (micrometres per pixel, say). With 1, `x` and `y` count pixels.

An image larger than 1024 pixels on a side would make a very long table (a 4000 × 3000 photograph is twelve million rows), so the dialog asks what to do:



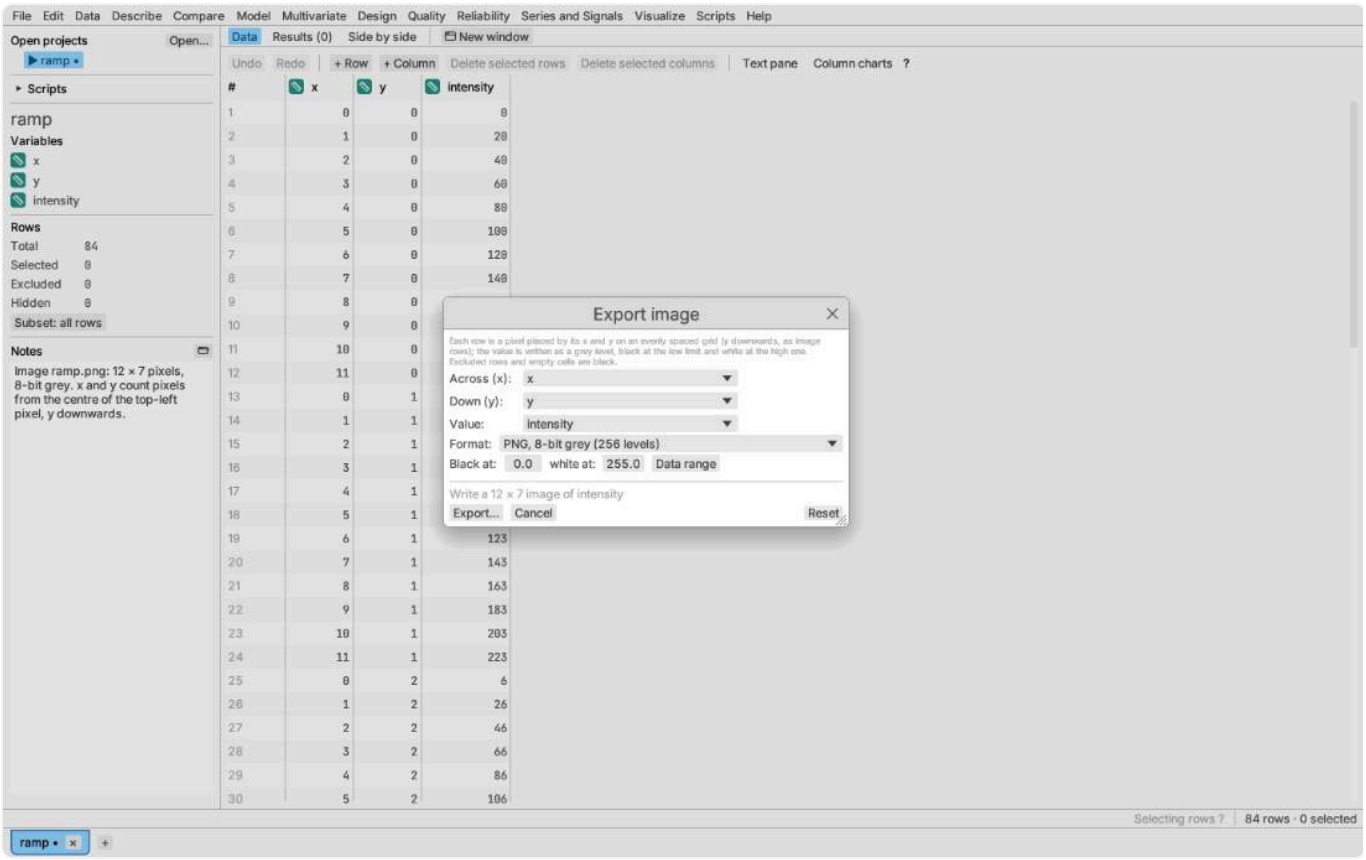
- **Reduce** averages blocks of *k* × *k* pixels, with *k* the smallest whole number that brings both sides within 1024 (⌈side ÷ 1024⌉). Each row is one block, placed at the block's centre **in the original pixel units**, so distances and frequencies measured on the reduced table keep their meaning. Pixels at the right and bottom edges that do not fill a whole block are left out (the note says how many).
- **Crop** keeps a region at full resolution: drag in the preview to place it, or type its left, top, width and height. `x` and `y` keep the original pixel numbers, so a crop at column 138 starts at `x` = 138.
- **Keep full size** opens every pixel, for images up to 4096 × 4096.

Under the choices the dialog gives the size of the table and the memory it needs, marked with ⚠ when it is over 200 MB. What was done (the file, its size, the reduction or crop, the pixel size) is shown in the status line and added to the project's **Notes**, so the table's origin stays with it.

With **Settings ▸ Open files directly when the table starts in the top-left cell** on, a grey image within 1024 × 1024 pixels opens without the dialog. Scripts open images with `open_table(path)`, which reads a colour image as its luminance and reduces a large image as **Reduce** does.

Exporting an image

File ▸ Export data as ▸ Image (PNG or 16-bit TIFF)… writes a value column as a grey picture. Choose the **Across (x)** and **Down (y)** columns, which must place the rows on an evenly spaced grid (any table made from an image does, and so do results computed from one), the **Value** column, and the format:



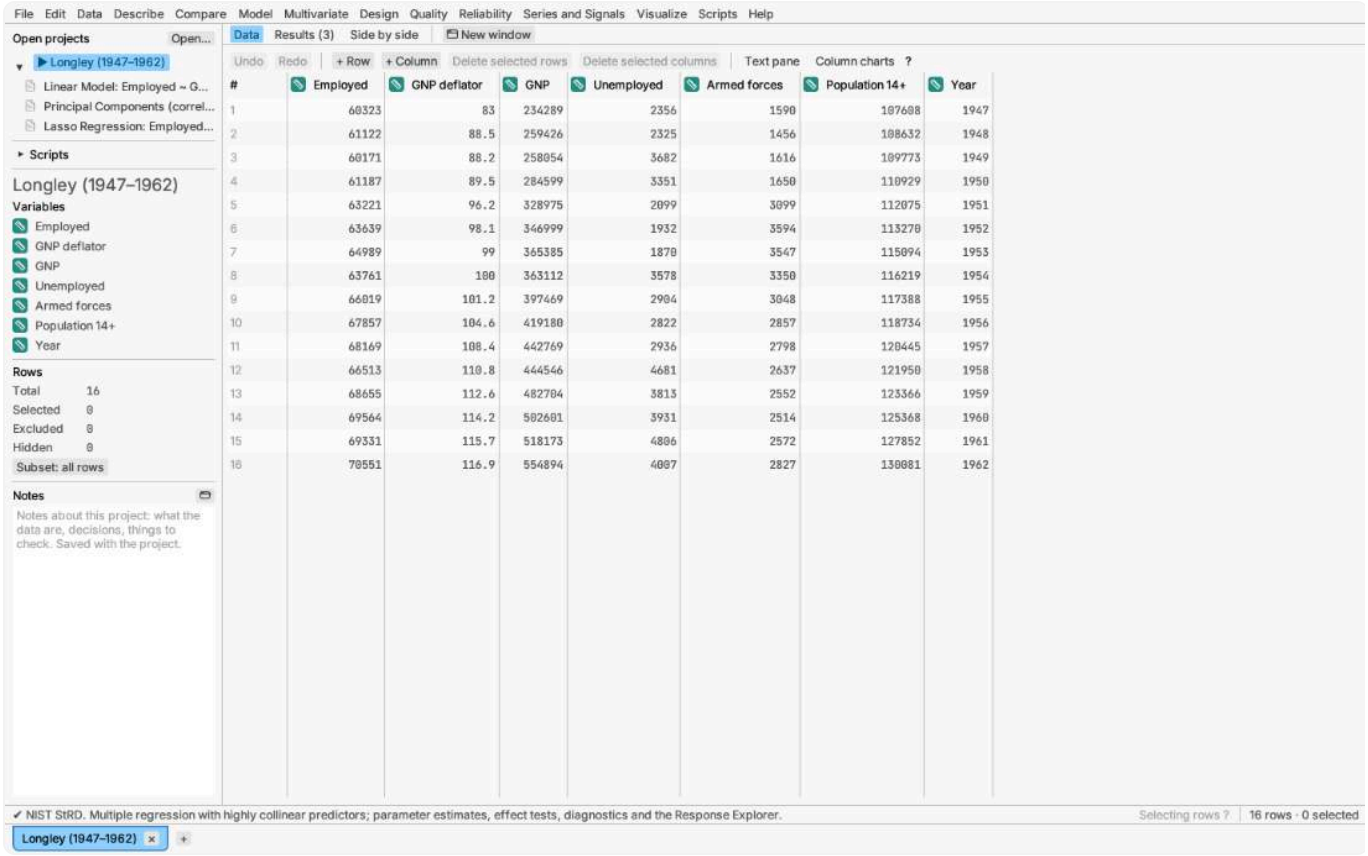
- **PNG, 8-bit grey** (256 levels) or **TIFF, 16-bit grey** (65,536 levels, for measurements that need the finer steps).
- **Black at** and **white at**: the values written as black and white; values between are spaced evenly in grey level, and values outside are clipped. **Data range** sets them to the least and greatest values.

Rows run downwards from the least `y`, as in the image they came from. Excluded rows and cells no row fills are written black. Two rows that fall in the same cell, or values that are not on an even grid, stop the export with a message naming them.

See Image plots to look at the table as a picture.

Editing the data grid

The **Data** tab shows the active table as a grid that works like a spreadsheet. It scrolls smoothly through large tables because only the visible rows are drawn. The first column shows row numbers, followed by × for an excluded row and ◌ for a row hidden in plots. Each column heading shows the column's measurement level as a coloured icon (a ruler for continuous, a tag for nominal, a numbered list for ordinal) and its name; hover over either for the level and the data type (numbers, text, dates). Columns open wide enough for their whole name and their first values, up to a limit; drag a column's edge to resize it. Numbers and dates are right-aligned so their digits line up; a missing number or date shows as a faint dot. A computed number with more than ten significant digits (a group mean, say) is shown to eight; whole numbers, such as identifiers, are always shown in full. Hover over a shortened number for the full value, which is also what editing, copying and saving use. Hover over the **?** in the toolbar for a summary of the mouse and keyboard controls.



Entering values

- Click a cell to move the cell cursor there. Clicking a cell ends any selection of rows or columns.
- Move with the arrow keys; ⌘/Ctrl with an arrow key jumps to the first or last row or column; Home and End go to the first and last column (with ⌘/Ctrl, the first and last cell); Page Up and Page Down move 20 rows.
- Double-click, press Enter or F2, or just start typing to edit the cell. When the value is wider than the cell, or holds more than one line, it is edited in a larger box that opens over the cell and shows the whole text (scrolling when it is long). Enter commits as usual; in a text column, Shift+Enter starts a new line.
- Typing in the main window's grid always edits the table that grid shows, even if you last used a window of another dataset.
- Enter or Tab commits the value and moves down or right (Shift reverses); Escape cancels. Enter on the last row adds an empty row and moves to it, so a column can be typed in without stopping.
- Entry is type-checked: a numeric column accepts numbers, a date column accepts dates, and a value that does not fit is refused with a message. A column shown as **Date** keeps only the day: a time typed or pasted with it (`2026-03-04 10:00`) is dropped, so the cell equals the others showing 2026-03-04 when grouping, sorting or joining. Show the column as **Date and time** to keep times. Typing `NA` (or another missing-value marker listed under Opening data) into a number or date cell makes it missing, as a blank does.

Formula column cells cannot be typed into; edit the formula instead (see Formula columns).

Selecting cells, columns and rows

- **A block of cells:** drag across the cells, Shift-click a second cell, or hold Shift with the movement keys. ⌘A / Ctrl+A (Edit ▸ Select all cells) selects every cell.
- **Columns:** click a column heading to select the whole column. Shift-click another heading to select every column between the two, ⌘/Ctrl-click to add or remove one, or drag across headings.
- **Rows:** click a row number to select the row. Shift-click another row number to select every row between the two (add ⌘/Ctrl to keep the rows already selected), ⌘/Ctrl-click to add or remove one row, ⌥/Alt-click to remove one, or drag down the row numbers. Row selection is shared with every plot (see Row states).

Selecting rows ends a selection of columns, and selecting columns (in the grid or the sidebar) ends a selection of rows, so only one kind is selected at a time.

Dragging past the top or bottom edge of the grid keeps extending the block or the rows, scrolling as it goes.

Clicking a variable in the sidebar also selects its column and scrolls the grid to it, which helps in wide tables.

Finding and replacing

Edit ▸ Find… (⌘F / Ctrl+F) opens a Find bar above the grid. Type some text, then press **Next** (or Enter) to move the cell cursor to the next cell that contains it, reading across each row and then down, and wrapping round at the end. Cells match on the text the grid shows, so numbers match as displayed. **Match case** makes upper and lower case letters count.

Select matching rows selects every row with a match, in the selected columns if any are selected and otherwise in any column. The selection is shared with every plot, so this is a quick way to find, say, every row for one supplier. Escape or × closes the bar.

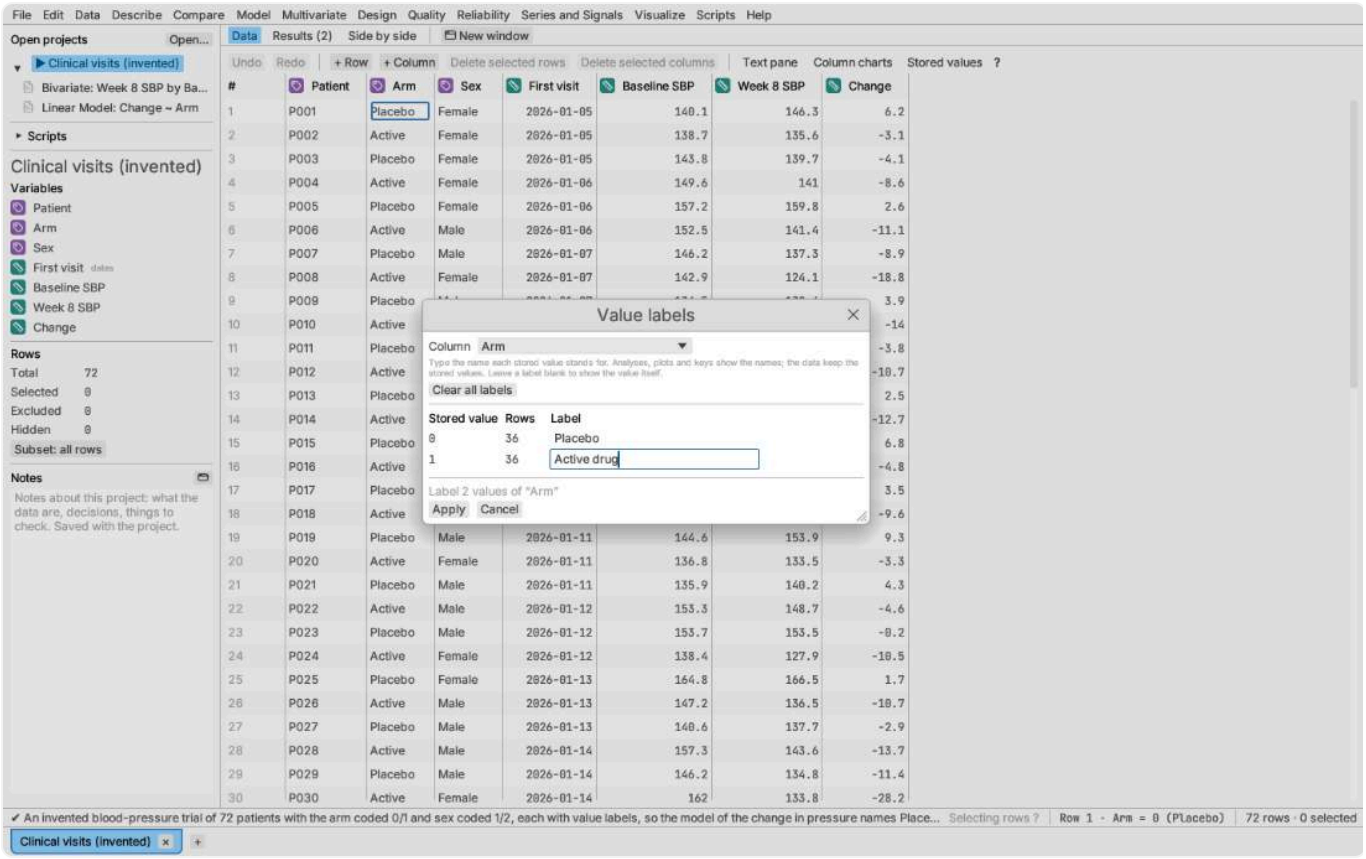
Replace… shows a second row. **Replace** changes the found text in the cell at the cursor (if it matches) and moves on to the next match; **Replace all** changes every match, in the selected columns or in every column, as one undoable step. The new text must still be a valid value for its column: replacing digits with letters in a numeric column leaves those cells alone, and the bar says how many were skipped. Formula cells are never changed.

Value labels

Data from surveys and case-report forms often store a category as a code: 1 for female and 2 for male, 0 for placebo and 1 for the active drug. **Data ▸ Value labels…** (or *Value labels…* in the column heading's right-click menu, which shows how many labels the column has) names each code. The dialog lists every value the column holds, plus any value that has a label but no rows now, each with a *Label for …* field. A blank field removes that label; two values cannot share a label. **Apply** changes the labels as one undoable edit, and every report that uses the column re-runs.

With labels set:

- The grid shows each labelled cell's label; hovering it shows the stored value. **Stored values** on the grid toolbar (shown when any column has labels) switches the grid to the codes and back. The status bar shows both, as "1 (Active)".
- Typing a label into a cell stores its code; typing a code still works.
- Analyses, plots, keys, summaries and saved predictions name the levels by their labels. Levels sort by their names, so labels change the order in which levels appear. A level chosen in a report before the labels were added (an event level, say) is stored by its code and may need choosing again.
- **File ▸ Export data as ▸ Write value labels** (a tick, remembered between sessions) writes labels in place of the codes; untick it to write the codes. Projects always keep both.
- Scripts read and set labels with `t.value_labels`, `t.set_value_labels` and `t.clear_value_labels`.



Columns shown on hover

Tick **Show on hover in plots** in a column heading's right-click menu to add that column's value (by its label) to the tooltip of every point in report plots and on the graph canvas, for

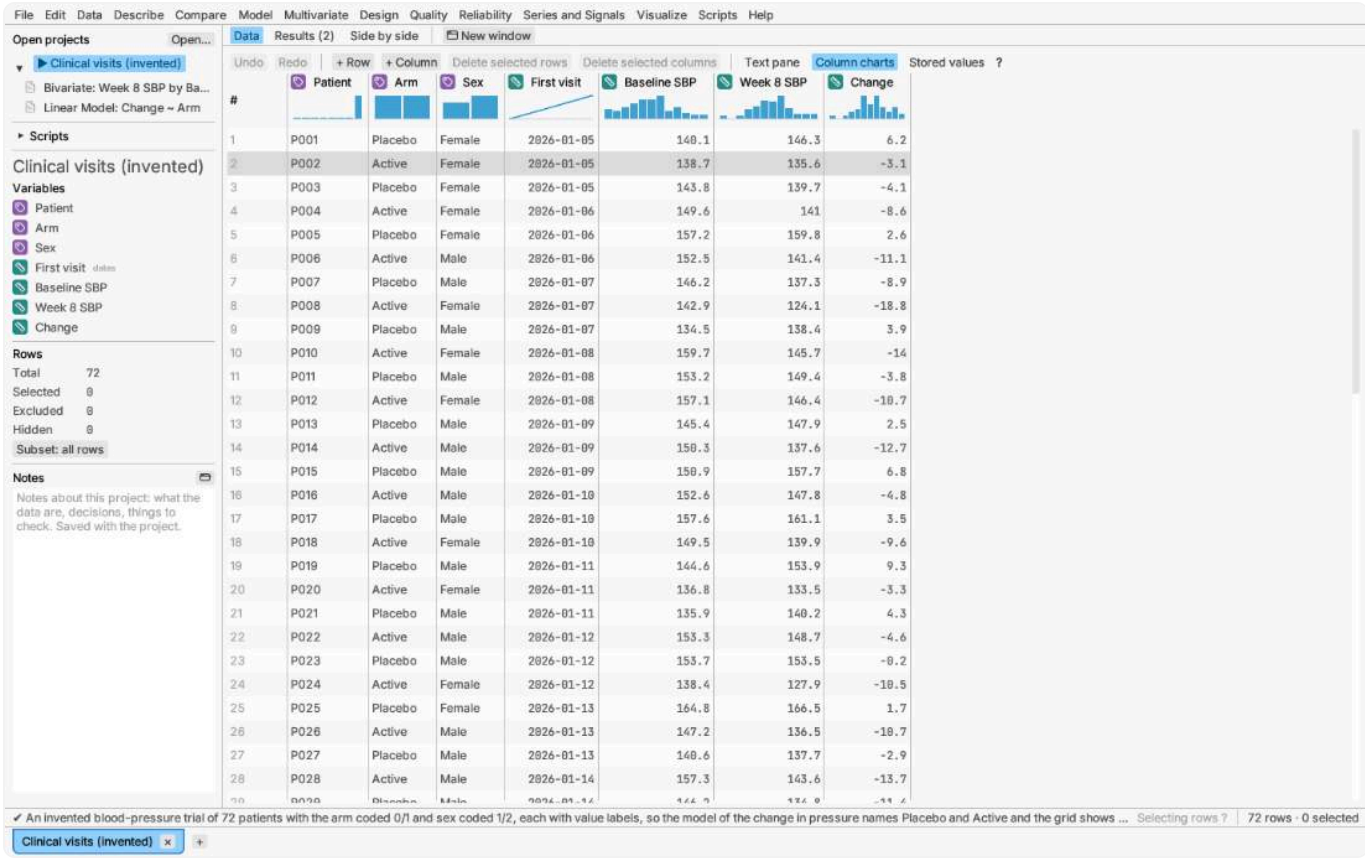
example a patient or batch identifier. Several columns can be shown; the tick is undoable and saved in projects.

Column charts

Column charts on the grid toolbar draws a small chart under each column heading, over the rows analyses use (excluded rows and rows outside the active subset are left out):

- a continuous column: a 12-bin histogram;
- a nominal or ordinal column: a bar per level, at most eight, the most frequent kept and the rest pooled into one "other levels" bar;
- a date column: the values in row order, as a line.

Hover a bar for its range or level and count (and the column's missing values); click it to select its rows, with the usual Shift, ⌘/Ctrl and ⌥/Alt modifiers. The charts are recomputed only when the data or the excluded rows change.



A column heading's level badge can also be dragged onto a dialog's column list or a graph canvas shelf.

Recoding values

Right-click a column heading (or a variable in the sidebar) and choose **Recode values…**, or use Data ▸ Recode values…, to give each distinct value of a column a new one. The dialog lists every value with how many rows hold it (· marks a space, so stray spaces show), and a box for its new value.

- Give several values the same new value to merge them, for example "north", " North" and "N" into "North".
- Leave a new value blank to make those cells missing.
- **Trim spaces**, **lower case**, **UPPER CASE** and **Title Case** tidy every new value at once; **Reset** puts the current values back.

The result goes **into a new column** (named "… recoded" unless you change it, placed right after the original), or **replaces the values in this column** as one undoable step. A new column holds numbers if every new value is a number, and text otherwise. Recoding numbers into labels (2 → "low") needs a new column, since a numeric column cannot hold text. The line at the dialog's foot says how many values and rows will change; **Recode** stays unavailable, with a note saying why, until a value has a new one (in place) or the new column has an unused name.

Random columns

Data ▸ New random column… (also Edit ▸ Add column ▸ Random values…) adds a column of random draws, one per row, from any distribution of the distribution calculator. Choose the distribution and its parameters, the column's name and a **seed**: the same seed always gives the same values, so a random column can be recreated exactly. The column is added at the end of the table as one undoable step. (Formula columns also have seeded random-number functions for values that should follow the rows.) Gamma and beta draws with very small shapes (down to about 0.005) are formed on the log scale, so they are never missing; a gamma draw is 0 only when its true value is below the smallest number a computer can store.

Rank columns

Data ▸ New rank column… adds a formula column of the ranks of a numeric column, 1 at the smallest value, optionally restarting in each group of another column (**Rank within groups of**). **Tied values** chooses how equal values are ranked:

Rule	Ties in 10, 20, 20, 30 get	Formula
Average	1, 2.5, 2.5, 4	`rank_ties("x", "average")` (same as `rank("x")`)
Minimum	1, 2, 2, 4	`rank_ties("x", "min")`
Maximum	1, 3, 3, 4	`rank_ties("x", "max")`
Dense	1, 2, 2, 3	`rank_ties("x", "dense")`
First	1, 2, 3, 4 (row order)	`rank_ties("x", "first")`

Missing values get no rank and are not counted. Because the column is a formula, the ranks follow later edits; Convert formula to values freezes them. Scripts rank an array with `ranks(values, "min")`.

Clearing and filling

- **Delete** (or Backspace, or Edit ▸ Clear cells) makes the selected cells, or the cell at the cursor, missing.
- **Cut** (⌘X / Ctrl+X) copies the selection and then clears it.
- **Fill down** (⌘D / Ctrl+D, or Edit ▸ Fill down) copies the first row of the selected block into the rows below it, in each selected column.

Each of these undoes as one step. Cells of formula columns are computed, so they are left as they are.

Rows and columns

The **Edit** menu adds rows at the end (**Add rows ▸**, with **How many**), deletes selected rows or selected columns (one undoable step; a column used by an open report cannot be deleted until that report is closed), and adds columns at the end (**Add column ▸** Numbers,

Text, Dates, Formula… or Random values…; the first four are the toolbar's choices). The toolbar's **+ Row** opens a menu with **How many** and **Insert rows above**, **Insert rows below** or **Add rows at the end**. Above and below are relative to the selected rows (above the first, below the last) or, with no rows selected, the cell cursor's row; the menu says which, and the two are greyed out when there is neither. With several rows selected, **How many** starts at their number. **+ Column** works the same way for columns, with a **Type** (Numbers, Text or Dates) and a **Name**: **Insert columns to the left**, **to the right** (of the selected columns, or the cell cursor's column) or **Add columns at the end**, and the same three places for one formula column (**Insert formula column to the left…**, **to the right…**, **Add formula column at the end…**). Opening either menu ends a cell edit in progress, keeping what was typed.

Every add or insert action except Formula… has a **How many** field (starting at 1; up to 100,000 rows or 1,000 columns), and the menu items say what they will do ("Insert 3 columns to the right"). The number is remembered for the rest of the session, separately for rows and columns. When the action applies to a selection (a row number among several selected rows, or a heading among several selected headings), **How many** starts at the selection's size and the new rows or columns go beside the whole selection: above its first row or below its last, left of its first column or right of its last.

Column inserts also have a **Name** field. Left empty, new columns are named Column 1, Column 2, …; a name is used as typed for one column and numbered for several (**Rep** gives Rep 1, Rep 2, …). Names already in use are skipped. The name is cleared once used.

⌘⇧I / Ctrl+Shift+I inserts without a menu: with column headings selected, as many number columns to their left; otherwise as many empty rows as the cell block spans (one for just the cell cursor), above it.

Everything one action adds undoes as one step, and when it adds more than one row or column the status bar says so ("Added 5 rows. Undo removes them.").

Right-click a **row** for insert above or below (as many as **How many** says), delete, and to select, exclude, include, hide or show it (or the selected rows).

Right-click a **column heading** for:

- **Rename…** (formulas that use the column are rewritten to the new name). Double-clicking the heading renames it too.
- **Sort rows ascending / descending** by this column (missing values last; one undoable step). Data ▸ Rows ▸ Sort… sorts by several columns.
- **Recode values…** (see Recoding values).

- **Insert column to the left / right**: as many as **How many** says, of the type picked above them (Numbers, Text or Dates, remembered for the session), with the **Name** given. **Insert formula column to the left… / right…**
- **Delete column**, or **Delete N selected columns** when the heading is one of several selected. The toolbar's **Delete selected columns** does the same.
- **Data type:** numbers, text or date. Converting text to numbers or dates makes blanks and missing-value markers such as `NA` missing; any other value that cannot be read is also made missing, and a message says how many. An ordinal column stays ordinal (so ratings typed as text keep their order); a nominal text column becomes continuous.
- **Measurement level:** continuous, nominal or ordinal. The level decides how an analysis treats the column (for example, a nominal predictor in a linear model is fitted as a categorical effect). Text columns cannot be continuous.
- **Date display** for date columns (see Dates and times).
- **Compute with a formula…**, **Edit formula…** and **Convert formula to values** (see Formula columns).

A column that a report uses, or that a formula uses, cannot be deleted until that report is closed or that formula changed. Asking to delete a column a report uses brings up a box, *Column not deleted*, that names the reports using it; **Show the reports** goes to them so you can close them (× on a report's heading), and **OK** keeps everything as it was. A column used by a dashboard tile is refused the same way.

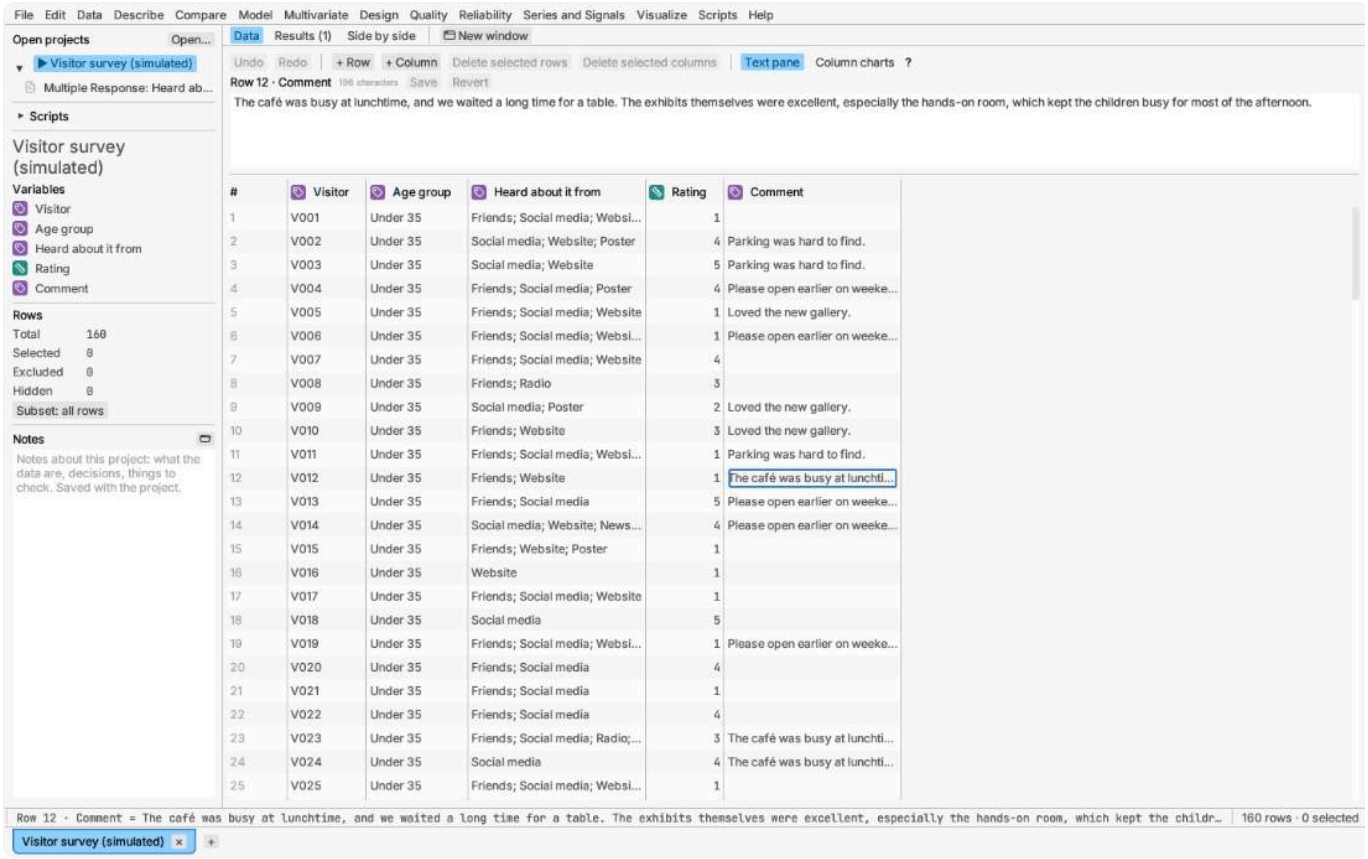
Inserting or deleting a column (or undoing either) in front of other columns closes any open analysis dialog for that table, so that it never runs on columns that have moved; open it again to choose the columns afresh. Adding a column at the end leaves open dialogs alone. An open Recode dialog stays on its column, and closes if that column is deleted.

Long text: the text pane

A grid cell shows only one line. To read or edit a long value, such as a comment or a description, turn on **Text pane** in the grid's toolbar. A pane above the grid then shows the cursor cell's whole value in a multi-line editor, with its row, column and length in characters.

- Type in the pane, then **Save** (or ⌘/Ctrl + Enter) to store the text. Moving the cell cursor to another cell stores it too; **Revert** puts back the cell's value.
- The edit is one cell edit: Edit ▸ Undo reverses it, and reports re-run as for any edit. The text is checked against the column's type, so a numeric column still takes only

numbers (the reason shows above the pane).
- Cells of a formula column are shown but cannot be edited.



Entry errors

A value the column cannot hold (letters in a numeric column, an unreadable date) keeps the cell open, outlined in red, with the reason just below it. Correct it, press Escape to cancel, or change the column's data type.

Undo and redo

Edit ▸ Undo and Redo (⌘Z / Ctrl+Z and the usual redo shortcut) step through the last 500 edits. Edits that replace whole columns of a large table take a lot of memory, so the history also stops at about 512 MB, forgetting the oldest edits first (the latest edit can always be undone). The menu names the edit, for example "Undo Paste". Pastes, sorts and formula changes each undo as one step.

Reports follow your edits

When you change the data, every report that depends on it re-runs, so results always reflect the current table.

Dates and times

A date column holds points in time (or lengths of time) and has four displays, chosen from the column's **Date display** menu:

Display	Example
Date	2026-10-13
Date and time	2026-10-13 15:57:58
Time of day	15:57:58 (seconds after midnight)
Duration	1:30:00, or 2 03:15:00 for two days and more

The same menu sets **Decimal places of seconds** (up to six) and **Show at** a UTC offset.

Importing dates

Import recognises ISO 8601 and RFC 3339 text, slash dates (`10/13/2026`), month-name dates (`13 Oct 2026`) and spreadsheet date cells (in both the 1900 and 1904 serial systems). In `.ods` files, times of day arrive as durations.

Slash dates such as `03/04/2026` are ambiguous. When a column's order cannot be told apart from its values, it is read month first and the import says so. If the column was really day first, choose **Swap day and month** from its Data type menu.

Converting

A date column converts to and from text, and to and from numbers (seconds since 1 January 1970, UTC). Text converted to a column shown as **Date** keeps only the day of each value, as typing does: a time of day in the text is dropped. In formulas, the date functions extract parts, take differences, add intervals and format dates (see Formula columns).

Dates in analyses and plots

Dates analyse as continuous values. In Describe ▸ Distribution, locations (mean, quantiles, minimum and maximum) are reported as dates and spreads (standard deviation, MAD, IQR) as lengths of time. Plot axes for dates tick on calendar units (seconds, minutes, hours, days, months or years) and are labelled as dates and times, with no zero baseline.

Exporting

Text and JSON export write ISO 8601. `.xlsx` export writes date cells with number formats, so spreadsheets show them as dates.

Copying and pasting

Selecting what to copy

Drag across cells, Shift-click, or hold Shift with the arrow keys to select a block of cells; click column headings to select whole columns (see Editing the data grid).

Copy

Edit ▸ Copy (⌘C / Ctrl+C) puts text on the clipboard, tab-separated so it pastes straight into a spreadsheet:

- the selected columns, every row, with a first line of column names;
- otherwise, a selected block of cells;
- otherwise, when two or more rows are selected, those rows with a first line of column names;
- otherwise, the cell under the cursor.

Edit ▸ Cut (⌘X / Ctrl+X) copies the selected columns or block of cells (or the cell under the cursor) and then clears them. It never takes whole selected rows; use Delete selected rows for that.

Edit ▸ Copy selected rows with column names copies the selected rows (any number) with their column names.

Paste

Edit ▸ Paste (⌘V / Ctrl+V) reads tab- or comma-separated text into the grid, starting at the cell cursor (or at the top of the first selected column). Text with a tab is split at tabs. Text without tabs is split at commas only when every line splits into the same number of fields, not every comma is followed by a space (so a column of names written `Smith, John` stays one column), the table has more than one column, and the pieces fit the columns; otherwise each line is one value. The status line says when the text was split on commas.

- Rows and columns are added as needed. New columns take their type from the pasted values.
- Every value is checked against its column's type. One bad value stops the whole paste, and nothing changes.
- A first line that repeats the column names is skipped.
- One Undo removes the whole paste.

File ▸ New table from clipboard makes a new table from the clipboard text through the Open Data dialog, which proposes a first line of names as the column names and skips title lines above the table.

Formula columns

A formula column computes its values from the other columns, one row at a time, and recomputes them whenever the data changes: after an edit, a paste, a sort, rows added or deleted, an undo or redo, or a script's changes.

Only what a change affects is recomputed. A formula that reads just its own row's values recomputes only the rows edited or added (and nothing at all after a sort or a deletion, since its values move with their rows), so editing a cell of a large table stays quick. A formula that reads other rows or the row's position — `lag`, `lead`, `col("name", row)`, the column summaries such as `col_mean`, ranks, cumulative sums, smoothing and filters, `row()`, `n_rows()` and the random numbers — recomputes every row.

Making one

- **Edit ▸ Add column ▸ Formula...** (or the grid toolbar's **+ Column ▸ Formula...**) adds a new column at the end.
- Right-click a column name for **Insert formula column to the right...**, **Compute with a formula...** (replaces that column's values; Undo puts them back), **Edit formula...** and **Convert formula to values** (keeps the current values as entered data).
- In a script: `t.set_formula("BMI", "Weight / Height ** 2")` (see scripting).
- From a model report (linear, generalized linear, lasso/ridge/elastic net, nonlinear regression): **Save prediction formula** in the report's options menu (see below).

Prediction formulas

Each of these reports ends with a **Prediction Formula** section that shows the equation (with a **Copy formula** button). **Save prediction formula** writes it as a formula column named "Predicted *y* formula". It computes the same values as **Save predicted values**, but keeps working as the data changes: edit a predictor or add a row and its prediction updates. The coefficients are written with every digit, so the values agree with the report's predictions. For example, a model with a continuous `Dose`, a categorical `Site` and their interaction gives:

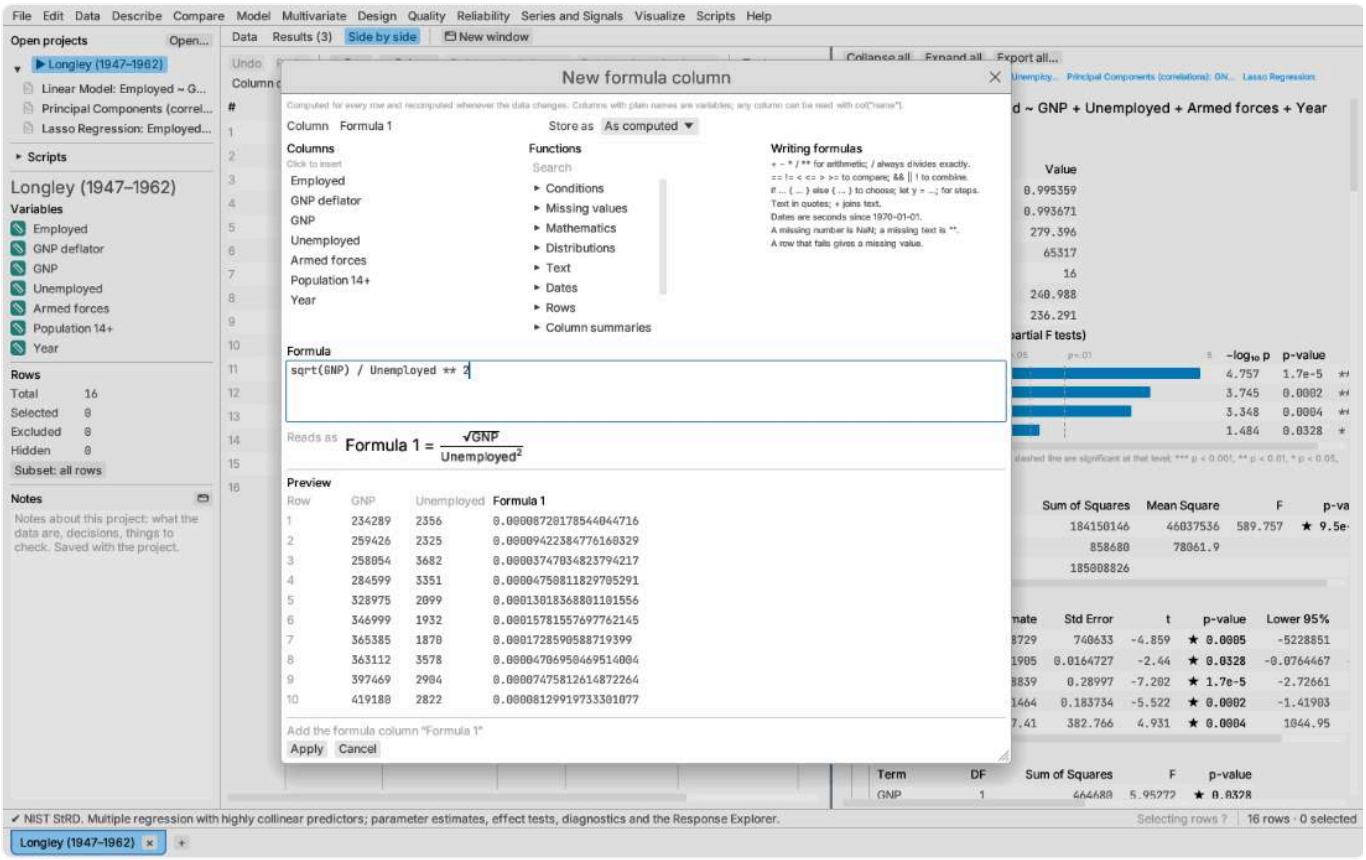
```
-1.25
+ 0.75 * Dose
+ (switch text(Site) { "East" => 0, "North" => 0.4, "West" => -0.2, _ => 0.0 /
0.0 })
+ (switch text(Site) { "East" => 0, "North" => 0.1, "West" => 0.3, _ => 0.0 /
0.0 }) * (Dose - 2.5)
```

- Each categorical effect is a `switch` giving every level its effect (for an interaction of two categorical factors, every pair of levels, joined by a tab). A level the model was not fitted to falls to `0.0 / 0.0` (not a number), which gives a missing prediction without counting as a failed row.
- In interactions and powers a continuous factor is centred at its mean over the rows the model was fitted to (here `Dose - 2.5`), as in the report.
- Logistic models give `logistic(…)` (the probability of the modelled level) and Poisson models `exp(…)` (the mean count). Penalized models use the λ_1SE fit and leave out predictors with a zero coefficient.
- Nonlinear regression gives its curve with the estimates written in, such as `53.52 / (1 + exp(-0.2777 * (Day - 17.68)))` (with every digit), or a model formula preceded by one `let` per parameter: `let Vmax = 212.68…; let Km = 0.0641…; Vmax * Conc / (Km + Conc)`. The model formula itself (Model ▸ Nonlinear regression…) is written in the same language, with the parameters as variables.
- A spline term (Model ▸ Linear model… or Generalized linear model…, More options) is written as one basis column per coefficient: `natural_spline(x, [knots], k)` for a natural cubic spline and `bspline(x, [knots], degree, k)` for a B-spline, where `[knots]` lists the lower boundary knot, the interior knots and the upper boundary knot, and `k` (from 1) picks the column. Beyond the boundary knots a natural spline continues as a straight line and a B-spline as its end polynomial.
- A date-time column used as a category cannot be written into a formula, so that option is not offered for such a model.
- The formula is fixed when saved: refitting the model (for example after excluding rows) does not change it. Save it again for the new fit.

The formula editor lists the table's columns and the functions below; a click inserts one at the text cursor. The preview shows the first rows as you type, and a formula that cannot be read is reported with its line and column and a caret under the place.

Under the text, **Reads as** draws the formula as mathematics, so a long expression can be checked at a glance: `/` becomes a fraction bar, `**` a raised power, `sqrt(x)` a radical sign, `abs(x)` |x|, `exp(x)` *e* to a power, `floor` and `ceiling` ⌊x⌋ and ⌈x⌉, `normal_cdf(z)` Φ(z), `col_sum("x")` Σ x, `cum_sum("x")` a running sum Σ over rows $j \le i$, and `lag("x")` and

`lead("x", 2)` the row subscripts x_{i-1} and x_{i+2} (*i* is this row, as `row()`, and *n* the number of rows, as `n_rows()`). Parentheses appear where the order of computing needs them: `-x ** 2` is drawn $(-x)^2$, because a leading minus applies before the power; write `-(x ** 2)` for $-x^2$. Comparisons read ≤, ≥, ≠ and `&&`, `||`, `!` read *and*, *or*, *not*. Only a single expression is drawn: a formula with steps (`let`, `;`), `if` or `switch` has no typeset view, and nor does one that cannot be read.



Before the formula is applied every row is computed; if some rows fail (for example `ln` of text) the editor says how many and why, and **Apply anyway** stores them as missing values. **Apply** is unavailable, with a note saying what is missing, until the column has a name and a formula; inside the editor Enter starts a new line, so press ⌘/Ctrl+Enter to apply from the keyboard.

Store as chooses the column's type: *As computed* keeps numbers as numbers and makes the column text when any value is text; a date format shows the numbers (seconds) as dates.

A formula column's cells cannot be typed into, and a column that a formula uses cannot be deleted until that formula is changed or converted to values. Renaming a column rewrites the formulas that use it. Formulas may use other formula columns, but not in a circle. Formulas are saved in project files with the values they computed, and recorded scripts

include them. If a project file holds formulas that cannot be computed (they use each other in a circle, or no longer compile, as can happen in a hand-edited file), those columns keep their saved values and the others are computed as usual; each such column is named in a note when the project is read.

Tables made by table operations (sort into a new table, subset, join, …) hold values, not formulas.

Writing formulas

Formulas are written in Rhai, the language scripts use, and are evaluated once per row:

- A column whose name is a plain identifier (letters, digits and `_`, not starting with a digit) is a variable holding this row's value: `Weight / (Height / 100) ** 2`. Any column can be read by name with `col("Wait (min)")`.
- Arithmetic: `+ - * / % **`. `/` always divides exactly (`1 / 2` is `0.5`).
- Comparison and logic: `== != < <= > >=`, `&&`, `||`, `!`.
- Choices: `if x > 5 { "high" } else { "low" }`, or `switch`.
- Several steps: `let bmi = Weight / Height ** 2; round(bmi, 1)` (the last expression is the value).
- Loops: `let s = 0; for k in 1..4 { s += k * x } s`, or `for (v, i) in [a, b, c] { … }` for each value with its position. A variable named in `let` or `for` is the formula's own, even when a column has the same name: inside the loop body `k` is the loop variable, not a column `k` (write `col("k")` for the column), and renaming column `k` leaves the loop alone.
- Text is written in quotes; `+` joins text (`concat` writes numbers the way the data view does).
- Dates and times are numbers of seconds since 1970-01-01 00:00 UTC.
- A missing number is `NaN`, so arithmetic on it gives a missing value and comparisons with it are false; a missing text value is `""`. Use `is_missing(x)` to test.
- The result may be a number, `true` / `false` (stored as 1 and 0), text, or `missing()`. A row whose formula fails gets a missing value.

Each row is limited to 200 000 operations, so a runaway loop cannot hang the program. Simple formulas compute a million rows in about a second.

Functions

Conditions

Function	What it gives
`if x > 0 { "positive" } else { "not positive" }`	Choose a value by a condition.
`switch x { 1 => "one", 2 | 3 => "two or three", _ => "other" }`	Choose a value by matching cases.
`x > 0 && y > 0`	Both conditions hold (`||` for either, ! for not).

Missing values

Function	What it gives
`is_missing(x)`	true when x is missing.
`if_missing(x, 0)`	x, or the second value when x is missing.
`missing()`	A missing value.

Mathematics

Function	What it gives
`sqrt(x)`	Square root.
`x ** 2`	Power (also pow(x, y)).
`exp(x)`	e to the power x.
`ln(x)`	Natural logarithm.
`log10(x)`	Base-10 logarithm (also log2).
`abs(x)`	Absolute value.
`round(x, 2)`	Round to a number of decimals (round(x) to a whole number).
`floor(x)`	Largest whole number not above x (ceiling for the smallest not below).

Function	What it gives
`min(x, y)`	Smaller of two values (max for the larger).
`modulo(x, y)`	Remainder after division, never negative.
`sign(x)`	−1, 0 or 1.
`logit(p)`	Log odds ln(p / (1 − p)); logistic(x) is its inverse.
`lambert_w(x)`	Lambert W: the w ≥ −1 with $w \cdot e^w = x$ ($x \geq -1/e$).
`wright_omega(z)`	Wright omega: the w with w + ln w = z, which is lambert_w(exp(z)) without overflow (used by the enzyme progress-curve template).
`natural_spline(x, [a, k1, …, b], k)`	Column k (from 1) of the natural cubic spline basis with boundary knots a and b and the interior knots between, as spline terms of linear models and GLMs use it.
`bspline(x, [a, k1, …, b], degree, k)`	Column k of the B-spline basis of that degree (1 to 3) without the intercept column.
`sin(x)`	Sine of an angle in radians (also cos, tan, asin, acos, atan, atan2, and the hyperbolic tangent tanh).

Distributions

Function	What it gives
`normal_cdf(x)`	Standard normal probability P(Z ≤ x); normal_cdf(x, mean, sd).
`normal_pdf(x)`	Standard normal density; normal_pdf(x, mean, sd).
`normal_quantile(p)`	Standard normal quantile; normal_quantile(p, mean, sd).
`t_cdf(x, df)`	Student's t probability P(T ≤ x).
`t_quantile(p, df)`	Student's t quantile.
`chisq_cdf(x, df)`	Chi-square probability P(X ≤ x).
`chisq_quantile(p, df)`	Chi-square quantile.
`f_cdf(x, df1, df2)`	F probability P(F ≤ x).
`f_quantile(p, df1, df2)`	F quantile.
`binomial_pmf(k, n, p)`	Binomial probability of k successes.

Function	What it gives
`binomial_cdf(k, n, p)`	Binomial probability of at most k successes.
`poisson_pmf(k, mean)`	Poisson probability of k events.
`poisson_cdf(k, mean)`	Poisson probability of at most k events.

Text

Function	What it gives
`concat(a, " ", b)`	Join values as text (up to six).
`text(x)`	A number as text, written as in the data view.
`number(s)`	Text read as a number (missing if it is not one).
`upper(s)`	Capital letters (lower for small letters).
`trim(s)`	Without spaces at either end.
`length(s)`	Number of characters.
`substr(s, 0, 3)`	Characters from a position (from 0), of a length.
`find(s, "x")`	Position of the first match (from 0), or −1.
`replace(s, "old", "new")`	Replace every match.
`word(s, 0, ",")`	The nth piece (from 0) between separators; word(s, n) splits at spaces.
`has_item(s, "Email", ";")`	true if a check-all-that-apply list holds the item (spaces around items ignored; separator `;` when left out). Use it as a row filter, or to make one yes/no column per item.
`item_count(s, ";")`	How many different items a check-all-that-apply list holds.
`s.contains("x")`	true if the text contains x (also starts_with, ends_with).
`regex_match(s, "^[A-Z]\\d+$")`	true if a regular expression matches.
`regex_find(s, "\\d+")`	The first match of a regular expression.
`regex_replace(s, "\\s+", " ")`	Replace every match of a regular expression ($1 for groups).

Dates

Function	What it gives
`year(d)`	Year of a date (also month, day, hour, minute, second).
`weekday(d)`	Day of the week, 1 (Monday) to 7 (Sunday).
`day_of_year(d)`	Day of the year, from 1.
`date_part(d)`	The date at midnight, without the time of day.
`make_date(2026, 1, 31)`	A date from year, month and day (make_datetime adds h, m, s).
`date_diff(start, end, "days")`	Time from start to end in a unit (whole months or years).
`date_add(d, 1, "months")`	A date moved by a number of units.
`format_date(d, "%d %b %Y")`	A date as text (%Y %m %d %H %M %S %j %B %b %A %a).

Rows

Function	What it gives
`row()`	This row's number (from 1).
`n_rows()`	Number of rows.
`lag("column", 1)`	The value a number of rows earlier (missing before the first row).
`lead("column", 1)`	The value a number of rows later.
`col("column")`	This row's value of any column, by name.
`col("column", 1)`	The value in a given row (from 1).

Column summaries

Function	What it gives
`col_mean("column")`	Mean of a column; col_mean("column", "group") within each group.

Function	What it gives
`col_sd("column")`	Standard deviation (n − 1), optionally by a group column.
`col_min("column")`	Smallest value (col_max for the largest), optionally by group.
`col_sum("column")`	Sum, optionally by group.
`col_count("column")`	Number of non-missing values, optionally by group.
`col_quantile("column", 0.5)`	Quantile (Hyndman–Fan type 8), optionally by group as a third argument.
`cum_sum("column")`	Running total down to this row, optionally by group.
`rank("column")`	Rank from 1 (ties averaged), optionally by group.
`rank_ties("column", "min")`	Rank from 1 with a tie rule: "average", "min" (1, 2, 2, 4), "max" (1, 3, 3, 4), "dense" (1, 2, 2, 3) or "first" (row order); optionally by group as a third argument.
`percent_of_range("column")`	The value as a percentage of the column's range (0 at the smallest value, 100 at the largest), optionally by group.
`fraction_of_total("column")`	The value divided by the column's total, optionally by group.
`subtract_baseline("column", 3)`	The value minus the mean of the column's first 3 values in row order (a baseline), optionally by group as a third argument.
`p_adjust("column", "bh")`	The column's p-values adjusted for multiple testing: `"bonferroni"`, `"sidak"`, `"holm"`, `"hochberg"`, `"bh"` (Benjamini–Hochberg), `"by"` (Benjamini–Yekutieli) or `"q"` (Storey's q-values, λ = 0.5); optionally within each group as a third argument. Missing p-values stay missing and are not counted.

Signals

These treat a column as a signal in row order (or, for the x–y functions, sorted by an x column). Each takes an optional group column as its last argument, and then works on each group separately. Missing values are skipped and stay missing.

Function	What it gives
`savgol("column", 11, 3)`	Savitzky–Golay smoothing: a polynomial of degree 3 fitted by least squares over 11 rows centred on this one (window odd, larger than the degree); the ends use the fit over the first or last window.
`moving_average("column", 5)`	Centred moving average of 5 rows (odd window; the series is mirrored beyond its ends).
`running_median("column", 5)`	Running median of 5 rows (odd window, mirrored ends), resistant to spikes.
`exp_smooth("column", 0.3)`	Exponential smoothing: $s_1 = y_1$, $s_t = 0.3 \cdot y_t + 0.7 \cdot s_{t-1}$.
`gaussian_smooth("column", 2)`	Gaussian kernel smoothing with σ = 2 rows (kernel cut at 4σ, mirrored ends).
`derivative("y", "x")`	dy/dx: central differences on the rows sorted by x (unequal spacing allowed; duplicate x averaged), one-sided at the ends.
`cum_integral("y", "x")`	Running integral of y over x by the trapezoid rule, from 0 at the smallest x.
`interpolate("y", "x", 2.5)`	y read off at x = 2.5 by straight lines between the points; missing outside the range of x.
`interpolate_spline("y", "x", 2.5)`	The same by a not-a-knot cubic spline.
`interpolate_pchip("y", "x", 2.5)`	The same by a shape-preserving piecewise cubic (PCHIP, Fritsch–Carlson), which never overshoots the data.

Signal processing

These work down a column in row order, optionally within each group (a last argument naming a column), on the rows with a value; missing values stay missing.

Function	What it gives
`lowpass_filter("column", 0.1)`	The column through a fourth-order Butterworth low-pass filter run forwards and backwards (zero phase). The cut-off is in cycles per row, between 0 and 0.5. `highpass_filter` keeps the frequencies above the cut-off instead.
`bandpass_filter("column", 0.05, 0.2)`	Keeps the frequencies between two edges (cycles per row); `bandstop_filter` removes them.
`envelope("column")`	The envelope: the magnitude of the analytic signal from the Hilbert transform. `instant_phase` gives its unwrapped phase in radians.
`wavelet_denoise("column", "db4")`	The column denoised by soft-thresholding its wavelet detail coefficients (universal threshold, noise level from the finest level, all levels the length allows). The wavelet (`haar` , `db1` – `db10` , `sym2` – `sym10` , `coif1` – `coif5`) defaults to `db4` ; a group column is the third argument.

Random numbers

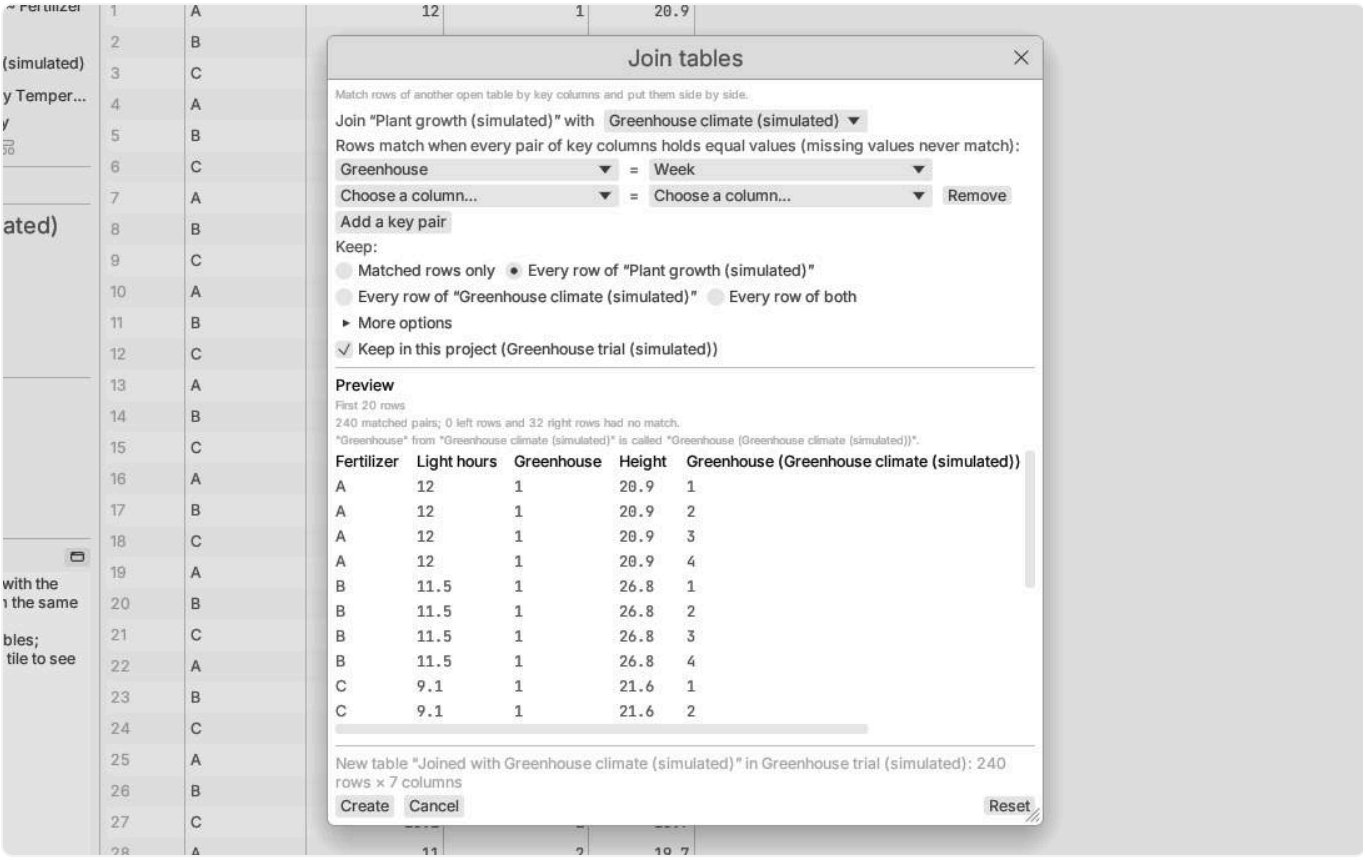
Function	What it gives
`random_uniform()`	Uniform on [0, 1); random_uniform(low, high).
`random_normal()`	Standard normal; random_normal(mean, sd).
`random_integer(1, 6)`	A whole number between two limits.
`random_seed(42); random_normal()`	Choose the seed (the default is 0).
Column names in `lag` , `lead` , `col` and the column summaries are written in	
quotes. Summaries skip missing values; `col_sd` divides by n − 1 and	
`col_quantile` uses Hyndman and Fan's type 8 definition, as the Distribution	

Function	What it gives
report does. Random numbers are drawn from a generator seeded by the formula's	
seed (`random_seed` , 0 by default) and the row number, so recomputing gives the	
same values; sorting the rows does not carry them along.	

Table operations

The **Data** menu reorganises tables. Every dialog previews its result before you apply it, and the line at its foot names the table it will make and its size. The main button says what it does: **Create** for a new table, **Sort** or **Remove duplicates** when working on the table in place. See Using analysis dialogs for the parts every dialog shares.

- **Sort** and **Remove duplicates** change the current table as one undoable edit (row states move with their rows), or optionally make a new table.
- The other operations open a **new table**. Where its rows map one-to-one to the source rows, it keeps their row states. It records how it was made, which shows in its tab's tooltip and is saved in the project.
- The new table gets a short name saying what it holds ("Summary by Fertilizer", "Fertilizer A only", "Stacked Employed, GNP"). With **Keep in this project** ticked (the default) it joins the source table's project and is listed under it; if the source is edited later, a **Source changed** banner offers to update it. Clear the box to open it as a separate project. See Tables made from other tables.



Rows

- **Sort…** by one or more columns, each ascending or descending. Missing values sort last; text sorts in natural order (`item2` before `item10`).
- **Remove duplicates…** keeps the first of each set of repeated rows, comparing all columns or the ones you choose.
- **Subset and sample…** makes a table of all, selected, included or matching rows and the columns you choose. Optionally take a random sample, as a count or a fraction, stratified by columns (proportional allocation) and seeded so it can be repeated.
- **Use row as column names** makes the row of the cell cursor (or the one selected row) the column names, removes it, and types text columns again (see The Open Data dialog); undoable.

Reshape

- **Stack columns…** (wide to long): chosen columns become a label column and a value column.
- **Unstack by label…** (long to wide): one column per level of a label column, rows matched by ID columns.
- **Transpose…**: rows become columns.
- **Distance matrix…**: the distance between every pair of included rows as a new table — an *Object* column naming the rows (by a label column you choose, or the row numbers) and one column per row, so row *i* of column *j* is the distance between rows *i* and *j*. Choose the distance (Euclidean, Manhattan, Gower, Jaccard, Bray–Curtis, Hamming or simple matching; see Distance measures) and the columns compared; Euclidean and Manhattan can standardize them first. Gower accepts text and ordinal columns and missing values; the other distances leave out rows with a missing value. Up to 3000 rows. The table feeds Multivariate ▸ Principal coordinates…, Dimension reduction… and hierarchical Clustering… as a distance matrix.

Merge

- **Combine rows…** appends the rows of other open tables, matching columns by name, with an optional column naming each row's source table.

- **Join…** matches rows of another table by key columns: inner, left, right or full outer. You can require the match to be many-to-one or one-to-one, and the join stops if the data break it. A missing key never matches anything, and a join in which no row matches (often the wrong key or an empty table) stops with a message instead of making a new table. If both tables have a column of the same name, the right table's copy gets its table's name in parentheses.

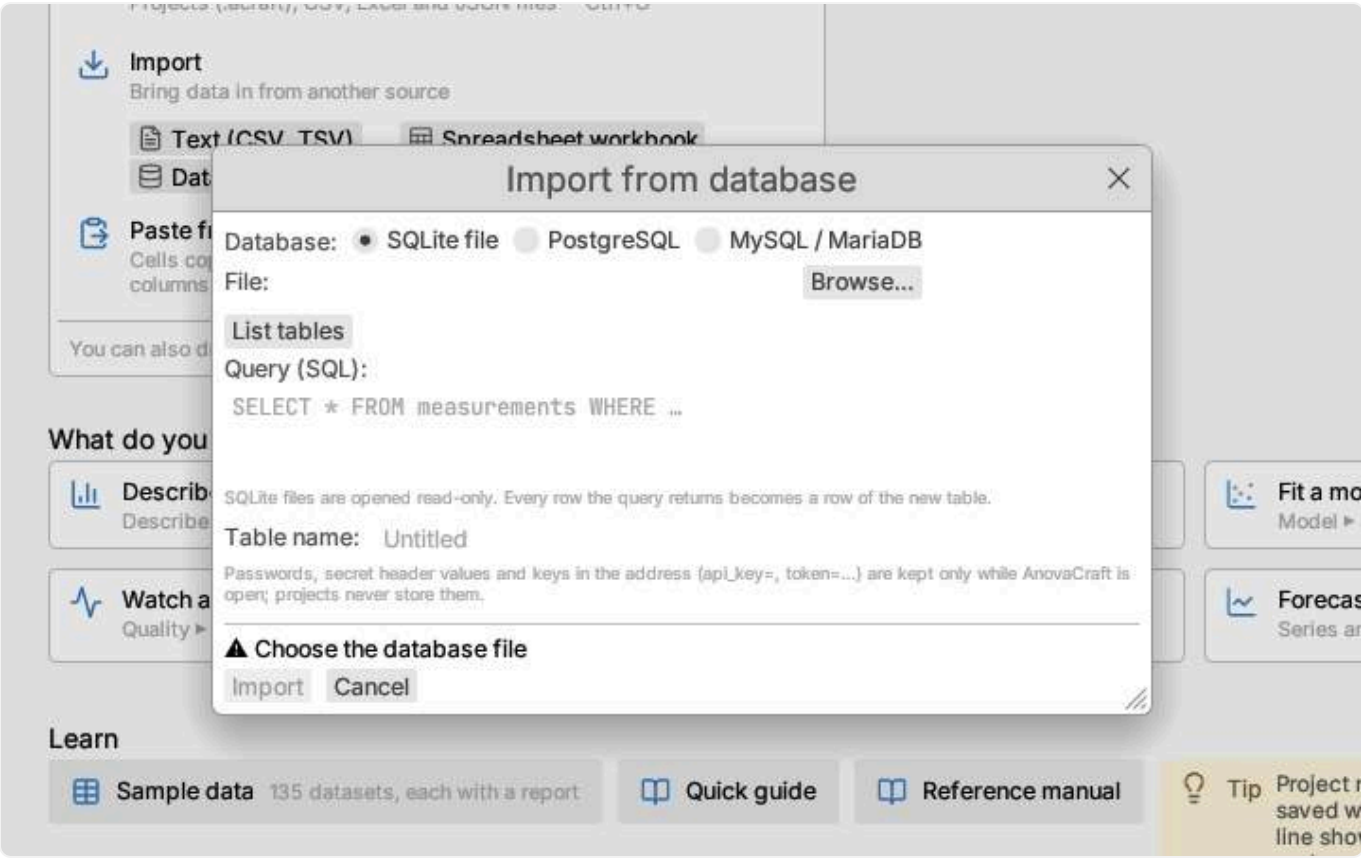
Summarize by group

Summarize by group… (Data ▸ Summarize by group…, also in the Describe menu) makes one row per group with N, missing count, mean, SD, minimum, maximum, median, quantiles and sum for each chosen column. Tick the columns to summarize in the left-hand list (**Summarize**) and the grouping columns in the right-hand list (**Group by**).

All of these operations are also available from scripts (see Scripting and recording).

Data from databases and the web

Besides files, a table can come from an SQL database or from an address on the web. **File ▸ Import from database…** and **File ▸ Import from the web…** read the rows into a new table; **Data ▸ Refresh from source** reads them again later.



Databases

Choose the kind of database at the top of the dialog:

Kind	What you give
SQLite file	The database file (**Browse…** or type its path). It is opened read-only.
PostgreSQL	Server, port (5432), database name, user and password.
MySQL / MariaDB	Server, port (3306), database name, user and password.

Kind	What you give
ODBC data source	A data source name or connection string (`DSN=plant;UID=ann` , or `Driver={…};Server=…`) and the password. Only in builds made with ODBC support (see below).

For a server, **Encryption** chooses whether the connection is encrypted: *If offered* (the default) encrypts when the server supports it (a MySQL connection is retried unencrypted only when encryption itself could not be set up, never after a refused login), *Required* refuses an unencrypted connection, and *Off* never encrypts. Encrypted connections trust the usual public certificate authorities.

List tables connects and lists the tables and views; click one to read it whole, or write any query in **Query (SQL)**. Every row the query returns becomes a row of the table, and the last result set is used when a query returns several. **Table name** defaults to the queried table's name.

Values arrive as text and are typed the way a CSV file is: a column of numbers becomes numeric, a column of dates or times becomes a date column, anything else is text. SQL `NULL` is a missing value. Errors from the database (a wrong password, a misspelt table) appear in the dialog, which stays open so you can correct the settings.

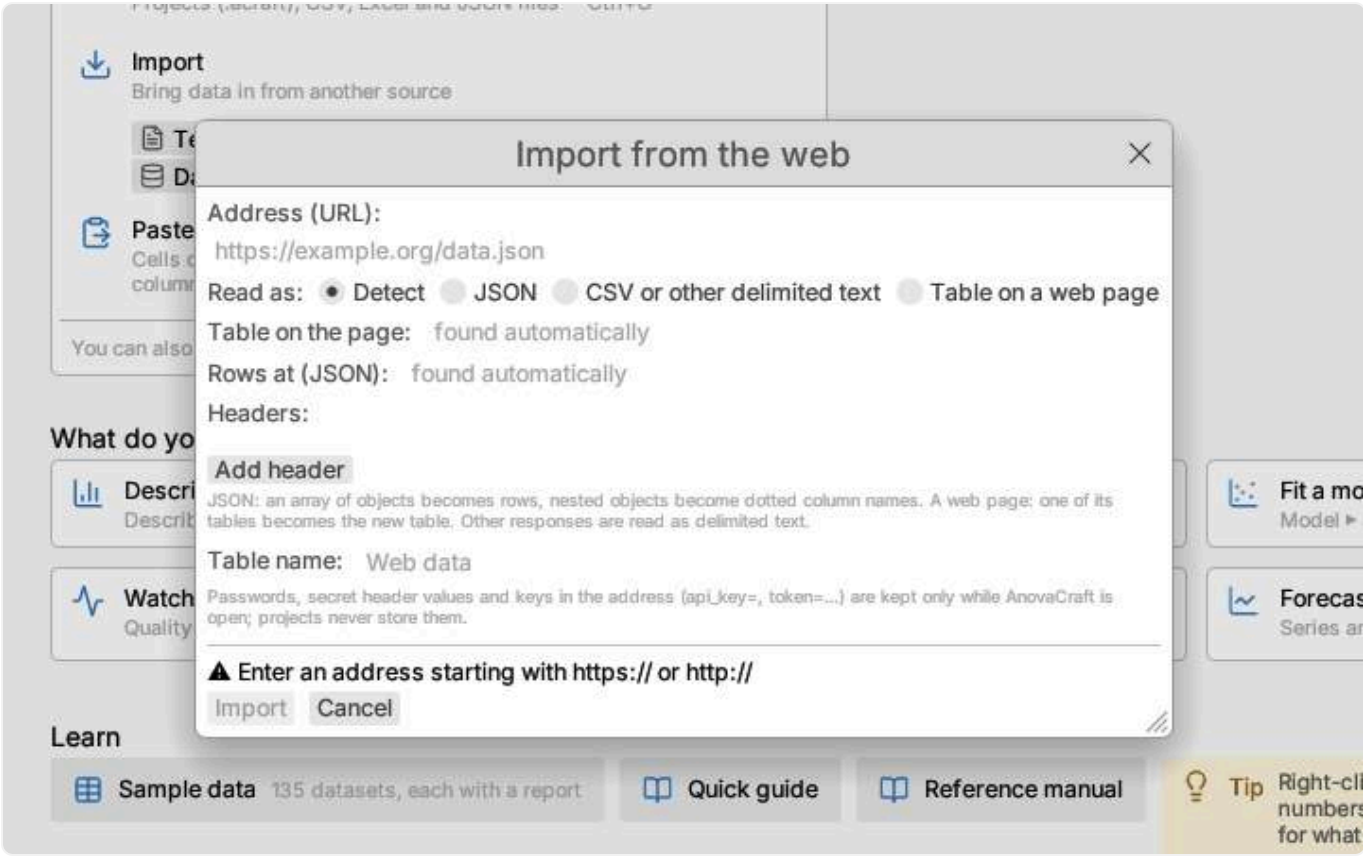
The web

Give the **Address (URL)** of a JSON or CSV document, or of a web page that shows its data in a table. The request is a plain HTTP GET; add **Headers** for services that need them, such as `Accept` or an API key. Tick **Secret** for a header holding a credential (it is ticked for you when the name looks like one: `Authorization` , `Cookie` , or a name mentioning a key, token, secret or password).

Read as is normally *Detect*: JSON when the server says so or the text starts with `[` or `{` , a table on a web page when the server sends HTML, otherwise delimited text with the delimiter detected. In a JSON document the rows are an array of objects: the document itself, the array at **Rows at** (a dotted path such as `data.items`), or, when that is left empty, the one member of the top-level object that holds such an array. Nested objects become dotted column names (`site.name` , `site.location.lat`); arrays inside a row are kept as their JSON text.

A web page (HTML) is read from one of its tables. Leave **Table on the page** empty to take the table the address's `#section` points to (for `https://example.org/specs#daily`, the first table after the heading with id `daily`), or the largest table when the address has no `#section`; or give a table's number (1 for the first table on the page) or its `id`. Heading rows become column names (a heading spanning several columns is repeated over them, so `Power` above `Min` and `Max` gives `Power Min` and `Power Max`), and cells that span rows or columns are repeated into each place they cover. The import note says which table was read and how many the page has. A table on a web page and delimited text then show the Open Data dialog, where you choose the row of names, the rows of values and the columns; **Refresh from source** reads the same block again. Pages that fill in their tables with JavaScript after they open cannot be read this way; look on the page for a download link or a data address instead.

When the import fails, the message says why in words. AnovaCraft identifies itself to web sites the way a desktop browser does, since many sites close the connection on other programs; a **User-Agent** header you give replaces it. On macOS and Windows, when a site drops the secure connection, AnovaCraft tries once more through the operating system's own secure connection, as the system's browser makes it. *Closed the connection without answering* means the site still turned the request away: check that the address opens in your browser, and look on the site for a download link or a data address.



Refreshing

A table read from a database or the web remembers where it came from (its tab's tooltip says so). **Data ▸ Refresh from source** reads it again and replaces its rows as one step that **Undo** reverses:

- columns keep their position, measurement level and formulas; new columns are added at the end; a column the source no longer has is removed, unless a report uses it (close that report first);
- row states (selection, exclusion, colours) are cleared, because the rows are new;
- every report re-runs on the new rows.

Passwords are never saved

Saving the project keeps the connection settings and the query, or the address and the headers, so the table can be refreshed after the project is reopened. It never keeps a password or a secret header's value: those stay in memory while AnovaCraft is open, and **Refresh from source** asks for them again in a new session. An ODBC connection string is saved without its `PWD=` part, and an address is saved without any password written into it (`https://user:password@…`) and with the values of query parameters that name a credential blanked (`?city=Oslo&api_key=…` is saved as `?city=Oslo&api_key=`; names containing `key`, `token`, `secret`, `password` or `auth`). **Refresh from source** asks for each of those again, as *name (in the address)*, in a new session.

Scripts

Scripts read the same sources with `read_database`, `database_tables` and `read_url`; see Scripting and recording.

ODBC support

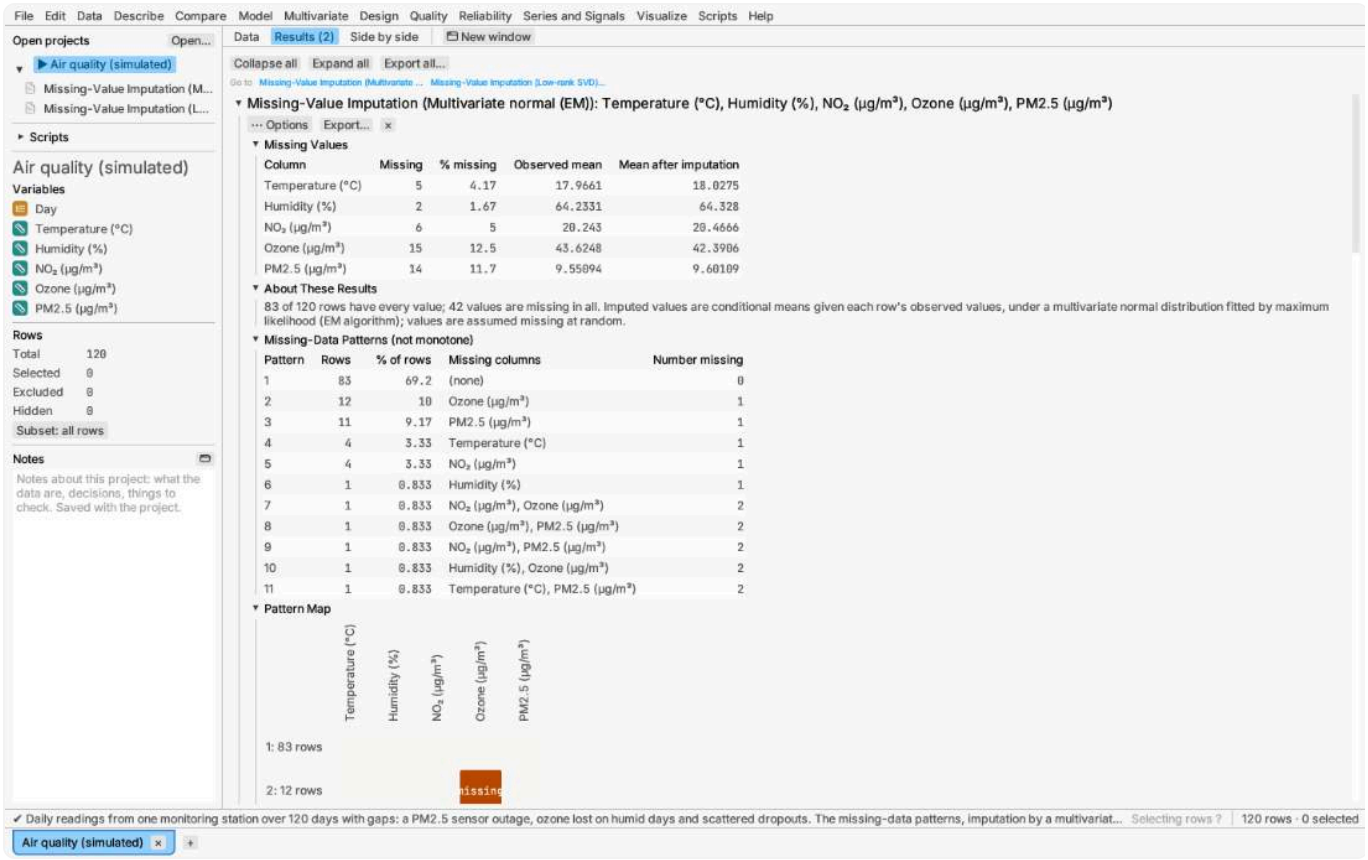
ODBC needs the operating system's ODBC driver manager (unixODBC or iODBC on macOS and Linux) and a driver for the database, so it is not part of the standard build. To build with it:

```
cargo run --release -p app-gui --features odbc
```

(on macOS, `brew install unixodbc` first).

Missing values and imputation

Data ▸ Impute missing values… shows where a set of numeric columns has missing values and fills them in from what each row does have. The completed columns are saved as new columns, so the original data stay as they were. The Air quality sample (File ▸ Open sample data ▸ Multivariate) has monitoring data with gaps and opens with two imputation reports.



Missing and excluded rows in reports

Reports that count rows (Missing, N used, and so on) count a row with a missing value as **missing** before they look at whether it is excluded: an excluded row whose value is missing adds to the missing count, not to the excluded count. Either way the row takes no part in the statistics. Every analysis follows the same order.

The dialog

Tick the **Columns** to impute; each is filled in using the others. Then choose a **Method**:

Method	How values are filled in	When it suits
Multivariate normal (EM)	Fits a multivariate normal distribution to the columns by maximum likelihood with the EM algorithm (Dempster, Laird & Rubin 1977), then fills each missing value with its expected value given the row's observed values.	Continuous columns that are roughly normal and related to each other.
Low-rank SVD	Fits the table as column means plus a few principal directions (a rank-*k* singular value decomposition) and refits until the filled-in values settle (Troyanskaya et al. 2001), shrinking the directions by the estimated noise so the refitting does not drift (regularized iterative PCA, Josse & Husson 2012).	Many related columns, or data that are not normal.
Column mean / **Column median**	The column's observed mean or median.	A baseline. It keeps the column's centre but understates its spread and its correlations.
Multiple imputation: chained equations / **Multiple imputation: multivariate normal**	Several completed copies drawn from the values' predictive distribution, for analyses pooled across them: see Multiple imputation.	Standard errors and tests that allow for the missing values.

For **Low-rank SVD**, **Rank** is either chosen for you (*Choose on held-out values* hides 10% of the observed values, imputes them at ranks 1 to 10 and keeps the rank that predicts them best) or fixed. **More options** holds *Regularize the fit (recommended)*, on by default: each kept singular value σ becomes $\sigma - n\hat{\sigma}^2/\sigma$, where $\hat{\sigma}^2$ is the noise variance estimated from the

discarded directions. Without it, refitting with more than one direction can chase its own filled-in values and drift far outside the data. It also holds the shrinkage λ (above 0 the singular values are soft-thresholded, the soft-impute method of Mazumder, Hastie & Tibshirani 2010, in place of the regularization), whether columns are standardized first, the seed for the held-out values, and the iteration limit and convergence tolerance of both iterative methods.

Rows excluded from analyses take no part, and their missing values are left missing. Values are assumed **missing at random**: whether a value is missing may depend on the row's other observed values, but not on the missing value itself. Imputed values are best guesses, not data; analyses of completed columns treat them as observed and so overstate precision. For honest standard errors, use multiple imputation.

The report

- **Missing Values:** for each column, how many values are missing, the mean of the observed values and the mean after imputation.
- **About These Results:** how many rows are complete, and what the method assumes.
- **Missing-Data Patterns:** each distinct set of missing columns with its number of rows. The title says whether the pattern is *monotone* (columns can be ordered so that a row missing one also misses every later one, as when people drop out of a study).
- **Pattern Map:** the patterns as rows of cells, one column per variable, with *missing* written in each missing cell (the 40 most frequent patterns).
- **Estimates:** the values filled in (mean and median), or the maximum-likelihood means, standard deviations and correlations (EM).
- **Rank Choice** (low-rank, rank chosen): the held-out error at each rank.
- **Warning: Not Converged** (first, only when it applies): the iteration limit was reached before the values settled, with what to try.
- **Convergence** and **Convergence Plot:** iterations, whether the method converged, whether the low-rank fit was regularized and its noise variance $\hat{\sigma}^2$, and the log-likelihood (EM) or the relative change (low-rank) at each iteration.
- **Imputed Values:** row, column and value of each imputed cell (the first 500).
- **Scatter plot** of the first two columns with every row's completed values, marked by which of the two values were imputed (colour and marker shape). Selecting points selects the rows everywhere.

Saving the results

From the report's ⋯ options menu:

- **Save *column* with imputed values** adds the column with its missing values filled in, named *column* imputed.
- **Save number of imputed values per row** adds a column counting the imputed values in each row (0 for complete rows), to flag or select the rows that were filled in.

Scripts do the same with `report.save_columns(...)`, using the keys `imputed1`, `imputed2`, … and `count`.

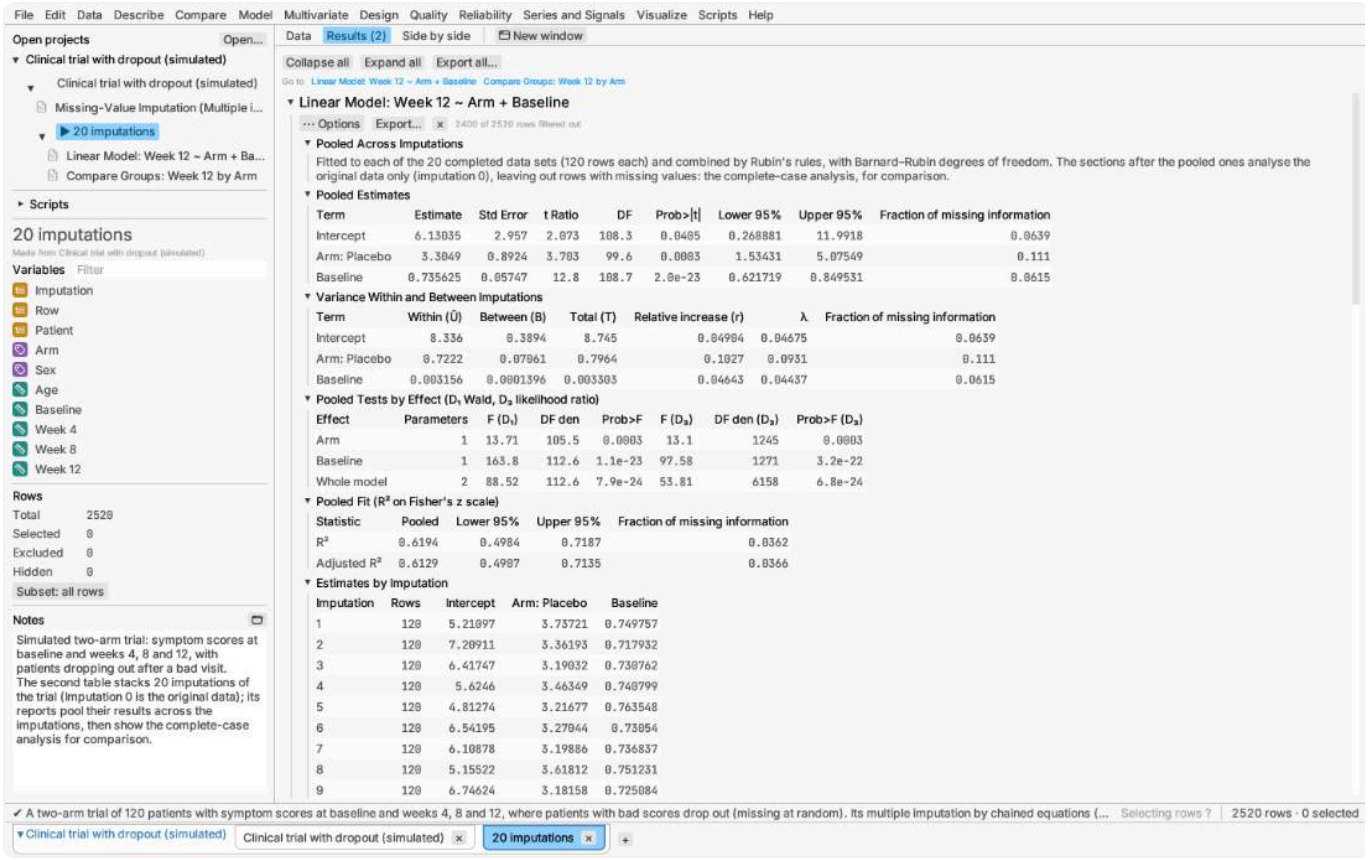
Multiple imputation

Filling each missing value once (Missing values and imputation) gives one completed table, and analyses of it treat the filled-in values as if they had been measured, so their standard errors are too small and their p-values too optimistic. **Multiple imputation** (Rubin 1987) fills the gaps *m* times instead, each time with values drawn from their predictive distribution, so the completed copies differ where the data say little. An analysis is run on each copy and the *m* results are combined; the spread between copies adds the uncertainty about the missing values to the usual sampling uncertainty.

The workflow has three steps:

1. **Data ▸ Impute missing values…** with one of the two *Multiple imputation* methods makes the imputations and a report to check them.
2. The imputations go into a **stacked table**: the original rows (Imputation 0) followed by each completed copy (Imputation 1 to *m*).
3. Analyses on the stacked table are **pooled across imputations**: fitted to each copy and combined by Rubin's rules.

To see it worked through, open **File ▸ Open sample data ▸ Projects ▸ Clinical trial with dropout**: a simulated two-arm trial in which patients with bad scores leave early, its multiple imputation, and the stacked table with a pooled analysis of covariance and a pooled comparison of the arms.

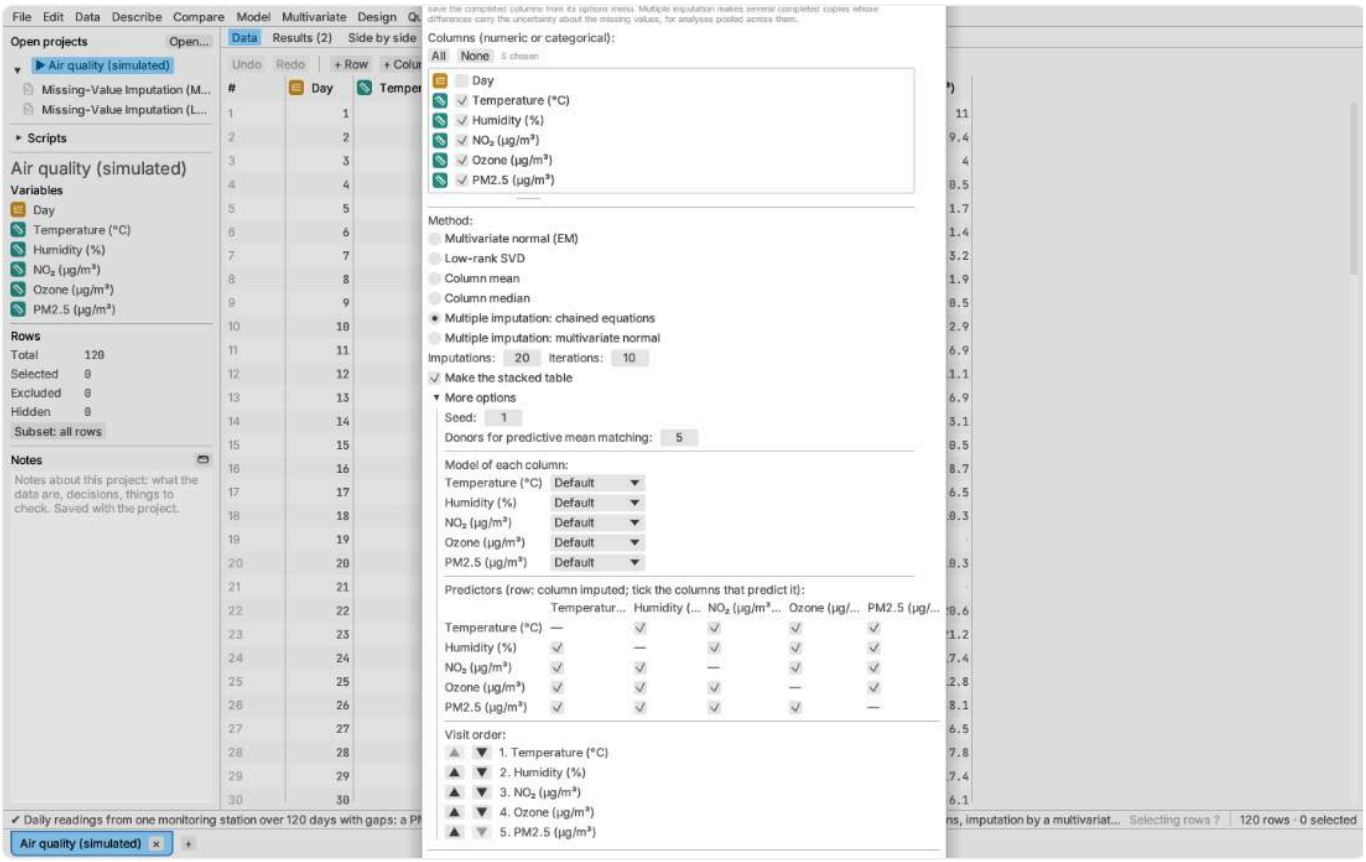


Making the imputations

In **Data ▸ Impute missing values...** tick the columns and choose a method:

Method	How copies are made	Columns
Multiple imputation: chained equations	Fully conditional specification (van Buuren & Groothuis-Oudshoorn 2011): each column with missing values in turn is filled from a model of it given the other columns, cycling through the columns for a number of *iterations*, one chain per imputation.	Continuous and categorical (text, nominal or ordinal)

Method	How copies are made	Columns
Multiple imputation: multivariate normal	Data augmentation (Schafer 1997): starting from the EM estimates, missing values are drawn given the current mean and covariance (I-step), then a mean and covariance are drawn given the completed data (P-step: inverse Wishart, then normal), and so on.	Continuous only



The settings are:

- **Imputations** (*m*, default 20). More imputations make pooled p-values and intervals repeatable; 20 is enough for most analyses, more when much information is missing (the *fraction of missing information* in pooled reports is a guide: *m* of about 100 × that fraction).
- **Iterations** of each chain before its copy is taken (10 for chained equations, 50 for the normal model). Check the convergence plots in the report.
- **Make the stacked table** (on by default) adds the stacked table to the project when the report is made.

- **More options:** the **Seed** (the same seed makes the same imputations; chain *l* uses seed + *l* − 1), and for chained equations:
    - **Donors for predictive mean matching** (5).
    - **Model of each column**. *Default* is predictive mean matching for a continuous column, logistic regression for a column of two categories, multinomial logit for more, and *not imputed* for a column without missing values (it still predicts the others).
        - *Predictive mean matching*: a Bayesian regression of the column on the others gives drawn coefficients; each missing value is copied from one of the 5 observed rows whose predictions are nearest, chosen at random. Imputed values are always values that occur in the data, so skewed or bounded columns stay in range.
        - *Bayesian linear regression*: σ^2 is drawn from its scaled inverse χ^2 posterior and the coefficients from their normal posterior given σ^2, and the value is the drawn prediction plus normal noise.
        - *Logistic regression* and *Multinomial logit*: the model is fitted by Newton's method with a small ridge, coefficients are drawn from their approximate normal posterior, and a category is drawn from the drawn probabilities.
        - *Not imputed*: the column's missing values are left missing.
    - **Predictors**: a grid with one row per imputed column; untick a box so that column no longer predicts the row's column (for example a later measurement that should not predict an earlier one).
    - **Visit order**: the order the columns are filled in within each iteration (▲ and ▼ move a column).

Rows excluded from analyses take no part. The methods assume values are **missing at random**: whether a value is missing may depend on the row's observed values but not on the missing value itself. Include the columns that predict missingness (in a trial, the earlier visits) among the imputation columns even when the analysis will not use them.

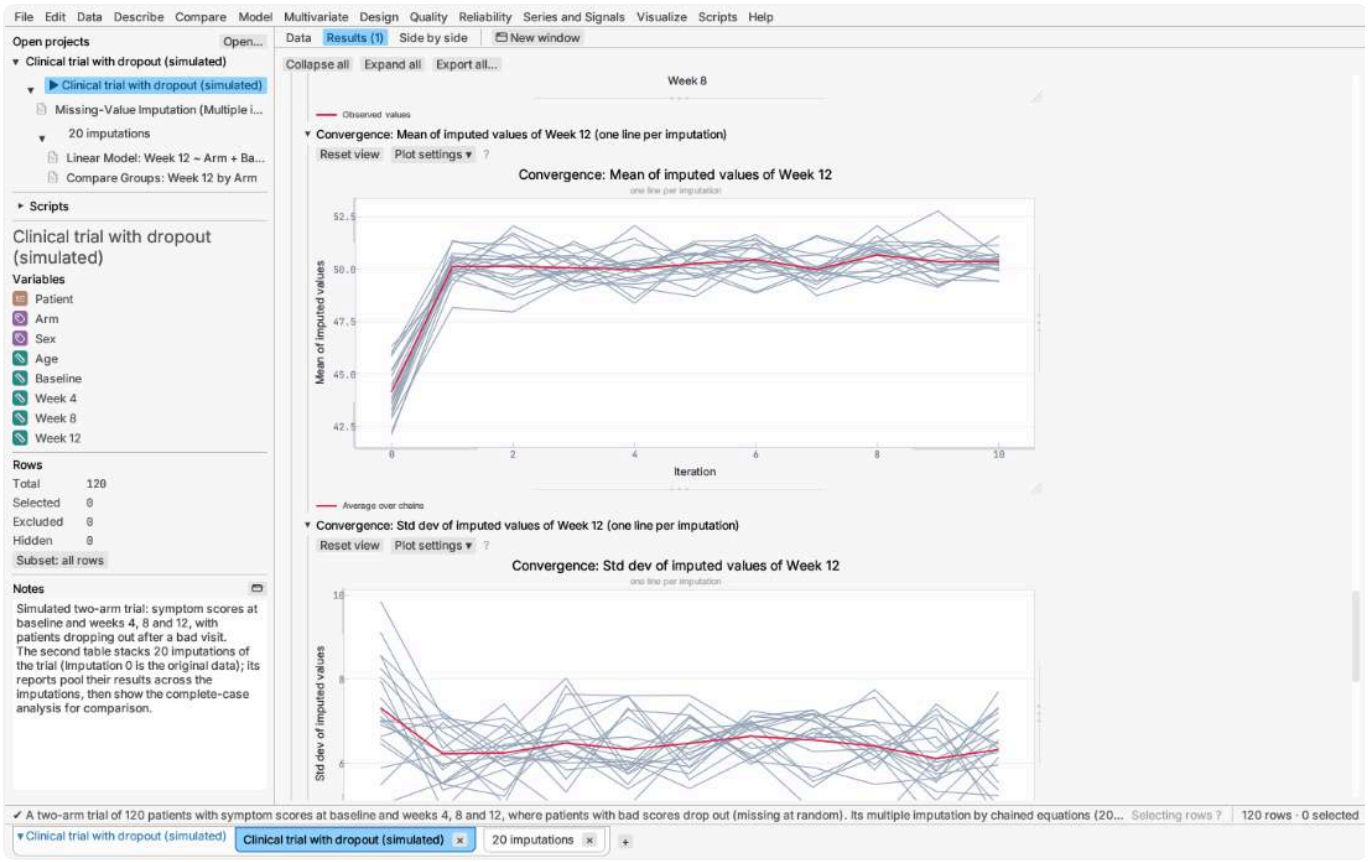
The report

Besides the missing-data patterns and pattern map of every imputation report:

- **Imputation Models:** each column's kind, number of missing values, method, predictors and place in the visit order.
- **Means Pooled Across m Imputations:** each continuous column's observed mean and its mean pooled over the completed copies by Rubin's rules, with standard error,

95% interval and fraction of missing information.

- **Convergence** plots (up to eight imputed columns): the mean and the standard deviation of the imputed values of a continuous column (for a two-category column, the share imputed as its second category) at each iteration, one line per imputation's chain, with their average. Chains that cross each other and show no trend have converged.
- **Observed and Imputed Values:** a strip plot of the observed values at 0 and each imputation's imputed values at its number. Imputed values should look plausible next to the observed ones; under missing at random they need not have the same distribution.
- **Densities:** the kernel density of the observed values (bold) and of each imputation's imputed values (thin).
- **Categories:** for a categorical column, the share of each category among observed and among imputed values.



Make stacked table in the report's ⋯ options menu makes the stacked table from the report's settings and seed, at any time.

The stacked table

The stacked table has m + 1 copies of the source table's rows: the column **Imputation** (0 for the original rows, 1 to m for the completed copies), the column **Row** (the source row number), then every column of the source with the imputed columns filled in (left missing in imputation 0). Rows keep their source row's selection, exclusion and colour. It is kept in the source's project and listed under it; like other derived tables it shows **Update** when the source changes, which makes the imputations again with the same settings and seed. Saving and reopening the project reproduces it exactly.

Pooled reports

A new report on the stacked table is pooled automatically when it can be: the ⋯ options menu has **Pool across imputations**, ticked. Untick it to analyse the stacked rows as they are (for example with a row filter on one imputation, to see that copy's full report). Pooling applies to:

Report	What is pooled	Complete-data df ν_com
Linear model	coefficients; D_1 and D_3 tests of each effect and of the whole model; R^2 and adjusted R^2	$n - p$
Generalized linear model, logistic regression	coefficients; D_1 test of each effect	$n - p$
Mixed model	fixed-effect coefficients; D_1 test of each fixed effect	$n - p$
One-sample test	the mean	$n - 1$
Two independent groups	the difference in means, with the pooled-variance standard error	$n_1 + n_2 - 2$
Three or more groups	each group's mean	smallest n_g − 1
Paired comparison of two columns	the mean difference	$n - 1$
Correlations	each Pearson correlation, on Fisher's z scale	(normal)

A pooled report starts with its pooled sections, then shows the report's usual sections computed from **the original rows only** (imputation 0), which is the complete-case analysis, for comparison:

- **Pooled Across Imputations:** how many copies and rows were combined.
- **Pooled Estimates:** each estimate $\bar{Q}$ (the mean of the m estimates), its standard error $\sqrt{T}$, t ratio, degrees of freedom, p-value, 95% interval and fraction of missing information.
- **Variance Within and Between Imputations:** $\bar{U}$ (the mean of the m squared standard errors), B (the variance of the m estimates), $T = \bar{U} + (1 + 1/m)B$, the relative increase in variance $r = (1 + 1/m)B/\bar{U}$, $\lambda = (1 + 1/m)B/T$ and the fraction of missing information.
- **Pooled Tests by Effect** (models): the D_1 multivariate Wald test of each effect (Li, Raghunathan & Rubin 1991) and, for linear models, the D_3 likelihood-ratio test (Meng & Rubin 1992), with the whole model when there is more than one effect.
- **Pooled Fit** (linear models): R^2 and adjusted R^2 pooled as $\sqrt{R^2}$ on Fisher's z scale (Harel 2009).
- **Estimates by Imputation:** each copy's estimates (the first 12), to see how much they vary.

The degrees of freedom are those of Barnard & Rubin (1999): $\nu = (1/\nu_{old} + 1/\nu_{obs})^{-1}$ with $\nu_{old} = (m - 1)/\lambda^2$ and $\nu_{obs} = (\nu_{com} + 1)/(\nu_{com} + 3) \cdot \nu_{com}(1 - \lambda)$, and the fraction of missing information is $(r + 2/(\nu + 3))/(1 + r)$. These follow R's `mice` package, whose pooled results the app reproduces (the validation record, `docs/validation.md` in the repository, lists the checks).

Walkthrough: the air quality sample

The **Air quality** sample (File ▸ Open sample data ▸ Multivariate) has daily readings with an ozone analyser that fails on humid days, so ozone is missing at random given humidity.

1. Choose **Data ▸ Impute missing values…**, tick all five measured columns and choose **Multiple imputation: chained equations**. Leave 20 imputations and **Make the stacked table** ticked, and click **Run**.
2. In the report, check the **Convergence** plots for ozone and PM2.5: the 20 lines should cross each other with no trend. Compare the **Densities** of ozone: the imputed values sit lower than the observed ones, as they should on humid, cooler days.

3. The stacked table *20 imputations* opens under the Air quality table. Choose **Model ▸ Linear model…** with ozone as the response and temperature and humidity as predictors. The report is pooled: compare its pooled estimates and their fraction of missing information with the complete-case model below them.

In scripts

`t.multiple_imputation(#{ columns: ["Ozone (µg/m³)", …], imputations: 20 })` makes the stacked table, and analyses take `pool_imputations: "Imputation"` to pool; see the scripting reference.

References

- Barnard, J. & Rubin, D. B. (1999). Small-sample degrees of freedom with multiple imputation. *Biometrika* 86, 948–955.
- Harel, O. (2009). The estimation of R^2 and adjusted R^2 in incomplete data sets using multiple imputation. *Journal of Applied Statistics* 36, 1109–1118.
- Li, K. H., Raghunathan, T. E. & Rubin, D. B. (1991). Large-sample significance levels from multiply imputed data using moment-based statistics and an F reference distribution. *JASA* 86, 1065–1073.
- Meng, X.-L. & Rubin, D. B. (1992). Performing likelihood ratio tests with multiply-imputed data sets. *Biometrika* 79, 103–111.
- Reiter, J. P. (2007). Small-sample degrees of freedom for multi-component significance tests with multiple imputation for missing data. *Biometrika* 94, 502–508.
- Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley.
- Schafer, J. L. (1997). *Analysis of Incomplete Multivariate Data*. Chapman & Hall.
- van Buuren, S. & Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software* 45(3).
- van Buuren, S. (2018). *Flexible Imputation of Missing Data*, 2nd ed. Chapman & Hall/CRC.

Row states: select, exclude, hide

Every row has a state shared by the whole document: the grid, every plot and every report. This is linked brushing: select rows anywhere and they are highlighted everywhere.

State	What it does
Selected	Highlighted in the grid and ringed in every plot. Selection does not change any result.
Excluded	Left out of every analysis; reports re-run without it. Plots draw excluded rows hollow and gray, with a key entry. The grid marks them ×.
Hidden	Not drawn in plots, but still analysed. The grid marks them ◌.
Colour and marker	A display colour and marker shape for the row's points.

The sidebar's *Rows* section counts total, selected, excluded and hidden rows, and so does the right end of the status bar.

Selecting

- In a plot, drag a box or click a point. For a freehand outline, turn on **Lasso selection** in a report scatter plot's right-click menu (the graph canvas has its own lasso button).
- In the grid, click a row number to select its row, Shift-click another row number to select every row between the two, or drag down the row numbers (see Editing the data grid). Clicking a data cell only moves the cell cursor. Right-click a row number to select only that row.
- In plots, Shift adds to the selection, ⌘/Ctrl toggles, ⌥/Alt removes. On the row numbers, ⌘/Ctrl and ⌥/Alt do the same, and Shift selects a run.
- In a decision tree diagram, click a node to select its training rows.
- Data ▸ Row states: Select all rows, Clear selection, Invert selection, Select excluded rows.
- Edit ▸ Find…: **Select matching rows** selects every row containing some text (see Editing the data grid).
- In a report, the faint "N of M rows excluded" note under the title selects the excluded rows when clicked.

Excluding and hiding

From Data ▸ Row states (or a row's right-click menu): Exclude selected rows from analyses, Include selected rows in analyses, Include all rows in analyses; Hide selected rows in plots, Show selected rows in plots, Show all rows in plots.

Colours and markers from a column

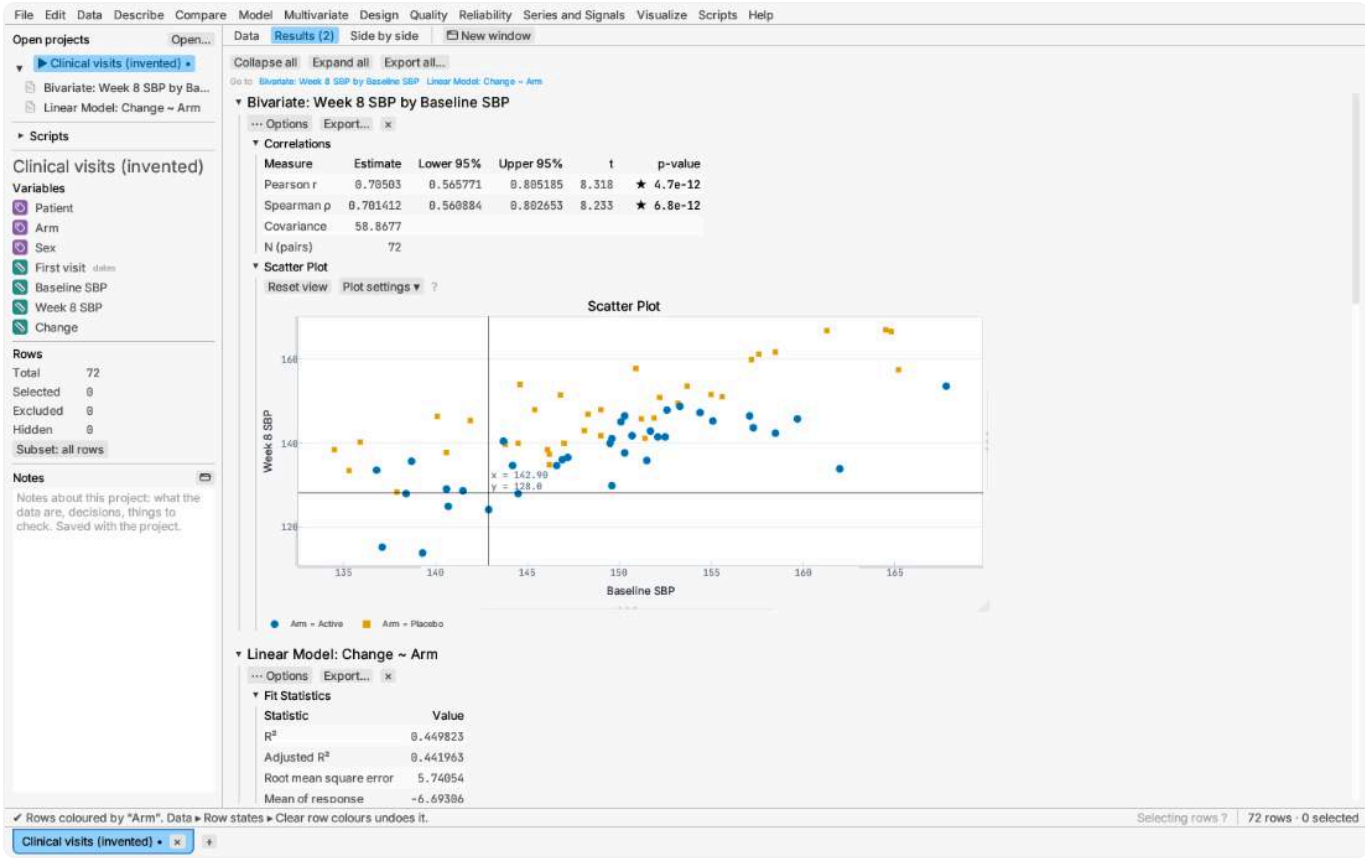
Data ▸ Row states ▸ Colour and marker by column… (also *Colour and marker rows by this column…* in a column heading's right-click menu) gives every row a colour from its value in one column:

- A **categorical** column (nominal or ordinal, or any text column) gives each level its own colour *and* marker shape, so the groups stay apart in print and for colour-blind readers. Levels take the seven palette colours and eight shapes in turn, giving 56 distinct pairs. Levels are ordered naturally by their names (value labels where the column has them).
- A **continuous** column gives a colour ramp from its smallest to its largest value, with the default circle marker. Rows with no value keep no colour.

The dialog previews each level's swatch with its row count (or the ramp with its two ends) before you press **Apply**.

Every report scatter plot and every point layer on the graph canvas that has no colour, shape or overlay grouping of its own then draws each row in its colour and marker. The plot's key names the column and each level the plotted rows show ("Arm = Placebo"), or the ramp's two ends. A plot with its own grouping keeps it.

Data ▸ Row states ▸ Clear row colours and markers removes every row's colour and marker. Colours given by a script (`t.color_rows`) are cleared too.



Row states, colours and markers are saved in projects, can be set from scripts, and are recorded when you record actions.

Row subsets

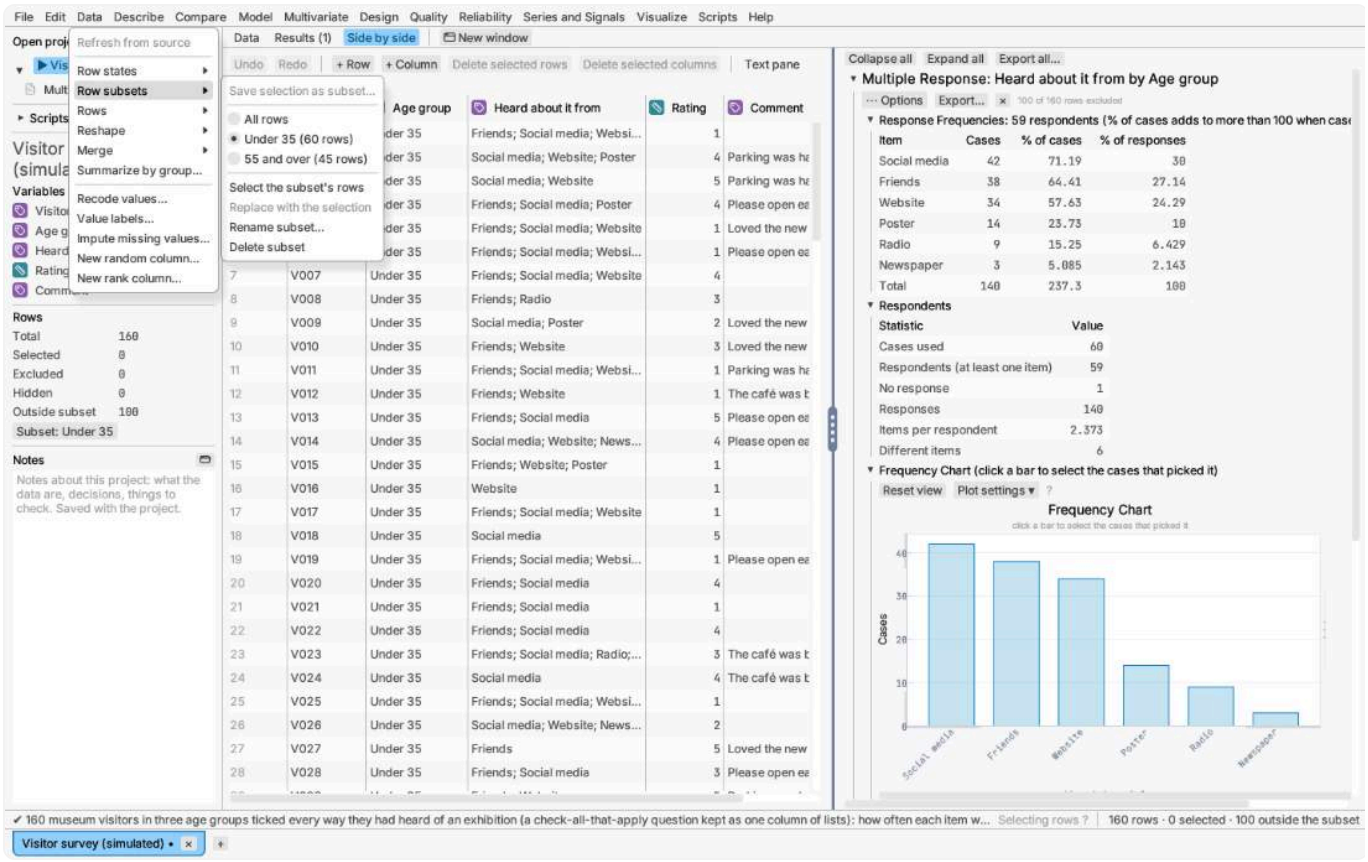
A row subset is a named set of rows that you can switch to and back from, such as one site of a study, the runs before a process change, or the rows of one customer type. One subset at a time can be *active*: rows outside it are then left out of every report, plot, dashboard tile and script exactly as excluded rows are, and every report re-runs at once. Switching back to **All rows** brings them all back.

Open **Data ▸ Row subsets**, or click the *Subset* line under the row counts in the sidebar (it names the active subset, or says *all rows*):

- **Save selection as subset…** names the selected rows. A name already in use replaces that subset's rows. A table keeps up to 64 subsets; saving does not switch to the new one.
- **All rows** and one entry per saved subset (with its row count) choose the active subset.

- With a subset active: **Select the subset's rows**, **Replace with the selection** (same name, the selected rows instead), **Rename subset…** and **Delete subset** (analyses use all rows again).

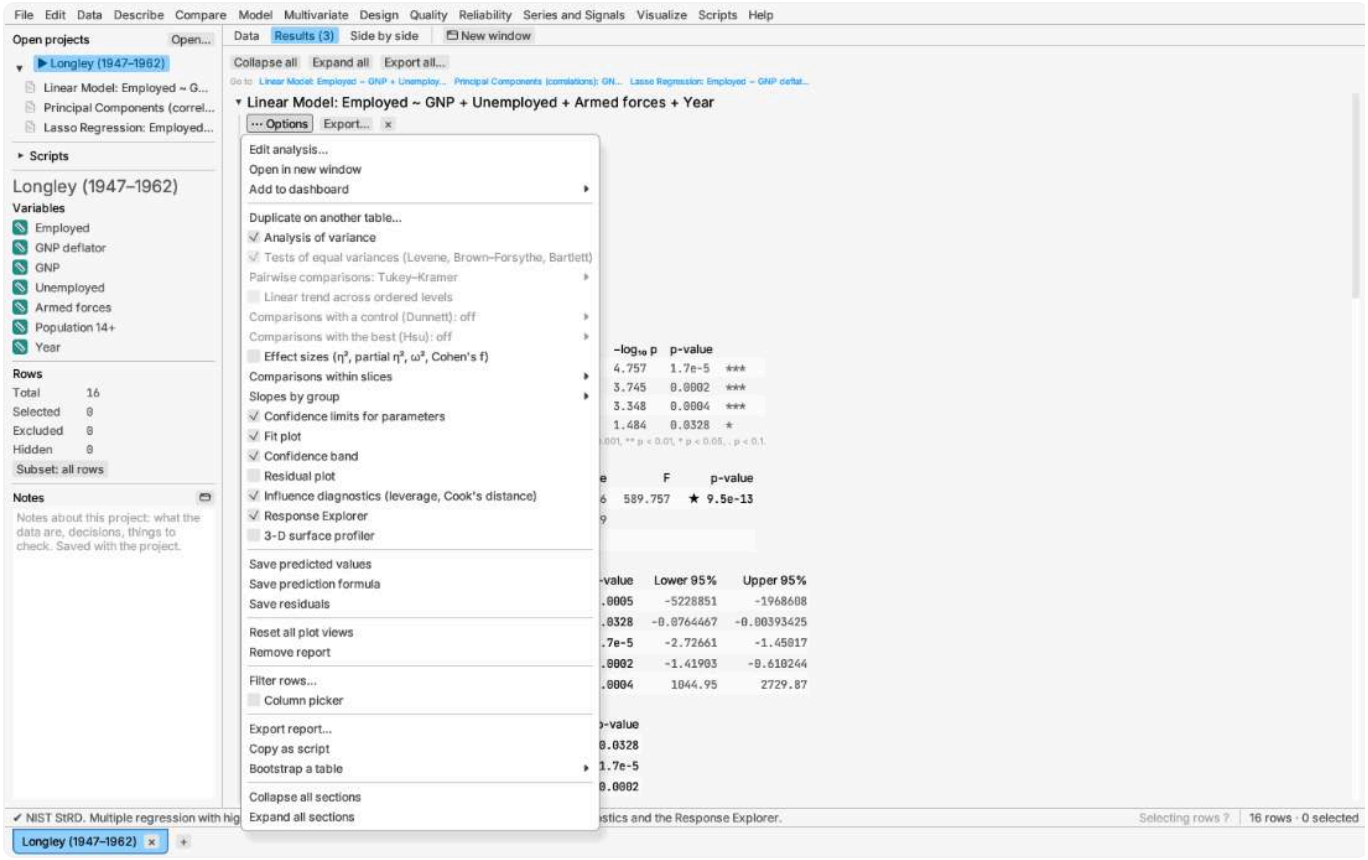
Rows outside the active subset show ⊘ in the grid and their text is faded; the sidebar and status bar count them apart (*Outside subset*). A subset is kept apart from your own exclusions and hidden rows: excluding a row while a subset is active, then switching to All rows, leaves exactly the rows you excluded.



Subsets and the active one are saved in the project file (in each table's `"rows"`: `"subsets"`, a list of names with their rows, and `"active_subset"`), recorded when you record actions, and can be set from scripts with `t.save_subset`, `t.use_subset` and `t.use_all_rows` (see Scripting).

Reading a report

Each analysis produces a **report**: a tree of collapsible sections, such as fit statistics, parameter estimates, test tables and plots. Reports appear on the document's **Results** tab (newest first) and in **Side by side** view.



Sections

Click a section's heading to collapse or expand it. Numeric table cells use a monospaced font, so digits line up.

Collapse all and **Expand all**, above the reports, close every report down to its title or open every report and all its sections. **Export all…**, beside them, saves all of the table's reports (or the ones you pick) as a document; see Saving plots and results. The × beside a report's ··· Options button removes that report; for 30 seconds the status bar offers **Undo remove**, which puts it back where it was. When rows are excluded, a faint "N of M rows excluded" note sits beside the buttons; click it to select the excluded rows.

With two to four reports, a **Go to** line lists their titles (newest first); click one to open that report and scroll to it. With more, a **Go to ▾** list beside the buttons holds the titles instead, so the line never wraps.

P-values below 0.05 in report tables are marked with a ★ in strong text, so they stand out without relying on colour. Turn this off in File ▸ Settings… (Mark p-values below 0.05 with ★). Settings can also separate thousands with commas (184,150,146) and line up the decimal points in each column. Copied tables carry the plain numbers. While the getting-started tips are on, the line beside the buttons is a reminder of how to work the plots; the **?** beside each plot shows the same reminder.

The options menu

Each report has a **⋯ Options** menu, also opened by right-clicking the report's heading or any section heading. It:

- **Edit analysis…** (at the top of the menu, and when right-clicking the report's heading) reopens the report's dialog with its settings filled in; **Update** changes the report in place (see Using analysis dialogs);
- **Open in new window** (just below it, also on the report heading's right-click menu) shows the report in a window of its own; in that window the item reads **Back to main window** (see Views in windows of their own);
- adds or removes optional sections in place (diagnostics, extra tests, plots). The chapters on each analysis list its options;
- saves columns the report offers back into the table, such as predicted values, residuals, component scores or cluster numbers;
- **Filter rows…** narrows this report alone to chosen rows, and **Column picker** steps it through other columns (see Filtering a report and switching its column);
- **Copy as script** copies the script call that recreates this report (see Scripting and recording);
- **Collapse all sections** and **Expand all sections** close or open every section of this report at once;
- **Reset all plot views** re-fits every plot in the report;

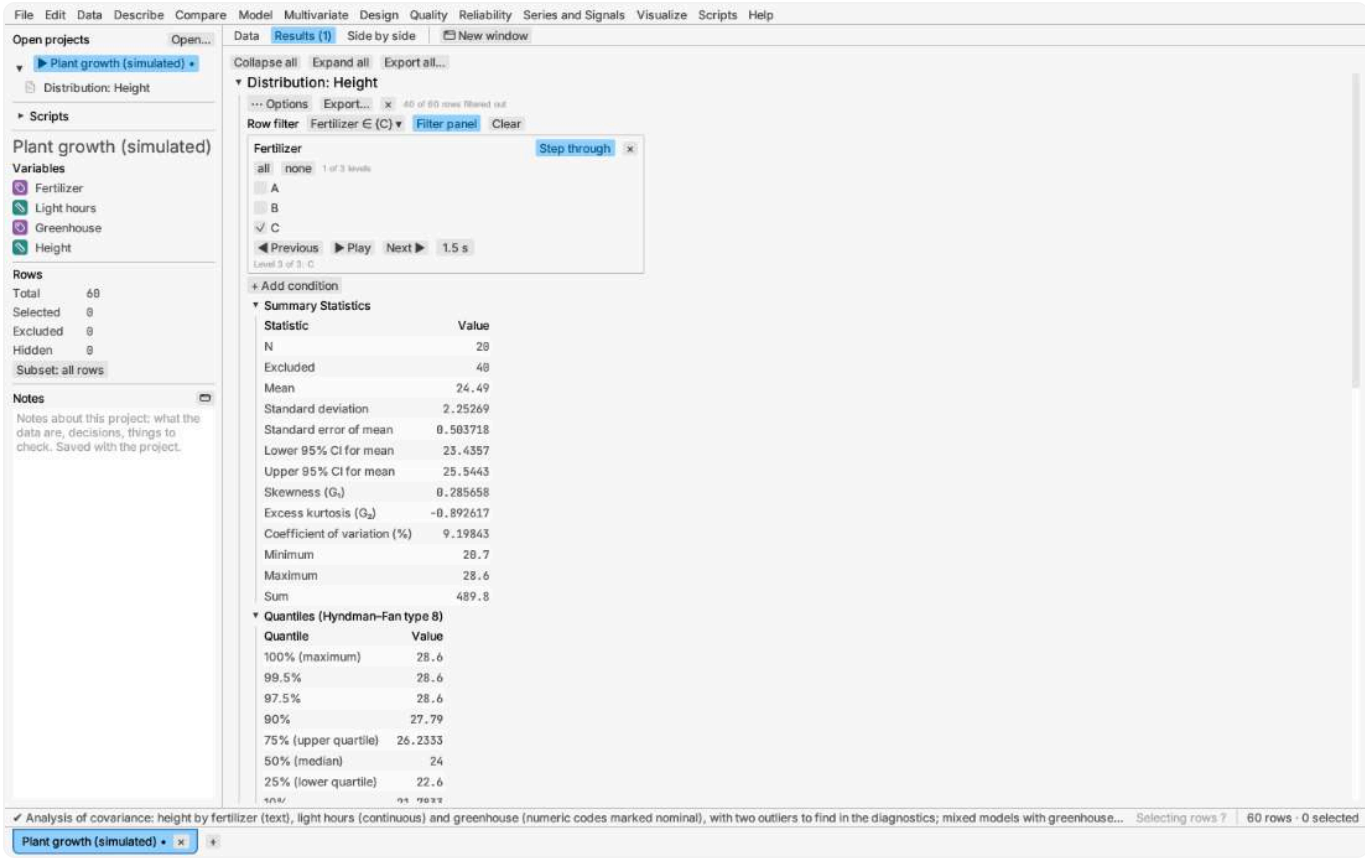
- **Remove report** closes it.

Right-clicking the heading of a plot section also offers to save the plot (see Saving plots and results).

Filtering a report and switching its column

⋯ Options ▸ Filter rows… gives a report a row filter of its own. A **Row filter** line appears under the report's title: **+ Add condition** lists the table's columns. A categorical column becomes a condition with a check box per level; a continuous one becomes a **from** / **to** interval (ends included). Each condition shows as a button such as `Region ∈ {East, West}` or `20 ≤ Age ≤ 40`; click it to change the levels or the interval, or to remove the condition. A row must pass every condition. **Clear** (or ⋯ Options ▸ **Remove row filter**) takes the filter away.

Filter panel on the same line opens the conditions as a form under it: each condition in a box with its column's name, a list of its levels with **all** and **none** (a categorical column) or a **from** and a **to** slider (a continuous one), and **×** to remove it; **+ Add condition** sits at the bottom. **Step through** on a condition shows one slice at a time: a categorical column one level at a time, a continuous column a window of fixed width (the interval you set, or a tenth of the column's range when the interval spans all of it) moved along by its own width. ◀ **Previous** and **Next** ▶ step by hand and wrap round at the ends; ▶ **Play** steps on by itself at the number of seconds shown (0.2 to 10) until ∎ **Pause**. A note such as "Level 2 of 3: B" or "Window 4 of 10: 1950 ≤ Year ≤ 1951.5" says where it stands. Each step recomputes the report, so this shows how an analysis changes across the levels of another column, in the manner of Cleveland's conditioning plots (*Visualizing Data*, 1993). Adjacent windows share their end points, so a value exactly on a boundary appears in both. The steps are ordinary filter edits: the filter shown when you save is the one saved.

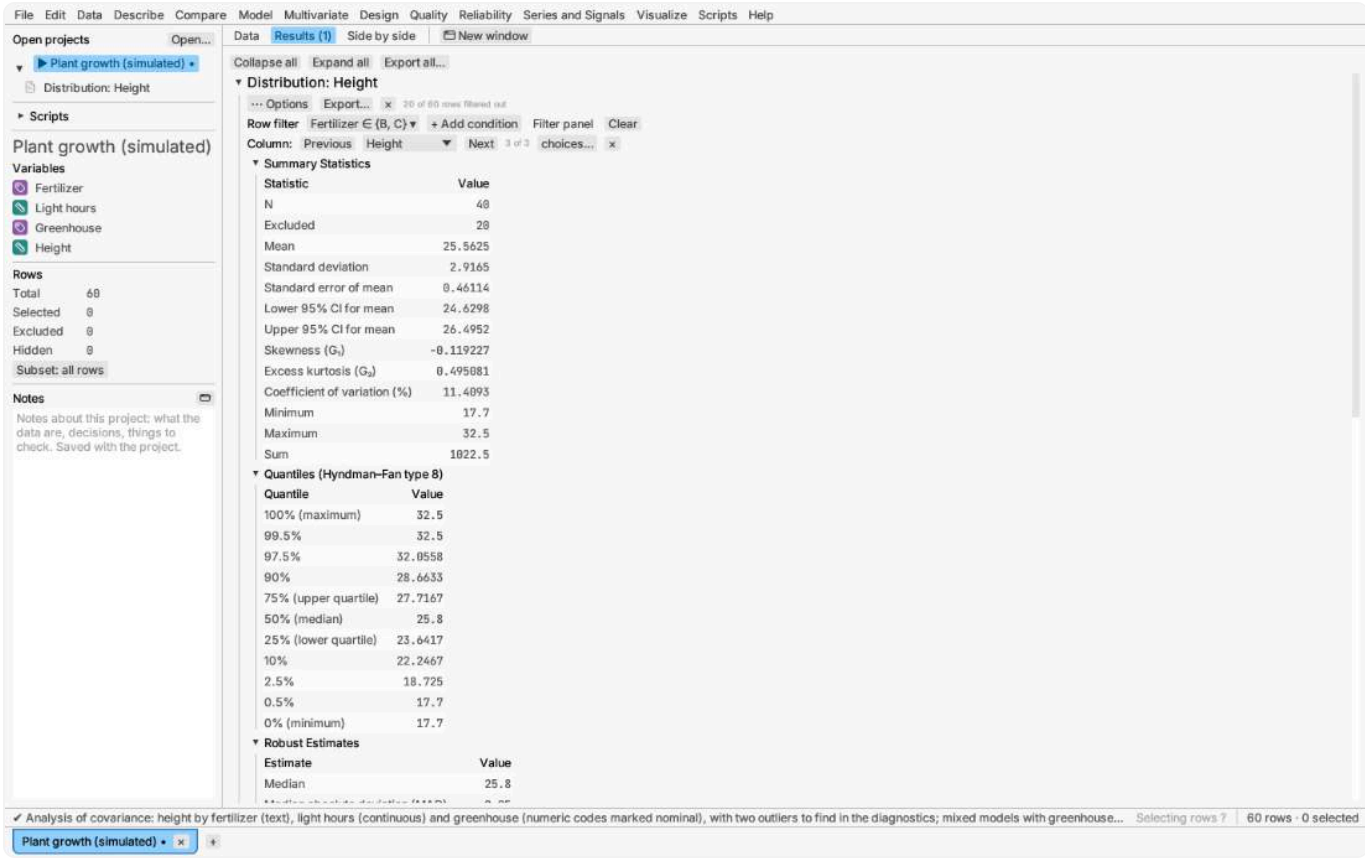


The filter changes this report only. The rows it leaves out are not excluded or hidden in the table, so the data grid and every other report still use them. The report recomputes from the rows that pass (an interval dragged with the mouse applies when you let go), its plots leave the other rows out, and a faint "N of M rows filtered out" note beside the title says how many it left out. Selecting points in a filtered report still selects those rows everywhere. The filter is saved with the report in a project and in **Copy as script**, and a column used by a filter cannot be deleted while the report is open (a renamed column is followed).

Reports built around one response or measured column (linear, generalized linear, penalized and stepwise models; distributions; control charts and capability studies; bivariate fits, whose Y column switches; multi-vari charts and analysis of means) also offer **⋯ Options ▸ Column picker**. A line under the title shows the column now in that role, with **Previous**, **Next** and a list to pick another, and "2 of 6" for its place in the list. Choosing a column recomputes the report on it, with every other setting kept, so you can step one analysis through many columns. The list starts with every column of the same kind (numeric or text) that the report does not already use in another role; **choices…** ticks which ones it holds, and **×** removes the picker. For a capability study, the line below holds the **specification limits for** the column shown: each column keeps its own limits as you step

through, and a column seen for the first time starts with the limits of the one before. A generalized linear model's event level goes back to its default when the response changes.

The same reports take a column dropped on a plot axis. Drag a column from the variables list over a plot: an axis whose title is the report's own column (the response of a model's fit plots, the column of a distribution's histogram or a control chart's values) is outlined, and a note beside the pointer says "Analyse Weight instead of Height". Letting go there switches the report to that column, just as choosing it in the column picker does (the picker need not be on). A column of another kind, or one the report already uses in another role, is refused with a note saying so; axes that show other quantities (counts, residuals, quantiles) take no drops.



Reports stay current

A report is bound to its table's data and row states. Editing the data or excluding rows re-runs it. Projects store each report's request and display options, and reopening the project re-runs it exactly.

Reports are computed in the background, so typing in the grid, excluding rows or scrolling never waits for a slow analysis. A report that takes more than a moment shows a spinner and **Updating…** in its title bar, with its previous result dimmed beneath until the new one arrives. Analyses with many repeated steps (random forests, boosted trees, Bayesian sampling, bootstrap intervals, the model search) count them: **Updating… 120 of 500**.

Cancel in the title bar stops the computation. The report keeps its previous result and says **Computation cancelled**; **Compute again** starts it once more. A further edit to the data also starts it again. When rows were added, removed or reordered, the previous result no longer lines up with the table, so the report shows a short note instead of it until the new result arrives.

If an analysis stops with an unexpected error, the report keeps its previous result and shows the error above it; the rest of the app carries on. New reports, and the reports of a project being reopened, appear at once and fill in as they finish. Exporting reports first finishes any that are still updating, so exported files always hold current results.

Model reports

Every model report includes an **Effect Pareto**, a bar chart of $-\log_{10} p$ for each effect, so the strongest effects stand out. Models keep the hierarchy principle: an interaction or a power implies its lower-order effects. Model dialogs enforce it, stepwise selection respects it, and a report notes when a model breaks it.

Linear, generalized linear, penalized, nonlinear and decision tree reports end with a **Prediction Formula** section showing the fitted equation, with a **Copy formula** button. **Save prediction formula** writes it as a formula column, so predictions follow edits to the data and are computed for new rows.

Working with plots

All plots in reports and on the graph canvas behave the same way.

Moving and rescaling

Plots never move when you scroll or pinch, so scrolling a long report is safe. Instead, each axis is a ruler you drag:

- Drag near **either end** of an axis to move that limit only; the value you grabbed follows the pointer.
- Drag the **middle** of an axis to shift the whole axis.
- **Reset view** (or double-click the plot) re-fits the plot to its data.

On a plot at equal scale (see Resizing below), dragging the end of one axis still moves only that limit; the other axis widens or narrows about its centre to keep one unit the same length on both. On the graph canvas, dragging one axis fixes that axis's range only; the other keeps fitting the data.

Dragging inside the plot area selects rows (see below). A short reminder of these gestures sits above the reports, and each plot's **?** shows it too.

After selecting points, right-click the plot and choose **Zoom to selected points** to frame them; Reset view goes back.

Resting the pointer on a point in a report plot shows a tooltip with its row number and its two values (the category's name on a categorical axis), and notes when the row is excluded.

Selecting points

Drag a box in the plot to select the rows inside it, or click a point. On the graph canvas, and in report scatter plots through the plot's right-click menu (**Lasso selection**), you can also draw a lasso. Shift adds, ⌘/Ctrl toggles and ⌥/Alt removes. Selected points are ringed, so the selection stays visible on any group colour, and the same rows are highlighted in every other plot and the grid.

Plots with very many points

A plot draws one marker for each row, which stays quick up to some tens of thousands of points. Above **20,000** points (in a report's scatter or quantile plot, among a box plot's values beyond the whiskers and its selection rings, or in one layer of the graph canvas) the points are drawn instead as one picture at the screen's resolution, and a note under the plot says so. Each pixel takes the colour of the points in it, with opacity growing with how many there are (on a logarithmic scale, so crowded areas read darker without hiding single points); excluded rows are drawn first in their own colour and selected rows last, in the selection colour, slightly larger. Group colours are kept; marker shapes and the dark theme's halo are not drawn.

Only the drawing changes. Box, lasso and click selection, hovering and zooming to the selection work on the rows as before; dragging an axis draws the picture again for the new view. Saved and copied figures (SVG, PDF, EPS and PNG) still contain every point.

Groups

Grouped data use a colour-blind-safe palette, and each group also gets its own marker shape, so groups can be told apart without colour.

Plot settings

Each plot's **Plot settings ▾** menu sets titles, axis ranges, major and minor ticks, the tick origin, grid lines, number format, axis position, equal scaling of the two axes, and the key. On scatter plots and on the graph canvas it also sets axis breaks on each continuous axis (see Axis breaks below), and on the graph canvas the right axis when the graph has one (see Graph canvas).

These settings are kept with the plot: saving the project, adding the report to a dashboard or copying its script call keeps them, and reopening draws the plot as you left it. A plot whose settings are all back at their defaults stores nothing. Ranges typed into the **min** and **max** fields are kept (the row then shows **fixed** and an **Automatic** button that lets the axis fit the data again); ranges you only drag to are not kept, and reopened plots fit their data again. **Reset view**, a double-click on the plot and **Fit view to data** clear typed ranges too.

Ticks

Each axis section has a **Ticks** group:

- **Tick direction** — **Inside** (the default) points the marks into the plot, **Outside** away from it towards the labels, **Across** half each way. Labels move out to make room for outside marks.
- **Major marks** and **Minor marks** — length and thickness in points (5 and 1 for majors, 3 and 1 for minors). Minor marks are drawn whenever the axis has minor ticks (set under **Minor ticks**).
- **Tick labels** — your own labels, each a value and a text read as math markup (`\pi` draws π, `10^3` a superscript). A label at a tick's value replaces its number; anywhere else it adds a labelled tick of its own. **Add a tick at…** adds a row to fill in; ✕ removes one. An empty label leaves the tick unlabelled.

On the plot itself, double-click a tick label (or right-click the axis ▸ **Edit tick label…**) to write that tick's label in place: **Apply** (or Enter) keeps it, **Use the number** goes back to the number, **Cancel** (or Escape) leaves it as it was. Where no labelled tick is near, the editor adds a tick at the value under the pointer.

Spine

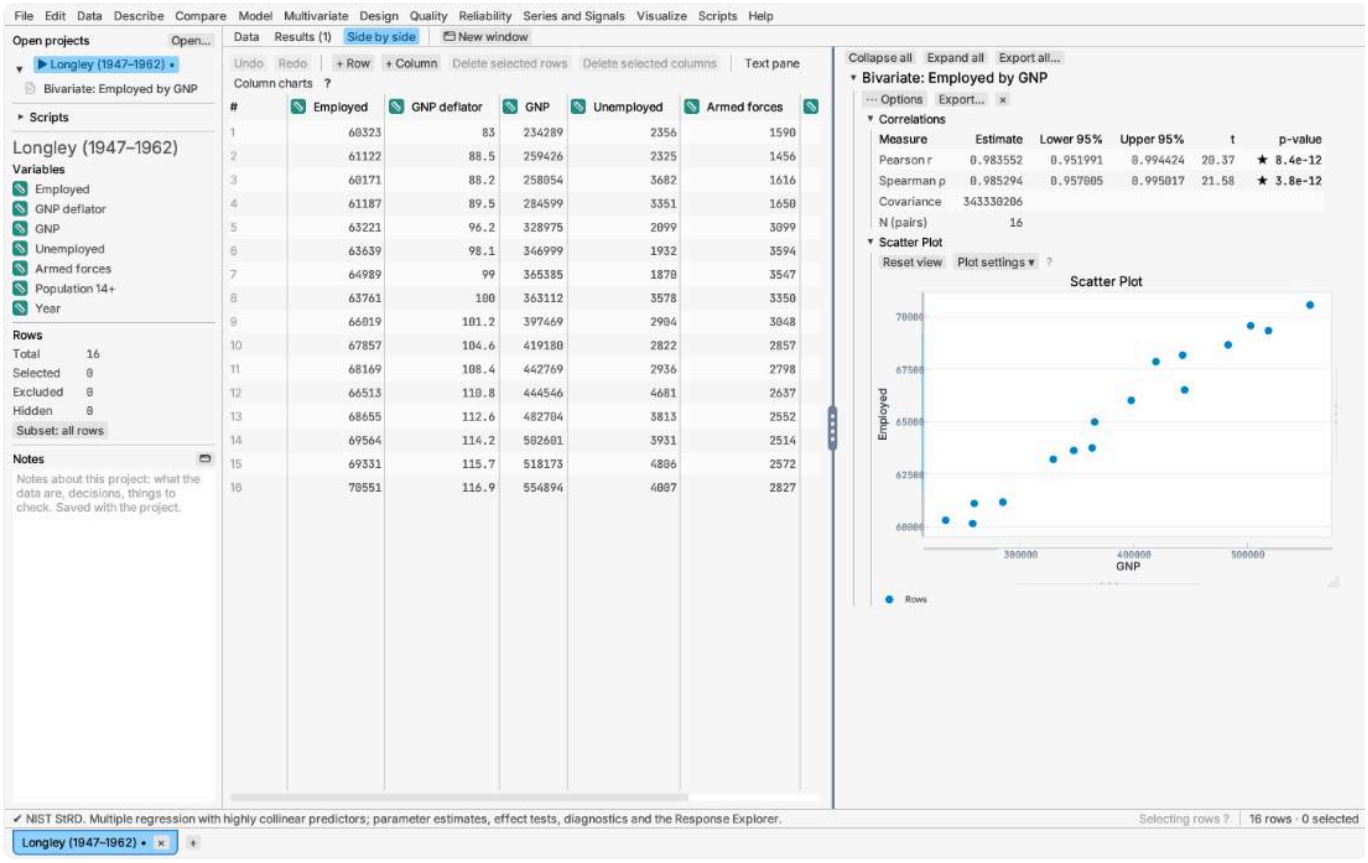
The **Spine** group of each axis has:

- **Offset from data** — moves the axis line, its marks and labels away from the data area by up to 40 points, a style common in scientific figures. The draggable axis ruler moves with the line.
- **Extend to nearest ticks** — widens an automatic range to the major ticks just outside it, so the axis starts and ends on a labelled value (off by default, when ranges keep their usual 6% padding). Typed or dragged ranges are left as they are.

Plot area

Besides **Equal scale**, the **Plot area** section sets a **Scale ratio** (one x unit drawn as long as *r* y units; 1 is equal scale) and a **Print scale**: saved figures draw one horizontal unit *N* millimetres long, so a printed figure can be measured with a ruler (report plots; the page grows or shrinks to fit the range shown).

Saved SVG, PDF, EPS and PNG figures draw the axes with these settings: tick direction, length and thickness, offsets, your tick labels (in markup), and on report plots also the tick intervals, minor ticks, number format, grid lines and the side each axis is on, so a saved figure matches the screen.



Copying a plot's style

Right-click a plot (or its section heading) and choose **Copy plot style** to keep its settings for the session; a graph canvas has it in its ⋯ Options menu and in its plot's right-click menu. Then right-click any other plot ▸ **Paste plot style**. The submenu names the plot the style came from and has a tick for each part that goes across:

Part	What it carries
Axis ranges	The typed minimum and maximum of each axis (off at first: ranges belong to one plot's data)
Ticks, grid and spines	Tick spacing and marks, minor ticks, your tick labels, number format, grid lines, axis sides and offsets, category label direction, equal scale and its ratio, print scale

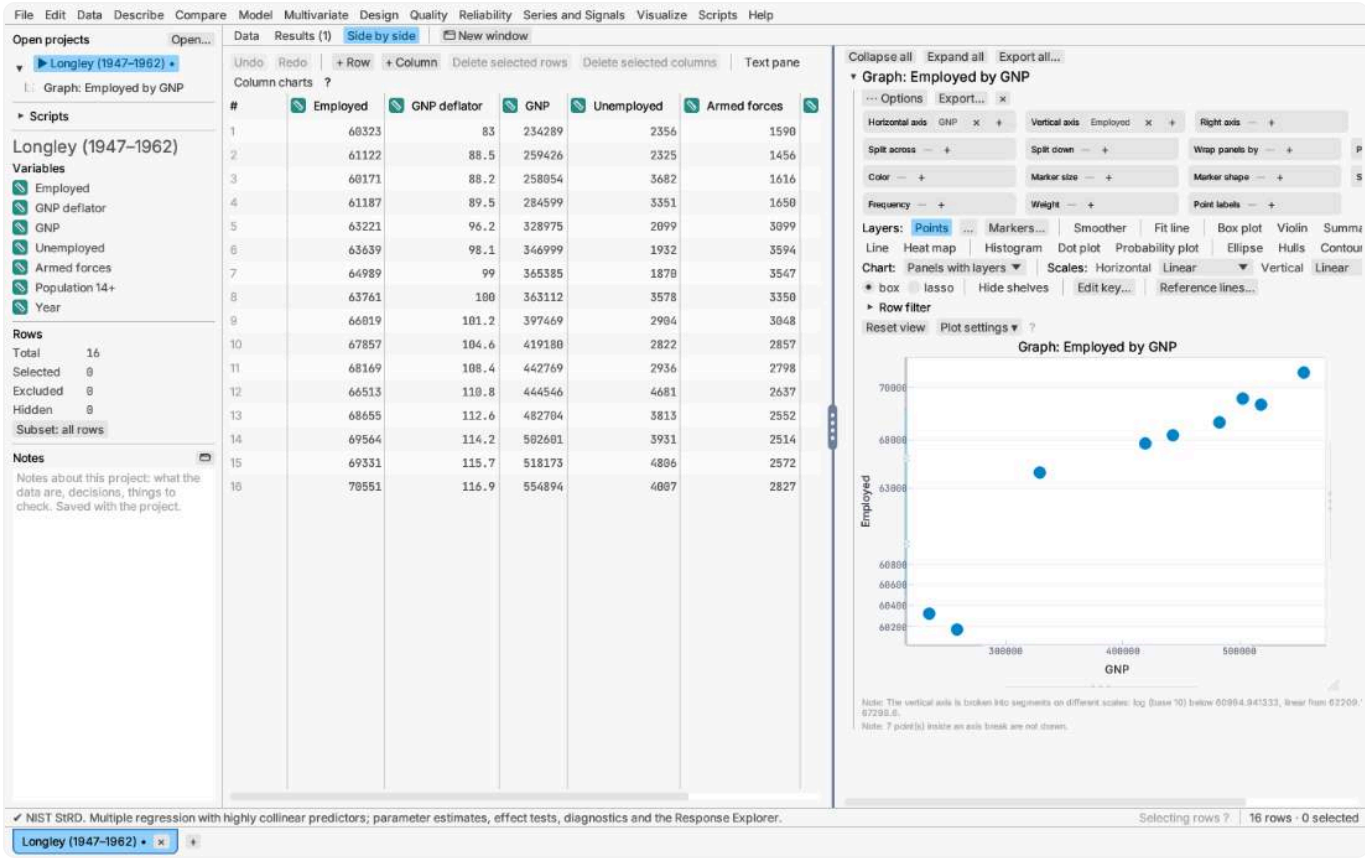
Part	What it carries
Font and size	The plot font, text size, saved figure size and background
Colours and markers	Marker and line colours, shapes and sizes (between graph canvases)
Reference lines	Reference lines and shaded bands
Key	Whether the key shows, where it sits (graph canvases), and renamed, hidden or reordered entries

Titles, axis breaks and annotations never go across: they describe one plot's data. Paste **Onto this plot**, **Onto every plot in this report** (collapsed sections too) or **Onto every plot in this project** (every report and graph canvas of every table in the project). Report plots and graph canvases share axes, fonts, reference lines and the key; colours, markers and the key's placement exist only on canvases. The pasted settings are the plot's own from then on and are saved with it.

On a plot with categories along the horizontal axis (a bar chart, Pareto chart, control chart, or a plot against a categorical column), **Category labels** sets how their names are drawn: **Automatic** (the default) keeps them level when they fit, slants them at 45° when level names would overlap, and stands them upright when even slanted ones would; **Horizontal**, **Slanted (45°)** and **Vertical** fix the direction. The axis grows to make room for turned names. Only when even upright names would overlap is every second or third one shown. Exported figures follow the same setting.

Axis breaks

On a scatter plot in a report (and on the graph canvas), each continuous axis has an **Axis breaks** row: tick **Leave out** to leave a stretch of the axis out, **Add a break** for more, and open **Segments and marks** to put a piece on a scale of its own (a log piece below a linear one, say), give a piece a fixed share of the axis, or choose slashes or a zigzag for the marks. Points inside a break are not drawn and a note under the plot counts them; a note also names the scales when the pieces use different ones. Tick labels come from each piece's own values, hovering a point shows its data values, and box and click selection pick what is drawn. Axes labelled by categories, probability scales or dates, plots divided into panes, contour maps and ternary plots take no breaks. The full description is under Graph canvas ▸ Axis breaks. Breaks are kept with the plot, and saved figures draw them as on screen.



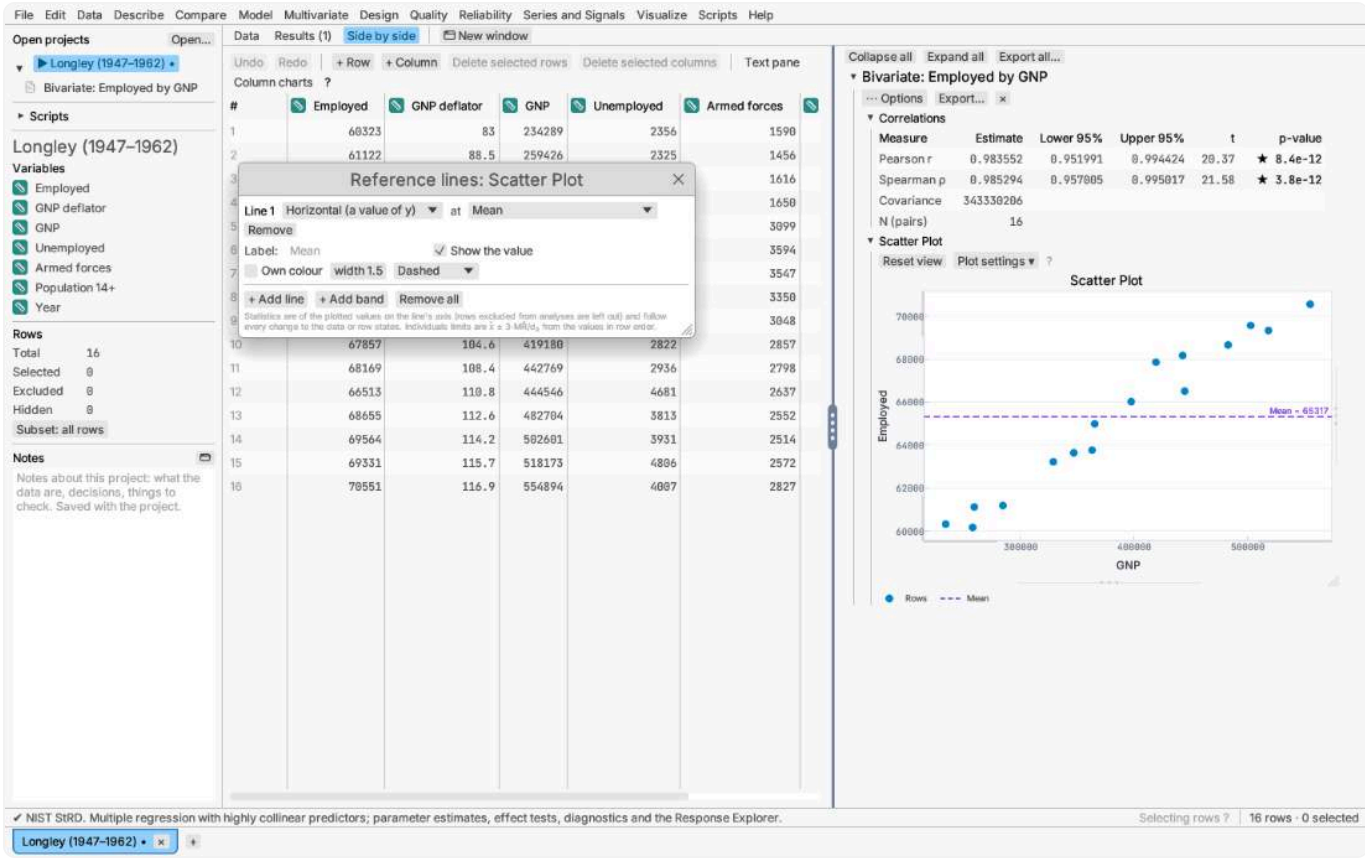
Reference lines and bands

Reference lines…, in the plot's **Plot settings ▾** menu and in its right-click menu, adds horizontal or vertical lines and shaded bands to a report plot (scatter and fit plots, control charts, box plots, survival curves, bar and Pareto charts) or a graph canvas.

- **+ Add line** adds a dashed line. Choose *Horizontal* or *Vertical*, then where it sits: a **Typed value**, or a statistic of the plotted values on that axis: **Mean**, **Median**, **Quantile** (with its percentage), **Mean ± k standard deviations** (with k; a negative k is below the mean), or the **Individuals chart** centre line, upper or lower limit ($\bar{x} \pm 3 \cdot M\bar{R}/d_2$, from the values in row order). Each line has its own label (empty: its value or statistic), colour, width and pattern (solid, dashed, dotted or dash-dot). *Show the value* adds the number after the label.
- **+ Add band** adds a shaded stretch between two values, each typed or a statistic, drawn behind the data with its fill colour and opacity. *Edge lines* draws its two edges; *Edge values* writes their values beside them. A band has an entry in the key, a filled square with its label.

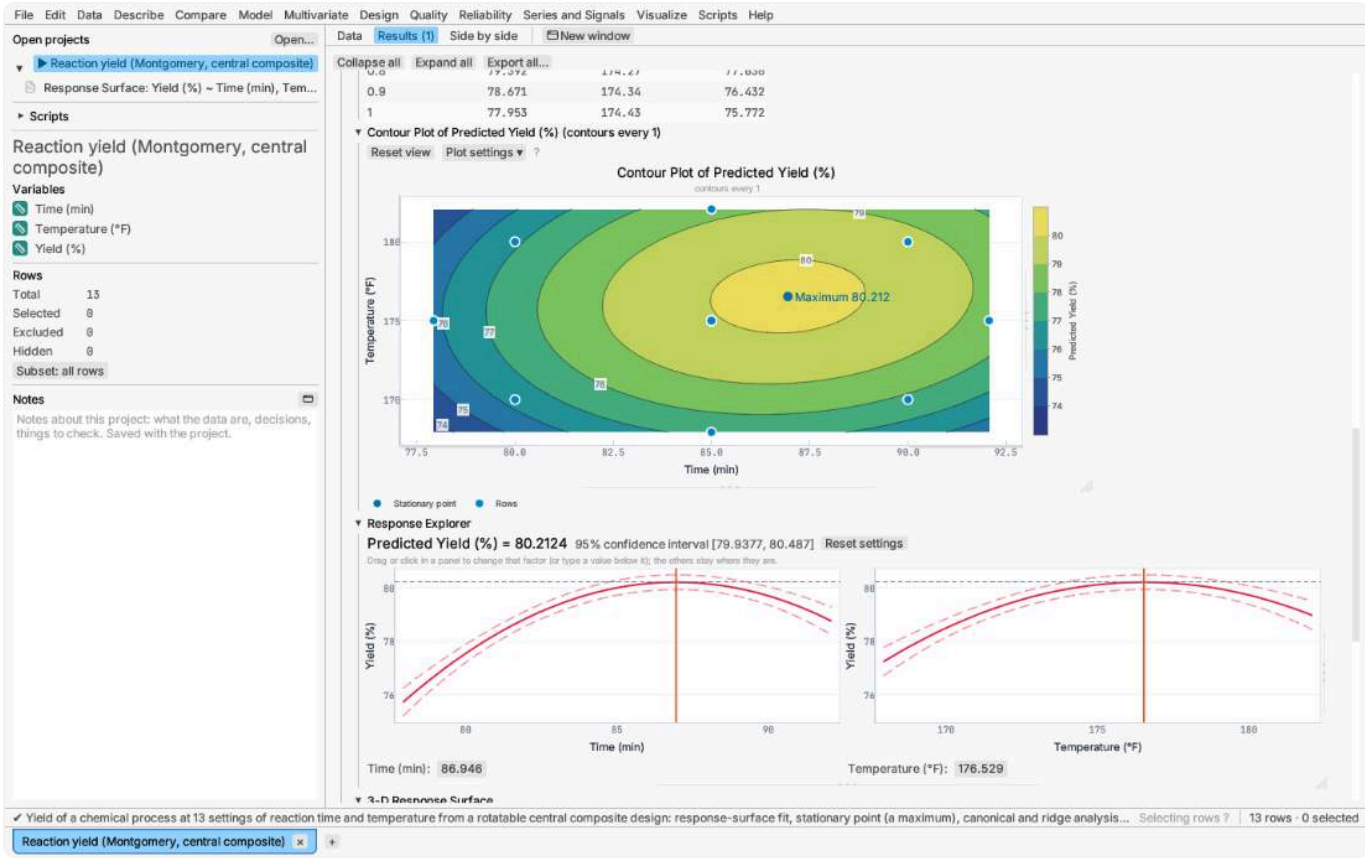
Statistics leave out rows excluded from analyses and follow every change to the data or the row states, so a line at the mean moves when rows are excluded. On the graph canvas a statistic is computed for **Whole graph**, **Each panel** (the default) or **Each subgroup**, the last drawing one line per colour group in the group's colour and naming it in the label. Lines are drawn on top of the data, bands underneath; the default view widens to show a line that falls outside the data. A statistic needs a continuous axis; on a categorical or date axis, type the value. A value that cannot be drawn on the axis (such as a negative value on a log axis) is left out with a note under the plot.

Lines and bands are saved with the plot (as `reference_lines` in its plot style), and appear in saved plot images and on figure pages.



Contour maps

The contour plots of the response surface, mixture (ternary), nonlinear surface and contour-and-surface reports colour the bands between contour levels and show a **colour bar** beside the plot with the level values at the boundaries between its boxes. The level lines and their values stay on top, so the map still reads without colour.



The end of the plot's **Plot settings ▾** menu holds the contour options. They apply to every contour plot of the report (and to the colours of the 3-D surface in a contour-and-surface report), and they are saved with the project, in dashboards and in exports:

- **Fill between levels**, **Level lines** and **Level values** switch the three layers on or off. **Lines only** gives the plain look of level lines and values with no fill; **Default** goes back to the filled map.
- **Colours** chooses the scale:
  - *Sequential (blue to yellow)*: lightness rises steadily, so high and low read in grayscale too. The default.
  - *Heat (yellow to red)*: pale yellow through orange to dark red (after Brewer's yellow–orange–red scheme).
  - *Spectrum (blue to red)*: blue, cyan, green, yellow, orange and red, the usual choice for elevation and response maps. It is not colour-blind safe, and its lightness does not rise steadily, so keep the level lines and values on with it.
  - *Diverging (blue, neutral, red)*: blue below a centre value, a light neutral at it and vermillion above, for a response with a target. **Centre at** sets the value in the middle (by default the middle of the range); the farther end of the range takes the strongest colour.
  - *Grey (for print)*: white to near black.

- **Reverse colours** runs the scale from high to low.
- **Opacity** fades the fill (0.2 to 1), for example to see the grid.
- **Number of levels** sets about how many levels to draw, at round values (steps of 1, 2, 2.5 or 5 × 10^n); unticked, each plot keeps its own number. A title that says *contours every …* follows the new step.
- **Fixed range** spans the levels and colours over a range you type instead of the surface's own, so maps of different responses or data can be compared colour for colour. Parts of the surface outside the range take the end colours.

Each band is coloured at its middle value. In dark mode the colours lean a little towards the dark canvas. Points over a filled map get a rim in the canvas colour, and selected and excluded rows keep their ring and cross.

The bands are true shapes, not pictures: **Save plot as** SVG, PDF or EPS writes each band as one filled outline (with holes where the band surrounds a higher or lower region), and PNG renders them, so a saved map matches the screen. The method: the surface is taken as linear over the triangles of its grid (each grid cell is split into four triangles around its centre, whose value is the mean of the corners), so each band is cut from each triangle exactly and the pieces join without gaps; the level lines are cut from the same triangles and lie on the bands' edges.

Math in labels

A title typed in **Plot settings** (the plot title or an axis title) is read as a small markup, so it can carry subscripts, Greek letters and fractions. The same markup is set in saved SVG, PDF, EPS and PNG figures, and in the vertical title of the y axis.

Type	To get
`x_1`, `x_{ij}`	a subscript: one character, or a group in braces
`x^2`, `e^{-t}`	a superscript
`x_i^2`	a subscript and a superscript at the same place
`\alpha`, `\beta`, `\mu`, `\sigma`, …	Greek letters by name; a capital first letter gives the capital (`\Delta`, `\Sigma`)
`\pm`, `\times`, `\cdot`, `\le`, `\ge`, `\ne`, `\approx`, `\infty`, `\partial`, `\deg`	±, ×, ·, ≤, ≥, ≠, ≈, ∞, ∂, °

Type	To get
`\frac{a}{b}`	a fraction, a over b with a rule between
`\sqrt{x}`	a square root, the radical sign as tall as what is under it
`\sum`, `\prod`	Σ, ∏
`$a/b$`	inside dollar signs a slash is a fraction too; outside them `/` is an ordinary slash
`\_`, `\^`, `\{`, `\}`, `\$`, `\/`, `\\`	the character itself

For example `Yield (kg m^{-2})`, `\sigma_{within}` or `$\Delta y / \Delta x$`. Scripts are set at 70% of the text size, fraction parts at 80%. A title with none of `_ ^ \ $ { }` is drawn exactly as plain text. Something the markup cannot read — an unknown name such as `\foo`, a `_` or `^` with nothing after it, an unmatched `$` — is shown as typed.

Default titles (column names) are shown as written. On the graph canvas, **⋯ Options ▸ Read math markup in column names, levels and labels** reads them as markup too, along with key entries and text labels, so a column named `x_1` gets a subscript.

Resizing

Every plot can be resized by dragging the faint handles beside it:

- the strip under the plot changes its height;
- the strip along its right edge changes its width;
- the corner where the two meet (marked with diagonal lines) changes both.

Double-click either strip to go back to the default size. A plot is never made wider than the visible part of the pane, so the whole of it stays in view. The same handles resize the 3-D scatter and surface plots, the plots of the graph canvas, and the panels of a Response Explorer, where one strip under the panels resizes them all together (see Response Explorer).

Plots of one variable against another, and the 3-D plots, open at most about 760 points wide (wider with larger text), so a wide window does not flatten them into a strip; drag a plot's right edge to widen it. Plots whose shape depends on equal units on both axes open at equal scale: observed against predicted or fitted values, probability plots against the line y = x, and score plots. Turn equal scaling off in Plot settings. Control charts, Pareto charts, bar charts and box plots use the full width.

Drawings laid out to fit their text (correlation heat maps, decision trees, cause-and-effect diagrams and the Effect Pareto) have no size handles: they grow with the text size instead, and scroll sideways when wider than the pane.

The key below each plot names what each line, band and marker shows: a fitted model line is "Fit", but reference lines and curves carry their own names ("Reference line y = x", "4/n threshold", "OC curve", "Mean difference", "Limits of agreement"). Category names that would overlap are turned rather than dropped (see **Category labels** under Plot settings).

By default, plots draw horizontal major grid rules only, and axis ranges frame the data with a little padding (6% per side), keeping a zero baseline when all the data are non-negative. Points at or just above zero (a 0/1 response, the leaves of a dendrogram, Cook's distances) get padding below as well, so the frame does not cut them in half, and categorical axes leave half a category beyond the first and last. Date axes tick on calendar units instead, and axes that count samples or subgroups tick on whole numbers only. A tick label that would be cut off at the top of the vertical axis, or run into the corner at the left end of the horizontal axis, is left out.

Labels drawn inside a plot keep clear of each other: group names at the ends of survival curves and run-rule numbers on control charts move up or down where they would collide, and on a Pareto chart a cumulative percentage that would run into the one before it is left out (hover over its bar to read it). Specification limits and other reference lines on a histogram are named beside their lines, near the top, rather than across them.

Themes

Plots follow the light or dark colour scheme (File ▸ Settings…). In dark mode, points get a soft halo so they stand out, the darkest group colours (the first, blue, and the grey) are lightened so they stay visible on the dark canvas, and the zero line of residual plots is a neutral grey in both modes.

Saving plots and results

Plots

Right-click any plot, or its section heading, and choose **Save plot as PNG/SVG/PDF/EPS…**.

- **PNG** is drawn from the plot itself at 300 pixels per inch, not taken as a screenshot, so it is sharp at any size.
- **SVG**, **PDF** and **EPS** are vector figures for publications.
- The figure keeps the plot's on-screen proportions and current axis ranges, is titled with the section name, keeps each group's colour and marker shape, and draws excluded rows hollow and gray with a key entry.

On a **Response Explorer**, the same menu saves every panel at its current settings as one figure. On the **3-D surface profiler**, it saves the surface at its current angle and settings, and the 3-D scatter plot and 3-D surface of data do the same. The graph canvas offers the same items in its ⋯ Options menu (**Save as PNG…** and so on).

Copy plot

Copy plot puts the plot on the clipboard in three formats at once, so the program you paste into takes the best one it reads:

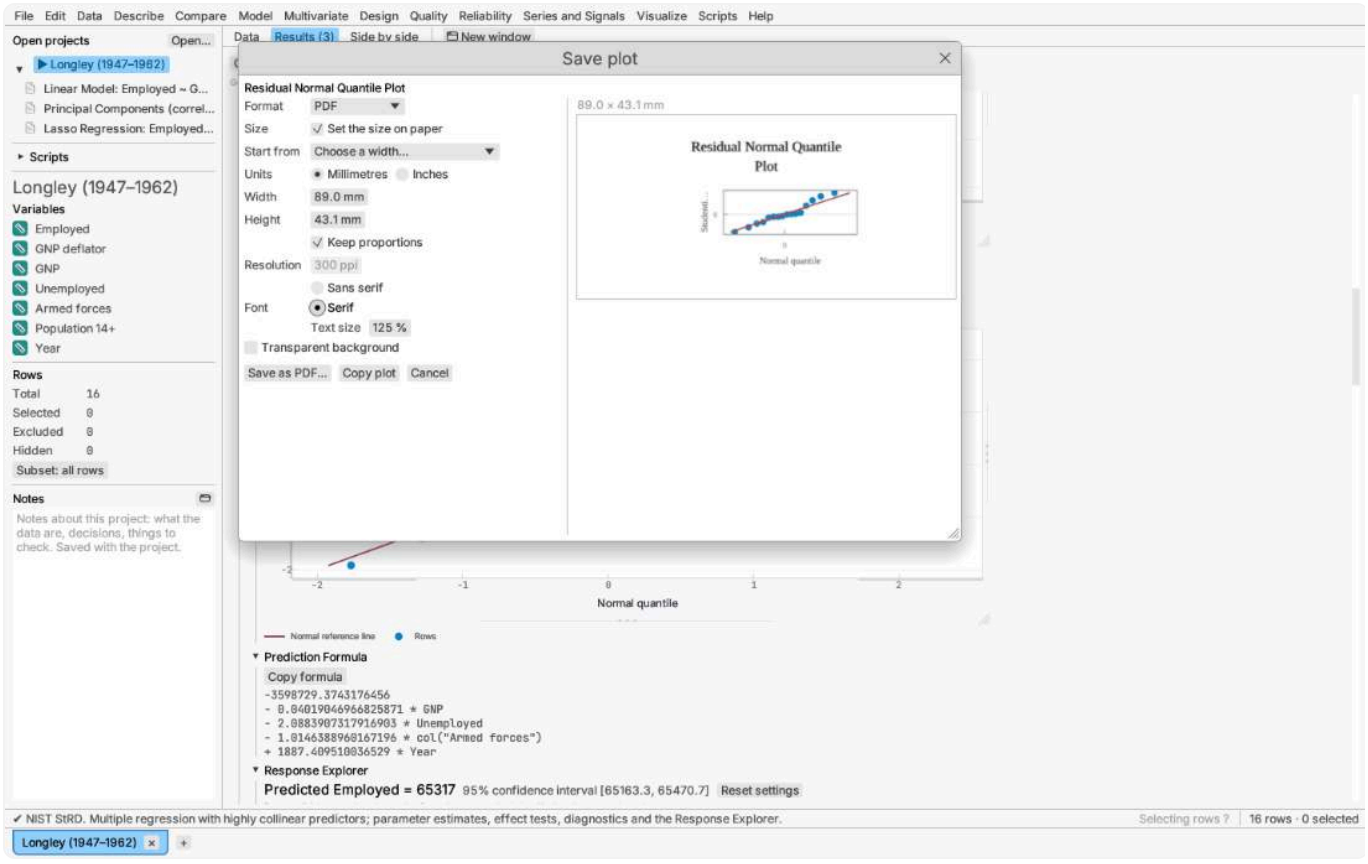
System	Formats offered
macOS	PDF (`com.adobe.pdf` ; Keynote, Pages, Word and PowerPoint paste it as a vector picture), PNG, SVG
Windows	a bitmap (what most programs paste), PNG (the `PNG` format Office reads), SVG
Linux	`image/png` , `application/pdf` , `image/svg+xml`

Windows programs paste vector pictures in the EMF format, which AnovaCraft does not write, so there the plot pastes as a picture. If the clipboard refuses the vector formats, the picture alone is copied and the message under the report says so.

Save plot as…

Save plot as… opens a dialog for one figure:

- **Format**: PNG, SVG, PDF or EPS.
- **Size**: off, the figure keeps the plot's on-screen proportions at about its on-screen size. **Set the size on paper** gives a width and height in millimetres or inches, with journal-column widths to start from (89, 120 and 183 mm) and **Keep proportions** (changing one side changes the other). The figure is laid out at that size, so its text keeps its point size: a larger figure gives the data more room rather than larger text.
- **Resolution** (PNG): 36–1200 pixels per inch; the pixel size is shown.
- **Font**: **Sans serif** (Helvetica in PDF and EPS, Liberation Sans in PNG) or **Serif** (Times, Liberation Serif in PNG). SVG files name the font and its common stand-ins (Helvetica, Arial, Liberation Sans; Times New Roman, Times, Liberation Serif). **Text size** (50–300%) enlarges the text with the marks and lines: the figure is laid out on a page that much smaller and enlarged to the size asked for.
- **Transparent background** leaves the page clear instead of white (EPS has no transparency, so translucent marks are still blended with white).



The preview shows the figure as it will be written. **Save as …** writes the file and closes the dialog; **Copy plot** copies the figure as set. Either keeps the settings with the plot: the plot's

Save plot as PNG/SVG/PDF/EPS… items and **Copy plot** use them from then on, and they are saved with the report (or the graph) in projects, dashboards, scripts and graph files. Pictures that are kept as drawn (3-D views, trees, diagrams, word clouds) are scaled to the size on paper, keep the settings for the session only, and ignore the text size.

Transparent background and Plot font

The same right-click menu has **Transparent background** (a check box) and **Plot font** (Sans serif or Serif, and the text size), which change the plot's figure settings without opening the dialog. On screen, the plot font and text size apply to the plot's titles and tick labels (tick numbers stay in the monospaced face, so digits line up).

Add to figure page, in the same menus, puts the plot on a page with other plots, lettered A, B, C…, saved as one file at a chosen size in millimetres or inches, pixel density, CMYK colour or transparent background (see Figure page).

Report tables

Right-click any report table, or its section heading:

- **Copy table** puts the table on the clipboard as tab-separated text with its headings, ready to paste into a spreadsheet or document. Typographic minus signs become plain `-` and "—" (not applicable) becomes an empty cell, so spreadsheets read the numbers as numbers.
- **Save as data table** adds the table to the current project as a new table; **Save as data table in a new tab** opens it as a separate project instead. Columns whose cells are all numbers become numeric columns; the rest are text. The new table is a copy: it is not updated when the report changes.
- **Bootstrap this table…** re-runs the report on resamples of its rows and gives bootstrap standard errors, bias and intervals for chosen cells (see Resampling). The report's ⋯ Options ▸ Bootstrap a table lists every table of the report.

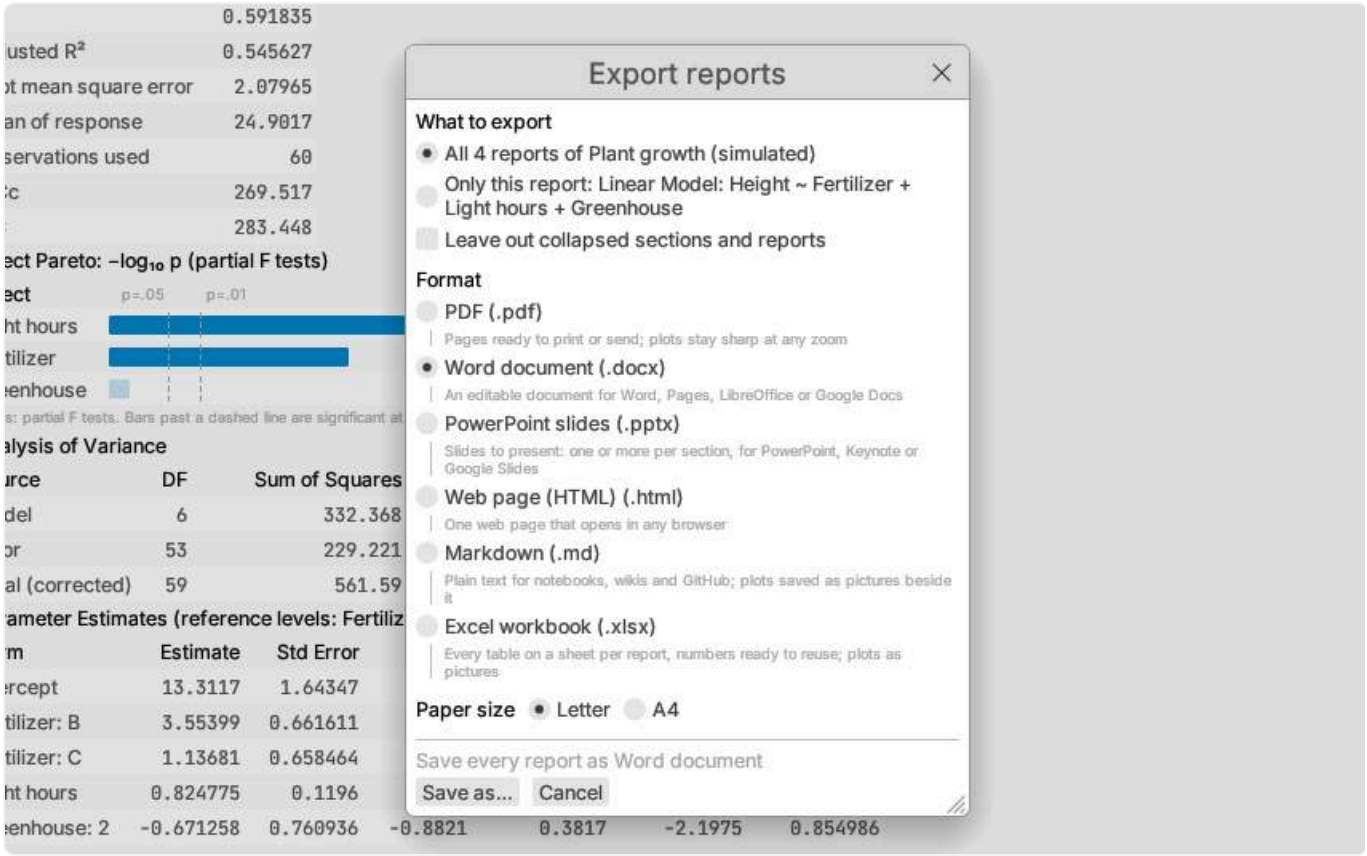
Numeric columns in report tables are right-aligned so their digits line up.

Data and report columns

- Export a table as CSV, TSV, JSON or `.xlsx` (see Documents, files and projects).
- Report options menus save computed columns into the table: predicted values, residuals, prediction formulas, principal component scores, cluster numbers, Box–Cox transformed values and others.

Whole reports as documents

Every report has an **Export…** button beside its ⋯ Options button (the same item is in ⋯ Options ▸ **Export report…** and the report's right-click menu). **File ▸ Export reports…** (⌘⇧E on a Mac, Ctrl+Shift+E elsewhere; also in the table's right-click menu in the sidebar and on its tab) opens the same dialog for every report of the table, as does **Export all…** at the top of the Results pane, beside Collapse all and Expand all (so reports can be exported with every report collapsed). The dialog opens in the window you asked from, so a report or results pane in a window of its own gets it there.



In the **Export reports** dialog choose:

- **What to export:** all the table's reports in the order they are shown (the default), or, when the dialog was opened from a report, **Only this report**.
- **Leave out collapsed sections and reports:** off by default, so every section is exported whether it is open or collapsed on screen. Tick it to export only what is open: collapsed sections are skipped, and so are reports collapsed down to their title (unless you export only that report).
- **Format:**

Format	What you get
PDF (`.pdf`)	Pages ready to print or send. Plots are vector drawings that stay sharp at any zoom; text uses the bundled Inter and JetBrains Mono fonts, embedded. Bookmarks list every report and section, and each page is numbered.
Word document (`.docx`)	An editable document for Word, Pages, LibreOffice or Google Docs. Report and section titles use Word's heading styles, so the navigation pane and a table of contents pick them up; table header rows repeat on every page; plots are pictures (Word 2016 and later draw them as vector graphics).
PowerPoint slides (`.pptx`)	A 16:9 presentation for PowerPoint, Keynote or Google Slides: a title slide, a title slide for each report when there are several, then a slide for each plot, table or note, titled with its section. Long tables continue on further slides (" (continued)") and very wide ones are split into groups of columns.
Web page (HTML) (`.html`)	One page that opens in any browser (see below).
Markdown (`.md`)	Plain text for notebooks, wikis and GitHub, with tables as Markdown tables. Plots are saved as PNG pictures in a folder named after the file (`results_files` beside `results.md`).
Excel workbook (`.xlsx`)	A sheet per report, its sections down the sheet. Cells that show a plain number are stored as numbers with the same decimals, so you can compute with them; plots are pictures.

- **Paper size** (PDF and Word): Letter or A4.

Then **Save as…** asks where to save the file. The dialog remembers the format and paper size you chose last.

What every format shares:

- It starts with the report's title (or "*table* reports"), the table, its row counts and the date it was exported.
- Every section is exported, collapsed or not, unless you tick **Leave out collapsed sections and reports**.
- Numbers are right-aligned so their digits line up.
- Plots are drawn as when saved with **Save plot as…**: titled, groups with their colours and marker shapes, excluded rows hollow and gray, on a white canvas.
- In PDF, tables too wide for the page are set smaller, then their text columns wrap, and if they are still too wide they are split into groups of columns, each starting with the first column. A table that runs onto the next page repeats its header row there.
- Interactive views that are not report sections (the Response Explorer and 3-D views) are not included; save them with their own **Save plot as…** items.

The web page

- Tables are real HTML tables, so they paste into a word processor or spreadsheet with their columns; numbers are in a monospace font.
- Plots are vector drawings (SVG) inside the page.
- The page follows the reader's light or dark appearance, using the application's colours.
- The page is self-contained: it loads nothing from the web. The bundled Inter and JetBrains Mono fonts (SIL Open Font License) are embedded in it, unless that would make the page larger than about 1.5 MB, in which case it uses the reader's system fonts.

A dashboard exports the same way, its tiles laid out on its grid (**Export as HTML…** in its toolbar, or **File ▸ Export dashboard as HTML…**).

Whole reports

Saving the document as a project keeps every report, and reopening it re-runs them. Scripts can also save a report, or every report of a table, in any of the formats above, sections as CSV or TSV, and plots as image files (see Scripting and recording).

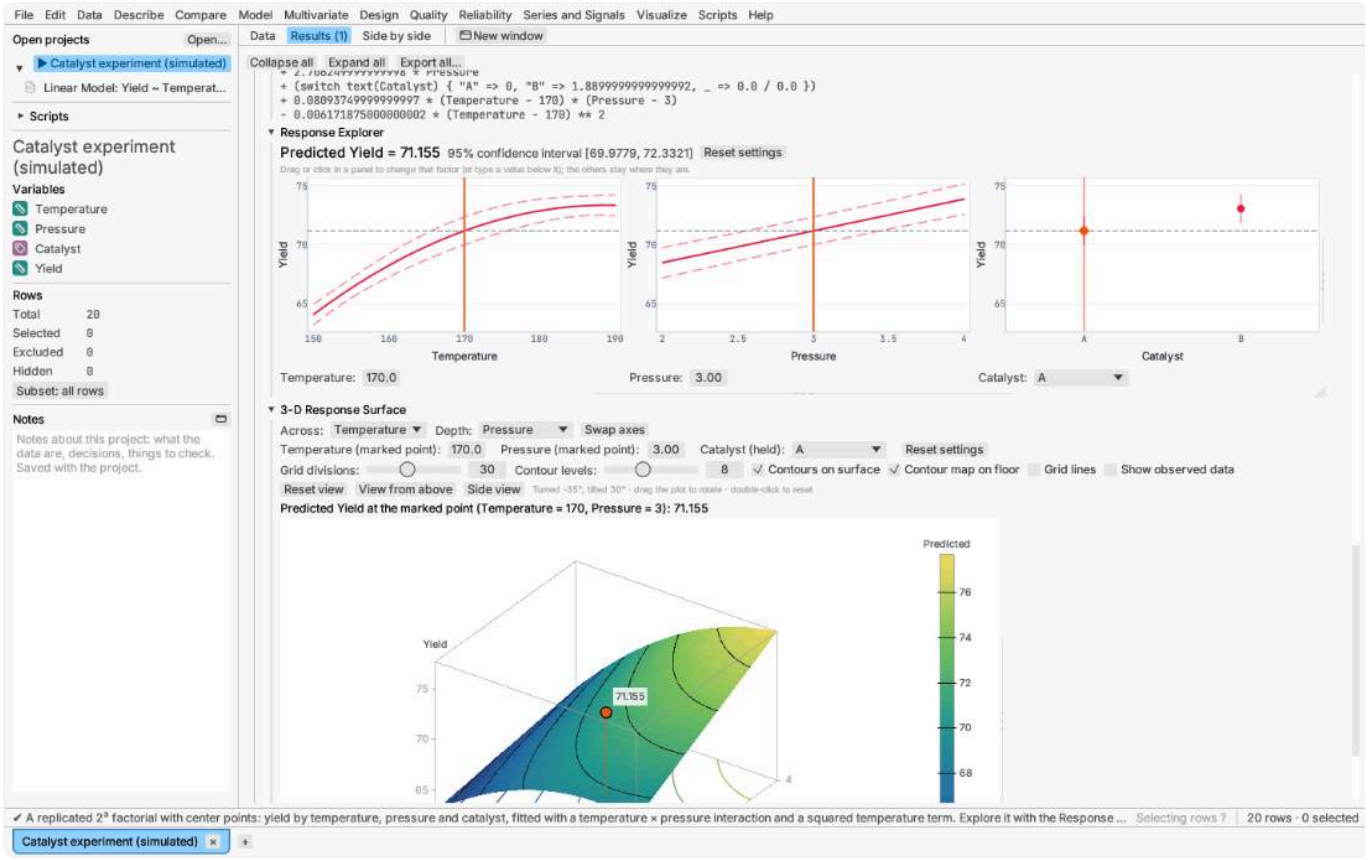
Response Explorer and 3-D profiler

Response Explorer

For a fitted model, the Response Explorer shows a row of slice plots, one per factor. Each plot shows the predicted response as that factor varies, with a confidence band, while the other factors are held at their current settings.

- Drag a factor's vertical reference line (or click in its panel) to change its setting. Every panel and the predicted value update as you drag. You can also type an exact value below each panel.
- A categorical factor's panel shows the prediction at each level with its confidence interval; click a level to choose it.
- All panels share one vertical scale, so you can compare the size of each factor's effect directly. Penalized fits are shown without bands.
- A button returns to the starting settings: continuous factors at their means, categorical factors at their most common level.
- In a **mixture** model, moving one component rescales the others so the mixture still sums to one.
- In the **several responses** report there is one row per response, plus a row for the overall desirability, all linked (see Model: several responses).
- The panels share one size. Drag the strip under them to make every panel taller or shorter together; drag its right-hand corner, or the strip beside the last panel, to make every panel wider or narrower. Double-click the strip to go back to the default size, which fills the pane's width. Panels that no longer fit side by side wrap onto another row. The size is kept for each report while the application is open, and a saved figure uses it.

Add it from a model report's ⋯ Options menu. It is available for linear, response-surface, mixture, generalized linear (population level for mixed models), penalized, decision tree, random forest, boosted tree, partial least squares, support vector machine and adaptive regression spline models, for the model chosen by stepwise selection, for proportional hazards (the hazard ratio, or survival at a chosen time) and for parametric survival regression (a predicted percentile). Right-clicking the 3-D profiler saves the surface at its current angle and settings.



3-D surface profiler

The 3-D surface profiler draws the fitted surface over two continuous factors, with the observed points. Drag to rotate it. Other factors are held at their current settings. Resize it with the handles under it and along its right edge, as any other plot (see Plots).

Both views can be saved as figures from their right-click menu (see Saving plots and results).

Graph canvas

Visualize ▸ Graph canvas… builds a graph by dragging columns onto shelves and stacking layers. It is the place for exploratory and presentation graphics that are not tied to one analysis.

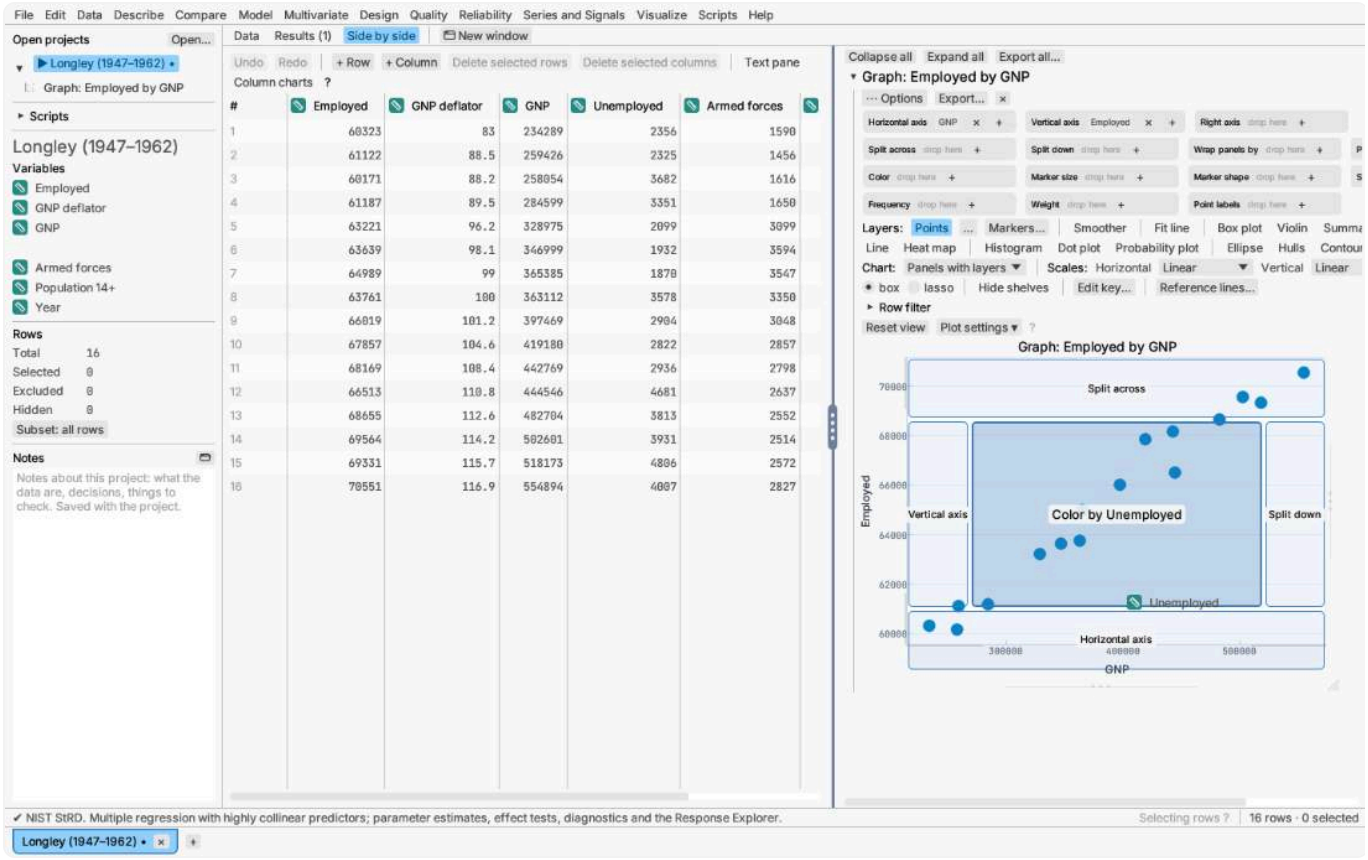
Shelves

Drag a column from the list onto a shelf:

- **Axes:** X and Y, and the **Right axis** (see below).
- **Panel splits:** across, down, wrapped, or onto separate pages, to make linked small multiples.
- **Colour, size and shape** encodings, and **subgroups**.
- **Frequency** and **weight** columns. A frequency counts as that many repeated rows everywhere, including fit lines and smoothers.

A graph canvas opened in a window of its own (⋯ Options ▸ Open in new window) lists the columns at the window's left, since columns cannot be dragged from one window to another: drag them from there.

While you drag a column, **drop zones** appear over the panels: the bottom edge for the horizontal axis, the left edge for the vertical axis, the top edge to split across, the right edge to split down and the centre for colour. The zone under the pointer is highlighted and says what it will do ("Color by Region"); let go there and the graph redraws. A column dropped on an axis takes that axis's first place (on an empty axis it is added); a chip dragged from a shelf moves, while a column from the variables list stays on any other shelf it is on. The shelves remain the way to reach every role, and to work from the keyboard.



A **column picker** lets you flip a shelf between several columns, and a per-graph **row filter** (+ Add condition) limits the graph to matching rows without changing the table's row states. The filter's form is the filter panel described under Filtering a report and switching its column: level lists, sliders and **Step through** a column one level (or one window of values) at a time, by hand or with ▶ **Play**. **Show beside the graph** moves the filter into a panel left of the panels, so its sliders and stepping controls sit next to the picture they change; **Move above the graph** puts it back under the Row filter heading. Below the conditions, **Show only matching rows**, **Analyse only matching rows** and **Select matching rows** choose what happens to the other rows.

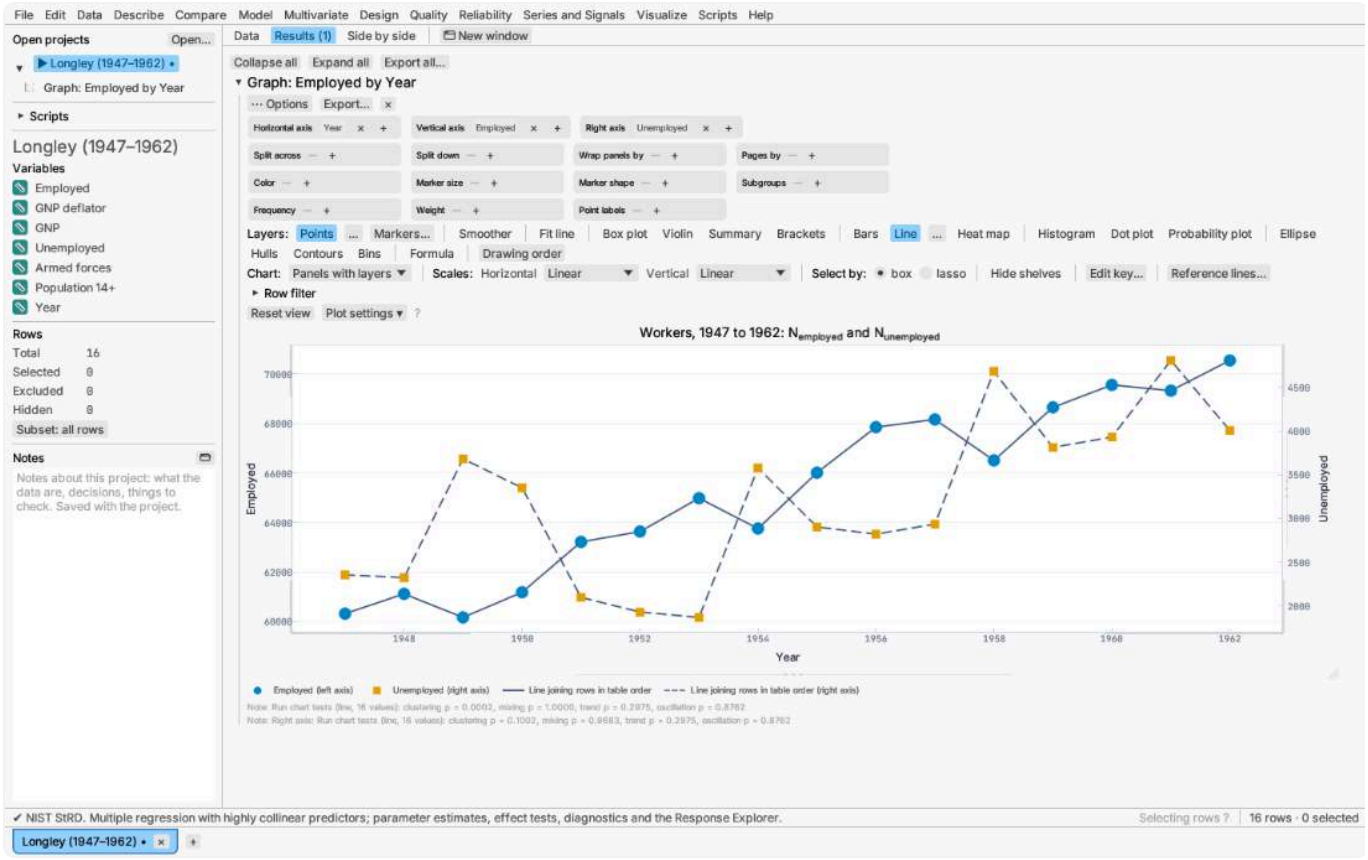
Axis scales

Linear, $\log_{10}$, ln, logit, square root and reciprocal. Date and time columns get calendar axes. Changing a scale (or the chart form) re-fits the axes to the data, dropping any ranges set by dragging an axis.

Panels showing the same column share its range, so dragging an axis in one panel moves it in all of them. The ranges under **Plot settings** are those of the first panel; typing new ones applies them to every panel, and **Fit view to data** (like **Reset view**) re-fits them all.

Right axis

Put one or more numeric columns on the **Right axis** shelf to draw them against a second vertical axis on the right, with its own scale, ticks, title and spine: for example temperature on the left and rainfall on the right of the same time axis. The right-axis columns get the same layers as the left (points, lines, fits…), and share the left axis's scale (linear, log…). The key starts with which series uses which axis: the left column "(left axis)", each right column " (right axis)", and every right-axis layer entry ends in "(right axis)". Right-axis lines are dashed and their markers differ in shape (hollow when the graph is split into groups by colour), so the axes are told apart without relying on colour. A categorical column cannot go on the right axis.



Select right-axis points by box, lasso or click as usual: they select the same rows everywhere. The right axis is a ruler like the left one: drag near an end to move that limit, elsewhere to shift it; double-click it to re-fit. Right-click it (or use **Plot settings ▸ Right axis**) to type its title and turn its tick marks and tick labels on or off. The right axis is drawn for the panels chart form (not for treemaps, parallel coordinates and the other whole-graph forms); its range is shared by every panel.

Axis breaks

Plot settings ▸ Horizontal axis / Vertical axis ▸ Axis breaks leaves stretches of a continuous axis out: tick **Leave out** and type where the gap starts and ends, in data units. The axis then runs straight on from the start of the gap to its end, with a short mark on the axis line showing where. Use it when a few values lie far from the rest, so the bulk of the data is not squashed. Values inside a gap are not drawn: markers there are hidden (a note under the graph counts them), lines are cut at the gap, and bands and areas stop at its edges. Fits and statistics are still computed from all the data. A break needs a continuous axis (not categories or dates), and applies in the scale shown (on a log axis, the gap is

between the logs of the two values). Saved figures draw the same gaps and marks. Clearing **Leave out** removes every break and restores the axis unchanged.

- **Add a break** leaves another stretch out, starting as the middle third of the widest piece of the axis still shown; each further break has its own **from** and **to** fields and a ✕ that removes it. Breaks are kept in axis order, and overlapping ones act as one.
- **Segments and marks** lists the pieces of the axis between the breaks (*Below 10*, *150 to 900*, *Above 2000*, …). For each piece:
  - **Scale** — *Same as axis* (the default), or a scale of its own: linear, log (base 10 or natural), square root, logit or reciprocal, for example a log segment for small values below a linear one. Each piece is ticked and labelled on its own scale, and a note under the plot names the scales, since the labels alone would not say the spacing changes.
  - **Automatic share** — off, the piece takes a fixed **share** of the axis length in percent (a long tail squeezed into the top 20%, say); pieces left automatic share what is left equally. Shares are laid out over the extent of the data on that axis, and the lowest piece holding data keeps its own units, so dragging and zooming the axis still work. With no share and no scale set anywhere, each piece keeps the axis's own spacing.
  - **Break marks** — **Slashes** (two short parallel strokes, the default) or **Zigzag** (a short zigzag in the axis line), drawn on both spines of the plot.

A piece on another scale bends straight lines only at their vertices, so a line joining two distant points across such a piece is drawn straight in the broken drawing; fitted curves are sampled finely enough that this does not show. Pieces with very different scales can mislead: Cleveland (*The Elements of Graphing Data*, 1994, §2.4) suggests a separate panel for each piece when the difference is large.

Report scatter plots take the same breaks (see Working with plots).

Titles and math

Type a graph title or axis titles in **Plot settings**; they accept the math markup described in Math in labels. Every Plot settings choice (ticks, grid, number format, key, titles, breaks) is saved with the graph, in projects and in graph files. **⋯ Options ▸ Read math markup in column names, levels and labels** reads the default titles, key entries and text labels as markup too.

Layers

Layers combine on the same panels:

- jittered or swarmed points and point labels;
- polynomial fits with confidence and prediction bands, and a movable label with the equation, R^2 and p-value;
- smoothing splines and LOESS with confidence bands;
- box plots (optionally notched) and violins;
- summary statistics with error bars;
- density ellipses, kernel-density contours (with **Shade the regions**, a translucent wash inside each contour, stacking so the densest region is darkest), and hexagonal or square binning;
- formula curves;
- bars (side by side, stacked or 100%) with error bars;
- line and run charts (a line joining rows in table order adds the four run-chart tests under the graph: clustering and mixing from runs about the median, trend and oscillation from runs up and down, each a one-sided p-value);
- category heat maps;
- histograms, dot plots and probability plots of one continuous column (below);
- significance brackets and convex hulls (below).

Each layer's **…** button holds its settings. At the end of the row below (the **Chart:** row), **Reference lines…** opens the reference lines and bands of the graph (see Working with plots); the button shows how many there are.

Error bars. The summary and bars layers draw error bars (standard error, standard deviation or a confidence interval for the mean). With bars chosen, their **…** menu also sets the bars' look: *Own colour* (otherwise each subgroup's colour), the line *width*, the cap width (*caps* ×, a multiple of the usual; 0 draws no caps), and the direction: **Both ways**, **Upper only** or **Lower only**, where a one-way bar runs from the mean to one end of its interval.

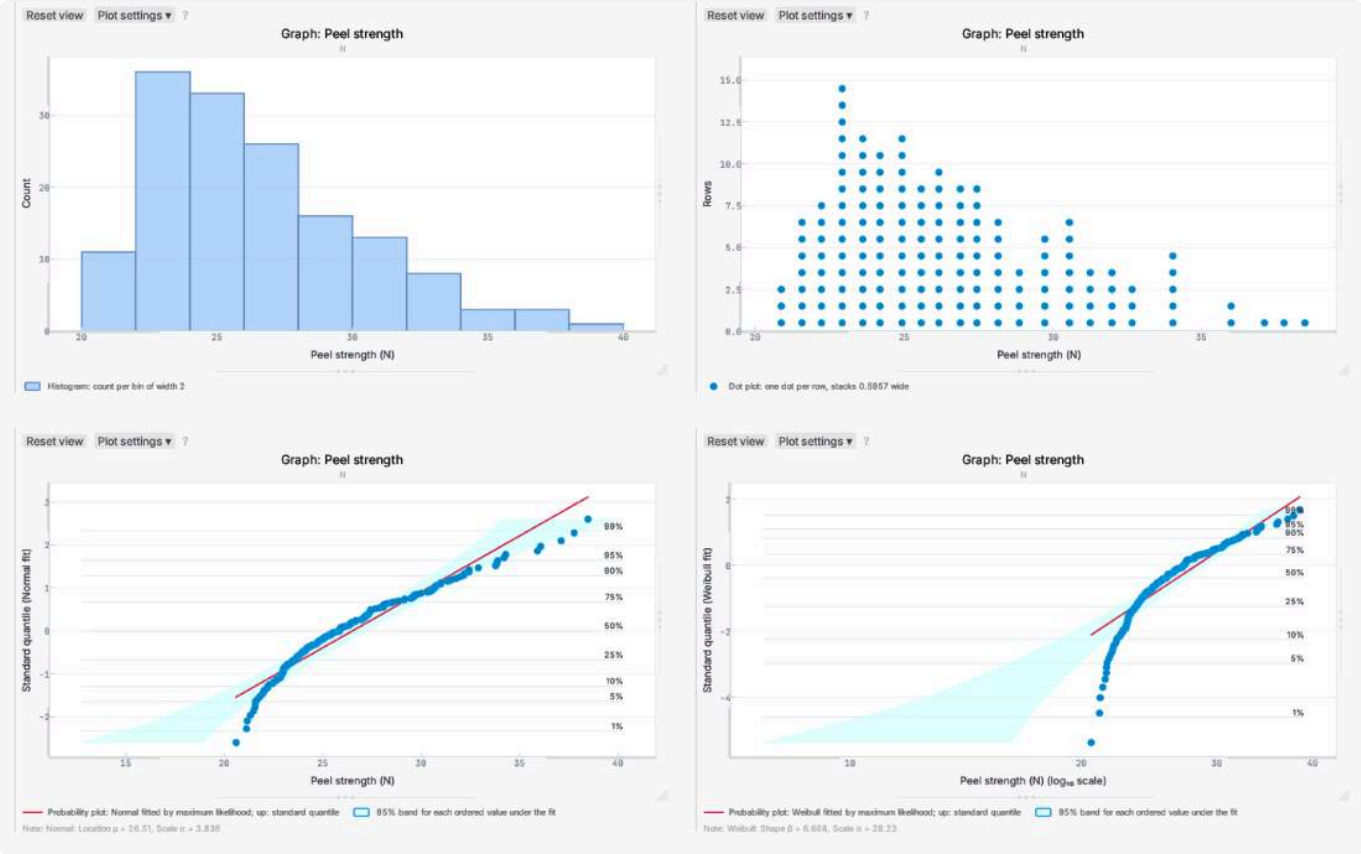
Drawing order. Layers are drawn back to front: area fills (heat maps, bins, contours, bars, violins, boxes), then bands and lines, then summaries and points, then brackets. **Drawing order** at the end of the layer bar lists the layers that are on, front first, each with ▲ (*Bring forward*) and ▼ (*Send backward*); right-clicking a layer's toggle offers the same two commands. The order is saved with the graph (`layer_order` in the graph file, back to front); a layer turned on later takes its usual place among the others.

Distributions of one column

Put a continuous column on X and leave Y empty, then turn on one of these layers. The vertical axis is titled by the layer.

- **Histogram:** counts per bin, or density (each subgroup's bars then have area 1). Bins are automatic (Sturges' rule widened to a round width such as 0.25 or 5), about a number you choose, or a width you type. With a colour or subgroup column the groups' bars are overlaid translucently, or stacked. Bins follow the horizontal axis scale, so on a log scale they are equal in ratio. A typed width that would need more than 10,000 bins is widened to the nearest round width that fits, and a note says so. The selected rows' share of each bar is shaded dark with an orange outline, and clicking a bar selects its rows.
- **Dot plot:** one dot per row (Wilkinson's dot-density method), stacked where values lie within the stack width of each other (by default 1/30 of the range). Selected rows are ringed, and clicking a dot selects its row.
- **Probability plot:** the ordered values against the quantiles of a distribution fitted to them by maximum likelihood, chosen from the box beside the toggle (any of the 14 in Describe ▸ Identify distribution). Points along the line mean the distribution fits. The shaded band shows, for each ordered value, where it would fall 95% of the time (or the level you set) if the fitted distribution were true; points well outside it point to a poor fit. Dotted guides mark cumulative percentages, and the fitted parameters are listed under the graph. For the lognormal, Weibull and log-logistic the line is straight on a log horizontal scale.

The dot and probability plots use the vertical axis for their own scale, so turning one on turns the points layer off.



Comparing groups

Swarm points. In the points layer's **…** menu, *Swarm (no overlap)* lays the markers of each category side by side instead of scattering them at random. Rows are placed in order of their value; each goes at the offset from the category's centre line nearest zero (left before right on a tie) at which its marker does not touch any marker already placed. The spacing is worked out for a panel about 480 × 300 points and then narrowed, if need be, to the jitter width, so a crowded category stays in its slot (its markers may then touch); a category with more than 2000 rows is jittered at random instead. The layout depends only on the values, so it is the same each time and in exported files.

Significance brackets. With a categorical horizontal axis and a continuous vertical one, the *Brackets* layer compares the categories in pairs and draws a bracket over each pair, written with the adjusted p-value, stars or both. Its **…** menu chooses the procedure:

Procedure	Compares	Adjustment
Tukey–Kramer (default)	every pair	studentized range, pooled variance

Procedure	Compares	Adjustment
Games–Howell	every pair	studentized range, each pair's own variances (Welch degrees of freedom)
Holm-adjusted t	every pair	pooled-variance t, Holm's step-down
Dunnett	each category with the first	multivariate t, pooled variance

Each bracket sits a gap above the tallest point within its own span (the *gap above the data* slider, as a share of the axis), and a bracket over a span that overlaps one already placed goes a step above it; shorter spans are placed first, so they sit lowest. *Only pairs with p below* hides the others. **Pairs to draw** is **Every pair compared** (the default) or **Only the pairs ticked**, which lists every pair of categories as a tick box (such as *A* – *B*); a pair the procedure does not compare (Dunnett compares each category with the first only) is named in the notes instead. *Write "ns" on pairs that are not significant* labels those brackets "ns" when stars are written. **Stars below p =** sets the four thresholds for *, **, *** and **** (each below the one before; **Usual** restores 0.05, 0.01, 0.001 and 0.0001). The notes under the graph name the procedure and the star key. Panel splits get their own brackets; subgroups within a category are pooled; categories with fewer than two values are left out. The p-values come from the same routines as the pairwise comparisons of the group comparison reports.

Estimation plot. The *Estimation plot* chart form shows every value of each group (the horizontal-axis column) as a swarm, with the group's mean ± one standard deviation drawn as a gap and line beside it, and the mean difference of every other group from a reference group with its confidence interval (Gardner & Altman 1986; Cumming 2012; Ho et al. 2019).

- With **two groups** the difference has a floating axis of its own on the right, whose zero is level with the reference group's mean, so the difference reads straight off the data (a Gardner–Altman plot).
- With **more groups** the differences go in a lower panel, one under each group, with a line at zero (a Cumming plot).

The interval is by default a **BCa bootstrap** interval: the two groups are resampled separately 5000 times (200 to 100 000), with the bias correction and acceleration of Efron (1987), the acceleration from the jackknife. The bootstrap distribution of the difference is drawn as a half violin. The seed is shown and can be changed; the same seed gives the same interval (group *k* uses seed + *k*). **Welch t** gives the unequal-variance t interval instead. The confidence level (default 95%) and the reference group (default the first) are set in the bar under the chart list. The differences, limits, level and method are written in the notes and listed in the summary table.

Convex hulls. On a scatter panel (both axes continuous) the *Hulls* layer draws the smallest convex polygon around each color or subgroup group's points (Andrew's monotone chain), outlined in the group's color and in a line pattern of its own (solid, dashed, dotted, in turn), and lightly shaded unless *Shade inside each hull* is turned off. Clicking inside a hull selects its rows.

Other chart forms

Besides panels with layers: **treemap**, **parallel coordinates**, **multi-vari chart**, **scatterplot matrix**, **clustered heat map**, **estimation plot** (above), **Sankey diagram**, **chord diagram** and **network** (these five have their settings in a bar under the chart list), and the round and special forms in Pie, radar, polar, ternary and other charts below.

The **treemap** draws nested rectangles, one level per horizontal-axis column in order, laid out by the squarified algorithm (Bruls, Huizing & van Wijk 2000). Each rectangle's area is its count of rows (weighted by the frequency column), or, if you choose, the sum of the first vertical-axis column; labels can be shown inside the tiles.

Parallel coordinates gives each column (horizontal shelf, then vertical shelf) a vertical axis scaled from its minimum to its maximum and draws every row as a line across them, so rows that behave alike run together. Box and click selection work on both forms.

The multi-vari form plots the first vertical-axis column against up to four horizontal-axis columns, outermost first: every row at its cell, each factor's level means marked in its own colour and shape and joined within each level of the factor above, and the outermost factor's levels as labelled sections. The means count the frequency and weight columns; excluded rows, and rows whose frequency or weight is zero or missing, are drawn but left out of the means. The report form, with its variation table and ranges view, is Quality ▸ Multi-vari chart.

The **scatterplot matrix** form crosses every continuous column on the axis shelves with every other in a square grid (row *i* plots column *i* up, column *j* across). The diagonal panels show each column's histogram (or the dot plot or probability plot layer, if one is on); the other panels show the remaining layers — points, density ellipses (turned on when you choose the form), fit lines, smoothers. Panel splits are not used; colour, shape and size columns are. Brushing any panel selects the rows in all of them. Multivariate ▸ Scatterplot matrix… sets one up from a column list.

The **clustered heat map** takes the numeric columns on the horizontal shelf as its columns and every row (up to 1000) as a row, labelled by the *Point labels* column or by row number. Each cell is shaded on a sequential ramp, with the ramp as the key. By default each column is standardized (mean 0, standard deviation 1, divisor $n - 1$), so columns in different units can be compared, and both rows and columns are reordered by hierarchical clustering of the Euclidean distances between them (average linkage by default; complete, single and Ward are offered, as in Multivariate ▸ Cluster analysis), with the dendrograms drawn to the left and above. Clustering can be turned off for rows or columns, and the values can be written in the cells. Click a cell to select its row; selected rows are outlined.

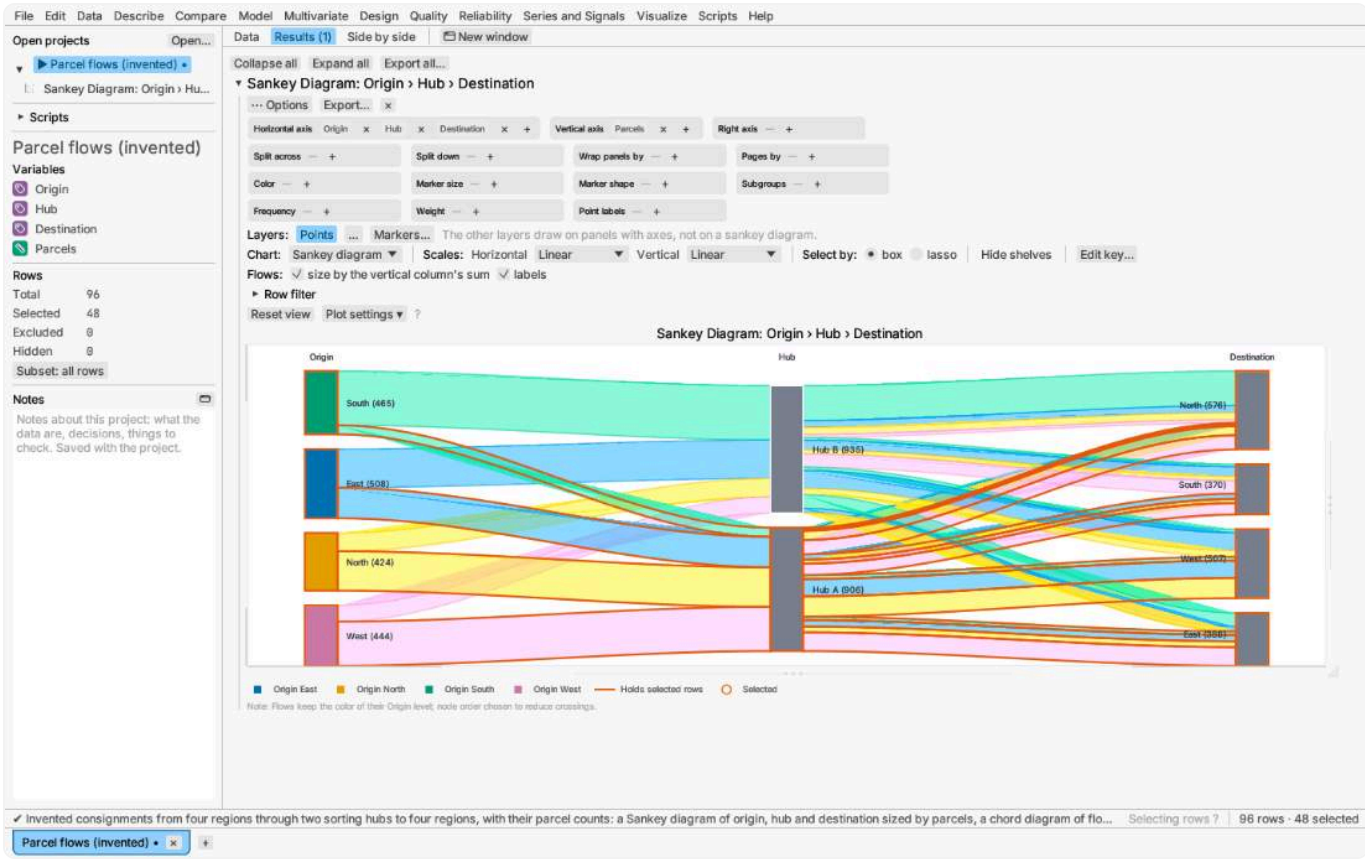
The **Sankey diagram** treats each horizontal-shelf column as a stage, in order. Each level of a stage is a node, its height the number of rows at that level (or, with *size by the vertical column's sum*, the sum of the first vertical-shelf column), and flows join the levels of neighboring stages, colored by their level in the first stage. Nodes are ordered within each stage to reduce crossings by barycentre sweeps (Sugiyama, Tagawa & Toda 1981): each stage is sorted by the flow-weighted mean position of its neighbors in the stage before, then after, and the order with the fewest weighted crossings is kept.

The **chord diagram** takes a "from" and a "to" column (the first two on the horizontal shelf) with the same levels: each level is an arc of a circle sized by its total flow, out and in, and

ribbons join the arcs, as wide at each end as the flow they carry, colored by their "from" level.

The **network** form draws the two ends of each row (the first two horizontal-shelf columns) as nodes joined by an edge, the edge's width the number of rows (or the sum of a numeric vertical-shelf column), and each node's size its number of neighbors. Nodes are placed by the Fruchterman–Reingold (1991) force-directed algorithm from seeded random starts: the same seed and number of iterations give the same picture.

On all three, clicking a flow, ribbon, arc, node or edge selects its rows, and the selection is outlined. Box selection does not apply.



Pie, radar, polar, ternary and other charts

Choose these from the **Chart** list; each form's settings appear beside it. The pie, radar, polar, wind-rose and ternary forms are drawn true to shape (a circle stays round and the triangle equilateral however the report is sized, on screen and in exported files) and draw their own scales instead of numeric axes. The other layers do not draw on these forms, except the points layer on the polar, ternary and vector forms.

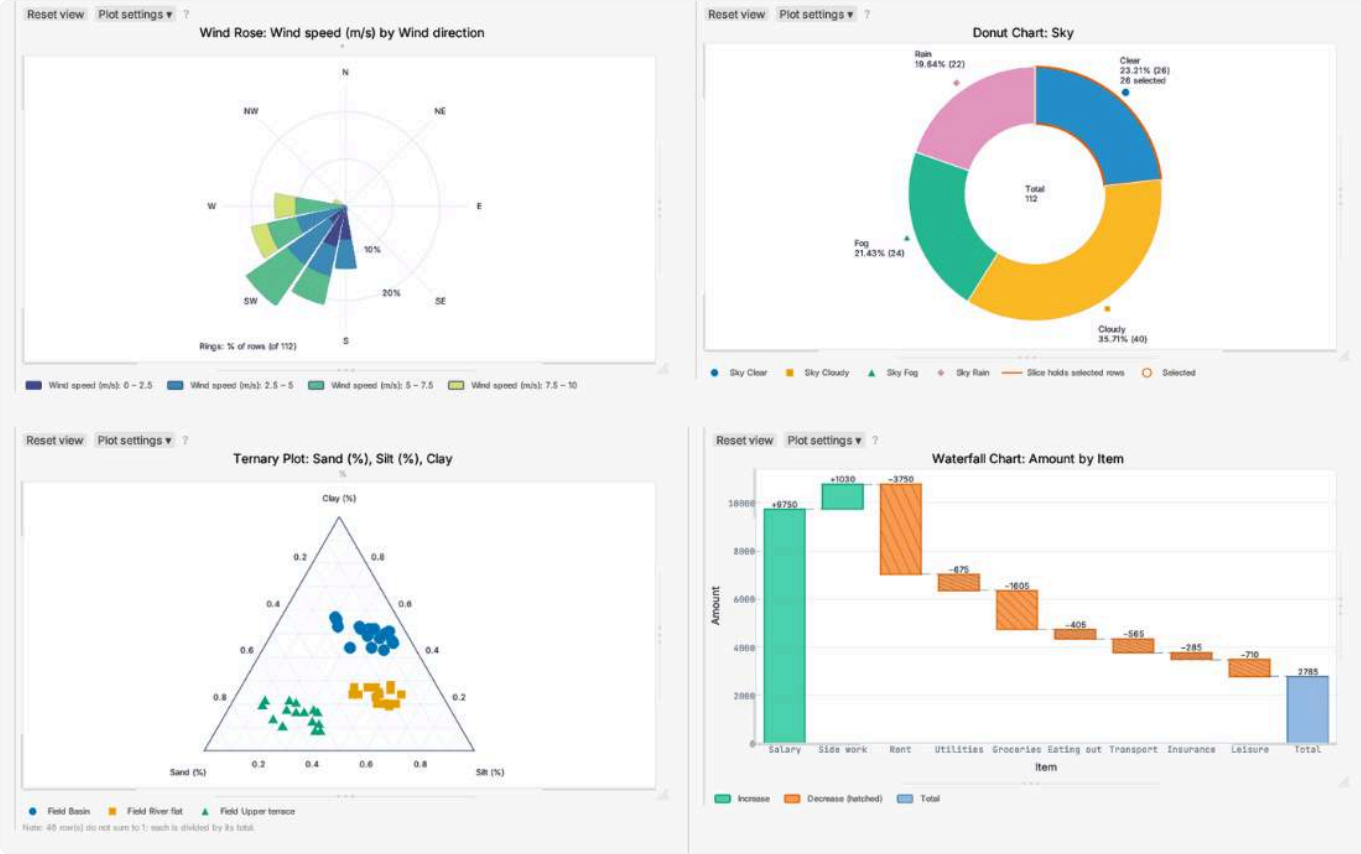
- **Pie chart.** The first horizontal-axis column's levels are the slices, clockwise from the top in the order of the column's levels. A slice's angle is its count of rows (weighted by the frequency column) or, with *size by the vertical column's sum*, the sum of the first vertical-axis column (rows with a missing or negative value are left out, with a note). Each slice is labelled with its level, its share in percent and its value, and carries a marker in its level's shape, so slices can be told apart without colour. *Donut* opens the centre and writes the total there. Clicking a slice selects its rows; a slice holding selected rows is ringed and its label says how many are selected. Samples: *Harbour weather* (sky), *Household budget* (spending by category).

- **Radar chart** (also called a star or spider chart). Every continuous column on the axis shelves (horizontal, then vertical; at least three) is a spoke, scaled from its minimum (near the centre) to its maximum (at the rim); its range is written under its name. Each row is a polygon through its values, or, with *group means*, each colour (or subgroup) level's means. Groups each have their own colour, marker shape and line pattern (solid, dashed, dotted); mean polygons are labelled "*level* mean", and clicking one selects its group's rows. *Fill* shades the polygons translucently. Sample: *Harbour weather*.
- **Polar chart.** The first horizontal-axis column is the angle and the first vertical-axis column the radius (zero at the centre, the largest radius at the rim; negative radii are left out with a note). Angles can be read as compass degrees (0° up, clockwise), mathematical degrees (0° to the right, anticlockwise) or radians. *Join in row order* draws a line through the points. Spokes every 30° and the rings are labelled.
- **Wind rose.** The first horizontal-axis column is a direction in compass degrees (where the wind comes from), divided into 8, 16 or 36 sectors centred on north. Each sector's bar is the count of its rows, stacked by bins of the first vertical-axis column (edges at round values, such as 0 – 2.5 and 2.5 – 5; a categorical column's levels are used as they are), coloured along an ordered ramp and listed in the key. Rings read in percent of all rows or in counts. Clicking a piece selects its rows. Sample: *Harbour weather*.
- **Ternary plot.** Three continuous columns (the axis shelves in order) are the components; each row is divided by its total, so the components need not already sum to 1. Rows with a negative or missing component or a zero total are left out with a note. The first component's corner is at the bottom left, the second's at the bottom right and the third's at the top; gridlines every 0.1 run parallel to the edges and are labelled along them. Points take colour, shape and size from the usual shelves; box, lasso and click selection work as on any scatter plot. Sample: *Soil texture*. (The mixture-design reports draw their ternary plots with the same geometry.)
- **Waterfall chart.** The first horizontal-axis column's levels, in the order they first appear in the table, are steps; each step's bar runs from the running total before it to the total after adding that level's sum of the first vertical-axis column. Increases and decreases differ in colour and in pattern — decreases are hatched — and each bar is labelled with its signed value (+120, −35). Dashed lines join each bar's end to the next bar's start. *Total bar* adds a last bar from zero to the overall total. Clicking a bar selects its rows. Sample: *Household budget*.
- **Ridgeline plot.** The first horizontal-axis column's distribution is drawn as a kernel density (Gaussian kernel, Silverman's rule-of-thumb bandwidth for each group) for each level of the first vertical-axis column, which must be categorical — or of the colour column when the vertical shelf is empty. The ridges are stacked down the page, overlapping, all on one height scale so that their areas compare; *height %* sets

the tallest peak as a percentage of the gap between ridges. *Rug* draws a tick for each row under its ridge; selected rows' ticks are highlighted. Sample: *Harbour weather*.

- **Vector field** (quiver plot). The horizontal shelf holds the x position and then the arrow's horizontal component; the vertical shelf holds the y position and then its vertical component. Each row is an arrow from its point. The arrows are scaled together so that the longest is about the typical spacing between points (*arrow length %* adjusts this); the note under the graph says how long one unit of the components is drawn. Selected rows' arrows are highlighted. Sample: *Harbour weather* (the wind through the fortnight).

Lollipop charts are a layout of the bars layer: in the bars' **…** settings choose *Lollipops (stem and dot)*. Each bar becomes a stem from zero with a dot at its value, in its group's colour and marker shape; clicking one selects its rows, and stems holding selected rows are ringed.



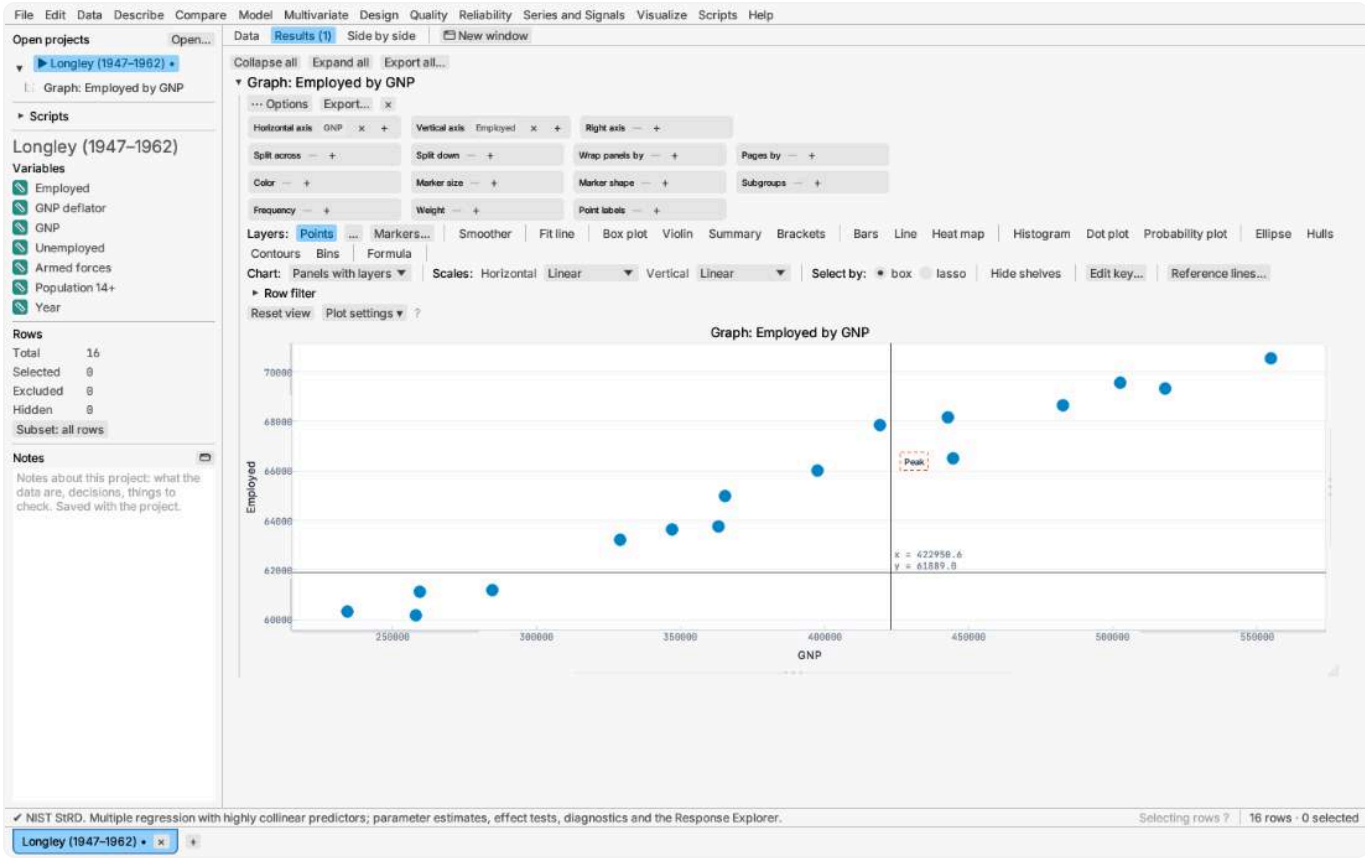
Annotations

Right-click a panel and choose **Add text**, **Add arrow**, **Add line** or **Add rectangle** to place one at the point clicked. New text opens its editor at once; text left empty is dropped. Text

is read as math markup (`x_1`, `x^2`, `\alpha`, `\frac{a}{b}`; see Titles and math), and an arrow can carry text at its tail.

- **Move:** drag the annotation (an arrow, line or rectangle is caught near its strokes, text anywhere on it).
- **Nudge:** click it to select it (a dashed outline), then press the arrow keys: one point per press, ten with Shift. Escape clears the selection; Delete removes the selected annotation. A movable statistics label (a fit line's equation and R^2, a density ellipse's correlation) is selected and nudged the same way.
- **Edit:** double-click it (a double-click elsewhere still re-fits the view), or right-click it and choose *Edit …*. The editor holds the text and what the position is measured in:
  - *Data values (moves with the axes)*, the default: the annotation keeps its data coordinates, so it moves when the axes are dragged and follows a change of scale (a log axis, an axis break).
  - *Plot frame (stays in place)*: its position is a fraction of the visible view, so it stays in the same place on the plot.

  Switching keeps the annotation where it is drawn.
- **Order and delete:** right-click it for *Bring forward*, *Send backward* (among the annotations, which are drawn above every layer) and *Delete*.



Annotations belong to the panel they were placed on, are drawn in exported figures, and are saved with the graph (`axes.style.annotations` in the graph file: shape, anchor, panel, the two points and the text).

Markers and key

Set marker size, colour and shape per plot or per group (right-click a marker in the key, or use **Markers...**), and choose where the key goes.

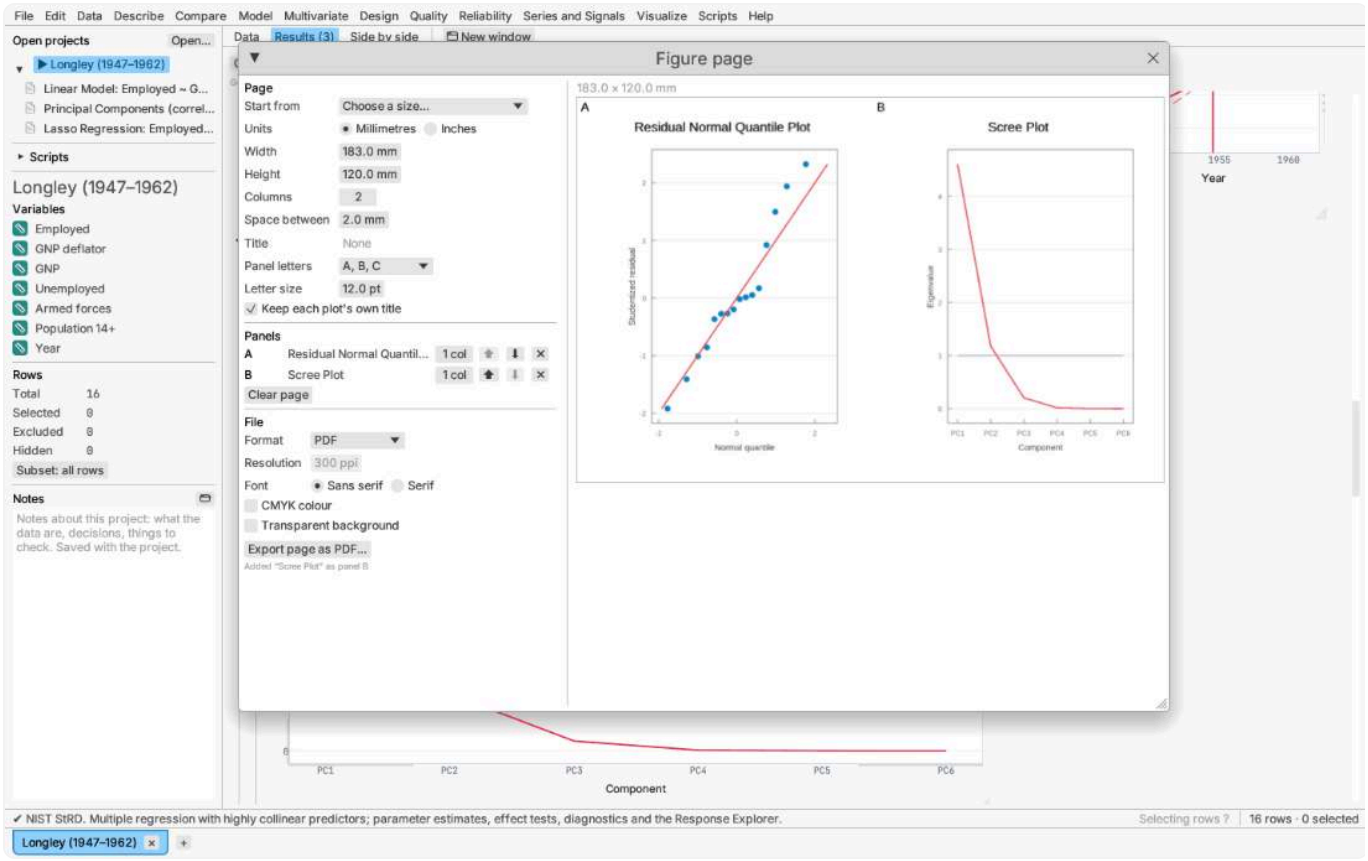
Edit key... (in the Chart bar, or right-click the key) lists every entry the key can show, in its order. Untick an entry to hide it, type over its text to rename it (math markup is read as in other key entries), and move it with ▲ and ▼. **Reset key** undoes every change. Each change follows what the entry stands for, a layer's entry or a column and one of its levels, so the changes stay put when the data change and a level that appears later still gets its entry. Exported figures show the key as edited. The changes are saved with the graph (`axes.style.key`).

Saving

Export the graph as PNG (300 ppi), SVG, PDF or EPS, or view its summary table. The file draws the same grid rules, titles, axis breaks and right axis as the graph on screen: horizontal rules only by default, and vertical rules too when they are turned on in the graph's axis settings. Save the graph as a JSON **pipeline file**: Visualize ▸ Graph from pipeline file… reopens it on any dataset with the same column names, so one graph design can be reused across data files.

Figure page

Visualize ▸ Figure page… gathers several plots on one page, letters them A, B, C…, and saves the page as one PNG, SVG, PDF or EPS file at the size a journal or poster asks for.



Adding plots

Right-click any plot (or its section heading) and choose **Add to figure page**, below the **Save plot as…** items. On a graph canvas the item is in the report's **⋯ Options** menu, after the **Save as …** items. The figure page window opens and the plot becomes the next panel.

- **Report plots, graph canvases and the Response Explorer** are kept as a copy of what they show when added (the same rows, groups and axis ranges) and are redrawn at the size of their panel whenever the page changes, so tick labels and titles keep their usual point size.
- **Other pictures** (3-D views, decision trees, cause-and-effect diagrams, word clouds) are kept as they were drawn and shrunk to fit their panel; hovering a panel's name in

the list says which kind it is.

Later changes to the report do not reach the page: add the plot again and remove the old panel. The page lasts while AnovaCraft is open.

Page settings

Setting	What it does
Start from	Sets the width to one journal column (89 mm), one and a half (120 mm) or two (183 mm), or the whole page to A4 or US Letter, upright or turned.
Units	Millimetres or inches for every length in the window.
Width, **Height**	The page size (1 inch to about 1.2 m a side). The default is 183 × 120 mm.
Columns	Columns of the grid (1–6). Panels fill it in reading order, in rows of equal height.
Space between	The gap between neighbouring panels.
Title	Optional text across the top of the page.
Panel letters	A, B, C · a, b, c · (A), (B), (C) · (a), (b), (c) · none, in bold in each panel's upper left corner; after Z come AA, AB, …
Letter size	In points (4–48; default 12).
Keep each plot's own title	Off leaves the letters to name the plots, as most journals ask (pictures kept as drawn keep theirs).

The **Panels** list shows each panel's letter and name, and:

- the **columns it spans** (a panel two columns wide fills a row of a two-column page; one that does not fit in what is left of a row starts the next row);
- ⬆ and ⬇ to move it, and ✕ to take it off the page;
- **Clear page** removes every panel.

The preview on the right shows the page as it will be saved, with its size.

Saving the page

Setting	What it does
Format	PNG, SVG, PDF or EPS.
Resolution	Pixels per inch of a PNG page (72–1200; default 300), recorded in the file; the pixel size is shown beside it. Vector formats have no resolution.
Font	**Sans serif** (Helvetica in PDF and EPS, Liberation Sans in PNG) or **Serif** (Times, Liberation Serif in PNG) for every panel's text, the letters and the title.
CMYK colour	PDF and EPS only: every colour is written as cyan, magenta, yellow and black. The conversion is the simple complement rule without a colour profile (K = 1 − max(R, G, B), C = (1 − R − K)/(1 − K), likewise M and Y), so black is pure K; a print shop may still adjust colours to its press. PNG and SVG hold RGB colour only.
Transparent background	Leaves the page clear instead of white, for slides and posters with a coloured background (PNG, SVG, PDF). EPS has no transparency: its page is left unpainted, and translucent marks are blended with white.

Export page as … asks where to save the file. Text in PDF and EPS is set in the standard Helvetica or Times without embedding, as in single plots (see Saving plots and results).

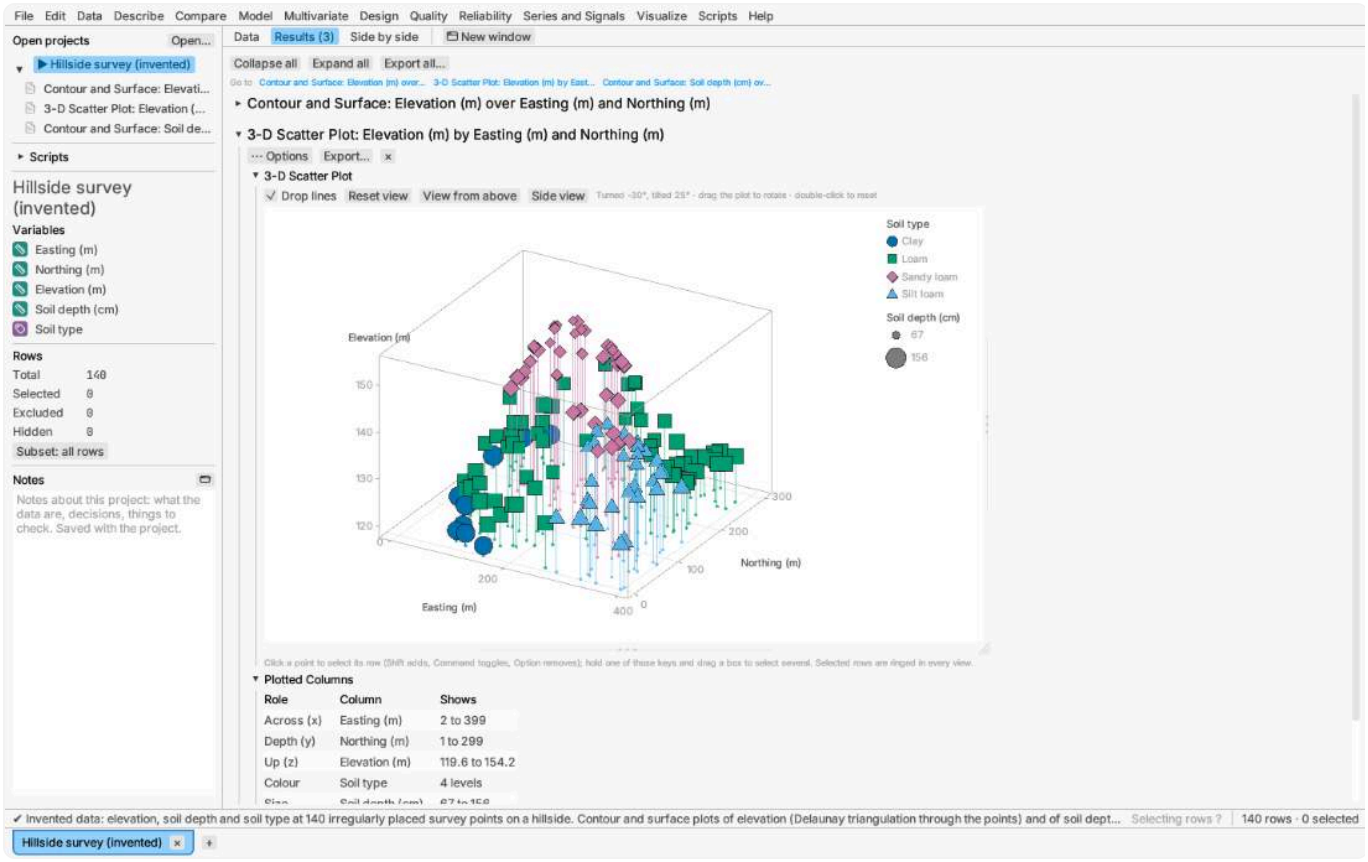
3-D scatter, contour and surface plots

Two entries on the **Visualize** menu show a response over two other columns in three dimensions. The *Hillside survey* sample opens with both plots already made.

3-D scatter plot

Visualize ▸ 3-D scatter plot… places each row in a box by three numeric columns: **Across (x)**, **Depth (y)** and **Up (z)**. Three optional columns add more information to each marker:

- **Colour:** a continuous numeric column is shown on a colour ramp with a key bar; any other column gets one colour *and* one marker shape per level, so the groups stay distinct without colour.
- **Size:** marker area grows in proportion to a numeric column; the key shows the smallest and largest values.
- **Shape:** one marker shape per level of a column (used when the colour column does not already set the shapes).



Turning and selecting:

- **Drag** inside the plot to turn it; **double-click** or **Reset view** returns to the starting angle. **View from above** looks straight down the up axis, and **Side view** looks along the depth axis.
- **Shift-, ⌘- or ⌥-drag** draws a selection box, and a **click** selects the nearest point; the modifiers combine selections as in the other plots. The selection is shared with the data grid and every other plot.
- **Drop lines** join each point to the floor, which helps judge where it sits in depth.
- Excluded rows are drawn hollow; hidden rows are left out.
- The plot opens at most about 760 points wide; drag the handles under it and along its right edge to resize it, and double-click either to restore its size (see Plots). The 3-D surface below resizes the same way.

The report also lists the plotted columns with their ranges, and how many rows were left out for missing values.

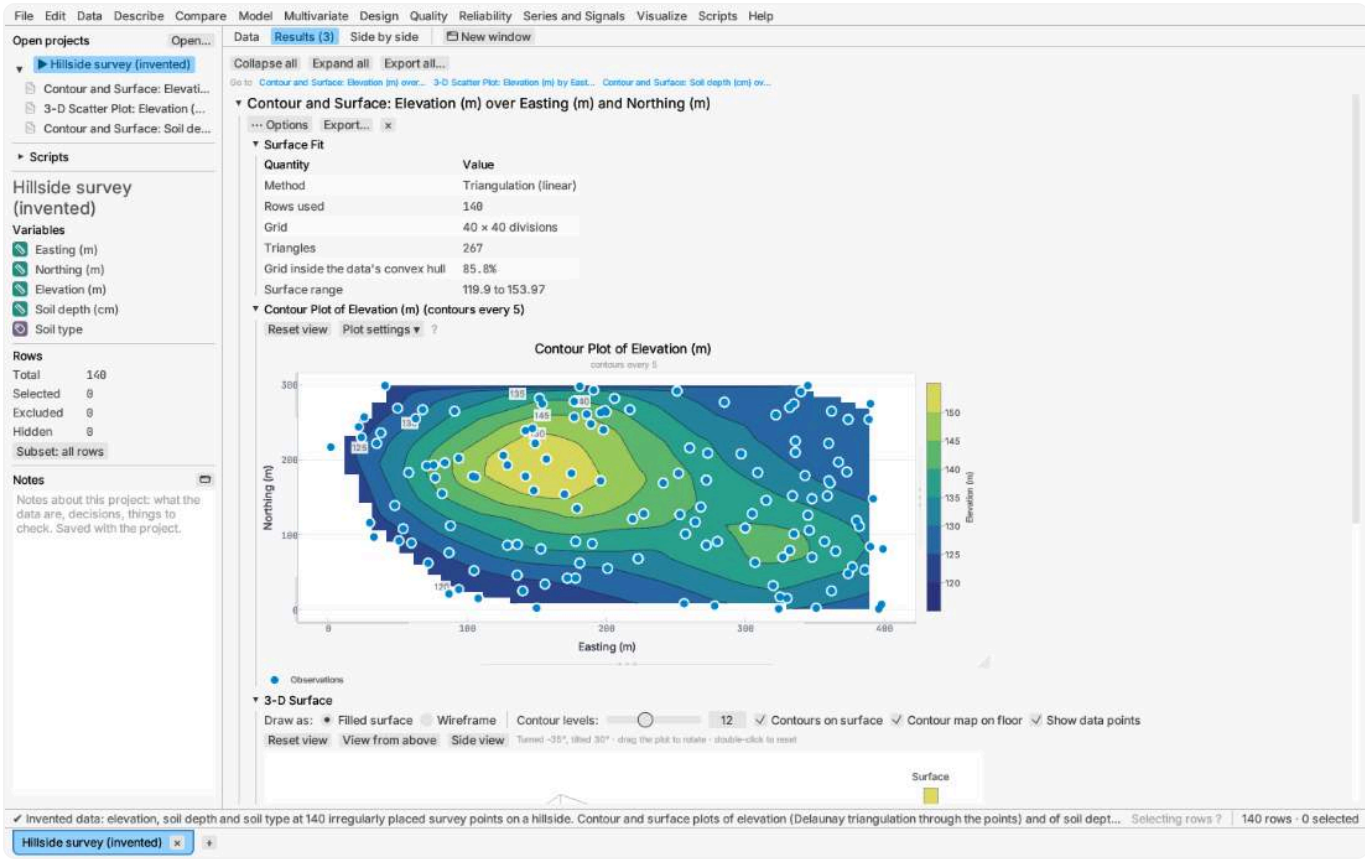
Contour and surface plot

Visualize ▸ Contour and surface plot… makes a surface of a **Response** column over **Across (x)** and **Depth (y)** columns, then draws it as a contour plot and as a 3-D surface you can turn. Choose how the surface is made:

- **Triangulation (linear):** joins the observations into Delaunay triangles and makes each triangle flat. The surface passes through every observation and stops at the edge of the data (the convex hull), so grid cells outside the data are left blank.
- **Thin-plate spline (interpolating):** the smoothest surface, in the sense of least bending, that passes through every observation (Wahba 1990). It extends to the corners of the plot.
- **Thin-plate spline (smoothing):** trades closeness to the observations for smoothness, with the balance chosen by generalized cross-validation. Use it for noisy measurements. The fit table reports the smoothing parameter λ, the effective degrees of freedom, the GCV score and the residual standard deviation.

Rows that share the same x and y are averaged before the surface is made. The smoothing spline is then fitted to those averages, so when rows were averaged its residual sum of squares and residual standard deviation are labelled *(distinct locations)*: they measure the scatter of the location averages about the surface on n − df degrees of freedom (n distinct locations, df the effective degrees of freedom), on the same footing as λ and the GCV score,

not the scatter of single rows. Under **More options**, **Grid divisions** (8 to 120, default 40) sets how finely the surface is drawn. The spline methods use at most 1,000 distinct locations; use triangulation for more.



The **contour plot** shows the surface as a filled contour map with a colour bar, its contour lines with their levels, and the observations as points you can select; its Plot settings choose the colours, lines and levels (see Contour maps). The 3-D surface, its floor map and its colour bar use the same colour scale. The **3-D surface** has its own toolbar: **Filled surface** or **Wireframe**, the number of contour levels, contours on the surface, a contour map on the floor, and the data points. It turns and selects in the same way as the 3-D scatter plot. Hovering over the surface shows the x, y and surface value under the pointer. The section menu (⋯ or right-click on the report title) switches the contour plot and the 3-D surface on or off and changes the method.

Saving the plots

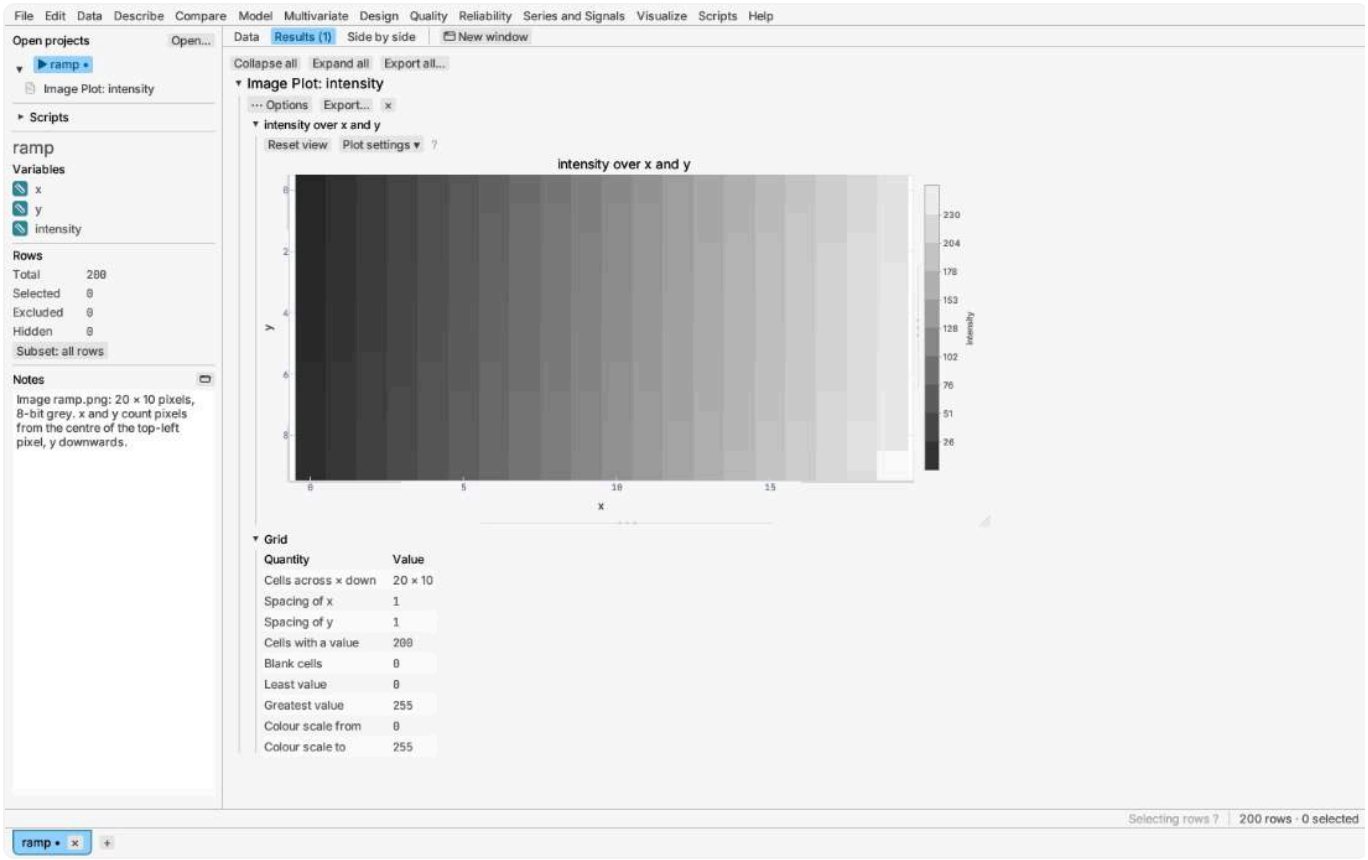
Right-click either 3-D plot and choose **Save plot as** PNG, SVG, PDF or EPS. The file is drawn from the plot itself at the current angle, with the same markers, key and axes as on screen (see Saving plots and results).

Scripts

Both plots run from scripts through `run_report` with the analysis keys `"scatter-3d"` and `"data-surface"`; see Scripting and recording.

Image plots

Visualize ▸ Image plot… draws a value column as a picture: each row is a cell placed by its **Across (x)** and **Down (y)** columns on an evenly spaced grid, coloured by its **Value**. Tables opened from image files are laid out this way (see Images as tables), and so are results computed on a grid, such as a field of measurements taken at regular steps. The dialog fills in `x`, `y` and `intensity` (or `luminance`) when the table has them.



Cells are drawn as they are, with no smoothing between them, so the picture shows every pixel exactly. The x and y columns must place the rows on one grid: the spacing on each axis is the smallest gap between distinct values, and every value must fall on it. Two rows in the same cell stop the plot with a message naming them; cells no row fills are left blank.

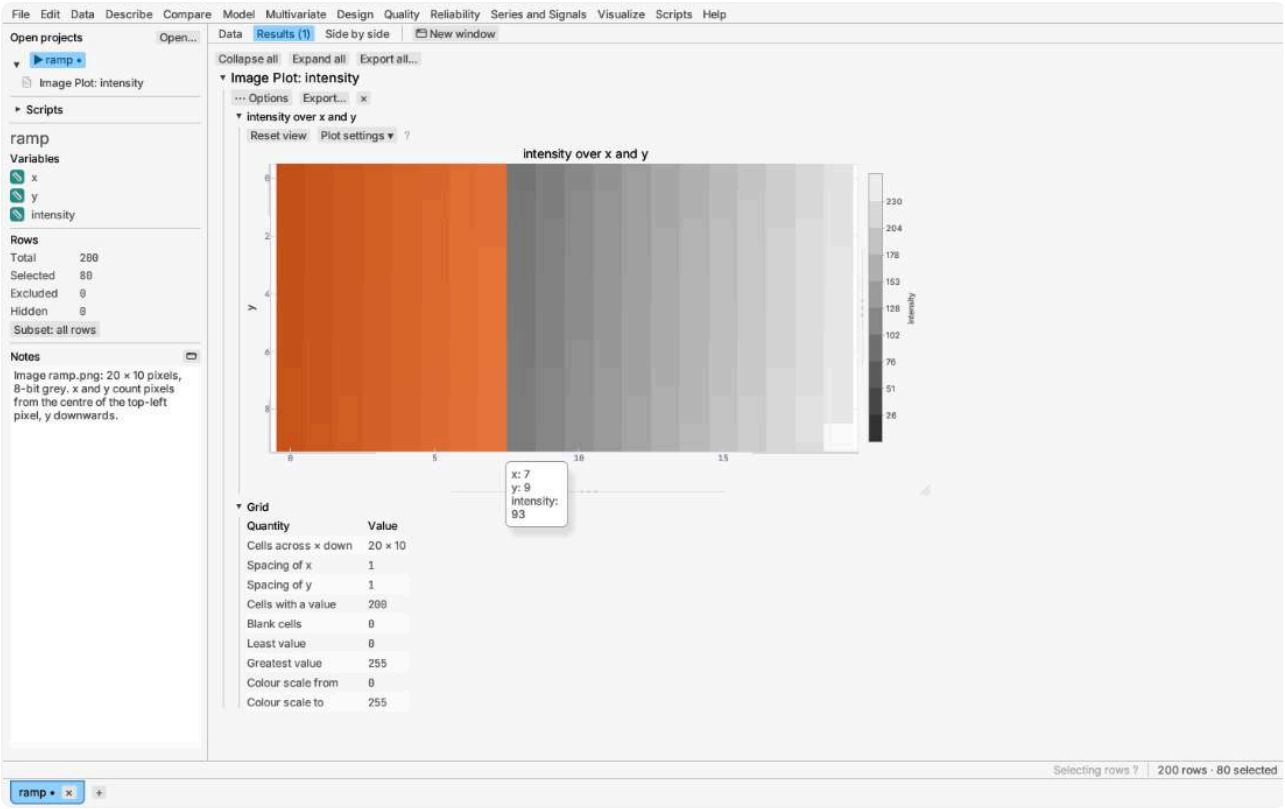
Settings

The dialog and the report's **Plot settings** hold the same choices:

- **Colours**: sequential (blue to yellow), heat, spectrum, diverging or grey (for print), shown in the colour bar beside the plot. The plot starts in grey with **Reverse the colours** ticked, so low values are black and high ones white, as in the image file; untick it to run any scale the other way.
- **Fixed range**: colour between two values instead of the data range, for example to compare several pictures on one scale. Values outside are drawn in the end colours.
- **Vertical axis**: **downwards** (as image files store their rows, the first row at the top; the default) or **upwards**.
- **Equal scale**: one unit is the same length on both axes, so cells are square (on by default).

Reading and selecting

- Point at a cell to see its position and value.
- **Drag a box** to select the rows of every cell inside it; **click** a cell to select its row. The modifier keys add, remove or intersect as in other plots, and the selection is shared with the data grid and every other view. Selected cells are tinted in the selection colour.



- Excluded rows are left blank.
- The axes drag and reset like any other plot's (see Working with plots).

The **Grid** table under the plot gives the grid's size, the cell spacing on each axis, how many cells are filled and the value range.

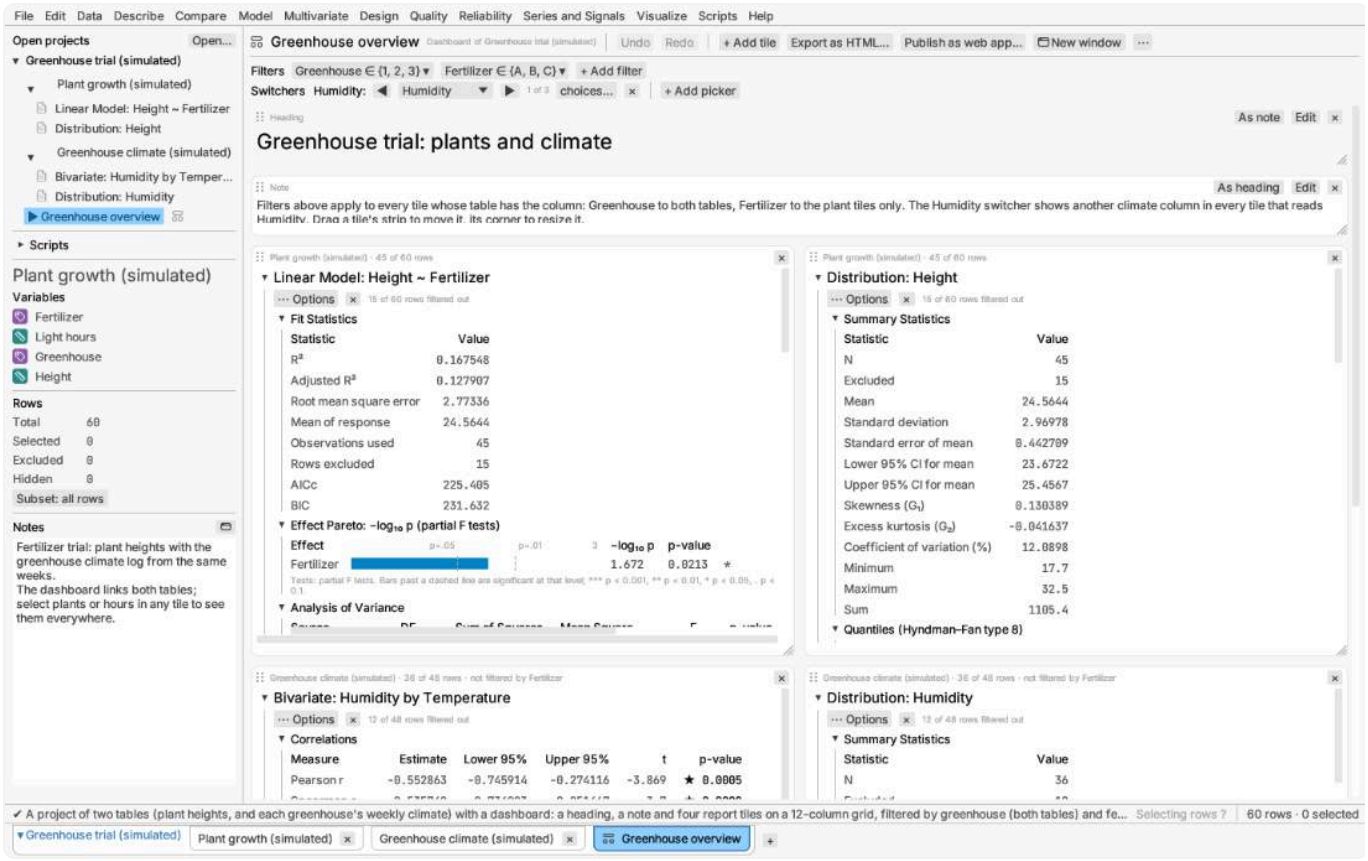
Saving

Save plot as writes PNG, SVG, PDF or EPS. The picture goes into the file as a bitmap, pixel for pixel: PNG data inside SVG, an image object in PDF (with transparency for blank cells) and an inline image in EPS, with the axes, titles and colour bar drawn as text and lines around it. Image plots can also go on a figure page. Scripts run the plot as `image-plot` (see Scripting).

Dashboards

A dashboard arranges reports and notes from a project's tables on one page, with filters that narrow every report on it at once. Use one to put the results you check most often side by side, or to show a summary to someone else, either in the application or exported as a web page.

To try one, open **File ▸ Open sample data ▸ Projects ▸ Greenhouse trial (simulated)**. The project has two tables, plant heights and each greenhouse's weekly climate, and opens on its dashboard, "Greenhouse overview".



Where dashboards live

A dashboard belongs to a project and uses only that project's tables. It is listed in the sidebar under the project's tables (with a small grid symbol), and has a tab after the project's tables' tabs along the bottom of the window. Click either to show the dashboard in the main window; click a table to go back to it, or press ⌘W / Ctrl+W while the

dashboard is shown. Right-click a dashboard's entry or tab to show it, open it in a window of its own, export it as HTML or remove it from the project.

Removing a dashboard asks first (**Keep** is the default, so Enter or Escape leaves it alone). After **Remove**, **Edit ▸ Undo remove dashboard** puts it back where it was until you edit data again; save the project to make the removal permanent. Closing a table that is the last one some dashboard uses also asks, naming the dashboards that would be left empty.

To make one, choose **File ▸ New dashboard** or **New dashboard** in the project's right-click menu, or use **Add to dashboard ▸ New dashboard** in a report's ⋯ Options menu, which starts the new dashboard with a copy of that report. New dashboards are named "Dashboard", "Dashboard 2" and so on; rename one from the ⋯ menu at the end of its toolbar.

Dashboards are saved in the project file, with their tiles, filters, column pickers and name, which dashboard was shown and whether it was in a window of its own. Save the project to keep them.

Tiles

The page is a grid twelve columns wide. Each item on it is a **tile**:

- **A report tile** holds a copy of a report bound to one of the project's tables. It is made by **Add to dashboard** in the report's ⋯ Options menu (or the menu you get by right-clicking its title), or by **+ Add tile ▸ From "table"** in the dashboard's toolbar, which lists that table's reports and offers an empty graph canvas. The copy is the tile's own: changing its display options (sections shown, plot settings) changes the tile, not the report it came from, and the other way round. A new report tile is six columns wide and six rows tall, placed in the first free space.
- **A heading** is a line of large text across the dashboard; **a note** is a paragraph. Add them from **+ Add tile**. Double-click the text (or choose **Edit** in the tile's strip) to change it, then **Done**; **As heading** and **As note** switch between the two.

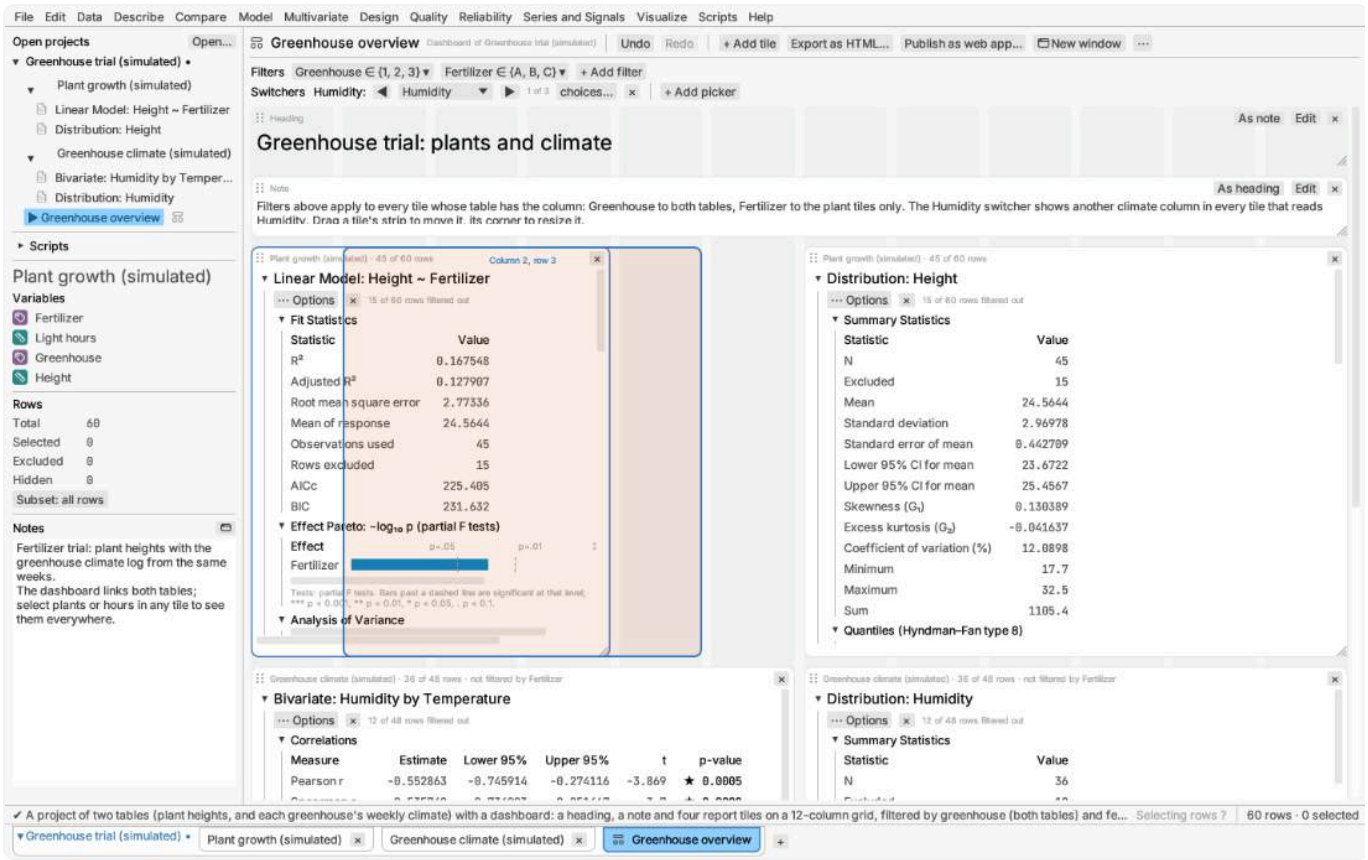
Each tile has a strip along its top. For a report tile it names the table, how many of its rows the filters keep, and which filters do not apply to it. Reports that are taller than their tile scroll inside it.

- **Move** a tile by dragging its strip (start on the dotted handle at its left end). Faint columns show the grid while you drag, and an outline shows where the tile will land;

"Overlaps another tile" in red means it would cover another tile, and it stays where it was.

- **Resize** a tile by dragging the grip in its lower right corner. It snaps to whole columns and rows.
- **Remove** a tile with the × at the right end of its strip, or **Remove report** in the tile report's ⋯ Options menu.

Every change to a dashboard — adding, moving, resizing and removing tiles, filters, column pickers, text, display options and its name — can be undone. Use **Undo** and **Redo** in the dashboard's toolbar, ⌘Z / Ctrl+Z and ⌘⇧Z / Ctrl+Y while the dashboard is shown, or **Edit ▸ Undo** and **Redo**, which act on the dashboard while it is the last thing you used. Edits to the tables' data are undone from the table, as before.



Filters

The strip under the toolbar holds the dashboard's filters. **+ Add filter** lists every column of the project's tables. A filter on a categorical column keeps the levels you tick (with **all** and **none**); a filter on a continuous column keeps a range between two sliders. Click a filter to

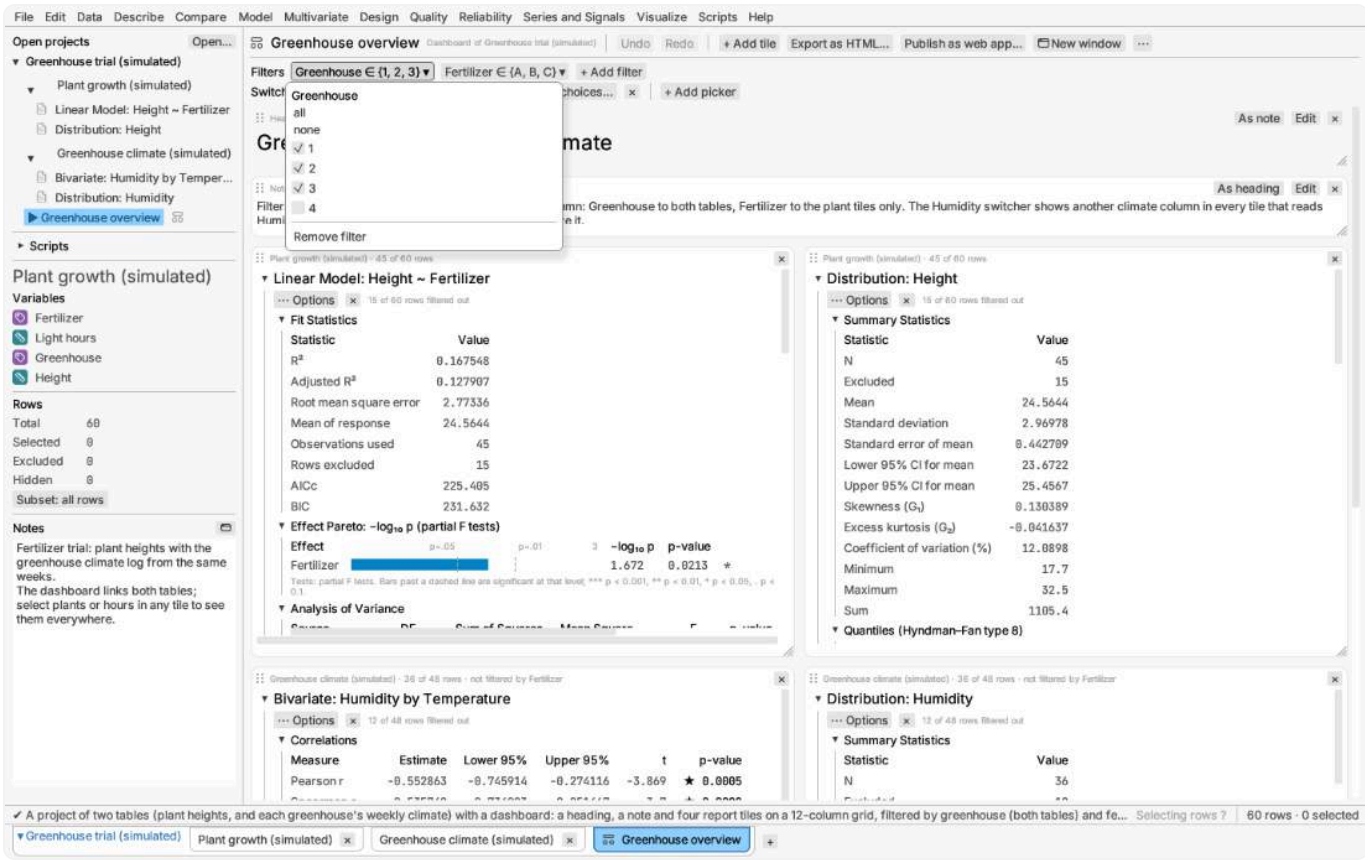
change it or to remove it. A change applies when you let go of the mouse button, so dragging a slider is one edit that Undo puts back in one step.

Filters are matched to tables by column name:

- a filter applies to every tile whose table has a column of that name;
- a tile whose table has no such column is not filtered by it, and its strip says so ("not filtered by Fertilizer"; point at it for the reason).

In the sample, the Greenhouse filter applies to both tables and the Fertilizer filter only to the plant tiles.

Filters never change the tables themselves. Each tile runs its report on a copy of its table's rows with the rows outside the filters left out, so the data grid, the table's own reports and its row counts are as they were. A tile's report says how many rows the filters leave out ("15 of 60 rows filtered out"), apart from any rows the table itself excludes, which it still calls excluded.



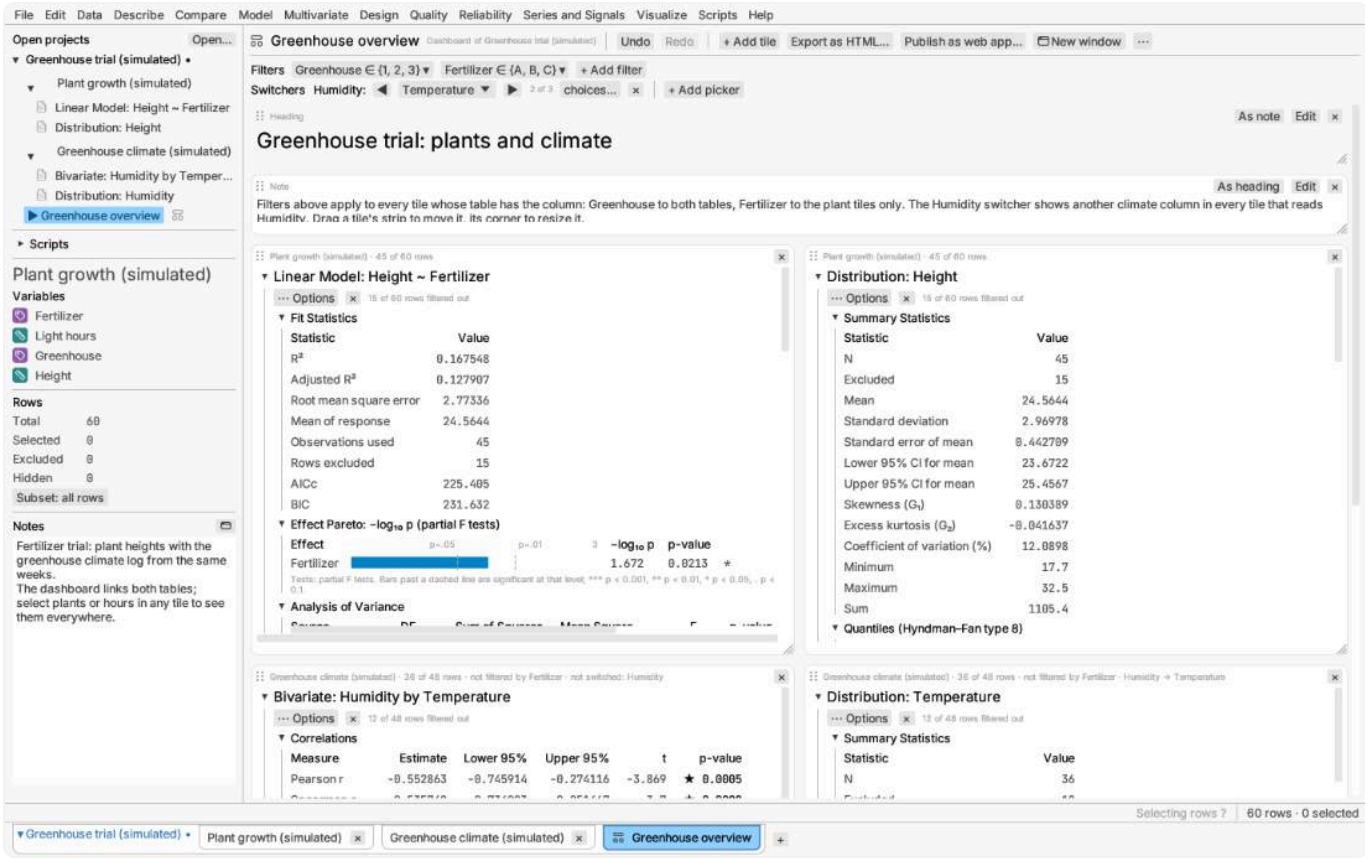
Column pickers

The strip under the filters holds the dashboard's **column pickers**. A picker names a column and a list of columns to show instead of it; every report tile that uses that column shows the chosen column in its place, in whatever role the report uses it (a model's response, a distribution's column, a graph's axis or colour). Use one to step a whole dashboard through several measurements without editing each tile.

The sample has a picker on Humidity offering Temperature and Week. Click ▶ to show the next column in the list, ◀ for the previous one, or pick one from the list between them; "2 of 3" says where you are.

- **+ Add picker** lists every column of the project's tables. A new picker offers the project's other columns of the same type (numbers, text or dates) and shows its own column, so nothing changes until you pick another. **choices…** ticks the columns it offers.
- Pickers are matched to tiles by column name, like filters. A tile that does not use the picker's column is not affected. A tile that uses it but whose table has no column of the chosen name, or that already uses the chosen column in another role, is left as it is, and its strip says "not switched: Humidity". In the sample, choosing Temperature switches the humidity distribution but not the humidity-by-temperature plot, which already uses Temperature; choosing Week switches both.
- A switched tile's strip names the switch, for example "Humidity → Week".
- The tiles keep their own reports: choosing the picker's own column again shows every tile as it was. A display option you change in a switched tile is kept when you switch back.
- **×** removes the picker, and every tile shows its own column again. Each pick is one edit, so Undo steps back through them.

Pickers are separate from the column picker a single report can have (its ⋯ Options ▸ Column picker): that one changes only its own report.



Selecting rows

Selecting points in a tile's plot selects those rows in the tile's table, so they are highlighted in its data grid, its own reports and every other tile of that table. Tiles of other tables are not affected: brushing works within one table. Rows the filters leave out are not drawn, so they cannot be selected from the dashboard. Using a tile also makes its table the one the menus, dialogs and the sidebar act on.

A window of its own

New window in the dashboard's toolbar (or **Open in new window** in its right-click menu) shows the dashboard in a window of its own, titled "Greenhouse overview · Dashboard · AnovaCraft"; the main window goes back to the project's tables. Undo, Redo and the other shortcuts work in that window on the dashboard. **Back to main window**, the window's close button or ⌘W / Ctrl+W in it shows the dashboard in the main window again. Clicking

the dashboard in the sidebar or its tab brings its window to the front. The window and its place on screen are saved with the project.

Exporting as a web page

Export as HTML… in the dashboard's toolbar (also **File ▸ Export dashboard as HTML…** and the dashboard's right-click menu) saves the whole dashboard as one self-contained web page:

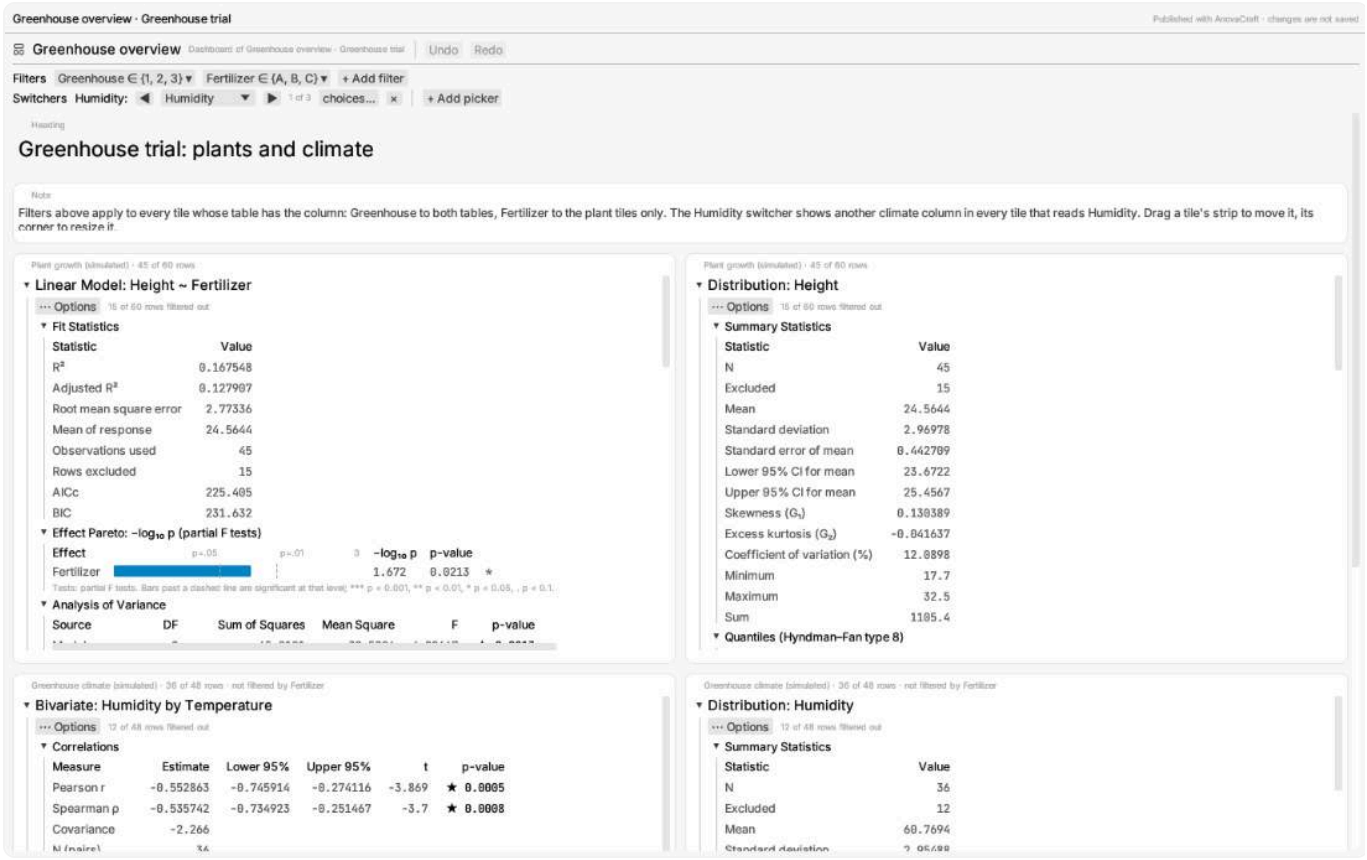
- the dashboard's name, its project, its filters, the columns its pickers show in place of others (for example "columns switched: Humidity → Week") and the export date at the top;
- the tiles on the same twelve-column grid, at the same places and spans (each row grows to fit its tiles); on a narrow screen they are stacked in reading order;
- in each report tile, its title, its table and rows kept, and the sections open in the tile, with tables as HTML tables and plots as SVG drawings, as in a report's own export;
- headings and notes as text.

Publishing as a web app

Publish as web app… in the dashboard's toolbar writes a folder that shows the project's dashboards in a web browser, still interactive. Choose an empty folder (or one published before, which is replaced); it gets:

- `index.html`, the page, titled with the dashboard and project names;
- `project.acraft`, the project with this dashboard first (its other dashboards follow; window layout and scripts are left out, and row selections cleared);
- `anovacraft_viewer.js` and `anovacraft_viewer_bg.wasm`, the viewer;
- `THIRD-PARTY-NOTICES.txt`, the licences of the open-source software the viewer is built with (the page links to it).

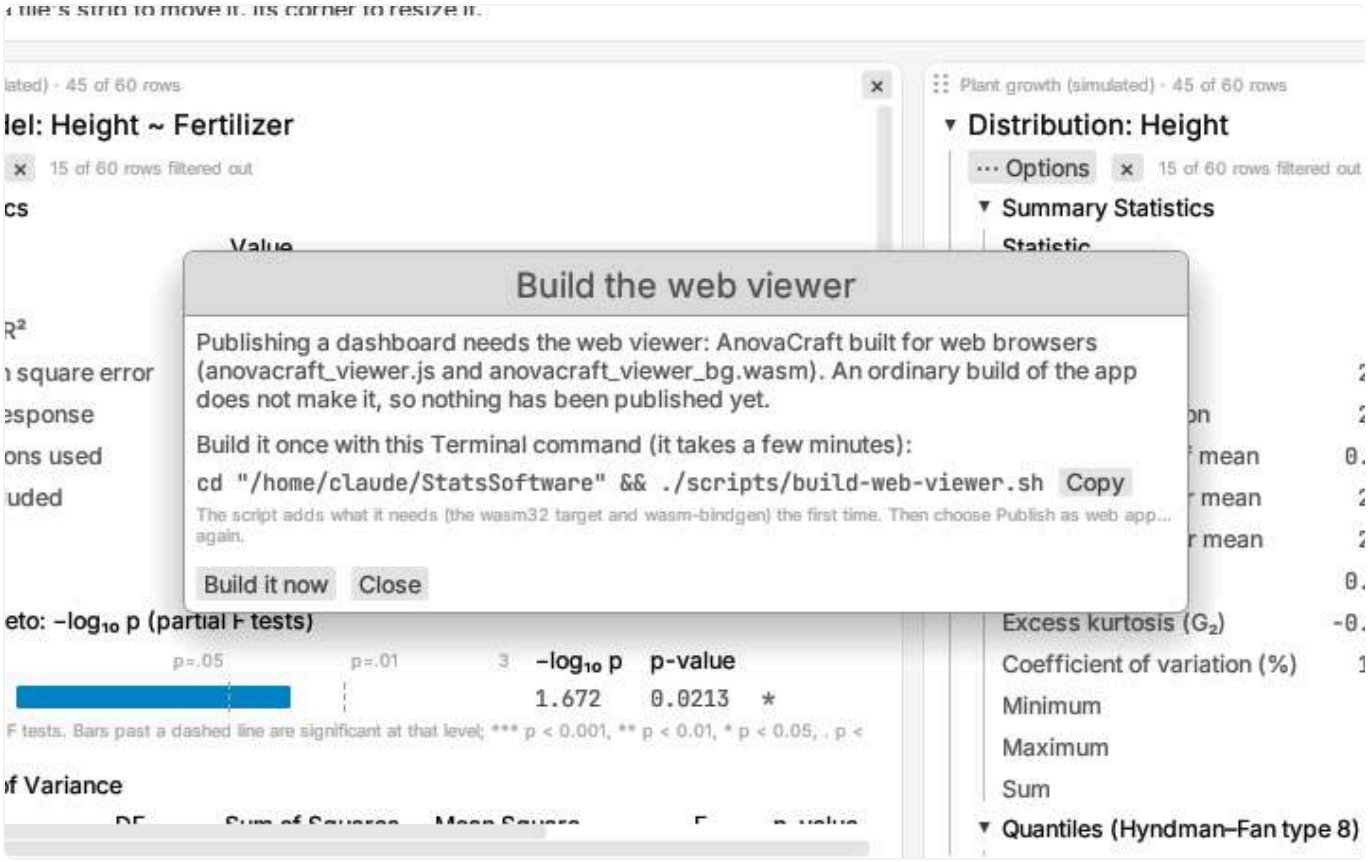
Put the folder on any web server (a static host such as GitHub Pages or an intranet share) and open its address. Browsers do not run a page like this from a file on disk, so to try it on your own computer run `python3 -m http.server` in the folder and open http://localhost:8000.



In the browser the reports are computed again from the project's data, and the dashboard works as in the app, except that nothing can be saved:

- filters and column pickers change, with Undo and Redo, and every tile recomputes;
- selecting points in a tile selects those rows in the tile's table, and a tile's ⋯ Options change what it shows;
- several dashboards show as choices in the top bar;
- tiles cannot be added, moved, resized, removed or edited, and commands that would add columns, reports or tables say so instead;
- reloading the page starts again from the published project.

The viewer files come from a WebAssembly build of AnovaCraft, which an ordinary build of the app does not make. Until they are built, Publish as web app… writes nothing and opens a **Build the web viewer** window instead. It shows the Terminal command that builds them (`scripts/build-web-viewer.sh`, see Web build) with a **Copy** button. In a source checkout, **Build it now** runs that build in the background (a few minutes the first time, which also installs the wasm32 target and wasm-bindgen) and then asks for the folder to publish to.



To look at a project the way the viewer shows it without a browser, run the app as `app-gui --viewer project.acraft`.

Limits

- A column picker on one tile's graph canvas, or a report's own column picker, changes only that tile; use the dashboard's column pickers to change every tile at once.
- A tile's report runs on its filtered rows, so its own tables can still count the rows the filters leave out as "excluded" (for example a distribution's "Excluded" statistic); the note under the report's title gives the filtered-out count.
- **Edit analysis…** and **Open in new window** are not offered in a tile's ⋯ Options menu: edit the table's report and add it again.
- Closing a table that has tiles on a dashboard takes those tiles off the dashboard and clears the dashboard's undo history.

Describe

The Describe menu summarises one column, the relationship between two, or several categorical columns at once in a multi-way table. It also lists Summarize by group…, which makes a table of summary statistics for each group.

Distribution

Describe ▸ Distribution… summarises one column.

Report: summary statistics: N, mean, standard deviation, moments, skewness, excess kurtosis, coefficient of variation and a confidence interval for the mean.

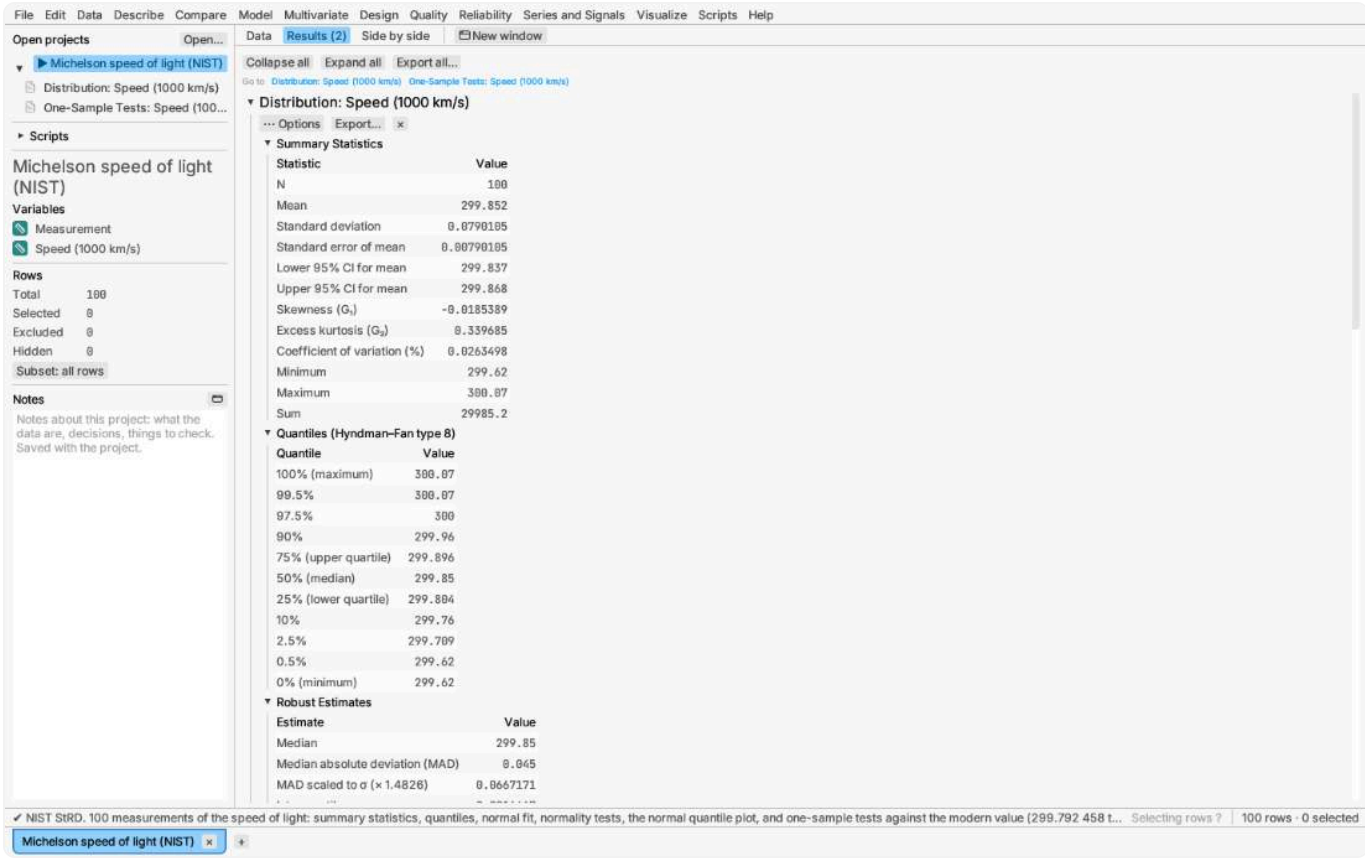
Options (⋯ Options menu):

- **Quantiles** by any of the nine Hyndman–Fan definitions (types 1–9).
- **Robust estimates:** median, MAD, IQR, trimmed and winsorized means.
- **Histogram** and **box plot**.
- **Normal fit** with Shapiro–Wilk and Anderson–Darling tests, and a **normal quantile plot**.
- **Box–Cox transformation:** the estimated power, with an option to save the transformed column.
- **Tolerance interval:** limits that contain a stated proportion of the population (90% by default) with stated confidence (95%). Choose two-sided limits or one limit only, and the normal-theory interval (mean ± k × SD, with the exact factor k) or the distribution-free interval from order statistics. The distribution-free table gives the confidence actually achieved, with a note when the sample is too small to reach the one asked for. The limits are drawn on the histogram.
- **Outlier tests:** Grubbs' test or Dixon's ratio test for one outlier (at either end, the largest value or the smallest), or Rosner's generalized ESD test for up to a given number of outliers (a tenth of the values by default). Set the significance level, and for Dixon the ratio (chosen by sample size unless you pick r10, r11, r21 or r22). The table gives the value and row tested, the statistic, critical value and p-value; the generalized ESD adds a table of its steps. **Select flagged rows** in the options menu selects the outliers in every view. These tests assume the other values are normal.
- **Runs tests for randomness:** runs above and below the median (or a value you give) and runs up and down, in row order, with the expected number of runs and p-values

for too few runs (clustering, trend) and too many (alternation). Exact p-values are used where they can be computed.
- For categorical columns, a **frequency table** and chart, with the levels in natural order (numbers by value, then text), so the cumulative column follows an ordinal scale.

Date columns report locations as dates and spreads as lengths of time.

Try it: Michelson light speed (tolerance interval); Clinic wait times for skewed data and Box–Cox; Rosner's measurements (generalized ESD outlier test); Beam deflection (runs tests).



Censored values and detection limits

Laboratory results below a detection limit ("non-detects", reported as "< 2 µg/L") and values known only to lie above a limit or between two limits are *censored*: dropping them, or replacing them by the limit or half of it, biases the mean and the spread (Helsel 2012). Under **Censored values (non-detects and detection limits)** in the Distribution dialog, choose:

- **A column flags censored values:** the analysed column holds each measured value, or for a censored one its limit; a numeric flag column is non-zero for censored rows.

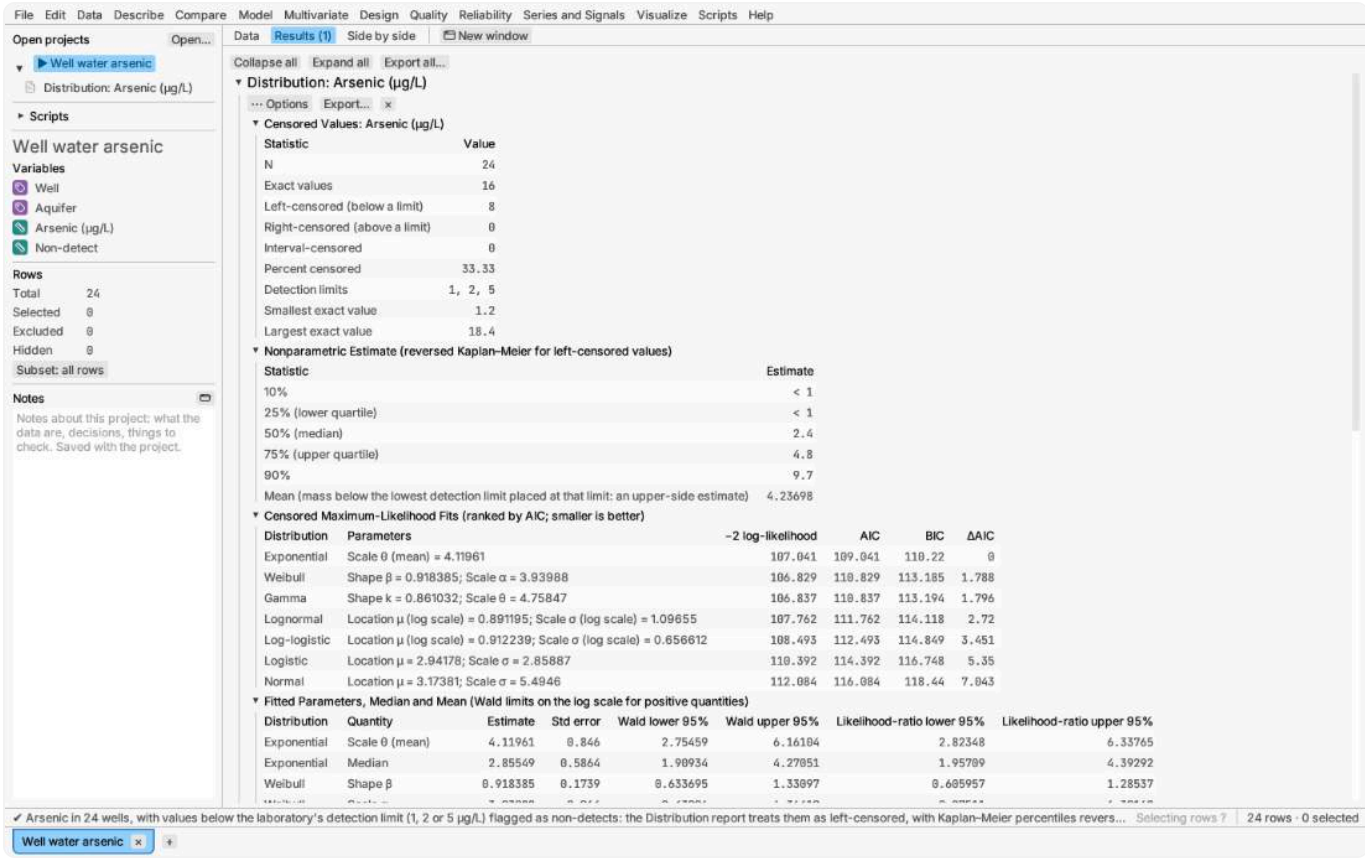
Censored values lie *below* the value (left-censored, the usual non-detect, and the default) or *above* it (right-censored).
- **Lower and upper ends in two columns:** the analysed column is the lower end and a second column the upper end of each value's interval. Equal ends are an exact value, an empty lower end a value below the upper end, an empty upper end a value above the lower end.

The flag column cannot also be analysed, and the analysed columns must be numeric. The report replaces the usual sections with:

- **Censored Values:** N, exact values, left-, right- and interval-censored values, and the detection limits that occur with how often.
- **Nonparametric estimate:** the distribution function without a model. With left censoring only it is the Kaplan–Meier estimate on the flipped scale (Helsel 2012, ch. 6), and with any other mix Turnbull's (1976) self-consistent estimate, which reduces to it. A non-detect "< d" is a value strictly below d, so a value detected at d is not counted with it. The table gives percentiles (5, 10, 25, 50, 75, 90, 95%) — shown as "< 1" when the estimate only says the percentile is below the lowest limit — and the mean, restricted to the observed range when the estimate leaves mass below the lowest limit.
- **Fits by maximum likelihood**, ranked by AIC: normal, lognormal, Weibull, gamma, exponential, logistic and log-logistic. Each censored value contributes the probability of its interval, F(d) for a non-detect.
- **Parameter estimates** of each fit with standard errors, and its median and mean with intervals.
- **Distribution function plot** (an option): the nonparametric estimate as points with the three best fits as curves.
- **Count fits:** when every value and limit is a whole number ≥ 0, Poisson and negative binomial (mean μ, size k, variance $\mu + \mu^2/k$) fits with likelihood-ratio limits; a non-detect "< d" counts as 0 … d − 1.

Verified: R `fitdistrplus::fitdistcens` (run to a tight tolerance) and `survival::survfit` on the flipped data; count fits against an independent SciPy implementation.

Try it: Well water arsenic (24 wells, nine non-detects at limits of 1, 2 and 5 μg/L).



Identify distribution

Describe ▸ Identify distribution… finds which distribution describes one continuous column. Use it before a capability study or a reliability calculation on data that are not normal, or simply to learn the shape of a process.

Dialog: choose the column and the distributions to try (all fourteen by default), whether to also try transformations to normality, and how many quantile plots to draw.

Distributions: normal, lognormal, exponential, Weibull, gamma, logistic, log-logistic, smallest extreme value and largest extreme value, plus a **threshold** form of the lognormal, exponential, Weibull, gamma and log-logistic. A threshold is a lower bound below which no value can fall (a seal that cannot be weaker than 20 N); the threshold forms also fit data with zero or negative values. Forms without a threshold need every value to be positive, and are listed as "not fitted" otherwise; a **Not Fitted** note under the table says why for each one.

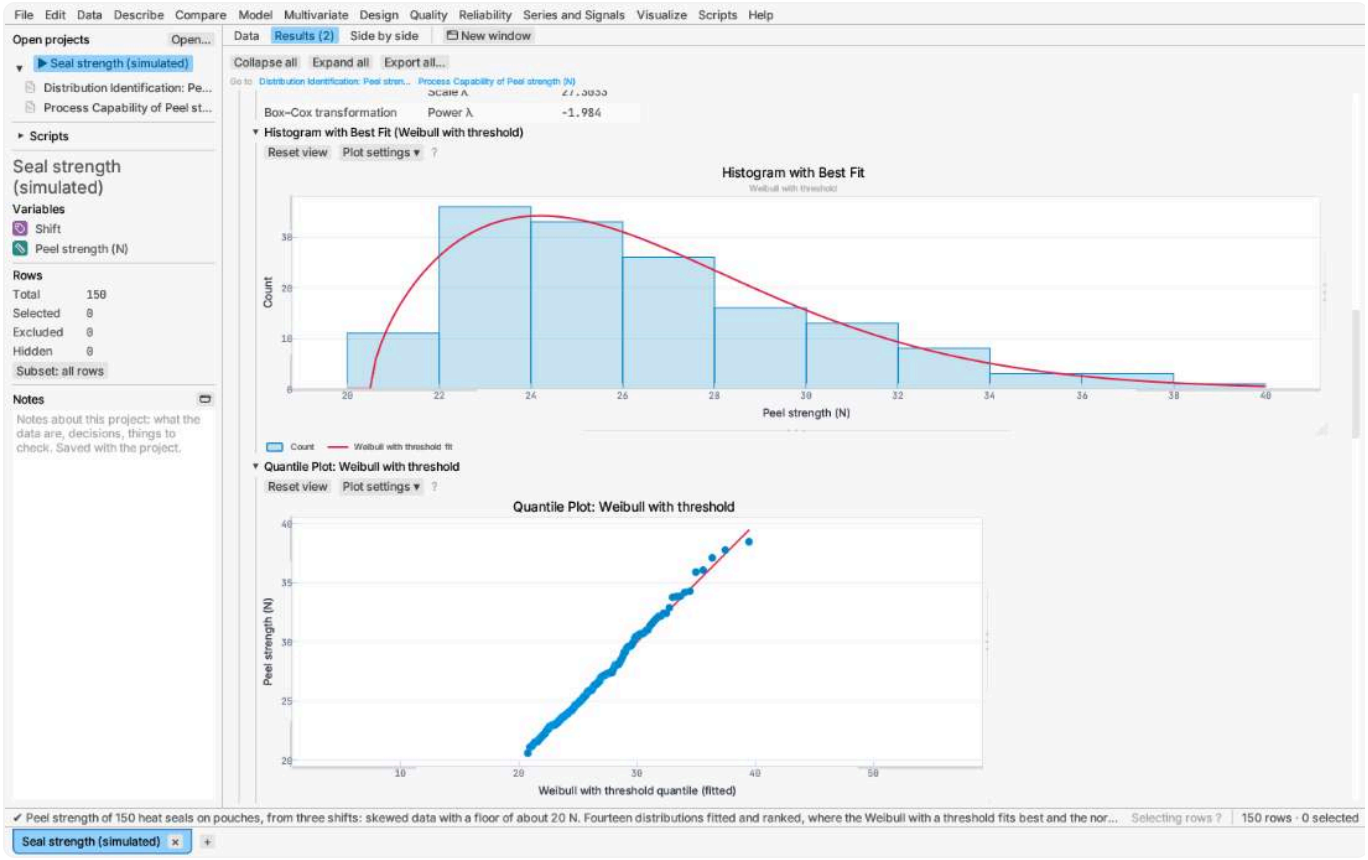
Report:

- **Goodness of Fit:** one row per distribution, best first by AIC (Akaike's information criterion; smaller is better, and a difference of more than about 2 is worth noticing). Each row gives the number of parameters, the Anderson–Darling statistic A^2 and its p-value (a small p-value means the distribution does not fit), and for a threshold form the p-value of the likelihood-ratio test of the threshold (a small value means the threshold is needed). Transformation rows give the normality test of the transformed values.
- **Parameter Estimates** for each distribution, in its usual form (for example a Weibull's shape β and scale α).
- **Histogram with Best Fit:** the histogram with the best distribution's density drawn over it.
- **Quantile plots** for the best fits: each value against the fitted distribution's quantile at its rank. Points along the diagonal line mean a good fit; a bend at either end shows where the distribution misses the tails. Points can be selected and brushed like any plot.
- With transformations on, the **Johnson** transformation (one of the bounded SB, lognormal SL or unbounded SU forms, chosen from the data's percentiles) and the **Box–Cox** power transformation, each with a normal quantile plot of the transformed values.

The p-values for the normal come from a published formula. For every other distribution they come from simulation: samples drawn from the fitted distribution are refitted, and the p-value is the share whose A^2 is at least the data's. Reports repeat exactly. When no simulated sample fits as badly as the data, the p-value shows as "< 0.0005".

Only included rows with a value are used; excluding rows re-runs the fits.

Try it: Seal strength, where the Weibull with a threshold near 20 N ranks first and the normal is rejected.



Bivariate

Describe ▸ Bivariate… looks at two columns. What you get depends on their measurement levels:

Columns	Report
continuous × continuous	Pearson and Spearman correlations with tests and 95% intervals, covariance, scatter plot
categorical × categorical	contingency table with margins and expected counts, Pearson and likelihood-ratio χ^2 tests, Cramér's V and φ (2×2) with 95% intervals from the noncentral χ^2 distribution, a warning when expected counts are small
continuous × categorical	means by group with intervals, one-way ANOVA, pooled and Welch t (two groups) or Welch ANOVA, tests of equal variances, rank tests, pairwise comparisons of three or more groups, a values-by-group plot

With three or more groups the report compares every pair of groups with 95% intervals: Tukey–Kramer by default, or Games–Howell (no equal-variance assumption), Holm-adjusted t, Bonferroni, Šidák, Holm–Šidák, Fisher's LSD, Newman–Keuls, Tamhane's T2, Dunnett's T3, or none, chosen from the report's options menu under **Pairwise comparisons**.

Contingency tables (two categorical columns) also show:

- **Adjusted residuals**: each cell's (observed − expected) divided by its standard error under independence (Agresti 2013, §2.4.5), approximately standard normal. Cells beyond ±1.96 carry ▲ (more than independence predicts) or ▼ (fewer) and are drawn in stronger ink, so the direction never rests on colour alone.
- **Differences of proportions** between every pair of row groups (up to 10 groups): for each column level (only the second when there are two), the difference of the proportions with Newcombe's hybrid-score 95% interval (Newcombe 1998, method 10), which stays inside [−1, 1] and behaves well with small counts, and a forest plot of the differences.

Try it: Customer survey (two categorical columns).

Multi-way table

Describe ▸ Multi-way table… lays out several categorical columns at once. The dialog's lists run, left to right: **Continuous columns** (optional measures summarized in each cell), then the two classifications. Tick the **Rows** columns and the **Columns** columns in nesting order (outermost first; the number after a ticked name is its place): with Centre then Treatment as rows, each centre's rows are split by treatment. Only combinations that occur in the data get a row or column. Each cell shows any of:

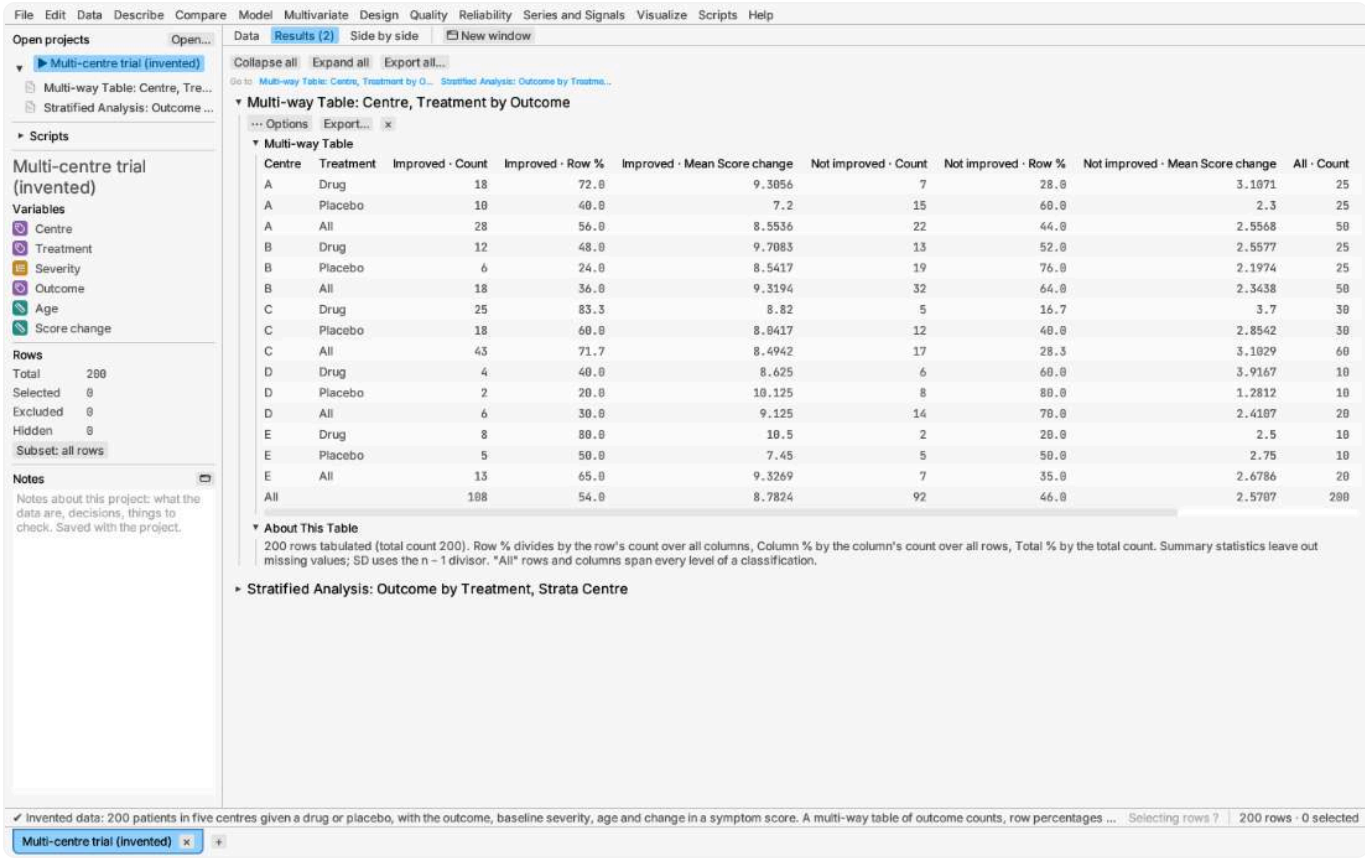
- **Count**, **Row %** (of the row's count over all columns), **Column %** (of the column's count over all rows) and **Total %** (of all rows);
- for each ticked **Continuous column**: N, Mean, SD (n − 1 divisor), Median, Min, Max and Sum, leaving out missing values.

Totals adds an *All* row and column; **Subtotals of outer levels** adds, after each outer level's rows (and columns), a row spanning all its inner levels. A **Frequency column** makes each row count as many times as its value (summary statistics then weight each value the same way). Rows with an empty classification are left out and counted in the note under the

table. With column classifications, each heading names the column group and the statistic, such as *Improved · Count*; with none, the statistic alone.

The statistics and totals can be changed afterwards from the report's ⋯ Options menu. Right-click the table to copy it or save it as a new data table (see Saving plots and results).

Try it: Multi-centre trial (outcome by centre and treatment, with row percentages and the mean score change).



Multiple response

Describe ▸ Multiple response… counts the answers to a check-all-that-apply question ("Which of these do you use? Tick all that apply"), where each person can pick any number of items. The answers can be kept two ways:

- **One column of lists**: a text column whose cells list the items picked, separated by a delimiter (`;` unless you type another; leave it empty to treat each cell as one item). Spaces around items are ignored and an item listed twice in one cell counts once. An empty cell is a person who picked nothing.

- **One column per item**: a column for each item, where a non-zero number, *yes*, *true*, *x* or ✓ means picked (anything else, or missing, means not). The column names are the items.

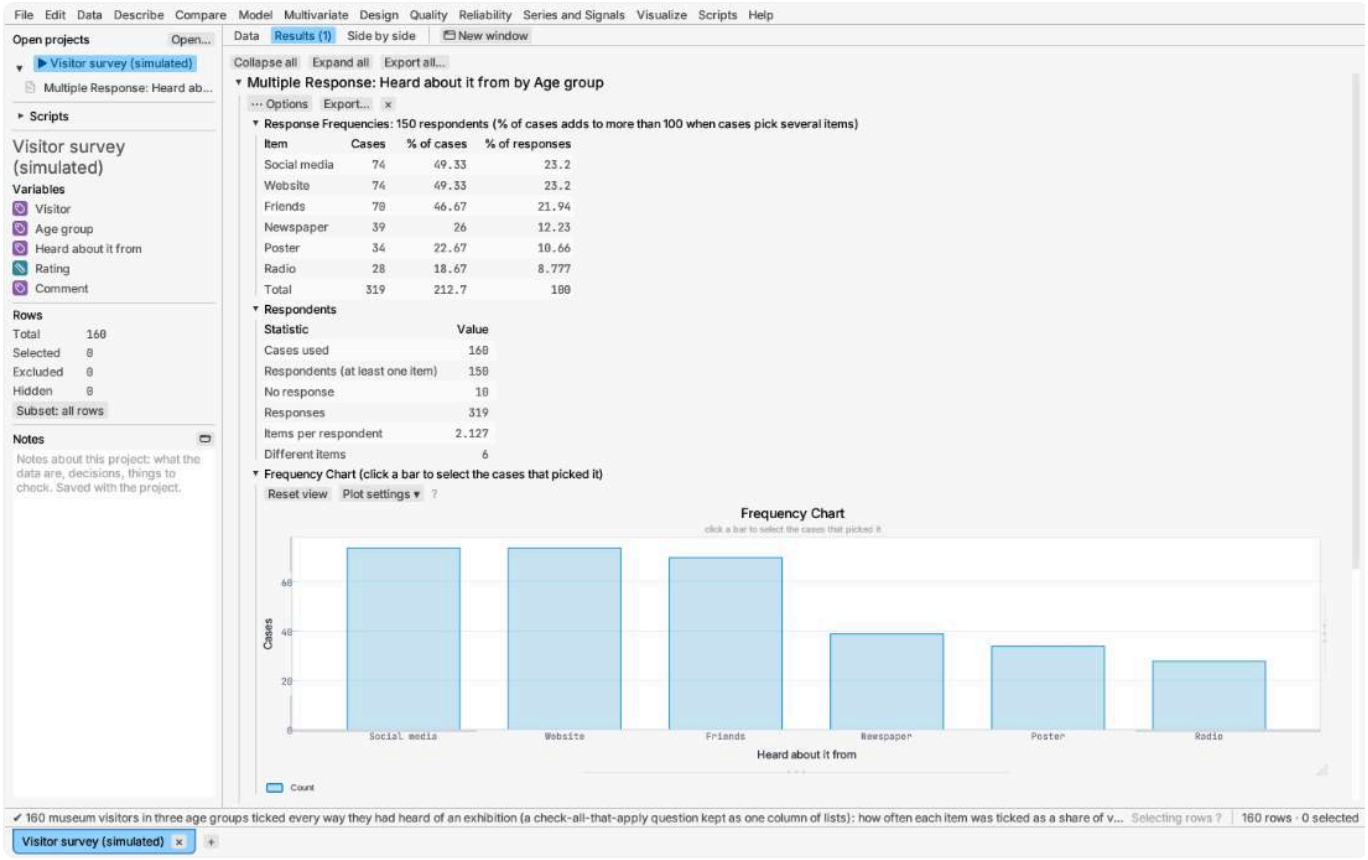
Cross-tab by (optional) names a group column, such as age group or region. **Order items** lists them most picked first, or in the order they were found (first appearance in the lists, or column order).

The report shows:

- **Response Frequencies**: each item's count of cases that picked it, as a percentage of cases (respondents who picked at least one item; these percentages add to more than 100, because people pick several) and of responses (all picks; these add to 100).
- **Respondents**: cases used, respondents, cases with no answer, total responses, items per respondent and the number of different items.
- **Frequency Chart**: a bar per item. Click a bar to select the cases that picked that item, in the grid and every plot.
- With a group column, **Cross-tabulation**: for each group, how many of its respondents picked each item and what percentage that is; and **Tests of Marginal Independence**: for each item a 2 × G Pearson χ^2 test of whether the share picking it differs between groups, with Bonferroni-adjusted p-values, and the overall Bonferroni test (the smallest p-value times the number of items, at most 1). Because one person answers for several items, the ordinary χ^2 test of the whole items-by-groups table is not valid; testing each item and adjusting is the simple valid alternative (Agresti & Liu 1999; Bilder & Loughin 2004). Items picked by everyone or by no one cannot be tested and say so; a note marks tests with expected counts below 5.

To filter on one item, make a formula column or a row filter with `has_item(Heard, "Radio")` (true when the list holds the item), or count the items with `item_count(Heard)`; see Formula columns.

Try it: Visitor survey (how 160 museum visitors heard about an exhibition, by age group).



Area under a curve

Describe ▸ Area under a curve… takes an x column (time, dose…), a response column and optionally a column that splits the rows into one curve per level. The curve joins the mean response at each distinct x with straight lines (rows with the same x are replicates).

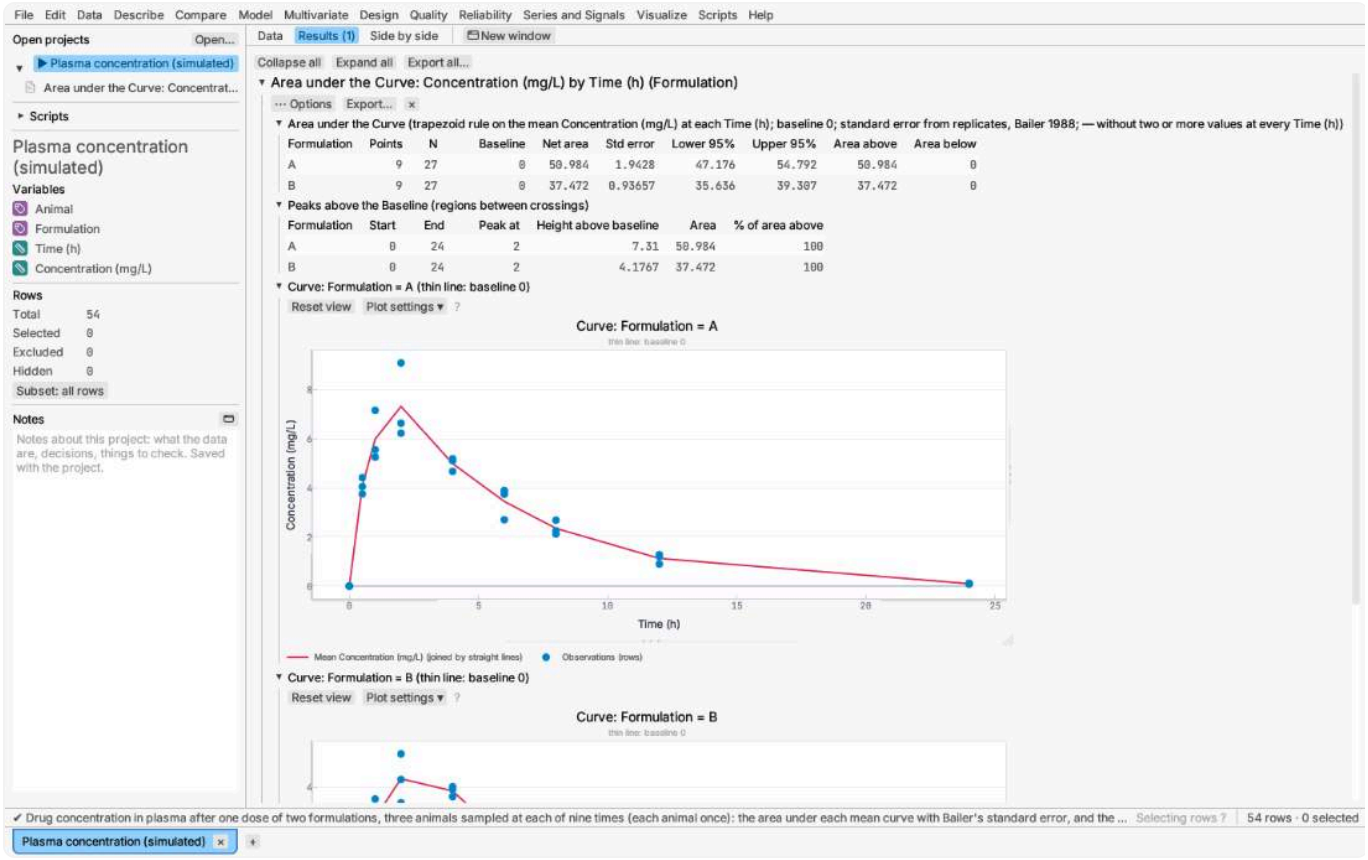
- **Area under the Curve:** for each curve, the number of x values and of rows, the baseline, the **net area** (trapezoid rule: area above the baseline minus area below it), its standard error and a normal confidence interval, and the areas above and below the baseline separately. The standard error (Bailer 1988) comes from the variance of the replicates at each x, so it needs two or more values at every x and shows — otherwise; it suits destructive sampling, where each subject is measured once.
- **Baseline:** a value (0 by default), or the mean of the curve's first few points (for example the readings before a dose); a baseline from the data adds its own uncertainty to the standard error.
- **Peaks above the Baseline:** each stretch of the curve above the baseline (between crossings, found by straight-line interpolation), with where it starts and ends, the x

and height of its highest point, its area and its share of the area above. Peaks lower than a chosen share of the tallest can be left out.
- **Curve:** the observations, the mean curve and the baseline, one plot per curve (the first 12).

Saving: the report's ⋯ Options menu saves the response minus its curve's baseline, as a percentage of its curve's range (0 at the smallest value, 100 at the largest) or as a fraction of its curve's total. The formula functions `subtract_baseline`, `percent_of_range` and `fraction_of_total` make the same transforms (see the formula reference).

Verified: hand calculation of the trapezoid areas, crossings and peaks; Bailer's standard error against R.

Try it: Plasma concentration (two formulations, three animals at each of nine times).



Pharmacokinetic parameters

Describe ▸ Pharmacokinetic parameters… summarises concentration–time profiles without fitting a compartmental model (non-compartmental analysis; Gibaldi & Perrier 1982, ch. 11;

Gabrielsson & Weiner 2016, ch. 2). Data are long: one row per sample with **subject**, **time** and **concentration** columns, and optionally a **treatment or period** column (one profile per subject and level, for crossover studies). Enter concentrations below the limit of quantification as 0. The **dose** is none (clearance and volume are then left out), one value for all, or a column (the first value in each profile). **Administration** is extravascular (oral, subcutaneous…; clearance and volume are then apparent, CL/F and Vz/F), an intravenous bolus, or an intravenous infusion of a given duration. Excluded rows and rows with a missing time or concentration are left out.

For each profile, with times $t_1 < \ldots < t_n$ and concentrations $C_1 \ldots C_n$ (a missing sample at time 0 counts as C = 0 after extravascular dosing or an infusion):

- **Cmax**, **Tmax** (first time of the peak), **Clast** and **Tlast** (last positive concentration).
- **AUC_0–t and $AUMC_0$–t** (to Tlast), by the **linear trapezoid** $\Delta AUC = \Delta t\,(C_1 + C_2)/2$, or **linear up / log down** (the default): where the concentration falls and both values are positive, $\Delta AUC = \Delta t\,(C_1 - C_2)/\ln(C_1/C_2)$, and for the first moment $\Delta AUMC = \Delta t\,(t_1C_1 - t_2C_2)/\ln(C_1/C_2) + \Delta t^2\,(C_1 - C_2)/\ln^2(C_1/C_2)$; elsewhere the linear rule ($\Delta AUMC = \Delta t\,(t_1C_1 + t_2C_2)/2$).
- **λz,** the terminal rate constant: minus the slope of ln C on t over the terminal points. *Best fit* (the default) tries the last 3, 4, … positive points after Cmax (Cmax excluded), drops fits with $\lambda z \le 0$, takes the largest adjusted R^2 and, among fits within 0.0001 of it, the one with most points. *Last k points* uses the last $k \ge 3$ positive values, *Times from … to …* the positive values in a time range (at least 3). No λz means the parameters below are shown as —.
- **t½** = $\ln 2/\lambda z$; **AUC_0–∞** = AUC_0–t + Clast/λz with the **% extrapolated**; **$AUMC_0$–∞** = $AUMC_0$–t + Clast·Tlast/λz + Clast/λz^2.
- **MRT** = $AUMC_0$–∞/AUC_0–∞ (minus half the duration for an infusion).
- With a dose, **CL** = Dose/AUC_0–∞ and **Vz** = Dose/(λz·AUC_0–∞).
- **Intravenous bolus:** C0 is the observed concentration at time 0 if positive; otherwise it is back-extrapolated log-linearly from the first two positive values when they decline, or set to the first positive value when they do not. It enters the areas but not Cmax or the terminal phase.

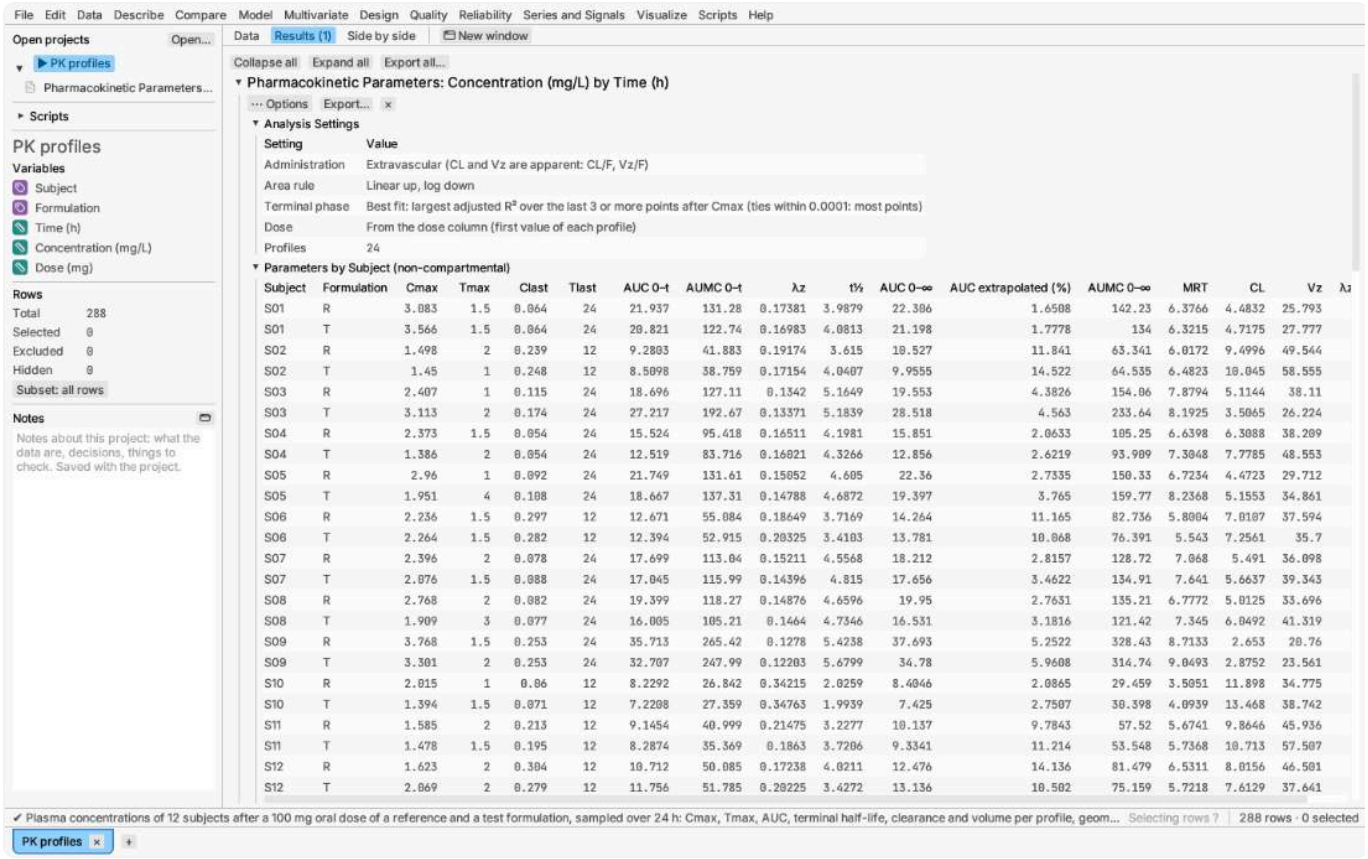
Report:

- *Analysis Settings* — route, area rule, terminal-phase rule, dose and number of profiles.
- *Parameters by Subject* — one row per profile (a profile that cannot be analysed, for example with no positive concentration, says why).
- *Summary* (by treatment) — for each parameter and treatment: N, the **geometric mean** and **geometric CV** = $\sqrt{\exp(s^2) - 1}$, where s is the standard deviation of the logs

(for concentrations, areas, rate, half-life, MRT, CL and Vz), the arithmetic mean, standard deviation, median, minimum and maximum.
- *Terminal Phase Regressions* — each profile's λz, intercept, points used, time range, R^2 and adjusted R^2.
- *Concentration–Time Profiles* and *Profiles on a Log Scale* — concentration against time per profile, the second on a log axis with each fitted terminal line drawn over its points.

Verified: an independent NumPy implementation written from the formulas (every parameter of the 24 sample profiles under both area rules, to 10^{-10}) and hand-worked closed forms.

Try it: PK profiles (12 subjects, reference and test formulations, a 100 mg oral dose).



Distribution calculator

Describe ▸ Distribution calculator… works with a distribution rather than data. Choose one of 22 distributions — normal, lognormal, exponential, gamma, Weibull (the last four with an optional threshold), logistic, log-logistic, smallest and largest extreme value, uniform,

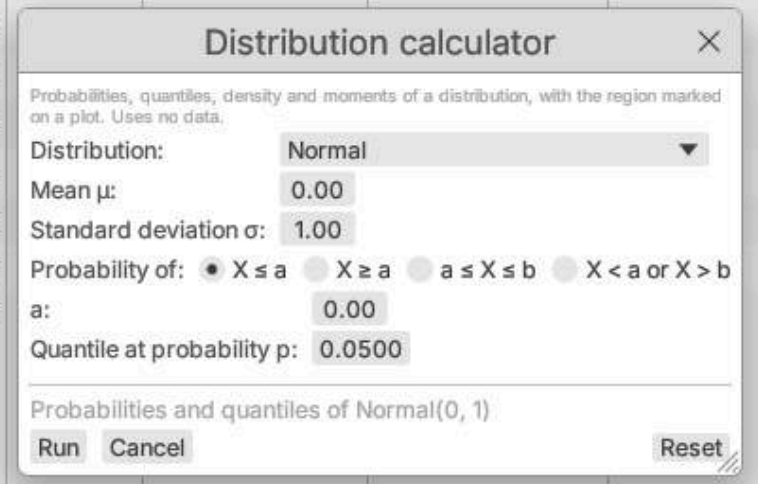
triangular, beta, t, χ^2, F, noncentral t, χ^2 and F, studentized range, binomial, Poisson and hypergeometric — and its parameters. Then choose a region: $X \le a$, $X \ge a$, $a \le X \le b$, or $X < a$ or $X > b$, and a probability p to invert.

The report gives the parameters with the mean, standard deviation and range of values; the probability of the region and of its complement, the density (or, for counts, the probability $P(X = x)$) at a and b, and the quantiles at p and $1 - p$; and a plot of the density with the region hatched. For the binomial, Poisson and hypergeometric the plot shows a stick per value, with a marker on each stick in the region, and the bounds are inclusive at whole numbers: $P(2 \le X \le 4)$ includes 2 and 4. The studentized range needs at least 0.05 degrees of freedom; below that its probability spreads too thinly to compute.

Extreme arguments give their limiting or exact values rather than a blank: a value so far out that its square would overflow (such as 10^{200} for the normal, t or F) gives probabilities of exactly 0 and 1, and upper-tail probabilities are computed directly, so a tail of 10^{-18} keeps its digits instead of rounding to 0 or to a few times 10^{-16}. Quantiles keep their relative precision in the tails too (the Cauchy, t with 1 degree of freedom, at $p = 10^{-20}$) and for very small degrees of freedom, where they can lie near 10^{-200} (χ^2 with 0.002 degrees of freedom) or 10^{68} (t with 0.01). A variance larger than the largest representable number (a Weibull shape below about 0.005) is shown as ∞.

Change the settings from the report's ⋯ Options menu and the report updates. To draw random values from the same distributions, use Data ▸ New random column....

63639	98.1	346999	1932	3594	113270	1952
64989	99	365385	1870	3547	115094	1953
63761	100	363112	3578	3350	116219	1954
66019	101.2	397469	2904	3048	117388	1955
67857	104.6				118734	1956
68169	108.4				120445	1957
66513	110.8				121950	1958
68655	112.6				123366	1959
69564	114.2				125368	1960
69331	115.7				127852	1961
70551	116.9				130081	1962



Compare

The Compare menu holds hypothesis tests and intervals for comparing a sample with a value, groups with each other, and paired measurements, goodness of fit for counts, crossover trials and stratified contingency tables, plus power and sample size, many two-group tests at once, and the adjustment of p-values for multiple testing. Tests are verified against published textbook examples, SciPy and R.

One sample vs a value

One-sample t and z tests, Wilcoxon signed-rank, the sign test, a test of a standard deviation, and an equivalence test (two one-sided tests, TOST).

Try it: Michelson light speed.

Two or more groups

Data can be stacked (a value column and a group column) or side by side (one column per group).

- **Tests of location:** pooled and Welch t, Wilcoxon rank-sum (Mann–Whitney), one-way and Welch ANOVA, Kruskal–Wallis.
- **Two-sample Kolmogorov–Smirnov** (two groups): the largest gap D between the two empirical distribution functions, sensitive to any difference in shape, not only location. The p-value is exact for samples whose sizes multiply to under 10 000 when there are no ties, and asymptotic otherwise (the row says which). A one-sided alternative asks whether the first group's values tend to be larger (or smaller).
- **Median test (Mood):** the counts above and not above the grand median in each group, with Pearson's χ^2 test and the smallest expected count (with a warning below 1). Choose how values equal to the median count: as not above (the default), as above, or left out.
- **Equal variances:** variance ratio, Levene, Brown–Forsythe, Bartlett.

- **Effect sizes:** Cohen's d, Hedges' g and Glass's Δ (the difference divided by the second group's standard deviation, for when one group is a control whose spread is the yardstick). For the one-way ANOVA, the **Effect Sizes** section gives η^2, ω^2 and Cohen's f, with confidence intervals for η^2 and f from the noncentral F distribution (Steiger 2004).

- **Linear trend** (three or more groups, ticked in the dialog or report options): a contrast with equally spaced coefficients across the groups in their order, with its estimate, t test and interval, the share of the between-group sum of squares it explains, and an F test of departure from a straight line. The groups' order is the column's value order.

- **Equivalence:** TOST.

- **Ratio of means** (two groups; **More options** or the report's options): the first group's mean divided by the second's, with Fieller's (1954) interval and a t test of ratio = 1, both with a pooled variance and with unequal variances (Welch–Satterthwaite degrees of freedom; Tamhane & Logan 2004). It suits relative effects such as relative potency or percent change. The interval is exact for normal data; when the second mean is not clearly different from zero the interval is unbounded (one or both limits infinite, or the complement of an interval), and the table says so.

- **All-pairs comparisons:** Tukey–Kramer (the default), Games–Howell, Holm-adjusted t, Bonferroni, Šidák, Holm–Šidák, Fisher's LSD (unadjusted; only after a significant overall test), Newman–Keuls (stepwise studentized range; p-values only, no intervals), and for unequal variances Tamhane's T2 (Welch t per pair with Šidák's adjustment) and Dunnett's T3 (Welch t against the studentized maximum modulus), or Dunn's rank test. The intervals use the dialog's confidence level, and the *Significant* column marks a pair when its adjusted p-value is below one minus that level ($p < 0.05$ at 95%, $p < 0.01$ at 99%).

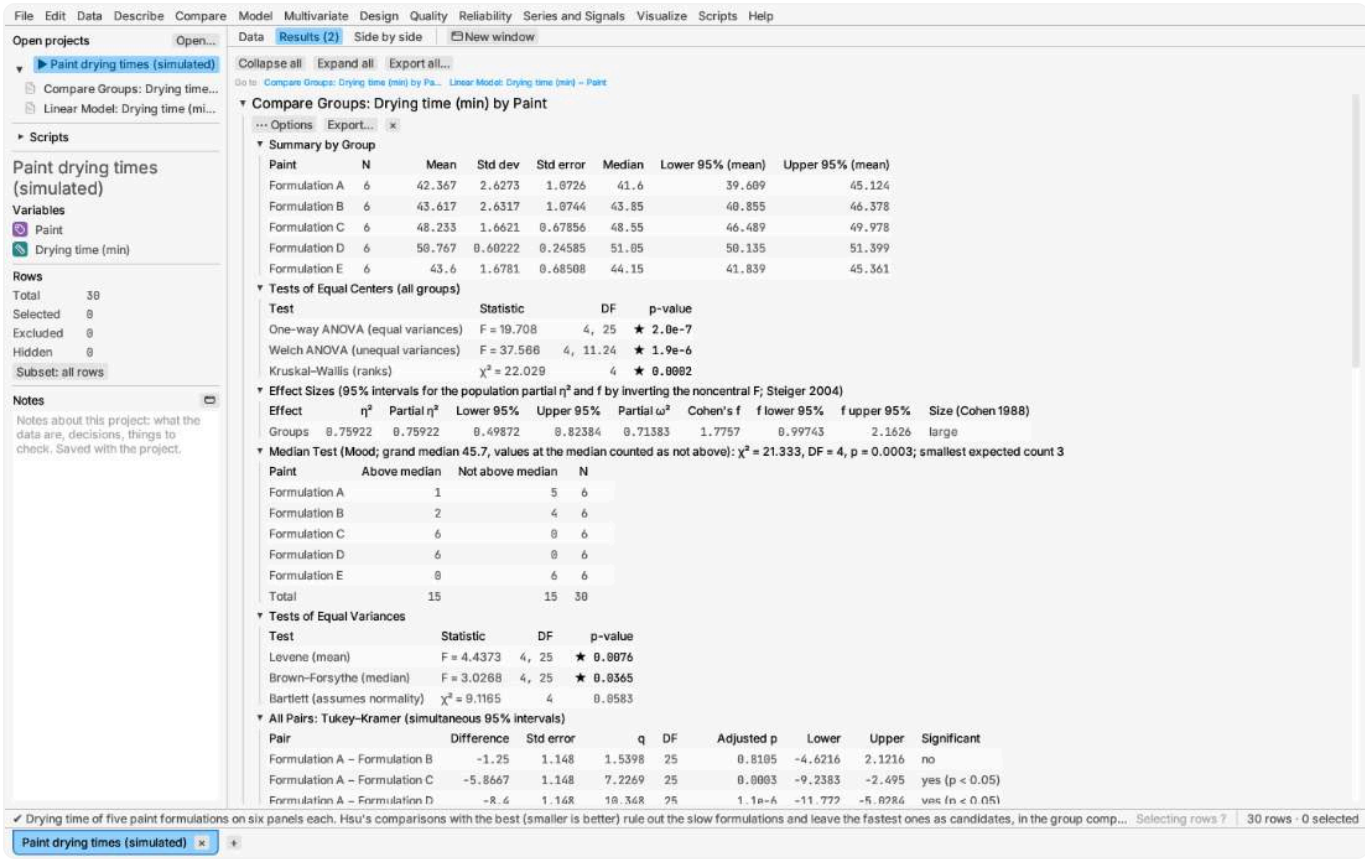
- **Comparisons with a control (Dunnett):** pick a control group in **Compare with a control** and every other group is compared with it: the difference, its standard error, t, the adjusted p-value and a simultaneous interval, with the critical value and error degrees of freedom in the heading, and an interval plot of the differences with a line at zero. The report's alternative (two-sided, above or below the control) and confidence level apply. Up to 100 groups can be compared with a control; with more (a grouping column with hundreds of levels) the section says so instead of stalling the report.

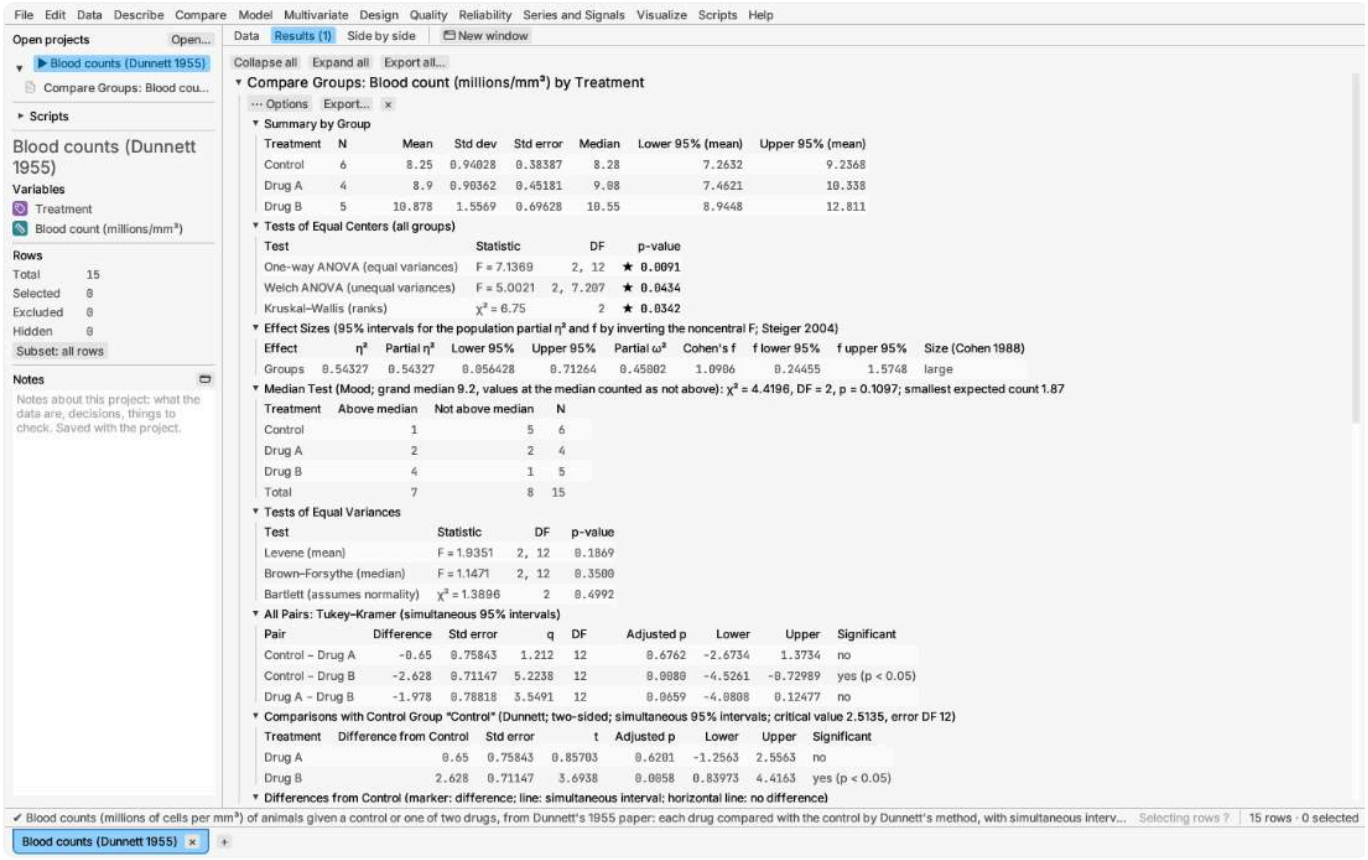
- **Comparisons with the best (Hsu):** choose whether larger or smaller is better. Each group is compared with the best of the *other* groups: the table gives the group's mean, its difference from the best other mean, the critical value (Dunnett's, with that

group as the control) and a constrained simultaneous interval, which always includes zero at one end. A group whose interval lies at or above zero is **the best**; one whose interval lies at or below zero is **not the best**; when the interval crosses zero the group **may be the best**. The interval plot shows each difference with its interval and a line at zero. The report's confidence level applies.

The settings, the control group and the direction of "best" included, can be changed afterwards from the report's ⋯ Options menu.

Try it: Clinic wait times; Silicon resistivity (median test); Blood counts (comparisons with a control); Paint drying times (comparisons with the best, smaller is better).





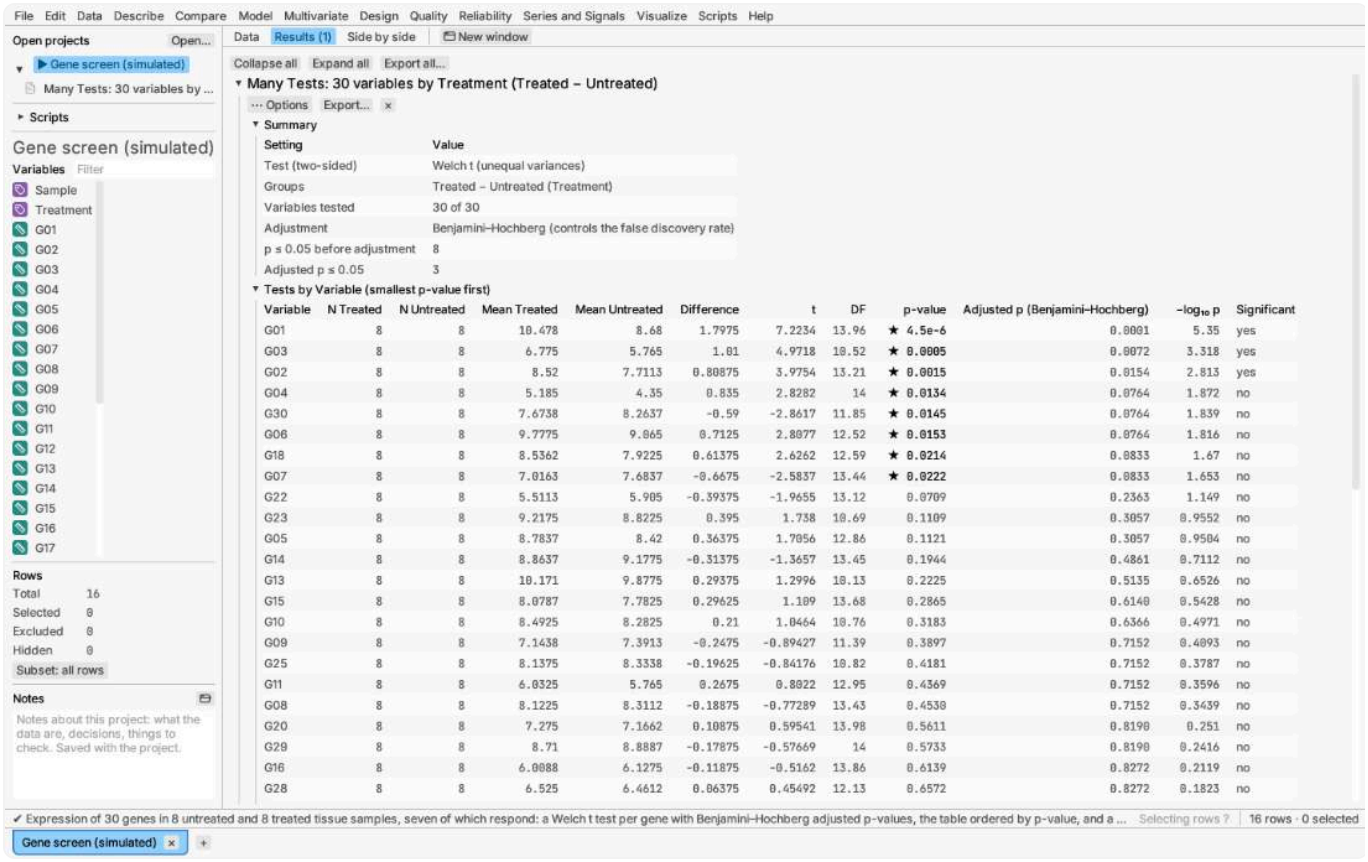
Two groups, many variables

Compare ▸ Two groups, many variables… runs the same test of two groups on every chosen column: put the measured columns in the left list and the grouping column (exactly two values) on the right. Choose the test — Welch t (the default), pooled t, or the Wilcoxon rank-sum test — and the adjustment for multiple testing (Benjamini–Hochberg by default; any method of Adjust and combine p-values).

- **Summary:** the groups, how many variables were tested, and how many have p ≤ α before and after adjustment.
- **Tests by Variable:** one row per column with the group sizes and means, the difference of means (or the Hodges–Lehmann shift for the rank-sum test), the statistic, degrees of freedom, p-value, adjusted p-value, $-\log_{10} p$ and whether the adjusted p-value is at most α. Order the table by column order, smallest p-value first or largest difference first (dialog, or the report's ⋯ Options menu). A column that cannot be tested (fewer than two values in a group) shows why instead.
- **Volcano plot:** the difference against $-\log_{10} p$, one point per column, with a line at p = α and at zero difference. Columns significant after adjustment have their own colour

and marker and are named (the 25 smallest p-values).

Try it: Gene screen (30 simulated genes, 8 untreated and 8 treated samples; seven genes really change).



Paired or repeated measures

Paired t, signed-rank test with the Hodges–Lehmann estimate, sign test; repeated-measures ANOVA and the Friedman test for three or more measurements per subject.

With three or more measurements the repeated-measures table also checks **sphericity** (equal variances of all differences between conditions), which the F test assumes: **Mauchly's test** gives W, χ^2, its degrees of freedom and p-value, and the **Greenhouse–Geisser** and **Huynh–Feldt** rows repeat the F test with both degrees of freedom multiplied by the ε estimate (1 when sphericity holds, down to 1/(k − 1)). Huynh–Feldt ε uses Lecoutre's correction; an estimate above 1 is reported and the test uses 1. A small Mauchly p-value means read a corrected row; Greenhouse–Geisser is the more cautious.

With two conditions, **Equivalence test** adds two one-sided tests that the mean difference lies between the bounds you give. Tick **Test the ratio of geometric means** for positive data such as concentrations: the test is run on logs and the estimate, bounds (for example 0.80 and 1.25) and interval are reported as ratios first / second.

Try it: Sleep drug trial.

Factorial repeated measures

Compare ▸ Factorial repeated measures… analyses designs in which each subject is measured under every combination of one to three **within-subject** factors (task × cue, time × dose), optionally with **between-subject** factors that split the subjects into groups (age group, treatment): two- and three-way within, between and mixed designs, four factors at most in all.

The data are stacked: one row per subject and within-subject cell, with a response column, a subject column and one column per factor. Several rows for the same subject and cell are averaged; subjects missing a cell are left out and counted. A between-subject factor must be constant within each subject. Tick a column in **Within-subject factors** or **Between-subject factors** (a column ticked in one list leaves the other). Tick **Simple effects** and choose a factor and the factor at whose levels to test it.

The report shows:

- **Design:** each factor, its type and levels, and how many complete subjects were used.
- **Cell means:** subjects, mean, standard deviation and standard error of each cell, each subject's cell value counting once.
- **Univariate tests:** every effect and interaction with its own error term (between-subject effects against subjects within groups, each within-subject effect against its interaction with subjects), the F test, the Greenhouse–Geisser and Huynh–Feldt ε and corrected p-values for within-subject effects with three or more levels, and partial and generalized η^2 (Olejnik & Algina, 2003). Unbalanced between-subject groups use Type III (effect-coded) sums of squares.
- **Sphericity:** Mauchly's test and both ε for each within-subject effect with more than one degree of freedom (an effect with two levels always meets sphericity).
- **Simple effects:** the test of the chosen factor within the data of each level of the other, each analysed separately with its own error term (unadjusted p-values).
- **Profile plot:** cell means joined across the first within-subject factor, one line per combination of the other factors, over the measurements.

The sums of squares, ε estimates and tests are computed from orthonormal contrasts of each subject's cell values (the multivariate form of the univariate tests, O'Brien & Kaiser 1985) and match R's `car::Anova` with Type III hypotheses.

Try it: Reaction times by task and cue.

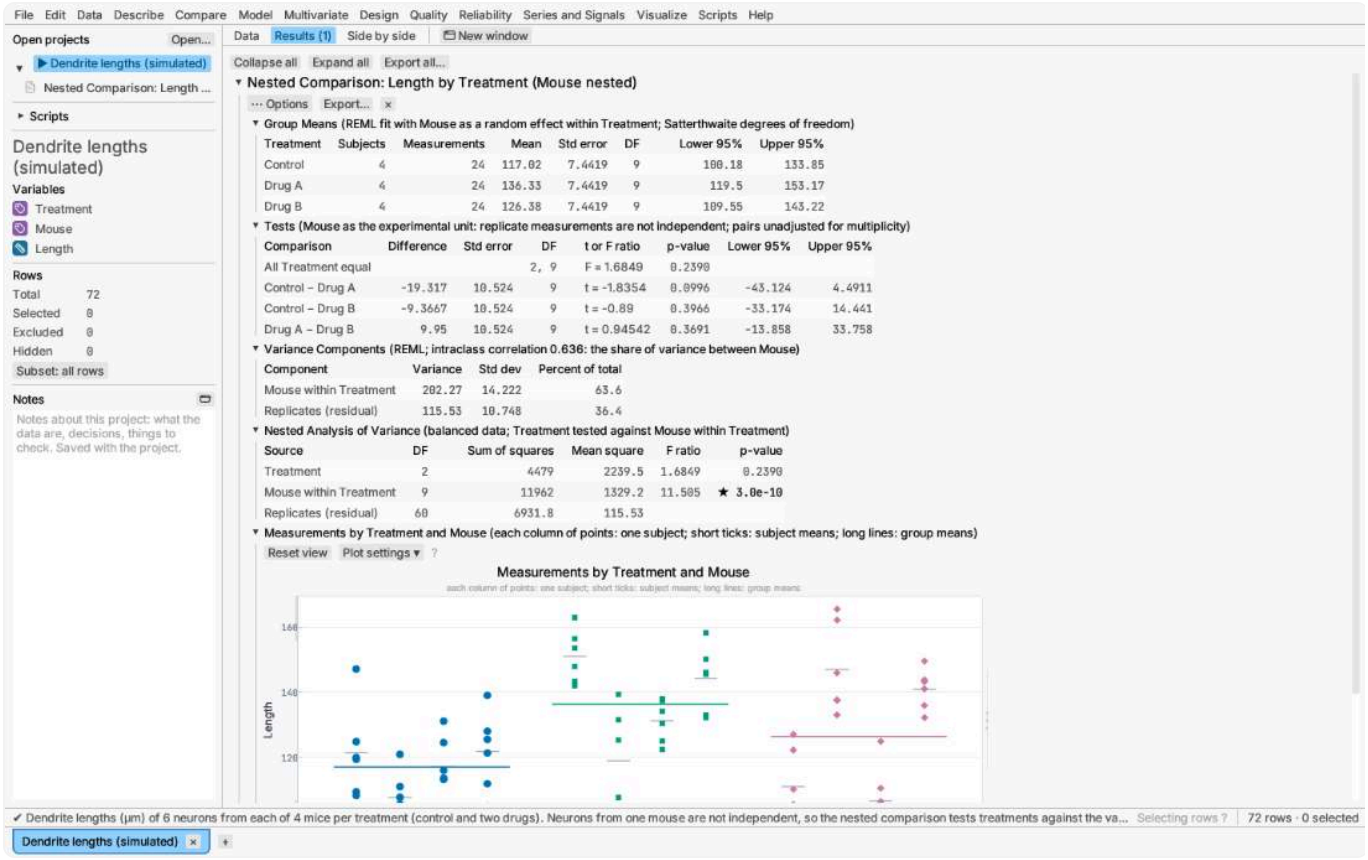
Nested comparison

Compare ▸ Nested comparison… compares groups when each subject (an animal, a culture plate, a batch) contributes several measurements. The measurements of one subject are not independent, so treating them as separate observations (pseudoreplication) overstates the evidence; the subject is the experimental unit. Choose the response, the group column and the subject column; subject labels are read within their group, so "Mouse 1" in two treatments is two mice. With two groups the analysis is a **nested t test**, with more a **nested one-way ANOVA**.

The report shows:

- **Group means:** subjects and measurements per group, and the model-based mean of each group with its standard error, Satterthwaite degrees of freedom and confidence interval.
- **Tests:** the F test that all groups are equal and each pair's difference with a t test and interval, from a linear mixed model with subjects as a random effect fitted by REML; denominator degrees of freedom by Satterthwaite's approximation (as in R's `lmerTest`). Pairwise p-values are not adjusted for multiplicity.
- **Variance components:** the variance between subjects within groups and between replicates, and the intraclass correlation (the share of variance between subjects).
- **Nested analysis of variance:** the classical table with groups tested against subjects within groups, shown when every subject has the same number of measurements and every group the same number of subjects (it then gives the same F test as the mixed model).
- **Plot:** each subject's measurements in a column, with subject and group means.

Try it: Dendrite lengths per mouse.



Proportions

Exact binomial test; two-proportion score z test; Pearson and G^2 tests of equal proportions across groups; Fisher's exact test; McNemar's test for paired outcomes.

Event rates

Exact Poisson tests of a rate and of a ratio of two rates.

Goodness of fit for counts

Compare ▸ Goodness of fit for counts… tests whether counts follow a hypothesized distribution, with Pearson's χ^2 and the likelihood-ratio G^2.

- **Categories in given proportions:** choose a category column (each row one observation, or give a **frequency column**). Leave **Equal proportions** ticked, or untick it and enter a proportion or ratio for each category (9, 3, 3, 1 for a 9 : 3 : 3 : 1 ratio; they are rescaled to sum to 1). Degrees of freedom: categories − 1.
- **Counts from a Poisson distribution:** choose a column of whole-number counts. The mean is estimated from the data, and neighbouring counts are pooled (from the low end, and into an open upper cell) until each cell expects at least the minimum you set (5 by default). Degrees of freedom: cells − 2.

The report gives the two tests, a table of observed and expected counts with each cell's contribution to χ^2 and its Pearson residual ((observed − expected)/√expected), and a plot of observed and expected counts per cell. A note warns when cells expect fewer than five.

Try it: Horse-kick deaths.

Crossover trial and equivalence

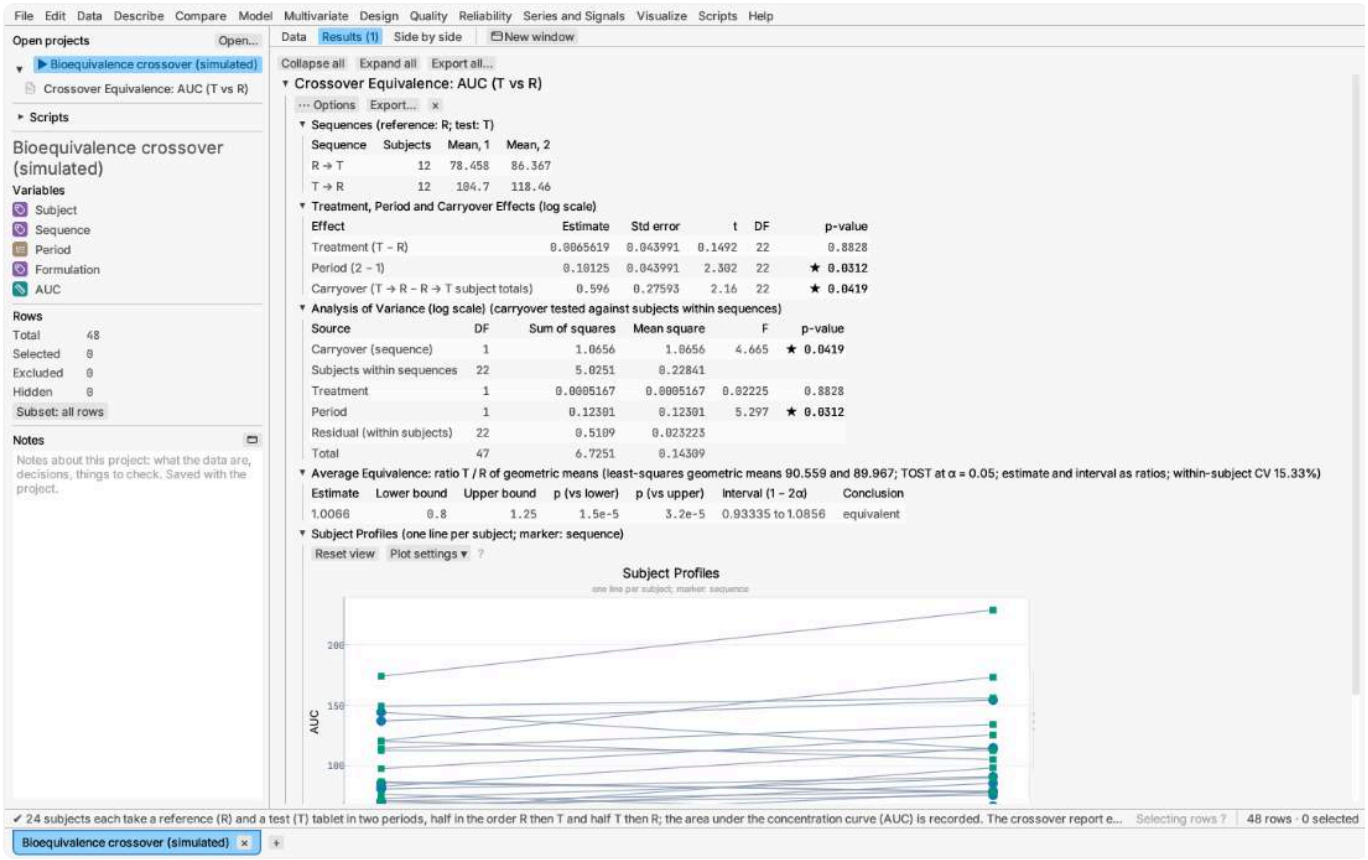
Compare ▸ Crossover trial and equivalence… analyses a two-period, two-treatment (2×2) crossover, in which each subject receives a reference and a test treatment in one of the two orders. The data are stacked: one row per subject and period, with the response and columns naming the subject, the period and the treatment. Pick the **reference treatment**

(the first in order by default), whether to compare the **ratio of geometric means** (on logs, the usual choice for bioequivalence) or the **difference of means**, the equivalence bounds (0.80 to 1.25 for a ratio) and the level α of each one-sided test (0.05 gives the usual 90% interval).

The report shows:

- **Sequences:** the subjects and mean responses in each order. Subjects without one row for each treatment are left out and counted.
- **Treatment, period and carryover effects:** estimates with standard errors, t tests and p-values (Chow & Liu's formulation, from each subject's period difference and total).
- **Analysis of variance:** carryover tested against subjects within sequences; treatment and period against the within-subject residual.
- **Average equivalence:** the least-squares (geometric) means, the estimate and its 1 − 2α interval, the two one-sided p-values and the conclusion; on the ratio scale also the within-subject coefficient of variation.
- **Subject profiles:** each subject's two responses joined by a line, with a marker shape per sequence.

Try it: Bioequivalence crossover.



Stratified tables and exact tests

Compare ▸ Stratified tables and exact tests… cross-classifies a **Rows** column (for 2 × 2 tables, the treatment or exposure) by a **Columns** column (the outcome) within each **stratum** — each combination of the ticked stratum columns, such as the centres of a multi-centre trial. One row per observation, or a **Frequency column** of whole-number counts. The levels and strata are those of the rows counted (not excluded, with every label and a frequency above zero), so a label found only in excluded rows adds no empty row, column or stratum. The report gives:

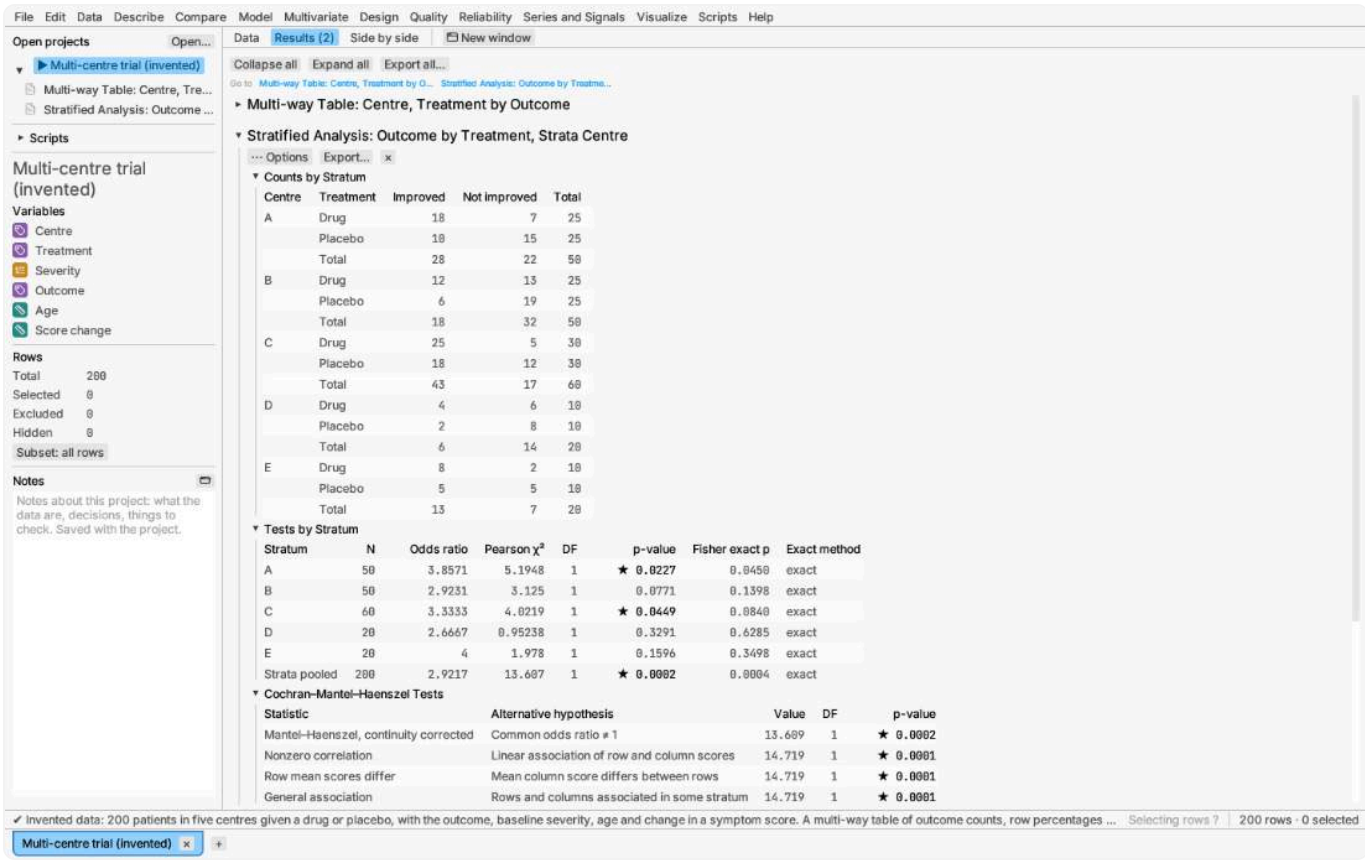
- **Counts by Stratum**, and **Tests by Stratum**: each stratum's size, odds ratio (2 × 2 tables), Pearson χ^2 with its p-value, and **Fisher's exact test** for any r × c table (the Freeman–Halton extension: the probability, given the margins, of a table no more probable than the one observed), plus the same for the table pooled over strata.
- **Cochran–Mantel–Haenszel Tests** of conditional independence across the strata: *nonzero correlation* (1 df; ordered rows and columns), *row mean scores differ* (r − 1 df; ordered columns) and *general association* ((r − 1)(c − 1) df; nominal), and for 2 × 2 tables the Mantel–Haenszel test with the continuity correction. Scores are the levels' values when every label is a number, otherwise 1, 2, … in level order; the note lists them.
- For 2 × 2 tables, the **Common Odds Ratio** — the Mantel–Haenszel estimate with the Robins–Breslow–Greenland confidence interval, beside the crude odds ratio of the pooled table (a large difference is a sign of confounding by the strata) — and **Homogeneity of Odds Ratios**: the Breslow–Day test, with and without Tarone's correction (a small p-value says the odds ratio differs between strata, so a common one is not a fair summary). Odds ratios compare the first column level's odds in the first row with the second row, in the levels' natural order.

With no stratum column the report is one table with its Pearson and exact tests. The exact test uses the network algorithm of Mehta & Patel (1983); a table too large for it within a fixed amount of work gets a Monte Carlo estimate from random tables with the same margins instead (10 000 by default, seeded so that the result repeats), marked *Monte Carlo* with its standard error in the table and explained in the note. The random tables are drawn cell by cell (Patefield's algorithm), so even tables with hundreds of thousands of observations take about a second. A table whose total count is above 2^{53} (about $9{\cdot}10^{15}$, possible with a frequency column) is beyond exact arithmetic: its row says *exact test not available* and the note gives the reason, while the rest of the report is unaffected.

Adjust each stratum's p-values for the false discovery rate (More options or ⋯ Options) adds **Tests by Stratum Adjusted for the False Discovery Rate**: the Pearson

(and, when computed, exact) p-value of each stratum beside its q-value across the strata, by Benjamini–Hochberg, Benjamini–Yekutieli or Storey's q-values ($\hat{\pi}_0$ at $\lambda = 0.5$, given in the heading). With many strata, such as many sites or many subgroups, some tables come out significant by chance; the q-values keep the expected share of false findings among those called significant at the chosen level. The confidence level, the exact test and the Monte Carlo settings can be changed from the report's ⋯ Options menu.

Try it: Multi-centre trial (treatment by outcome, stratified by centre).



Permutation test

Two groups, paired columns or a correlation tested against every rearrangement of the data (or random ones, with a seed), without a normal assumption. See Resampling.

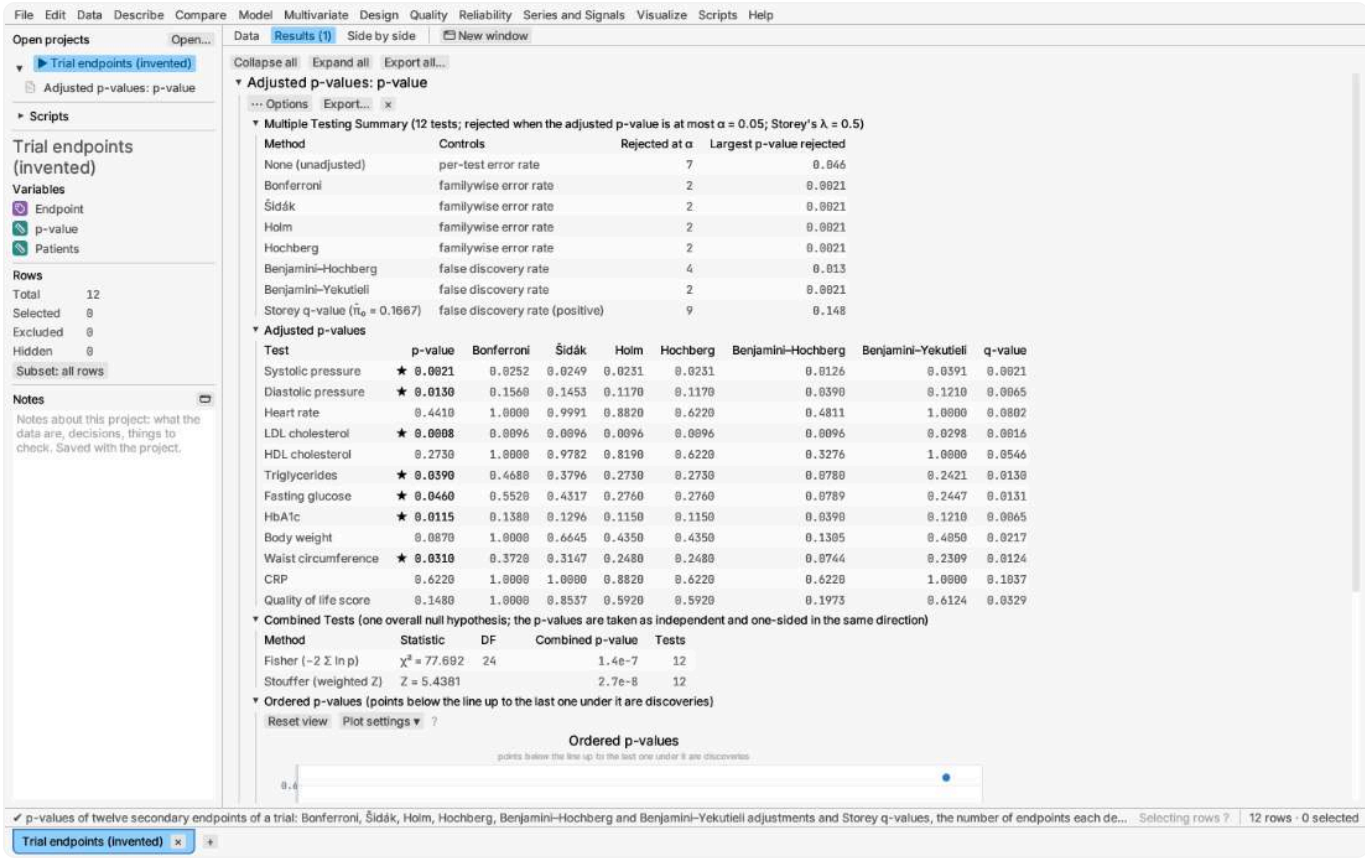
Adjust and combine p-values

Compare ▸ Adjust and combine p-values… takes a column of p-values, one test per row, with an optional column naming the tests. Missing p-values are left out, and the number of tests m counts the others.

- **Multiple Testing Summary:** for no adjustment and each method, the error rate it controls, how many tests are rejected (adjusted p ≤ α) and the largest raw p-value rejected.
- **Adjusted p-values:** one row per test with Bonferroni (m·p), Šidák (1 − (1 − p)^m), Holm (step-down), Hochberg (step-up), Benjamini–Hochberg and Benjamini–Yekutieli (false discovery rate; BY holds under any dependence) adjusted p-values, and Storey's q-values. Bonferroni, Šidák, Holm and Hochberg control the family-wise error rate.
- **Storey's q-values** estimate the share of true null hypotheses as $\hat{\pi}_0$ = #{p > λ} / (m(1 − λ)) (λ = 0.5 by default; set it in the dialog) and scale the Benjamini–Hochberg values by $\hat{\pi}_0$, so they are smaller when many tests have real effects.
- **Combined Tests:** one test of the overall null hypothesis that no test has an effect. Fisher's method refers −2 Σ ln p to χ^2 with 2k degrees of freedom; Stouffer's method sums the normal scores $\Phi^{-1}(1 - p)$, divided by √k, or weighted by a column you choose under **More options** (such as each test's sample size). Both assume independent, one-sided p-values in the same direction.
- **Ordered p-values:** the sorted p-values against their rank with the Benjamini–Hochberg line α·i/m.

Saving: the report's ⋯ Options menu saves any adjusted p-values or the q-values as a new column, in the same rows as the p-values. In formulas, `p_adjust("p", "bh")` gives the same values (see the formula reference).

Try it: Trial endpoints (twelve endpoints of a clinical trial, weighted by the number of patients measured).



Power and sample size

Compare ▸ Power and sample size… plans a study. Choose a design, then what to **Solve for** — power, sample size, or the detectable effect — and give the other two. Designs:

- **Tests:** one-sample t and z, paired t (from the standard deviation of each condition and their correlation), two-sample t, one-way ANOVA, one and two proportions, one and two Poisson rates (with the exposure per unit), one standard deviation, two variances, and equivalence for one or two samples.
- **Margins of error** for a mean, a proportion (Wilson interval), a standard deviation, or a rate: the probability that the interval's margin (half-width) is at most E. Solving for the sample size gives the n that reaches the target probability; solving for the effect gives the smallest margin reached with that probability.
- **Tolerance intervals** (one- or two-sided, with the proportion covered and the confidence): the probability that the interval's content stays below a limit P' — the larger the sample, the closer the interval hugs the proportion asked for.
- **Distribution-free tolerance intervals** (no assumption about the population's shape): the interval runs between order statistics of the sample, leaving *m* of them

outside (m = 1 takes the smallest or largest value as one limit, m = 2 with two sides takes both extremes). Its "power" is the confidence that it covers at least the proportion P, the exact binomial sum $\Sigma_{j \le n - m} C(n, j) P^j (1 - P)^{n-j}$ (Wilks 1941; Guenther 1977). Solving for the sample size gives the smallest n with the target confidence (59 for 95% content with 95% confidence and one limit, 93 for two); solving for the effect gives the largest proportion covered with that confidence.

- **Factorial experiments:** two-level factorial (factors, fraction 1/2^q, center points, main effects or with two-factor interactions), screening designs (runs per replicate, factors) and general full factorials (levels of the factor tested and cells of the other factors). The sample size is the number of replicates, and the report also gives the total runs.

Every report plots the power (or probability) against the sample size and against the effect, with a marker at this study and a line at the target.

Try it: Sleep drug trial (how many subjects a follow-up study needs).

Resampling: bootstrap and permutation tests

Resampling methods replace a formula for a statistic's sampling distribution with the data themselves: the **bootstrap** re-computes a statistic on samples drawn with replacement from the rows, and a **permutation test** re-computes it over rearrangements of the data that the null hypothesis makes equally likely. Both are useful for small or skewed samples, and for statistics that have no standard error formula.

Every resampling run is **seeded**: the same seed gives the same results, whatever the number of processor cores used. Change the seed to see how much the Monte Carlo part varies.

Try it: Repair times (invented). It opens with a distribution report, a bootstrap of its mean, standard deviation and coefficient of variation, a permutation test of the two crews' median repair times, and a permutation test of the correlation between parts replaced and repair time.

Bootstrap of any report table

Any table in a report can be bootstrapped. Right-click the table (or its section heading) and choose **Bootstrap this table…**, or use the report's **⋯ Options ▸ Bootstrap a table** menu, which lists the report's tables.

The dialog shows the table's columns and rows that hold numbers:

- **Columns** and **Rows:** the cells to collect. Every chosen row of every chosen column is one bootstrapped statistic. The first numeric column and every row start chosen.
- **Replicates:** the number of bootstrap samples B (1000 by default; at least 1000 is advisable for intervals).
- **Seed:** the start of the random draws.
- **Confidence level** of the intervals (0.95 by default).
- **Resample within levels of:** a column whose levels are resampled separately, keeping each level's number of rows (the stratified bootstrap, for example to keep the size of each group of a two-group comparison).

Each replicate draws as many rows as the report uses, with replacement, from the rows that are not excluded and have a value in every column the report refers to. The report is re-run on the resample exactly as it was set up (its options included), and the chosen cells are read as the report shows them — with the report's displayed precision, typically five or six significant digits.

The bootstrap report shows:

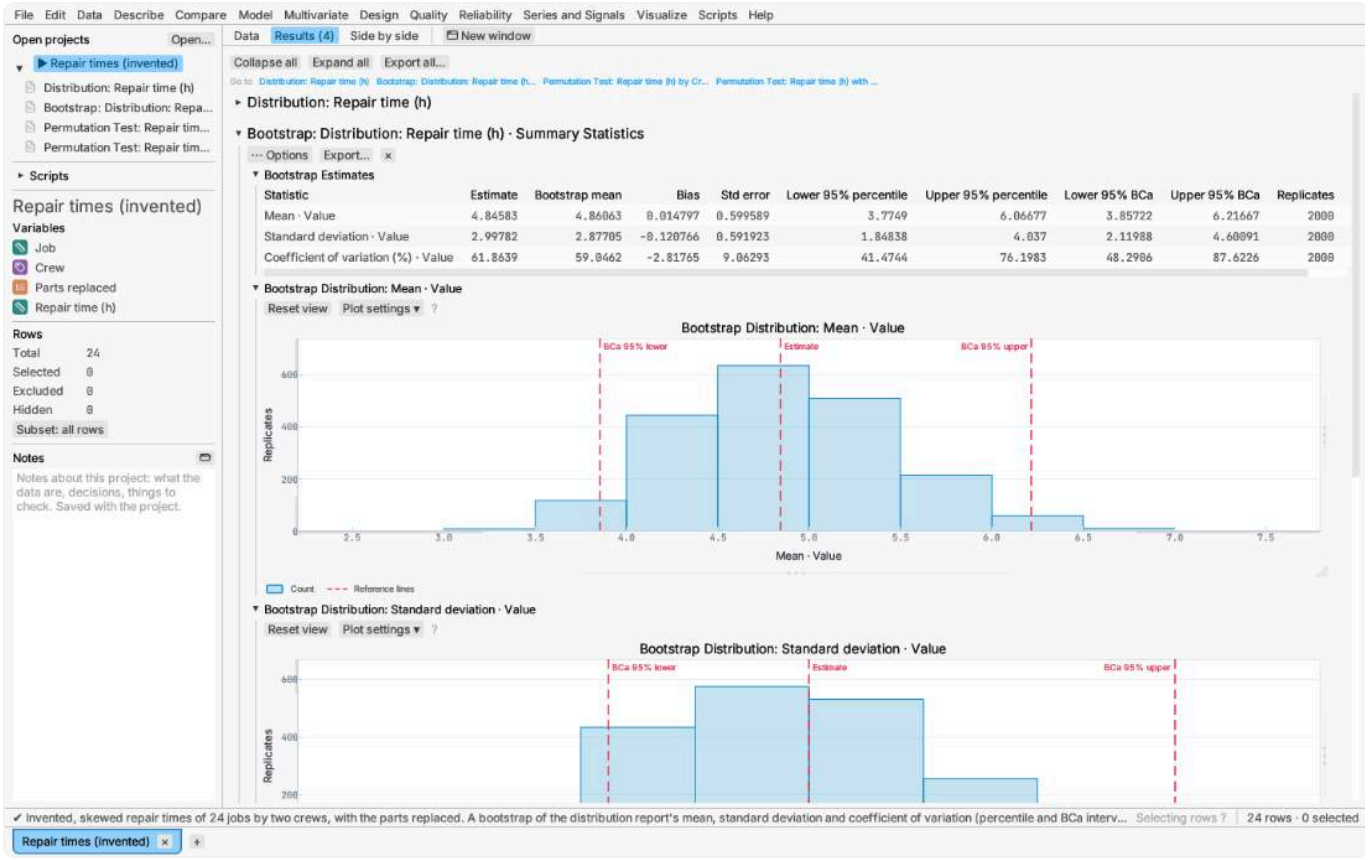
- **Bootstrap Estimates:** for each statistic, the estimate (the cell in the original report), the bootstrap mean, the **bias** (bootstrap mean minus estimate), the **standard error** (standard deviation of the replicates), the **percentile** interval (quantiles of the replicates) and the **BCa** interval (bias-corrected and accelerated; Efron 1987), the number of replicates used and the number that **failed**.
- **Bootstrap Distribution** of each statistic: a histogram of its replicates with the estimate and the BCa interval (or the percentile interval when BCa is not available) marked.
- **About This Bootstrap:** the settings, how many resamples the report could not be computed on, and the jackknife used for BCa.

A replicate **fails** when the report cannot be computed on that resample (for example a model with a factor level missing from the resample) or the cell is not a number on it (a dash for "not defined"). Failed replicates are left out of that statistic's summary and counted, never dropped silently; many failures mean the resampled report often breaks, and the intervals should be read with care.

The BCa interval adjusts the percentile interval for bias and skewness. Its acceleration comes from the **jackknife**: the report is re-run once with each row left out. That is done for up to 500 rows; with more rows the report gives percentile intervals only. When every replicate falls on one side of the estimate, BCa is not defined and shows "—".

The bootstrap treats the rows as independent observations. It does not suit reports whose meaning depends on the order of the rows (control charts, run charts, time-ordered data) or on a fixed design (a designed experiment's runs), where resampling rows breaks the structure the report relies on.

Bootstrap reports are saved in projects with the report they bootstrap, and re-run when the data or the excluded rows change. Reports that use no data (power, distribution calculator), graphs and bootstrap reports themselves cannot be bootstrapped.



Permutation test

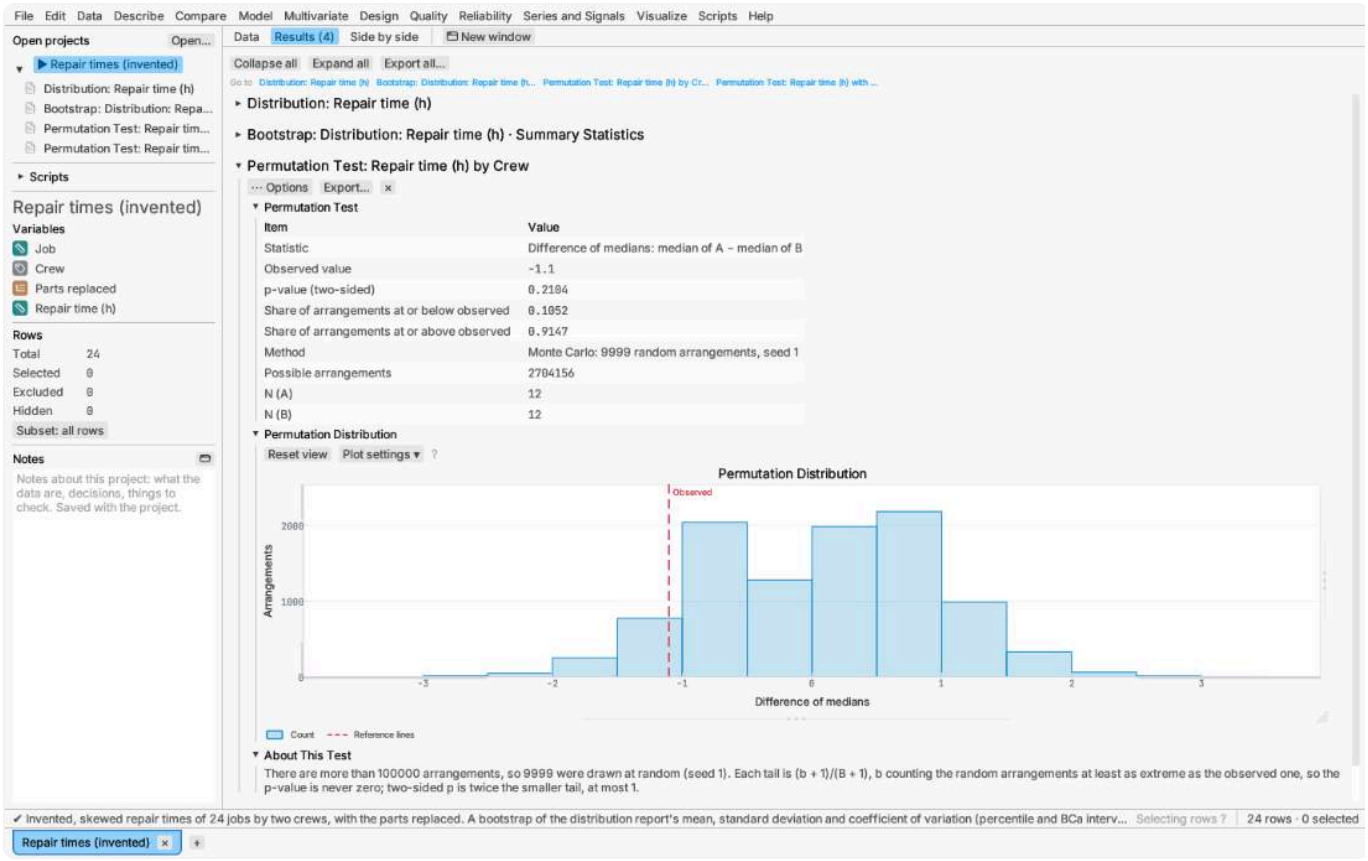
Compare ▸ Permutation test... tests one of three designs:

- **Two groups:** a response column and a grouping column with exactly two levels. The statistic is the difference of means, or of medians, of the first level minus the second (levels in natural order). Under the null hypothesis the group labels are exchangeable.
- **Paired columns:** two columns measured on the same rows. The statistic is the mean of the differences first − second; under the null hypothesis each difference is equally likely to have either sign.
- **Correlation:** two numeric columns, by Pearson's r or Spearman's ρ. Under independence every pairing of the values is equally likely.

Choose the **alternative**: two-sided, or one-sided with the statistic lower or higher than its null distribution. When the number of possible arrangements is at most **Exact when arrangements are at most** (100 000 by default), every arrangement is evaluated and the p-value is exact. Otherwise the **random arrangements** (9999 by default) are drawn with the **seed**, and each tail is (b + 1)/(B + 1), where b counts the random arrangements at least

as extreme as the data: the p-value is never zero. A two-sided p-value is twice the smaller tail, at most 1.

The report gives the statistic and its observed value, the p-value, both tail shares, the method (exact or Monte Carlo) and the number of possible arrangements, the sample sizes, and a histogram of the permutation distribution with the observed value marked.



Scripting

Both reports run from scripts like any other: `t.analyze("permutation-test", #{ design: "two-groups", response: "Repair time (h)", group: "Crew", statistic: "median" })`, and `t.analyze("bootstrap", #{ report: #{ analysis: "distribution", column: "Repair time (h)" }, section: "Summary Statistics", columns: ["Value"], rows: ["Mean"] })`. See Scripting and recording for the settings.

References

- Efron, B. (1987). Better bootstrap confidence intervals. *Journal of the American Statistical Association* 82, 171–185.
- Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall (chapters 6, 10, 13 and 14).
- Davison, A. C. and Hinkley, D. V. (1997). *Bootstrap Methods and Their Application*. Cambridge University Press (§4.3).
- Phipson, B. and Smyth, G. K. (2010). Permutation p-values should never be zero. *Statistical Applications in Genetics and Molecular Biology* 9(1).

Model: linear and response-surface models

Linear model

Model ▸ Linear model… fits ordinary least squares with continuous and categorical predictors, interactions and powers.

Building the model. Choose a numeric response and tick the predictors; each enters as a main effect, which you can untick in the effect list (a predictor left with no effect is not in the model). The **Main effects**, **+ Two-factor interactions** and **+ Squares** buttons build common models quickly. The dialog keeps models hierarchical: an interaction or power brings in its lower-order effects, and unticking an effect also removes the interactions and powers built on it (turn this off under **More options**, which also notes how predictors are centered and coded). Click **Run**.

Report:

- Fit statistics: R^2, adjusted R^2, RMSE, AICc and BIC.
- Effect Pareto ($-\log_{10}$ p per effect).
- Parameter estimates with standard errors, t tests and p-values.
- Effect tests.
- A fitted line plot (one predictor) or observed-versus-predicted plot.

Options: ANOVA table, effect sizes, confidence limits for the estimates, confidence band, residual plot, influence diagnostics (leverage, Cook's distance and a residual quantile plot), the Response Explorer and 3-D surface profiler.

Saving: predicted values as a new column (for every row with its predictors present, excluded rows and rows without a response included, so new rows are scored), residuals (for every row with complete data), or the **prediction formula** as a formula column, so predictions update with the data and are computed for new rows.

One-way models. When the only predictor is one categorical factor, the report adds tests of equal variances (Levene, Brown–Forsythe, Bartlett) and a **Pairwise comparisons** choice: Tukey–Kramer (the default), Games–Howell, Holm-adjusted t, Bonferroni, Šidák, Holm–Šidák, Fisher's LSD (unadjusted; only after a significant overall test), Newman–Keuls

(stepwise studentized range; p-values only, no intervals), and for unequal variances Tamhane's T2 (Welch t per pair with Šidák's adjustment) and Dunnett's T3 (Welch t against the studentized maximum modulus), or none. **Linear trend across ordered levels** (three or more levels) tests a straight-line trend in the means across the levels in their order, with an F test of departure from linearity. **Comparisons with a control (Dunnett)** compares every other level with the level you pick, with simultaneous 95% intervals from the model's error mean square and an interval plot of the differences. **Comparisons with the best (Hsu)** compares every level with the best of the others, larger or smaller being better, and says which levels may be the best (see Compare).

Effect sizes (options menu) adds a table with, for each effect, η^2 (the effect's share of the total sum of squares), partial η^2, ω^2 and Cohen's f, with confidence intervals for partial η^2 and f found by inverting the noncentral F distribution of the effect's F statistic (Steiger 2004). The intervals use the report's confidence level.

Spline terms. Under **More options** each continuous predictor that is not in an interaction or power can enter as a curve instead of a straight line: a **natural cubic spline** (cubic between the knots, straight beyond the outer ones, so it does not swing at the ends), or a **linear**, **quadratic** or **cubic B-spline** (a linear B-spline is a broken line). **Knots** sets the number of interior knots, placed at equally spaced quantiles of the predictor (3 by default, up to 20), or choose **at** and type the knot positions, separated by commas. The boundary knots are the smallest and largest values. The spline's columns appear as one effect: the effect test is the joint F test of all its coefficients, and the estimates table lists one coefficient per basis column (`x: spline 1`, `x: spline 2`, …). The fit plot and Response Explorer show the curve with its band, and the saved prediction formula writes the basis with `natural_spline(…)` or `bspline(…)` (see formula columns). The bases follow de Boor (1978) and Harrell (2015, §2.4); with the same knots they give the same fits as R's `splines::ns` and `splines::bs`.

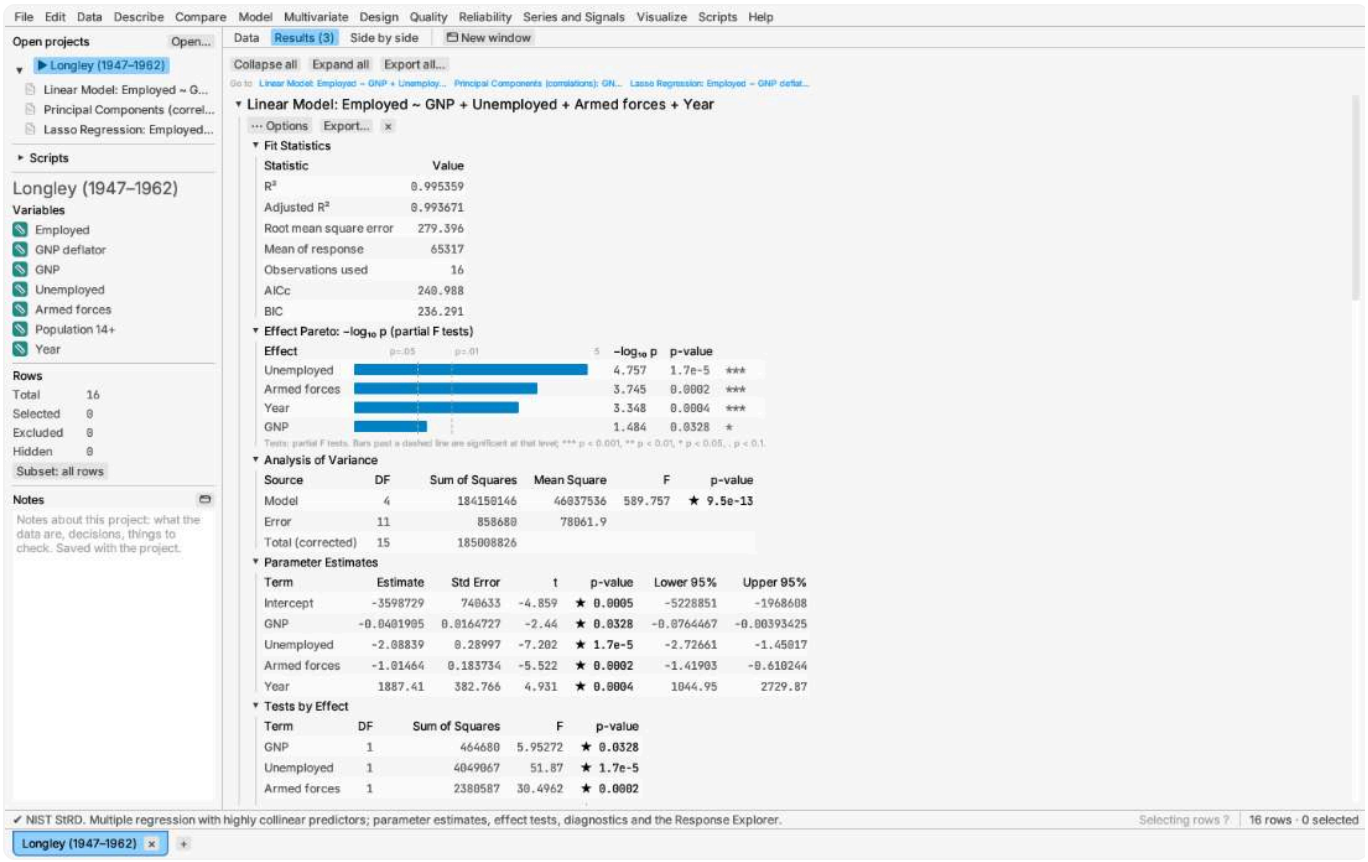
Comparisons within slices. For each interaction of two categorical factors A × B, **Comparisons within slices** (in **More options** or the report's ⋯ Options) tests A at each level of B and B at each level of A (*simple effects*), each with an F test from the model's error mean square, and lists the least-squares means of every A × B cell (other factors averaged over their levels, covariates at their means). Choosing Tukey, Holm or Bonferroni adds the pairwise differences of the means within each slice with adjusted p-values and intervals; **Tests only** leaves them out. The least-squares means are the estimable functions of Searle, Speed & Milliken (1980), the same as R's `emmeans`.

Slopes by group. For an effect of a categorical factor with a continuous predictor (separate slopes, as in an analysis of covariance with unequal slopes), **Slopes by group** gives each

group's slope with its interval, the F test that all slopes are equal, and, with an adjustment chosen, every pairwise difference of slopes (Milliken & Johnson 2002).

Verified: the NIST StRD linear regression and ANOVA suites; effect-size intervals against R; spline fits against `lm` with `splines::ns` / `bs`; slices against `emmeans` (`joint_tests`, `pairs(by =)`) and slopes against `emmeans::emtrends`.

Try it: Norris calibration; Longley economy (multiple regression); Plant growth (ANCOVA); Silicon resistivity (one-way).



Response surface

Model ▸ Response surface (second-order model)… fits the full second-order model (linear, two-factor interaction and squared terms) on coded factors, and adds:

- the **stationary point** and **canonical analysis** (eigenvalues tell you whether it is a maximum, a minimum or a saddle);
- **ridge paths** of maximum and minimum response;

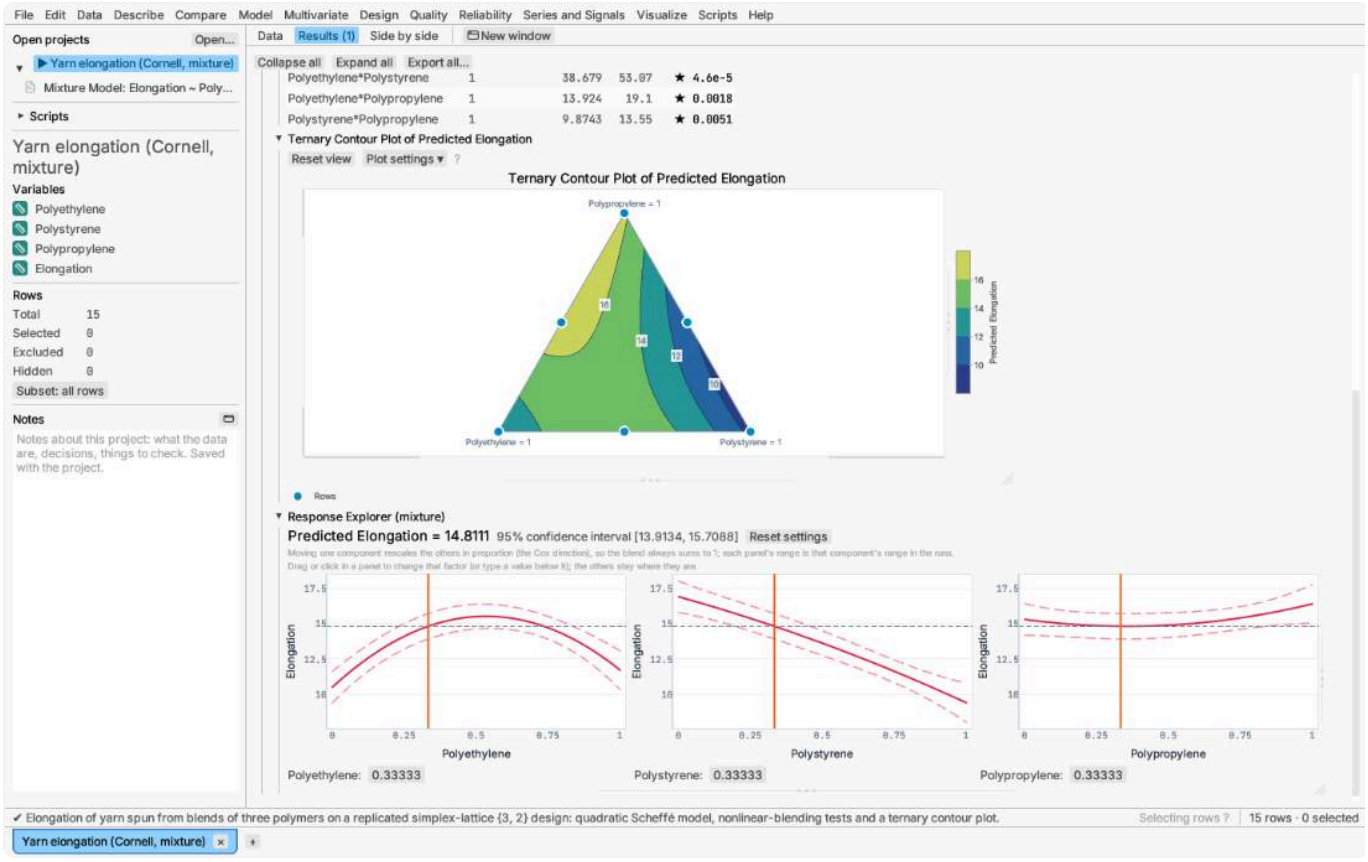
- **contour plots** over any two factors (choose the horizontal and vertical factors; editing the analysis down to fewer factors goes back to the first two), filled with colour and a colour bar; the colours, lines and levels are set in the plot's Plot settings (see Contour maps);
- the Response Explorer and 3-D profiler.

The method follows Box & Draper, *Response Surfaces, Mixtures, and Ridge Analyses*.

Try it: Reaction yield.

Mixture model

Model ▸ Mixture model… fits Scheffé linear, quadratic and special-cubic models for components that sum to a constant. The mixture Response Explorer rescales the other components when you move one, and ternary plots show the response over three components as a filled contour map (see Contour maps).



Try it: Yarn elongation.

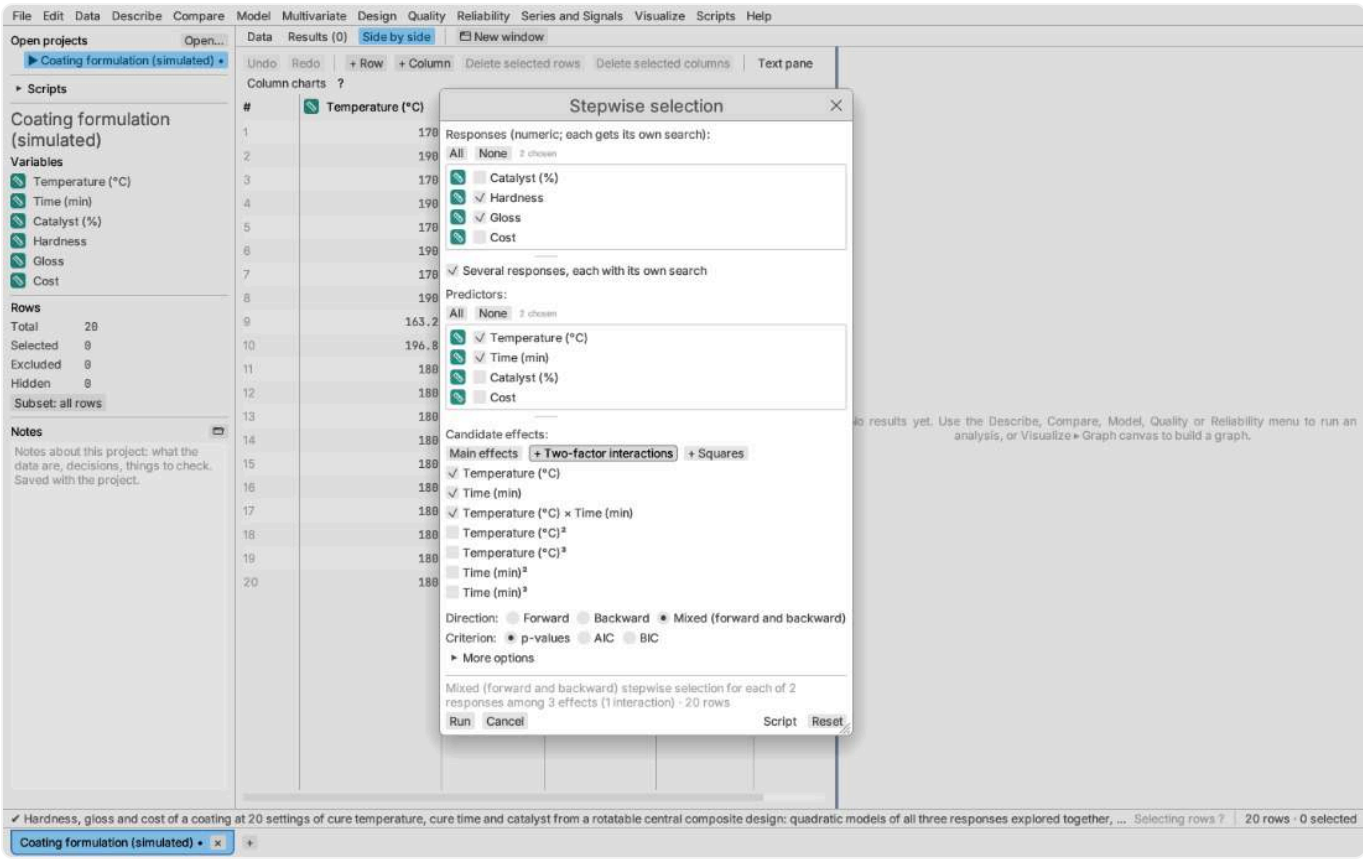
Stepwise selection

Model ▸ Stepwise selection… chooses model effects from a set of candidates:

- direction: forward, backward or mixed;
- criterion: p-values, AIC or BIC;
- under **More options**: the **Enter if p <** and **Remove if p >** thresholds and the heredity rule, so the selected model stays hierarchical.

Click **Run**.

To search several responses at once, tick **Several responses, each with its own search** and choose the responses: each gets its own search among the same candidates, and the selected models open together in a several-responses report, with desirability goals and a search for the best settings.



The report shows the step history. For the selected model you can show the Response Explorer and 3-D profiler, or **Open as linear model report** for full estimates and diagnostics.

Try it: Hald cement.

Best subsets

Model ▸ Best subsets… searches every subset of the candidate effects and reports the best models of each size. The dialog is the stepwise dialog's, with:

- candidate effects built the same way (interactions and powers allowed);
- **Best overall by:** R^2, adjusted R^2, Mallows' Cp, AICc or BIC (the default);
- under **More options**: a heredity rule (strong, weak or none), **Effects in every model** (effects forced into each model), **Models kept per size** (3 by default) and an optional **Largest model size**.

Click **Run**. The report lists the models of each size, best first, with R^2, adjusted R^2, Cp, AICc and BIC; the best overall by the chosen criterion is labelled "best by …" in the last column. A plot shows the criterion against the number of effects (for Cp, with the line Cp = number of coefficients). **Open the best model … as a linear model report** sits under the report, and any other listed model opens from the ⋯ Options menu. The search uses branch and bound (Furnival & Wilson's "leaps and bounds"), so it gives the same answer as fitting every subset; with many candidates it can still take a while, and the dialog warns above 25 free candidates.

Try it: Hald cement.

Screening many responses

Model ▸ Screen many responses… fits the same model to every chosen response column against the same factors, as in screening gene or protein expression, many quality characteristics, or a panel of assays, and says which responses the factors affect while controlling the false discovery rate across them.

Dialog. Tick the responses (numeric columns) on the left and the factors on the right; a column can be in one list only. **Screen each level of** (optional) runs a separate screen within each level of another column, such as each tissue. Then choose:

- **Model:** *Linear model* (F tests of each main effect from `lm`), *Poisson regression* for counts (likelihood-ratio χ^2 tests, log link), *Negative binomial regression* for overdispersed counts (the dispersion θ estimated by maximum likelihood with each fit; likelihood-ratio tests with θ refitted under the smaller model, as R's `anova` of `MASS::glm.nb` models), or *Kruskal–Wallis* (ranks; one categorical factor only).

- **False discovery rate by:** Benjamini–Hochberg (independent or positively dependent tests), Benjamini–Yekutieli (any dependence), or Storey's q-values with the proportion of true nulls $\hat{\pi_0}$ estimated at $\lambda = 0.5$. Each effect is adjusted across the responses (within each subgroup).
- **α** for the discoveries ($q \le \alpha$), and the **order** of the tables.

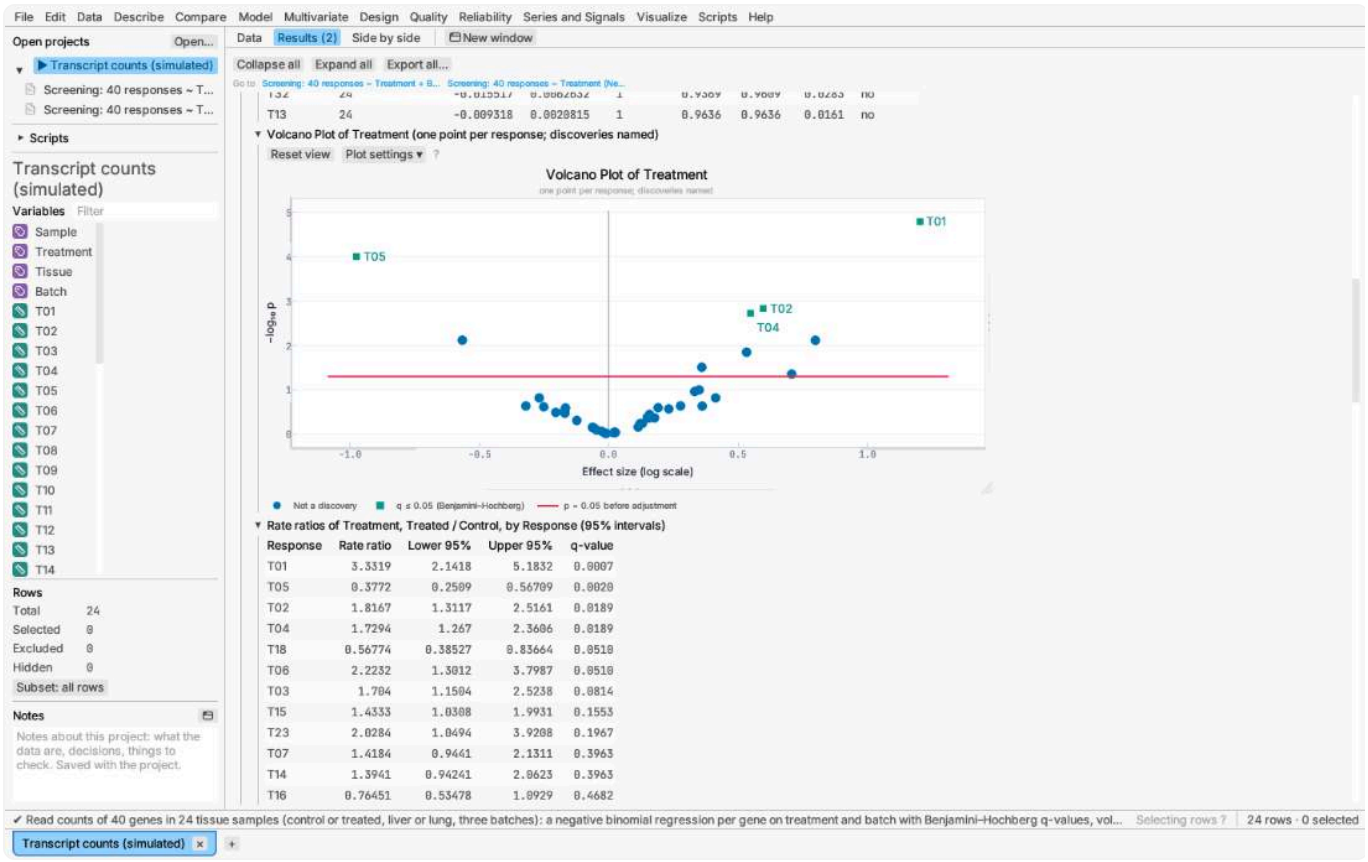
Report:

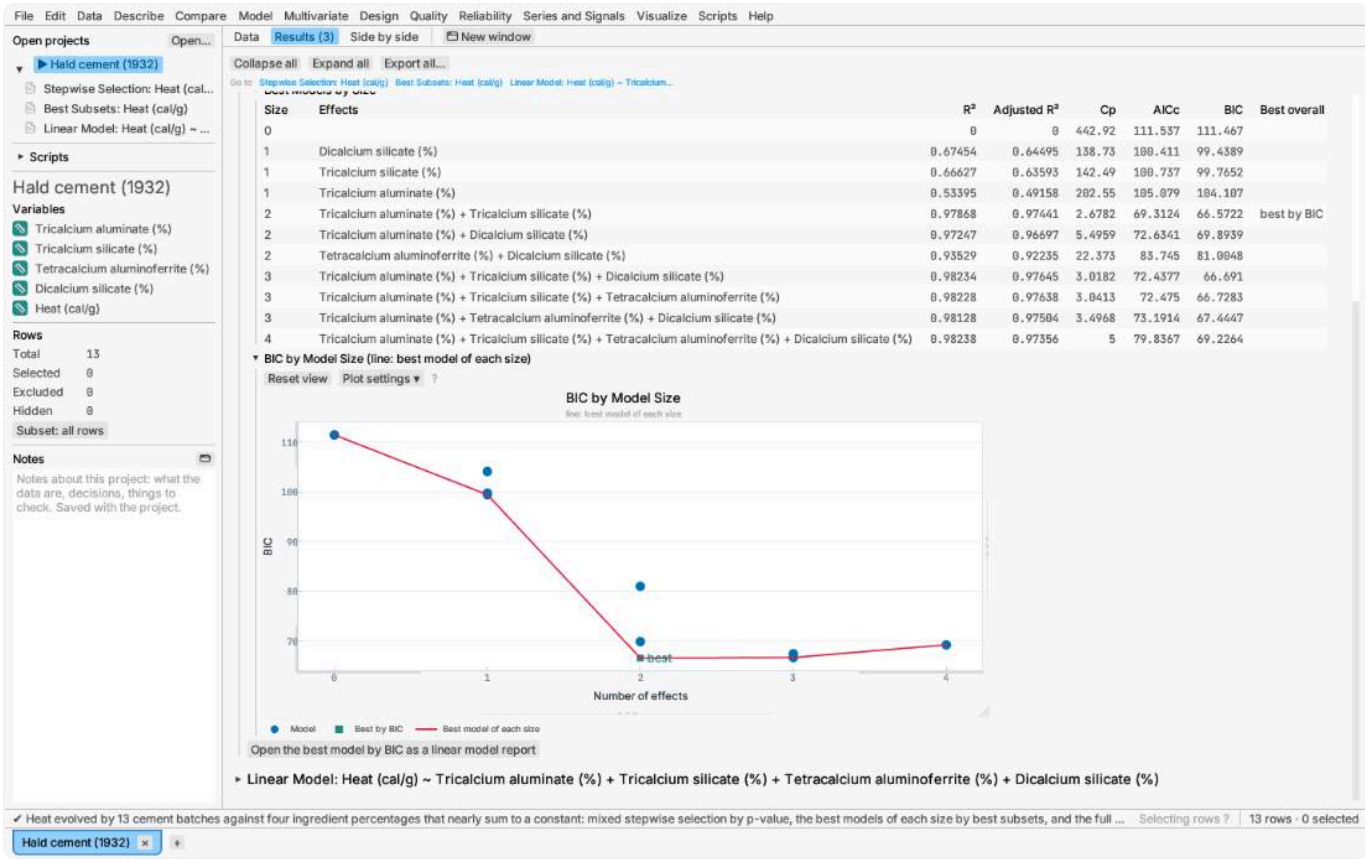
- Summary: model, responses fitted, the adjustment, and the number of discoveries per effect (with $\hat{\pi_0}$ for Storey).
- A table per effect: each response's n, effect size, test statistic, df, p-value, q-value, $-\log_{10} p$ and whether it is a discovery. The effect size is the slope for a continuous factor; for a categorical one the difference from the first level (linear model) or the log rate ratio (counts) when it has two levels, and the range of the level effects otherwise; for Kruskal–Wallis it is $\varepsilon^2 = H/(n - 1)$. Responses that could not be fitted show the reason.
- A volcano plot per effect (and subgroup): the effect size against $-\log_{10} p$, one point per response, discoveries drawn with a different marker and named, and a reference line at $p = \alpha$ before adjustment.
- For a two-level factor: the difference of means (linear model) or rate ratio (counts) with its 95% interval for each response, and a forest plot marking the discoveries.
- With a single response, the model's coefficients.

⋯ Options → **Open a response's full report** lists the responses, smallest q first, and opens the chosen one's full model report (linear model, Poisson GLM, two-or-more-groups comparison, or a one-response negative binomial screen).

Verified: against R's `lm` with `drop1`, `glm` (Poisson), `MASS::glm.nb`, `kruskal.test` and `p.adjust` on simulated data (`crates/app-model/tests/data/screening_reference.R`).

Try it: Transcript counts (simulated): counts of 40 transcripts in 24 samples, four of which respond to the treatment; one screen with Treatment and Batch, and one of Treatment within each tissue.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Transcript counts (simulated)
Screening: 40 responses ~ T...
Screening: 40 responses ~ T...
Scripts
Transcript counts (simulated)
Variables
Sample
Treatment
Tissue
Batch
T01
T02
T03
T04
T05
T06
T07
T08
T09
T10
T11
T12
T13
T14
Rows
Total 24
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (2)
Side by side
New window
Collapse all
Expand all
Export all...
T13 24 -0.009318 0.0020815 1 0.9636 0.9636 0.0161 no
Volcano Plot of Treatment (one point per response; discoveries named)
Reset view
Plot settings
Volcano Plot of Treatment
T01
T05
T02
T04
−log10 p
Effect size (log scale)
Not a discovery
q ≤ 0.05 (Benjamini–Hochberg)
p = 0.05 before adjustment
Rate ratios of Treatment, Treated / Control, by Response (95% intervals)
Response Rate ratio Lower 95% Upper 95% q-value
T01 3.3319 2.1418 5.1832 0.0007
T05 0.3772 0.2509 0.56709 0.0020
T02 1.8167 1.3117 2.5161 0.0189
T04 1.7294 1.267 2.3606 0.0189
T18 0.56774 0.38527 0.83664 0.0518
T06 2.2232 1.3012 3.7987 0.0518
T03 1.704 1.1504 2.5238 0.0814
T15 1.4333 1.0308 1.9931 0.1553
T23 2.0284 1.0494 3.9208 0.1967
T07 1.4184 0.9461 2.1311 0.3963
T14 1.3941 0.94241 2.0623 0.3963
T16 0.76451 0.53478 1.0929 0.4682
✔ Read counts of 40 genes in 24 tissue samples (control or treated, liver or lung, three batches): a negative binomial regression per gene on treatment and batch with Benjamini–Hochberg q-values, vol...
Selecting rows ?
24 rows · 0 selected
Transcript counts (simulated)


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Hald cement (1932)
Stepwise Selection: Heat (cal...
Best Subsets: Heat (cal/g)
Linear Model: Heat (cal/g) ~ ...
Scripts
Hald cement (1932)
Variables
Tricalcium aluminate (%)
Tricalcium silicate (%)
Tetracalcium aluminoferrite (%)
Dicalcium silicate (%)
Heat (cal/g)
Rows
Total 13
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
Size Effects R² Adjusted R² Cp AICc BIC Best overall
0 0 0 442.92 111.537 111.467
1 Dicalcium silicate (%) 0.67454 0.64495 138.73 100.411 99.4389
1 Tricalcium silicate (%) 0.66627 0.63593 142.49 100.737 99.7652
1 Tricalcium aluminate (%) 0.53395 0.49158 202.55 105.079 104.107
2 Tricalcium aluminate (%) + Tricalcium silicate (%) 0.97868 0.97441 2.6782 69.3124 66.5722 best by BIC
2 Tricalcium aluminate (%) + Dicalcium silicate (%) 0.97247 0.96697 5.4959 72.6341 69.8939
2 Tetracalcium aluminoferrite (%) + Dicalcium silicate (%) 0.93529 0.92235 22.373 83.745 81.0048
3 Tricalcium aluminate (%) + Tricalcium silicate (%) + Dicalcium silicate (%) 0.98234 0.97645 3.0182 72.4377 66.691
3 Tricalcium aluminate (%) + Tricalcium silicate (%) + Tetracalcium aluminoferrite (%) 0.98228 0.97638 3.0413 72.475 66.7283
3 Tricalcium aluminate (%) + Tetracalcium aluminoferrite (%) + Dicalcium silicate (%) 0.98128 0.97504 3.4968 73.1914 67.4447
4 Tricalcium aluminate (%) + Tricalcium silicate (%) + Tetracalcium aluminoferrite (%) + Dicalcium silicate (%) 0.98238 0.97356 5 79.8367 69.2264
BIC by Model Size (line: best model of each size)
Reset view
Plot settings
BIC by Model Size
BIC
best
Number of effects
Model
Best by BIC
Best model of each size
Open the best model by BIC as a linear model report
Linear Model: Heat (cal/g) ~ Tricalcium aluminate (%) + Tricalcium silicate (%) + Tetracalcium aluminoferrite (%) + Dicalcium silicate (%)
✔ Heat evolved by 13 cement batches against four ingredient percentages that nearly sum to a constant: mixed stepwise selection by p-value, the best models of each size by best subsets, and the full ...
Selecting rows ?
13 rows · 0 selected
Hald cement (1932)

Model: generalized, mixed and penalized models

Generalized linear model

Model ▸ Generalized linear model… fits binomial, Poisson, Gaussian and Gamma families, with categorical predictors. Choose the response first: the dialog then suggests a distribution from its values (binomial for a column with two values or text, Poisson for whole numbers of zero or more, normal otherwise), which you can change. For a binomial response, **Event** chooses which of the two values the model predicts the probability of; it starts at the second in sorted order ("Yes" against "No"). The dialog offers each family's links, the canonical one first:

- **Binomial:** logit, probit or complementary log-log. Probit and complementary log-log reports drop the odds ratios, which belong to the logit link.
- **Poisson:** log or identity.
- **Normal:** identity or log.
- **Gamma** (positive, right-skewed responses with a constant coefficient of variation): inverse, log or identity. The dispersion is estimated from Pearson's statistic and the tests use t; with the log link the report adds mean ratios.

For grouped yes/no data (a count out of a number of trials per row) and the effective doses of probit analysis, see Dose–response analysis.

More options adds **two-factor interactions** (tick any pair of predictors), **spline terms** for continuous predictors that are not in an interaction, and **comparisons within slices** and **slopes by group**, as for the linear model. In a GLM the slices are Wald tests from the coefficients' covariance and the least-squares means and differences are on the link scale (log odds, log mean), as R's `emmeans` gives them by default.

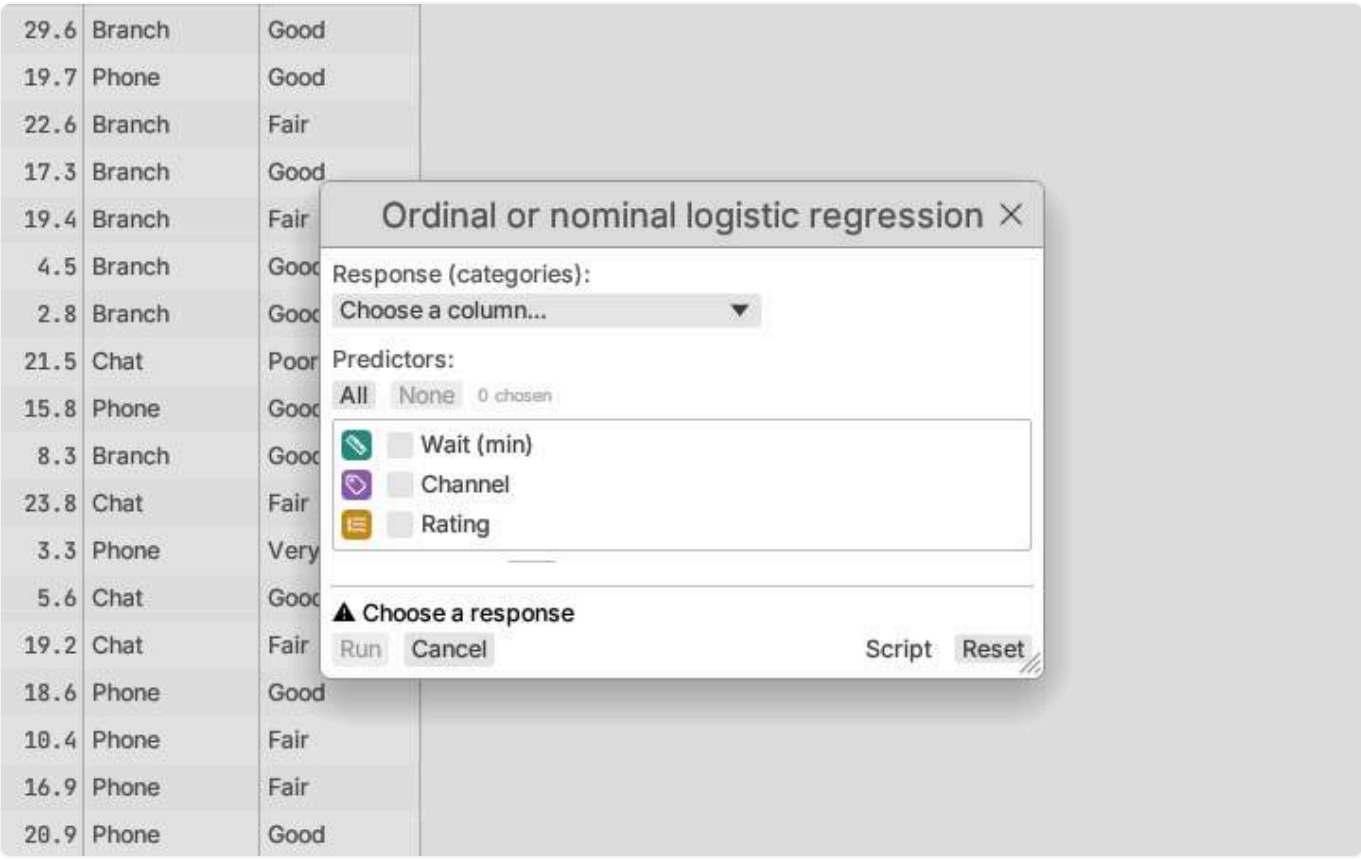
Report: parameter estimates, a likelihood-ratio test for each term, a fitted curve (one predictor) or observed-versus-fitted plot, and the Response Explorer and 3-D profiler. The explorer's confidence band is the interval for the linear predictor mapped through the inverse link. With the inverse link (mean = 1/η), where that interval includes η = 0 the mean is not bounded at all, so the band is left out there rather than drawn as a range that would not even contain the prediction.

Saving: predicted values (the fitted mean), or the prediction formula as a formula column: the inverse link applied to the linear predictor — `logistic(…)`, `normal_cdf(…)`, `1.0 - exp(-exp(…))`, `exp(…)` or `1.0 / (…)`.

Try it: Customer churn (logistic and Poisson models).

Ordinal and nominal logistic regression

Model ▸ Ordinal or nominal logistic regression… models a response with several categories (a rating scale, a choice among options). Any column can be the response; its categories are the distinct values of the rows used.



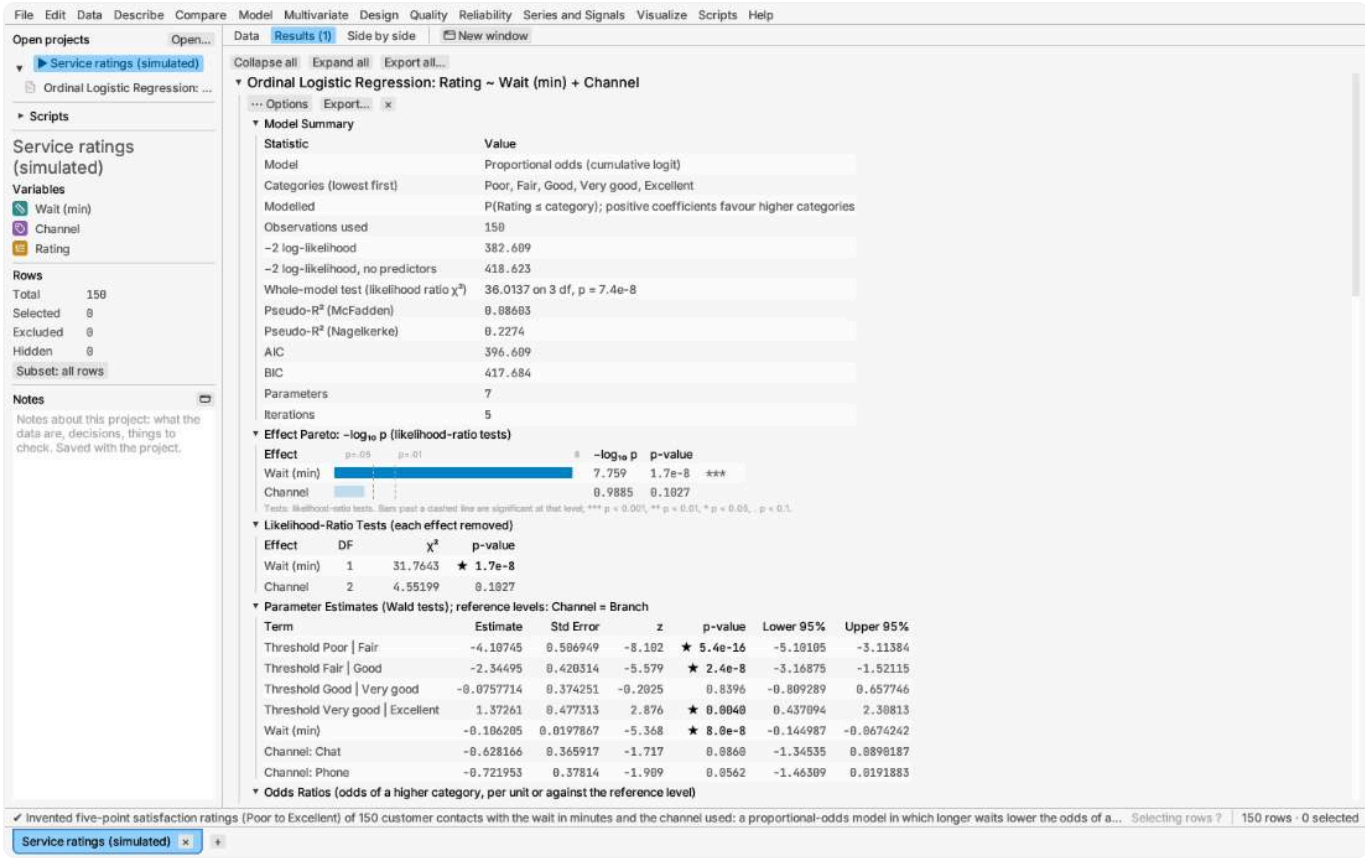
- **Ordinal (ordered categories):** the proportional-odds model, P(response ≤ category j) = logistic($\alpha_j - x\beta$). There is one threshold α_j between each pair of neighbouring categories and one set of coefficients for all of them, so a positive coefficient moves the response up the scale. The dialog lists the categories in sorted order (numbers numerically, text alphabetically); use the ▲ and ▼ buttons to put them in their real order, lowest first. The order is saved with the report.

- **Nominal (unordered categories):** baseline-category logits, ln(P(category j) / P(reference)) = $\beta_{0j} + x\beta_j$, with an intercept and its own coefficients for every category but the reference. Choose the reference category in the dialog (the first category by default).

A column whose measurement level is nominal starts as a nominal model, any other as an ordinal one; you can switch either way. Predictors and model effects (interactions, powers, keeping the model hierarchical) are chosen as for the linear model; categorical predictors are coded the same way.

Report:

- *Model Summary:* the categories, −2 log-likelihood with and without the predictors, the whole-model likelihood-ratio χ^2, McFadden's and Nagelkerke's pseudo-R^2, AIC, BIC, and a warning when the fit did not converge (usually because a predictor separates the categories).
- *Effect Pareto* and *Likelihood-Ratio Tests*: each effect tested by refitting the model without it (for a nominal model the test covers the effect in every equation, so its degrees of freedom are multiplied by the number of equations).
- *Parameter Estimates* with standard errors, Wald z tests and 95% intervals: the thresholds and coefficients (ordinal), or one block per category (nominal).
- *Odds Ratios* with intervals: for the ordinal model, the factor by which the odds of a higher category change per unit of a continuous predictor, or against a categorical predictor's reference level; for the nominal model, the odds of each category against the reference category.
- *Parallel-Lines Test* (ordinal only): the score test of the proportional-odds assumption. A small p-value says the effects differ between the cumulative splits; consider the nominal model then.
- *Classification:* counts of each actual category by the most likely predicted one, with the share correct.
- *Predicted Probabilities* against the first continuous predictor, one curve per category, with the other predictors at their mean or most frequent level. Each row is drawn at its fitted probability of the category it has, and can be brushed.
- *Prediction Formulas* for each category's probability.



Saving: each category's probability ("Prob *y* = *category*") and the most likely category ("Most likely *y*"), all as formula columns that follow later edits to the predictors.

Try it: Service ratings (ordinal) and Commute modes (nominal), two invented datasets.

Mixed model

Model ▸ Mixed model… fits linear and generalized linear mixed models.

- **Random effects:** random intercepts, and random slopes that are correlated or independent, for grouping columns that are crossed or nested.
- **Families:** normal (REML or ML, with Satterthwaite t and F tests), binomial, Poisson, Gamma, negative binomial, and **ordinal** (cumulative logit link).
- **Estimation** for non-normal families: the Laplace approximation, or adaptive Gauss–Hermite quadrature when there is a single random intercept.
- **Frequency and weight:** a *frequency* column (whole numbers) makes each row count as that many identical observations; a *weight* column gives prior weights, so the residual variance of a row is σ^2/w (normal) or its dispersion is divided by w (Poisson, Gamma, negative binomial). Weights are not offered for the binomial family, whose

number of trials already plays that role. An **offset** column (GLMMs) is added to the linear predictor with its coefficient fixed at 1, such as log exposure in a Poisson rate model.

Normal-family options (under *More options* unless noted):

- **Residual variance differs by** (in the main dialog): a grouping column whose levels each get their own residual variance, $\sigma^2 \cdot \delta^2_g$ with $\delta = 1$ for the first level. The report adds a *Residual variance test* section: the likelihood-ratio test of equal variances against the fit with one variance (χ^2 with one fewer degree of freedom than levels; both fits use the same REML or ML criterion), and the variance table lists the residual variance of each level.
- **Denominator df:** *Satterthwaite* (default) or *Between–within*. The between–within method gives each fixed effect the degrees of freedom of the outermost grouping level at which it varies: a column that is constant within each group of a grouping level (and that level's coarser levels) gets that level's df, $G_\ell - G_{\ell-1} - p_\ell$, and the intercept and effects that vary within the innermost groups get the residual df, $N - G_L - p$ (Pinheiro & Bates 2000, §2.4.2). Simple and quick, it suits balanced designs; Satterthwaite's approximation is better for unbalanced data.
- **Robust standard errors, clustered on the outermost grouping:** the cluster-robust (sandwich) covariance of the fixed effects with the bias-reduced CR2 correction (Bell & McCaffrey 2002; Pustejovsky & Tipton 2018). Each coefficient's t test uses the Satterthwaite degrees of freedom of its CR2 variance, and each effect's test is the Hotelling T^2 test with its approximate F distribution (HTZ). The fixed-effects table keeps the model-based standard errors in a column of their own. The robust errors hold even when the variance model is wrong, but need a fair number of clusters (20 or more is a common guide).

Ordinal responses. With the ordinal family, the response categories are ordered (natural order, or the order set in the dialog) and the model is logit $P(Y \le k) = \alpha_k - \eta$, where η holds the fixed and random effects (Agresti 2010, ch. 10). A positive coefficient means higher categories are more likely. The model has no intercept: its place is taken by the **thresholds** $\alpha_1 < \ldots < \alpha_{K-1}$, one between each pair of adjacent categories, which the report lists with standard errors. The fixed-effects table adds **odds ratios** (e^β, the factor by which the odds of a higher rather than a lower category change) with confidence limits, and a *Category probabilities* section compares, for every category, its count and observed proportion with the mean predicted probability over the rows, once including each group's random effects and once with the random effects at zero (a typical group). Saving offers one probability column per category and the most likely category. Residual plots use the expected category number (1 for the lowest category).

Report: fixed effects and tests, variance components with Satterthwaite χ^2 limits (always positive), correlations between random effects, and conditional modes (the estimated random effects). The Response Explorer works at the population level.

Comparisons (More options): **comparisons within slices** of a two-factor interaction and **slopes by group**, as for the linear model, on the fixed effects. For a normal model each slice test is an F test with Satterthwaite's denominator degrees of freedom (as R's `lmerTest::contest` with `joint = TRUE`); for other families the tests are Wald tests on the link scale.

Saving: population-level predictions, predictions including the random effects, or residuals.

Verified: against an independent dense reference implementation and the Dyestuff data; weights, offsets, CR2, unequal variances, between–within df and the ordinal model against R (lme4, lmerTest, nlme, clubSandwich and ordinal) on the Orthodont and wine data.

Try it: sample *Wine bitterness (Randall 1989) — ordinal mixed model*; sample *Plant growth (simulated) — ANCOVA, outliers* has a second mixed model with a residual variance for each fertilizer and robust standard errors.

Penalized regression

Model ▸ Penalized regression (lasso, ridge)… fits the lasso, ridge regression and the elastic net.

- The penalty λ is chosen by k-fold cross-validation, reporting both λ_min (lowest cross-validated error) and λ_1SE (the largest λ within one standard error of it). A predictor that happens to be constant within one fold's training rows gets a zero coefficient in that fold; a predictor whose values are all equal in the whole table is refused.
- **Coefficient paths** show each coefficient as λ changes.
- Coefficients are reported in the original units of the predictors.
- Response Explorer and 3-D profiler.

Saving: predicted values, or the prediction formula of the λ_1SE fit (dropped predictors are left out).

Try it: Longley economy; Customer churn.

Bayesian analysis

The **Bayesian** submenu, listed under both **Compare** and **Model**, holds four analyses that describe what the data say about a quantity as a probability distribution (the *posterior*), given a stated *prior*:

Menu item	What it does
Bayesian estimation…	Posterior of a mean, proportion or event rate; one group, two groups or paired columns. Closed form, no simulation.
Bayes factor tests…	Evidence for an effect against no effect: one-sample, paired and two-sample t tests, a correlation, a contingency table.
Bayesian linear model or ANOVA…	Every model that keeps heredity compared by Bayes factors; inclusion probabilities and model-averaged estimates.
Bayesian regression or hierarchical model…	Normal, logistic, Poisson or negative binomial regression with group effects, sampled by Markov chain Monte Carlo (MCMC).

Every report opens with what was analysed and the prior used, and every setting it shows can be changed from its **⋯ Options** menu (the report updates as you change it) or in full with **Edit analysis…**.

Reading a posterior summary

Each estimated quantity gets one row:

- **Mean, SD and median** of the posterior.
- **Equal-tailed interval** (the central 95% by default): 2.5% of the posterior lies below it and 2.5% above.
- **HPD interval** (highest posterior density): the shortest interval holding the same probability. It differs from the equal-tailed one when the posterior is skewed, as for a rate, a ratio or a standard deviation.

The mean and SD of a ratio or odds ratio do not always exist. When the second group has no events, its posterior puts so much weight near zero that the average of 1 ÷ (its proportion or rate) is infinite; the same happens to an odds ratio when every trial in the first group succeeds. The table then shows — for the mean and SD, with a note, while the median and both intervals are still defined and are the numbers to report.

Unlike a confidence interval, a 95% credible interval *is* a 95% probability statement about the quantity, given the model and the prior. Change the level under **More options** or in the report's options.

Bayesian estimation

Choose what to estimate and the design:

- **Mean** of values: one group, two groups (difference of means), or two paired columns (mean of their difference). With the default prior the posterior of a mean is a Student t centred on the sample mean, so its interval equals the classical t interval. For two groups, **Assume equal variances** pools the SDs; otherwise the posterior of the difference is the Behrens–Fisher distribution (Box & Tiao 1973), the difference of the two groups' t posteriors.
- **Proportion** of an outcome column (choose the **Event**; by default the second value in sorted order), with an optional **Frequency** column, or successes out of a **Trials per row** column. Two groups are compared by their **difference**, **ratio** or **odds ratio**.
- **Event rate**: counts with an optional **Exposure per row** column (for example years of follow-up). Two groups are compared by their rate ratio, whose posterior is an exact scaled F distribution.

Priors. The default is Jeffreys' prior: Beta(½, ½) for a proportion, Gamma(½, 0) for a rate and $1/\sigma^2$ for a normal mean. A proportion or rate can instead take the flat prior (Beta(1, 1), Gamma(1, 0)) or an **informative** beta or gamma prior given by its mean and SD. A mean's informative prior is a prior mean worth a chosen number of observations.

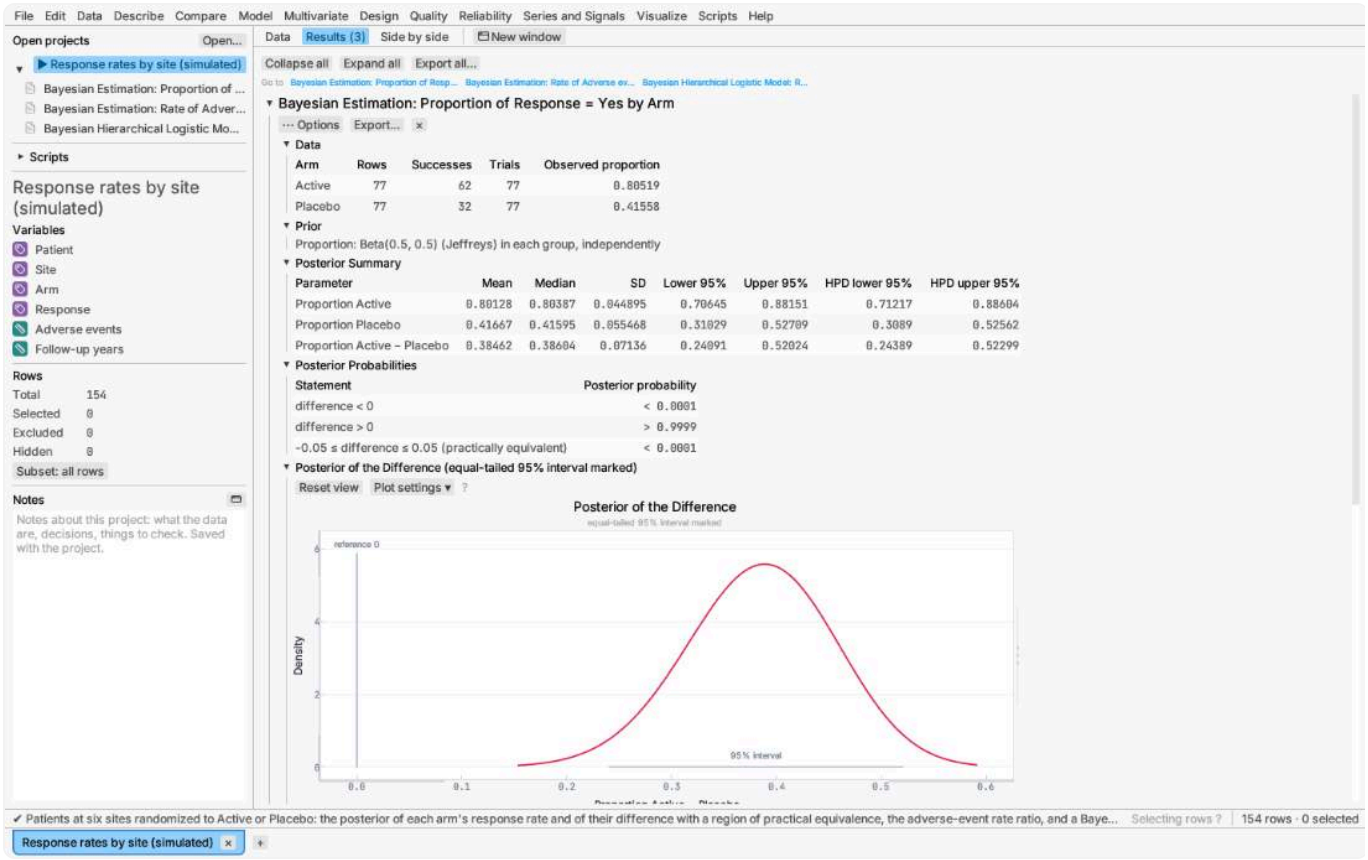
Report sections:

- **Data**: counts, totals or sample statistics per group.
- **Prior** and **Posterior Summary** (for two groups, each group and the contrast).
- **Posterior Probabilities**: the probability that the quantity (or contrast) is above and below the reference value (one group: 0 for a mean, ½ for a proportion, 1 for a rate, or a value you set), and, with a **region of practical equivalence** (ROPE), the

probability of lying inside it. A ROPE states which values count as no practical difference, such as proportions within ±0.05. With a proper (informative) prior the heading also gives the Bayes factor against the reference value, by the Savage–Dickey density ratio (prior density ÷ posterior density there).

- **Prior and posterior density** plot, with the credible interval marked and the prior as a dashed band; for two groups the groups' posteriors overlaid.

Try it: sample *Response rates by site (simulated)* (two proportions with a ROPE; the rate ratio of adverse events).



Bayes factor tests

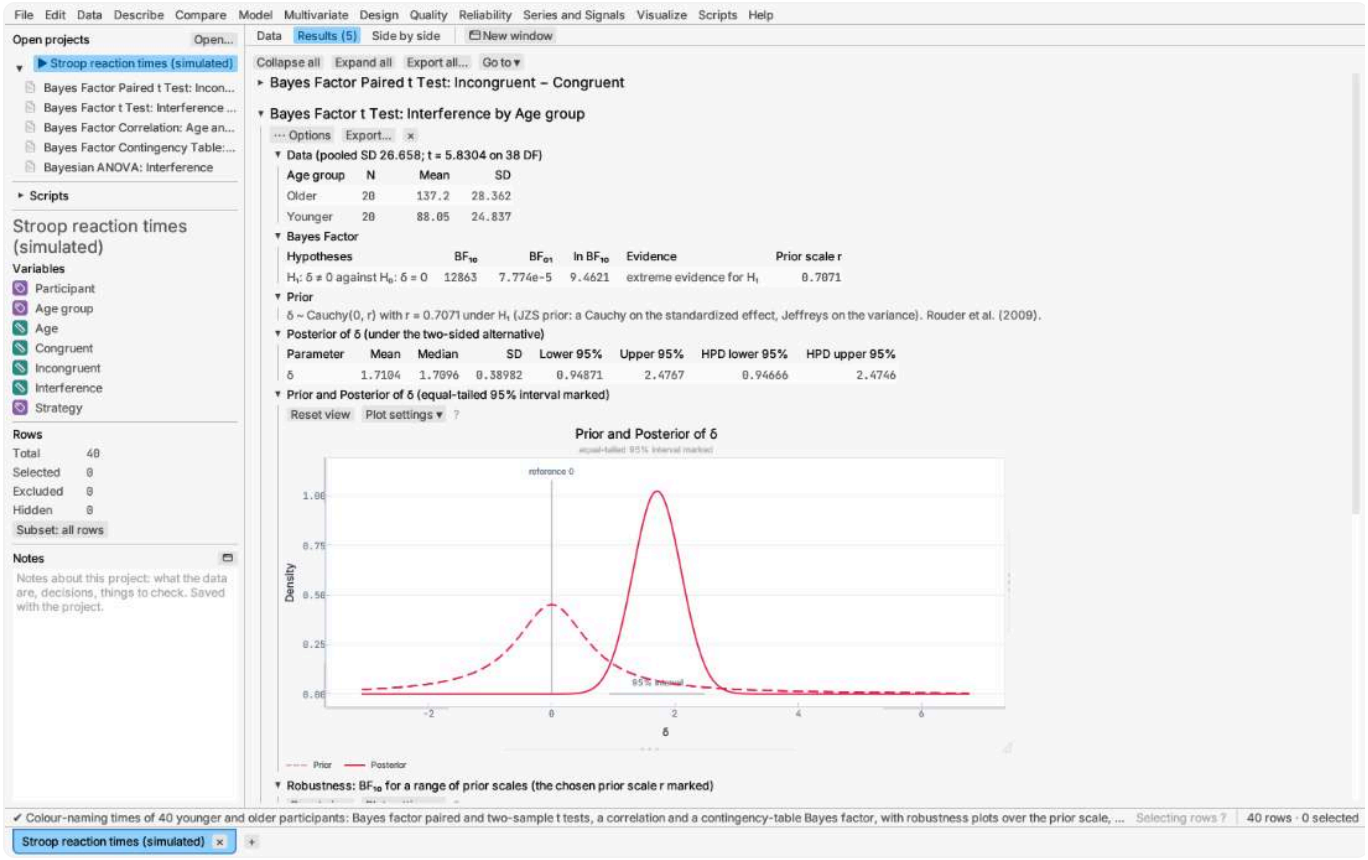
A Bayes factor BF_{10} is how many times more likely the data are under the alternative (an effect) than under the null (no effect); $BF_{01} = 1/BF_{10}$. Values above 1 favour the alternative and below 1 the null, so unlike a p-value a Bayes factor can show evidence *for* no effect. The report labels its size (anecdotal, moderate, strong, very strong, extreme; Lee & Wagenmakers 2013).

Test	Prior on the effect	Settings
One-sample, paired and two-sample t	Cauchy on the standardized effect δ with scale r (JZS; Rouder et al. 2009); r = √2/2 is "medium"	hypothesized mean (one sample), alternative (two-sided, δ > 0, δ < 0)
Correlation	stretched beta on ρ with width κ (Ly, Verhagen & Wagenmakers 2016); κ = 1 is uniform	alternative (two-sided, ρ > 0, ρ < 0)
Contingency table	Dirichlet with concentration a on the cell probabilities (Gunel & Dickey 1974)	how the table was collected: nothing fixed (Poisson), grand total fixed, row totals fixed, column totals fixed

The t tests also give the posterior of δ (median and credible interval). The **Robustness** plot shows BF_{10} across a range of prior scales with the chosen one marked, so you can see whether the conclusion depends on the prior. A one-sided alternative uses the same prior folded onto one side.

The contingency-table report gives the Bayes factor under all four sampling plans, marking the chosen one; they usually agree in direction but not in size.

Try it: sample *Stroop reaction times (simulated)* (paired, two-sample, correlation and contingency-table tests).



Bayesian linear model or ANOVA

Choose a numeric response and predictors, then the effects (interactions and powers bring in the effects they are built from). Categorical factors are coded with orthonormal contrasts, so every level effect has the same prior.

Prior (Rouder et al. 2012; Liang et al. 2008): a flat prior on the intercept and $1/\sigma^2$ on the residual variance; the continuous effects of a model together get a Zellner–Siow (Cauchy) g-prior with scale r ($\sqrt{2}/4$ is "medium"), or a hyper-g prior with parameter a; each categorical effect gets its own g with scale ½. Models start equally likely, or with a beta-binomial prior on how many effects they contain: each model size gets the same total probability, shared equally by the models of that size that keep heredity. The prior is the same whether the models are all fitted (up to 14 effects) or sampled by MC³ (more).

Report sections:

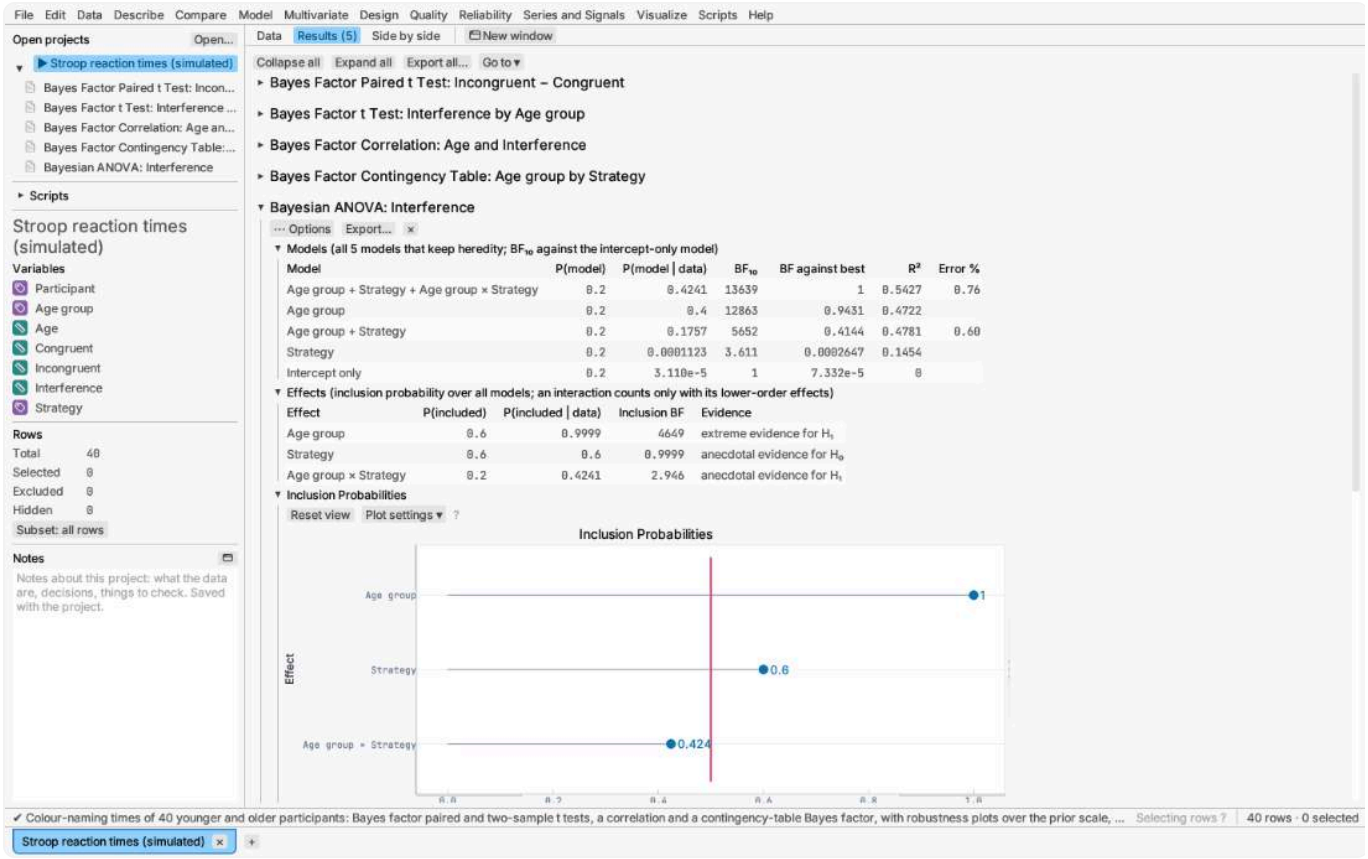
- **Models**: every model that keeps heredity (the best 20 shown), with its Bayes factor against the intercept-only model, against the best model, and its posterior probability. With one continuous block the Bayes factors are exact one-dimensional integrals;

models with several categorical effects use importance sampling over the g's, with its error shown.

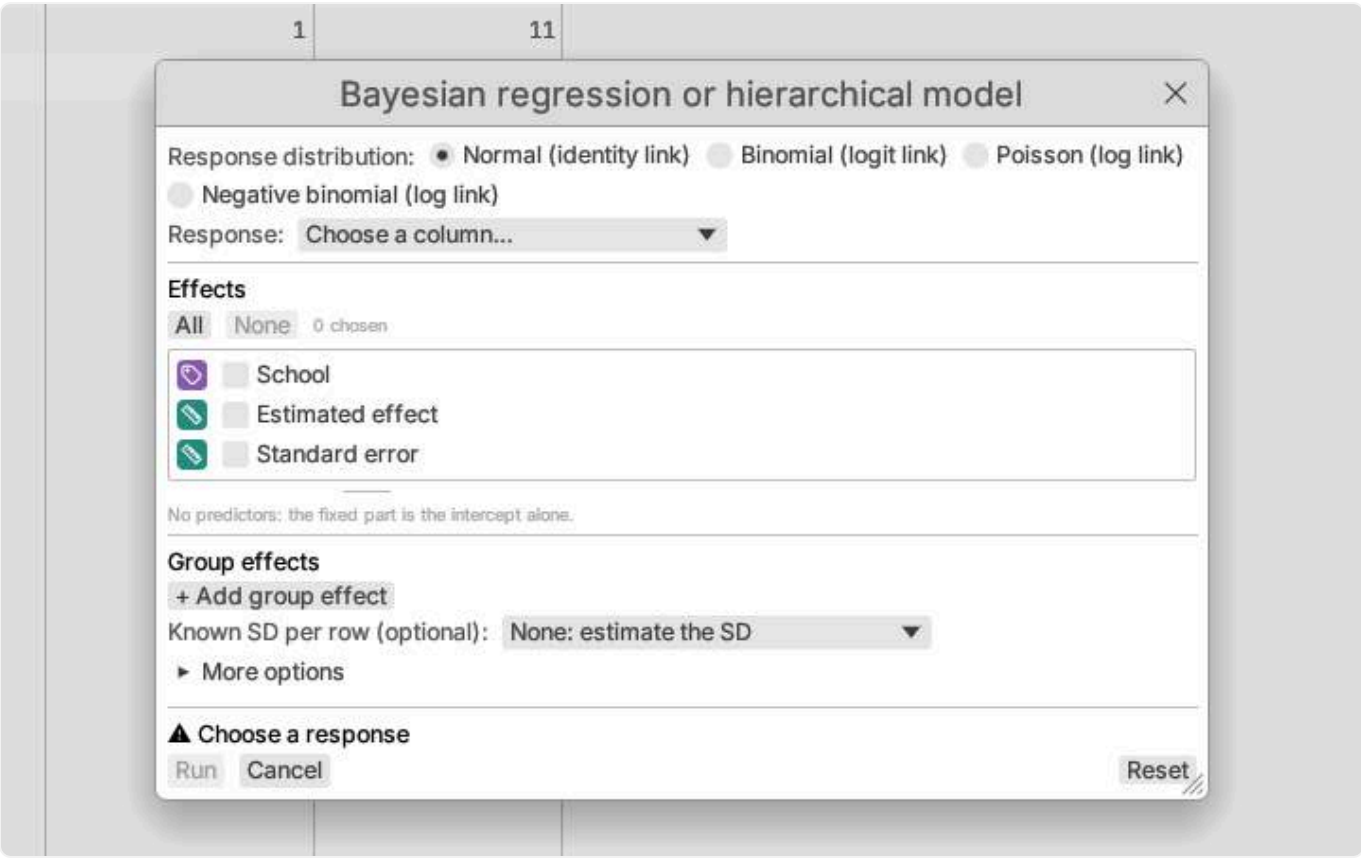
- **Effects**: each effect's prior and posterior inclusion probability and inclusion Bayes factor, and a plot of the inclusion probabilities. When the posterior probability is 1 to the precision of the arithmetic the factor shows as > 1e12.
- **Model-averaged estimates** (each coefficient averaged over the models, zero in the models without it) and **estimates in the best model**, with credible intervals.

Fit only the full model skips the comparison. The title is *Bayesian ANOVA* when every effect is categorical.

Try it: sample *Stroop reaction times (simulated)* (interference by age group and strategy, with their interaction).



Bayesian regression or hierarchical model



Choose the **response distribution** (normal, binomial with a logit link, Poisson or negative binomial with a log link), the response, the effects, and any number of **group effects**: a grouping column (optionally nested in another), with an intercept and slopes of continuous columns that vary by level. Group effects are independent normals whose SDs are estimated from the data, which pulls (*shrinks*) small or noisy groups toward the overall fit. Options:

- **Binomial:** a two-valued response (the second value is the event) or successes out of a trials column.
- **Offset** (non-normal families), such as the log of exposure time.
- **Known SD per row** (normal only): each row's standard error, for a meta-analysis; no residual SD is estimated.

Priors (under More options), weakly informative by default: coefficients Normal(0, 2.5), the intercept Normal centred at the response's mean with scale 2.5, and SDs Exponential with mean 1, all scaled to the data (multiplied by the response's SD for a normal model, divided by each predictor's SD). Coefficients may instead have a Student t or Cauchy prior, the

intercept a fixed centre, and the SDs a half-normal, half-t (3 DF) or half-Cauchy prior. Turn off **Scale the priors to the data** to use the scales as given.

Sampler. The No-U-Turn sampler (Hoffman & Gelman 2014) with step-size and diagonal mass-matrix adaptation runs 4 chains in parallel, each with 1000 warm-up and 1000 kept draws, from a fixed seed so the report reproduces exactly. Raise the **target acceptance** towards 0.99 when the report warns of divergent transitions.

Report sections:

- **Model** and **Priors** (the actual scales after autoscaling).
- **Posterior Summary** of the coefficients, SDs and dispersion, with R̂, bulk and tail effective sample sizes (rank-normalized; Vehtari et al. 2021) and Monte Carlo standard errors.
- **Group Effects**: each level's deviation from the fixed effects.
- **Sampler Diagnostics** per chain (step size, mean acceptance, divergent transitions, maximum tree depth hits, E-BFMI) and **Sampler Warnings** in plain words: divergent transitions, R̂ above 1.01, fewer than 100 effective draws per chain, or E-BFMI below 0.3.
- **Predictive Accuracy**: PSIS-LOO and WAIC (Vehtari, Gelman & Gabry 2017) with their standard errors, and how many rows have a Pareto k above 0.7 (whose LOO terms are unreliable).
- **Coefficient intervals** plot, and for one parameter (chosen in the report's options): a **trace** plot per chain, **rank** histograms per chain (flat when the chains agree), the **autocorrelation**, and its posterior density.
- **Posterior predictive check**: the observed distribution of the response against the range of datasets simulated from the posterior.

Try it: sample *Eight schools (Rubin 1981)* (the classic hierarchical normal model with known standard errors); *Response rates by site (simulated)* (a logistic model with a random intercept per site).

File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help

Open projects Open...

Eight schools (Rubin 1981)

Bayesian Hierarchical Norma...

Scripts

Eight schools (Rubin 1981)

Variables

School
Estimated effect
Standard error

Rows

Total	8
Selected	0
Excluded	0
Hidden	0

Subset: all rows

Notes

Notes about this project: what the data are, decisions, things to check. Saved with the project.

Data | Results (1) | Side by side | New window

Collapse all | Expand all | Export all...

Bayesian Hierarchical Normal Model: Estimated effect

··· Options | Export... | ×

Model

Item	Value
Response	Estimated effect
Family	Normal (identity link)
Known standard deviations	Standard error
Rows used	8
Groups: School	8 levels; effects Intercept
Sampling	No-U-Turn sampler, 4 chains × 1000 draws after 1000 warm-up iterations (seed 1)

Priors (as set)

Parameter	Prior
Intercept (at the predictors' means)	Normal(location 0, scale 5)
Standard deviations	Half-Cauchy, scale 5

Posterior Summary (equal-tailed and highest-density 95% intervals; R̂ and ESS rank-normalized)

Parameter	Mean	SD	Median	Lower 95%	Upper 95%	HPD lower 95%	HPD upper 95%	R̂	Bulk ESS	Tail ESS	MCSE
Intercept	4.4727	3.2614	4.5191	-2.0075	10.947	-2.1938	10.708	1.001	4423	2989	0.049
SD(Intercept | School)	3.5989	3.2112	2.7174	0.13152	11.937	0.00080385	9.8707	1.002	2431	1748	0.0575

Group Effects (deviations from the fixed effects, shrunk toward zero)

Parameter	Mean	SD	Median	Lower 95%	Upper 95%	HPD lower 95%	HPD upper 95%	R̂	Bulk ESS	Tail ESS	MCSE
School[A]: Intercept	1.847	4.8114	0.53757	-5.0491	14.551	-6.0177	12.967	1.000	3701	3345	0.0804
School[B]: Intercept	0.5426	3.8035	0.10923	-7.0104	9.7914	-7.6451	8.9364	1.000	3957	3435	0.0618
School[C]: Intercept	-0.5261	4.2862	-0.11788	-10.31	8.1509	-9.9739	8.3973	1.001	4380	3210	0.0692
School[D]: Intercept	0.29927	3.9137	0.044423	-7.989	9.6329	-7.4028	10.011	1.001	4746	3663	0.0587
School[E]: Intercept	-0.78138	3.7958	-0.15656	-9.9766	6.4767	-9.5882	6.796	1.000	3793	3594	0.0619
School[F]: Intercept	-0.3623	3.8509	-0.099622	-9.1343	7.4279	-8.4618	7.9869	1.003	4854	3652	0.0577
School[G]: Intercept	1.9243	4.4939	0.60389	-4.775	14.267	-5.9974	12.457	1.000	3483	3342	0.0763
School[H]: Intercept	0.36937	4.2862	0.050593	-8.3328	10.18	-9.1083	9.2946	1.002	4659	3667	0.066

Sampler Diagnostics (no problems found)

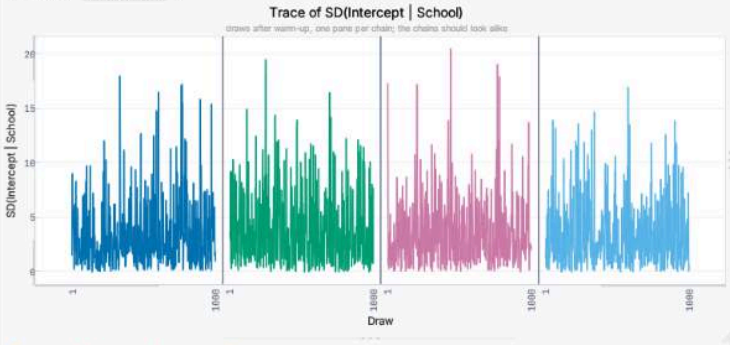
Chain	Step size	Mean acceptance	Divergent transitions	Maximum tree depth hits	E-BFMI
1	0.3695	0.931	0	0	1.088
2	0.2872	0.962	0	0	0.848
3	0.2314	0.988	0	0	0.918
4	0.1966	0.986	0	0	0.887

Predictive Accuracy (Pareto-smoothed importance-sampling LOO and WAIC; all Pareto k ≤ 0.7 (3 between 0.5 and 0.7); lower criteria are better)

Measure	Estimate	Std error
Expected log predictive density (WAIC)	-30.649	1.435
Effective number of parameters (WAIC)	0.8149	
WAIC	61.297	2.871

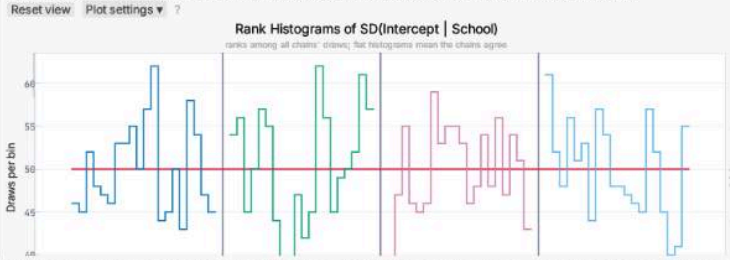
Trace of SD(Intercept | School) (draws after warm-up, one pane per chain; the chains should look alike)

Reset view | Plot settings ▾ | ?



Rank Histograms of SD(Intercept | School) (ranks among all chains' draws; flat histograms mean the chains agree)

Reset view | Plot settings ▾ | ?



✔ Coaching effects estimated in eight schools with their standard errors: a hierarchical normal model sampled with NUTS shrinks each school toward the common mean, with trace, rank and autocorrelati... Selecting rows ? 8 rows · 0 selected

Eight schools (Rubin 1981) × +

Scripting

Each analysis runs from a script with `t.analyze(name, #{...})`, using the same keys as the project file: `"bayesian-estimation"`, `"bayes-factor"`, `"bayesian-linear-model"` and `"bayesian-model"`. See the scripting reference.

References

- Box, G. E. P. & Tiao, G. C. (1973). *Bayesian Inference in Statistical Analysis*. Addison-Wesley.
- Gelman, A. et al. (2013). *Bayesian Data Analysis*, 3rd ed. CRC Press.
- Gunel, E. & Dickey, J. (1974). Bayes factors for independence in contingency tables. *Biometrika* 61, 545–557.
- Hoffman, M. D. & Gelman, A. (2014). The No-U-Turn sampler. *Journal of Machine Learning Research* 15, 1593–1623.
- Liang, F., Paulo, R., Molina, G., Clyde, M. A. & Berger, J. O. (2008). Mixtures of g priors for Bayesian variable selection. *JASA* 103, 410–423.
- Ly, A., Verhagen, J. & Wagenmakers, E.-J. (2016). Harold Jeffreys's default Bayes factor hypothesis tests. *J. Math. Psych.* 72, 19–32.
- Rouder, J. N., Speckman, P. L., Sun, D., Morey, R. D. & Iverson, G. (2009). Bayesian t tests for accepting and rejecting the null hypothesis. *Psychonomic Bulletin & Review* 16, 225–237.
- Rouder, J. N., Morey, R. D., Speckman, P. L. & Province, J. M. (2012). Default Bayes factors for ANOVA designs. *J. Math. Psych.* 56, 356–374.
- Vehtari, A., Gelman, A. & Gabry, J. (2017). Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing* 27, 1413–1432.
- Vehtari, A., Gelman, A., Simpson, D., Carpenter, B. & Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: an improved R̂. *Bayesian Analysis* 16, 667–718.

Model: decision trees and validation

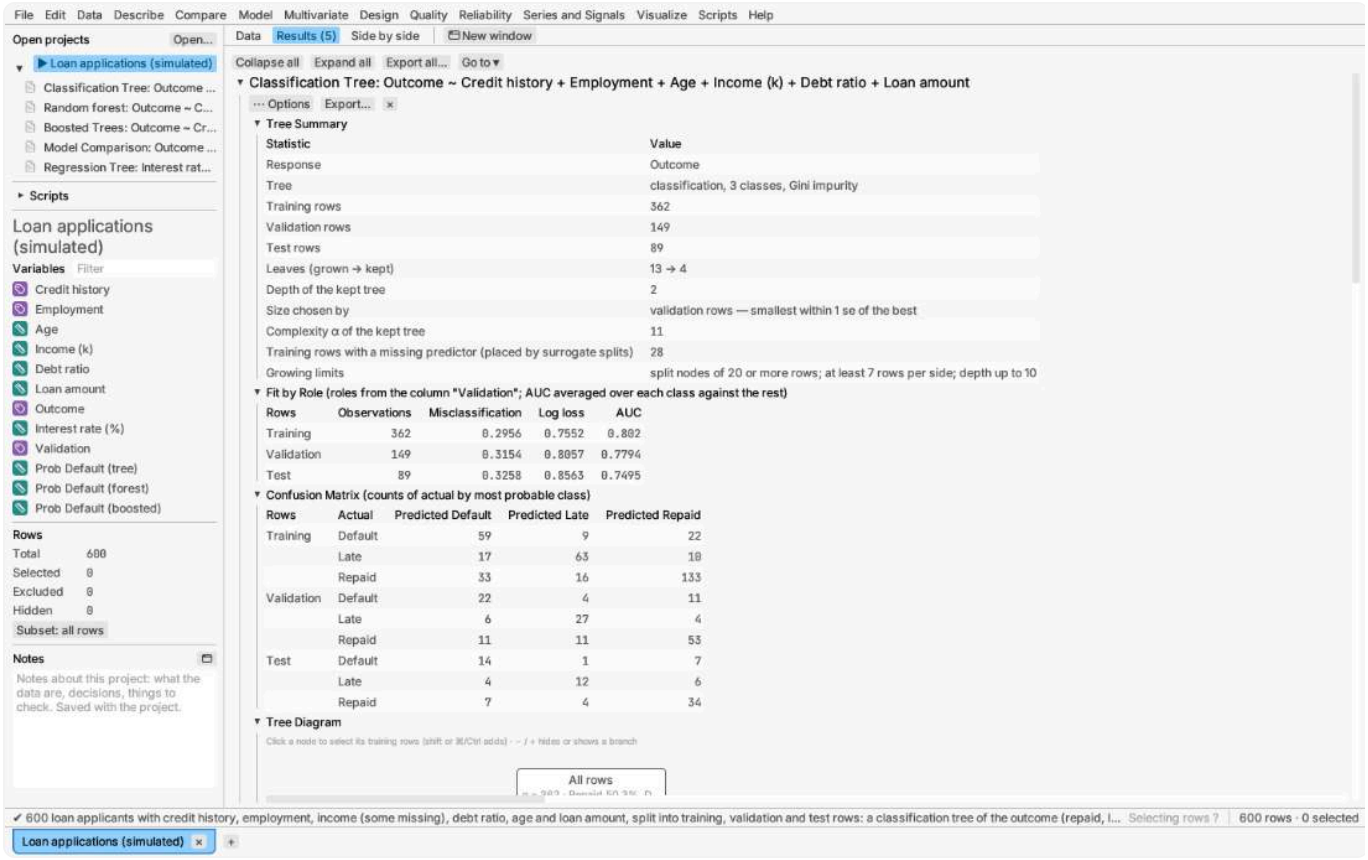
Decision tree

Model ▸ Decision tree… grows a classification tree for a categorical response, or a regression tree for a continuous one, following Breiman, Friedman, Olshen & Stone (1984).

Growing. Classification trees split by Gini impurity or entropy; regression trees by squared error. A categorical predictor splits its levels into two groups. You set the minimum node and leaf sizes and the maximum depth.

Missing values. Surrogate splits send a row with a missing predictor the way its best-agreeing other predictor goes, or the majority way.

Pruning. Weakest-link cost-complexity pruning. The tree's size is chosen on validation rows, or by cross-validation when there are none. Choose the smallest tree within one standard error of the best (the default), the tree with the lowest held-out error, or no pruning.



Report:

- summary, and fit measures for each role (training, validation, test);
- confusion matrix (classification);
- the **tree diagram**: each node shows its rule, count, prediction and a bar of class shares. Click a node to select its training rows in every view (Shift or ⌘/Ctrl adds). The − / + buttons hide or show a branch;
- variable importance;
- a leaf table with each leaf's full rule;
- held-out error against tree size, and the pruning sequence;
- the prediction formula, the Response Explorer and 3-D profiler.

Saving: predicted values, or each class's probability and the most likely class, as formula columns (nested `if` expressions that route missing values the same way the tree does). The validation roles can be saved as a column.

Try it: Loan applications: a three-class outcome with validation and test rows, and a regression tree of the interest rate sized by 10-fold cross-validation.

Validation and fit measures

Predictive models give each row a **role**:

- **Training** rows fit the model.
- **Validation** rows choose the model's size (for example, how far a tree is pruned).
- **Test** rows are kept out of both, for an honest final estimate.

Roles come from:

- a column (values 0/1/2 or Training/Validation/Test);
- a seeded random draw, optionally stratified by the response's classes (or its quartiles, for a continuous response), or by whole groups of a column so related rows stay together;
- or k-fold cross-validation, used instead of a validation set.

Fit measures per role:

- continuous response: RMSE, MAE and R^2;
- categorical response: misclassification rate, log loss, AUC (one class against the rest, averaged when there are three or more classes) and the confusion matrix.

Verified: AUC against direct counting of positive–negative pairs and the trapezoid area under the ROC curve; the pruning sequence of a small tree against hand computation; saved formulas reproduce every row's prediction, missing values included.

Model: random forests and boosted trees

Both models combine many decision trees, and both use the validation roles and fit measures described in Model: decision trees and validation. Rows missing the response, and excluded rows, are left out.

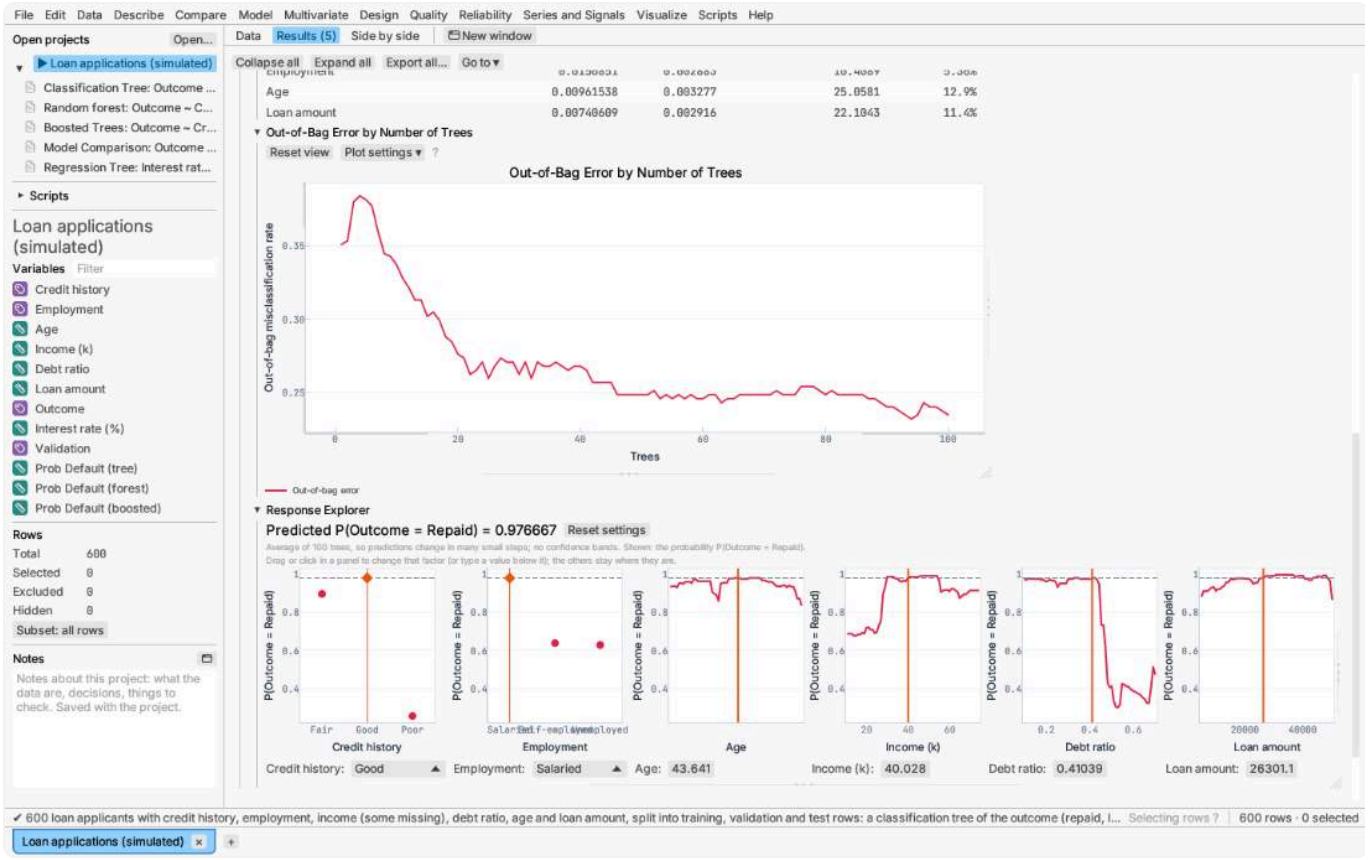
Random forest

Model ▸ Random forest… grows many trees, each on a bootstrap sample of the training rows. At every split a tree tries only a random subset of the predictors. Predictions are averaged over the trees: class probabilities for a categorical response, values for a continuous one (Breiman, 2001).

Settings: number of trees; predictors tried per split (by default √p for classification and p/3 for regression); bootstrap sample size as a share of the training rows; smallest node that may be split; smallest leaf; maximum depth; impurity (Gini or entropy) for classification; seed. Trees grow in parallel, and each tree's seed is derived from the seed you set, so the same settings always give the same forest.

Out-of-bag rows. Each tree's bootstrap sample leaves out about a third of the rows. Predicting each row only from the trees that never saw it gives a held-out error without a separate validation set, so by default every row trains.

K-fold cross-validation refits the forest without each fold in turn and reports those held-out predictions instead of the out-of-bag ones. With fewer training rows than folds, there is one fold per row (at least two). A fold whose remaining rows cannot grow a forest (for example, they hold a single class) leaves its rows without a held-out prediction rather than stopping the report.



Report:

- summary and fit measures for each role (with an out-of-bag row);
- confusion matrix (classification);
- variable importance: permutation importance (how much the out-of-bag error rises when a predictor's values are shuffled) and impurity importance;
- out-of-bag error as trees are added, so you can see when more trees stop helping;
- the Response Explorer and 3-D profiler.

Saving: predicted values, or each class's probability and the most likely class, and the validation roles.

Try it: Loan applications: a forest of the outcome on the same roles as the tree. Credit history comes out as the most important predictor.

Boosted trees

Model ▸ Boosted trees... adds small trees one at a time. Each tree is fitted to what the model so far gets wrong (the gradient of the loss), then shrunk by the learning rate before

it is added (Friedman, 2001).

Losses: squared or absolute error for a continuous response (absolute error is robust to outlying values); logistic for two classes; multinomial (one tree per class per round) for more.

Trees. Split gains and leaf values use second-order (Newton) steps with an L2 penalty on leaf values (Chen & Guestrin, 2016). Continuous predictors are binned at training-row quantiles. A categorical predictor splits its levels into two groups. Each split learns which way to send missing values.

Settings: greatest number of rounds, learning rate, tree depth, rows per leaf, L2 penalty, share of training rows used each round, rounds without improvement before stopping, and seed.

Rounds kept:

- with validation rows, boosting stops once their loss has not improved for the set number of rounds, and keeps the best round;
- with k-fold cross-validation, the number of rounds with the smallest cross-validated loss;
- without validation, every round.

Report: summary, fit measures by role, confusion matrix, variable importance (each predictor's total gain, its share and its number of splits), and the training and held-out loss by round with the kept round marked. Also the Response Explorer and 3-D profiler.

Saving: as for random forests.

Try it: Loan applications: boosted trees of the outcome, stopped early on the validation rows.

Model: partial least squares

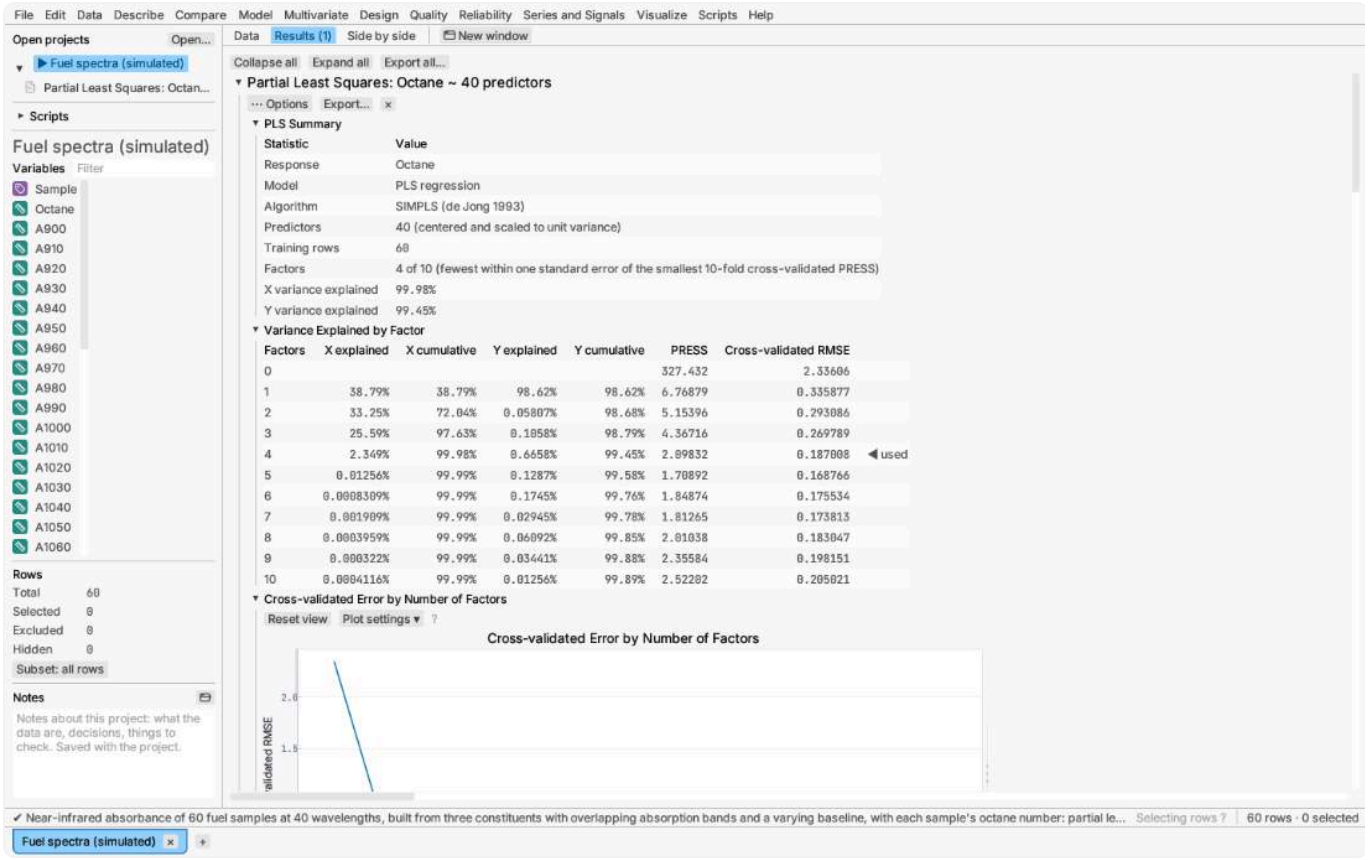
Model ▸ Partial least squares… regresses the response on a few **factors**: weighted sums of the predictors, chosen to explain both the predictors and the response (Wold; de Jong, 1993). It suits many strongly correlated predictors, such as the absorbances of a spectrum, even when there are more predictors than rows.

Predictors must be numeric. They are centered, and by default standardized to unit variance so that predictors on large scales do not dominate. Rows missing a predictor are left out.

Method: SIMPLS (the default) or NIPALS. The two agree for a single response.

Number of factors. Fix it, or let held-out rows choose it: the fewest factors whose held-out mean squared error is within one standard error of the smallest. Held-out error comes from the validation rows, or from k-fold cross-validation (10 folds when every row trains). Roles work as for decision trees.

Categorical response. PLS discriminant analysis (PLS-DA) fits one 0/1 indicator per class. The fitted indicators, clipped to [0, 1] and rescaled to sum to one, serve as class probabilities.



Report:

- summary;
- factors table: X and Y variance explained by each factor and in total, held-out PRESS and RMSE, with ◀ marking the number used;
- PRESS against the number of factors;
- fit measures by role, and the confusion matrix (PLS-DA);
- variable importance in projection (VIP): above 1 is important, below 0.8 minor;
- coefficients in the predictors' original units;
- X loadings, and a score plot of the first two factors (points grouped by class for PLS-DA; every point is a data row, so selections link);
- the Response Explorer and 3-D profiler.

Saving: predicted values with their prediction formula (continuous response), or each class's probability and the most likely class, and the validation roles.

Verified: with every factor, PLS equals least squares (to 10^{-9}); one factor matches its closed form; NIPALS and SIMPLS agree for one response; scores are orthogonal; VIP scores have mean square 1.

Try it: Fuel spectra: octane from 40 near-infrared absorbances, with the number of factors chosen by 10-fold cross-validation. Flower measurements also opens with a PLS-DA of the variety.

Model: support vector machines

Model ▸ Support vector machine… fits a maximum-margin classifier for a categorical response (C-support vector classification; Cortes & Vapnik, 1995), or ε-support vector regression for a continuous one, which ignores errors smaller than ε. The fit is found by sequential minimal optimization (Platt, 1998; Fan, Chen & Lin, 2005).

Kernels let the boundary bend:

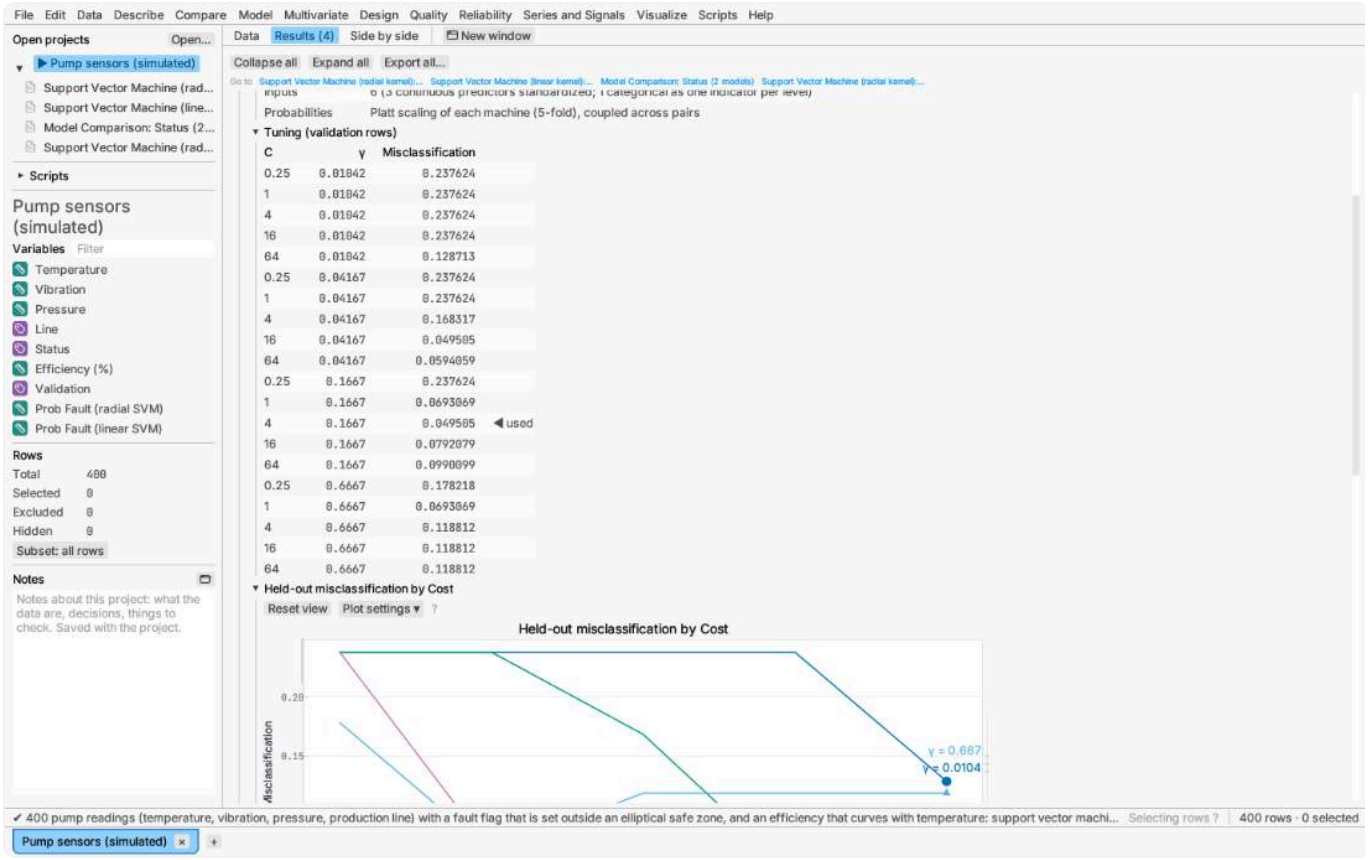
- radial basis, $\exp(-\gamma\|u - v\|^2)$ (the default): smooth, flexible boundaries;
- polynomial, $(\gamma u^T v + 1)^d$, of degree 2 to 5;
- linear, $u^T v$: a flat boundary, whose weights the report lists.

Inputs. Continuous predictors are standardized on the training rows, and each level of a categorical predictor becomes a 0/1 indicator. Rows missing a predictor are left out. For regression the response is standardized too, so ε is in response standard deviations (default 0.1).

Tuning. The cost C (how heavily margin violations count) is tried at 0.25, 1, 4, 16 and 64. For the radial and polynomial kernels γ is tried at g/16, g/4, g and 4g, where g is 1 divided by the number of inputs. The pair with the smallest held-out error wins. Held-out error comes from the validation rows, or from k-fold cross-validation (5 folds when every row trains); roles work as for decision trees. Either value can be fixed in the dialog instead.

Several classes. One machine is fitted for each pair of classes, and the class with the most votes is predicted.

Probabilities. Each machine's decision value is turned into a probability by a sigmoid fitted to cross-validated decision values (Platt scaling; Lin, Lin & Weng, 2007). With three or more classes, the pairwise probabilities are combined by pairwise coupling (Wu, Lin & Weng, 2004).



Report:

- summary: kernel and γ, cost C and how it was chosen, and the number of support vectors, including how many sit at the bound C. When a machine finds no separating direction (its best weights are zero, so it puts every row in one class), a warning says so and its probabilities are the same for every row, giving an AUC of 0.5;
- tuning table, with ◀ marking the setting used;
- held-out error against $\log_2$ C, one labelled line per γ;
- fit measures by role (cross-validated with k-fold), and the confusion matrix (classification);
- for a linear machine with a single decision function, its weights on the standardized inputs;
- the Response Explorer and 3-D profiler.

Saving: predicted values, or each class's probability and the most likely class, and the validation roles.

Verified: the closed-form maximum-margin solutions of small separable problems (weights, offset, and support vectors on the margin); ε-regression residuals within ε on exact linear data; Platt scaling and pairwise coupling recover known probabilities; the same seed gives the same model.

Try it: Pump sensors: a fault flag set outside an elliptical safe zone, fitted with radial and linear kernels and compared on the test rows (only the radial kernel can follow the curved boundary; the linear machine finds no separating line at all and says so), and an ε-regression of efficiency tuned by 5-fold cross-validation.

Model: adaptive regression splines

Model ▸ Adaptive regression splines… fits multivariate adaptive regression splines (Friedman, *Annals of Statistics* 19, 1991) to a continuous response. The model is a sum of *hinge functions*

- max(0, x − t), zero below the knot t and rising above it, and
- max(0, t − x), its mirror image,

with knots t placed at values in the data, and of products of hinges on different predictors. Each predictor's effect is therefore made of straight pieces that bend at the knots, so the model finds thresholds and changes of slope on its own, and products let one predictor's effect depend on another.

Interactions. *None (additive)* keeps every term to one predictor; *Pairs* (the default) allows products of two; *Up to three predictors* allows three.

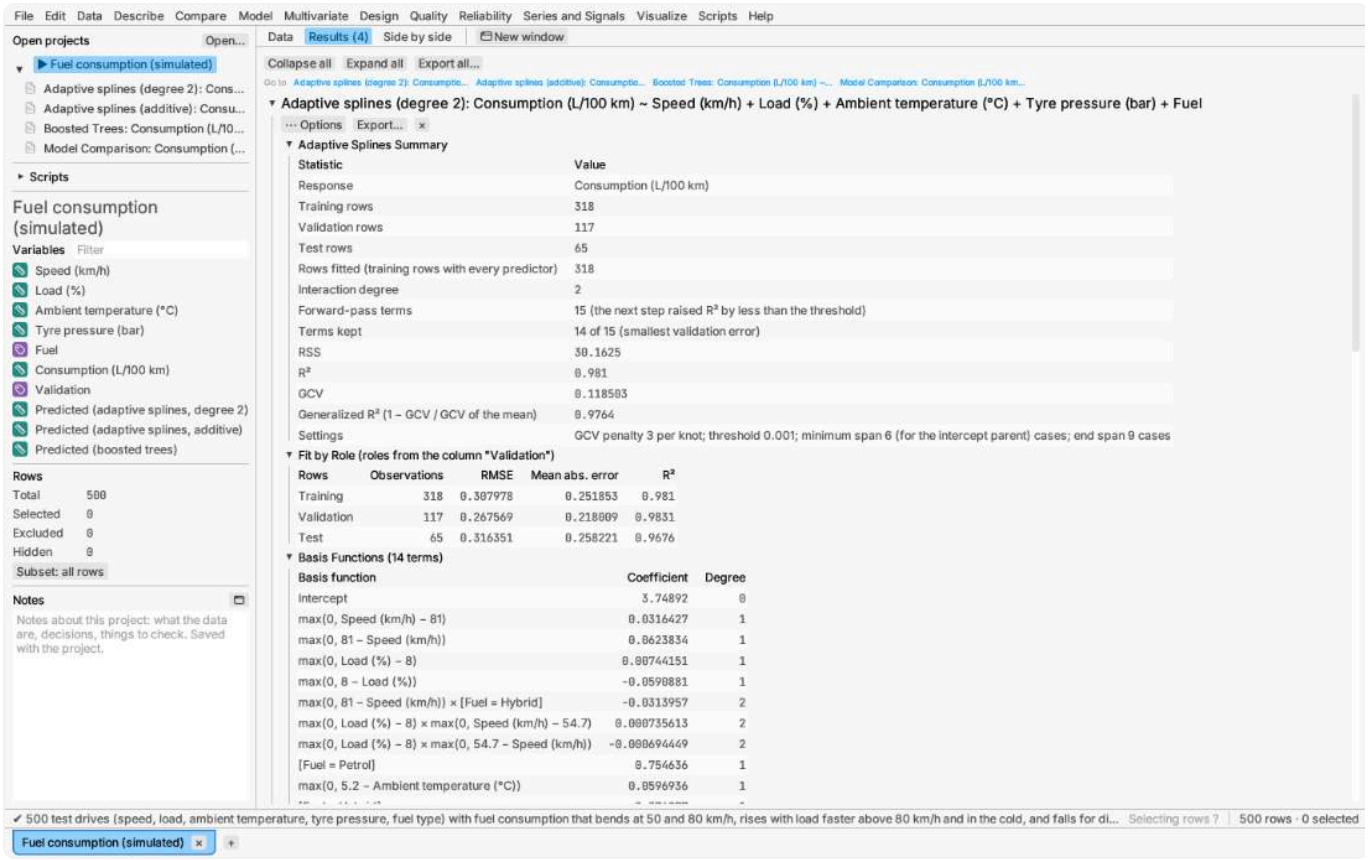
Predictors. Continuous predictors enter through hinges. Those ticked under *Enter linearly* enter only as x times another term, with no knots. A categorical predictor enters through 0/1 indicators of each of its levels except the first, so a level effect, or a level-specific slope when the indicator multiplies a hinge, is just another term. Training rows missing a predictor are left out; rows missing a predictor the final model uses get no prediction.

Forward pass. Starting from the mean, each step tries every existing term as a *parent*, every predictor not already in it, and every allowed knot, and adds the pair of terms (parent × each hinge) that most reduces the residual sum of squares. A term that adds nothing new (for example the second hinge of a predictor that already has a pair at another knot) is dropped, so a step can add a single term. The pass stops when the maximum number of terms is reached (each pair counts two; by default min(200, max(20, 2 × variables)) + 1), when the best step raises R^2 by less than the threshold (0.001), when R^2 reaches 1 − threshold, or when no step helps. Knots keep Friedman's *minimum span* (cases between knots) and *end span* (cases outside the outermost knots), which guard against knots fitted to runs of noise; both can be set under More options.

Pruning. The forward model deliberately has too many terms. The backward pass removes, one at a time, the term whose removal hurts least, giving the best model of each size. The size kept is the one with the smallest *generalized cross-validation* criterion,

$GCV = RSS / (n (1 − C/n)^2)$, with C = terms + d × (terms − 1)/2,

which charges d (2 for an additive model, 3 with interactions; settable) for each knot. With validation rows the size with the smallest validation error is kept instead; with k-fold cross-validation each fold repeats both passes without its rows and the size with the smallest cross-validated error is kept. Roles work as for decision trees.



Report:

- summary: rows by role, forward-pass terms and why it stopped, terms kept and how the size was chosen, RSS, R², GCV and the generalized R² (1 − GCV / GCV of the mean alone), and the settings;
- fit measures by role (cross-validated with k-fold);
- basis functions with their coefficients, written like `max(0, Speed (km/h) − 80.2) × max(0, 61 − Load (%))` (knot values as found in the data);
- variable importance: the number of backward-pass subsets (up to the size kept) that use each predictor, and the reductions in GCV and RSS they bring, with a relative scale (square root of the GCV reduction, largest 100);
- model size: GCV, and the validation or cross-validated error when used, against the number of terms, the size kept marked;
- the forward-pass steps (parent, terms added, R²);
- the Response Explorer and 3-D profiler: each continuous predictor's slice is piecewise linear, bending at its knots.

Saving: predicted values; the prediction formula as a formula column, a sum of coefficients times `max(Speed - 80.2, 0.0)`-style hinges (and a `switch` on the label for a level indicator), so it follows later edits and works on new rows (see formula columns); and the validation roles. Saved predictions can be compared with other models in Compare models.

Verified: functions built from known hinges are recovered exactly (knots and coefficients); additive fits agree with the R package earth to rounding, and fits with interactions to a fraction of a percent (details in the validation record).

Try it: Fuel consumption: consumption bends at 50 and 80 km/h, rises with load faster above 80 km/h, rises in the cold and is lower for diesel and hybrid cars. The degree-2 model finds the knots and the speed × load interaction and ignores tyre pressure; the additive model and boosted trees are compared with it on the test rows.

Model: neural networks

Model ▸ Neural network… fits a fully connected feed-forward neural network (a multilayer perceptron) to a continuous response or to a categorical one. The predictors feed one or two *hidden layers* of nodes; each node takes a weighted sum of the layer before it, adds a bias and passes the result through an *activation* function, and the output layer combines the last hidden layer:

- a continuous response gets one linear output node, fitted by least squares;
- two classes get one logistic node, the probability of the second class;
- three or more classes get one node per class, turned into probabilities by the softmax function.

Because every hidden node is a smooth curve of a different combination of the predictors, a network can follow curved effects and interactions without being told where they are. The price is that its weights are hard to read directly, so the report leans on the profilers to show what the network has learned.

Hidden layers and activation. Choose one or two layers and the number of nodes in each (three nodes in one layer by default). The activation is the hyperbolic tangent tanh(z) (the default), the logistic function 1 / (1 + e^(−z)), the rectified linear function max(0, z), or the identity (which makes the whole network linear, useful as a check).

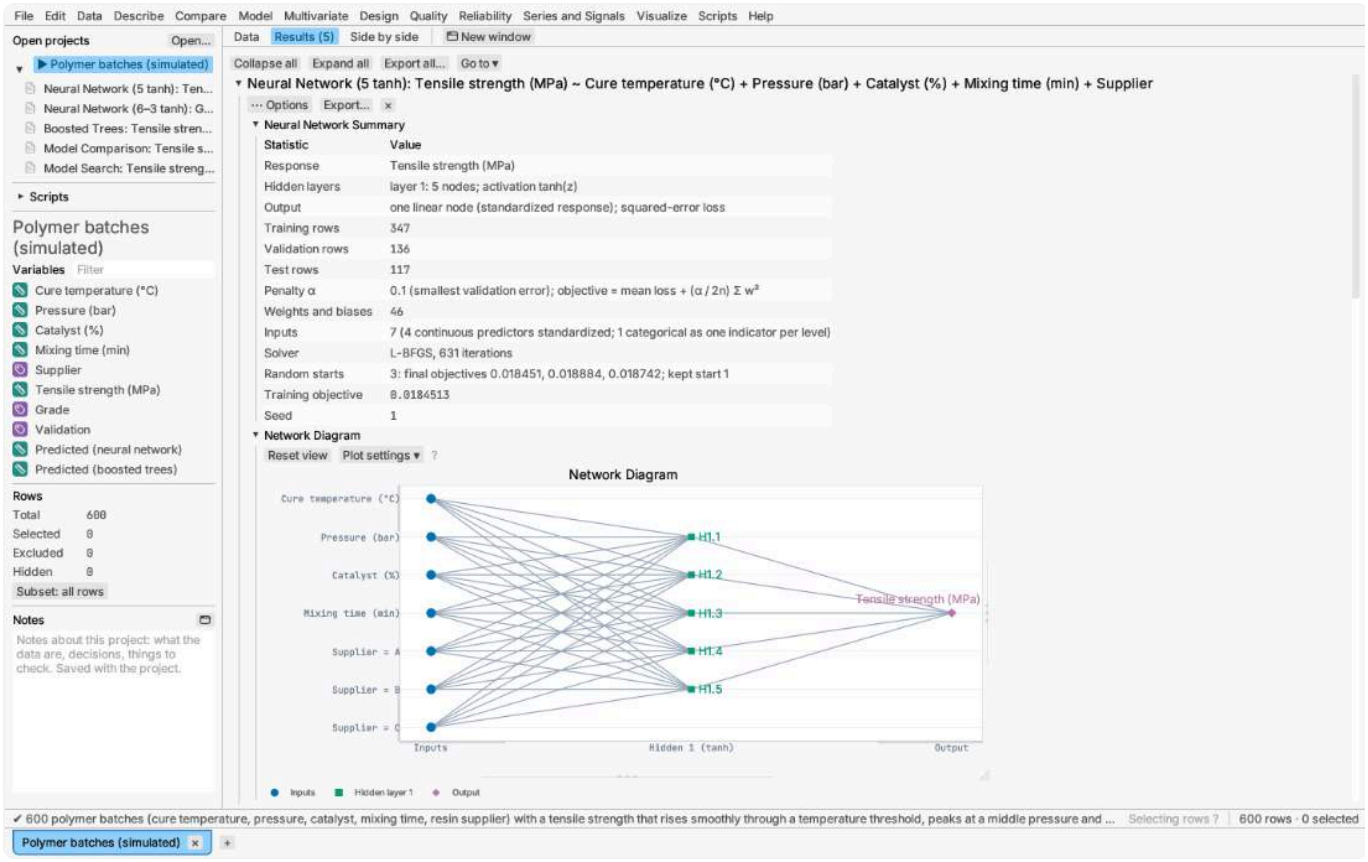
Inputs. Continuous predictors are standardized with the training rows' mean and standard deviation; a categorical predictor becomes one 0/1 indicator per level. A continuous response is standardized too, and every prediction is returned in its units. Rows missing a predictor are left out.

Fitting. The network minimizes the mean loss over the training rows (half the squared error, or the cross-entropy for classes) plus a penalty (α / 2n) Σ w² on the size of the weights (*weight decay*; biases are not penalized). The penalty keeps the network from bending to fit noise: a larger α gives a smoother, more cautious fit.

- *Penalty.* By default α is chosen from 0.0001, 0.001, 0.01, 0.1, 1 and 10 by the smallest held-out error (root mean square error, or log loss for classes): on the validation rows, or by k-fold cross-validation (5 folds when every row trains). Ties go to the larger penalty. Under More options the penalty can be fixed instead. Roles work as for decision trees.
- *Solver.* L-BFGS, a quasi-Newton method that uses every training row at each step, or Adam, which takes many small steps on random batches of 200 rows. *Automatic* (the

default) uses L-BFGS up to 2,000 training rows and Adam above. *Most iterations* caps the L-BFGS iterations or Adam passes (2,000 by default); the summary says when a fit stopped at the cap.

- *Random starts.* A network's loss has many local minima, so it is fitted from several random sets of starting weights (3 by default) and the start with the smallest final objective is kept. Every start's objective is listed in the summary. The seed fixes the starting weights, so the same request always gives the same network.



Report:

- summary: the layers and activation, the output node and loss, rows by role, the penalty and how it was chosen, the number of weights, the inputs, the solver and its iterations, every random start's objective and the one kept;
- network diagram: the inputs (named on the left), the hidden nodes (H1.1, H1.2, ...) and the output nodes, joined by their connections; each layer has its own colour and marker shape;
- penalty tuning: the held-out error of each penalty, ◀ marking the one used, and a plot of held-out error against $\log_{10}$ α;
- fit measures by role (cross-validated with k-fold) and, for classes, the confusion matrix;

- weights: one table per layer, each row a node and each column a node feeding it (and the bias), shaded from negative to positive with the value printed in each cell;
- the Response Explorer and 3-D profiler (for classes they show the probability of one class; the second of two, or the most common of more).

Saving: predicted values, or each class's probability and the most likely class; for a continuous response, the prediction formula as a formula column (the standardized inputs, each hidden node as `tanh(…)`, `logistic(…)` or `max(…, 0.0)`, and the output in the response's units), so it follows later edits and works on new rows (see formula columns); and the validation roles. Saved predictions can be compared with other models in Compare models.

Verified: at weights fitted by scikit-learn, the objective, its gradient and the predictions agree to 10^{-10} for a tanh regression network, a two-layer logistic network for two classes and a rectified-linear softmax network for three classes, and fits from random starts reach objectives at least as small as scikit-learn's (details in the validation record).

Try it: Polymer batches: tensile strength rises through a temperature threshold, peaks at a middle pressure and gains most from the catalyst when the cure is cool. The strength network follows these curves (drag the temperature line in the Response Explorer and watch the catalyst slice flatten) and beats boosted trees on the test rows; a second network classifies each batch's grade, and a model search ranks the network against the other models.

Model: Gaussian processes and sequential optimization

A *Gaussian process* treats the unknown response surface as a random smooth function: any set of its values is jointly normal, and two settings close together have strongly correlated responses. Fitted to the runs, it gives a prediction everywhere together with a standard error, which is small near the runs and grows between and beyond them. That makes it the usual model for computer experiments and small designed experiments, and the basis of *sequential optimization*: choosing the next runs where the model expects the largest gain.

Two menu items use it:

- **Model ▸ Gaussian process…** fits the model and reports it, with a Response Explorer and contour maps.
- **Design ▸ This table ▸ Propose next runs…** fits one model per response to the runs made so far and proposes the next batch, which you can add to the table as new rows.

The model

The response is standardized (the training mean subtracted, divided by the standard deviation) and modelled as

$y(x) = f(x) + \varepsilon$, with $\mathrm{Cov}(f(x), f(x')) = \sigma^2_f \cdot r(x, x')$ and $\varepsilon \sim N(0, \sigma^2_n)$,

where the correlation r depends on the scaled distance

$d = \sqrt{\sum_j ((x_j - x'_j) / \ell_j)^2}$.

Inputs are rescaled to 0–1 over their observed range first; a categorical predictor becomes one 0/1 indicator per level, each with its own length scale.

Kernel. Two correlation functions are offered:

- *Matérn 5/2* (the default): $r = (1 + \sqrt{5}\, d + 5d^2/3)\, e^{-\sqrt{5}\, d}$. Its surfaces are twice differentiable, which suits most physical responses and rarely overshoots between runs.

- *Squared exponential*: r = e^(−d²/2). Infinitely smooth surfaces, suited to simulators whose response changes gently.

Length scales. Each input has its own length scale ℓ_j (*automatic relevance determination*): a short length scale means the response changes quickly with that input; a very long one (at the upper bound of 1000 times the range) means the input has no detectable effect.

Nugget. The noise variance σ^2_n, as a fraction of the response's variance, is estimated by default. Under the dialog's options it can be fixed instead; a tiny value such as 10^{-6} makes the surface pass through every run, as suits a deterministic simulator.

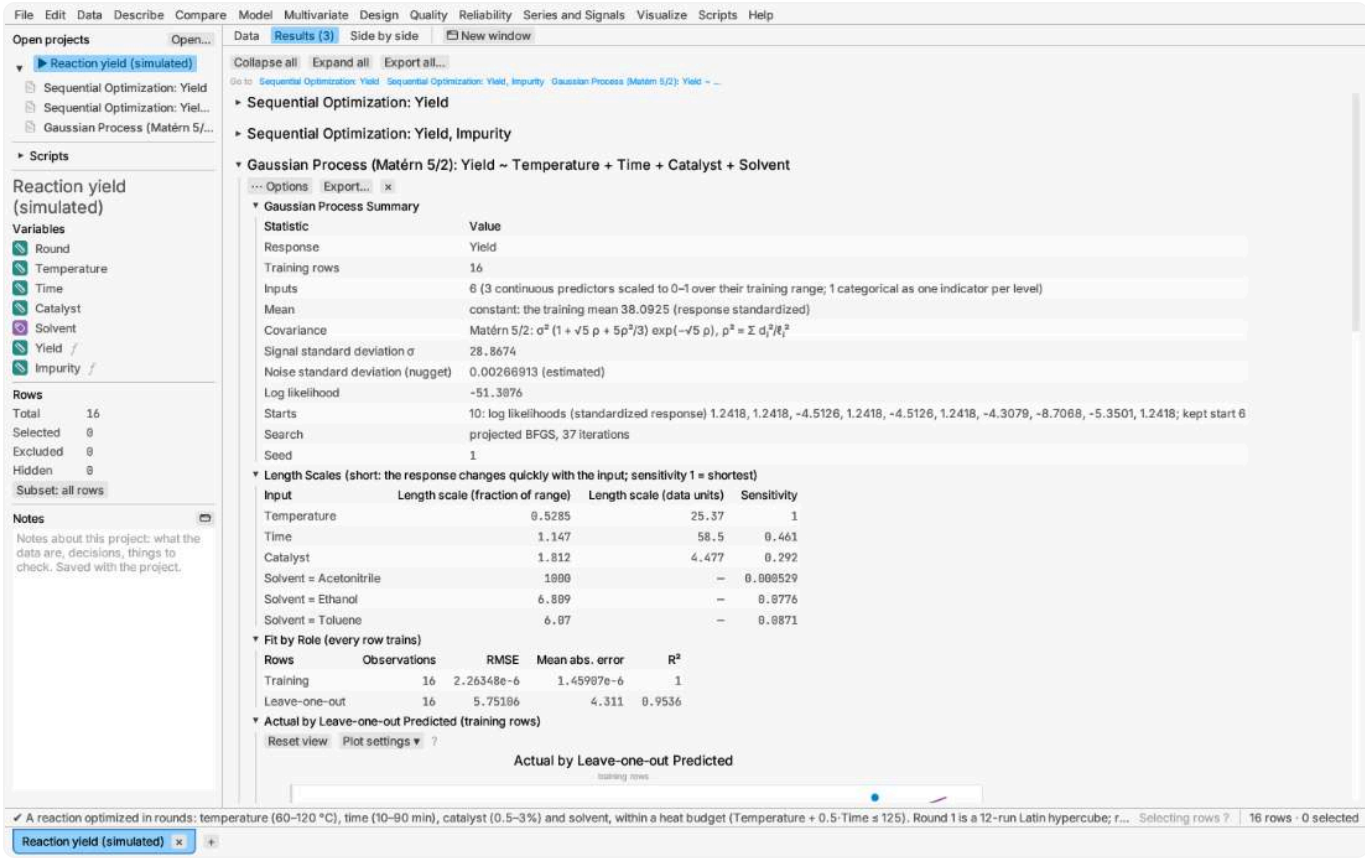
Fitting. The signal variance, the length scales and the nugget maximize the log marginal likelihood

$$\log p(y \mid \theta) = -\tfrac{1}{2}\, y^{\mathsf{T}} K^{-1} y - \tfrac{1}{2} \log |K| - (n/2) \log 2\pi,$$

with K the covariance matrix of the runs. The search works on the logarithms of the parameters, within $\sigma^2_f \in [10^{-3}, 10^{3}]$, $\ell \in [10^{-3}, 10^{3}]$ and $\sigma^2_n \in [10^{-8}, 10]$, by a quasi-Newton (BFGS) method with the exact gradient $\tfrac{1}{2}\, \mathrm{tr}((\alpha\alpha^{\mathsf{T}} - K^{-1})\, \partial K/\partial\theta)$, $\alpha = K^{-1}y$. The likelihood often has several local maxima, so the search starts from several points spread by a Latin hypercube (10 by default, set under More options with the seed) and keeps the best; every start's log likelihood is listed in the summary. Up to 2,000 rows can be fitted.

Predictions. At a new setting x, with k the vector of covariances with the runs, the predicted mean is $k^{\mathsf{T}}K^{-1}y$ and the variance of the mean $\sigma^2_f - k^{\mathsf{T}}K^{-1}k$ (both returned in the response's units). Intervals use ±1.96 standard errors.

Validation. By default each row's held-out prediction is its *leave-one-out* prediction, computed exactly without refitting (Rasmussen & Williams 2006, eq. 5.12). Validation roles from a column, random holdout and k-fold cross-validation work as for decision trees.



Report:

- summary: rows by role, the inputs, the kernel and its formula, the signal and noise standard deviations (the nugget, estimated or fixed), the log likelihood, and every start's log likelihood with the start kept;
- length scales: each input's as a fraction of its range and in data units, with a sensitivity (shortest length scale divided by this one, so 1 marks the input the response is most sensitive to);
- fit measures by role (RMSE, MAE, R^2), with leave-one-out or cross-validated rows;
- actual by held-out predicted plot;
- contour maps of the predicted mean and of its standard error over the first two continuous predictors, the other predictors held at the Response Explorer's starting settings (named in the titles); the runs are marked, so the low-error valleys around them show where the model is sure;
- Response Explorer and 3-D surface profiler (from the options menu), with confidence bands from the standard error.

Save (options menu): the predicted values, their standard errors and the validation roles.

Try it: Reaction yield (simulated): the Gaussian process of yield, with its length scales showing how quickly yield changes with each factor.

Proposing the next runs

Design ▸ This table ▸ Propose next runs… works on a table with one row per run: the factor settings and the measured responses. Rows with every factor set but a response still missing are *runs in progress*: they are not used to fit the models, and the proposals keep away from them.

Responses and goals. Each response is maximized, minimized or matched to a target. With one response the criterion works on it directly; with several, they are combined into an overall desirability (the geometric mean of each response's desirability, weighted by importance; Derringer & Suich 1980), each scaled between its smallest and largest observed values unless limits are set under More options.

Factors. Continuous factors are searched between the table's smallest and largest settings, or a range set under More options; categorical factors take their levels. Linear constraints and disallowed combinations are typed in the factors' units, one per line, as for optimal designs: for example `Temperature + 0.5*Time <= 125` or `exclude Temperature > 110 and Catalyst > 2.5`.

Round column. An optional column numbering the batches of runs; the progress chart then shows the best value after each round, and proposed runs get the next round number.

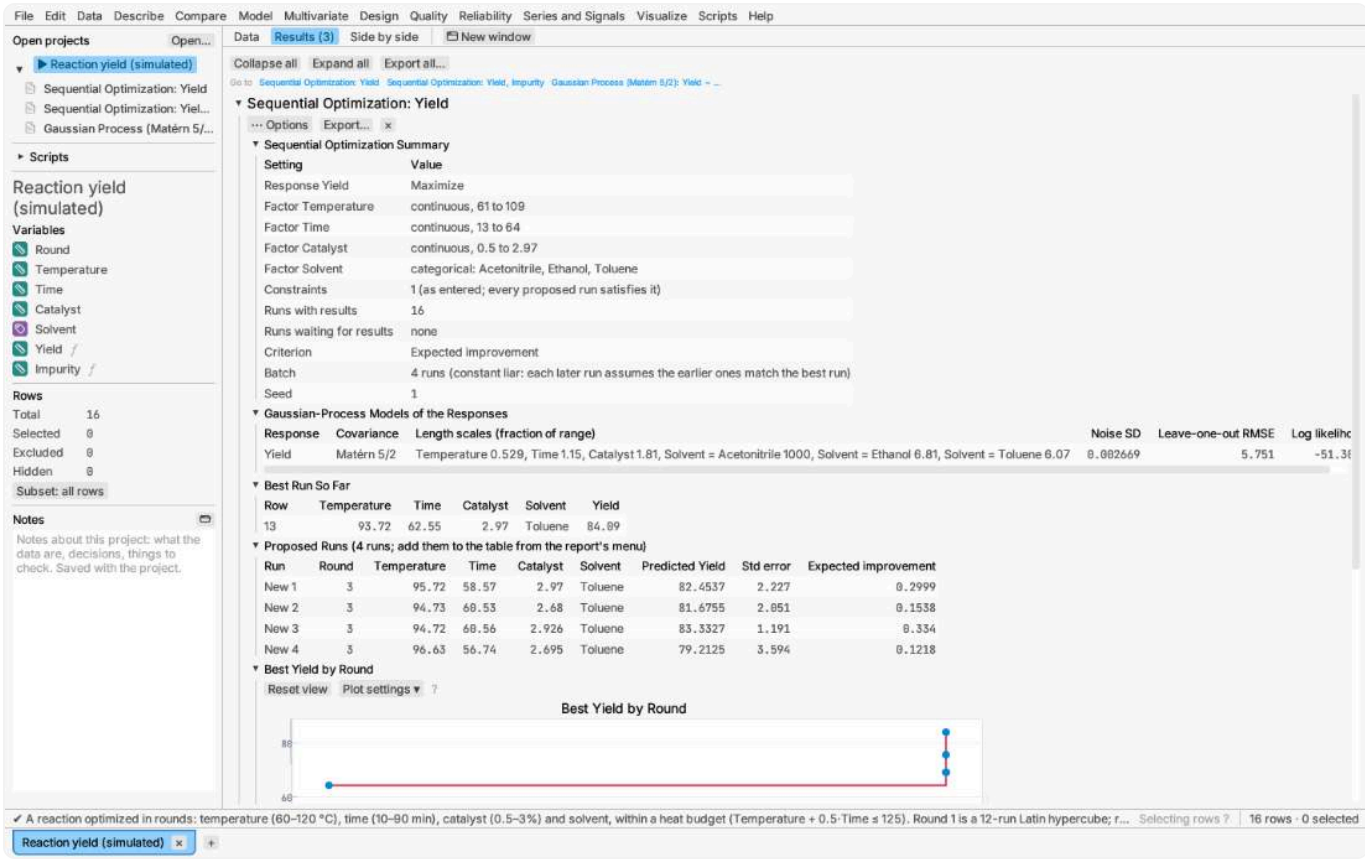
Criterion. Each candidate setting is scored from the model's predicted mean μ and standard error s, with f* the best run so far:

- *Expected improvement* (the default; Jones, Schonlau & Welch 1998): E[max(0, Y − f*)] = (μ − f*) Φ(z) + s φ(z), z = (μ − f*) / s, when maximizing (mirrored when minimizing). For a target T, the improvement is in the distance from it, E[max(0, b − |Y − T|)] with b the best run's distance, which also has a closed form. For several responses the improvement in overall desirability is averaged over 256 draws from the models, the same draws for every candidate.
- *Upper confidence bound* (Srinivas et al. 2010): μ + κs (μ − κs when minimizing), with κ standard errors (2 by default). A larger κ explores more.

Batches. A batch of runs (4 by default) is chosen one run at a time by the *constant liar* heuristic (Ginsbourger, Le Riche & Carraro 2010): after each proposal the models are updated as if that run had come out at the best value so far, which lowers their uncertainty there and moves the next proposal elsewhere. Runs in progress are treated the same way first.

Search. For each run, 2,000 random settings (a Latin hypercube over the continuous factors, rejecting those outside the constraints, with random levels) are scored; the best five are refined by a bounded Nelder–Mead search and by trying every level of each categorical factor.

Stop rule. The report says *stop* when the best run reaches a value you set (a response value, a distance from the target, or an overall desirability), or when the largest expected improvement falls below a fraction of the response's observed range (1% by default; of desirability 1 for several responses). Otherwise it says *continue*.



Report:

- summary: responses and goals, factors and their ranges, the number of constraints (their text stays in the dialog, opened with Edit), the runs with results and the runs in progress, criterion and batch;
- the model of each response: kernel, length scales, noise SD, leave-one-out RMSE, log likelihood;
- the best run so far;
- the proposed runs with their round, predicted responses, standard errors and criterion values;
- progress: each run's value by round (or by run) with the best so far as a step line;

- the stop rule's verdict.

Settings and adding the runs (options menu): the batch size, criterion and stop rule are edited in place, and the report updates as you change them. *Add proposed runs to the table* appends the proposed runs as new rows with their factor settings and the next round number, the responses left empty; Undo removes them. Measure the responses, type them in, and the report proposes the next batch.

Try it: Reaction yield (simulated): round 1 is a 12-run Latin hypercube, round 2 the four runs the yield report proposed. Its yield is a formula column standing in for the experiment, so the added runs are measured at once: add runs a few times and watch the best yield approach the optimum under the heat budget. A second report trades yield against impurity by desirability.

References. Rasmussen, C. E. & Williams, C. K. I. (2006) *Gaussian Processes for Machine Learning*, MIT Press. Santner, T. J., Williams, B. J. & Notz, W. I. (2018) *The Design and Analysis of Computer Experiments*, 2nd ed., Springer. Jones, D. R., Schonlau, M. & Welch, W. J. (1998) Efficient global optimization of expensive black-box functions, *Journal of Global Optimization* 13, 455–492. Srinivas, N., Krause, A., Kakade, S. & Seeger, M. (2010) Gaussian process optimization in the bandit setting, *ICML*. Ginsbourger, D., Le Riche, R. & Carraro, L. (2010) Kriging is well-suited to parallelize optimization, in *Computational Intelligence in Expensive Optimization Problems*, Springer. Henkenjohann, N. & Kunert, J. (2007) An efficient sequential optimization approach based on the multivariate expected improvement criterion, *Quality Engineering* 19, 267–280.

Model: model search

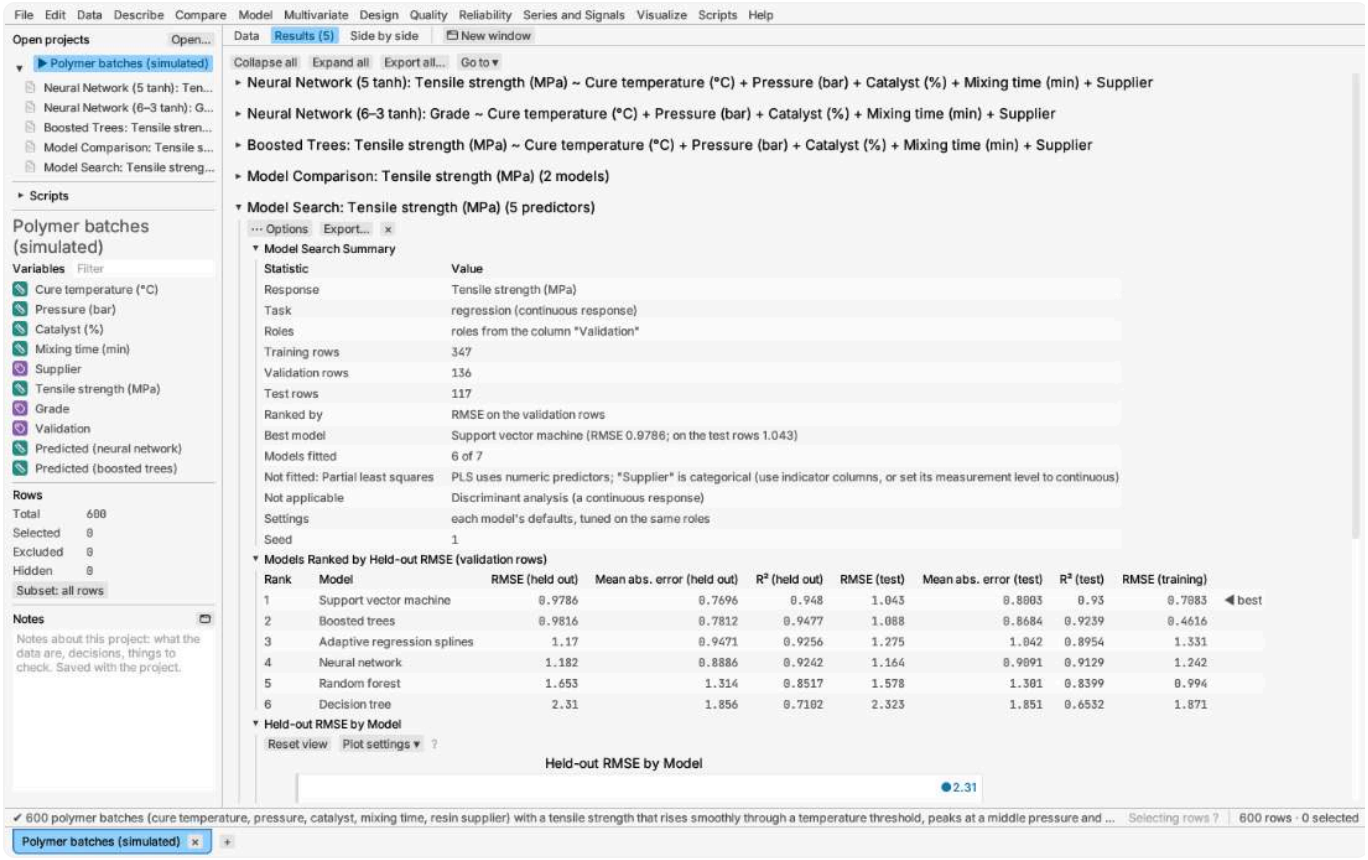
Model ▸ Model search… fits every kind of predictive model that suits the response, each with its own default settings and the same validation roles, ranks them by their error on rows they were not fitted to, and opens the best one's full report below the ranking.

Models. Decision tree, random forest, boosted trees, neural network, support vector machine, adaptive regression splines (continuous response only), partial least squares, and discriminant analysis (categorical response only, numeric predictors). Untick any under *Models to try*; those that do not suit the response are greyed out. Each model runs exactly as its own report would with default settings, so the report it opens shows the same numbers as its row in the ranking. A model that cannot be fitted to the data (for example discriminant analysis with a text predictor) is listed in the summary with the reason.

Ranking. Each model is tuned and ranked on held-out rows:

- *Hold back random rows* or *Roles from a column*: the validation rows tune every model and rank them; test rows are used by none of them, so the test columns of the ranking are an honest estimate for new data. A search needs some validation rows (test rows alone cannot rank without biasing the test).
- *K-fold cross-validation*, or *None (all rows train)*, which uses 5 folds: each row's held-out prediction comes from the model fitted without its fold.

The measure is the root mean square error for a continuous response. For a categorical one it is the log loss (the mean of −ln of the probability given to the observed class), which rewards probabilities that are both right and well calibrated and separates models the misclassification rate often ties; ties go to the lower misclassification rate. The seed (under More options) seeds every model, so a search repeats exactly.



Report:

- summary: the task, rows by role, the measure, the best model with its held-out and test error, models fitted, models that could not be fitted (with the reason) or do not apply;
- the models ranked by held-out error (◀ marks the best), with the held-out and test measures (RMSE, mean absolute error and R^2, or misclassification, log loss and AUC) and the training error;
- held-out error by model, best at the left.

Repeated and nested cross-validation. One split into folds is a single draw: with another seed the ranking can change. Under **More options**, **Repeat k-fold cross-validation** reruns the k-fold ranking that many times (up to 50) with fresh folds, the seed for repeat *i* being the search's seed plus *i*, so repeat 1 is the ordinary search. The report adds a table of each model's mean held-out error across the repeats with a 95% t interval, its standard deviation, and its best and worst repeat, and a forest plot of the means with their intervals (Kim 2009). **Nested** goes further for models that tune themselves on held-out rows: each outer fold is held out while the model tunes by cross-validation on the remaining rows only, so the outer error is not flattered by tuning on the rows it is scored on (Varma & Simon 2006). Both need k-fold cross-validation (or every row training); with validation rows the

section says so. Fitting takes the number of repeats times longer, and nested folds about the number of folds times longer again.

Opening reports. A new search opens the best model's full report below it. ⋯ Options lists every ranked model; choosing one opens its full report, with the profilers, fit measures and columns to save. To compare models on more measures and plots (ROC curves, lift, calibration), save their predictions from their reports and use Compare models.

Verified: on simulated data where the true model class is known, the true class wins: a sum of hinge functions is won by adaptive regression splines, and a curved class boundary made of tanh units by the neural network; seeded searches repeat exactly, and each ranked model matches its own report (see the validation record).

Try it: Polymer batches: the search of tensile strength ranks the six models that suit it on the validation rows.

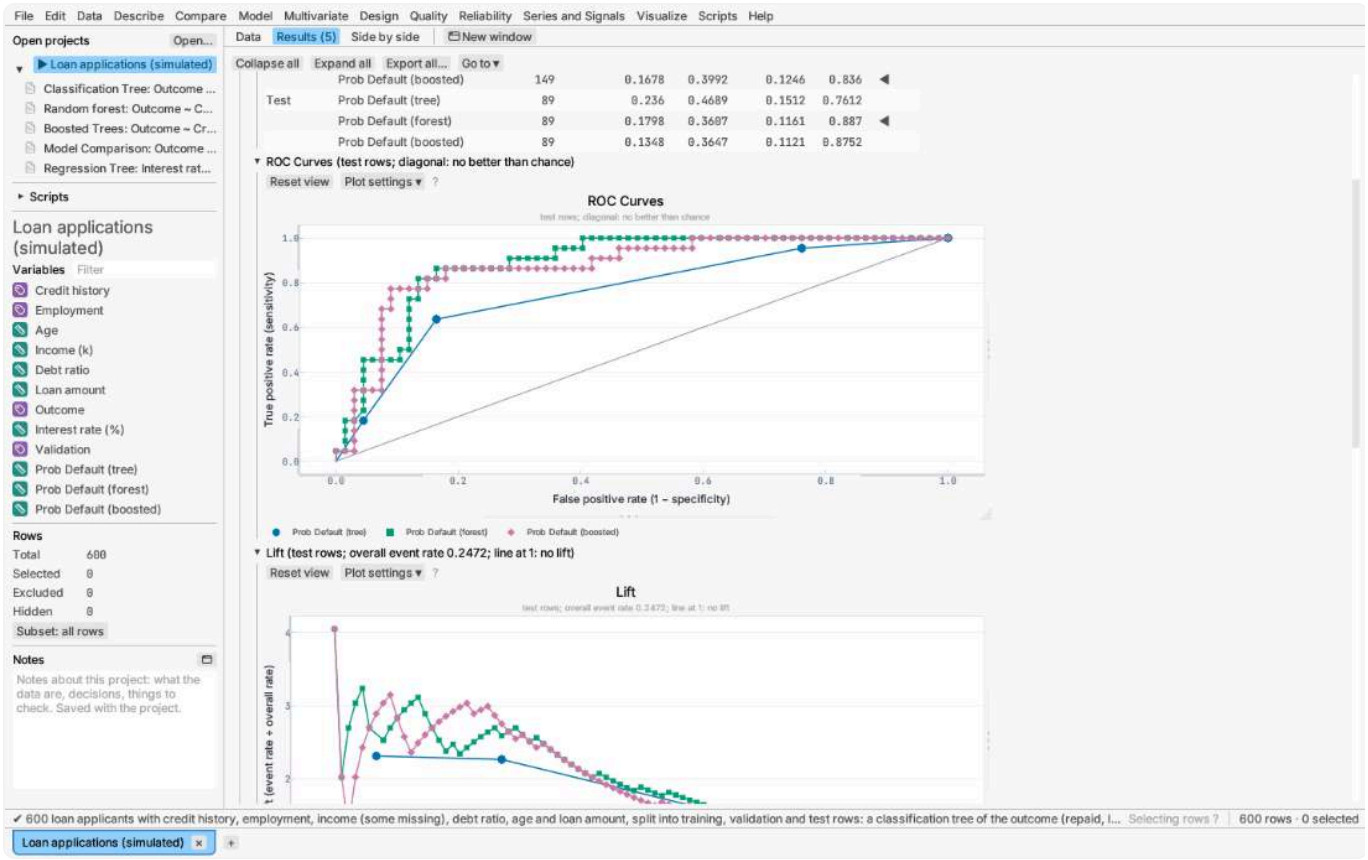
Model: comparing models

Model ▸ Compare models… compares predictive models through the prediction columns they saved (each model's options menu offers *Save* for its predictions). It uses the rows where the response and every chosen column are present.

Choose: the response, two or more prediction columns, and optionally a column of validation roles. For a categorical response, choose the columns holding each model's probability of one event class (by default the last class), which is compared against all the others.

Measures, for each role when a role column is given, with ◀ marking the best model in each (lowest RMSE, or highest AUC):

- continuous response: RMSE, MAE and R^2;
- categorical response: misclassification at a 0.5 cut-off, log loss, Brier score and AUC.



Plots use the test rows when there are any, otherwise the validation rows, otherwise all rows. Each model has its own colour and marker shape, and every point is a data row, so selecting points selects those rows everywhere.

- Categorical response: ROC curves with the chance diagonal; lift and cumulative gains by depth; a calibration plot of the observed event rate against the mean predicted probability in tenths.
- Continuous response: actual by predicted, and residual by predicted.

Every plot exports as PNG, SVG, PDF and EPS from its right-click menu.

Try it: Loan applications compares the tree's, the forest's and the boosted trees' default probabilities on the test rows. Pump sensors compares radial and linear support vector machines. Fuel consumption compares two adaptive spline models with boosted trees. Polymer batches compares a neural network with boosted trees. To fit and rank every kind of model in one step, use Model search.

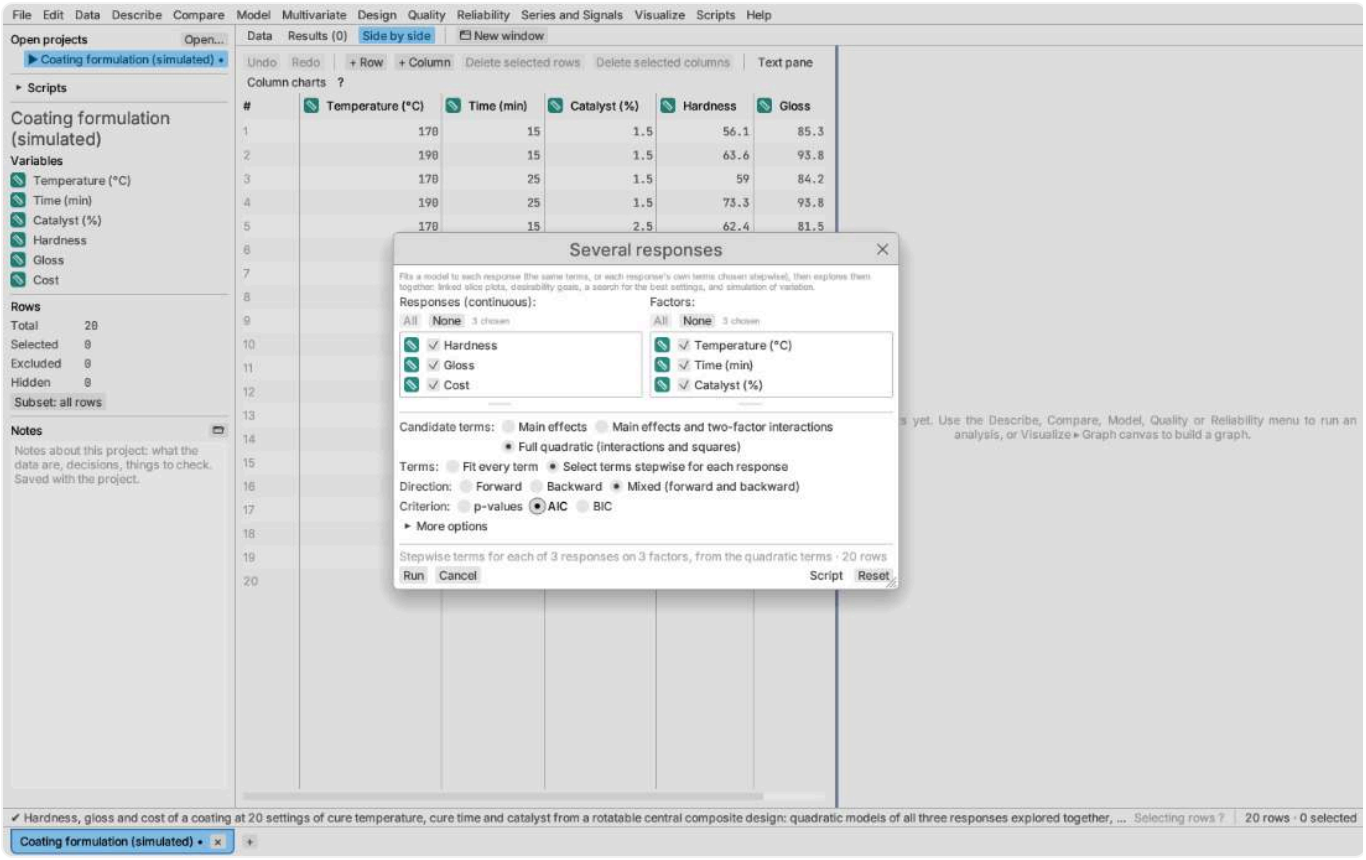
Model: several responses

Model ▸ Several responses (desirability, simulation)… fits a model to each of several responses on shared factors (main effects, with interactions, or quadratic, either in full or with each response's own terms chosen stepwise), and lets you explore and optimise them together.

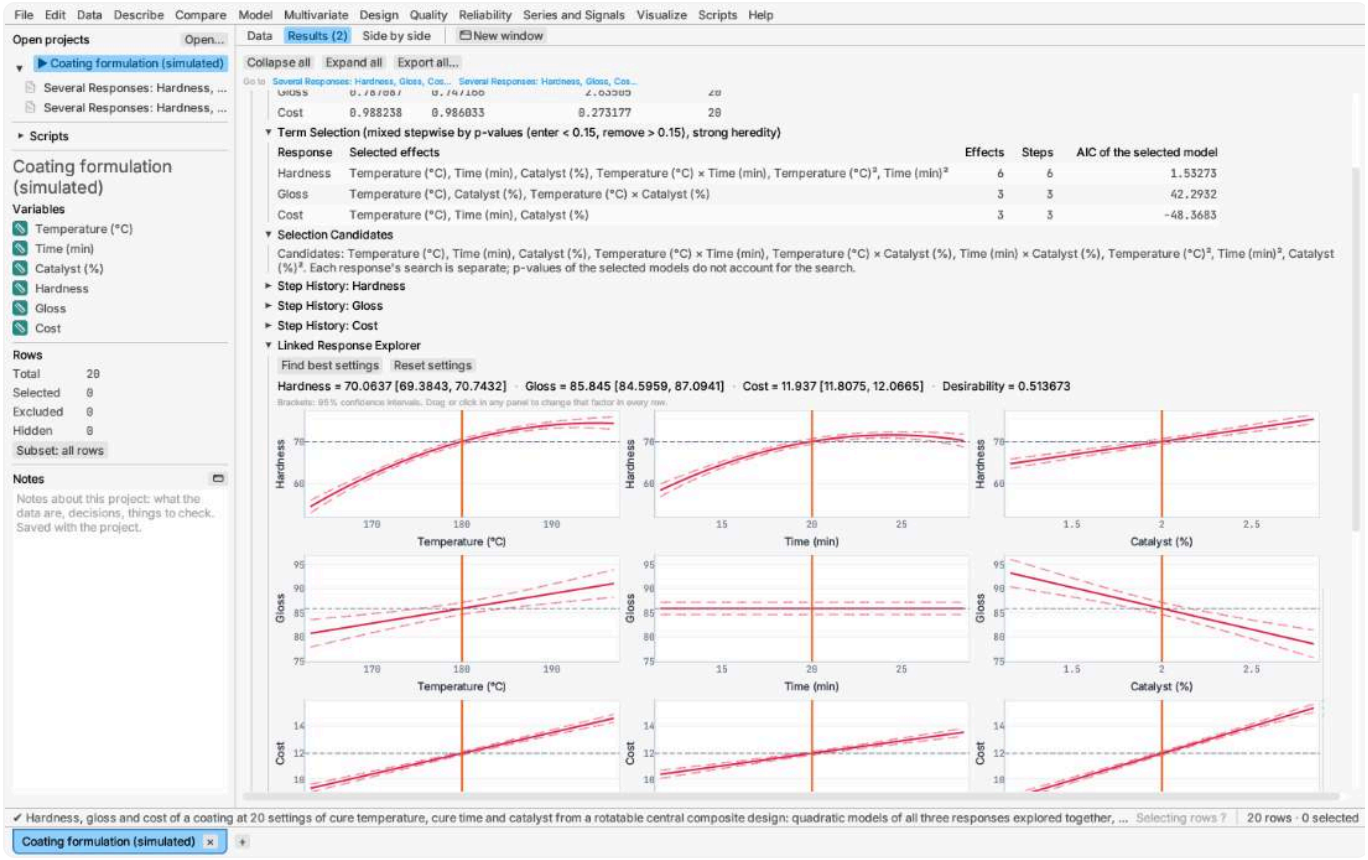
Try it: Coating formulation: three factors and three responses from a central composite design. Its second report selects each response's terms stepwise.

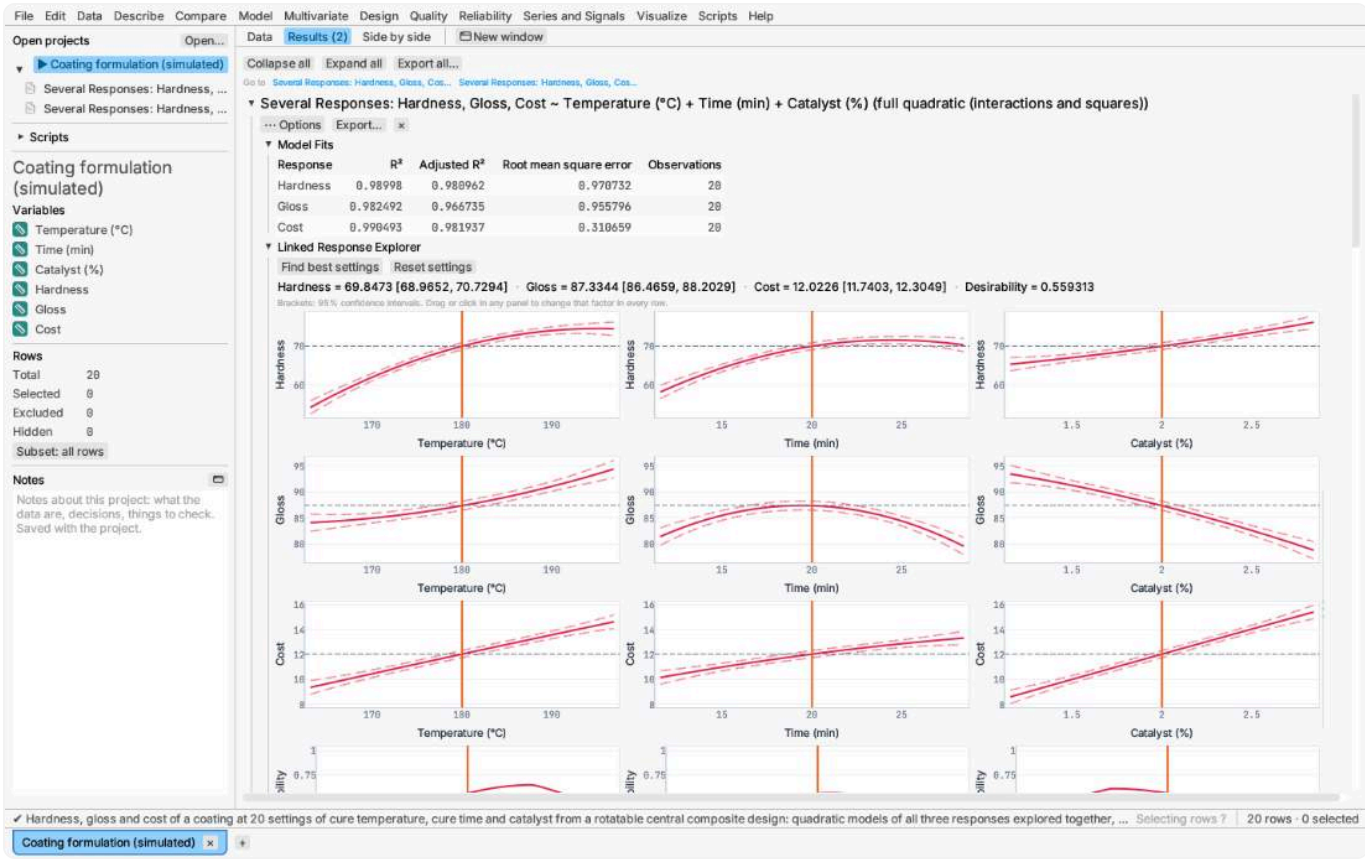
Terms: every term, or stepwise for each response

Under **Terms**, **Fit every term** gives every response the same model. **Select terms stepwise for each response** runs a separate stepwise search (see Stepwise selection) for each response among the terms of the chosen model, so a response that depends on an interaction or a square keeps it while another can drop it. The candidates include two-factor interactions and squares when the model form has them. Choose the direction (forward, backward or mixed) and the criterion (p-values, AIC or BIC); under **More options** set the heredity rule (strong by default: an interaction or square enters only with the main effects it is built from) and the p-value thresholds (enter below 0.15, remove above 0.15 by default). To pick the candidates one by one (cubes included), use Model ▸ Stepwise selection… and tick **Several responses, each with its own search**: it opens the same report.



The report adds **Term Selection**, one row per response with the effects selected, how many, the number of steps and the criterion of the selected model; **Selection Candidates**, the candidate list (with parent effects added for heredity); and a **Step History** per response (closed at first). A factor that no selected term of a response uses draws flat in that response's explorer row. A response for which no term is selected is predicted by its mean, with the interval $\bar{y} \pm t_{0.975,n-1}\, s/\sqrt{n}$, and shown as "(mean only)" in **Model Fits**. The explorer, goals, best settings, simulation and specification region then work on the selected models. p-values of the selected models do not account for the search.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Coating formulation (simulated)
Several Responses: Hardness, ...
Several Responses: Hardness, ...
Scripts
Coating formulation (simulated)
Variables
Temperature (°C)
Time (min)
Catalyst (%)
Hardness
Gloss
Cost
Rows
Total 20
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (2)
Side by side
New window
Collapse all
Expand all
Export all...
Term Selection (mixed stepwise by p-values (enter < 0.15, remove > 0.15), strong heredity)
Response
Selected effects
Effects
Steps
AIC of the selected model
Hardness
Temperature (°C), Time (min), Catalyst (%), Temperature (°C) × Time (min), Temperature (°C)², Time (min)²
6
6
1.53273
Gloss
Temperature (°C), Catalyst (%), Temperature (°C) × Catalyst (%)
3
3
42.2932
Cost
Temperature (°C), Time (min), Catalyst (%)
3
3
-48.3683
Selection Candidates
Step History: Hardness
Step History: Gloss
Step History: Cost
Linked Response Explorer
Find best settings
Reset settings
Hardness = 70.0637 [69.3843, 70.7432] · Gloss = 85.845 [84.5959, 87.0941] · Cost = 11.937 [11.8075, 12.0665] · Desirability = 0.513673
Temperature (°C)
Time (min)
Catalyst (%)
✓ Hardness, gloss and cost of a coating at 20 settings of cure temperature, cure time and catalyst from a rotatable central composite design: quadratic models of all three responses explored together, ...
Selecting rows ?
20 rows · 0 selected
Coating formulation (simulated)


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Coating formulation (simulated)
Several Responses: Hardness, ...
Several Responses: Hardness, ...
Scripts
Coating formulation (simulated)
Variables
Temperature (°C)
Time (min)
Catalyst (%)
Hardness
Gloss
Cost
Rows
Total 20
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (2)
Side by side
New window
Collapse all
Expand all
Export all...
Several Responses: Hardness, Gloss, Cost ~ Temperature (°C) + Time (min) + Catalyst (%) (full quadratic (interactions and squares))
Options
Export...
Model Fits
Response
R²
Adjusted R²
Root mean square error
Observations
Hardness
0.98998
0.988962
0.970732
20
Gloss
0.982492
0.966735
0.955796
20
Cost
0.990493
0.981937
0.310659
20
Linked Response Explorer
Find best settings
Reset settings
Hardness = 69.8473 [68.9652, 70.7294] · Gloss = 87.3344 [86.4659, 88.2029] · Cost = 12.0226 [11.7403, 12.3049] · Desirability = 0.559313
Temperature (°C)
Time (min)
Catalyst (%)
✓ Hardness, gloss and cost of a coating at 20 settings of cure temperature, cure time and catalyst from a rotatable central composite design: quadratic models of all three responses explored together, ...
Selecting rows ?
20 rows · 0 selected
Coating formulation (simulated)

Linked Response Explorer

One row of slice plots per response, plus a row for the overall desirability. Moving a factor in any row moves it in all of them. Right-clicking saves every row as one figure.

Goals and desirability

For each response, choose a goal: **maximize**, **minimize**, **hit a target**, or **ignore**. Set lower and upper limits, shape exponents (separately above and below a target) and an importance. Each response's prediction becomes a desirability between 0 and 1, and the **overall desirability** is their importance-weighted geometric mean (Derringer & Suich, 1980). The current predictions and desirabilities update live.

Find best settings

Maximizes the overall desirability within the factor ranges, using Nelder–Mead from a Latin hypercube of starting points, and searching every combination of categorical levels. The five best distinct solutions are listed, each with a **Set to** button that moves the explorer there.

Constraints on the factors

Under **Constraints on the factors**, type linear limits such as `Temperature (°C) + 2*Time (min) <= 230` or `Temperature (°C) - Time (min) = 150 , and combinations to leave out such as ` exclude Temperature (°C) > 185 and Time (min) > 21 `, one per line in the factors' units (the same syntax as constraints in optimal designs). Find best settings and Find robust settings then keep to them: the searches start inside the allowed region and settle on its boundary when the unconstrained best lies outside it. The best-settings title counts the constraints. A line that does not parse is reported and not applied.

Simulation

Simulation shows how variation in the factors and noise in the responses carries through to the results at the current settings:

- each continuous factor is fixed, or varies normally, uniformly or triangularly around its setting;
- each response gets normal noise (by default the model's RMSE);
- optional specification limits per response.

The report gives each response's simulated mean, SD and defect rate, and the share of runs with any response out of specification, with 95% Wilson intervals and a histogram per response. It re-runs as you change settings.

Find robust settings moves the continuous factors to the lowest simulated defect rate (using common random numbers so comparisons are fair). The simulated runs can be saved as a new table.

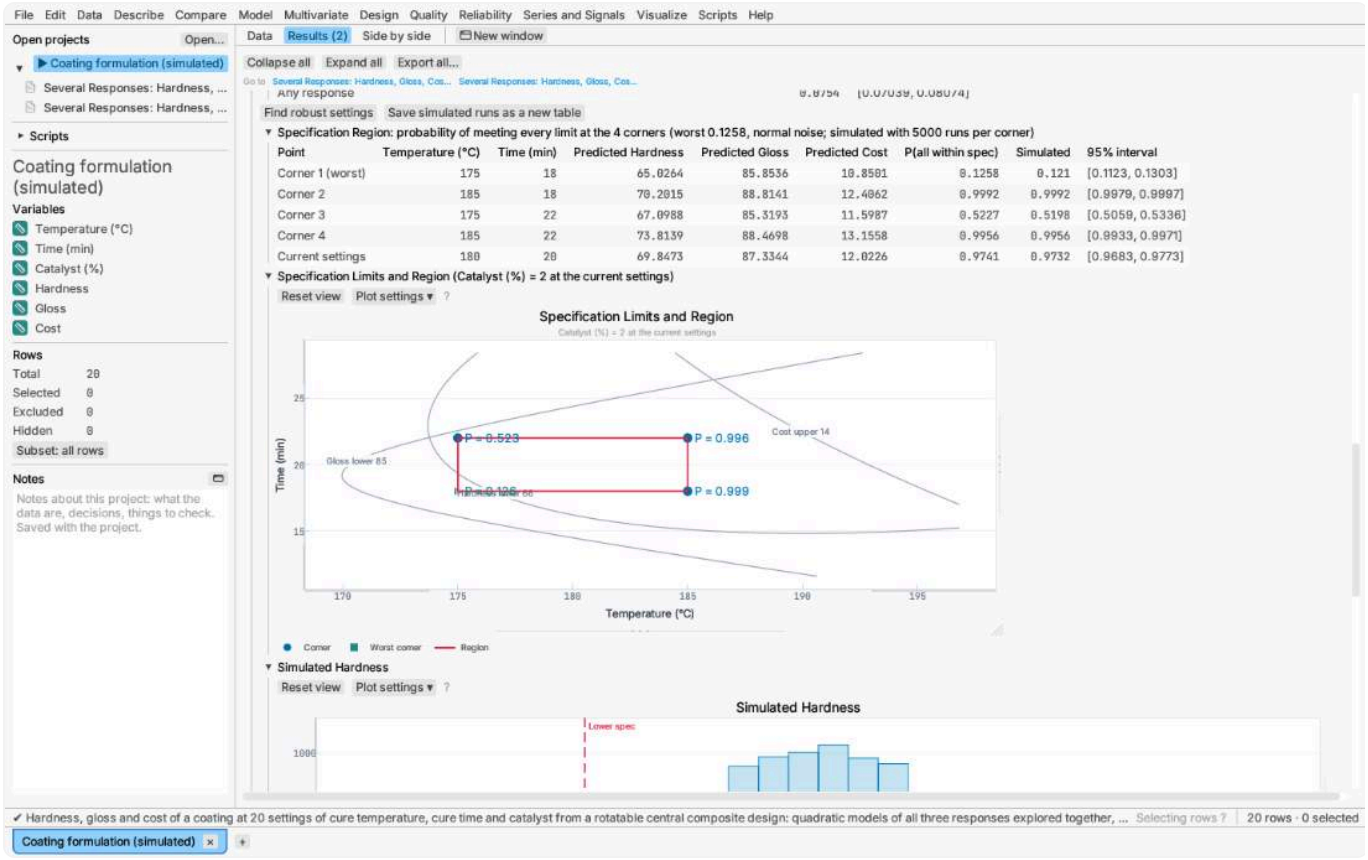
Specification region

A process is run not at one setting but anywhere within its operating ranges. Tick **Check a specification region around the settings** and give each continuous factor a region low and high (by default its setting ± a twentieth of its range; **Center on the current settings** resets them). At every corner of that box the report gives the predictions and the probability that every response is within its specification limits:

$$P = \prod_i [\Phi((U_i - \hat{y}_i)/\sigma_i) - \Phi((L_i - \hat{y}_i)/\sigma_i)]$$

from the normal noise of each response, alongside the simulated share (the same random draws at every corner, at most 5000 per corner) with a 95% Wilson interval. The lowest corner is marked worst. When the probability falls steadily away from the current settings, as it usually does near an optimum, the worst corner is the least favourable point of the box; with strongly curved responses an edge or face can be lower still, so check the contour plot (Peterson 2004; del Castillo 2007). Up to 8 factors can vary (256 corners); a factor whose low equals its high stays at its current setting.

A contour plot shows each response's specification limits over the first two varying factors, the others at their current settings, with the box, its corners and the worst corner marked by a different symbol.



Try it: the Coating formulation report has a constraint and a region set up.

Full reports

The options menu's **Open each response's linear model report** opens a full report for every response, with estimates, tests and diagnostics.

Verified: desirability functions against hand-computed values; simulated defect rates against analytic normal tail probabilities; uniform and triangular spreads against their theoretical standard deviations; corner probabilities against normal tails and simulation; constrained optima of problems solved by hand.

Model: dose–response, method comparison and shelf life

Three specialised regressions on the Model menu: probit analysis of quantal responses, regression for comparing two measurement methods, and shelf life from stability studies.

Dose–response (probit) analysis

Model ▸ Dose–response (probit)… fits the proportion of subjects responding (killed, cured, failed) against a dose, and estimates the dose that gives a chosen response — the median effective dose ED50, or ED10, ED90 and so on.

Dialog:

- **Dose**, **Number responding** and **Number of subjects** columns. Each row is a group of subjects given one dose; rows with the same dose are pooled. With *One subject per row* the response column must be 0 or 1.
- **Link:** probit (the normal curve, the classical choice), logit, or complementary log-log (an asymmetric curve, for one-hit models).
- **Dose scale:** $\log_{10}$ dose (the default), ln dose or the dose itself. On a log scale, rows with a zero dose (untreated controls) are left out and counted in the summary.
- **Effective doses for (%):** the response percentages to report, such as `10, 50, 90`.
- **Fiducial level** (95%) and **Heterogeneity test level** (0.05).

Report:

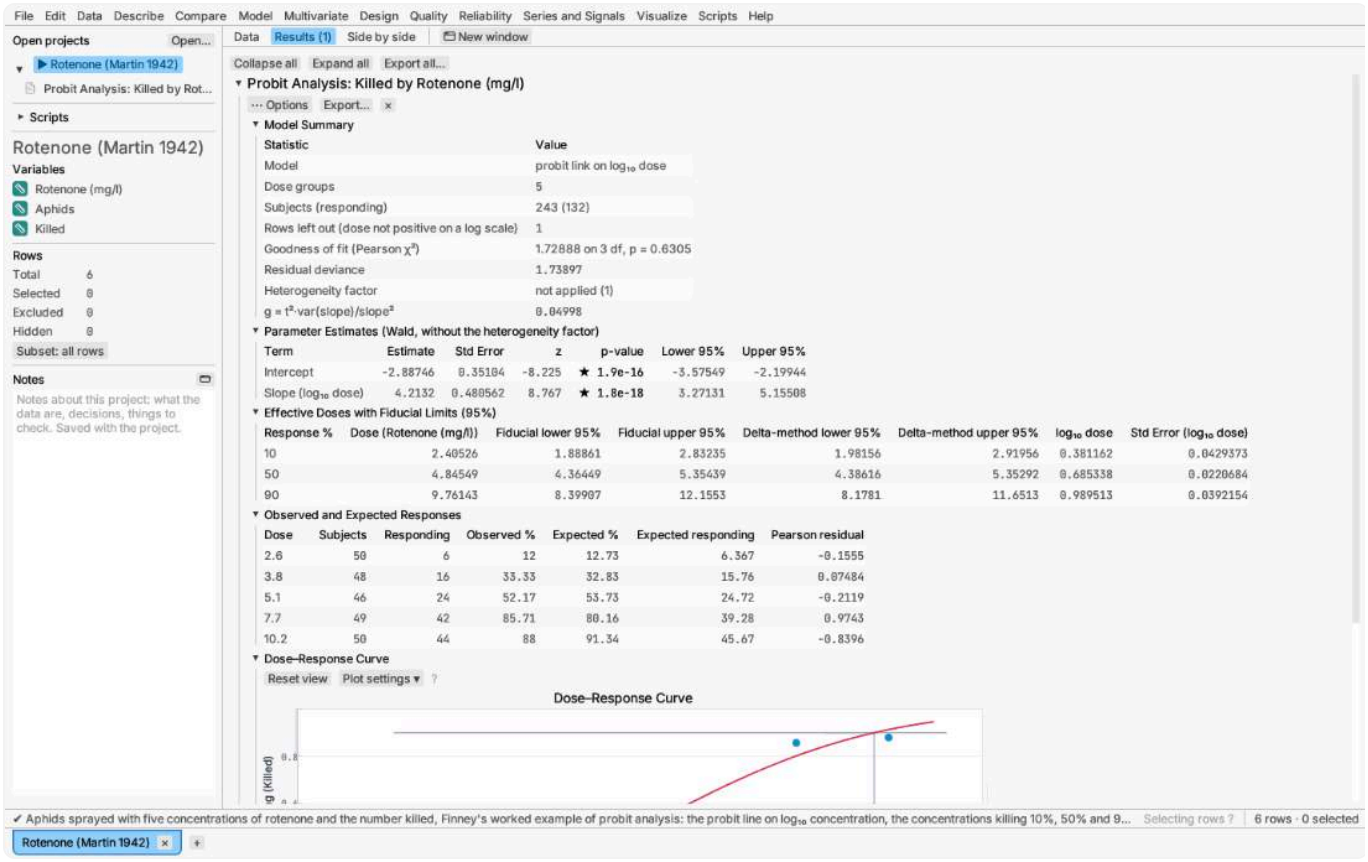
- *Model Summary:* dose groups, subjects, Pearson's χ^2 test of fit on $k - 2$ degrees of freedom, the residual deviance, and whether the heterogeneity factor was applied. When the χ^2 test is significant at the chosen level the groups vary more than binomial sampling allows; as Finney recommends, the variances are then multiplied by the heterogeneity factor $\chi^2/(k - 2)$ and Student's t on $k - 2$ df replaces the normal deviate. *g* measures how well the slope is determined: when $g \geq 1$ the fiducial limits are unbounded.
- *Parameter Estimates:* intercept and slope on the model scale.
- *Effective Doses with Fiducial Limits:* each requested percentage, its dose, the fiducial limits (Fieller's theorem, asymmetric about the dose) and the simpler delta-method

limits for comparison.

- *Observed and Expected Responses:* per dose group, with Pearson residuals.
- *Dose–Response Curve:* observed proportions against the (log) dose with the fitted curve; each effective dose is marked with a drop line and its fiducial interval. Points are linked to their rows.

With only two doses the curve passes through both observed proportions (there is nothing left to test its fit), so each dose needs some but not all subjects responding: a dose at 0% or 100% gives an infinitely steep curve, and the analysis says so. Add a dose between them.

The same links are available for any binomial model in Model ▸ Generalized linear model….



Try it: Rotenone (Martin 1942), Finney's worked example.

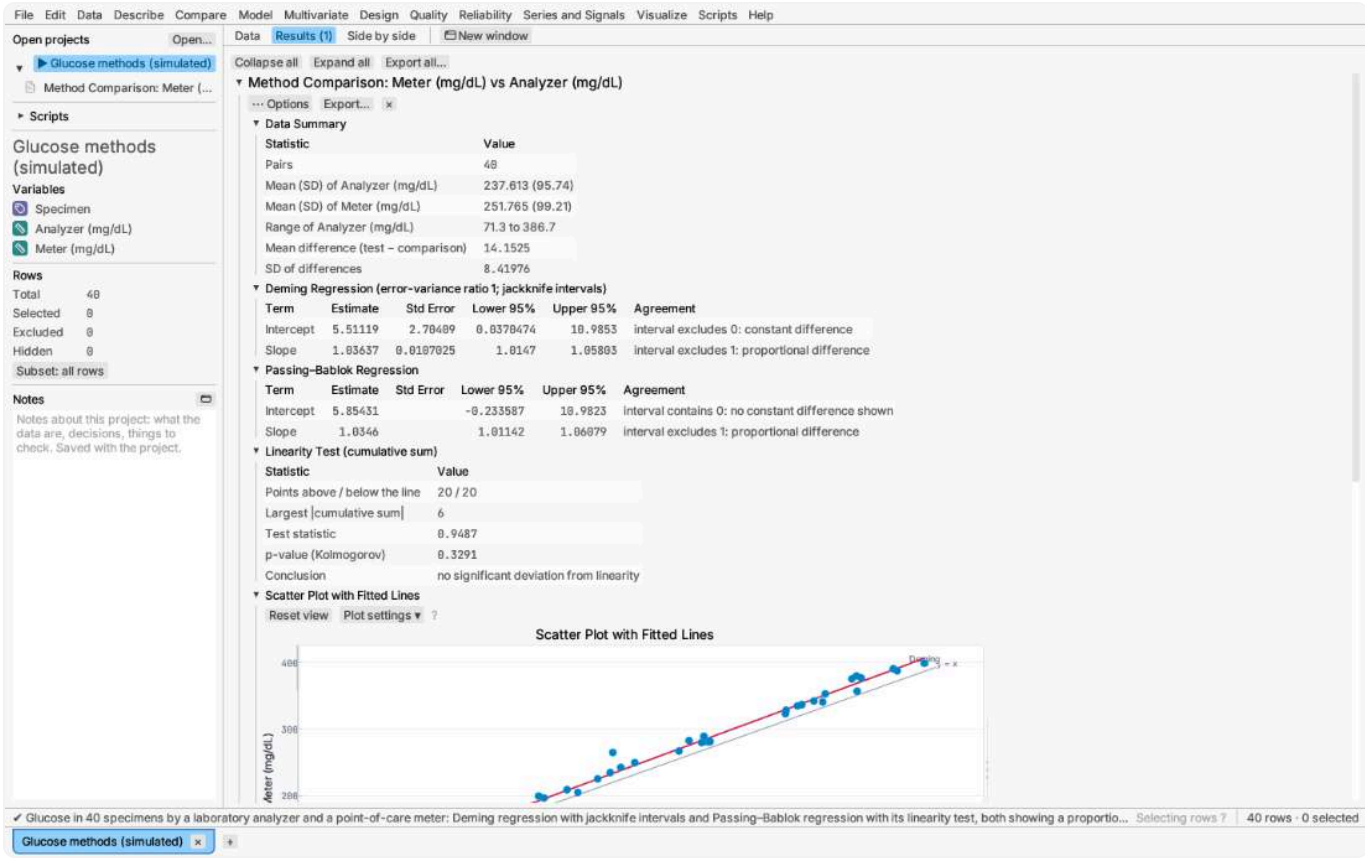
Method comparison

Model ▸ Method comparison… regresses a test method (Y) on a comparison method (X) when both are measured with error, so ordinary least squares would underestimate the slope.

Dialog: the two columns; **Deming**, **Passing–Bablok** or both; the **error-variance ratio** Var(error of Y) / Var(error of X) for Deming regression (1 gives orthogonal regression; use the ratio of the two methods' squared repeatability standard deviations when known); the confidence level.

Report:

- *Data Summary:* pairs, means, the range of X, and the mean and standard deviation of the differences.
- *Deming Regression:* intercept and slope with jackknife standard errors and t intervals on n − 2 df (Linnet 1993). The *Agreement* column reads the intervals: an intercept interval excluding 0 shows a constant difference between the methods, a slope interval excluding 1 a proportional one.
- *Passing–Bablok Regression:* the median-based slope and intercept with their confidence limits, robust to outliers. The method assumes the two methods are positively related. When they are not (most pairwise slopes below −1, as when one method falls while the other rises) the upper slope limit is undefined: it shows as ∞ and a *Passing–Bablok Note* explains why, rather than a finite limit that would mean nothing.
- *Linearity Test:* Passing and Bablok's cumulative-sum test. A small p-value means the relation is not linear and Passing–Bablok regression does not apply. The largest cumulative sum is divided by √(I + L), the numbers of points above and below the line, which makes it the two-sample Kolmogorov–Smirnov statistic. Some texts print the divisor as √(L + 1); that form flags linear data as non-linear far more often than 5% of the time, so results from software that uses it can show smaller p-values.
- *Scatter Plot with Fitted Lines:* the data, the line of identity and each fitted line, labelled.
- *Difference Plot:* the difference Y − X against the mean of the two, with the mean difference and the limits of agreement mean ± 1.96 SD (Bland and Altman).



Try it: Glucose methods (simulated), where one specimen has a gross error that pulls the Deming line but not Passing–Bablok's.

Shelf life

Model ▸ Shelf life… estimates how long a product stays within its acceptance limits from stability data of several batches, following ICH Q1E.

Dialog: the **Time**, **Response** and **Batch** columns (no batch for a single batch); whether the batches are **fixed** (the ICH Q1E pooling analysis below) or **a random sample of batches** (see *Batches as a random effect*); a **lower limit** (for an attribute that falls, like assay), an **upper limit** (one that rises, like a degradation product) or both; the **poolability test level** (0.25, as ICH Q1E sets), the confidence level (95%) and the **time unit** of the time column (months, weeks, days or years; months by default, or *Not stated*).

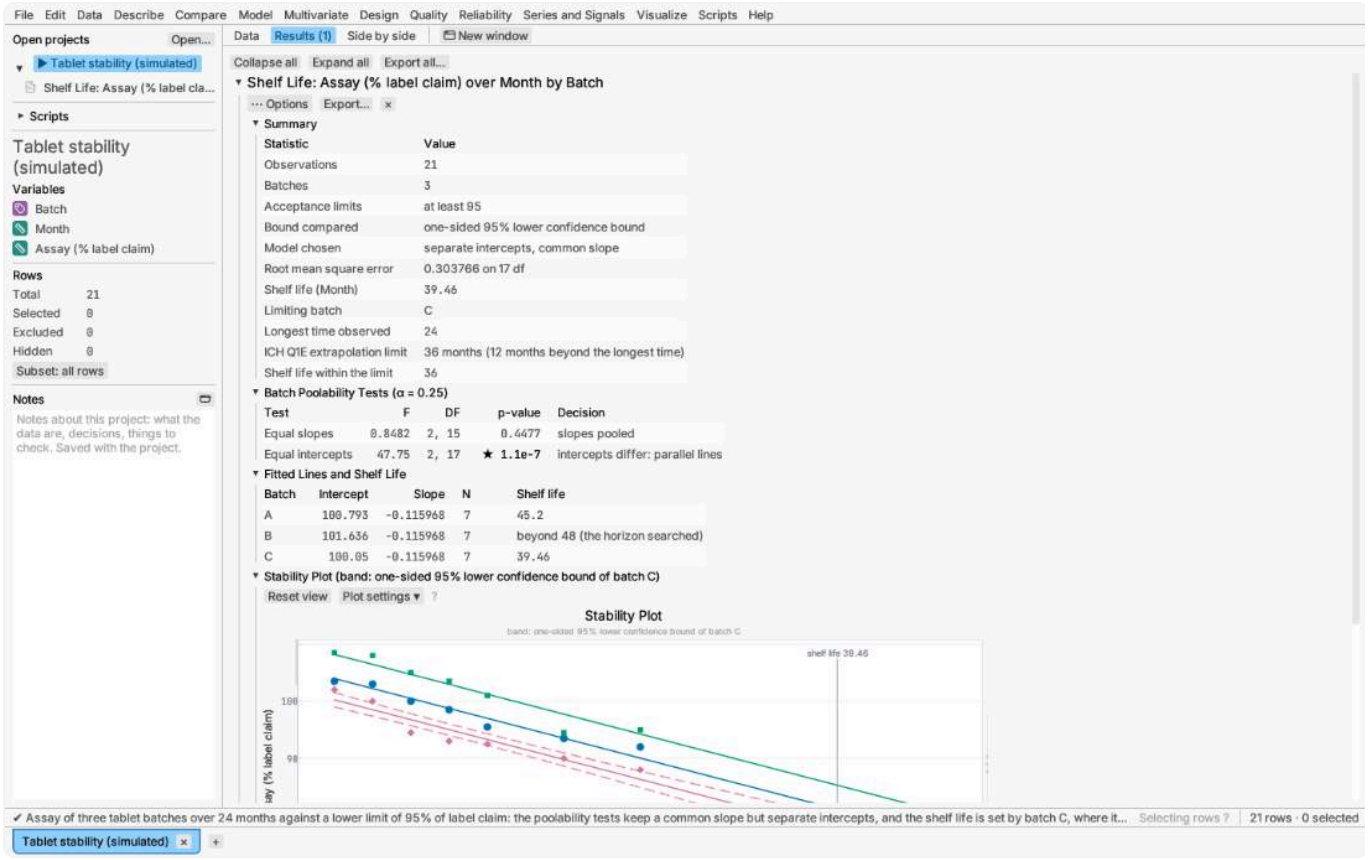
How the model is chosen: an analysis of covariance first tests whether the batches share a slope. If they do not ($p < 0.25$), each batch keeps its own line. If they do, a second test asks whether they share an intercept: if not, the batches get parallel lines; if so, all data are pooled into one line.

When every batch has only two measurement times, the separate-lines model fits each batch exactly and leaves nothing to test the slopes against. The slope test is then skipped, the slopes are assumed common (the notes say so), and the intercept test alone chooses between parallel lines and one line. Measure at three or more times to test the slopes. If every batch lies exactly on its line, identical lines pool ($F = 0$, $p = 1$) and different slopes count as differing with certainty ($F = \infty$, $p = 0$).

Shelf life is the earliest time at which the 95% confidence bound for the mean response meets a limit: the one-sided lower bound against a lower limit, the one-sided upper bound against an upper limit, or two-sided bounds when both limits are set. With separate or parallel lines the shortest batch shelf life applies. The search runs to twice the longest time observed.

Extrapolation limit: ICH Q1E allows a shelf life of at most twice the period the long-term data cover, and no more than 12 months beyond it. With data to 24 months that is 36 months; with data to 6 months, 12. The time unit tells the analysis how long 12 months is, so with *Not stated* only the twice-the-data limit is applied. The summary shows the limit and the **shelf life within the limit**, the smaller of the estimate and the limit.

Report: a summary (model chosen, shelf life, limiting batch, extrapolation limit, shelf life within the limit), the poolability tests, each line with its own shelf life, a plot of the data by batch with the fitted lines, the confidence bound of the limiting batch, the limits and the shelf life marked, and notes. When the shelf life lies beyond the data the notes recall the ICH Q1E rule: extrapolation up to twice the period covered, and no more than 12 months beyond it.



Batches as a random effect

When the batches tested are a sample of the batches that will be made, the dialog's **Batches are: A random sample of batches** choice fits $y = (\beta_0 + b_0) + (\beta_1 + b_1)\cdot\text{time} + \varepsilon$, with each batch's intercept and slope deviations (b_0, b_1) drawn from a normal distribution with an estimated covariance G, by REML. It needs three or more batches. **Bound for** picks the bound compared with the limits:

- **A future batch** (default): the prediction bound for the mean of a batch not yet made, whose variance adds the batch-to-batch variance x^TGx to the variance of the population line; this is the shelf life a new batch can be expected to meet.
- **The mean of all batches:** the confidence bound for the population line alone, which is narrower.

Degrees of freedom come from Satterthwaite's approximation for the bound's variance, using the covariance of the REML variance parameters (Chow & Shao 1991 for random batches in stability studies). The report shows the shelf life, the population line, the batch-to-batch variances and correlation, each batch's predicted line (its conditional modes) with the shelf life where it meets the limit, and the plot with the chosen bound. The same extrapolation limit applies.

Verified: against bounds and degrees of freedom computed in R from lme4's fit and lmerTest's covariance of the variance parameters.

Try it: sample *Capsule stability, six lots (simulated) — shelf life with random lots*, which holds the random-lot report beside the fixed-lot one.

Try it: Tablet stability (simulated). Batch C's bound meets 95% at about 39.5 months, beyond the 36-month limit for 24 months of data, so the shelf life within the limit is 36 months.

Model: treatment effects from observational data

Model ▸ Treatment effects (observational data)… estimates the effect of a two-level treatment (enrolled or not, exposed or not, new process or old) on an outcome when the treatment was not assigned at random. Rows that received the treatment usually differ from those that did not in ways that also affect the outcome (confounding), so a plain comparison of the two groups is biased. The analysis adjusts for covariates measured before treatment through the **propensity score**, each row's probability of being treated given its covariates (Rosenbaum & Rubin 1983).

The adjustment removes the bias from the covariates you choose, and only those: it cannot help with a confounder that was not measured. Choose covariates that affect both the treatment and the outcome, and never ones the treatment itself could have changed.

Dialog

- **Treatment:** a column with exactly two values. **Treated value:** which of them is the treatment (by default the second in sorted order).
- **Outcome** (numeric) and its kind: **Continuous**, **Binary (0/1)** or **Count**. The kind sets the outcome models of the doubly robust method (normal linear, logistic or Poisson regression) and the effect scales. Choosing a 0/1 column sets Binary.
- **Covariates:** the confounders. A categorical covariate enters the models as indicators of each of its levels but the first.
- **Subgroups** (optional): columns whose levels each get their own estimate (for example age group or sex).
- **Method:**
  - *Inverse-probability weighting* weights each row by the inverse of the probability of the treatment it received, so that the weighted groups look alike (the normalized, or Hájek, estimator).
  - *Doubly robust (weighting with outcome models)* adds regressions of the outcome on the covariates in each group (augmented inverse-probability weighting). The estimate is consistent if either the propensity model or the outcome models are right.

    - *Nearest-neighbour matching* pairs each treated row with the control rows nearest to it in the logit of the propensity score, largest scores first. Set the **matches per treated row** (1 for pair matching), **with replacement** (a control may serve several treated rows) and a **caliper** in standard deviations of the logit (0.2 by default, as Austin 2011 recommends): treated rows with no control that close stay unmatched and are left out.
- **Effect:** the *average effect (ATE)* over everyone, or the *average effect among the treated (ATT)*. Matching for the ATE matches every row, treated and control, to the other group and needs matching with replacement.
- **More options:** *stabilized weights* (the saved weights are scaled by the share of rows in each group; the estimate does not change), *trim scores outside* α to 1 − α (rows with scores near 0 or 1, where the groups do not overlap, are left out; Crump et al. 2009), *bootstrap intervals* instead of sandwich intervals, and the confidence level.

Report

- *Summary:* the treated and control values, the method and estimand, the interval method, rows in each group and rows used, the effective sample size of each group ($(\Sigma w)^2/\Sigma w^2$: how many unweighted rows the weighted group is worth), rows trimmed, excluded or missing, and the largest standardized difference left after adjustment, with a note when it is 0.1 or more.
- *Treatment Effect:* the mean difference (continuous), the risk difference, risk ratio and odds ratio (binary) or the rate difference and rate ratio (count), each with its standard error, z test and interval. Ratios get their intervals on the log scale.
- *Estimated Outcome Means:* the mean outcome had everyone (or every treated row, for the ATT) been treated, and had they not.
- *Covariate Balance:* for every covariate (each level of a categorical one) and the propensity score, the means in each group, the standardized mean difference and, for continuous variables, the variance ratio, before and after adjustment. Differences are standardized by the unadjusted pooled standard deviation $\sqrt{((s_1^2 + s_0^2)/2)}$ for the ATE and by the treated group's for the ATT, so before and after are on the same scale; a 0/1 variable's variance is $p(1 - p)$. Weighted variances use $\Sigma w/((\Sigma w)^2 - \Sigma w^2) \cdot \Sigma w(x - \bar{x})^2$. A covariate counts as balanced when its adjusted difference is below 0.1 in absolute value (Austin 2009).
- *Balance Plot:* the absolute standardized differences before (one marker) and after (another marker) adjustment for each variable, joined by a line, with the 0.1 threshold.

- *Propensity Score Overlap:* each row's score, one strip per group (jittered). Scores near 0 or 1 in one group with no rows of the other group nearby show poor overlap; consider trimming. Points are linked to their rows, so a box selection shows the rows in the data grid and every other plot.
- *Propensity Score Model:* the logistic regression of the treatment on the covariates.
- *Outcome Models* (doubly robust): the regressions fitted in each group.
- *Subgroup Effects* and *Subgroup Forest Plot* (with subgroups): the whole analysis repeated within each level of each subgroup column, the effect (difference scale) with its interval, and Cochran's Q test that the subgroup effects are equal (the subgroup-by-treatment interaction), on G − 1 degrees of freedom. Covariates that are constant within a subgroup (such as the subgroup column itself) are left out of its models. Many subgroups make chance differences likely: treat them as exploratory (Wang et al. 2007).

The ⋯ Options menu switches the effect between ATE and ATT and changes the confidence level in place; *Edit analysis…* reopens the dialog. *Save propensity scores*, *Save weights* and, for matching without replacement, *Save matched sets* add columns to the table (NaN for rows the analysis did not use; weight 0 for rows left out). A saved weight column can be used as the weight of other analyses.

Methods

With treatment $t_i \in \{0, 1\}$, outcome y_i and estimated score $\hat{e}_i$:

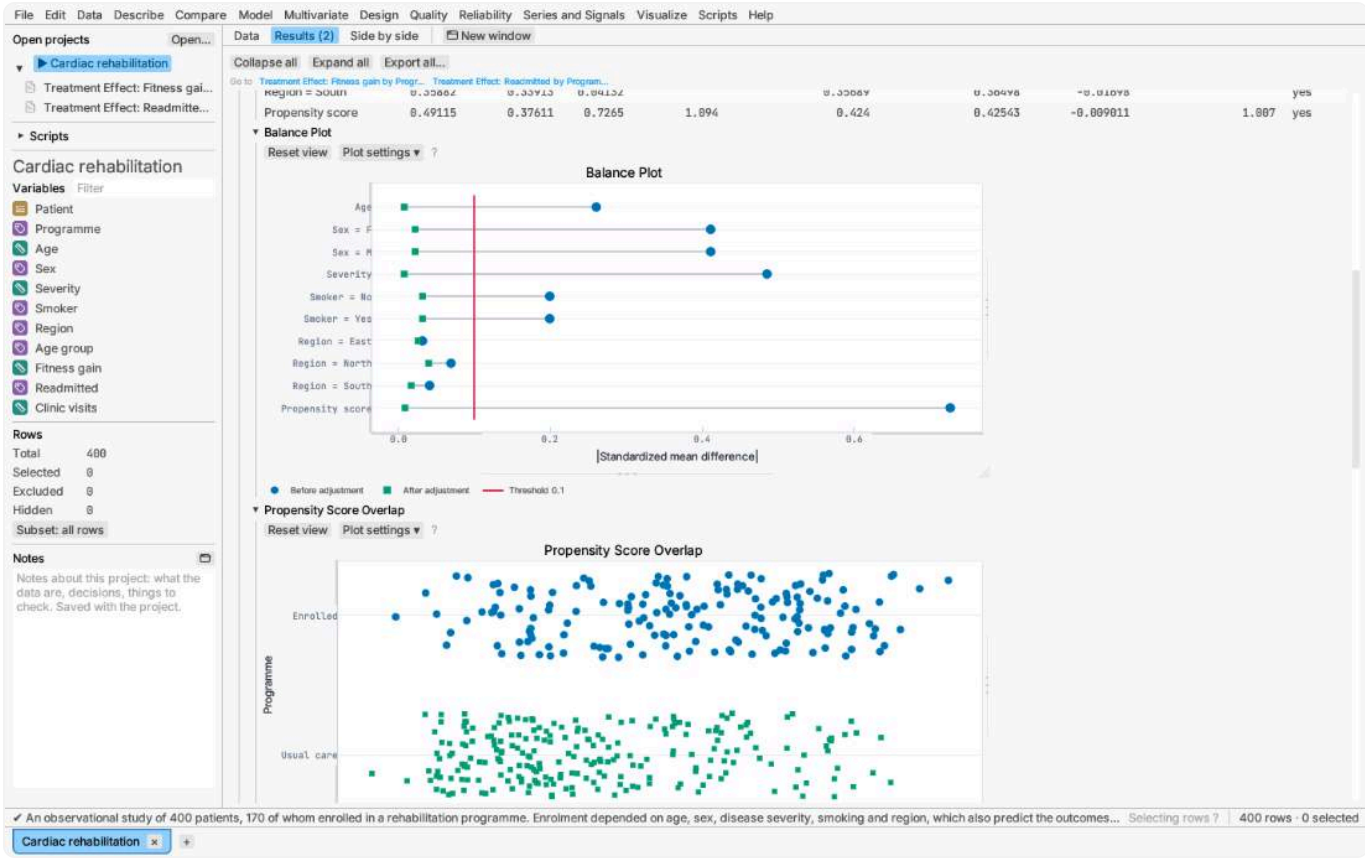
- **Weighting, ATE:** weights $t_i/\hat{e}_i + (1 - t_i)/(1 - \hat{e}_i)$; **ATT:** $t_i + (1 - t_i)\hat{e}_i/(1 - \hat{e}_i)$. Each group's mean is the weighted mean of its outcomes.
- **Doubly robust, ATE:** $\hat{\mu_1} = n^{-1} \Sigma\, [t_i y_i/\hat{e}_i - (t_i - \hat{e}_i)\, m_1(x_i)/\hat{e}_i]$ and $\hat{\mu_0} = n^{-1} \Sigma\, [(1 - t_i)y_i/(1 - \hat{e}_i) + (t_i - \hat{e}_i)\, m_0(x_i)/(1 - \hat{e}_i)]$, with m_1, m_0 the outcome models; **ATT:** $\hat{\mu_1}$ is the treated mean and $\hat{\mu_0} = \Sigma\, [t_i\, m_0(x_i) + (1 - t_i)\, \hat{e}_i/(1 - \hat{e}_i)(y_i - m_0(x_i))] \,/\, \Sigma\, t_i$ (Lunceford & Davidian 2004; Hernán & Robins 2020, ch. 13).
- **Matching:** matched rows get weight 1 (a treated row) or the sum, over the treated rows they serve, of 1/(that row's number of matches), the control weights rescaled to sum to the number of matched controls; the effect is the difference of weighted means. For the ATE each row also takes the mean of its matches as its missing potential outcome (Abadie & Imbens 2006), which is the same as weights $1 + K_i/k$.
- **Sandwich intervals** (weighting, doubly robust) treat the propensity model, the outcome models and the two means as one set of estimating equations, so the uncertainty of the estimated scores is included (Stefanski & Boos 2002). **Matching**

uses the robust variance of the weighted means with matched sets as clusters (rows, with replacement) and the small-sample factor G/(G − 1)·(N − 1)/(N − 2), as recommended for matched data (Austin & Small 2014); this conditions on the matching. **Bootstrap** intervals resample rows within each treatment group, repeat everything (score model, matching or weights, outcome models) and take percentiles; the p-value is twice the share of replicates on the far side of no effect.

Validation

The propensity and outcome models, weighting and doubly robust estimates and their sandwich covariances match statsmodels GLMs with a complex-step stacked sandwich (to 10^{-6}), the closed-form standard error of Lunceford & Davidian (2004) agrees to 1%, ATT matching (weights, estimate and robust standard error, with and without replacement and caliper) matches R `MatchIt` 4.5 with `sandwich` 3.1, ATE matching matches a brute-force implementation, and balance statistics match NumPy (the validation record, `docs/validation.md` in the source).

Try it: sample *Cardiac rehabilitation* (Model), an invented observational study in which younger patients, women, milder cases, non-smokers and patients in the South enrolled more often. The unadjusted difference in fitness gain overstates the programme's effect; the doubly robust estimate is close to the true average, and the subgroup forest plot shows the larger benefit for patients under 65.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Cardiac rehabilitation
Treatment Effect: Fitness gai...
Treatment Effect: Readmitte...
Scripts
Cardiac rehabilitation
Variables
Filter
Patient
Programme
Age
Sex
Severity
Smoker
Region
Age group
Fitness gain
Readmitted
Clinic visits
Rows
Total 400
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (2)
Side by side
New window
Collapse all
Expand all
Export all...
Propensity score 0.49115 0.37611 0.7265 1.894 0.424 0.42543 -0.009011 1.007 yes
Balance Plot
Reset view
Plot settings
Balance Plot
Age
Sex = F
Sex = M
Severity
Smoker = No
Smoker = Yes
Region = East
Region = North
Region = South
Propensity score
0.0
0.2
0.4
0.6
|Standardized mean difference|
Before adjustment
After adjustment
Threshold 0.1
Propensity Score Overlap
Reset view
Plot settings
Propensity Score Overlap
Enrolled
Usual care
Programme
An observational study of 400 patients, 170 of whom enrolled in a rehabilitation programme. Enrolment depended on age, sex, disease severity, smoking and region, which also predict the outcomes...
Selecting rows ?
400 rows · 0 selected
Cardiac rehabilitation

Model: nonlinear regression

Model ▸ Nonlinear regression… fits a curve whose parameters do not enter linearly — growth that levels off, a reaction rate that saturates, a decay towards a floor — by least squares. Pick a curve from the library, write the model as a formula with named parameters, give it as a differential equation, or describe a curve implicitly by an equation that is zero on it. Several data sets can be fitted at once with some parameters shared, and the report compares models, flags outliers and reads unknowns off a standard curve.

Dialog

- **Response:** the numeric column to fit.
- **Model:** *Curve from the library*, *Formula with parameters*, *Differential equation* or *Implicit equation*.
    - A library **Curve** in one **Predictor (x)** column. The dialog shows the equation with its parameter names:

Curve	Equation
Exponential growth or decay	`Scale * exp(Rate * x)`
Exponential with asymptote	`Asymptote + Scale * exp(Rate * x)`
Logistic (3 parameters)	`Asymptote / (1 + exp(-Rate * (x - Midpoint)))`
Logistic (4 parameters)	`Lower + (Upper - Lower) / (1 + exp(-Rate * (x - Midpoint)))`
Gompertz	`Asymptote * exp(-exp(-Rate * (x - Inflection)))`
Michaelis–Menten	`Vmax * x / (Km + x)`
Power	`Scale * x ** Exponent` (x must not be negative)
Rational (linear / linear)	`(a + b * x) / (1 + c * x)`

Curve	Equation
Straight line	`Intercept + Slope * x`
Dose–response, log dose (4 parameters)	`Bottom + (Top - Bottom) / (1 + 10 ** ((LogEC50 - x) * HillSlope))`
Dose–response, dose (4 parameters)	`Bottom + (Top - Bottom) / (1 + (EC50 / x) ** HillSlope)`
Asymmetric logistic, log dose (5 parameters)	`Bottom + (Top - Bottom) / (1 + 10 ** ((LogC - x) * HillSlope)) ** Asymmetry`
Weibull growth	`Asymptote - (Asymptote - Initial) * exp(-((Rate * x) ** Shape))`
Beta growth	`Ymax * (1 + (End - x) / (End - Peak)) * (x / End) ** (End / (End - Peak))` (x capped at End)
Segmented line (one breakpoint)	`Intercept + Slope1 * x + (Slope2 - Slope1) * max(x - Breakpoint, 0)`
Sum of two exponentials	`Plateau + Span1 * exp(-Rate1 * x) + Span2 * exp(-Rate2 * x)`
Gaussian peak	`Baseline + Height * exp(-(x - Center)² / (2 Width²))`
Lorentzian peak	`Baseline + Height / (1 + ((x - Center) / HalfWidth)²)`

The two dose–response curves are the four-parameter logistic of pharmacology (Hill equation), on log dose and on dose; *Bottom* is always the lower plateau, and a falling curve has a negative *HillSlope*. The asymmetric form (Richards 1959; Gottschalk & Dunn 2005) lets the curve approach its two plateaus at different rates. Weibull and beta growth (Yin et al. 2003) describe growth that stops at a final size; the segmented line changes slope at an estimated breakpoint (its derivative is not smooth there, so start near the bend if the automatic value is poor). The peak shapes fit a single peak on a flat baseline.

Library curves find their own starting values (leave *Start* empty): for each value of the curve's rate, midpoint, Km or exponent on a grid scaled to the data, the remaining parameters follow by linear least squares, and the best combination starts the fit. Their derivatives are exact.

    - A **model formula** written like a formula column: columns and parameters are variables, for example `Vmax * Conc / (Km + Conc)`. List every parameter under **Parameters** with a starting value (**Add parameter**, **Remove**). Parameter names are letters, digits and `_`, and may not be column names. Derivatives are computed by central differences. The dialog reports a formula it cannot read before you run it. A formula in **two** columns is a surface `z = f(x, y)`: the report draws it as contours over the data.

    - A **differential equation** for the response in a **Predictor (x)** column, such as time: write the **rate of change** `dy/dx` using the response, the predictor and the parameters by name (for example `-Vmax * Substrate / (Km + Substrate)`), choose which parameter is the response's value **at the start**, and give the **starting predictor value** (empty means the smallest observed value). Each evaluation solves the equation from that point with an adaptive Dormand–Prince 5(4) Runge–Kutta method (Dormand & Prince 1980) to a relative tolerance of 10^{-8}; derivatives are by central differences.

    - An **implicit equation** in the response and one other column that is zero on the curve, for example `(X - a)**2 + (Y - b)**2 - r**2` for a circle. The fit minimises the sum of squared *shortest distances* from the points to the curve (orthogonal distance regression; Boggs & Rogers 1990), so both columns are treated as measured with error. Each point's nearest point on the curve is found by Newton projection along the gradient followed by steps along the curve.

- **Parameters:** a starting value and optional **lower** and **upper bounds** for each. A start close to the answer helps a hard model converge; bounds keep a parameter in a meaningful range (a rate that must be positive, an asymptote that cannot be negative). Equal lower and upper bounds fix a parameter at that value: it is a known constant, not estimated, so it is marked *(fixed)*, has no standard error and uses no residual degree of freedom (the fit summary counts the parameters as, for example, *3 (2 estimated, 1 fixed by equal bounds)*); the fit, its residual standard error, tests and intervals are those of the model with that value written in. A parameter that merely ends on one of its bounds is still counted, since the data put it there.
- **Data sets (optional):** a column whose values split the rows into data sets (compounds, batches, subjects) fitted together in one *global* fit (Motulsky & Christopoulos 2004, ch. 37). Each parameter is either **Shared by all data sets** (one value for all, the default) or fitted to each data set separately: untick it to estimate, say, a separate potency for each compound with a common slope and plateaus. Bounds and starting values apply to every copy of a parameter.

- **Weights:** *Equal*, *1/Y² (relative error)* when the scatter grows in proportion to the response, *1/Y (Poisson)* for counts, or a **weight column** (positive values; a weight of 2 counts as two observations). Weighted fits minimise $\Sigma\ w_i\ (y_i - f(x_i))^2$; the residual standard error is that of an observation of weight 1, and prediction limits for a new observation use the weight it would have ($1/\hat{y}^2$ or $1/\hat{y}$; 1 with a weight column).
- **Fitting:** *Least squares*; *Robust (Cauchy loss)*, which minimises $\Sigma\ \ln(1 + (r_i/s)^2)$ so that a few wild points barely pull the curve; or *Least squares after removing outliers*, which fits robustly, flags outliers by the ROUT rule with false discovery rate **Q** (1% by default; Motulsky & Brown 2006), and refits the remaining rows by least squares. The robust scale s is the 68.27th percentile of the absolute residuals times $N/(N - K)$, re-estimated at each reweighting step. ROUT ranks the absolute residuals and, from the 70th percentile upward, flags the first residual whose two-sided t p-value (residual over the robust scale, $N - K$ degrees of freedom) falls below $Q\cdot(N - i + 1)/N$, together with every larger one.
- **Compare with** (one data set and a library curve): another library curve to compare against this one.
- **Confidence level** (95%), **Iteration limit** (200) and **Profile intervals** (on).

Rows that are excluded, or missing the response, a column the model reads, the data set or a valid weight, are left out of the fit. Rows with a response but no predictor value are not fitted: their predictor is read off the curve (see *Inverse Prediction*).

Report

- *Fit Summary:* the model, the initial value of a differential equation, the data sets and which parameters they share, observations used, rows excluded or missing, the weighting and fitting method, the robust scale and outliers removed, the sum of squared errors, residual degrees of freedom, the residual standard error, AICc, whether and how the fit converged, the iterations, the relative offset at the solution, and whether derivatives were exact or numeric.
- *Warnings* (only when there is something to say): the fit did not converge (the estimates are those of the last iteration; try other starting values), a parameter ended on one of its bounds (it is held there, without a standard error), or a profile interval is open on one side.
- *Parameter Estimates:* each estimate with its asymptotic standard error, t ratio and p-value (for the hypothesis that the parameter is zero), the Wald interval $\text{estimate} \pm t \times \text{SE}$, the profile interval, and the starting value used (marked *automatic* when the

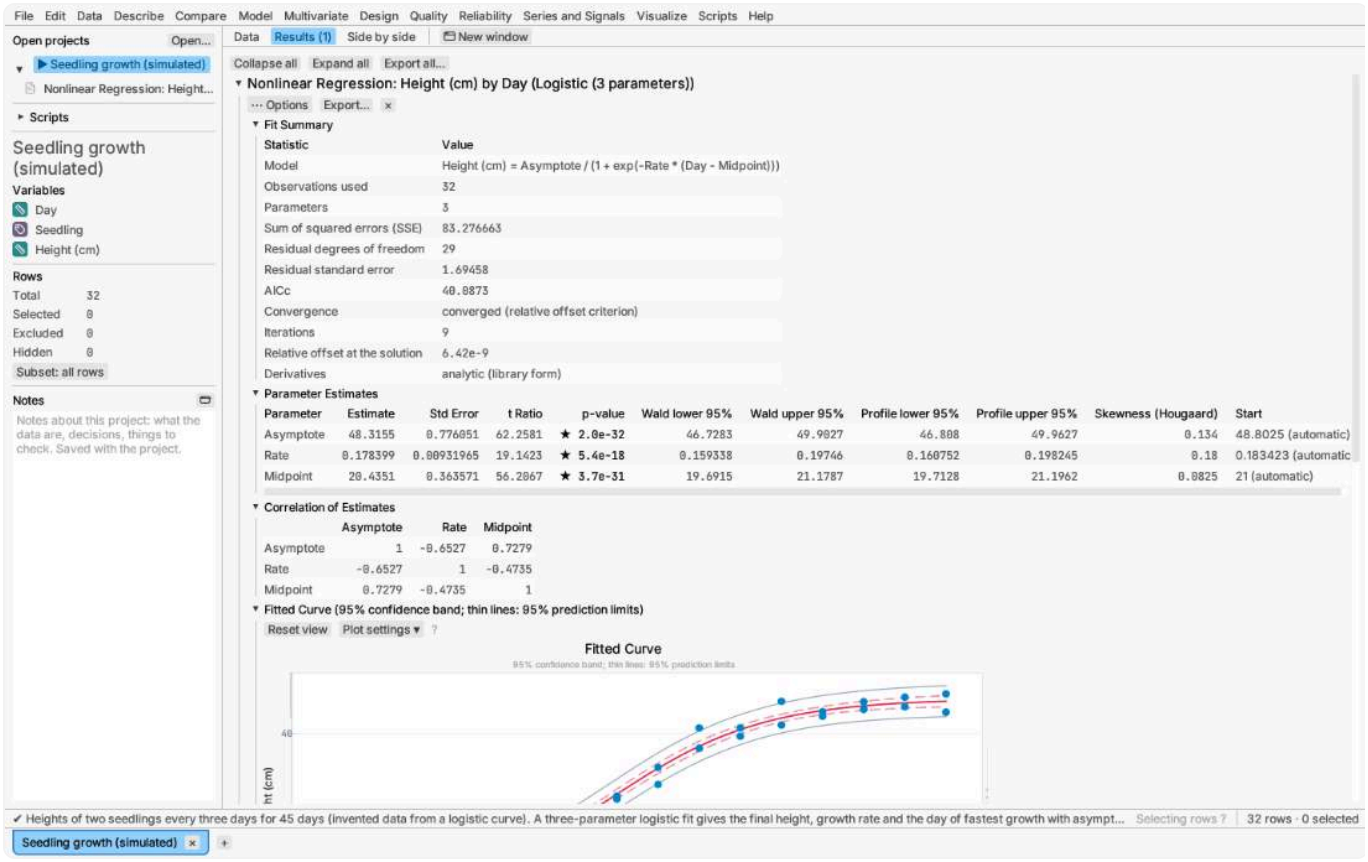
library chose it). Unshared parameters appear once per data set, as `LogEC50 (A)`. The profile interval holds the parameter fixed at trial values, refits the others, and keeps the values whose increase in the sum of squares is not significant (Bates & Watts 1988, ch. 6). It follows the curvature of the model, so it is often asymmetric, and it is the better interval when the two disagree. The *Skewness (Hougaard)* column measures how far each estimate's sampling distribution is from the symmetric one the Wald interval assumes (Hougaard 1985; Ratkowsky 1990, §2.6): below 0.1 in absolute value the parameter behaves close to linearly, above 0.25 the nonlinearity is noticeable and a reparameterization (for example log EC50 instead of EC50) usually helps. It is computed from the first and second derivatives of the model at the estimates.

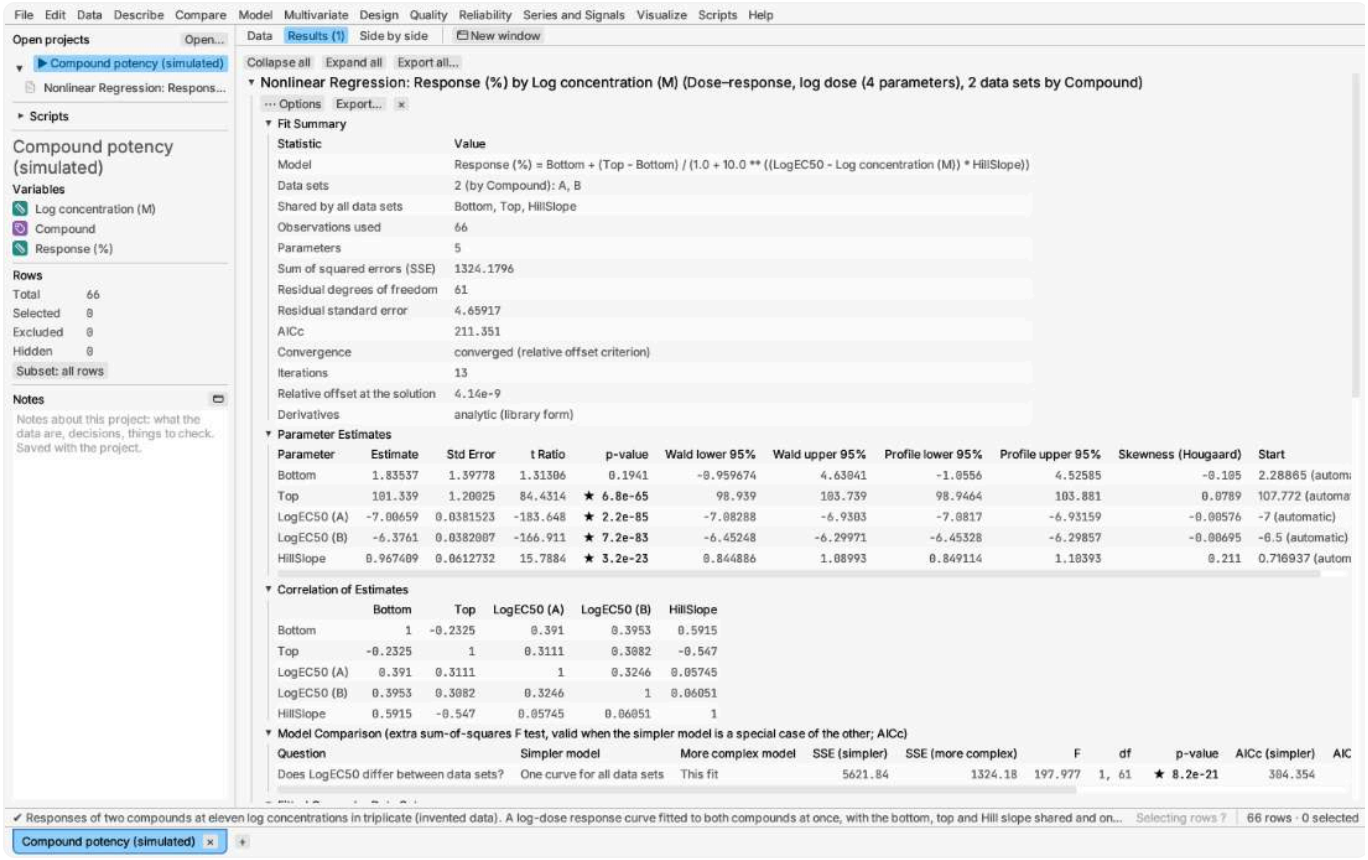
- *Correlation of Estimates:* correlations near ±1 mean the data barely separate two parameters; a different parameterization or more data over the curve's range helps.
- *Fitted Curve:* the data with the fitted curve, the confidence band for the mean (shaded) and prediction limits for single new observations (thin lines), both by the delta method. A formula that reads several columns shows observed against predicted values instead. Points are linked to their rows. With data sets, *Fitted Curves by Data Set* draws one curve per data set in the data set's colour and marker. An implicit curve is drawn over the points without bands.
- *Fitted Surface* (a formula in two columns): a filled contour map of the fitted surface with its colour bar and the observations (see Contour maps).
- *Model Comparison* (with data sets, or a curve to compare with): each row asks one question — *Does LogEC50 differ between data sets?* (the fit against the same model with that parameter shared), *Do the data sets differ?* (against one curve for all), or *Dose–response or Logistic?* — and answers it two ways. The **extra sum-of-squares F test** `F = ((SSE₁ − SSE₂)/(df₁ − df₂)) / (SSE₂/df₂)` applies when the simpler model is a special case of the other. **AICc** `N ln(SSE/N) + 2K + 2K(K + 1)/(N − K − 1)`, with `K` the estimated parameters plus one, also compares models that are not nested; the probability that the more complex model is the better one is `e^{−Δ/2} / (1 + e^{−Δ/2})` with `Δ` its AICc minus the simpler model's (Burnham & Anderson 2002). No comparisons are made for a robust fit.
- *Outliers* (when outliers are removed): each flagged row with its value and its residual from the robust fit, or a note that none was found.
- *Residuals vs Predicted:* residuals against fitted values with a zero line (*Distances to the Curve* for an implicit fit: each point's signed distance to the curve).
- *Inverse Prediction* (rows with a response but no predictor): the predictor value at which the curve reaches each such response, with an interval for one new observation — every `x` whose prediction limits contain the response (for a straight line this is the classical calibration interval, Draper & Smith 1998, §3.2). An interval that the data

range does not close is reported as open, and a response the curve reaches more than once is noted. This reads unknown samples off a standard curve.

- *Prediction Formula:* the fitted model as a formula, with **Copy formula**.

⋯ Options saves predicted values (every row the model can be computed on, excluded rows included), residuals, or the prediction formula as a formula column: a library curve with the estimates written in, or your formula preceded by `let Vmax = …;` for each parameter (with data sets, one branch per data set). It also saves inverse predictions, and for an implicit curve the distance of each row to it. A differential equation has no prediction formula. **Response Explorer** and **3-D surface profiler** (models with two or more inputs, such as a surface or a curve with data sets) show the fitted response as each input changes. Renaming a column that a model formula uses makes the report fail until you run it again with the new name.





Try it: Seedling growth (simulated), a three-parameter logistic fit; Compound potency (simulated), two compounds fitted together with a separate LogEC50 for each; Assay standard curve (simulated), a weighted dose–response standard curve with an outlier removed and six unknowns read off it; Substrate depletion (simulated), a Michaelis–Menten rate equation fitted as a differential equation.

How the fit is computed

The estimates minimise the sum of squared residuals by the Levenberg–Marquardt method: each step solves the linearised problem with a damping term that shrinks to zero near the solution, where the steps become plain Gauss–Newton steps. The fit has converged when Bates & Watts' relative offset — the remaining Gauss–Newton step measured against the residual standard error — falls below 10^{-8}. Standard errors and bands come from the derivative matrix at the solution. The validation record (`docs/validation.md` in the source repository) lists how the results are checked.

Model: curve templates

Model ▸ Curve templates… fits a ready-made curve for a common laboratory assay. Pick the kind of experiment and a template; the dialog shows the model, asks only for the columns and constants the template needs, and works out starting values from the data. The fit is an ordinary nonlinear regression, so its report has every section that one has (estimates with profile intervals, fitted curve with bands, residuals, model comparison, inverse prediction, prediction formula), plus the results the assay calls for: derived quantities such as Ki, kcat, pEC50 or a half-life with intervals, acceptance checks of a standard curve, a comparison of inhibition mechanisms, a Schild analysis or a relative potency.

Templates are named by the standard model names of the literature cited below. Parameters keep their usual symbols (Kd, Bmax, Vmax, Km, logEC50, Hill, pA2…).

Dialog

- **Experiment:** immunoassay standard curves, receptor binding, enzyme kinetics, pharmacological dose–response, or growth, decay and survival.
- **Template:** the models of that experiment (table below). The model and a sentence on the data layout appear under the choice.
- **Columns:** the **Response** and the **x** column the template names (concentration, log concentration, time, dose…). Some templates read a second column: the inhibitor concentration (inhibition mechanisms), the antagonist concentration (Gaddum–Schild), the radioligand or competitor concentration (kinetics), or which rows are total and which nonspecific binding (text containing "total", or 1 for total and 0 for nonspecific). **Data sets** are optional for most templates (one curve per data set, with the parameters shared or not as set below); the operational model needs the agonist column and relative potency the preparation column, where **Standard preparation** picks the reference (the first data set by default). Standard curves take an optional **Dilution factor** column.
- **Constants:** numbers that are not in the data, such as the radioligand concentration and its Kd for Cheng–Prusoff, the enzyme concentration for kcat, F for ECF, or the starting substrate concentration of a progress curve. A required one must be filled in; an optional one only adds the derived quantities that need it.
- **Parameters** (collapsed): a start (empty: computed from the data), lower and upper bounds (the template's defaults filled in, such as positive constants and a fraction

between 0 and 1), and, with data sets, whether each parameter is shared by all data sets. Equal bounds fix a parameter.
- **More options:** weights (1/Y² by default for 4PL and 5PL standard curves and the Arrhenius equation, 1/Y for clonogenic survival, otherwise equal), the confidence level, profile intervals, and for standard curves the acceptance limits and the CV limit of the quantitation range.

Edit analysis… on a template report reopens this dialog with its settings. Starting values are recomputed from the data.

Templates

Experiment	Template	Model	Derived quantities
Standard curves	Four-parameter logistic (4PL)	`Y = D + (A − D) / (1 + (X/C)^B)`	acceptance, quantitation range, sample concentrations
	Five-parameter logistic (5PL)	`Y = D + (A − D) / (1 + (X/C)^B)^G`	as 4PL
Receptor binding	Saturation, one site	`Y = Bmax·X / (Kd + X)`	
	Saturation, total and nonspecific together	total `Bmax·X/(Kd + X) + NS·X + Background`, nonspecific `NS·X + Background`	
	Saturation, two sites	`Bmax1·X/(Kd1 + X) + Bmax2·X/(Kd2 + X)`	
	Competition, one site	`Bottom + (Top − Bottom) / (1 + 10^(X − logIC50))`	IC50, Ki (Cheng–Prusoff), pKi
	Competition, two sites	two IC50s with fraction F	IC50s and Ki of each site
	Homologous competition	`Bmax·[L] / ([L] + 10^X + 10^logKd) + NS`	Kd, pKd

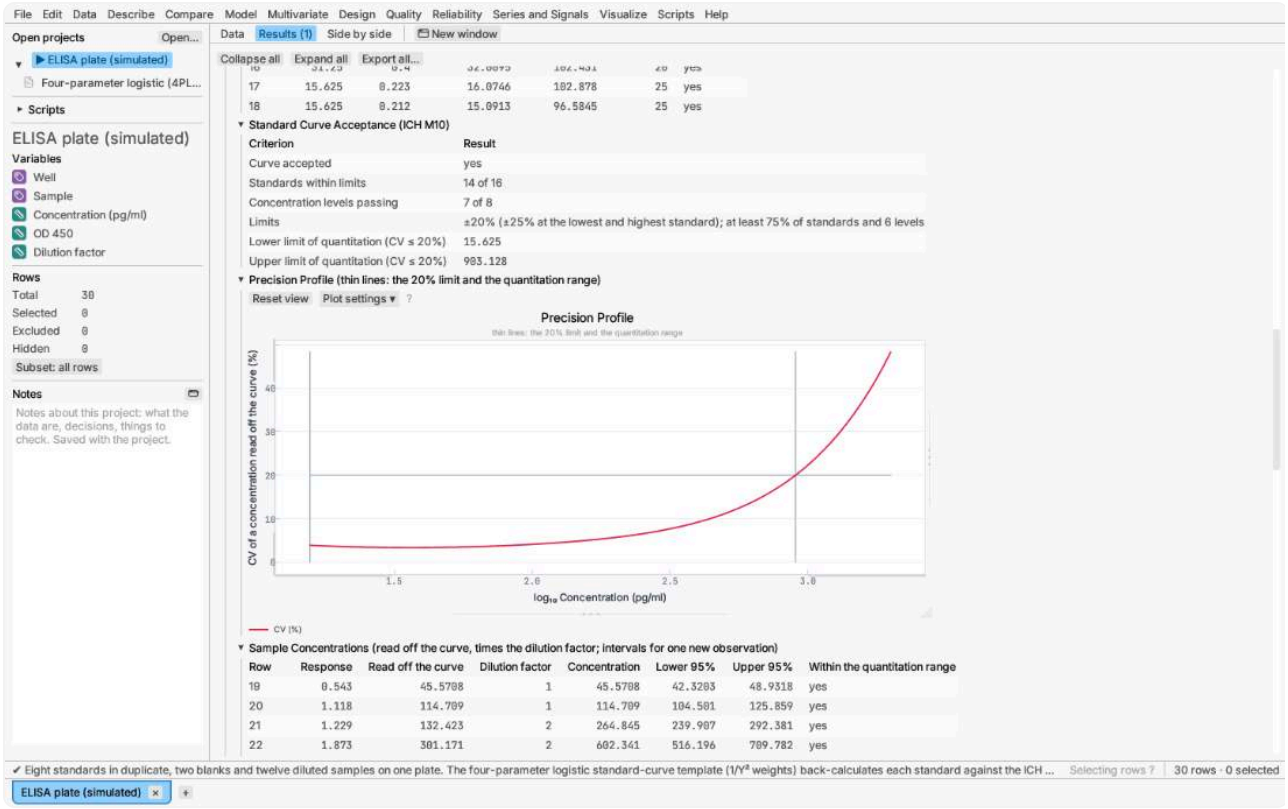
Experiment	Template	Model	Derived quantities
	Association, one ligand concentration	`Ymax·(1 − exp(−kob·X))`	half-time; kon and Kd given koff
	Association, several concentrations	global fit of kon and koff	Kd = koff/kon, dissociation half-life
	Dissociation	`(Y0 − NS)·exp(−koff·X) + NS`	half-life
	Kinetics of an unlabelled competitor	Motulsky–Mahan, k1 and k2 known	competitor Kd, residence time, half-life
Enzyme kinetics	Michaelis–Menten	`Vmax·X / (Km + X)`	kcat = Vmax/[E], kcat/Km
	Substrate inhibition	`Vmax·X / (Km + X·(1 + X/Ksi))`	
	Allosteric (Hill)	`Vmax·X^h / (Khalf^h + X^h)`	
	Competitive, uncompetitive, noncompetitive, mixed inhibition	global fits over substrate and inhibitor	alpha·Ki (mixed); mechanism comparison
	Tight-binding inhibition (Morrison)	quadratic in enzyme, inhibitor and apparent Ki	
	Progress curve	integrated Michaelis–Menten, `P = S0 − Km·W((S0/Km)·exp((S0 − Vmax·t)/Km))`	kcat given [E]
Dose–response	Agonist	`Bottom + (Top − Bottom) / (1 + 10^((logEC50 − X)·Hill))`	EC50, pEC50, ECF
	Inhibitor	`Bottom + (Top − Bottom) / (1 + 10^((X − logIC50)·Hill))`	IC50, pIC50, ICF

Experiment	Template	Model	Derived quantities
	Bell-shaped, biphasic	two Hill terms	EC50 of each phase
	Gaddum–Schild	global fit, EC50 shifted by `1 + ([B]·10^pA2)^SchildSlope`	KB, EC50; Schild plot; slope = 1 test
	Operational model of agonism	Black–Leff, τ and KA, global over agonists	KA, τ, log(τ/KA)
	Relative potency from parallel curves	common Bottom, Top, Hill; logEC50 per preparation	relative potency; parallelism test
Growth, decay, survival	Modified Gompertz	Zwietering form of ln(N/N0)	doubling time
	Baranyi–Roberts	lag, maximum rate and plateau	doubling time
	One- and two-phase decay, one-phase association, plateau then decay	exponential forms	half-lives, percent fast
	Linear-quadratic cell survival	`PE·exp(−(alpha·D + beta·D²))`	alpha/beta, SF2, D10
	Boltzmann	`Bottom + (Top − Bottom) / (1 + exp((V50 − X)/Slope))`	
	Arrhenius	`k = exp(lnA − 1000·Ea / (R·T))`, Ea in kJ/mol	A

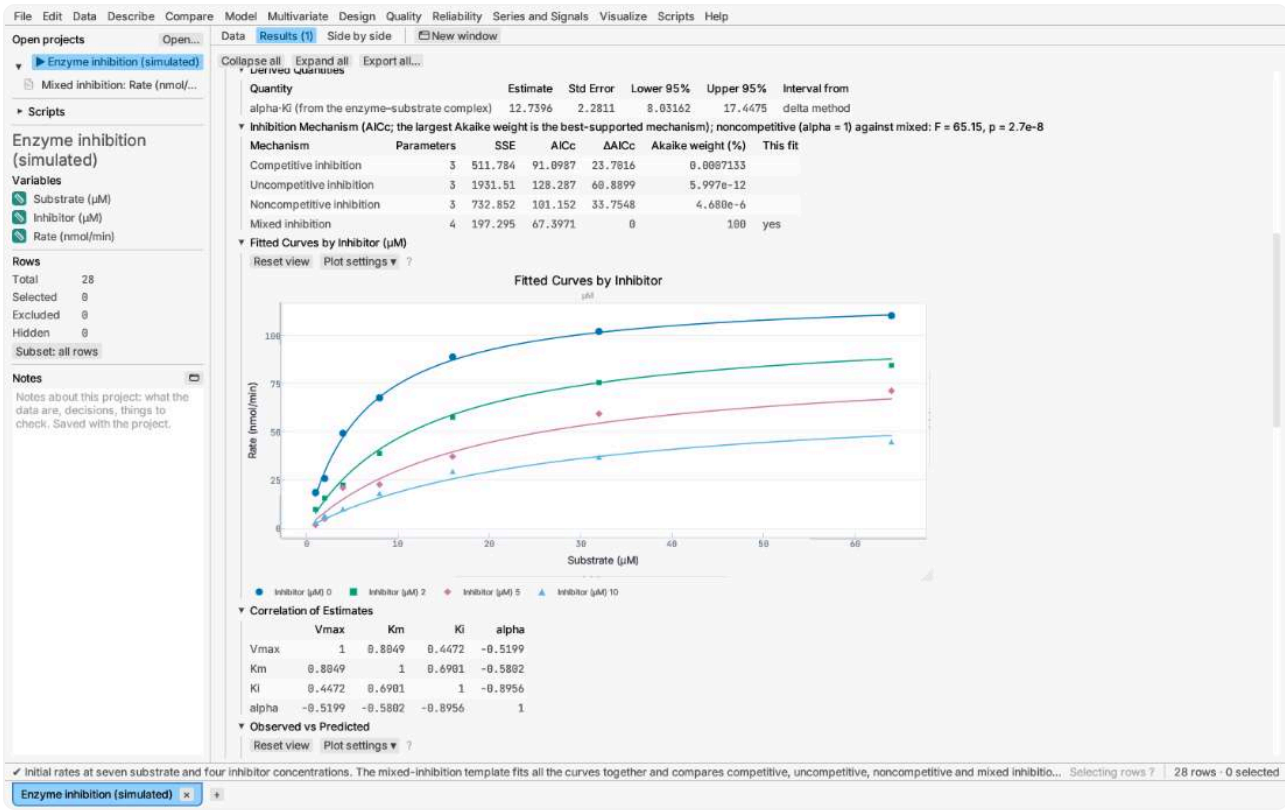
Report

Every template report opens with an **Assay Template** section (the template, its model and what each parameter means) and a **Derived Quantities** table (per data set when parameters are not shared). A derived quantity that is a monotone function of one parameter (EC50 from logEC50, a half-life from a rate) takes that parameter's profile interval mapped through the function; others (Ki by Cheng–Prusoff, kcat/Km, alpha·Ki) use the delta method with the estimates' covariance. Templates with a second column add **Fitted Curves by** that column. The rest depends on the template:

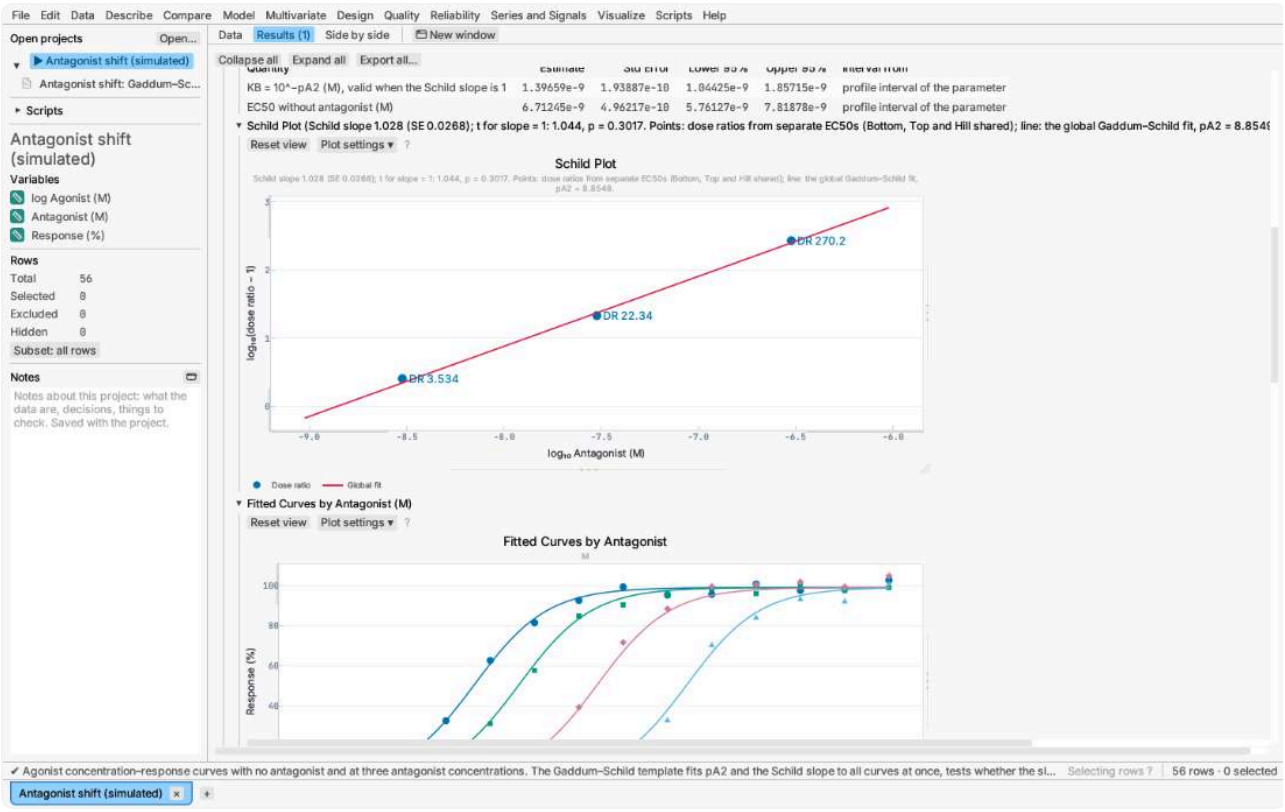
- **Standard curves:** the standard curve on a $\log_{10}$ concentration axis; **Back-Calculated Standards**, each standard read off the curve with its recovery against the limit (±20%, ±25% at the lowest and highest standard); **Standard Curve Acceptance**: the curve passes when at least 75% of the standards and at least six concentration levels are within their limits (ICH M10 for ligand-binding assays; all four limits can be changed), with the lower and upper limits of quantitation; the **Precision Profile**, the CV of a concentration read off the curve (prediction interval half-width ÷ slope ÷ concentration) across the standards' range, whose widest stretch below the CV limit (20% by default) is the quantitation range; and **Sample Concentrations**: every row with a response but no concentration read off the curve, times its dilution factor, with its interval and whether it lies inside the quantitation range. Rows with a concentration of zero (blanks) are fitted but not back-calculated.



- **Inhibition mechanisms:** the four reversible mechanisms fitted to the same rows, ranked by AICc with Akaike weights, and the F test of noncompetitive (alpha = 1) against mixed inhibition.



- **Gaddum–Schild:** the **Schild Plot** of log(dose ratio − 1) against log antagonist concentration, the dose ratios coming from separate EC50s per antagonist concentration (Bottom, Top and Hill shared), with the global fit's line; the test of a Schild slope of 1 is in the title and, as an F test, under Model Comparison.



- **Relative potency:** each preparation's EC50 against the standard's, with its interval from Finney's method on the difference of log EC50s; the parallelism test (common Bottom, Top and Hill against separate curves) is under Model Comparison.

Try it: ELISA plate (simulated), a 4PL standard curve with blanks, eight standards and twelve diluted samples; Receptor saturation (simulated); Competition binding (simulated), IC50 and Ki; Enzyme inhibition (simulated), a mixed inhibitor with the mechanisms compared; Antagonist shift (simulated), a Schild analysis; Bacterial growth (simulated), Gompertz curves of three strains; Clonogenic survival (simulated); Potency assay (simulated), a test lot against a reference.

Starting values and fitting

Each template computes its starts from the data: plateaus from the smallest and largest mean responses, a midpoint from where the curve crosses half its span (interpolated on a

log scale for concentrations), rates from the time to half the change, and for the global models (inhibition, binding kinetics, Schild) the starts of the simpler one-curve model. Starts are moved inside the template's bounds. The fit is then the nonlinear regression's Levenberg–Marquardt least squares with central-difference derivatives.

Every template is checked against an independent fit of the same model with SciPy's `least_squares` (the validation record, `docs/validation.md` in the source repository, has the tolerances).

References

ICH M10 (2022), *Bioanalytical method validation and study sample analysis*; Findlay & Dillard (2007), *AAPS J* 9(2); Gottschalk & Dunn (2005), *Anal. Biochem.* 343; Cheng & Prusoff (1973), *Biochem. Pharmacol.* 22; Motulsky & Mahan (1984), *Mol. Pharmacol.* 25; Hulme & Trevethick (2010), *Br. J. Pharmacol.* 161; Cornish-Bowden (2012), *Fundamentals of Enzyme Kinetics*; Morrison (1969), *Biochim. Biophys. Acta* 185; Copeland (2013), *Evaluation of Enzyme Inhibitors in Drug Discovery*; Arunlakshana & Schild (1959), *Br. J. Pharmacol.* 14; Lew & Angus (1995), *Trends Pharmacol. Sci.* 16; Black & Leff (1983), *Proc. R. Soc. B* 220; Finney (1978), *Statistical Method in Biological Assay*; Zwietering et al. (1990), *Appl. Environ. Microbiol.* 56; Baranyi & Roberts (1994), *Int. J. Food Microbiol.* 23; Fowler (1989), *Br. J. Radiol.* 62; Franken et al. (2006), *Nat. Protoc.* 1.

Multivariate

The Multivariate menu is in four groups, each under a small heading: **Relationships** (correlations, scatterplot matrix, outlier distances, item analysis), **Reduce dimensions** (principal components, factor analysis, correspondence analysis, and MDS, t-SNE and UMAP maps), **Clusters** (clustering, Gaussian mixtures, latent classes), **Groups** (multivariate analysis of variance, discriminant analysis) and **Curves** (functional data).

Correlations

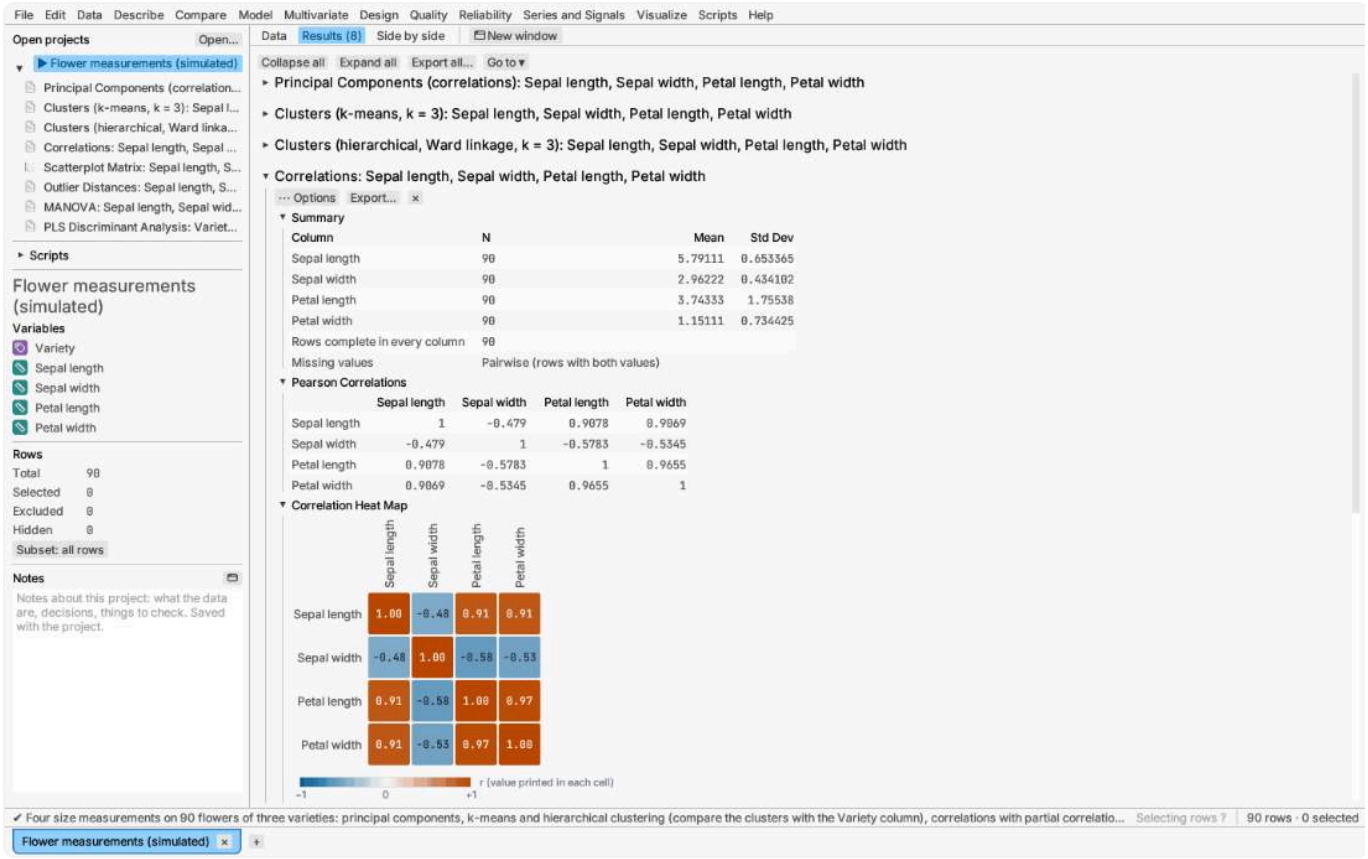
Multivariate ▸ Correlations… measures how each pair of numeric columns moves together. Choose the columns, then:

- **Rows with a missing value:** *pairwise* uses, for each pair, every row that has both values (pairs may then rest on different rows); *listwise* uses only rows complete in every chosen column, so all pairs share one set of rows. Excluded rows are always left out.
- **Confidence level** of the intervals (95% by default).
- Optional tables: **Spearman's rank correlations**, **partial correlations** (on by default) and the **covariance matrix**.

Report:

- *Summary* — each column's count, mean and standard deviation, and the number of rows complete in every column.
- *Pearson Correlations* — the matrix of r.
- *Correlation Heat Map* — the same matrix as coloured cells: blue for negative, orange for positive, stronger colour for a larger |r|. Every cell also prints its value, so the colour is never needed to read it; hover a cell to see which pair it is. The key under the grid shows the colour scale. Right-click the section heading to save it as a figure.
- *Pairwise Pearson Correlations* — every pair once with its N, r, the Fisher z interval and the p-value of the test of no correlation.
- *Spearman's ρ* — the same for rank correlations, which measure any monotone association and resist outliers.
- *Partial Correlations* — each pair's correlation with all the other chosen columns held fixed, from the complete rows, with its test on n − p degrees of freedom. A large correlation whose partial correlation is near zero is explained by the other columns.

Try it: Flower measurements (the petal measurements correlate strongly; the sepal width hardly at all); Course evaluation (rank correlations of the answers).



Scatterplot matrix

Multivariate ▸ Scatterplot matrix… (also Visualize ▸ Scatterplot matrix…) plots every pair of the chosen continuous columns in a square grid of linked panels: the panel in row *i*, column *j* has column *i* up and column *j* across, each with a 95% density ellipse (and the correlation written on it), and each column's histogram sits on the diagonal. Optionally colour (and mark) the points by a column, such as a group.

The matrix opens as a graph canvas with the **Scatterplot matrix** chart form, so everything the canvas offers works: drag a box in any panel to select rows, and they are ringed in every panel, in the histograms and in the data grid; add fit lines or smoothers, change the ellipse level, switch the histograms for dot plots, or use a colour, shape or size column.

Try it: Flower measurements (coloured by variety).

Outlier distances

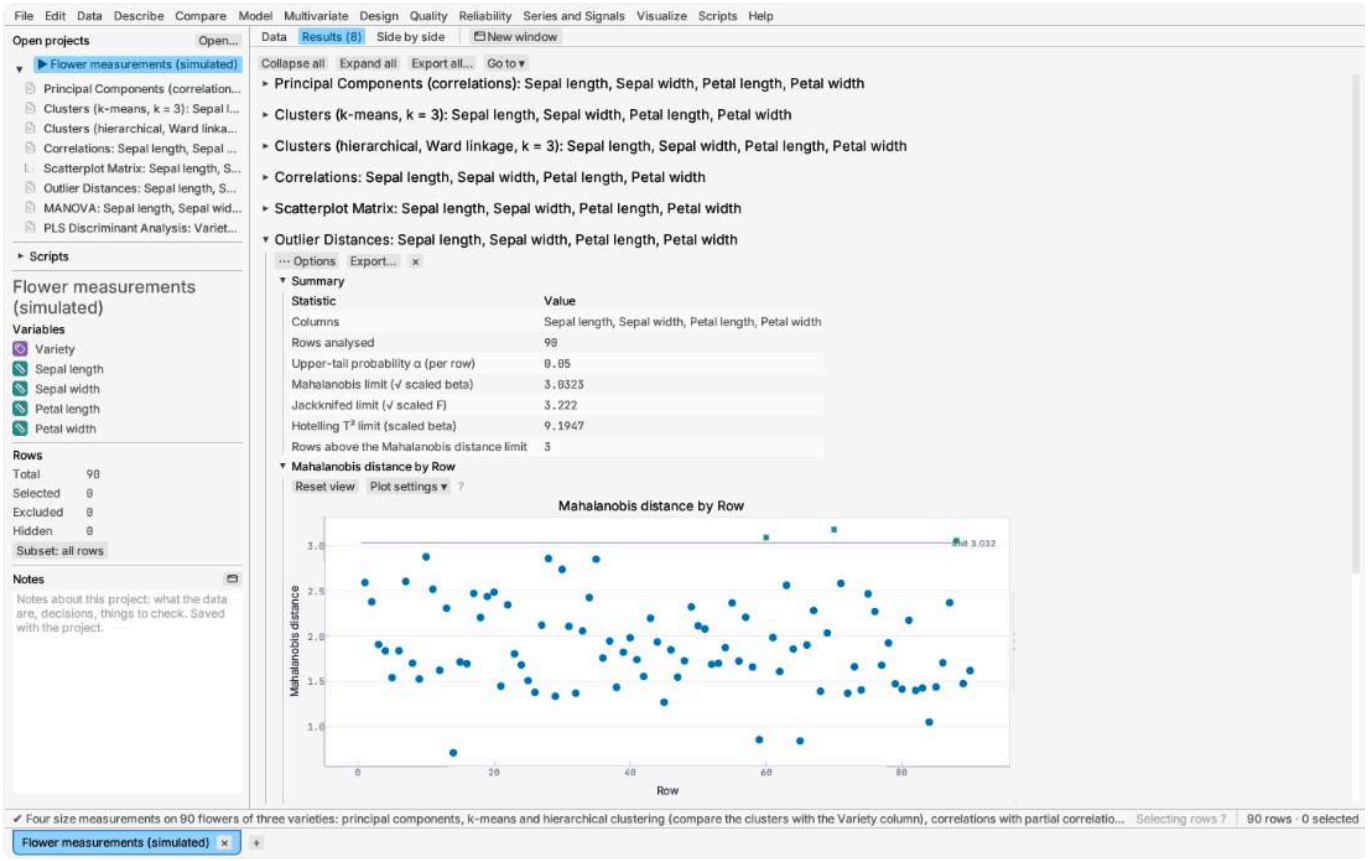
Multivariate ▸ Outlier distances… measures how unusual each row is across several columns at once, allowing for the correlations between them: a row can be ordinary in every column alone and still unusual in combination. Rows complete in every column and not excluded take part.

- **Mahalanobis distance** — distance from the mean of all rows, in units set by their covariance.
- **Jackknifed distance** — distance from the mean and covariance of all the *other* rows, so an outlier cannot hide by pulling the estimates towards itself.
- **Hotelling T^2** — the squared Mahalanobis distance.

Choose which to plot and the upper-tail probability α of the reference limit (0.05 by default). The limits assume multivariate normal data: T^2 uses a scaled beta distribution, the jackknifed distance a scaled F, and the Mahalanobis limit is the square root of the T^2 limit. With many rows, some will cross a per-row limit by chance; use a smaller α to allow for that.

Report: a summary with all three limits; the chosen distance by row number with its limit (rows above it drawn as a different marker, and selectable in the plot); a table of the rows above the limit with all three distances; and the mean and covariance used. The report's options menu saves any of the three distances as a new column.

Try it: Flower measurements.



Item analysis

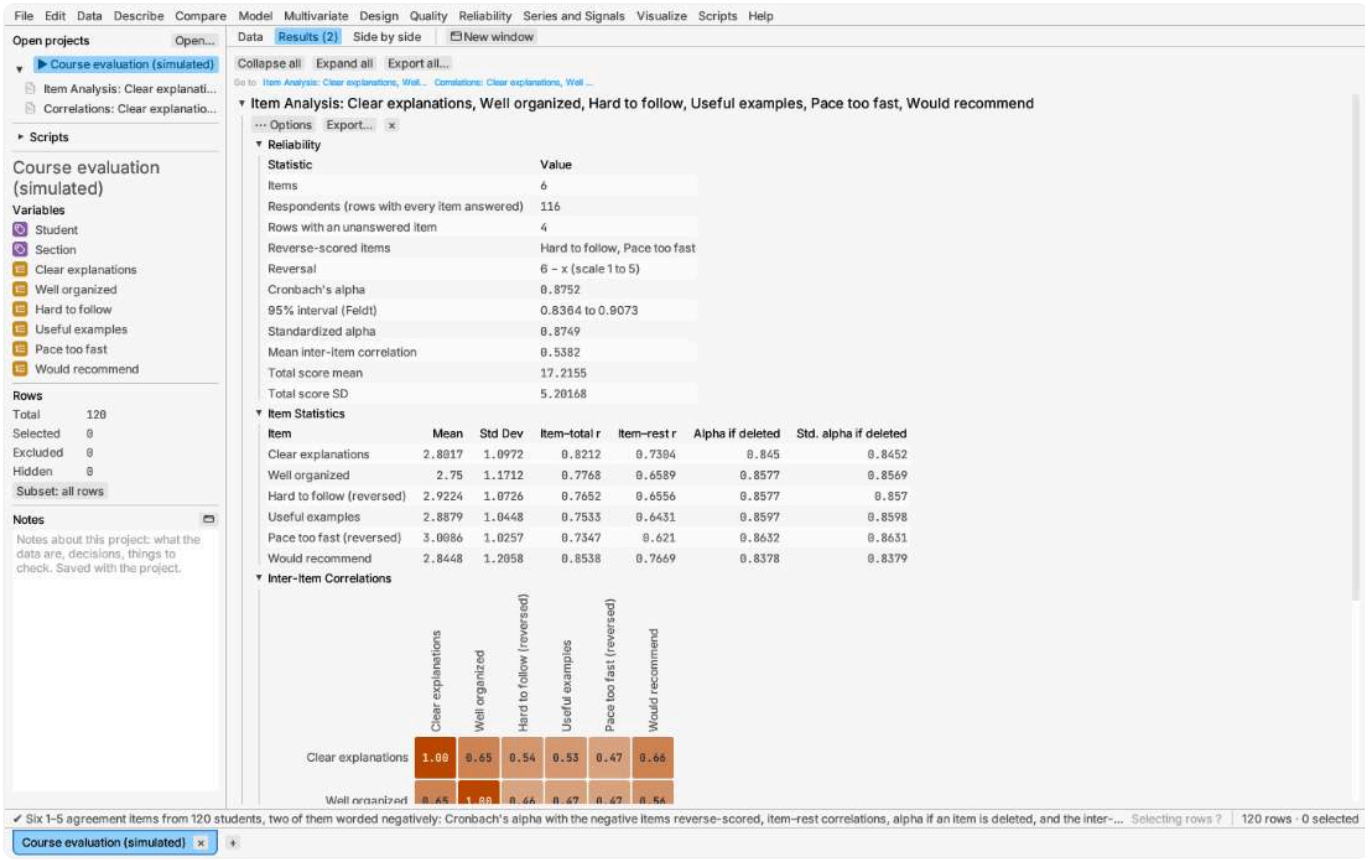
Multivariate ▸ Item analysis… checks the reliability of a scale scored by adding several items, such as the answers to a questionnaire. Tick the item columns; tick **reverse** beside negatively worded items ("The course was hard to follow") so that every item points the same way. A reversed answer *x* becomes low + high − *x*, where low and high are the ends of the answer scale: give them (1 to 5 by default), or clear the box to use the smallest and largest answers given. Only rows with every item answered take part.

Report:

- *Reliability* — **Cronbach's alpha** with Feldt's confidence interval, **standardized alpha** (alpha of the items scaled to equal variance, from the mean inter-item correlation), and the total score's mean and SD. Values around 0.7–0.9 are usual for a scale in use.
- *Item Statistics* — each item's mean and SD; its correlation with the total score (item–total) and with the total of the other items (item–rest, the better guide, since the item–total correlation includes the item itself); and alpha if the item were deleted. An item with a low or negative item–rest correlation, whose deletion raises alpha, does not measure what the others do — or needs reversing.
- *Inter-Item Correlations* — a heat map of the items' correlations, with values printed in the cells.

The options menu saves the **total score** (after reversal) as a new column.

Try it: Course evaluation — then run it again without reversing "Hard to follow" and "Pace too fast" and watch alpha fall and their item–rest correlations turn negative.



Principal components

Multivariate ▸ Principal components… finds the directions of greatest variation in several continuous columns, computed by singular value decomposition. Choose standardized (correlation-based) components, which put every column on an equal footing, or covariance-based ones.

Report: eigenvalues with the share of variation each component explains, a scree plot, loadings, and a score plot with lines showing each variable's loadings. Save the component scores as new columns.

Try it: Flower measurements; Longley economy.

Clustering

Multivariate ▸ Clustering… groups rows by similarity:

- **k-means** with a chosen number of clusters (a row equally close to two centres stays in its current cluster, so repeated rows give the same clusters whatever their order, and no cluster is left empty);
- **hierarchical** clustering with single, complete, average or Ward linkage, drawn as a dendrogram.

Starting centers (k-means). By default k-means runs ten seeded k-means++ starts and keeps the best. Under **Starting centers** choose **Given centers** to start once from centers you type (one row per cluster, one value per column, in the data's units; the number of rows is the number of clusters), or **Given and seeded** to add them as one more start beside the seeded ones, keeping whichever gives the smallest within-cluster sum of squares (the given start wins a tie). **Read from table** copies the centers from another open table that has columns with the same names and one row per center (up to 50). When the columns are standardized the centers are standardized the same way. The report adds a **Starting Centers** table saying which start won. Lloyd's iterations from given centers match R's `kmeans(x, centers = …, algorithm = "Lloyd")`.

Report: cluster means and plots, and for k-means each cluster's average silhouette width (how much closer its rows are to their own cluster than to the next nearest, from −1 to 1; Rousseeuw 1987). Save each row's cluster number as a column.

Choosing the number of clusters. For k-means, tick **Compare numbers of clusters (gap statistic and silhouettes)** and set the largest k to try and the number of reference data sets. The report adds a table and two plots over k = 1 … the largest k:

- the **gap statistic** (Tibshirani, Walther & Hastie 2001): how much tighter the clusters are than in reference data with no clusters, drawn uniformly over **the data range** of each column or over **the principal-component box** (the range of the data's rotated principal components, better when the columns are correlated). Each reference set has its own seed, so the result is reproducible. The suggested k is the smallest with Gap(k) ≥ Gap(k+1) − s(k+1);
- the **average silhouette width** for each k ≥ 2; the suggested k is the largest.

Both suggestions are marked in the table; they often agree, and when they do not, look at the cluster plot of each.

Distances. Hierarchical clustering compares rows by Euclidean distance unless you choose another **Distance** in the dialog: any of the measures in Distance measures — Gower lets the columns be text, ordinal or numeric, with missing values — or **The columns are a distance matrix** (one column per row, as made by Data ▸ Reshape ▸ Distance matrix…). k-means always uses Euclidean distances. With another distance the cluster plot maps the rows by their first two principal coordinates, and cluster means are of the numeric columns.

Try it: Flower measurements; Shrub traits (average linkage on Gower distances of mixed traits).

Correspondence analysis

Multivariate ▸ Correspondence analysis… maps the rows and columns of a two-way table of counts: an inertia table and a symmetric map where nearby categories are associated.

Try it: Customer survey.

Multivariate analysis of variance

Multivariate ▸ Multivariate analysis of variance… (MANOVA) tests whether the effects of a linear model shift several continuous responses together. Testing each response separately

ignores their correlations and multiplies the chances of a false alarm; MANOVA tests every effect on all the responses at once.

Choose two or more **responses** (continuous columns) and the **predictors**. As in the linear model dialog, continuous predictors enter as they are and text, nominal or ordinal columns as categorical factors, and the effect list adds interactions and powers (kept hierarchical unless you turn that off under More options). Rows not excluded with every response and predictor present take part.

More options:

- **Rank effects by** — the statistic whose p-values order the Effect Pareto (Pillai's trace by default, the most robust to unequal covariance matrices); the report shows all four.
- **Canonical plot of** — the categorical factor whose canonical plot is drawn (by default the first one in the model).
- **Contrasts among levels** — choose a factor, give each level a weight (weights adding to zero, such as 1 and −1 to compare two levels, or 2, −1 and −1 to compare one level with the average of two others), and Add contrast. Each contrast is tested on all the responses, on the factor's least-squares means (each level's mean with the other factors balanced).

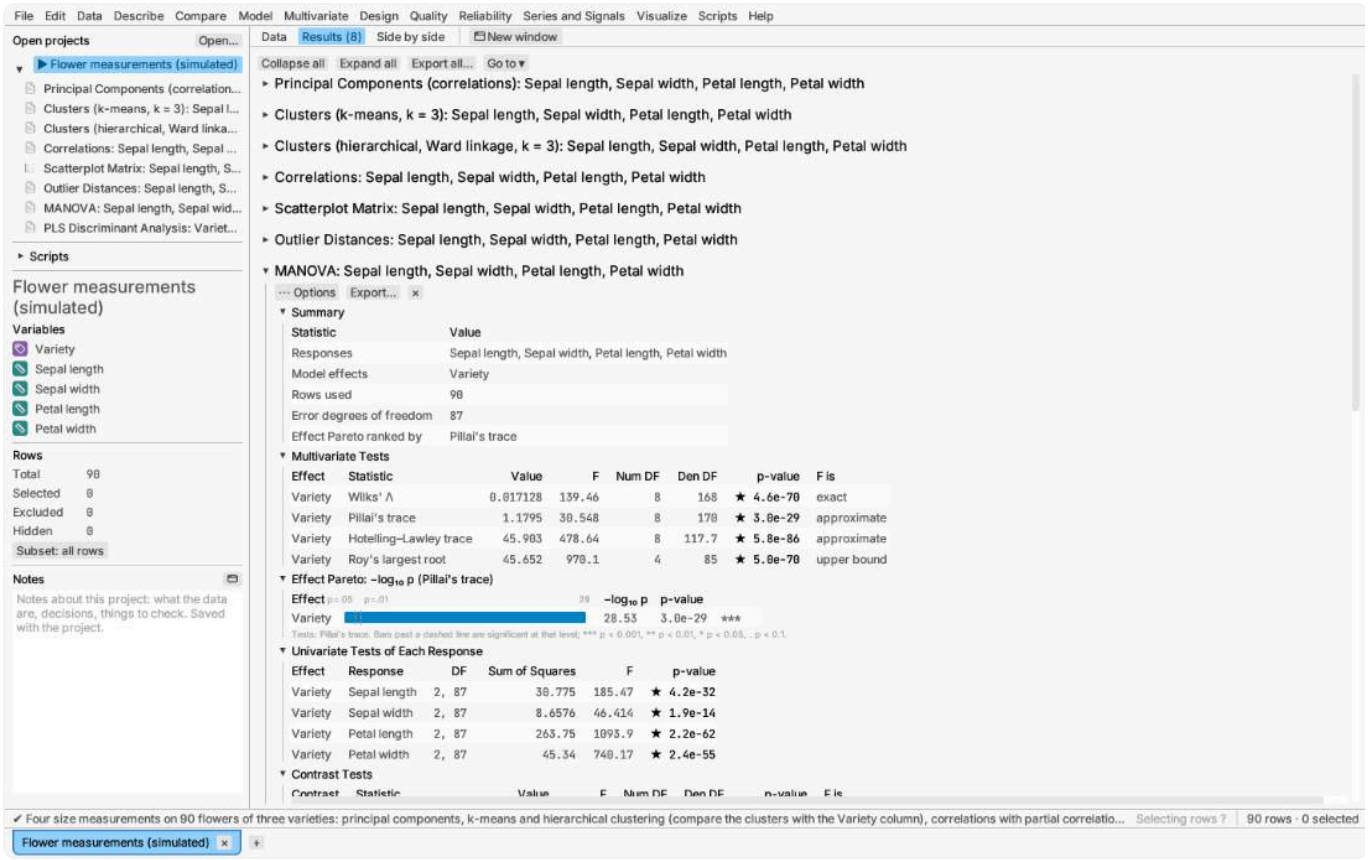
Report:

- **Multivariate Tests** — for each effect, Wilks' Λ, Pillai's trace, the Hotelling–Lawley trace and Roy's largest root, each with its F approximation, degrees of freedom and p-value. The last column says whether the F is exact (always when the effect has one degree of freedom or there is a single canonical variate), an approximation, or (for Roy's root) an upper bound, so its p-value is a lower bound.
- **Effect Pareto** — $-\log_{10} p$ of each effect by the chosen statistic.
- **Univariate Tests of Each Response** — each response's own F-test of each effect, the same as its linear model's effect tests.
- **Contrast Tests** and **Contrast Estimates** — each contrast's exact F test and its estimated value for each response.
- **Canonical Analysis of Each Effect** — the eigenvalues of $E^{-1}H$ (the effect's variation relative to the error variation), the share of each, the canonical correlations, and the standardized coefficients of each canonical variate: the combination of responses the effect separates most, scaled by each response's error standard deviation so they can be compared.
- **Canonical Plot** — the rows' scores on the first two canonical variates of the chosen factor, marked and coloured by level with each level's mean labelled; with a single

variate (a two-level factor), the scores by level. Points are linked to the data, so selecting them selects the rows.

- **Residual Correlations** — each response's error standard deviation and the correlations of the responses after the model (partial correlations).
- **Equal Covariance Matrices (Box's M)** — when every predictor is categorical, Box's test that every cell of the factors has the same covariance matrix, with its chi-square approximation. It is sensitive to non-normality; when it rejects, prefer Pillai's trace.

Try it: Plastic film (Johnson & Wichern), a two-way MANOVA; Flower measurements, a one-way MANOVA by variety with the contrast A vs B.



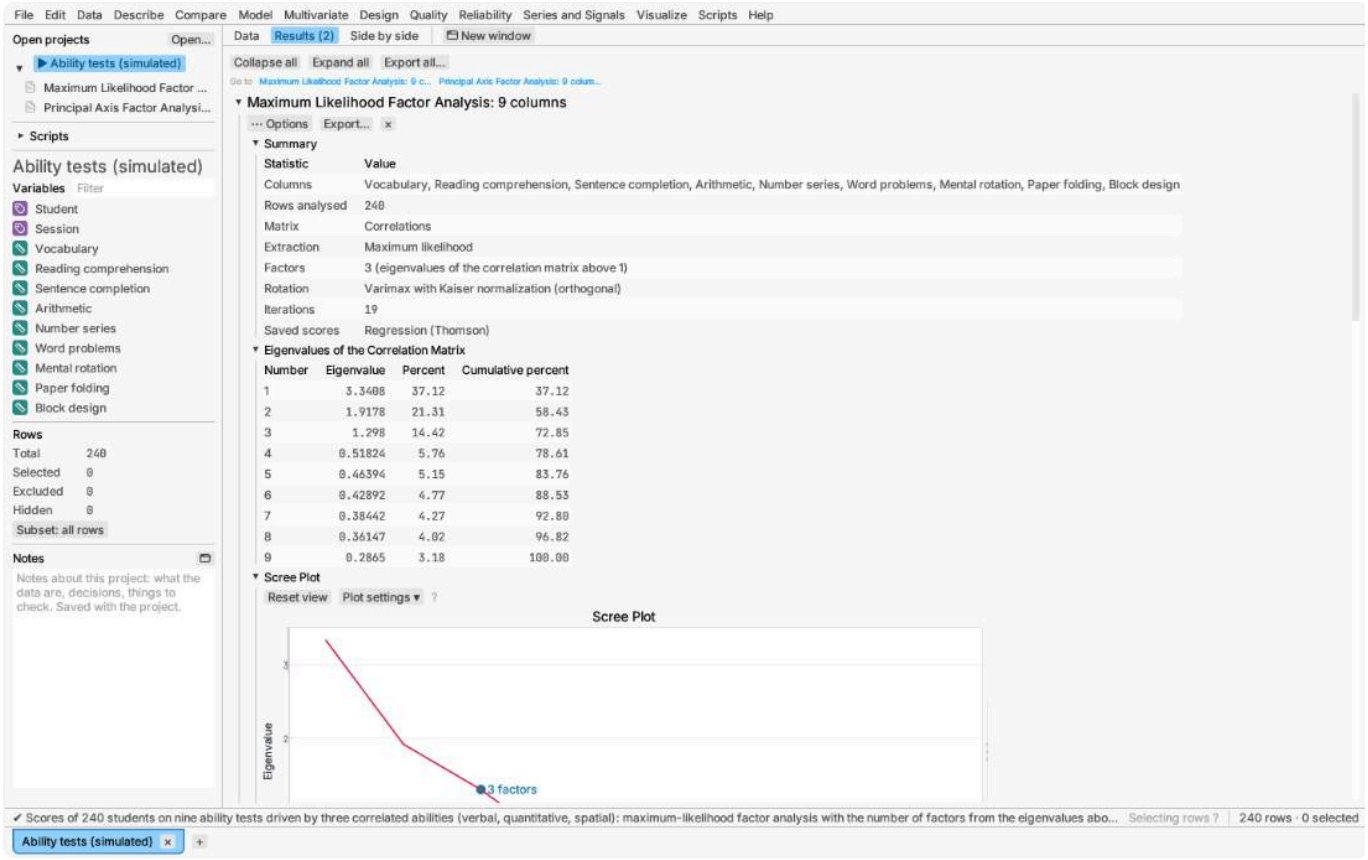
Factor analysis

Multivariate ▸ Factor analysis… looks for a few unobserved **common factors** that explain the correlations among several numeric columns — for example verbal, quantitative and spatial ability behind a set of test scores. Each column is modelled as a weighted sum of the factors (its **loadings**) plus a part of its own (its **uniqueness**); the share it has in common with the others is its **communality**. Rows complete in every column and not excluded take part.

- **Extraction.** *Maximum likelihood* finds the loadings that make the observed correlations most likely under multivariate normal data, and tests whether the number of factors is enough. *Principal axis* repeats an eigen-decomposition with the communalities on the diagonal until they settle; it makes no distributional assumption. When a column's communality reaches its whole variance (a **Heywood case**) it is held at that bound and named in the summary — usually a sign of too many factors or too few rows. Under principal axis such a column is reported with a communality equal to its variance and a uniqueness of 0, so Bartlett scores (which divide by the uniquenesses) are not available; use regression scores.
- **Number of factors.** By default, as many as the eigenvalues of the correlation matrix above 1 (above their mean for a covariance matrix), or a number you set. Maximum likelihood needs $((p - m)^2 - p - m)/2 \geq 0$ for p columns and m factors; when the eigenvalue rule asks for more, fewer are fitted and the summary gives both numbers.
- **Rotation.** Factors can be turned without changing the fit, to make each column load mainly on one factor. *Varimax* keeps them uncorrelated; *Promax* lets them correlate (more realistic for abilities or attitudes) and reports the **pattern** loadings (each factor's own contribution), the **structure** loadings (correlations of columns with factors, or covariances when covariances are analysed) and the factor correlations.
- **More options:** analyse covariances instead of correlations, the method of the saved scores (*Regression*, which shrinks scores towards 0 but correlates best with the factors, or *Bartlett*, unbiased for each row) and the promax power (4).

Report: a summary (with any Heywood cases); the eigenvalues and a scree plot marking the number of factors; for maximum likelihood, the chi-square test that the factors suffice (Bartlett's correction; a small p-value means more factors are needed); unrotated and rotated loadings; communalities with their starting values (squared multiple correlations) and uniquenesses; the variance each factor accounts for; and a plot of the columns' loadings on factors 1 and 2. The report's options menu saves each factor's scores as a new column.

Try it: Ability tests — a maximum-likelihood solution with varimax, and a principal-axis solution with promax showing correlated factors.



Discriminant analysis

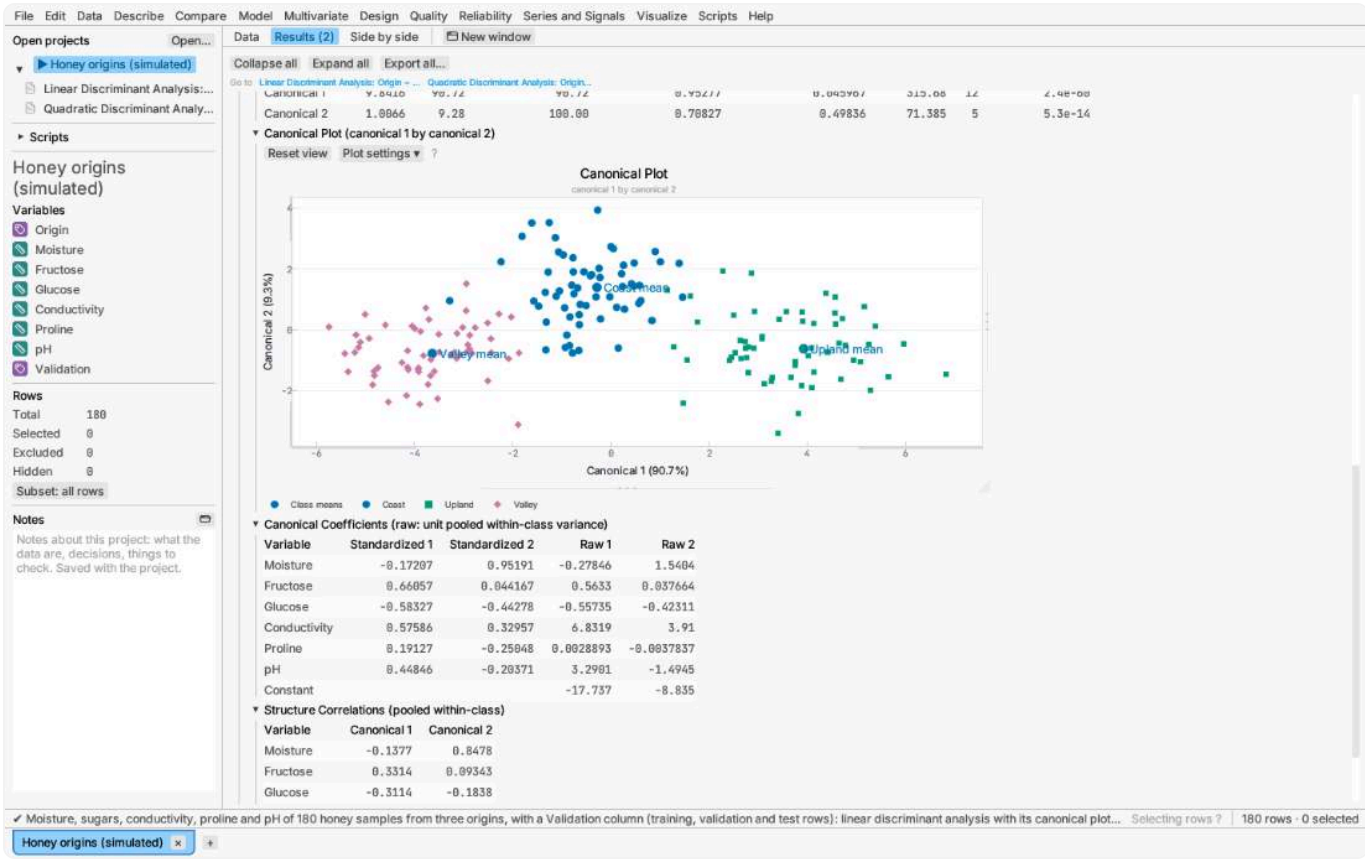
Multivariate ▸ Discriminant analysis… (also under Model with the other classifiers) assigns rows to the classes of a categorical response from numeric measurements, assuming the measurements are multivariate normal within each class.

- **Rule.** *Linear* uses one covariance matrix pooled over the classes, so the boundaries between classes are straight; *Quadratic* gives each class its own covariance matrix (curved boundaries), which needs more rows than predictors in every class.
- **Prior probabilities.** *Proportional* to the training rows of each class, or *Equal*. Each row's **posterior probability** of a class combines its prior with how typical the row is of that class; the most probable class is the prediction.
- **Validation.** The same choices as the other predictive models: every row trains (the classical analysis; its error rates are optimistic), random validation and test rows, roles from a column, or k-fold cross-validation. The rule is fitted to the training rows only; the others are classified with it, so their misclassification rates estimate the error on new data.

Report: a summary; fit measures by role (misclassification rate, log loss, AUC) and confusion matrices of actual by predicted class; the misclassified rows with their posterior probabilities; class means and priors; and **Fisher's canonical discriminant functions** — the weighted sums of the predictors that best separate the classes (at most one fewer than the classes): their eigenvalues, canonical correlations and Wilks' Λ tests (does this function and every later one separate the classes?), a canonical plot of the rows by class (colour and marker) with each class mean, the standardized and raw coefficients, and the structure correlations of each predictor with each function. Points in the canonical plot are rows, so selecting them selects the rows everywhere.

The options menu saves each class's posterior probability, the most likely class, the validation roles and the canonical scores. The saved probabilities can be compared with other models' under Model ▸ Compare models.

Try it: Honey origins — linear and quadratic rules with training, validation and test rows from the Validation column.



Gaussian mixture clustering

Multivariate ▸ Gaussian mixture clustering… treats the rows as drawn from a mixture of normal distributions, one per cluster, each with its own mean, spread and share of the rows. Unlike k-means, clusters can be elongated, tilted and of different sizes, and every row gets a *probability* of belonging to each cluster rather than a hard label.

Choose the numeric columns and the range of numbers of components to try (1 to 6 by default). The **covariance structure** says how the clusters' shapes may differ; each is named by three letters for volume, shape and orientation (E equal across clusters, V varying, I identity):

Structure	Clusters are
EII	round, all the same size
VII	round, of different sizes
EEI	axis-aligned ellipses, all alike
VVI	axis-aligned ellipses, each its own
EEE	tilted ellipses, all alike
VVV	tilted ellipses, each its own

Every ticked structure is fitted with every number of components by the EM algorithm, from several starts (the first from k-means or Ward clustering, the rest from seeded random k-means runs; set them under **More options**), and the model with the smallest **BIC** (−2 log-likelihood + parameters × ln n) is chosen: it balances fit against the number of parameters. Fits that break down — a cluster left with almost no rows, a covariance that cannot be estimated — are shown as *not fitted*. With one column only the two spherical structures differ (equal or unequal variances).

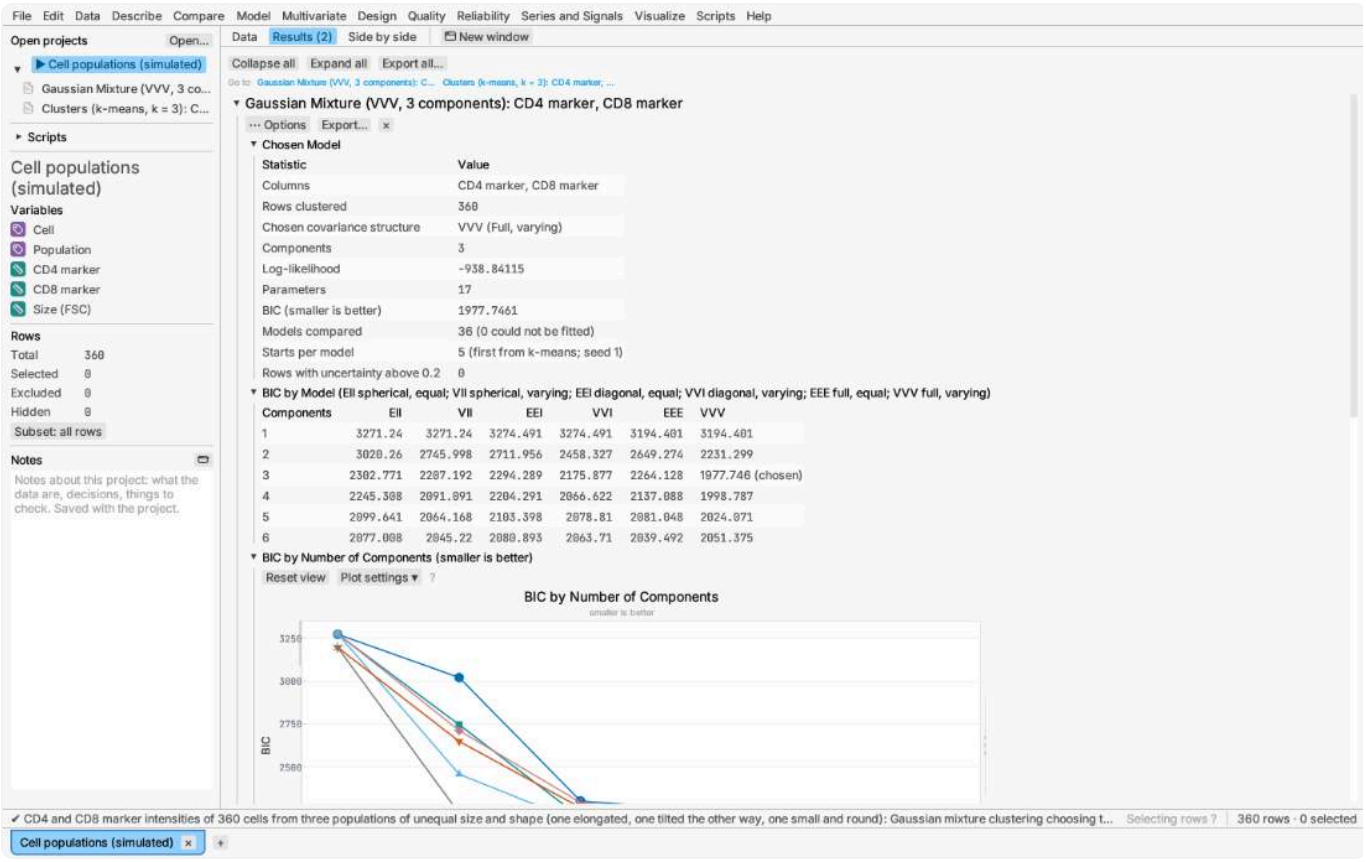
Report:

- *Chosen Model* — the structure, number of components, log-likelihood, parameters and BIC, and how many rows are uncertain (probability of their most likely cluster below 0.8). The log-likelihood and every BIC are those of the values in the columns' own units, also when the columns were standardized for fitting.
- *BIC by Model* — every structure and number of components, with the chosen one marked.
- *BIC by Number of Components* — one line and marker shape per structure.
- *Components* — each cluster's mixing proportion, the rows most likely in it, and its means.

- *Covariance Matrices* — each cluster's (or the shared) covariance matrix.
- *Rows by Most Likely Component* — the rows on the first two columns, coloured and marked by cluster, with each cluster's 95% ellipse. Drag a box to select rows; they are selected everywhere.

The options menu saves each row's **membership probabilities**, its **most likely component** (a nominal column) and its **uncertainty** (1 minus its largest probability).

Try it: Cell populations — the mixture finds the three populations with their own shapes (VVV, 3 components), where k-means cuts the long, tilted population in two; compare both with the Population column.



Latent class analysis

Multivariate ▸ Latent class analysis... looks for a few hidden groups (*classes*) of rows behind several categorical answers — survey items, symptoms, test results. Within a class the items are assumed independent (*local independence*), so everything the items have in common is explained by the class a row belongs to. Each class is described by its **size** and by the probability of each answer to each item.

Choose two or more nominal or ordinal columns as the **items** (continuous columns are not offered; set a coded column's measurement level to nominal or ordinal to use it). An item may have any number of levels — its levels are its distinct non-blank values. Choose the range of numbers of classes to try (1 to 5 by default). Under **More options** set the number of random starts per number of classes (10), the random seed, and a **smoothing** value (0 by default) that adds a small count to every level so that no probability is estimated as exactly 0 or 1.

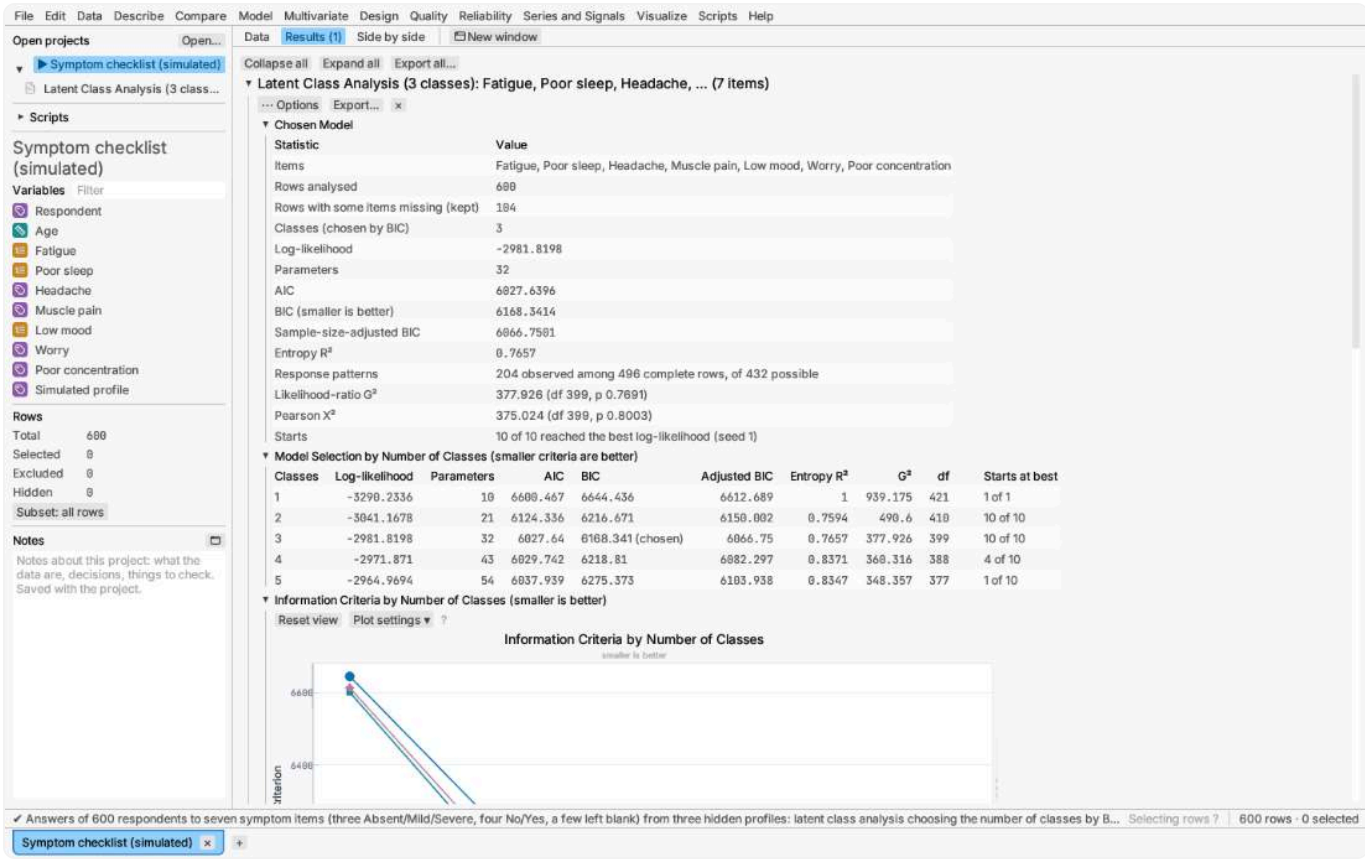
Each number of classes is fitted by the EM algorithm from several random starts, keeping the best; the report says how many starts reached it (if only one did, try more starts). The number of classes with the smallest **BIC** is chosen. A blank answer is simply left out of that row's likelihood, so rows with some items missing still count; rows with every item blank, and excluded rows, are left out. With few items not every number of classes can be estimated — with four yes/no items, at most three classes — and those are listed as *not fitted*. Classes are numbered from the largest to the smallest.

Report:

- *Chosen Model* — number of classes, log-likelihood, parameters, AIC, BIC and the sample-size-adjusted BIC, the **entropy R^2** (1 when every row clearly belongs to one class, near 0 when classes are indistinct), and the likelihood-ratio **G^2** and Pearson **X^2** comparing observed and expected counts of the response patterns. With many items most patterns are rare and these tests are unreliable; compare models by BIC instead.
- *Model Selection by Number of Classes* — the same measures for every number of classes tried, the chosen one marked.
- *Information Criteria by Number of Classes* — BIC, AIC and adjusted BIC against the number of classes.
- *Class Sizes* — each class's estimated share with its standard error, and the rows whose most likely class it is.
- *Item Response Probabilities by Class* — for every item and level, the probability of that answer in each class with its standard error (from the observed information). A probability on the boundary (0 or 1) has no standard error and shows —.
- *Response Probability Profiles* — one pane per item, one marker and stem per class at each level, so the classes' answer patterns can be compared at a glance.
- *Classification* — for the rows assigned to each class, the average probability of every class. Values near 1 on the diagonal mean clear assignments.

The options menu saves each row's **most likely class** (a nominal column) and its **probability of each class**.

Try it: Symptom checklist — BIC chooses three classes: a large group with few symptoms, one with fatigue, poor sleep and muscle pain, and one with low mood and worry. Save the most likely class and compare it with the Simulated profile column.



Dimension reduction

Multivariate ▸ Dimension reduction (MDS, t-SNE, UMAP)… draws the rows as points on a map (two dimensions by default) so that rows that are alike lie close together. Choose a method:

- **Classical scaling** (Torgerson) — the map whose distances best match the rows' distances in the least-squares sense of principal components; for Euclidean distances between rows it equals the principal component scores. Exact and fast.
- **Metric scaling (SMACOF)** — moves the points to make the map distances match the dissimilarities as closely as possible (smallest *stress*), starting from classical scaling and from four seeded random layouts. Better than classical scaling for non-Euclidean distances; up to 3000 rows.
- **t-SNE** — keeps each row's nearest neighbours together, so clusters stand out; distances *between* clusters and cluster sizes mean little. The **perplexity** (30) is about

how many neighbours each row pays attention to; it must be below the number of rows less one. Up to 600 rows use the exact gradient, more use the Barnes–Hut approximation.
- **UMAP** — like t-SNE, keeps neighbourhoods together, and tends to keep more of the larger-scale layout; **neighbours** (15) sets how local the view is and **minimum distance** (0.1) how tightly points may pack.

The columns either hold **variables** — the rows' distances are then the Euclidean distances between their values, standardized by default (under **More options**) so that columns in larger units do not dominate — or a **distance matrix**: one column per row, with row *i* of column *j* the distance between rows *i* and *j* (so the table has as many rows as distance columns). Excluded rows are left out, with their matching columns. **Colour and marker by** colours the map by a column (its categories, or quartiles of a continuous column), each with its own marker shape. Under **More options**: the number of dimensions (1–3), the iterations and the random seed. The same settings and seed always give the same map.

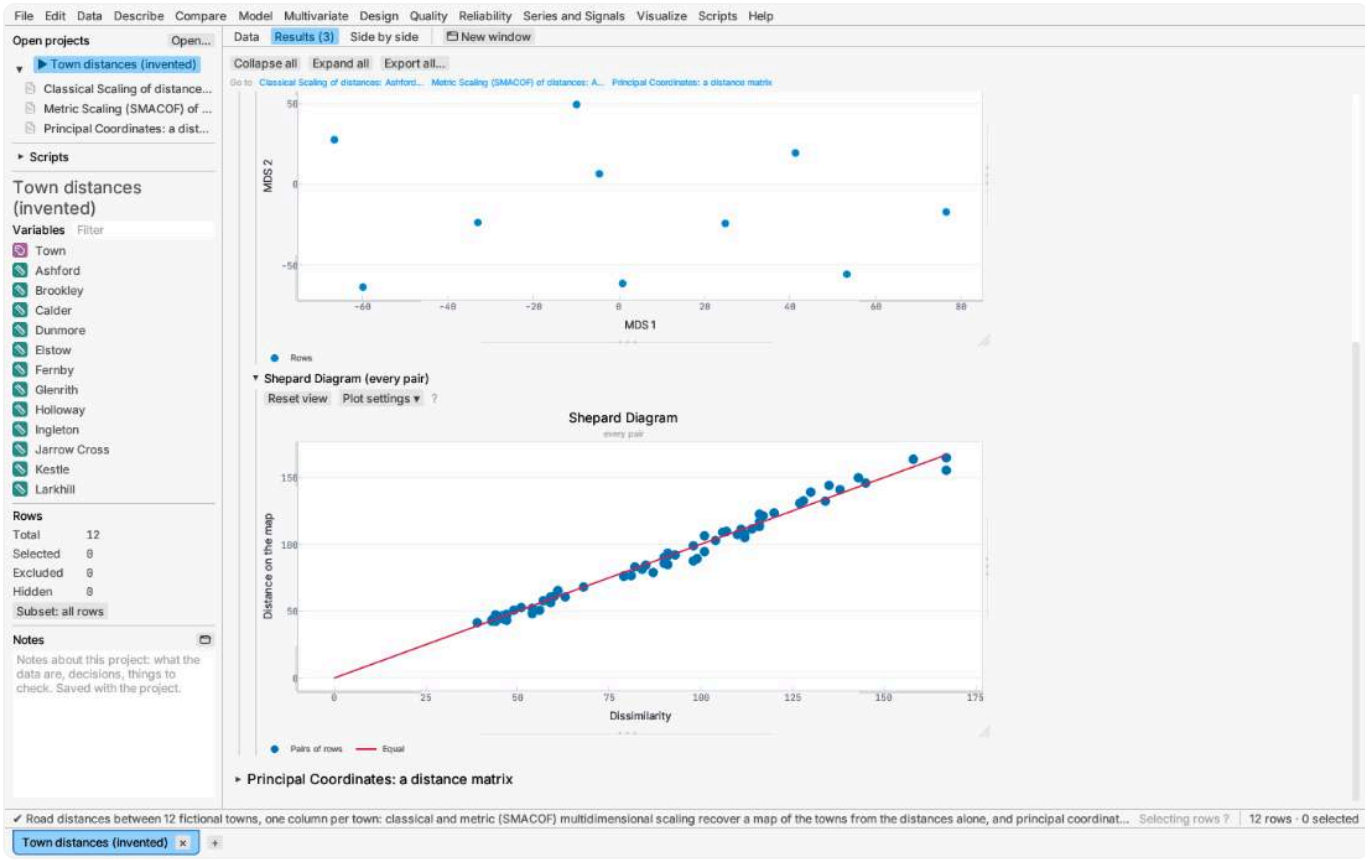
Report:

- *Fit* — the method and settings, and how good the map is: the share of the eigenvalues kept (classical scaling; negative eigenvalues mean the distances are not Euclidean), stress-1 (metric scaling; below about 0.1 is good), the KL divergence and the iterations, with how many of the first ones used early exaggeration (a quarter of them, at most 250; t-SNE), the curve parameters and graph size (UMAP); and for all, the **trustworthiness** — how far the map's nearest neighbours are true neighbours (1 is perfect, about 0.5 is random).
- *Eigenvalues* (classical scaling).
- *Map* — the rows on the first two dimensions; select rows by dragging a box, and they are selected in every other plot and the data grid.
- *Shepard Diagram* (scaling methods) — each pair's dissimilarity against its distance on the map, with the line of equality (up to 1500 pairs drawn at random when there are more).

The options menu saves the map coordinates as new columns.

Try it: Expression profiles (t-SNE, UMAP and classical scaling of 20 genes, coloured by cell type); Town distances (a map of twelve towns from their road distances alone).

File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Expression profiles (simulated)
t-SNE: Gene 01, Gene 02, Gene ...
UMAP: Gene 01, Gene 02, Gene...
Classical Scaling: Gene 01, Gen...
Scripts
Expression profiles (simulated)
Variables
Filter
Cell
Cell type
Gene 01
Gene 02
Gene 03
Gene 04
Gene 05
Gene 06
Gene 07
Gene 08
Gene 09
Gene 10
Gene 11
Gene 12
Gene 13
Gene 14
Gene 15
Rows
Total 300
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
t-SNE: Gene 01, Gene 02, Gene 03, ... (20 columns)
Options
Export...
Fit
Statistic
Value
Method
t-SNE
Input
Euclidean distances between rows on 20 standardized columns
Rows mapped
300
Dimensions
2
Perplexity
30
Gradient
exact gradient
Iterations
1000 (the first 250 with early exaggeration 12)
Learning rate
50
KL divergence
0.4731
Seed
1
Trustworthiness (10 neighbours)
0.9685
Map (coloured and marked by Cell type)
Reset view
Plot settings
Map
t-SNE 2
t-SNE 1
Basal
Immune
Luminal
Stromal
Twenty gene expression values for 300 cells of four types, with a gradient shared across types: t-SNE, UMAP and classical scaling maps coloured by cell type.
Selecting rows ?
300 rows · 0 selected
Expression profiles (simulated)


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Town distances (invented)
Classical Scaling of distance...
Metric Scaling (SMACOF) of ...
Principal Coordinates: a dist...
Scripts
Town distances (invented)
Variables
Filter
Town
Ashford
Brookley
Calder
Dunmore
Elstow
Fernby
Glenrith
Holloway
Ingleton
Jarrow Cross
Kestle
Larkhill
Rows
Total 12
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
MDS 2
MDS 1
Rows
Shepard Diagram (every pair)
Reset view
Plot settings
Shepard Diagram
Distance on the map
Dissimilarity
Pairs of rows
Equal
Principal Coordinates: a distance matrix
Road distances between 12 fictional towns, one column per town: classical and metric (SMACOF) multidimensional scaling recover a map of the towns from the distances alone, and principal coordinat...
Selecting rows ?
12 rows · 0 selected
Town distances (invented)

Principal coordinates

Multivariate ▸ Principal coordinates… (principal coordinates analysis, PCoA; Gower 1966) maps the rows from any distance between them. Choose **Measured from columns** — then the **distance** (see below; Euclidean and Manhattan can standardize the columns first, under More options) — or **The columns are a distance matrix**. Excluded rows are left out (with their matrix columns).

With D the n × n distances, PCoA double-centres $A = -\frac{1}{2}D^2$ (squared element by element): $B = (I - 11^T/n)\, A\, (I - 11^T/n)$, and takes the eigenvalues λ and eigenvectors of B; the coordinates on axis k are the eigenvector scaled by $\sqrt{\lambda_k}$. For Euclidean distances this is principal components; other distances can give **negative eigenvalues** (the distances cannot be drawn exactly in any number of real dimensions). The report keeps them, and under More options **Negative eigenvalues** can be corrected (Legendre & Legendre 2012, §9.3.4):

- **Lingoes** (1971) adds $2c_1$ to every squared distance off the diagonal, with c_1 the absolute value of the most negative eigenvalue;
- **Cailliez** (1983) adds a constant c_2 to every distance off the diagonal, the largest real eigenvalue of $[[0, 2B], [-I, -4B_1]]$ (B_1 built from the distances rather than their squares).

Either makes every eigenvalue non-negative; the correction is applied only when there is a negative eigenvalue, and the constant is reported.

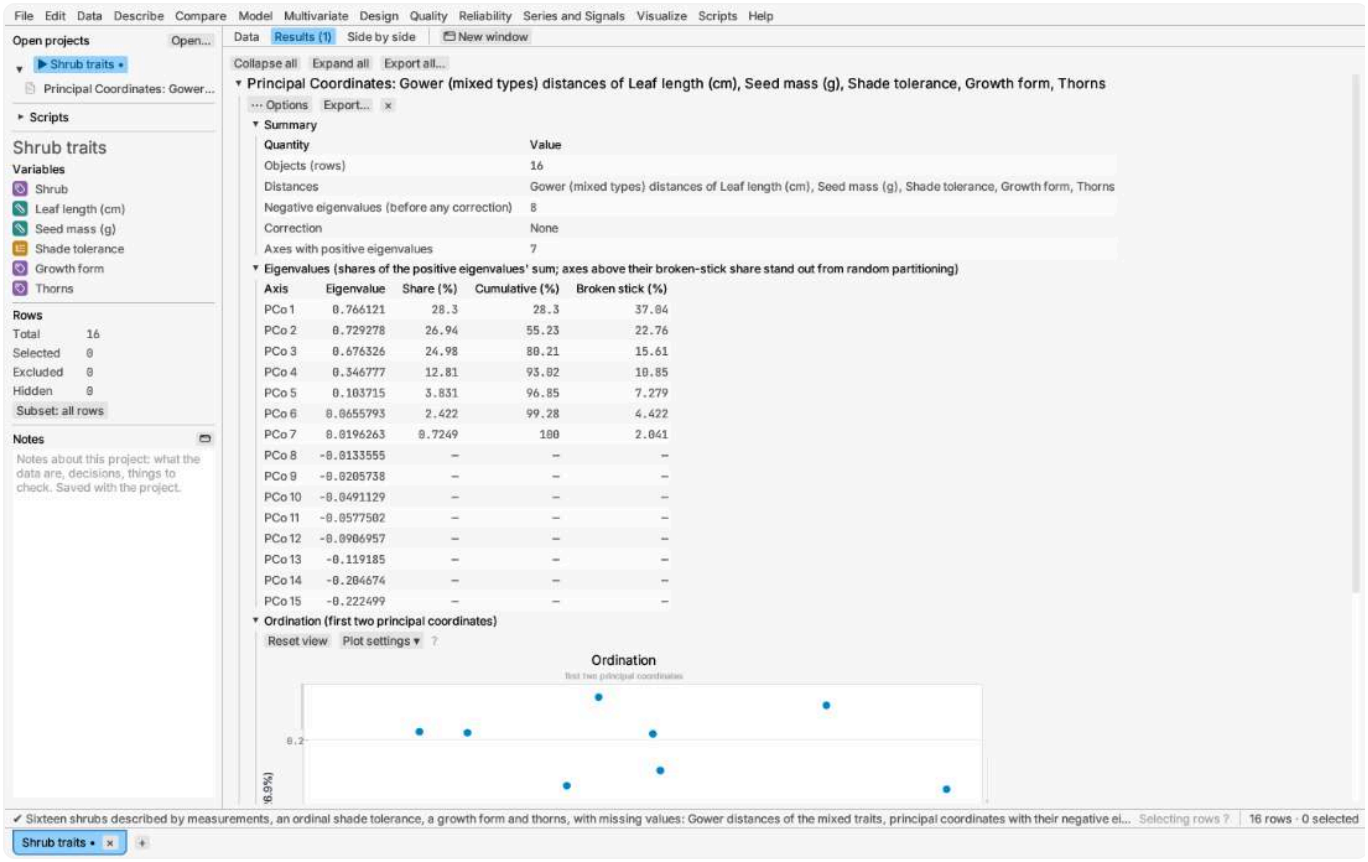
Report:

- *Summary* — distance, objects, rows left out, the correction and its constant, and how many eigenvalues are negative.
- *Eigenvalues* — every axis with its eigenvalue, its share of the sum of the positive eigenvalues, the cumulative share and the **broken-stick** share expected if the variation were split at random (axes above it are worth interpreting).
- *Ordination* — the rows on the first two axes (labelled with their shares), brushable, coloured and marked by an optional **Colour points by** column.
- *Correlations of Continuous Columns with the Axes* — of each continuous column used with the first axes (variables input).

The options menu saves up to three coordinate columns.

Verified: R `cmdscale` and `ape::pcoa` (eigenvalues, coordinates up to sign, both corrections, broken stick).

Try it: Shrub traits (Gower distances of mixed traits, coloured by growth form); Town distances (road distances: compare the negative eigenvalues with the classical scaling map).



Distance measures

Principal coordinates, hierarchical clustering and Data ▸ Reshape ▸ Distance matrix… share these distances between rows x and y over p columns:

Distance	Definition	Columns
Euclidean	$\sqrt{\Sigma(x_j - y_j)^2}$	numeric (optionally standardized)
Manhattan (city block)	$\Sigma \lvert x_j - y_j \rvert$	numeric (optionally standardized)

Distance	Definition	Columns
Gower (mixed types)	$\Sigma\, w_j\, d_j / \Sigma\, w_j$ over the columns present in both rows: numeric $\lvert x_j - y_j \rvert / \text{range}_j$; ordinal the same on the ranks; nominal 0 if equal, else 1	any; missing values allowed
Jaccard (presence/absence)	$1 - a/(a + b + c)$, a = columns present (non-zero) in both, b and c in one only	numeric (non-zero = present)
Bray–Curtis	$\Sigma \lvert x_j - y_j \rvert \,/\, \Sigma(x_j + y_j)$	non-negative numbers (abundances)
Hamming	number of columns that differ	any
Simple matching	share of columns that differ	any

Gower (1971) gives each column a weight w_j of 1 when both values are present and 0 otherwise, so missing values only drop that column from that pair. Text columns and numeric columns marked nominal count as nominal, numeric columns marked ordinal as ordinal. A 0/1 column is compared as an ordinary (symmetric) column: two absences count as agreement. For the other distances a row with a missing value in a chosen column is left out.

Functional data

Multivariate ▸ Functional data… analyses measurements that each trace a curve: a temperature profile of every batch, a growth curve of every plant, a spectrum of every sample. The data are in long form, one row per reading, with a **curve identifier** column (rows with the same value form one curve), a **time** column (where along the curve the reading was taken; any numeric scale, such as wavelength) and a **value** column. Curves need not be read at the same times or have the same number of readings; curves with fewer than three readings are skipped and listed in the summary. Missing values are left out.

- **Basis and smoothing.** Each curve is turned into a smooth function. *B-splines* (piecewise cubic polynomials by default, joined smoothly at equally spaced knots) suit most curves; a *Fourier series* suits curves that repeat with a period (the length of the time range unless you set one). *Smooth by GCV* uses many basis functions and a roughness penalty on the curvature, chosen by generalized cross-validation (one

penalty for all the curves, so they are smoothed alike); *Fixed smoothing* sets the penalty λ yourself; *No smoothing* fits the basis by least squares, so the number of basis functions sets the smoothness. The number of basis functions is chosen for you unless you clear *Automatic number of basis functions*.

- **Registration.** Curves that rise and fall at different times blur the mean curve and mix differences in timing with differences in size. *Add landmark* lines up each curve's maximum or minimum (anywhere, or within a time window you give) at its mean time, by stretching each curve's time axis smoothly between landmarks; several landmarks must occur in the same order in every curve. *Continuous registration to the mean curve* then aligns the whole curve shapes (Ramsay & Silverman's minimum-eigenvalue criterion) with smooth increasing warping functions; More options sets how many functions describe each warp and how strongly bends are penalized.
- **Principal components.** Functional principal components are the typical ways the curves differ from their mean, each an *eigenfunction* of time; every curve gets a score on each. By default enough components are kept to explain 90% of the variation (at most six).
- **Factors** (optional) describe each curve, such as the settings of a designed experiment; they must be constant within each curve. The scores of the kept components are then modelled on the factors (main effects, and optionally two-factor interactions), which gives a predicted curve for any factor settings.

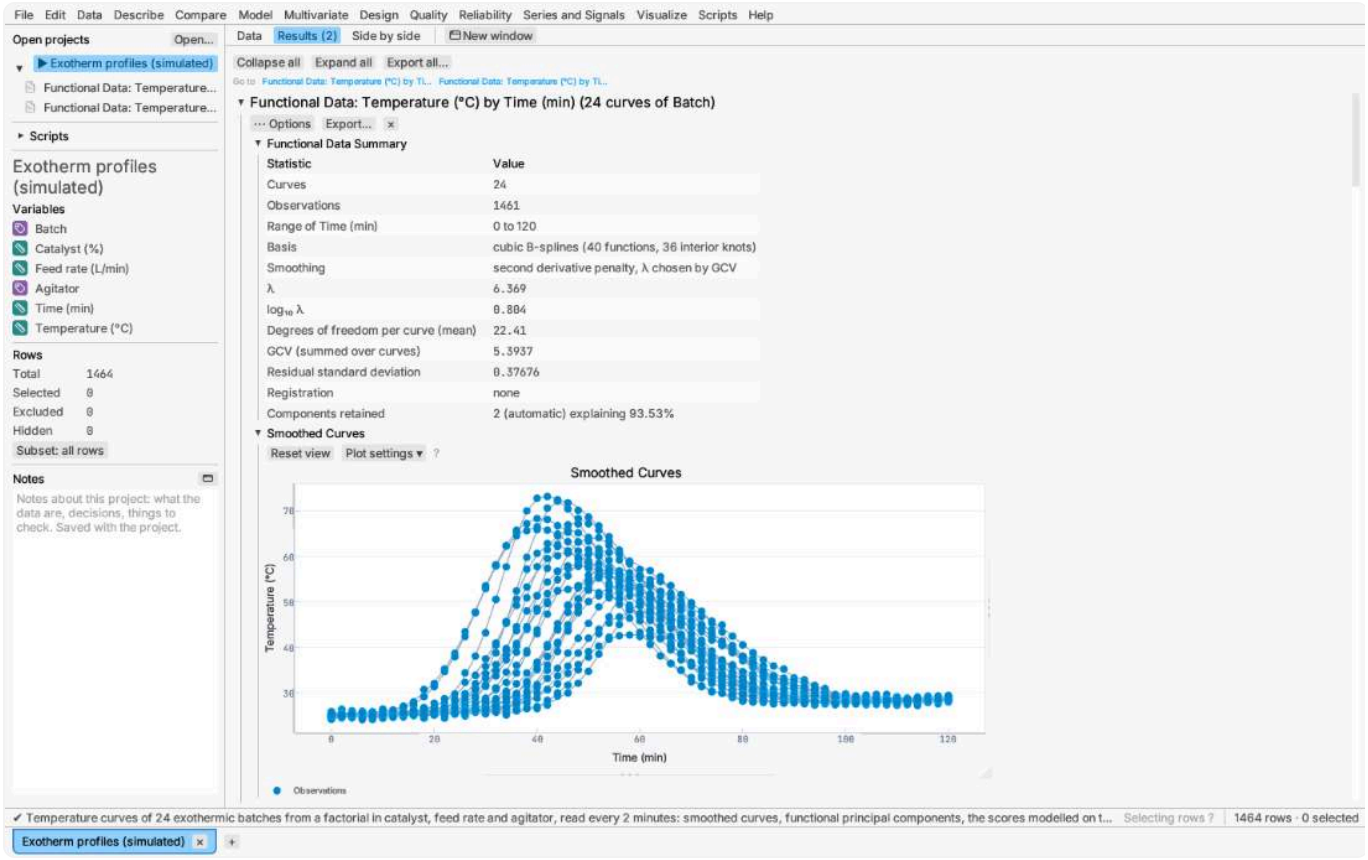
Report:

- *Summary* — curves, readings, basis, smoothing parameter and degrees of freedom, residual standard deviation, the components kept and the share of variation they explain.
- *Smoothed Curves* — each curve's smooth with its readings (curves are named in the key when there are ten or fewer). Select readings to select their rows everywhere.
- *Smoothing Parameter (GCV)* — $\log_{10}$ GCV against $\log_{10}$ λ with the chosen value marked.
- *Landmarks*, *Registered Curves* and *Warping Functions* (when registered) — each curve's landmark times and the mean they were moved to, the aligned curves, and each curve's warp of registered time to its own time (the diagonal is no warping).
- *Mean Curve* — the mean with a band of ±2 standard deviations, over the curves.
- *Functional Principal Components* — eigenvalues with the percent and cumulative percent of variation, the kept components marked.
- *Eigenfunctions*, each numbered at its largest value, and one *FPC Effect* plot per kept component: the mean curve and the mean plus and minus two standard deviations of that component's score (marked + and −), which shows what a high or low score means for the curve's shape.

- *Component Scores* — the curves on the first two components (coloured by the first categorical factor), and a table of every curve's scores.
- *Reconstruction* — each curve and its approximation from the kept components; *Fit by Curve* — readings, degrees of freedom, residual standard deviation and the reconstruction error of each curve.
- With factors: *Score Model* — Wilks' Λ test of each factor over the component scores, with the p-value for each component on its own; *Score Fits* (R^2 and RMSE of each component's model) and the *Effect Pareto*.
- *Curve Explorer* (with factors) — the predicted curve with its confidence limits: drag a factor's line to change its setting and watch the whole curve move, or drag the time line to read the prediction at any time. The *3-D Response Surface* shows the predicted value over time and one factor.

The options menu turns the Curve Explorer and 3-D profiler on or off and saves each curve's component scores, the smoothed value at each reading and (when registered) the registered time of each reading as new columns; every row of a curve gets its curve's score.

Try it: Exotherm profiles — 24 batch temperature profiles from a factorial in catalyst, feed rate and agitator. The first report models the component scores on the three factors (more catalyst makes the exotherm come earlier and higher); the second lines the peaks up by a landmark and continuous registration.



Text Analysis

The **Text** items at the foot of the **Multivariate** menu analyse a column of free text: survey comments, product reviews, incident notes. Each non-empty cell is a *document*. **Text exploration** counts words and phrases, draws a word cloud and finds the main directions of variation in the vocabulary; **Topic model** finds topics, each a mix of words, and describes every document as a mix of topics; **Sentiment scores** rates each document from negative to positive.

Excluded rows are left out, and empty cells are counted but not analysed (the report's summary says how many). Selecting documents anywhere, in a plot, the word cloud or the data grid, selects the same rows in every view.

Preparing the text

Text exploration and the topic model share the same preparation choices, under the **Text column**:

- **Leave out common words** (*the*, *and*, *of*, …) — on by default; a list of common English function words.
- **Combine word forms** — reduces each word to its Porter stem, so *price*, *prices* and *priced* count as one term. The tables then show the most common form found in the text, not the stem.
- **Keep terms found in at least N documents** — rarer words are dropped (default 2).

Under **More options**: further words to leave out (comma-separated), whether to lower-case every word (on by default), whether to keep numbers as terms, the shortest term kept (2 characters by default), the most terms kept (0 for all) and the random seed.

Words are split at the word boundaries of the Unicode text-segmentation standard, so contractions such as *don't* stay whole, and phrases never cross a sentence or comma.

Text exploration

Multivariate ▸ Text exploration… gives:

- **Summary** — documents, words, distinct terms, and the mean number of words per document;
- **Term Frequencies** — the most frequent terms with their counts, the number and percentage of documents containing each, and their total weight (when word forms are combined, the stem is shown beside the term);
- **Word Frequencies** — a bar chart of the most frequent terms. Each bar is made of the documents that contain the term, so clicking or box-selecting a bar selects those rows;
- **Word Cloud** — the most frequent terms with type sized by count (the area of the type follows the count; the most frequent tenth is set in bold). Hover a word for its count; click it to select the documents containing it (shift or ⌘/Ctrl adds them). Words of documents already selected are underlined. Right-click to save the cloud as PNG, SVG, PDF or EPS;
- **Phrases** — runs of 2 to 3 words (the length is an option) seen at least twice, with their counts and documents;
- **Semantic Dimensions** — latent semantic analysis: the singular values of the weighted document–term matrix and the share of its sum of squares each dimension carries;
- **Terms by Dimension** — the terms loading most strongly, positive and negative, on each dimension, which say what the dimension is about;
- **Document Map** — the documents on the first two dimensions, coloured and shaped by cluster when clusters are asked for, with the leading terms marked;
- **Document Clusters** — k-means clusters of the documents' directions in the semantic dimensions, with each cluster's size and its most typical terms.

The **Term weighting** choice sets the matrix the semantic dimensions are computed from: counts, presence (binary), log counts, TF-IDF (the default: count × ln(documents ÷ documents containing the term), which plays down terms found everywhere) or log TF-IDF. Each document's row is scaled to unit length, so long and short documents count alike. Set **Semantic dimensions** or **Document clusters** to 0 to leave those sections out.

Below the report, **Save semantic dimension 1, 2, …** and **Save document cluster** add the scores and cluster numbers as new columns.

Topic model

Multivariate ▸ Topic model… fits latent Dirichlet allocation: each topic is a probability distribution over terms, and each document a mix of topics. Choose the number of **Topics**

(default 5). Under **More options** you can set the document–topic prior α (default 50 ÷ topics; smaller values let each document lean on fewer topics), the topic–term prior β (default 0.1; smaller values give topics fewer, more distinct terms), the number of sampler sweeps (1000) and how many to discard as burn-in (200).

The model is fitted by collapsed Gibbs sampling, so results depend on the random seed; the same seed always gives the same report. The report has:

- **Summary** — documents, terms, topics, the priors and the perplexity (lower means the model predicts the words better);
- **Top Terms by Topic** — each topic's most probable terms with their probabilities, and its share of all words. Read the terms to name the topic;
- **Documents by Most Likely Topic** — a bar for each topic; its bar is made of the documents whose largest share is that topic, so clicking it selects them;
- **Sampler Log-Likelihood** — the log-likelihood at each sweep. It should level off after the burn-in; if it is still rising, run more sweeps;
- **Most Representative Documents** — the three documents with the largest share of each topic, with their text.

Save most likely topic and **Save proportion of topic 1, 2, …** add columns for further analysis, for example a contingency table of topic against product.

Sentiment scores

Multivariate ▸ Sentiment scores… rates each document from a lexicon of about 7,500 English words, each rated from −4 (very negative) to +4 (very positive) by human raters (the VADER lexicon of Hutto & Gilbert, 2014, used under its MIT licence). A word's sign is reversed and its valence weakened when a negation (*not*, *never*, *isn't*, …) comes within the three words before it. Intensifiers (*very*), capitals, exclamation marks and emoticons are not taken into account. The choices are:

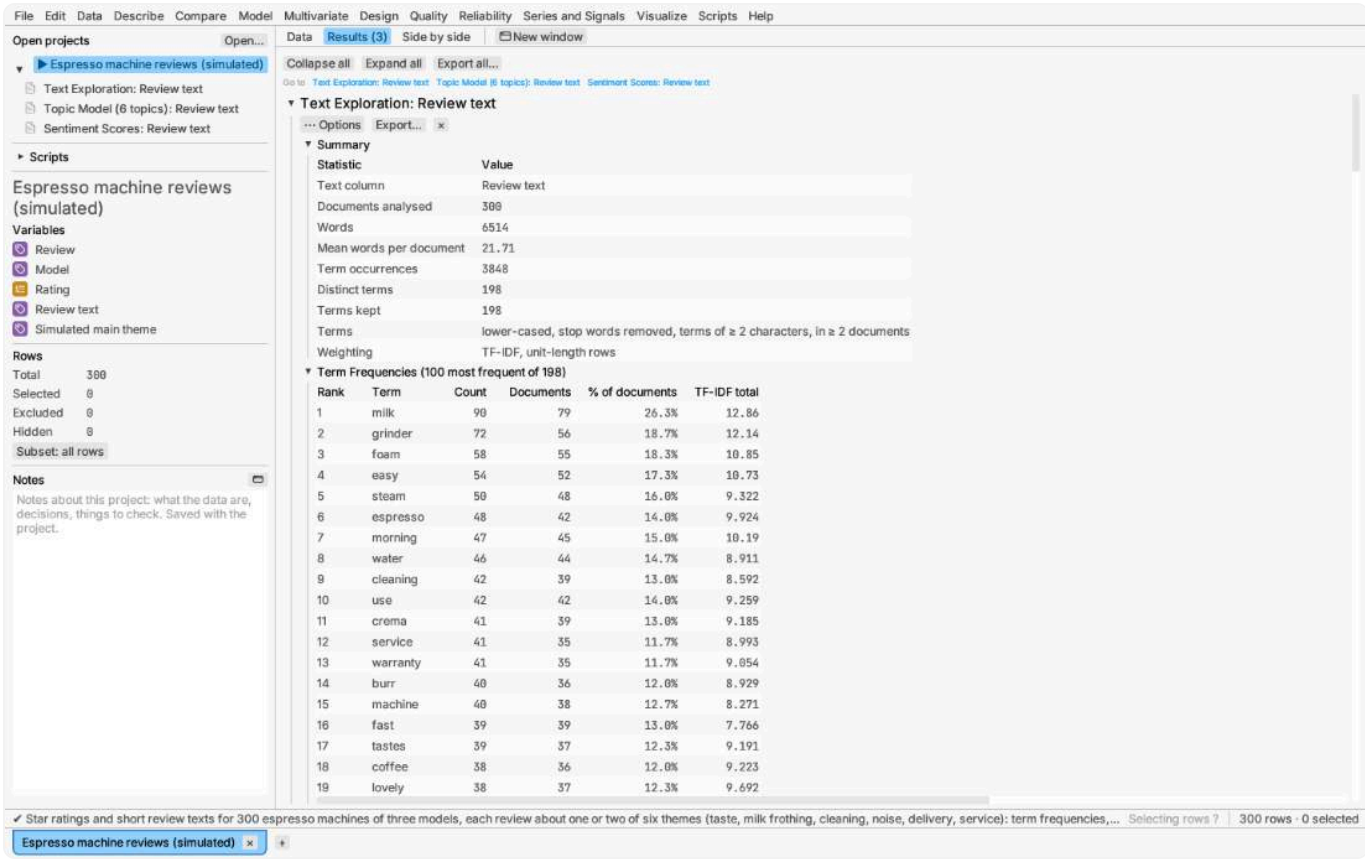
- **Reverse words after a negation within N words** — on by default, N = 3;
- **Neutral band** — documents scoring within ±0.05 are neutral;
- **Words of your own** — entries such as `crema = 2, limescale = -1.5` that add words to the lexicon or replace its valence (−4 to 4).

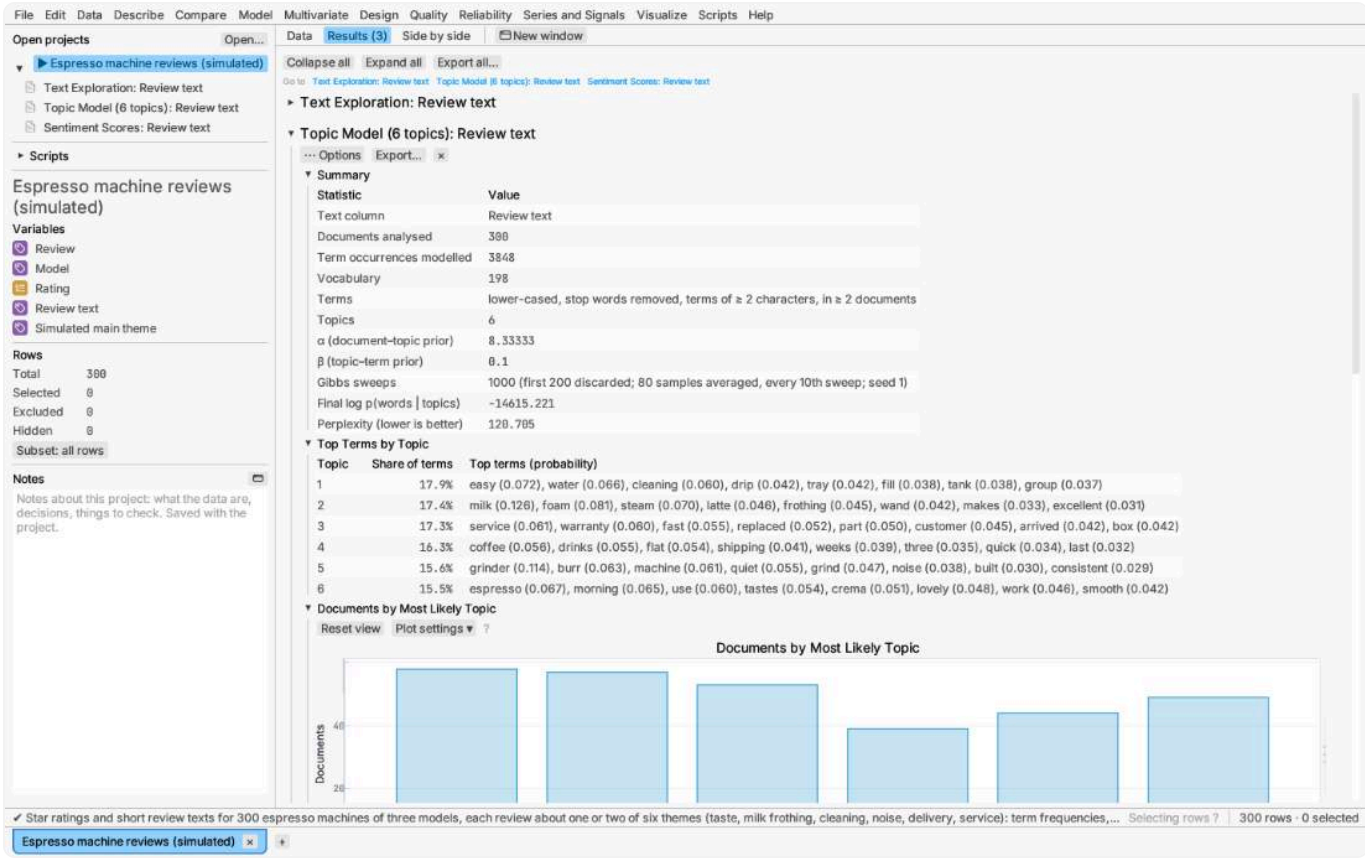
Each document's **score** is the sum of its valences s mapped to −1…1 by $s \div \sqrt{s^2 + 15}$. The report has:

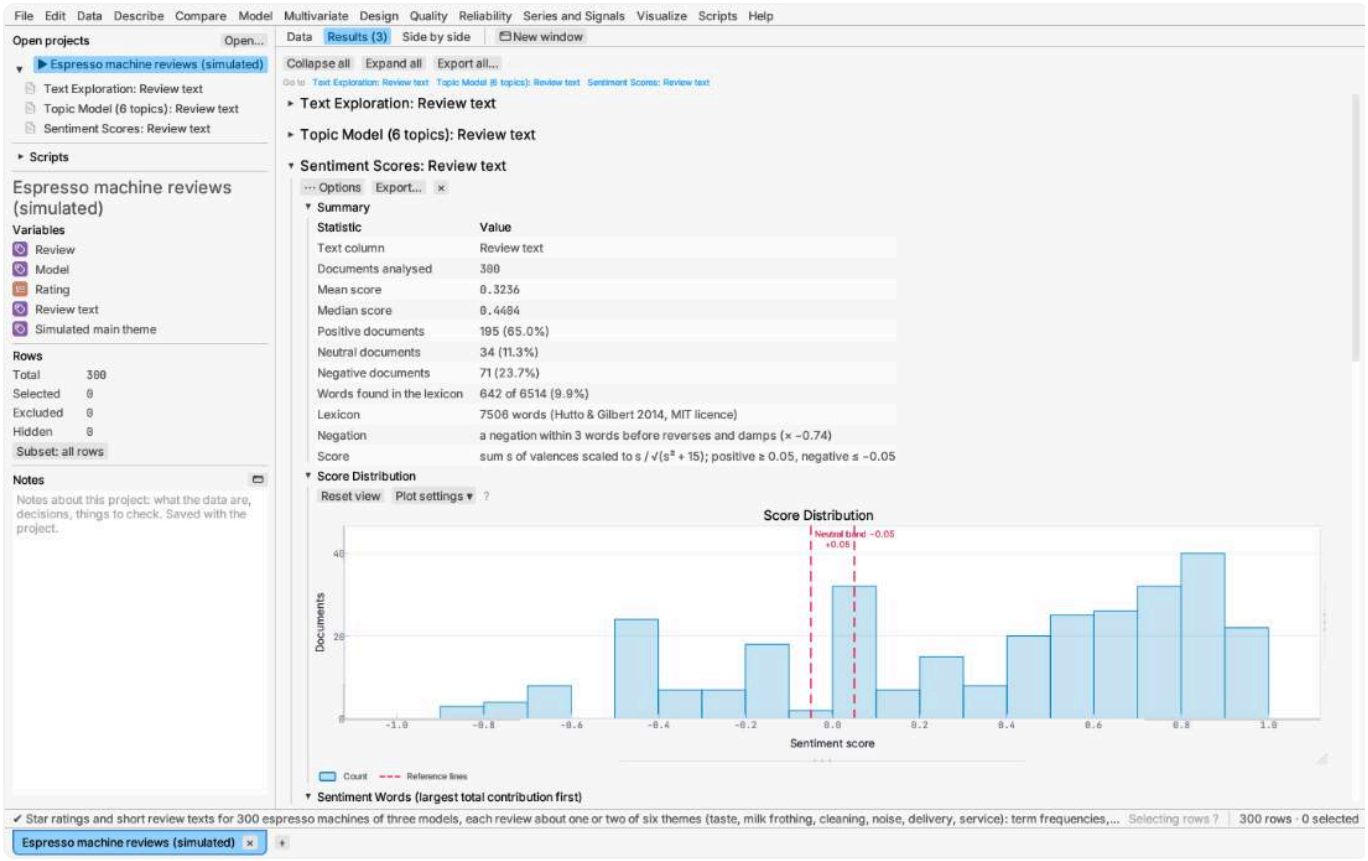
- **Summary** — documents scored, how many are positive, neutral and negative, and the mean and median score;
- **Score Distribution** — a histogram of the scores with the neutral band marked; selecting bars selects the documents;
- **Sentiment Words** — the lexicon words that contributed most, with their valence, occurrences and total contribution, and how often they were negated;
- **Most Positive Documents** and **Most Negative Documents** — the extremes, with their text.

Save sentiment score, **Save sum of valences**, **Save sentiment category** (positive, neutral or negative), **Save positive word count** and **Save negative word count** add columns. A lexicon score is a rough guide: it misses sarcasm and domain words, so read the extreme documents before relying on it, and add your own words where the lexicon is silent.

Try it: Espresso machine reviews — 300 simulated reviews of three machine models with star ratings. The text exploration finds six clusters that match the hidden main theme of each review (kept in the column *Simulated main theme*), the topic model recovers the same six themes, and the sentiment score rises with the star rating.




File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Espresso machine reviews (simulated)
Text Exploration: Review text
Topic Model (6 topics): Review text
Sentiment Scores: Review text
Scripts
Espresso machine reviews (simulated)
Variables
Review
Model
Rating
Review text
Simulated main theme
Rows
Total 300
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
Text Exploration: Review text
Topic Model (6 topics): Review text
Options
Export...
Summary
Statistic
Value
Text column
Review text
Documents analysed
300
Term occurrences modelled
3848
Vocabulary
198
Terms
lower-cased, stop words removed, terms of ≥ 2 characters, in ≥ 2 documents
Topics
6
α (document–topic prior)
8.33333
β (topic–term prior)
0.1
Gibbs sweeps
1000 (first 200 discarded; 80 samples averaged, every 10th sweep; seed 1)
Final log p(words | topics)
-14615.221
Perplexity (lower is better)
120.705
Top Terms by Topic
Topic
Share of terms
Top terms (probability)
1 17.9% easy (0.072), water (0.066), cleaning (0.060), drip (0.042), tray (0.042), fill (0.038), tank (0.038), group (0.037)
2 17.4% milk (0.126), foam (0.081), steam (0.070), latte (0.046), frothing (0.045), wand (0.042), makes (0.033), excellent (0.031)
3 17.3% service (0.061), warranty (0.060), fast (0.055), replaced (0.052), part (0.050), customer (0.045), arrived (0.042), box (0.042)
4 16.3% coffee (0.056), drinks (0.055), flat (0.054), shipping (0.041), weeks (0.039), three (0.035), quick (0.034), last (0.032)
5 15.6% grinder (0.114), burr (0.063), machine (0.061), quiet (0.055), grind (0.047), noise (0.038), built (0.030), consistent (0.029)
6 15.5% espresso (0.067), morning (0.065), use (0.060), tastes (0.054), crema (0.051), lovely (0.048), work (0.046), smooth (0.042)
Documents by Most Likely Topic
Reset view
Plot settings
Documents by Most Likely Topic
Documents
40
20
✓ Star ratings and short review texts for 300 espresso machines of three models, each review about one or two of six themes (taste, milk frothing, cleaning, noise, delivery, service): term frequencies,...
Selecting rows ?
300 rows · 0 selected
Espresso machine reviews (simulated)


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Espresso machine reviews (simulated)
Text Exploration: Review text
Topic Model (6 topics): Review text
Sentiment Scores: Review text
Scripts
Espresso machine reviews (simulated)
Variables
Review
Model
Rating
Review text
Simulated main theme
Rows
Total 300
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
Text Exploration: Review text
Topic Model (6 topics): Review text
Sentiment Scores: Review text
Options
Export...
Summary
Statistic
Value
Text column
Review text
Documents analysed
300
Mean score
0.3236
Median score
0.4484
Positive documents
195 (65.0%)
Neutral documents
34 (11.3%)
Negative documents
71 (23.7%)
Words found in the lexicon
642 of 6514 (9.9%)
Lexicon
7506 words (Hutto & Gilbert 2014, MIT licence)
Negation
a negation within 3 words before reverses and damps (× −0.74)
Score
sum s of valences scaled to s / √(s² + 15); positive ≥ 0.05, negative ≤ −0.05
Score Distribution
Reset view
Plot settings
Score Distribution
Neutral band −0.05 +0.05
Documents
40
20
0
-1.0
-0.8
-0.6
-0.4
-0.2
0.0
0.2
0.4
0.6
0.8
1.0
Sentiment score
Count
Reference lines
Sentiment Words (largest total contribution first)
✓ Star ratings and short review texts for 300 espresso machines of three models, each review about one or two of six themes (taste, milk frothing, cleaning, noise, delivery, service): term frequencies,...
Selecting rows ?
300 rows · 0 selected
Espresso machine reviews (simulated)

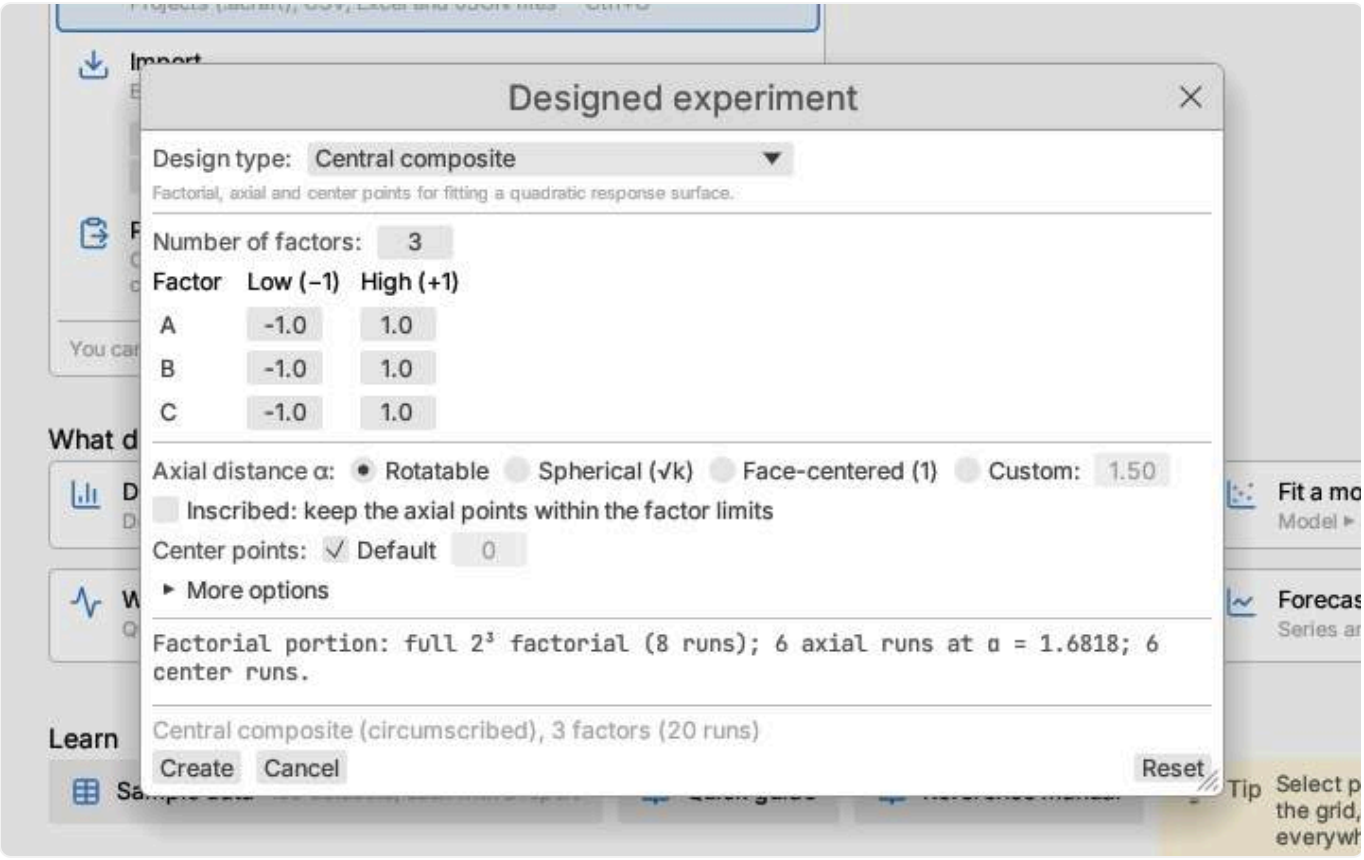
Design

The Design menu creates experimental designs. It works without an open dataset, because each design opens as a new data table ready for you to add the responses.

Submenu	Designs
Factorial	2-level and 3-level full factorials; full factorials with mixed levels; fractional factorials, with their resolution shown
Screening	Plackett–Burman; definitive screening designs (with extra runs and center runs); screening with categorical factors
Response surface	central composite (rotatable, spherical, face-centered or a custom α) and Box–Behnken
Optimal	D-optimal and I-optimal designs built by coordinate exchange, optionally within constraints; split-plot designs for hard-to-change factors
Space filling	Latin hypercube (maximin), maximin distance, minimax distance and uniform designs, for computer experiments
Mixture	simplex lattice, simplex centroid and extreme vertices
Robust design	orthogonal arrays with an inner and outer array
Measurement	gauge study worksheet
This table (with a table open)	signal-to-noise analysis; declare table as design; augment design; simulate responses; evaluate design; factorial analysis; propose next runs (sequential optimization)

Every design dialog:

- shows a live preview of the runs as you change the settings;
- can add center points (a rotatable central composite design starts with Box & Hunter's uniform-precision number for its factorial portion);
- randomizes the run order, with a seed so the order can be reproduced;
- opens the design as a new table with **Create**.



After running the experiment, type the responses into the new table and analyse it with Model ▸ Linear model, Response surface, Mixture model or Several responses.

Full factorials with mixed levels

Design ▸ Full factorial (mixed levels)… crosses factors that each have their own number of levels, for example a 2 × 3 × 4 design of 24 runs. For each factor give its lowest and highest setting and the number of levels; the levels are spaced evenly between them. Factors with named categories (suppliers, materials) are entered as numbered levels 1, 2, … and the column can then be recoded or changed to nominal in the grid.

Definitive screening: extra runs and center runs

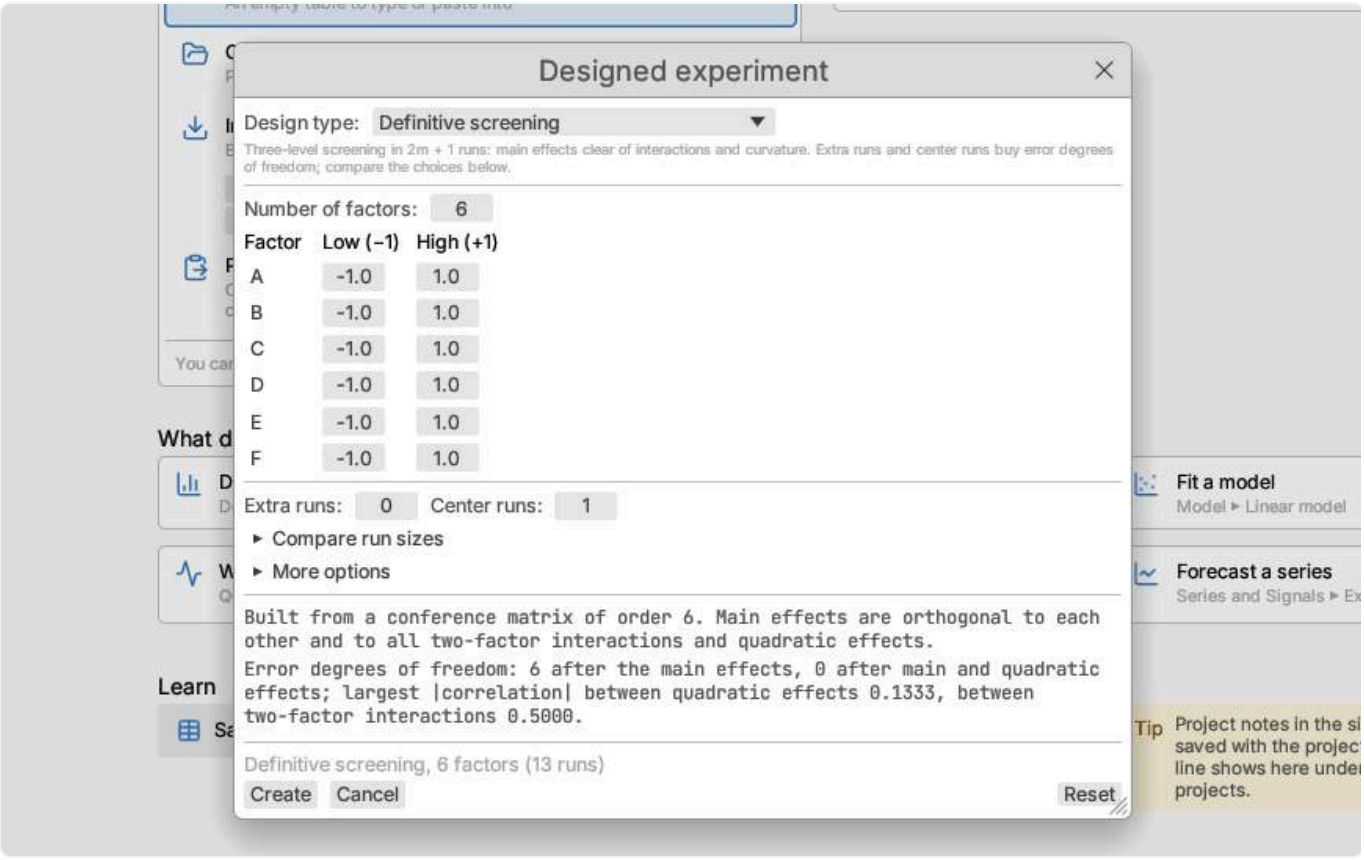
A definitive screening design for k factors has 2m + 1 runs, where m is the smallest even conference-matrix order of at least k (the fold-over of the matrix, plus one center run). Its main effects are clear of each other, of two-factor interactions and of quadratic effects, but

with few runs to spare the error estimate and the quadratic effects are weak. Two settings in the dialog buy precision:

- **Extra runs**: the design is built from a larger conference matrix and its extra columns are left out ("fake factors", Jones & Nachtsheim 2013), so the runs grow in pairs while the defining properties hold. Asking for 4 extra runs with 6 factors uses the 8-factor design: 17 runs instead of 13.
- **Center runs**: the number of center runs in all (default 1). Each one beyond the first adds a pure-error degree of freedom and sharpens the quadratic effects.

Compare run sizes lists nearby choices (the first few larger orders with one center run and with your number of center runs) side by side, your choice in bold:

Column	Meaning
Error df (main effects)	n − 1 − k: degrees of freedom left after fitting the main effects
Error df (main + quadratic)	n − 1 − 2k, after the quadratic effects too
D-efficiency (main effects, %)	100 · det(X^TX)^(1/p) / n for the main-effects model
Max |r| quadratic	the largest correlation between two quadratic columns: (n − 9)/(3(n − 3)) with one center run
Max |r| interactions	the largest correlation between two two-factor-interaction columns
Mean SE quadratic (σ)	the mean standard error of a quadratic effect in the main-effects-plus-quadratic model, in units of σ



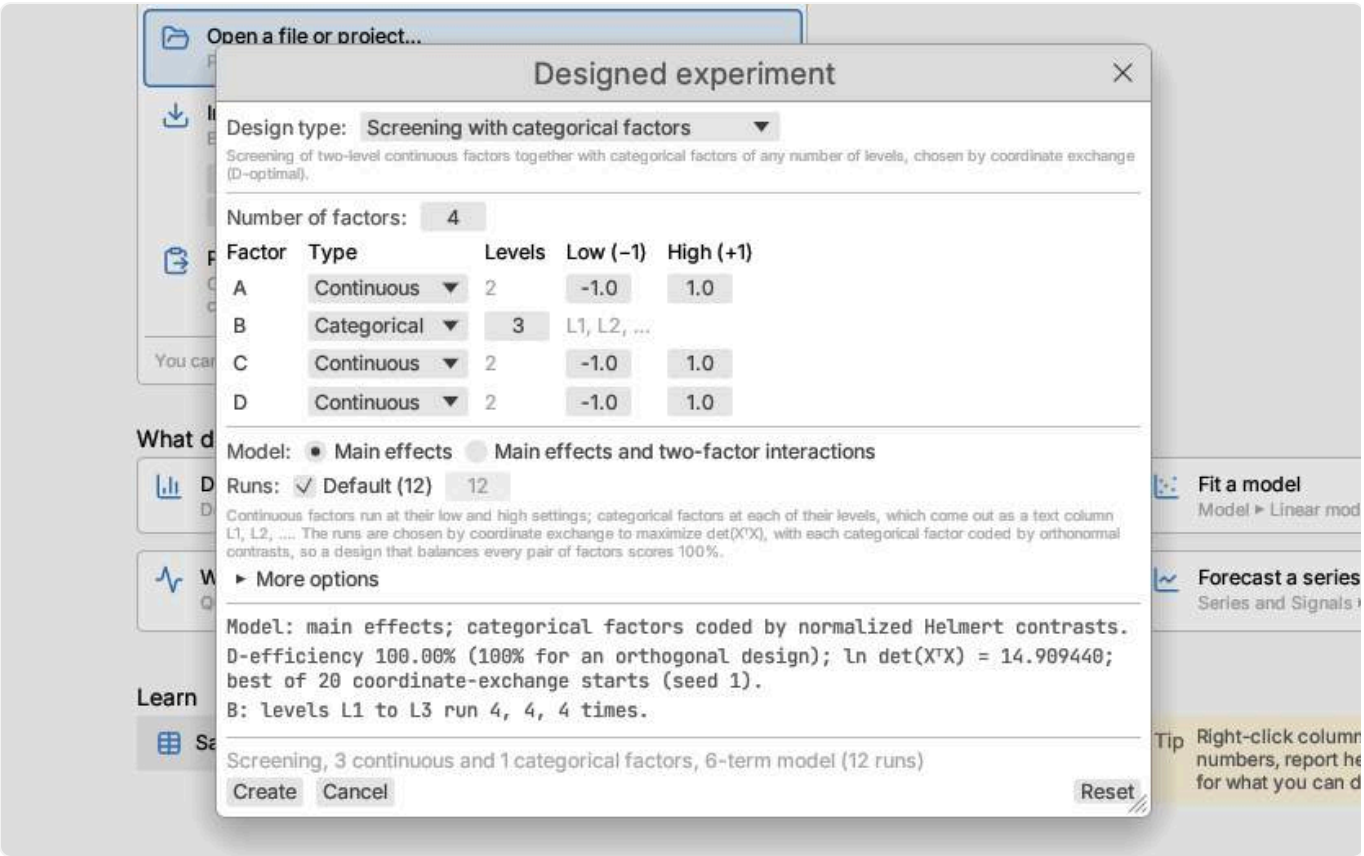
Script: `definitive_screening_design(#{ factors, extra_runs, center_runs })` and `compare_definitive_screening(#{ factors, extra_runs, center_runs })`.

Screening with categorical factors

Design ▸ Screening with categorical factors… screens two-level continuous factors together with categorical ones (suppliers, machines, materials) of 2 to 12 levels. Set each factor's **Type**; a categorical factor has a number of **Levels**, and its low and high settings do not apply.

The runs are chosen by coordinate exchange to maximize $\det(X^TX)$ for the main-effects model, or the model with every two-factor interaction too. Each categorical factor enters through orthonormal (normalized Helmert) contrasts, so a design that balances every pair of factors, an orthogonal array, scores 100% D-efficiency; the search reaches the known arrays (for example a four-level factor with four two-level factors in 8 runs). The default number of runs is the number of model terms plus 4, rounded up to a multiple of the levels' least common multiple when that costs at most half again, so that levels can appear

equally often; untick **Default** to choose. The notes give how often each level runs, the D-efficiency and ln det(X^TX).



In the new table a categorical factor is a nominal text column with levels L1, L2, … (rename them with Data ▸ Recode). Script: `screening_design(#{ factors: [#{ name, levels }, #{ name, low, high }], runs, model })`.

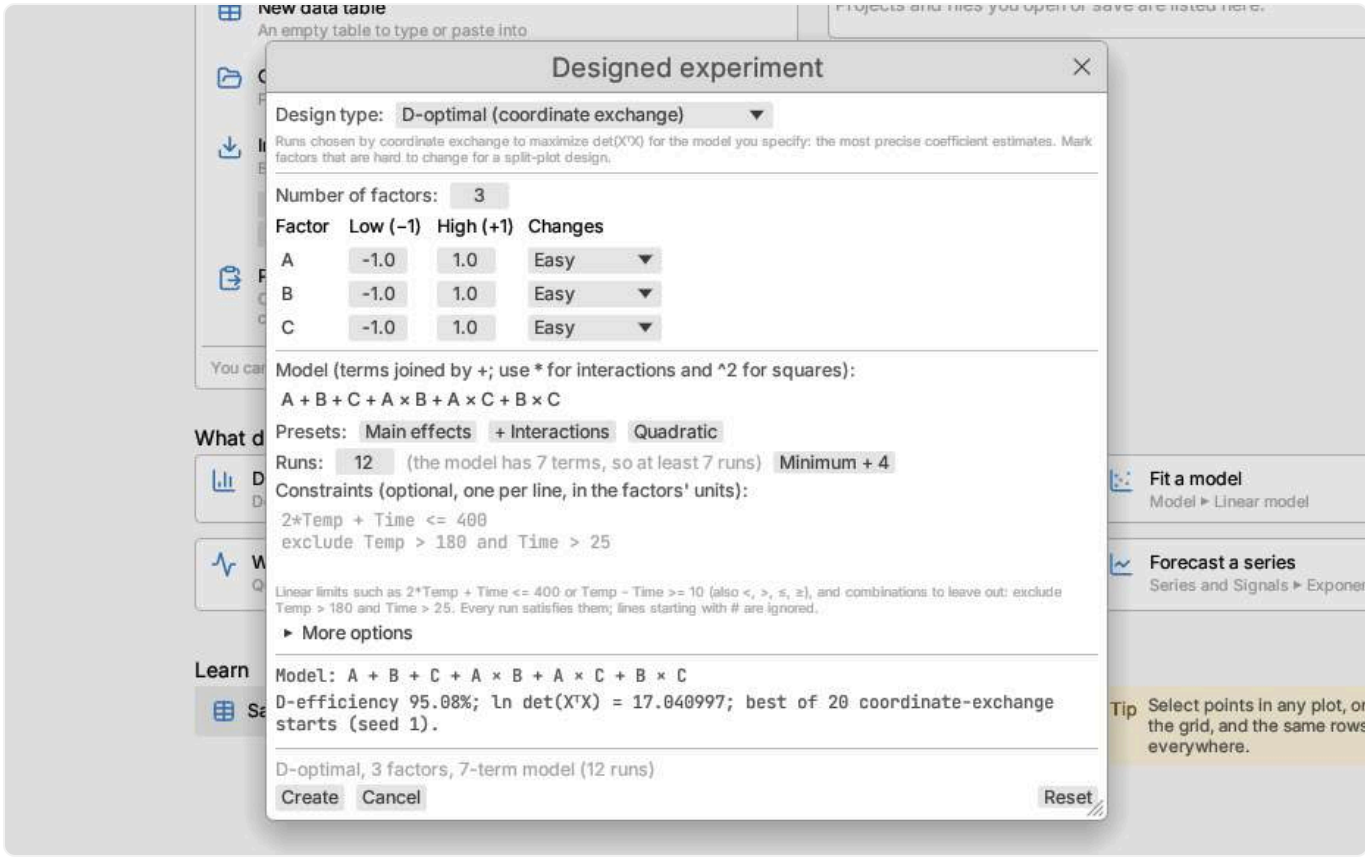
Constraints in optimal designs

Some combinations of settings cannot be run: an oven cannot be at its highest temperature for the longest time, or a pressure limit ties two factors together. The D-optimal and I-optimal dialogs have a **Constraints** box for these limits, one per line, written with the factor names in the factors' own units:

```
2*Temp + Time <= 400
Temp - Time >= 10
exclude Temp > 180 and Time > 25
```

- A **linear constraint** compares two sums of terms such as `Temp`, `2*Temp`, `Time/4` or a number, with `<=`, `>=`, `<`, `>`, `≤` or `≥`. Points on the boundary are allowed. An equality cannot be used (it would leave no room for an experiment).
- An **exclude** line leaves out a combination: every point meeting all its conditions (joined by `and`) is disallowed. Each condition compares one factor with a value; `>` and `<` leave the boundary value allowed, `>=`, `<=` and `=` exclude it.
- Lines starting with `#` are ignored. Factor names may contain spaces (write `"Reaction time"` in quotes if a name could be misread).

The search places every run inside the region: each setting is chosen from the grid levels, the ends of the range the constraints leave, and (for factors with squared terms) the middle of that range, and a run on a slanted limit can slide along it. The constraints are listed in the design's notes. A mistake is reported in the preview with its line number; a region with no room for runs says so. For an I-optimal design the average prediction variance is over the constrained region, estimated from 20 000 evenly spread points inside it (the notes say so); over the whole cube it is exact.



Split-plot designs for hard-to-change factors

Some factors are slow or costly to reset: a furnace takes hours to reach a new temperature, a batch of raw material is used up over several runs. Running every run in a fully random order would reset them every time. Instead, the runs are grouped into **whole plots**: the hard-to-change factors are set once per whole plot, and the easy factors vary from run to run within it. This is a split-plot design. With a third, very hard factor, the whole plots are split into **subplots**: very hard factors are fixed within whole plots and hard ones within subplots (a split-split-plot design).

Design ▸ Optimal ▸ **Split-plot (hard-to-change factors)…** opens the D-optimal dialog with the first factor marked hard. In the factor table, the **Changes** column sets each factor to Easy, Hard or Very hard; any factor that is not easy makes the design a split-plot design. Then give:

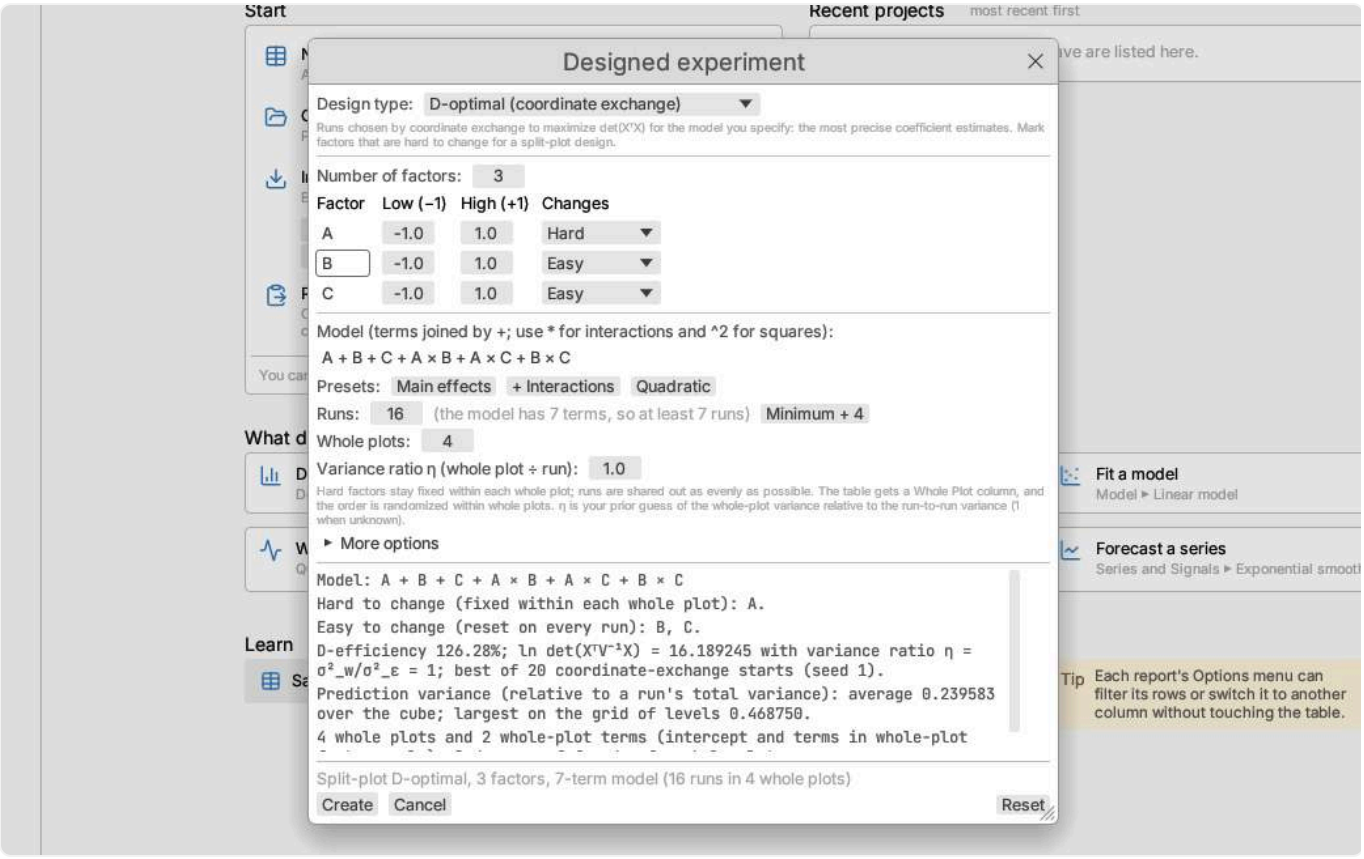
- the **model** and number of **runs**, as for any D-optimal design;
- the number of **whole plots** (and of **subplots** when there are very hard and hard factors). Runs are shared out as evenly as possible;
- the **variance ratio η**, your guess of the whole-plot variance relative to the run-to-run variance. 1 is a good default when you have no prior information; the design changes little over a wide range.

The search (coordinate exchange without a candidate list; Jones & Goos 2007) changes a hard factor's setting for a whole plot at a time and maximizes $\det(X^T V^{-1} X)$, which accounts for runs in the same whole plot being correlated (Goos & Vandebroek 2003). The notes give the D-efficiency (relative to a completely randomized orthogonal design with the same total variance per run; it can exceed 100% because comparisons within a whole plot are free of the whole-plot variance), the average and largest prediction variance, and the **degrees of freedom for whole-plot error**: whole plots minus the terms that involve only hard factors (with the intercept). With none, the effects of hard factors cannot be tested, so add whole plots.

The table has a nominal **Whole Plot** column (and **Subplot**), numbered in run order. The run order is randomized only within the restrictions: whole plots in random order, then the runs inside each one. Run them in that order.

The new table opens with the design declared (Design ▸ This table ▸ Declare table as design shows the plots, which factors are fixed within them and the analysis to use) and with a project script, *Fit split-plot model*. Once the responses are entered, choose Model ▸ **Mixed model**: it is filled in with the design's model and a random intercept for the whole plots (and for the subplots within them), fitted by REML. Its F tests use Satterthwaite degrees of

freedom, so a hard factor is tested against the variation between whole plots rather than the smaller run-to-run variation, which an ordinary linear model would wrongly use. The sample *Furnace coatings (simulated) — split-plot design* shows the whole cycle.



Space-filling designs for computer experiments

A computer model (a simulation or a formula) gives the same answer every time it is run with the same inputs, so replicates and center points tell you nothing, and designs that put their runs at the corners leave most of the region unexplored. Space-filling designs spread the runs evenly over the whole range of every factor, ready for fitting a flexible surrogate model. Design ▸ Space filling offers four:

Design	What it optimizes
Latin hypercube (maximin)	Each factor's range is cut into as many equal slices as there are runs and each slice is used once; among such designs, the runs are pushed apart by minimizing the φ_{50} criterion (Morris & Mitchell 1995)

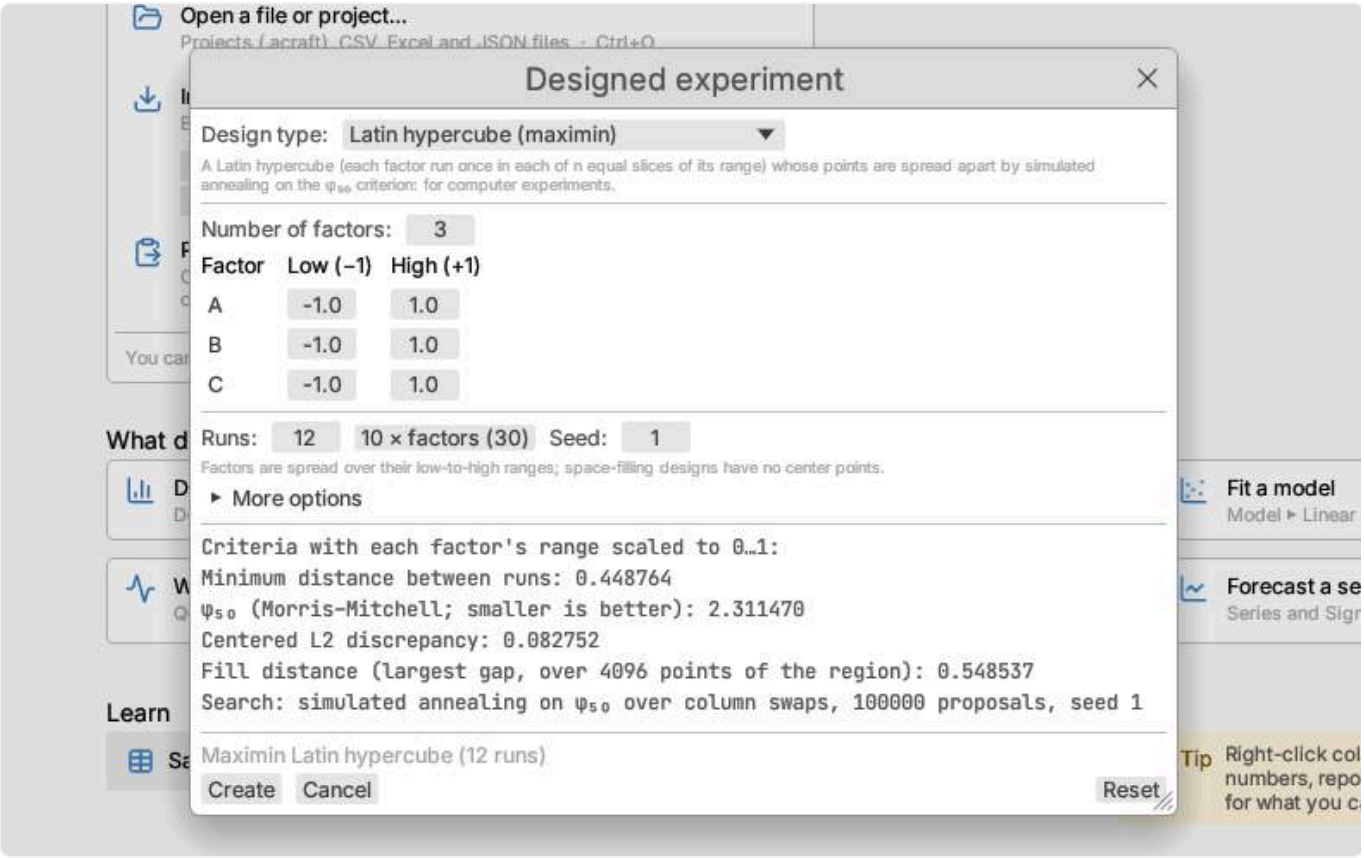
Design	What it optimizes
Maximin distance	The smallest distance between two runs is made as large as possible, with the runs free to go anywhere (often on the edges of the region)
Minimax distance	The largest gap (the distance from any point of the region to its nearest run, the *fill distance*) is made as small as possible, by clustering many points of the region (Mak & Joseph 2018)
Uniform (centered L2 discrepancy)	Like a Latin hypercube, one run per slice of each factor, with the runs spread as evenly as possible by the centered L2 discrepancy (Fang et al. 2000)

Set the number of factors and their low and high settings (the design spreads the runs between them), the number of **Runs** (up to 1000; the **10 × factors** button is a common starting size) and the **Seed**. Each design is found by a randomized search, so the same seed always gives the same design and another seed may give a slightly different one. There are no center points.

The preview lists four measures of each design, computed with every factor's range scaled to 0…1, so you can compare the designs:

- **Minimum distance between runs** — larger is better spread.
- **φ_{50}** — a smooth version of one over the minimum distance that also counts close pairs; smaller is better.
- **Centered L2 discrepancy** — how far the runs are from an even spread over every corner-anchored box; smaller is better.
- **Fill distance** — the largest gap, measured over a fine grid and a quasi-random set of points (so slightly optimistic); smaller is better.

The sample *Borehole water flow (Worley 1987)* is a computer experiment: a 40-run maximin Latin hypercube over the eight inputs of a groundwater model, with the flow computed from its formula. It opens with a surrogate linear model and the Response Explorer.

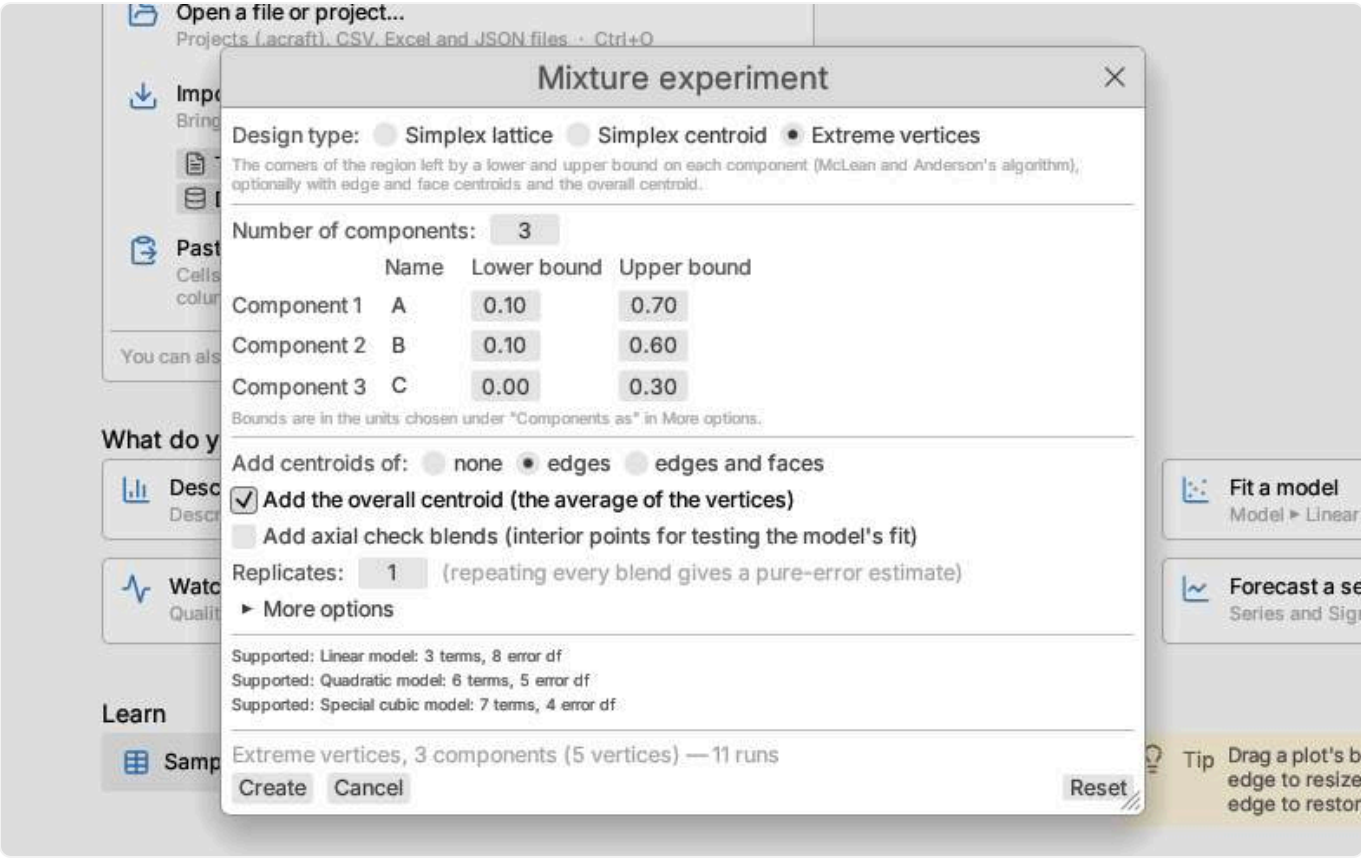


Extreme-vertices mixture designs

When each component of a mixture has its own lower and upper limit, the blends that satisfy them form a smaller region inside the simplex. Design ▸ Extreme vertices… finds the corners of that region (McLean & Anderson's algorithm), after tightening any bound that the others make unreachable. Enter each component's name, lower bound and upper bound in the units chosen under **Components as** (proportions, percentages or amounts). The options add:

- the centroids of the region's edges, or of its edges and two-dimensional faces;
- the overall centroid (the average of the vertices);
- axial check blends half-way between the overall centroid and each vertex.

The preview lists the Scheffé models the blends support. Fit the results with Model ▸ Mixture model.

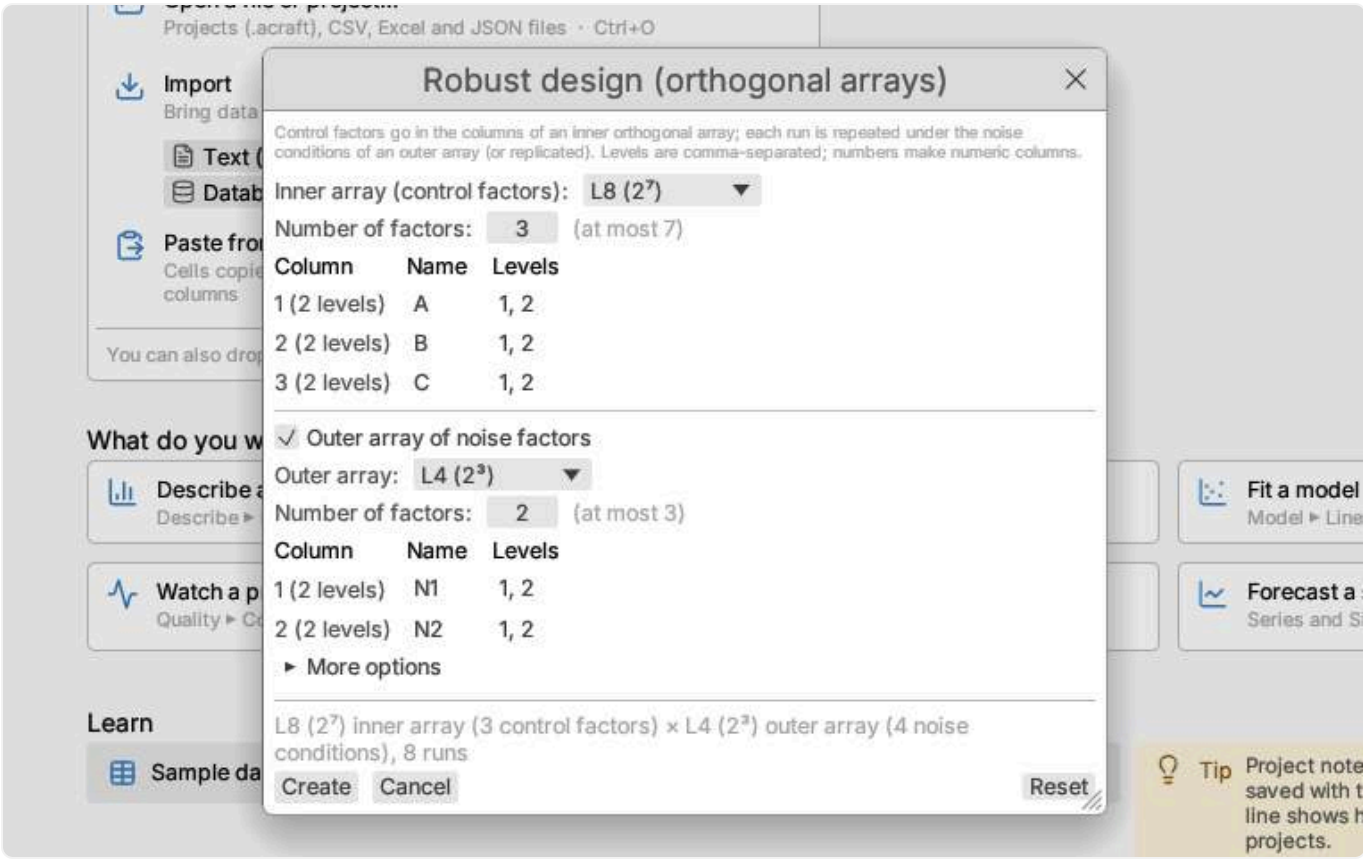


Robust design with orthogonal arrays

Design ▸ Robust design (orthogonal arrays)… lays out a robust-design experiment in the style described by Phadke (1989). Control factors (the settings you can choose) go in an **inner array**, one of the standard orthogonal arrays:

Array	Runs	Columns
L4	4	3 two-level
L8	8	7 two-level
L9	9	4 three-level
L12	12	11 two-level
L16	16	15 two-level
L18	18	1 two-level and 7 three-level
L27	27	13 three-level

Noise factors (conditions you cannot control in use but can set in the test) go in an optional **outer array**. Each run of the inner array is tested at every run of the outer array, and each outer run becomes one response column, named after its noise settings, such as `Shrinkage (Humidity=low, Age=new)`. Without an outer array, choose how many times each inner run is repeated. Factors take the array's columns in order; level names whose values are all numbers make numeric columns. The run order can be randomized with a seed.



Signal-to-noise analysis

With the results entered, Design ▸ Signal-to-noise analysis… summarizes each run's responses as a signal-to-noise ratio η (in decibels, larger is better whatever the goal):

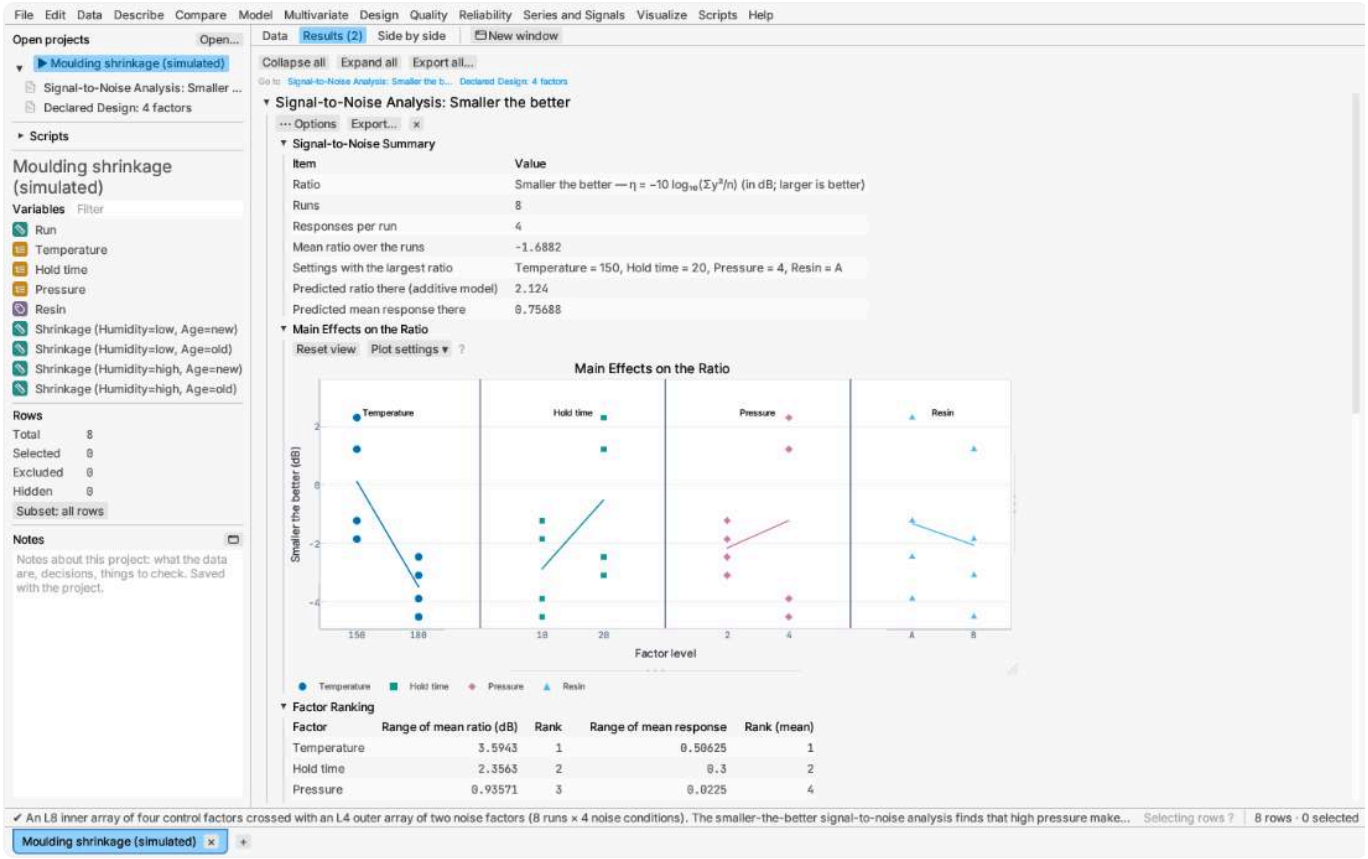
Goal	Ratio
Smaller the better	$\eta = -10 \log_{10}(\text{mean of } y^2)$
Larger the better	$\eta = -10 \log_{10}(\text{mean of } 1/y^2)$
Nominal the best	$\eta = 10 \log_{10}(\bar{y}^2 / s^2)$
Nominal the best (variance only)	$\eta = -10 \log_{10} s^2$

Choose the factor columns and the response columns (one per outer-array run or repeat). The report shows:

- **Summary:** the ratio used and the settings with the largest ratio, with the predicted η and mean response there from the additive model (overall mean plus the gain of each chosen level).
- **Main Effects on the Ratio:** the mean η at each level of each factor.
- **Factor Ranking:** each factor's range of level means (delta) and rank, for the ratio and for the mean response.
- **Response Table:** the number of runs, mean η and mean response at each level of each factor.
- **Main Effects on the Mean**, to pick a factor that moves the mean without changing η (an adjustment factor).
- **Ratio by Run:** each run's settings, number of responses, mean, standard deviation and η.

The ratio can be changed in place from the report's settings.

Try it: Moulding shrinkage (smaller the better).



Declaring a table as a design

Design ▸ Declare table as design… treats columns of any open table as the factors and responses of an experiment, for a design built elsewhere or an existing set of runs. For each factor you can give the settings coded −1 and +1; otherwise the column's smallest and largest values are used. A categorical factor (text, or numbers marked nominal or ordinal) must have two levels: the first in sorted order is coded −1 and the second +1. Responses must be numeric. The report shows:

- **Factors and Responses:** each factor's −1 and +1 settings, its number of distinct levels and how many runs fall outside the range; each response with its number of values;
- **Runs:** the number of runs and of distinct settings, settings run more than once and the pure-error degrees of freedom they give, and center runs;
- **Correlations of Coded Factors**, with a note saying whether the design is orthogonal for the main effects;
- **Models the Runs Support:** main effects, main effects with two-factor interactions, and the full quadratic, each with its parameters, whether it can be estimated, error degrees of freedom and D-efficiency.

For a split-plot experiment, also choose the **Whole plots** column (and **Subplots** for a split-split-plot design). The report then adds **Restricted Randomization** — the number and size of the plots and the factors held fixed within them — and a note on the analysis, with the whole-plot error degrees of freedom when the design's model is recorded (as it is for a design made by Design ▸ Split-plot). Model ▸ Mixed model on the table is then filled in with that analysis.

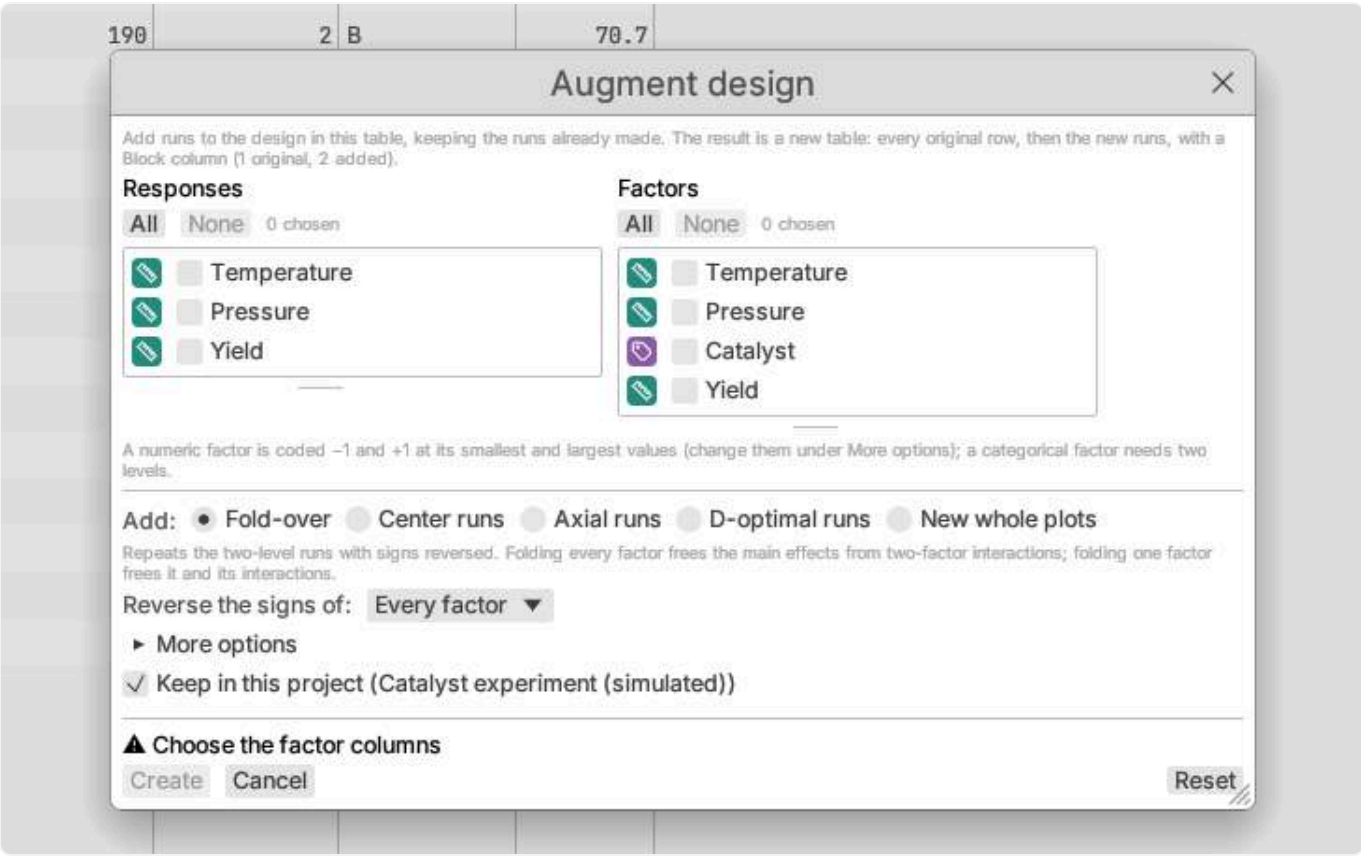
Try it: Moulding shrinkage (temperature, hold time and pressure); Furnace coatings for a split-plot design.

Augmenting a design

Design ▸ Augment design… (with a table open) adds runs to the experiment in the active table and keeps the runs already made. Choose the factor columns (a numeric factor is coded −1 and +1 at its smallest and largest values unless you give its settings under More options; a categorical factor needs two levels) and, optionally, the response columns, then what to add:

Add	What it does
Fold-over	Repeats the two-level runs with signs reversed. Reversing **every factor** frees all main effects from two-factor interactions (a resolution III fraction becomes resolution IV). Reversing **one factor** frees that factor and its two-factor interactions. Center runs are not repeated; a two-level text factor swaps its levels.
Center runs	Runs at the middle of every factor's range, for pure error and a curvature check.
Axial runs	Runs at ±α on each factor axis (α = 1, face-centered, by default) plus center runs: a two-level factorial becomes a central composite design. The notes give the rotatable α for the factorial runs.
D-optimal runs	The number of runs you ask for, chosen by coordinate exchange to maximize $\det(X^TX)$ of all runs for the model you give (for example a larger model with interactions or squares), with the existing runs fixed. Constraints can be given as for optimal designs.
New whole plots	For a split-plot design: the number of whole plots you ask for, each with the same number of runs. Choose the **Whole plot column**; the factors constant within every existing whole plot are taken as hard to change and stay fixed within each new whole plot, while the others vary freely. The settings maximize $\det(X^TV^{-1}X)$ of all runs, $V = I + \eta\, ZZ^T$, with the existing runs fixed (Goos & Jones 2011, ch. 10).

The preview shows how many runs are added and notes such as the aliases a fold-over removes or the D-efficiency before and after. **Create** opens a new table: every original row and column, then the new runs with their factor settings in actual units (other columns empty, ready for the responses), and a nominal **Block** column — 1 for the original runs and 2 for the added ones — to include as a factor in the analysis, since the new runs are usually made later. The order of the new runs can be randomized with a seed (More options). With **Keep in this project** ticked (the default) the augmented design joins the current table's project; clear it to open the design as a separate project.



New whole plots are numbered after the existing ones in the whole-plot column (5, 6, … after plots 1 to 4) and run in random order, the runs in random order within each. The notes name the hard-to-change factors and give the D-efficiency before and after and the whole-plot error degrees of freedom. Script: `t.augment_whole_plots(#{ factors, whole_plot, model, whole_plots, runs, ratio })`.

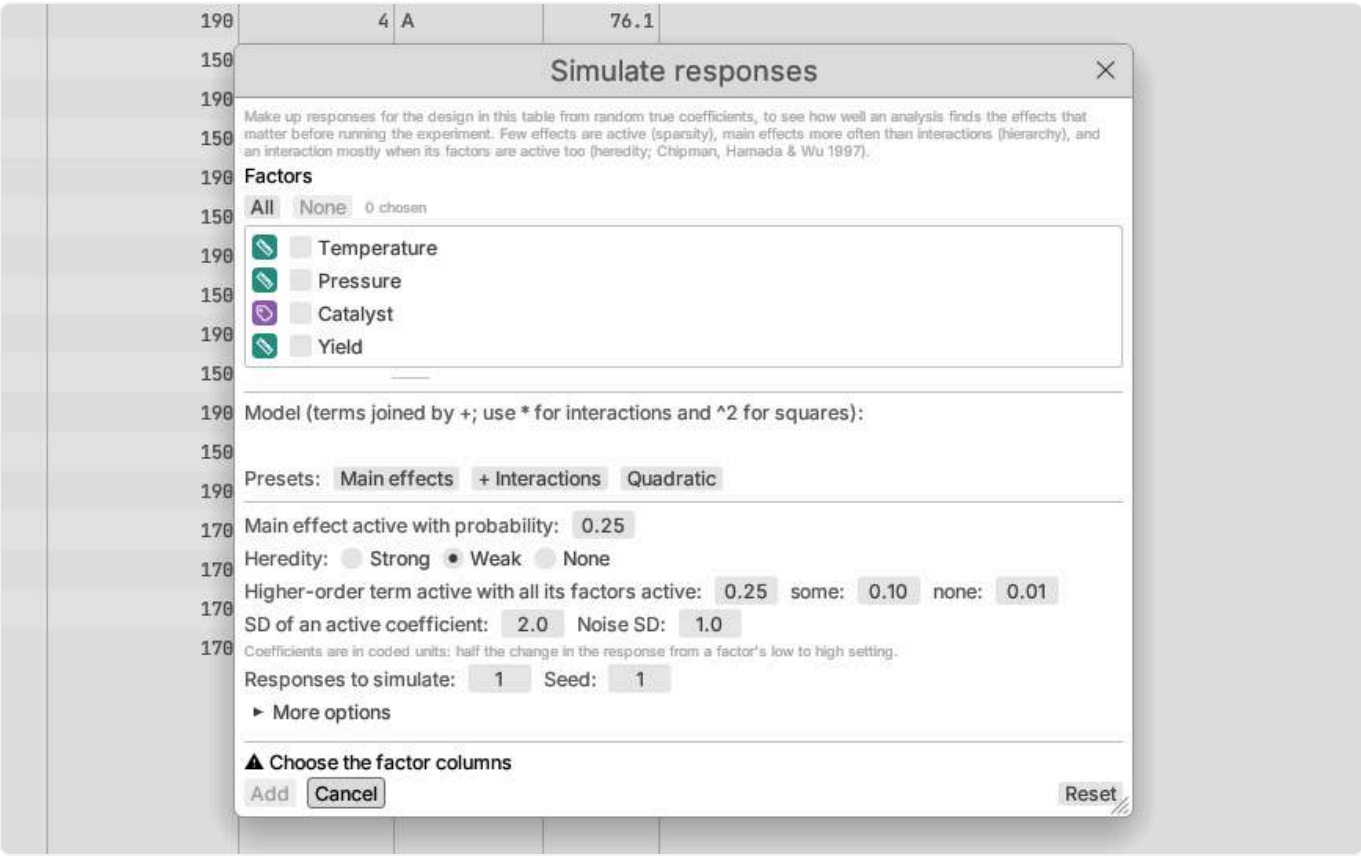
Simulating responses

Design ▸ Simulate responses… (with a table open) makes up responses for the design in the active table, to see before running the experiment whether the planned analysis finds the effects that matter. Each simulated response comes from its own random true coefficients for the model you give:

- **Sparsity**: each main effect is active with the main-effect probability p (default 0.25).
- **Hierarchy and heredity** (Chipman, Hamada & Wu 1997): an interaction or square is active with probability p_{11} when all its factors are active, p_{01} when some are and p_{00} when none is. **Strong** heredity sets (0.25, 0, 0), so an interaction can be active only

with its parents; **Weak** (the default) is (0.25, 0.10, 0.01); **None** gives every term the main-effect probability.
- Active coefficients are drawn from N(0, τ²) in coded units (half the change from low to high setting; τ = 2 by default), inactive ones are zero (or drawn with a small SD, under More options), and normal noise with the noise SD is added.

Factor columns are coded −1 and +1 at their smallest and largest values; a categorical factor needs two levels. **Add** puts the responses in the table as new columns (Simulated Y 1, 2, …; one edit, so Undo removes them) and adds a table of the true coefficients to the project, one row per response with its active terms, to compare with what the analysis finds.



Script: `t.simulate_responses(#{ factors, model, count, seed, main, all_parents, some_parents, no_parents, effect_sd, noise_sd })`.

Evaluating a design before running it

Design ▸ Evaluate design… judges the runs in the open table for a model before any responses are collected — a design made by the Design menu, one built elsewhere, or

several candidates to choose between. Choose the factor columns (coded −1 to +1 as in *Declaring a table as a design*: from each numeric column's smallest to largest value, or two-level categories in sorted order) and the model: main effects, main effects with two-factor interactions, the full quadratic, or a formula typed over the column names (`Temperature + Time + Temperature*Time + Temperature^2`). Variances are in units of the error variance σ^2, which is unknown until the experiment is run, and the anticipated coefficient is in units of σ: a coefficient of 1 for a main effect means the mean response changes by one standard deviation from the factor's midpoint to its high setting.

The report shows:

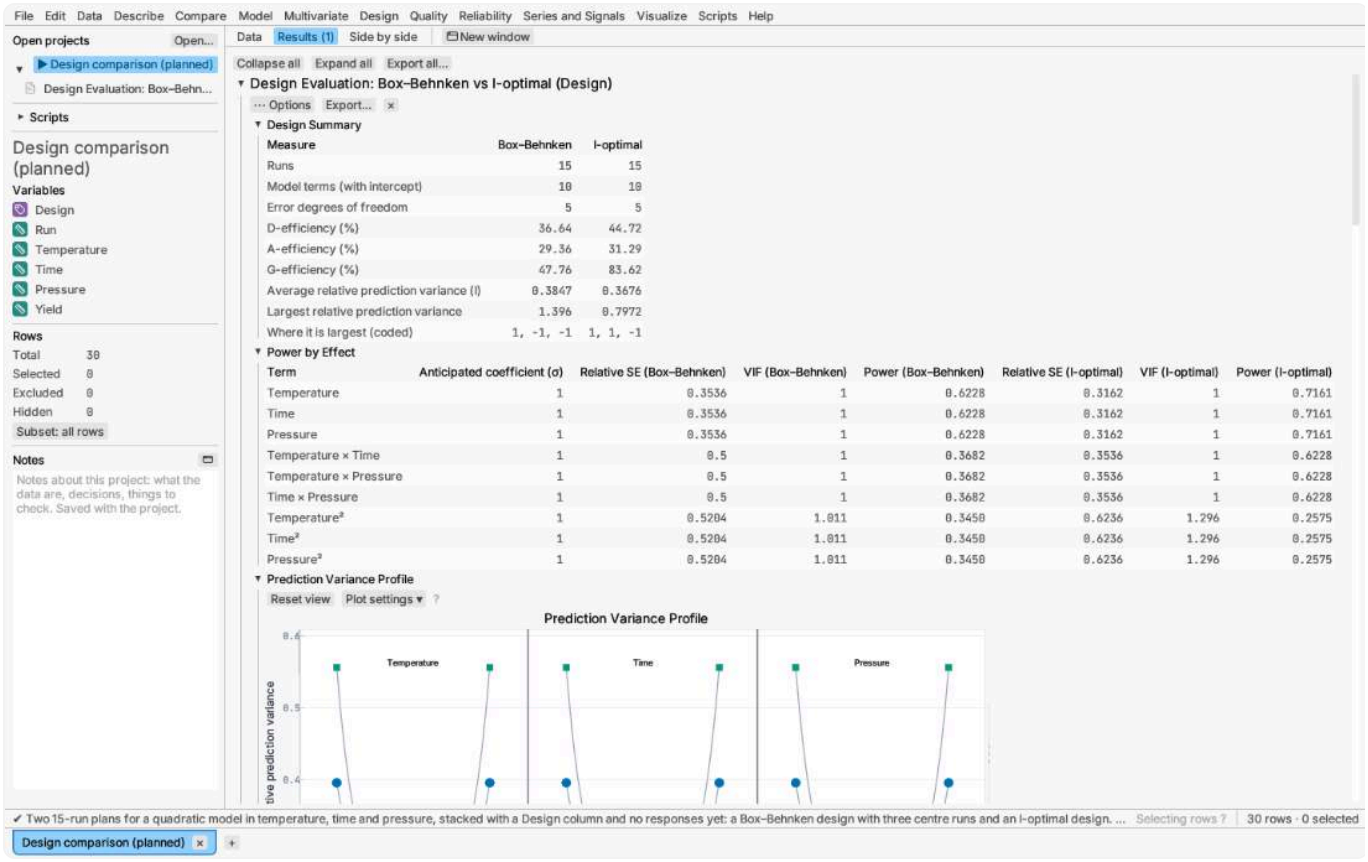
- **Design Summary:** runs, model terms, error degrees of freedom, and the D-, A- and G-efficiencies (100% for an orthogonal two-level design for its model) with the average and largest relative prediction variance;
- **Power by Effect:** for each term, its relative standard error $\sqrt{[(X^TX)^{-1}]_{jj}}$, variance inflation factor, and the power of the two-sided t test at level α for the anticipated coefficient. A design with as many runs as terms has no error degrees of freedom, so no power is shown;
- **Prediction Variance Profile:** the relative variance of the predicted mean along each factor, one pane per factor, with the other factors at their midpoints;
- **Fraction of Design Space:** the relative prediction variance at random points of the region, sorted, against the fraction of the region at or below each value (Zahran, Anderson-Cook & Myers 2003). A low, flat curve predicts well everywhere; a steep end shows corners predicted poorly. **Prediction Variance over the Region** lists its quantiles (**… over the Constrained Region** when constraints are given);
- **Alias Matrix:** how terms left out of the model would bias each estimate. By default these are the two-factor interactions not in the model (the three-factor ones when the model has every two-factor interaction); type others under *More options*. A 1 means complete aliasing; values such as ±1/3 mean partial aliasing;
- **Absolute Correlations of Terms:** a heat map of |r| among the model and alias columns, with each value printed in its cell;
- **About These Measures:** the model and how the measures are computed.

To compare two candidate designs, stack them in one table with a column naming each run's design and choose it as *Compare designs by*. Both designs are coded over the same region (all rows), each section shows both, the plots draw each design in its own colour and marker shape with a key, and a **Design Comparison** table gives the D, A, G and I criteria side by side with the first design's relative efficiency to the second: above 1 favours the first. The *per run* column divides out the numbers of runs.

The region is the cube of coded factor settings unless you type **Region constraints** in the dialog, in the factors' actual units and the same form as for optimal designs (`Temp + 4*Time <= 300` , `exclude Temp > 180 and Time > 25` , one per line). The largest and average prediction variance (and so the G- and I-criteria) and the fraction-of-design-space curve are then over the constrained region only, sampled at quasi-random points inside it, and the note says so; the variance profiles still run across the whole cube. A D- or I-optimal design created with constraints opens with this report already set up for its constraints.

The model, alias terms, coefficient, α, constraints and each factor's coded settings can be changed from the report's options menu; the report updates at once.

Try it: Design comparison (planned): a 15-run Box–Behnken design and a 15-run I-optimal design for the quadratic model in three factors. The I-optimal design, which uses the corners of the cube, has the smaller average prediction variance and a flatter fraction-of-design-space curve; the Box–Behnken design predicts poorly at the corners it never visits.



Factorial analysis

Design ▸ Factorial analysis… analyses a two-level factorial or fractional factorial experiment held in the open table, one row per run. Choose the factor columns and the response:

- **Factors** are coded −1 (low) and +1 (high) as for a declared design: a numeric factor's smallest and largest values (or the settings given when the report was declared), a categorical factor's two levels in sorted order. Centre runs (coded 0) are used in the fit and left out of the plotted means.
- **Response type:** *Continuous* is fitted by least squares; *Binary* (0/1, or any column with two levels, modelling the probability of the second) by logistic regression; *Count* by Poisson regression. Binary and count effects are on the log-odds or log-mean scale, with Wald *z* tests and the odds or rate ratio from low to high.
- **Model terms:** every interaction the runs can estimate (the default; in a fraction, a term aliased with an earlier one is left out and listed in the summary), interactions of up to a number of factors, or a formula such as `A + B + C + A*B`.

More options set the significance level α, whether the plots show the data means or the fitted model's predictions (factors not shown held at their centre), and, with more than three factors, which three make the cube.

The **effect** of a term is twice its coefficient in coded units: the change in the mean response from the term's low to its high setting. The report shows:

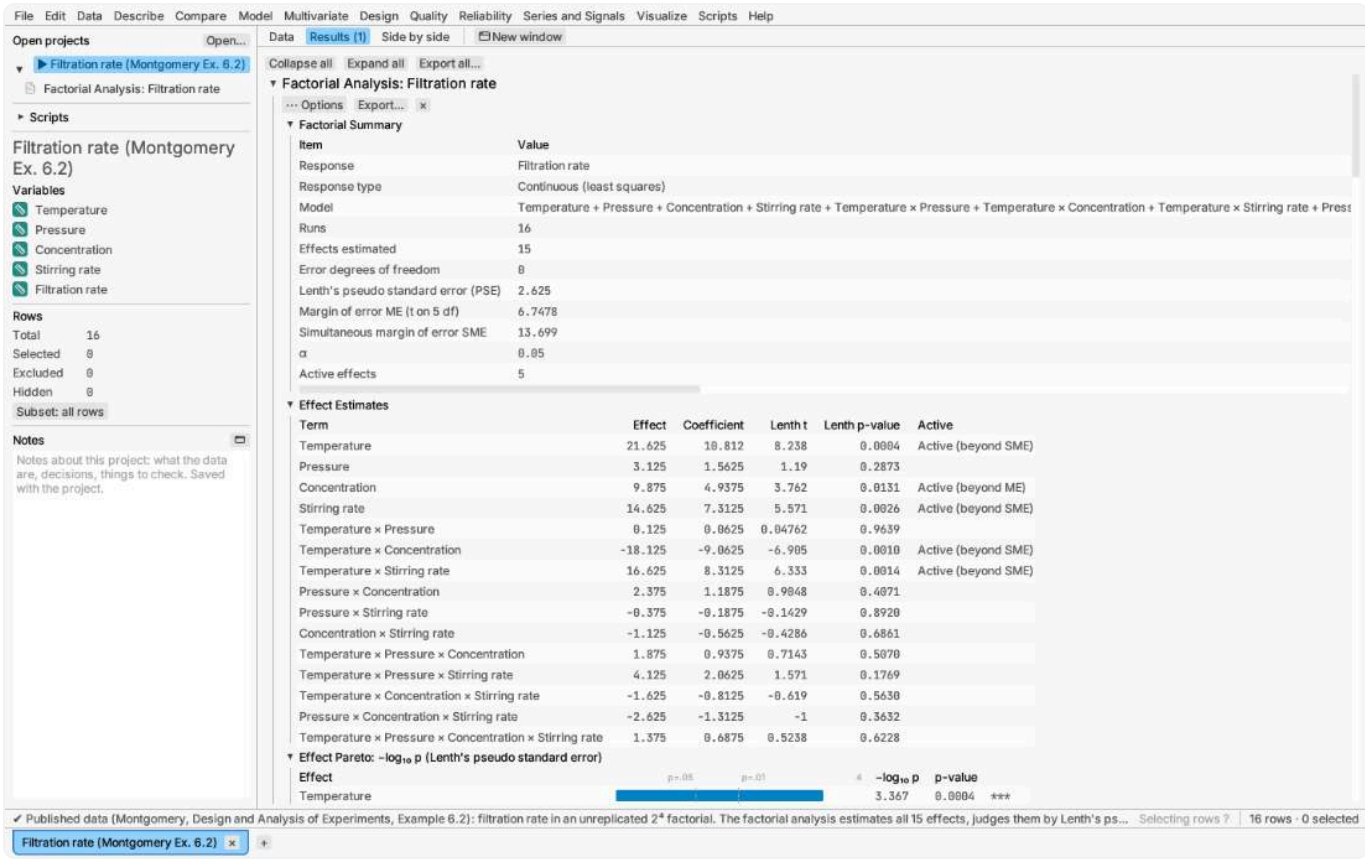
- **Factorial Summary:** model, runs, error degrees of freedom, root mean square error (or deviance), and Lenth's pseudo standard error (PSE), margin of error (ME) and simultaneous margin of error (SME) when they are used.
- **Effect Estimates:** each term's effect and coefficient. With error degrees of freedom (replicates, centre runs or terms left out) each effect has a standard error and t test. Without them (a saturated model) Lenth's method judges the effects: effects beyond ME are marked active, beyond SME active allowing for all the comparisons.
- **Effect Pareto** of $-\log_{10} p$.
- **Normal and half-normal plots of effects:** each effect labelled with its term, against normal (or half-normal) quantiles. Inert effects lie near the line through the origin whose slope is their standard error; active effects stand off it and are drawn with their own marker shape and colour. Vertical lines mark ME and SME.
- **Main Effects:** one pane per factor with each run's response and the line through the two level means.
- **Interaction** plots for each pair of factors (of the first five): the mean at each setting of the first factor, one line per setting of the second. Lines that are not parallel show an

interaction.

- **Cube Plot** (or square for two factors): the mean (or prediction) at each corner of three factors.
- **Dispersion effects:** for a replicated design, the standard deviation at each setting and the effects of ln s (Box & Meyer), with a normal plot; for an unreplicated design, the Bergman–Hynén test for each factor (the location model of the active effects refitted within the runs at each setting, comparing the residual variances by an F ratio) and Box–Meyer's ln ratio of residual sums of squares.
- **Notes**, among them how many runs missed their target settings. The model is fitted at the settings actually run, so the effects stay unbiased.

Terms, response type, α and the means shown are changed in place from the report's options menu.

Try it: Filtration rate (Montgomery's unreplicated 2^4, with four effects active beyond SME) and Coating thickness (a replicated 2^3 whose oven temperature changes the variability).



Gauge study worksheet

Design ▸ Gauge study worksheet… (also under Quality) creates the run sheet for a gauge study before any measurements are taken. Give the number of parts, the operators' names, the number of replicates and the layout:

- **Crossed:** every operator measures every part.
- **Nested:** each operator measures their own parts (for destructive tests), so parts are labelled within operators.

The run order can be standard, fully random, or random within each replicate (each operator–part combination once per round, in a new order each round), with a seed. The sheet opens as a new table with Run, Replicate, Operator and Part columns and an empty measurement column. Analyse a crossed study with Quality ▸ Gauge R&R… or Intraclass correlation….

Quality

The Quality menu is in five groups, each under a small heading: **Control charts**, **Process capability**, **Measurement systems** (gauge studies and attribute agreement), **Acceptance sampling**, and **Problem-solving tools** (analysis of means, Pareto chart, cause-and-effect diagram and multi-vari chart).

Control charts

Quality ▸ Control chart… draws variables control charts: **X̄ and R**, **X̄ and S** (subgroups) and **individuals and moving range**, with the eight **Nelson run rules**, and the **moving-average**, **three-way (between/within)**, **zone** and **short-run** charts described below. Flagged points are drawn as diamonds and labelled with the numbers of the rules they break. Hover over a chart type in the dialog for what it is for.

Building a chart from column roles

The dialog has five roles. Choose a column for each from its list, or drag a column from the variables list onto it; choose **None** to clear an optional role.

- **Measurement** (required): the numeric column to chart.
- **Subgroup ID:** rows sharing a value form one subgroup, in order of first appearance, so the rows of a subgroup need not be adjacent and subgroups may differ in size. Without it, consecutive rows are grouped by the **Subgroup size**. Giving a subgroup ID switches an individuals chart to X̄ and R.
- **Phase:** each run of consecutive rows with the same value (for example *before* and *after* a process change) is a phase with its own center line, σ and limits, computed from that phase's points only. Run rules restart at each phase, the moving range is not taken across a boundary, and the chart separates the phases with a solid rule, shades every other phase lightly and names each one in a label centred along the top edge of the chart (the key lists the *Phase boundary*). Exported charts show the same.
- **Point label:** names each point (from the first row of its subgroup) in the tooltip and the table of flagged points. Without it, a subgroup ID serves as the label.
- **Part number** (short-run charts only): a column of any type naming the part each row belongs to. Assigning it switches the chart type to short-run.

The measurement, subgroup ID, phase and part number must be different columns; the dialog names any column chosen for two of them. The point label may repeat another role.

When subgroups differ in size, the limits are computed for each size and drawn as steps: a smaller subgroup has wider limits (Montgomery, *Introduction to Statistical Quality Control*, §6.3.1). The Control Limits table then shows each limit as a range, and for several phases it has a row per phase.

Rows with a missing measurement, subgroup ID or phase are left out. $\bar{X}$ charts leave out subgroups with a single value. Excluded rows leave the chart and its limits, so excluding an assignable-cause point recomputes the limits without it.

Changing a chart in place

The report's **Options** menu switches the **chart type** (and the subgroup size, when there is no subgroup ID column) without reopening the dialog, and chooses the **run rules** applied to each chart. By default all eight apply to the mean or individuals chart, and only rule 1 (a point beyond the limits) to the dispersion chart. It also shows or hides the dispersion chart, the 1σ and 2σ zone lines, and the table of flagged points. Rules 5 to 8 measure distances in σ, so they are not applied when the limits have no spread (σ = 0, as for a series that never varies).

Hover over a point to see its sample number, label, phase and the rules it breaks.

Limit and display options

More options in the dialog set how the limits are drawn on individuals, $\bar{X}$–R and $\bar{X}$–S charts (Montgomery, *Introduction to Statistical Quality Control*, ch. 6–7; Wheeler, *Advanced Topics in SPC*):

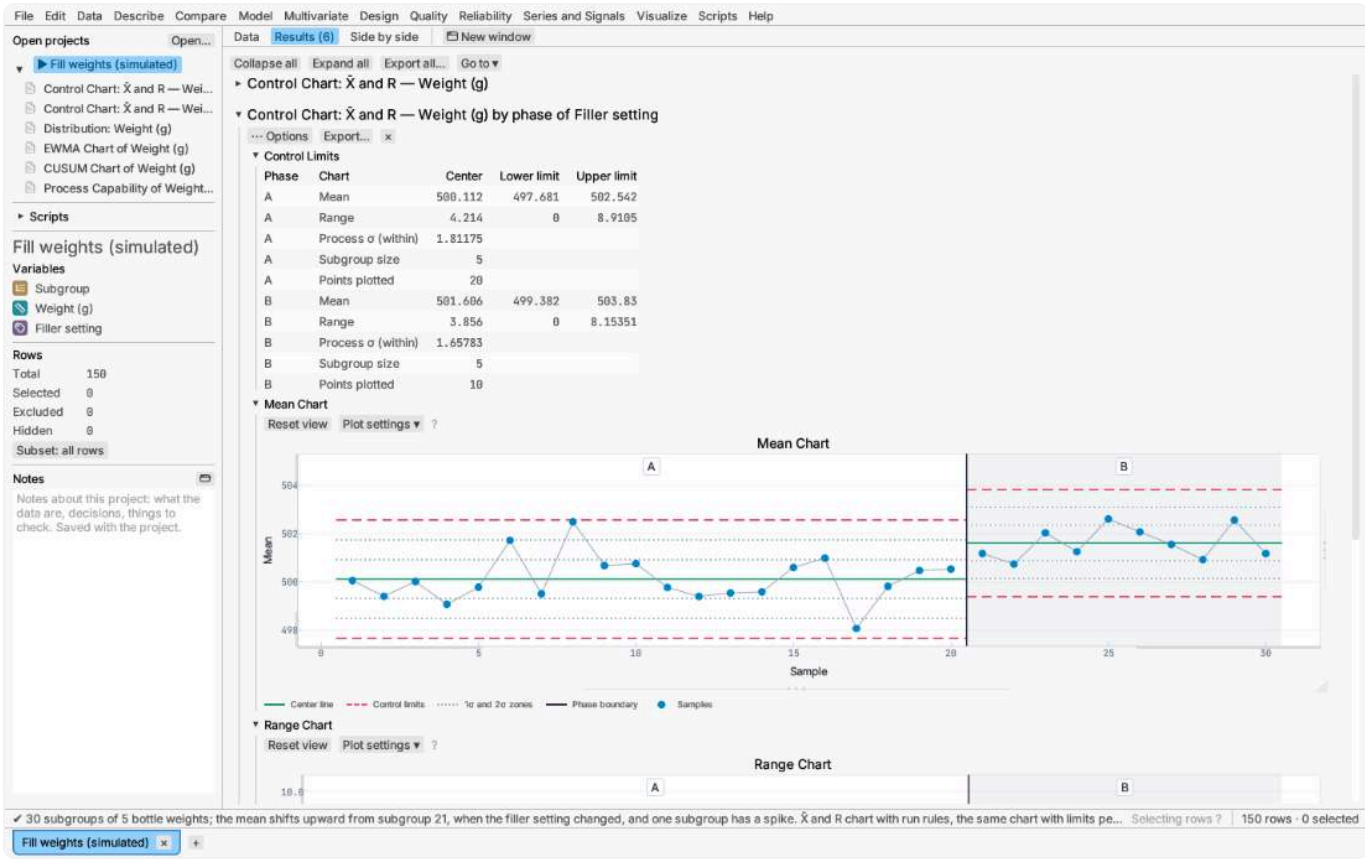
- **Limits at k σ:** the limits are center ± k$\hat{\sigma}$ (± k$\hat{\sigma}$/√n for means) and the dispersion limits use k in place of 3, for example 2.5σ for earlier warning. The run rules' zones stay at 1σ and 2σ.
- **Values per moving range** (individuals): each moving range is the largest minus the smallest of w consecutive values, $\hat{\sigma} = \bar{M}R / d_2(w)$, and the moving-range limits are $\bar{M}R\,(1 \pm k\, d_3(w)/d_2(w))$, the lower one at least 0. The first w − 1 points have no moving range; hovering over a moving range lists the w rows it spans.
- **Limits from standard (historical) values:** type a process center μ_0 and σ_0, or press **Read from …** to take them from a limits table saved earlier (any open table with

Measurement, Process center and Process sigma columns and a row for this measurement). The center is μ_0, the limits $\mu_0 \pm k\sigma_0/\sqrt{n}$, and the range, standard deviation or moving range chart is centered on $d_2\sigma_0$, $c_4\sigma_0$ or $d_2(w)\sigma_0$ with limits $(d_2 \pm k\, d_3)\sigma_0$, $(c_4 \pm k\sqrt{(1 - c_4^2)})\sigma_0$ or $(d_2(w) \pm k\, d_3(w))\sigma_0$. Use them to judge new data against limits set in an earlier, stable period.

- **Show only the last N subgroups** (any chart type): the limits come from all subgroups, with the earlier ones hidden, or from the N shown alone. The limits table says how many are shown.

The report's Options menu adds a **Run chart of every value**: each value at its subgroup's position, with the subgroup medians (or means) joined and a center line at the median (or mean) of all values. It also has **Save limits as data table**, which adds a table with each phase's center, limits, process center, process σ, subgroup size and σ multiple; open it with a later dataset and its values can be read back as standard values.

Try it: Fill weights (an $\bar{X}$ and R chart by the Subgroup column, and the same chart with Filler setting as the phase: the upward shift from subgroup 21 is flagged on the first chart, while the second gives each setting its own limits); Beam deflection (an individuals chart on which no rule fires: the readings swing so much from one to the next that the moving ranges, and so the limits, are wide. The runs test in its distribution report catches the oscillation).



Moving-average chart

Each point is the average of the last *w* values (individual values, or subgroup means with a subgroup size or ID), with *w* the **span** (default 5, set in the dialog or the Options menu). Averaging makes the chart more sensitive to small sustained shifts than an individuals chart. With σ estimated as for the individuals or $\bar{X}$ chart, the limits of point *i* are

$$\bar{\bar{x}} \pm 3\sigma \sqrt{(\Sigma_j 1/n_j)} / w_i,$$

summing over the w_i subgroups in its window; the first *w* − 1 points of each phase average fewer values and have wider limits. Because consecutive averages share values, only rule 1 (a point beyond the limits) applies; the other run rules are switched off for this chart. The dispersion chart is the usual moving-range, R or S chart.

Three-way (between/within) charts

When the subgroup means vary more than the spread within subgroups allows (for example readings across a coil, with coil-to-coil differences that are part of the process), an $\bar{X}$ chart's limits, built from the within-subgroup spread, flag nearly every subgroup. The **three-way** chart (Wheeler 1995, *Advanced Topics in Statistical Process Control*) draws three charts:

- the subgroup means as **individual values**, with limits from the moving range of the means: $\bar{\bar{X}} \pm 3\,M\bar{R}/d_2(2)$;
- the **moving range** of the means;
- a **within-subgroup** range (three-way R) or standard-deviation (three-way S) chart, titled *Within-Subgroup … Chart*, with its own run-rule choice in the Options menu.

Zone chart

The zone chart (Jaehn 1987) scores each point by the σ zone it falls in: 1 within 1σ of the center line, 2 between 1σ and 2σ, 4 between 2σ and 3σ and 8 beyond 3σ (a point exactly on a boundary belongs to the inner zone). Scores add up while points stay on the same side of the center line; the total restarts when a point crosses to the other side and after a signal. The chart **signals** when the total reaches 8. Each point is labelled with its total, signalling points are drawn as diamonds, and the flagged-point table lists them with the rule "zone". The usual run rules are switched off. Scoring 1 for points near the center line means eight points in a row on one side signal, so this chart signals by chance more often than a chart with rule 1 alone.

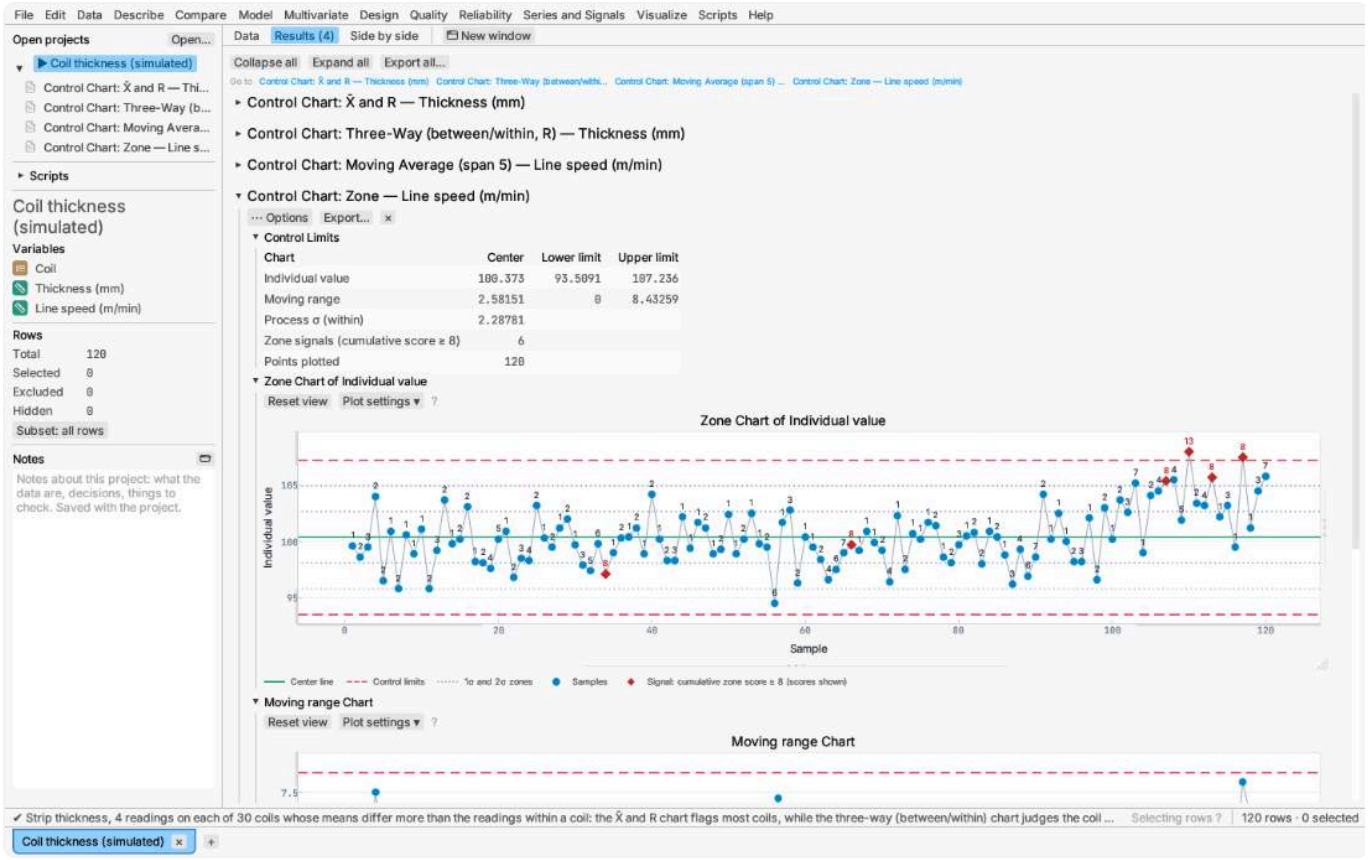
Short-run charts for several part numbers

When one process makes several part numbers in short runs, no single part has enough data for a chart of its own. The **short-run Z and moving-range chart** (Griffith 1996, *Statistical Process Control Methods for Long and Short Runs*) standardizes every value by its own part's estimates,

$Z = (x - \bar{x}_p) / \sigma_p$, with $\sigma_p = M\bar{R}_p / d_2(2)$,

where $M\bar{R}_p$ is the average moving range between consecutive values of the same part (within the phase). The Z chart has center line 0 and limits ±3; the moving range of the Z values has center line $d_2(2)$ and upper limit $d_2(2) + 3d_3(2)$. The **Part Estimates** table gives each part's count, mean and σ (per phase). Each part needs at least two values in each phase.

Try it: Coil thickness (the $\bar{X}$ and R chart flags most coils, the three-way chart judges the coil means against their own variation; the moving-average and zone charts of Line speed catch the shift from reading 101); Short-run machining (three part numbers on one chart, where the Flange parts made after the tool slipped stand out).



Count and rare-event charts

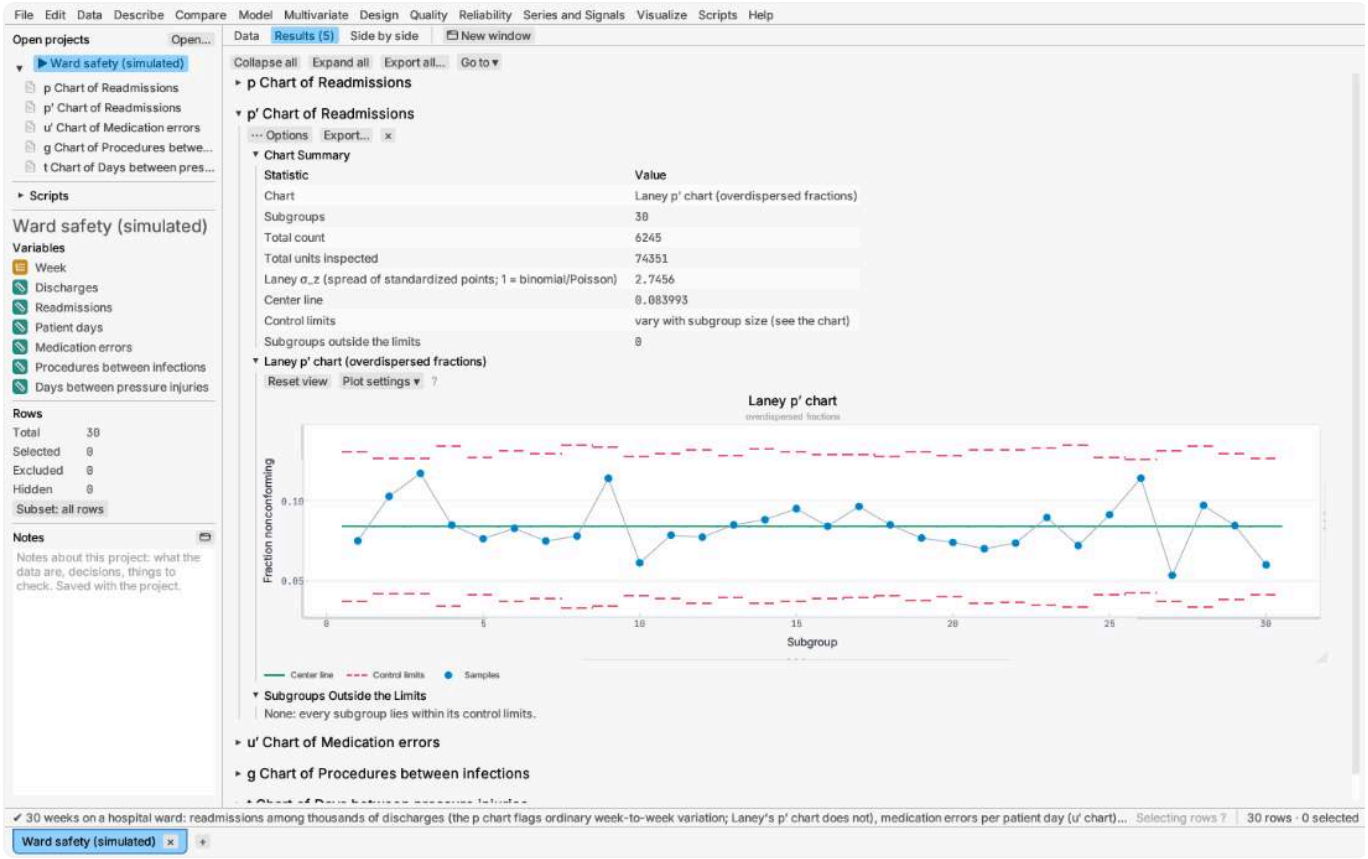
Quality ▸ Count and rare-event chart (p, np, c, u, p', u', g, t)... charts counts, one row per subgroup:

- **p** and **np** charts for defective units, **c** and **u** charts for defects per subgroup;
- **Laney p'** and **u'** charts (Laney 2002) for large subgroups, where real variation of the rate from subgroup to subgroup (overdispersion) makes the ordinary limits so narrow that most points fall outside. Each subgroup's value is standardized, $z_i = (p_i - \bar{p})/\sigma_{p_i}$, and σ_z, the moving range of the z values divided by $d_2(2)$, widens the limits to $\bar{p} \pm 3\sigma_z \sigma_{p_i}$. σ_z near 1 means no overdispersion and the chart is the ordinary one; the summary table reports it;
- the **g chart** (Benneyan 2001) for rare events, with one row per event giving the number of units (cases, procedures, opportunities) since the previous one. The counts are geometric, with center line $\bar{g}$ and limits $\bar{g} \pm 3\sqrt{(\bar{g}(\bar{g} + 1))}$ (the lower limit is usually 0). A point above the upper limit is an unusually long run without an event: an improvement;

- the **t chart** (Nelson 1994) for the time between rare events. Times are transformed by y = t^(1/3.6), which makes exponential or Weibull times roughly normal; an individuals and moving-range chart of y gives the limits, which are transformed back and drawn on the original time scale (so they are not symmetric).

The dialog takes the count column, the subgroup size (a column or a constant) for the charts that need one (for p, np and p' the number of units inspected, a whole number; for u and u' the number of inspection units, which may be fractional, such as 2.5 m² of panel), an optional **phase** column (each run of equal values gets its own center line and limits), and whether to apply the **run rules**. The rules are applied to the standardized values (or, for the t chart, the transformed values), so they work with limits that vary with subgroup size; the zones are measured in the plotted statistic's σ even where a proportion's upper limit is capped at one. The Options menu switches the chart type and the run rules in place.

Try it: Daily inspection (p, u and c); Ward safety (the p chart of readmissions flags many weeks, while the p' chart, with σ_z well above 1, flags almost none; a u' chart of medication errors; a g chart with one long run without infection; a t chart whose run rules catch pressure injuries becoming more frequent).



EWMA and CUSUM charts

Quality ▸ EWMA chart… and CUSUM chart… detect small, persistent shifts. The target and σ can be given or estimated from the data: without them the target is the mean and σ the average moving range ÷ d_2. A signals table lists where the chart signals.

- **EWMA** plots the exponentially weighted moving average with weight **λ** (default 0.2, from 0.01 to 1; smaller λ remembers more of the past and catches smaller shifts) and limits **L** standard errors wide (default 3).
- **CUSUM** accumulates deviations above and below the target, less a reference value **k** (default 0.5), and signals when a sum exceeds the decision interval **h** (default 5), both in units of σ; the settings table shows them in data units as K = kσ and H = hσ. Choose k as about half the shift you want to detect.

In scripts these are the `ewma-chart` (`lambda` , `width`) and `cusum-chart` (`k` , `h`) analyses.

Try it: Fill weights.

Multivariate control charts

Quality ▸ Multivariate control chart… watches two or more correlated measurements together, where separate charts for each would miss a point that is unusual only in combination (a high fill volume without the torque that normally goes with it). Tick the measurements, then choose:

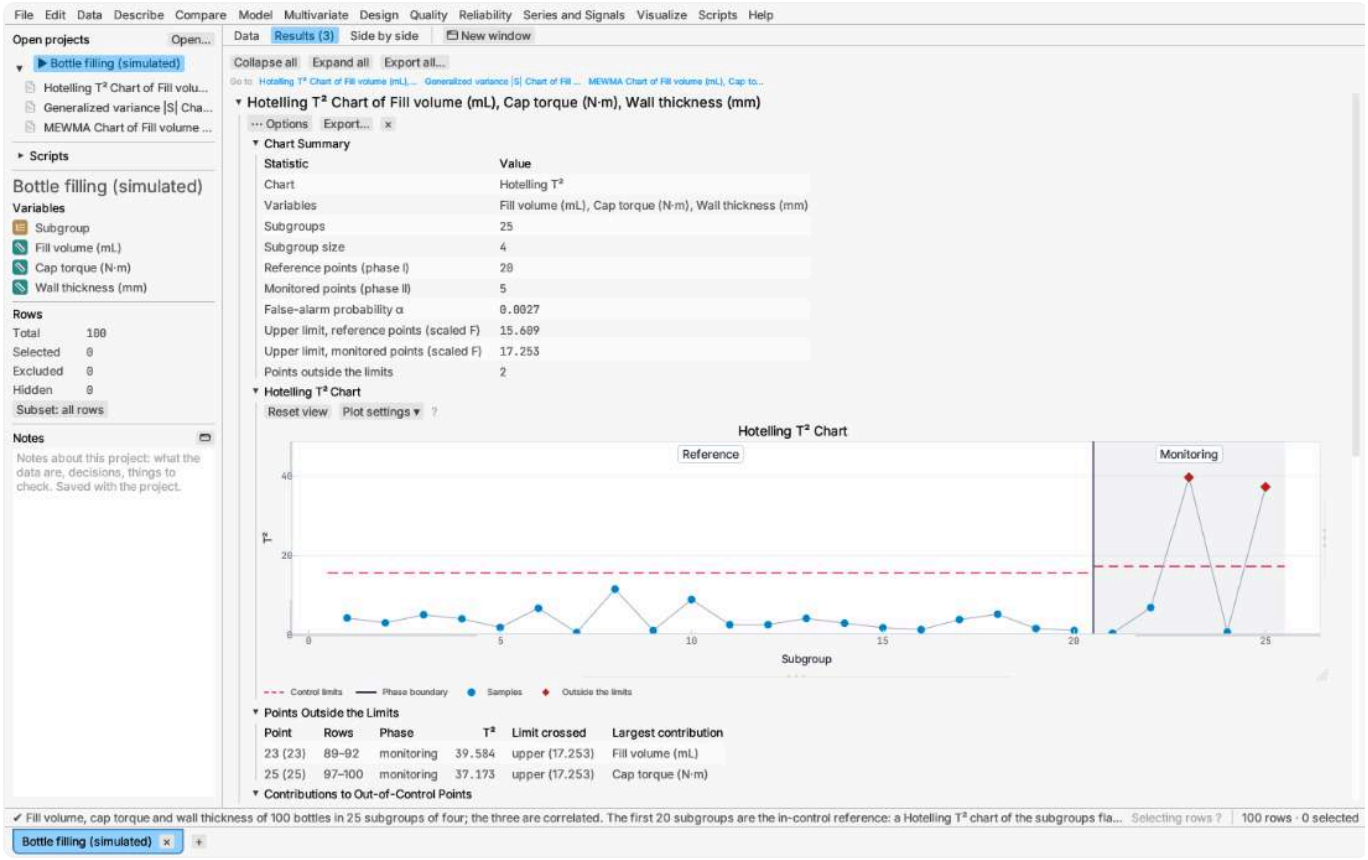
- **Hotelling T²**: the squared distance of each observation, or each subgroup mean, from the overall mean, scaled by the covariance. Subgroups come from an ID column or from consecutive runs of a fixed size (1 gives individual observations).
- **Generalized variance |S|**: the determinant of each subgroup's covariance matrix, which grows when the spread grows. It needs subgroups of at least two, and more rows per subgroup than measurements. Limits sit three standard errors either side of the expected |S| (Alt 1985); a negative lower limit is set to zero.
- **MEWMA**: an exponentially weighted moving average of the individual observations, charted as a T² distance (Lowry et al. 1992). Small, sustained shifts build up over several points. Set the weight λ (0.1 by default) and either the upper limit H or the in-control average run length the limit should give (200 by default, at most 2000); the app finds that limit by simulation, the same each time. The simulation follows 5000 runs up to a run length of 200 and fewer beyond it (500 at 2000), so the limit for a

long run length is a little less precise (about 4.5% in run length at 2000) but is found in about a second.

Estimate from the first *m* points splits the chart into a reference sample and a monitoring stretch. The mean and covariance come from the reference sample only. Reference points are compared with limits for the data that produced them (a beta distribution for individual observations, F for subgroups); later points are compared with F limits for new observations (Montgomery, *Introduction to Statistical Quality Control*, ch. 11). Leave it off to use every point for both. α (0.0027 by default, that of three-sigma limits) is the false-alarm probability per point.

T^2 and MEWMA charts have an upper limit only. A point above it is drawn as a diamond, with the colour as a second cue, and listed under **Points outside the limits**.
Contributions splits each such point's statistic by variable: a variable's contribution is T^2 minus the T^2 computed without it (Runger, Alt & Montgomery 1996), so a large value names a variable that drives the signal. Values above the χ^2 value with one degree of freedom that is exceeded with probability α are marked ▲, and the contribution chart draws them against that guide, one line per point. The estimates table gives the reference means and covariance. Points select their rows in every other view.

Try it: Bottle filling (a T^2 chart of the subgroups, a |S| chart and a MEWMA chart of the bottles, each estimated from the first part of the run).



Principal-components control chart

Quality ▸ Principal-components control chart… watches many correlated measurements, such as the sensors on a reactor, through a principal-components model of a reference period (Jackson & Mudholkar 1979; Nomikos & MacGregor 1995; Kourti & MacGregor 1996). Where a Hotelling T² chart of every variable needs more reference rows than variables and loses power when the variables are nearly collinear, this chart works on the few components that carry most of the variation, and adds a second chart for everything the components leave out. Tick the measurements; optionally choose a column that labels the points.

Fit the model to the first *m* rows marks the reference period (normal operation); the rest are monitored against it. Leave it off to use every row for both. The model centres each variable on its reference mean and, by default, divides by its reference standard deviation (a correlation model; untick **Standardize** in More options to use covariances, when the variables share units). The number of components is chosen by cross-validation, or set by hand. Cross-validation leaves out a fold of reference rows (row *i* goes to fold *i* mod *k*, 7 folds by default), refits the model, and predicts every value of each left-out row from that

row's other values; the number of components with the smallest sum of squared prediction errors (PRESS) is kept (Bro, Kjeldahl, Smilde & Kiers 2008). The table **Choosing the Number of Components** lists PRESS and the cumulative variance for each number.

Two charts follow, both with an upper limit only:

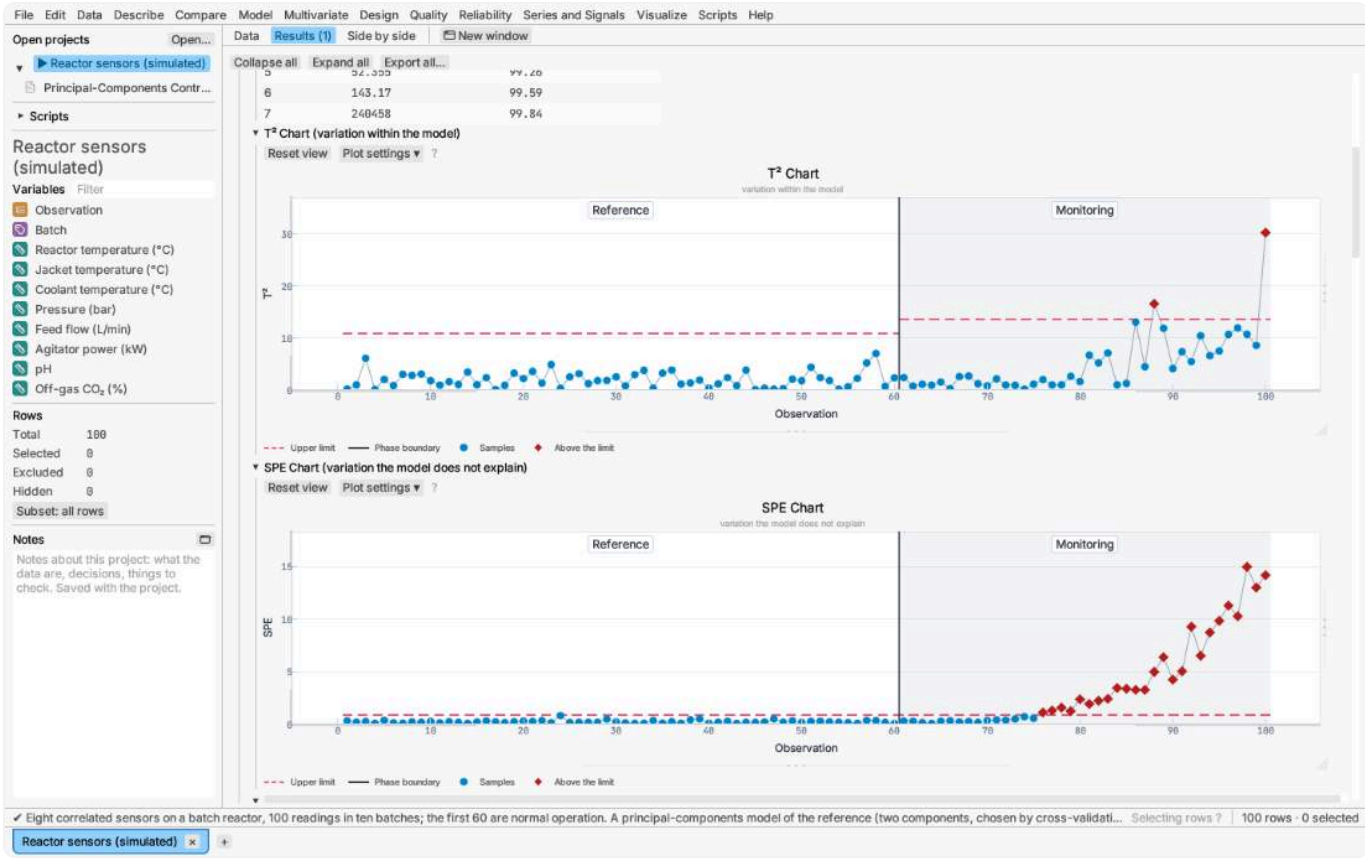
- **T^2 chart**: the scores of each row on the retained components, $T^2 = \Sigma\, t_a^2/\lambda_a$. It rises when a row moves along the usual patterns of variation, but further than usual. Reference rows have the scaled-beta limit and monitored rows the F limit for new observations (Tracy, Young & Mason 1992), as for a T^2 chart of that many variables.
- **SPE chart** (squared prediction error, also called Q): the squared distance between a row and its projection on the model. It rises when the usual correlations break, for example when one sensor drifts while the others do not follow. The limit is Jackson and Mudholkar's approximation from the eigenvalues the model leaves out, or Box's scaled χ^2 (More options). With every component retained nothing is left over and there is no SPE limit.

α (0.0027 by default, that of three-sigma limits) is the false-alarm probability per point. **Points Above the Limits** lists every point above either limit, with the variable that contributes most. **Contributions** split each signal by variable: for T^2, each variable's standardized value times its weight in the retained scores (the shares add up to T^2; a share can be negative); for SPE, each variable's squared residual (these add up to SPE). A contribution plot draws them across the variables, one line per point (the eight largest signals). **Signalling Points against the Reference** draws the same points' standardized values across the variables, in reference standard deviations, against the reference mean and ±3 SD, so you can see which way each variable moved.

When anything signals, **Change Point in the Mean** estimates where a single shift in the mean of the retained scores happened, over all rows: the split that maximizes the likelihood-ratio statistic for a change in the mean vector with a common covariance (Sullivan & Woodall 2000). Its p-value comes from 999 random reorderings of the rows (the same each time); the scan plot shows the statistic for every split. The last table gives the reference means, standard deviations, loadings and eigenvalues.

Every point is one row, so brushing a chart selects its row everywhere. The options menu changes the reference period, the model and the limits in place. In scripts this is the `pca-chart` analysis.

Try it: Reactor sensors (the coolant sensor drifts from reading 71, caught by the SPE chart; the production rate runs high from reading 86, caught by the T^2 chart).



Environmental monitoring limits

Quality ▸ Monitoring alert and action limits… sets alert and action levels for environmental monitoring of clean rooms and similar areas, from the data themselves, and charts each sampling location against them (USP ⟨1116⟩; PDA Technical Report 13). Choose what the readings are:

- **Counts**, such as colony-forming units per plate. The candidate families are the Poisson, the negative binomial (counts more spread out than a Poisson), and the zero-inflated Poisson and negative binomial (more plates with no growth than either expects; Lambert 1992; Hilbe 2011).
- **Positive samples out of samples taken**, such as contact plates with any growth: give the number taken, from a column or the same for every reading. The candidates are the binomial and the beta-binomial (positives that vary more from day to day than a binomial). Readings and levels are shown as proportions.
- **Continuous readings** of zero or more, such as particle counts per m³: the fourteen continuous families of distribution identification.

Each family is fitted by maximum likelihood and the one with the smallest AIC is used, unless you pick a family. A family whose extra parameter runs to its limit (no extra spread, or no extra zeros) is listed as not fitted, because it is then the simpler family. **Families Compared** lists every candidate's log-likelihood, AIC and ΔAIC.

Choose the location column (each location gets its own chart and summary row; leave it empty for one location) and a time-order column (a date or number; otherwise table order). **Set levels for** says which readings share levels: all locations together, each location, or each level of another column such as a room grade. A missing location or grade is a group of its own, "(missing)".

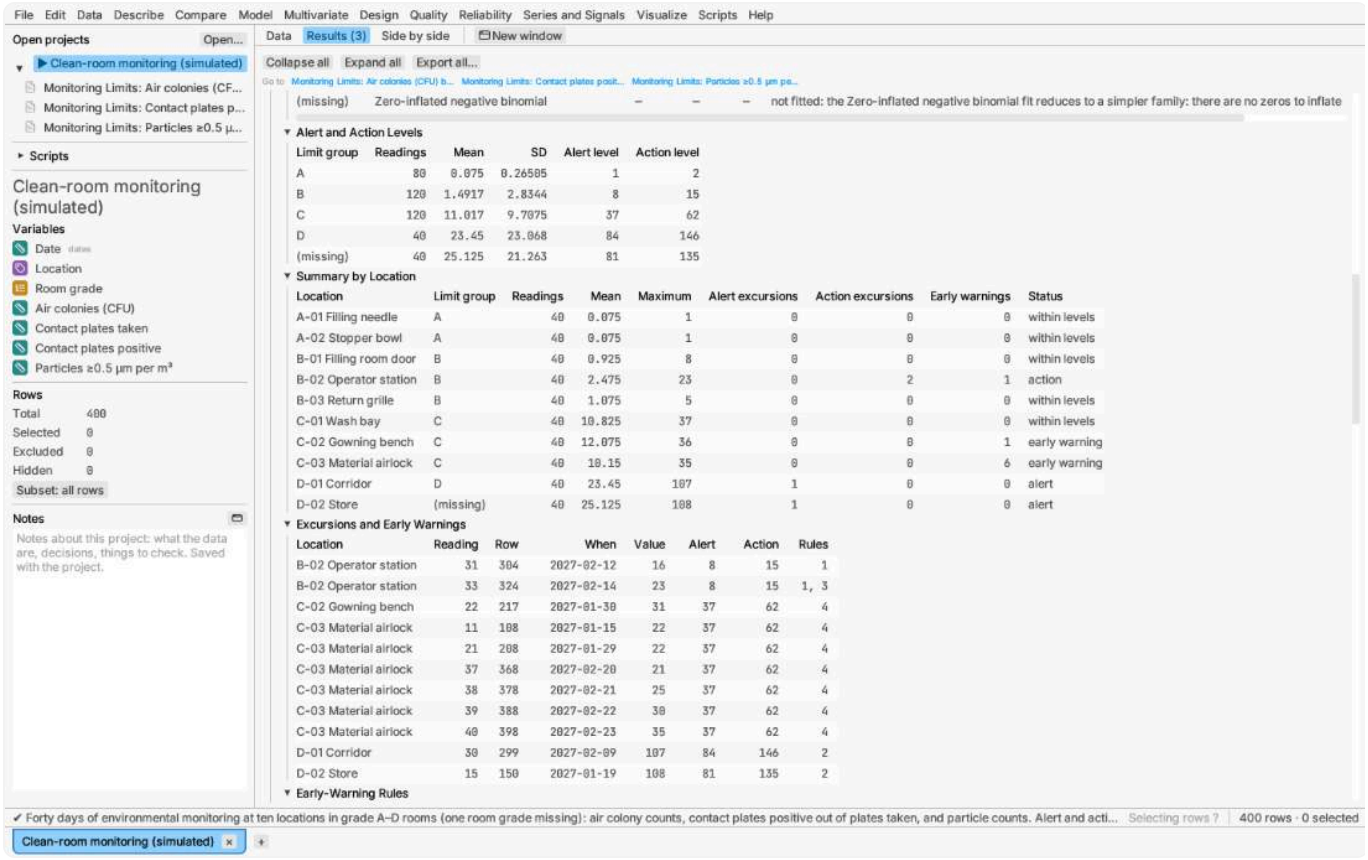
Levels from:

- **Quantiles of the fitted distribution** (default 0.975 for the alert level and 0.9985 for the action level). For counts the level is the smallest whole count whose cumulative probability reaches the quantile, so a count above it has at most that small a chance.
- **Mean + k standard deviations** (2 and 3 by default), from each group's readings; for positives $\bar{p} + k\sqrt{\bar{p}(1 - \bar{p})/n}$, so the levels follow the number of samples taken.
- **Typed in** for each group; a row with an empty group applies to every group not listed.
- **From columns of the data**: an alert-level and an action-level column (bring a table of levels in with Data ▸ Join first). A row without a value takes the first value of its group.

A reading **above** a level is an excursion; a reading equal to it is not. Each location's readings are checked in time order against numbered rules, shown beside the points: 1, an action excursion; 2, an alert excursion; 3, *n* or more alert excursions within *m* readings (2 in 5 by default); 4, *k* increases in a row (3 by default). Rules 3 and 4 can be changed or turned off in More options. **Summary by Location** gives each location's readings, mean, maximum, excursions, early warnings and a status; **Excursions and Early Warnings** lists every flagged reading with its date and row. Tick **Chart only the locations with a signal** to keep the report short. Each location chart draws the alert level (lower dashed line) and the action level (upper dashed line); its title gives both values.

Every point is one row, so brushing links the charts with the data. The options menu changes the levels and rules in place. In scripts this is the `monitoring-limits` analysis.

Try it: Clean-room monitoring (air colonies by room grade, contact plates, and particle counts by location).



Analysis of means

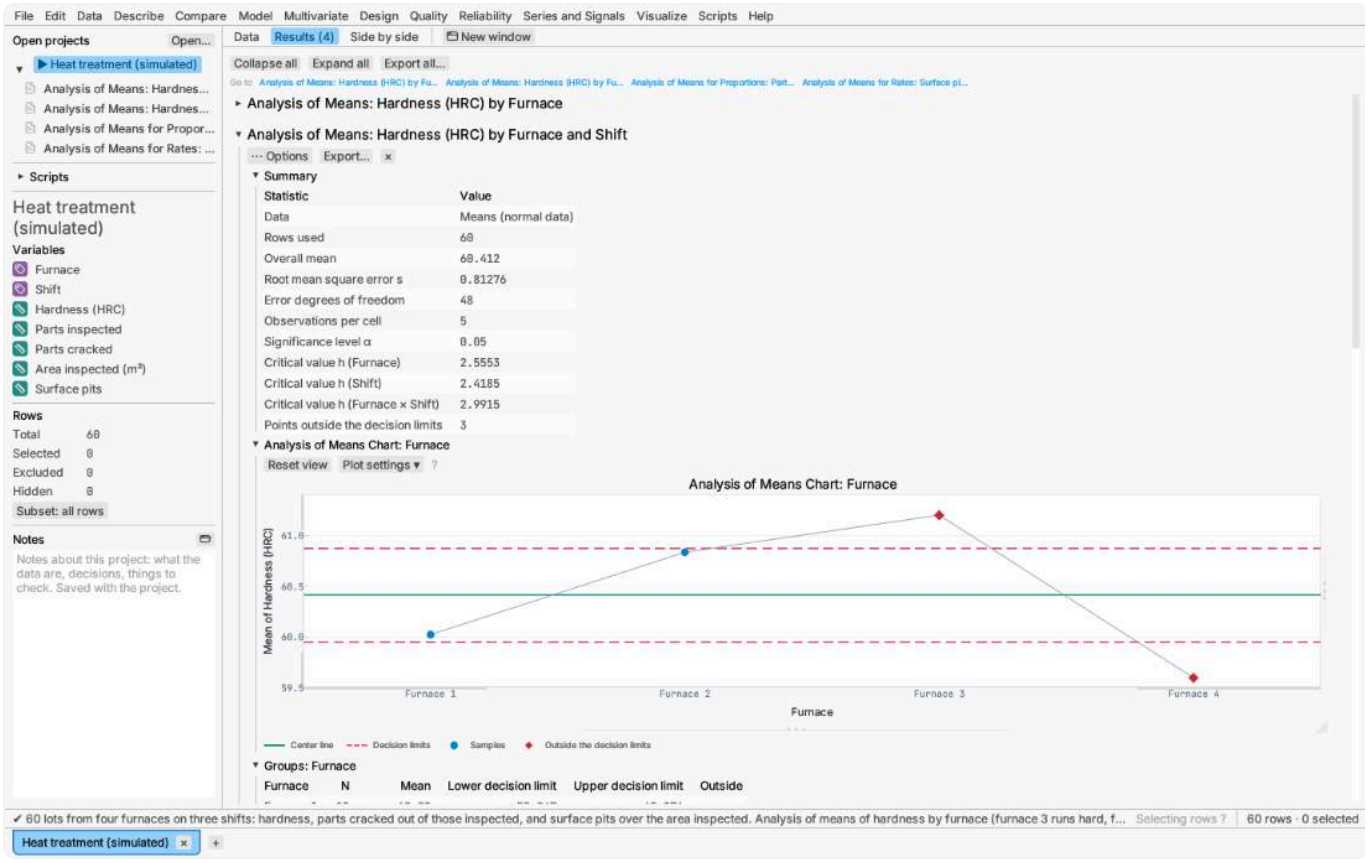
Quality ▸ Analysis of means... compares each group with the overall value, instead of comparing groups with each other: the chart shows each group's mean, proportion or rate, the overall value as a center line, and decision limits around it (Ott 1967). A group outside the limits differs from the overall value at significance level α (0.05 by default); it is drawn as a diamond as well as in colour and listed in the group table. Each column can fill only one role in the dialog: **Run** stays unavailable, with a note saying so, while a column is chosen twice.

- **Means (normal data):** a response column and a grouping column. The limits use the pooled within-group standard deviation and an exact critical value from the multivariate t distribution of the group deviations, for equal or unequal group sizes (Nelson 1982); with unequal sizes each group has its own limits. Add a **second factor** for a balanced two-way layout: one chart per factor's main effects and, when every cell has two or more rows, a chart of the interaction effects (cell mean minus row and column means plus the grand mean) whose limits use the studentized maximum modulus (exact for a 2 × 2 layout, conservative otherwise).

- **Proportions (binomial counts):** a column of events (for example parts cracked) and the trials behind them (parts inspected), from a column or the same for every row.
- **Rates (Poisson counts):** a column of counts and the exposure (area, time or units) behind them.

For proportions and rates the rows of each group are pooled; with no grouping column every row is its own group. These limits use the normal approximation, and the report notes the groups where fewer than five events (or non-events) are expected, as the approximation is rough there.

Try it: Heat treatment (hardness by furnace, by furnace and shift, the cracked proportion by furnace and the pit rate by shift).



Process capability

Quality ▸ Process capability… compares a process with its specification limits:

- within-subgroup indices **Cp, Cpk, Cpl, Cpu**; overall indices **Pp, Ppk, Ppl, Ppu**; and **Cpm**;

- confidence intervals for the indices: the χ^2 interval for Cp and Pp and Bissell's (1990) interval for Cpk and Ppk. Their width depends on the degrees of freedom of the σ estimate: n − 1 for the overall indices; for the within indices k(m − 1) with the pooled standard deviation of k subgroups of m, and an effective (Patnaik) value for the range estimators, about 0.9·k(m − 1) for the average range and about 0.6·(n − 1) for the moving range, because ranges carry less information than standard deviations. The within intervals are approximations;
- observed and expected nonconforming rates;
- a histogram with the specification limits and the fitted normal curve.

Also by group of a column (lines, machines, suppliers, shifts) adds two sections after the indices. **Capability by …** lists each group's N, mean, overall SD, Cp, Cpk, Pp, Ppk and the observed share of values outside the limits, from the same study run on that group's rows alone (in row order, so the within σ uses that group's own subgroups or moving ranges); with a non-normal model it adds each group's percentile-based Ppk. A group too small for the chosen within-σ method shows — and a note saying why. **Ppk by …** plots each group's Ppk as a dot on a stem from zero, with reference lines at 1 and 1.33; choose **Order the Ppk chart** by group label, smallest Ppk first or largest first under More options or in the report's options.

Probability plot with the limits (More options, or the report's options) adds a normal probability plot (or a quantile plot against the chosen distribution) with LSL, target and USL drawn across it, so it shows where the fitted line crosses each limit. The summary page always has it.

The report's options also offer **Save in-specification indicator**, a formula column that is 1 when a value is within the limits, 0 outside and missing without a value; as a formula it follows later edits.

Try it: Fill weights, by Filler setting: the setting B period sits nearer the upper limit and has the lower Ppk.

Non-normal processes

The normal-theory indices assume the process is normal: for skewed data they misstate the tails, and so the nonconforming rates, often by orders of magnitude. Choose another **Process distribution** in the dialog:

- **Normal after Box–Cox:** transforms the data and limits by a power λ and computes the usual indices on that scale.

- **A distribution** (lognormal, Weibull, gamma, … with or without a threshold), or **Best fit by AIC** among all fourteen.
- **Johnson transformation:** a transformation to normality chosen from the data's percentiles.
- **Nonparametric:** the sample's own percentiles. These need many values (about 740 before the outer percentiles leave the sample extremes); the expected rates are the observed ones.

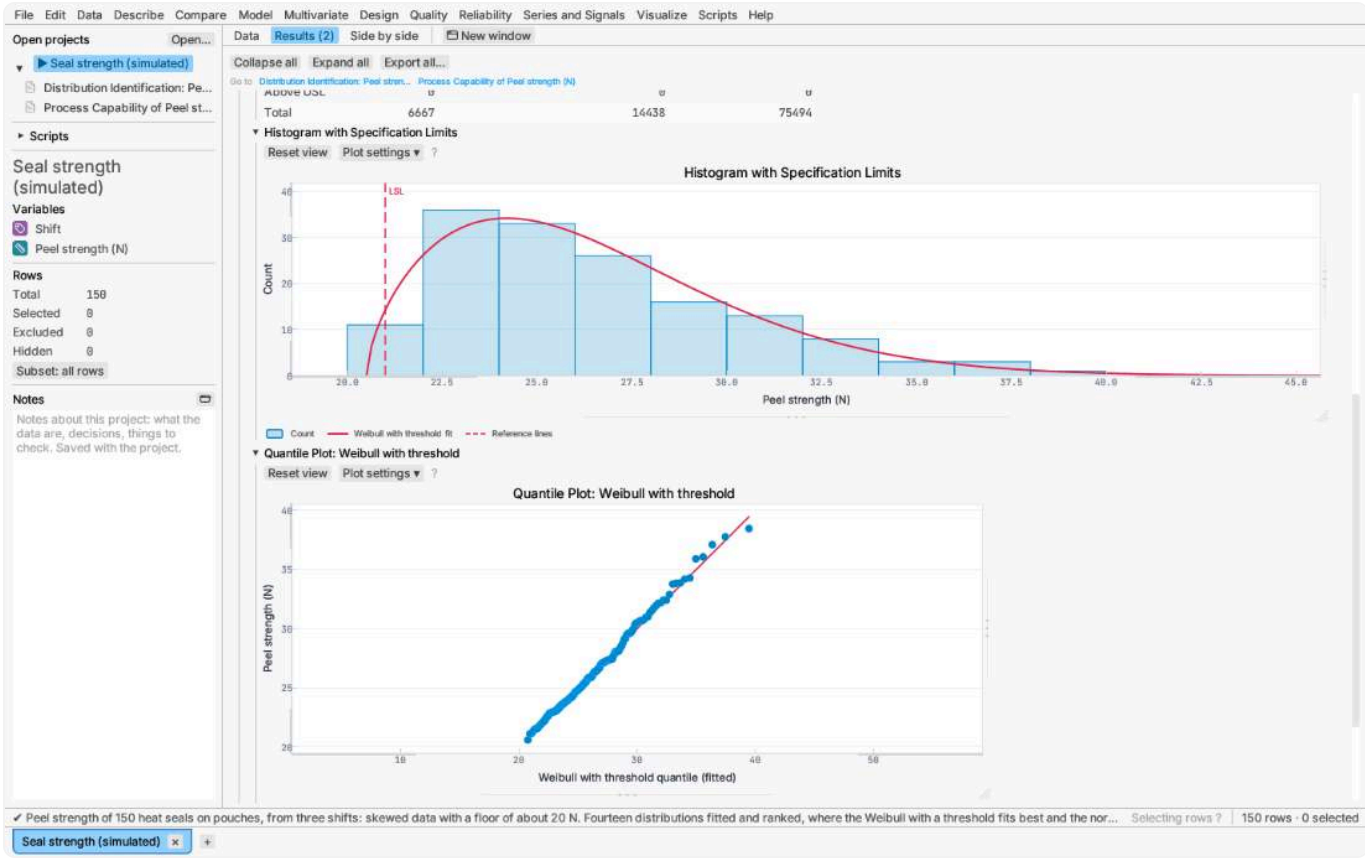
The indices then use percentiles of the fitted distribution in place of mean ± 3σ (ISO 22514-2, Clements 1989): with X0.135, X50 and X99.865 the 0.135%, 50% and 99.865% points,

- Pp = (USL − LSL) / (X99.865 − X0.135);
- Ppl = (X50 − LSL) / (X50 − X0.135) and Ppu = (USL − X50) / (X99.865 − X50);
- Ppk = the smaller of Ppl and Ppu.

Ppk (equivalent) is the normal-theory Ppk that would give the same expected nonconforming fraction on the worse side. The report also shows the fitted distribution and its percentiles (with the AIC of every family for best fit), the expected rates under the fit beside the observed and normal ones, the fitted density over the histogram, and a quantile plot against the fit. The normal-theory indices follow at the end for comparison. Only overall (Pp-type) indices are given: a non-normal fit has no short-term σ.

Bootstrap intervals. Tick *Bootstrap intervals for the percentile indices* in the dialog (or the report's Options menu) to add percentile bootstrap intervals (Efron & Tibshirani 1993, ch. 13) for Pp, Ppl, Ppu and Ppk: the data are resampled with replacement, the model is refitted to each resample (best fit refits the chosen family), and the 2.5% and 97.5% points of the resampled indices form a 95% interval. Choose the number of **resamples** (default 1000, at least 20) and the **seed**; the same seed gives the same intervals every time. Resamples whose model cannot be fitted are skipped and counted in the table's Bootstrap row.

Try it: Seal strength, peel strength with a lower limit of 21 N and the Weibull with threshold.



Between/within capability

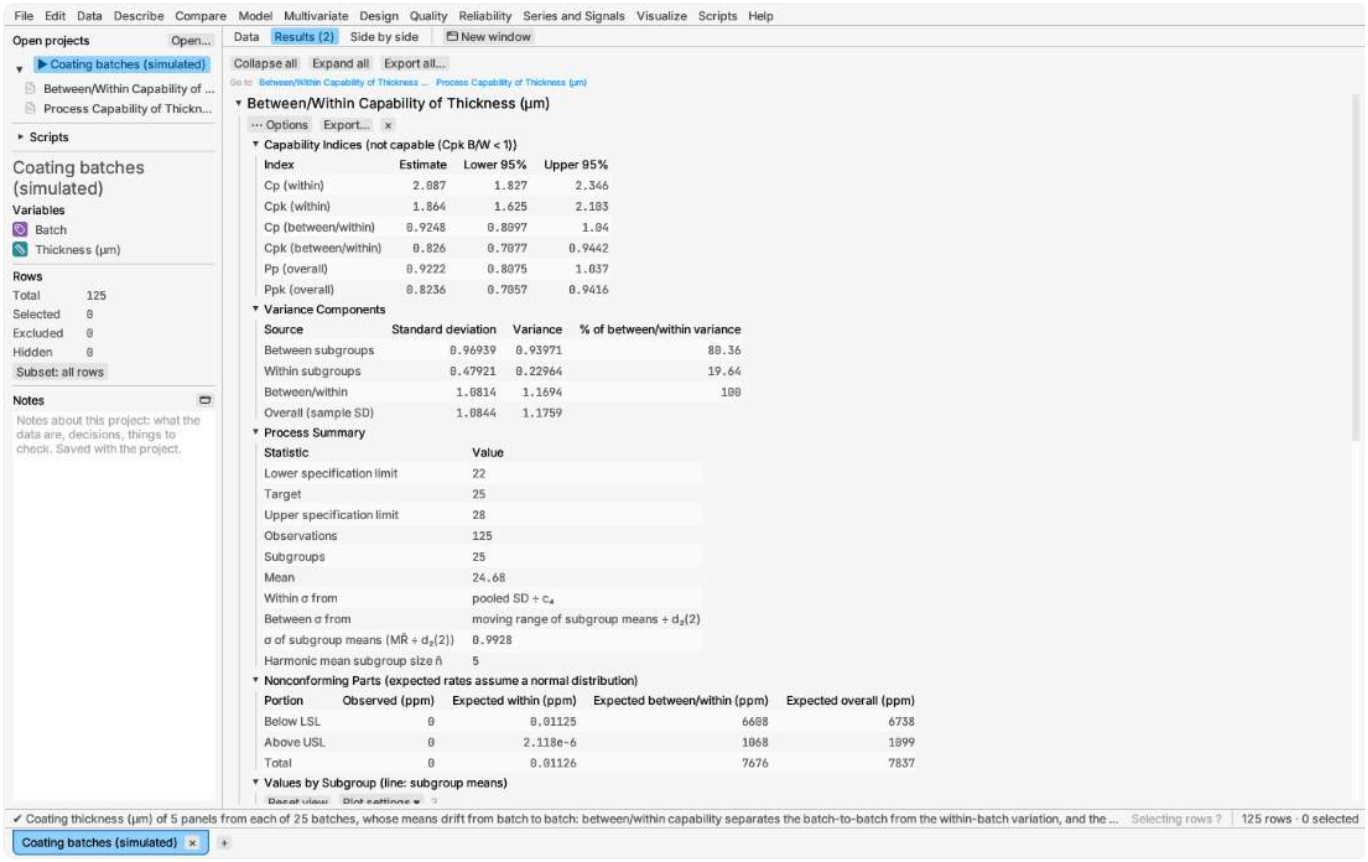
Quality ▸ Between/within capability… is for a process run in batches, or in parallel streams such as mould cavities or filling heads, whose means differ from one another. The ordinary within-subgroup σ ignores that batch-to-batch variation and makes the process look better than it is; the between/within σ adds it back.

Choose the measurement, then the subgroups: rows with the same value of a column (a batch or stream ID), or consecutive rows a fixed number at a time. Give at least one specification limit. The between-subgroup σ comes from the **moving range of subgroup means** (batches in time order), or from **variance components** of a one-way layout (streams, where order does not matter). The within σ is the pooled standard deviation or the average range.

The report shows Cp and Cpk within, Cp and Cpk between/within, and Pp and Ppk overall, with 95% intervals; a variance components table (between, within, between/within and overall σ); a process summary; observed and expected nonconforming parts; the values by subgroup with the subgroup means and the limits (points are linked to the data); and a histogram.

Options: the limits and both σ estimators, edited in place.

Try it: Coating batches. Compare the between/within report with the ordinary capability report opened beside it.



Attribute capability

Quality ▸ Attribute capability… estimates capability from counts, one sample per row. **Binomial** takes the number of defective units and the units inspected, and reports the percent defective and defective PPM with exact (Clopper–Pearson) intervals and the process Z, the normal deviate with the same tail area. **Poisson** takes the number of defects and the units inspected (a column or a fixed number, which need not be whole, but not the count column itself), and reports the defects per unit with an exact (Garwood) interval. The dialog lists only columns of whole, non-negative numbers as counts (and, for binomial data, as units inspected).

The estimate only describes a stable process, so the report includes the p chart (binomial) or u chart (Poisson; a c chart with no sizes) and counts the samples outside the limits. It

also plots the cumulative estimate, which should settle, and each sample's rate against its size, which should show no trend, and lists the samples.

Options: the model and the confidence level.

Try it: Supplier lots: defective units with units inspected (binomial), and cosmetic defects (Poisson).

Acceptance sampling plans

Quality ▸ Acceptance sampling plan... designs a lot-by-lot inspection plan. It uses no data columns. Enter the **producer's point**: the acceptable quality level p_1 and the risk α of rejecting such a lot. Then enter the **consumer's point**: the lot tolerance level p_2 and the risk β of accepting such a lot. Also give the lot size.

The plans it can design are:

- **Single attribute plan** (n, c): the plan with the smallest sample meeting both points.

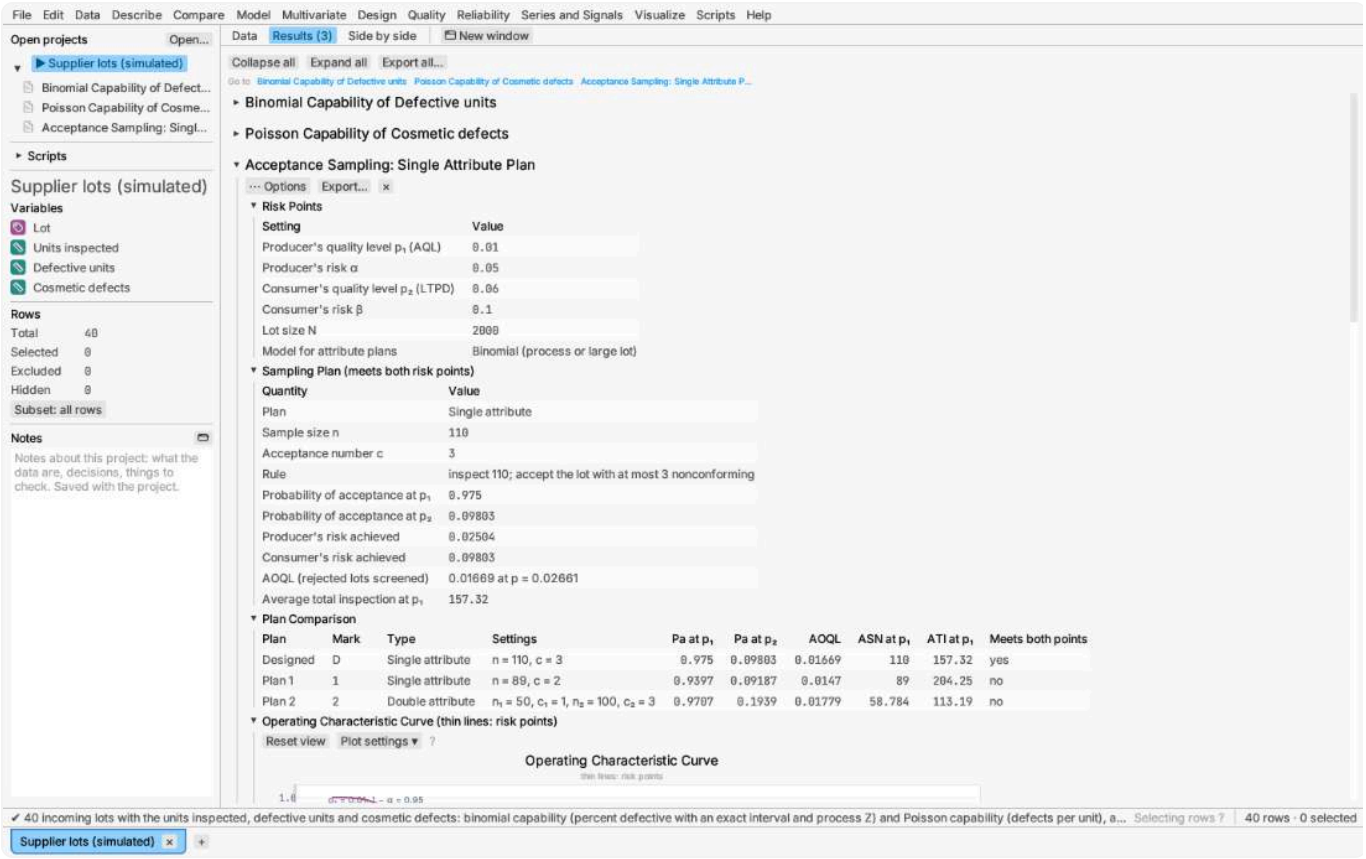
- **Double attribute plan** (n_1, c_1, n_2, c_2, with n_2 a chosen multiple of n_1): among the plans meeting both points, the one with the smallest average sample number at p_1.
- **Variables plan** (n, k) by the k-method, for a normally distributed measurement with one specification limit, with σ known or unknown.

Attribute plans are worked out with a binomial model (a continuing process or a large lot), a hypergeometric model (an isolated lot of the given size) or a Poisson model. Nothing is looked up in published sampling tables: every plan is computed from the risk points.

The report gives the plan and its decision rule, the risks it achieves and its AOQL, then the curves: operating characteristic (probability of acceptance, with the risk points marked), average outgoing quality with the AOQL (rejected lots screened and nonconforming items replaced), average total inspection and, for double plans, average sample number. Plans you enter under *Plans to compare* are drawn on the same plots. Each plan has its own labelled marker (D for the designed plan, 1, 2, … for yours), and a comparison table says which plans meet both points. Choose *None* as the design to compare entered plans only.

Options: every setting, edited in place.

Try it: Supplier lots: the 1%/6% plan compared with n = 89, c = 2 (which misses the producer's point) and a double plan.

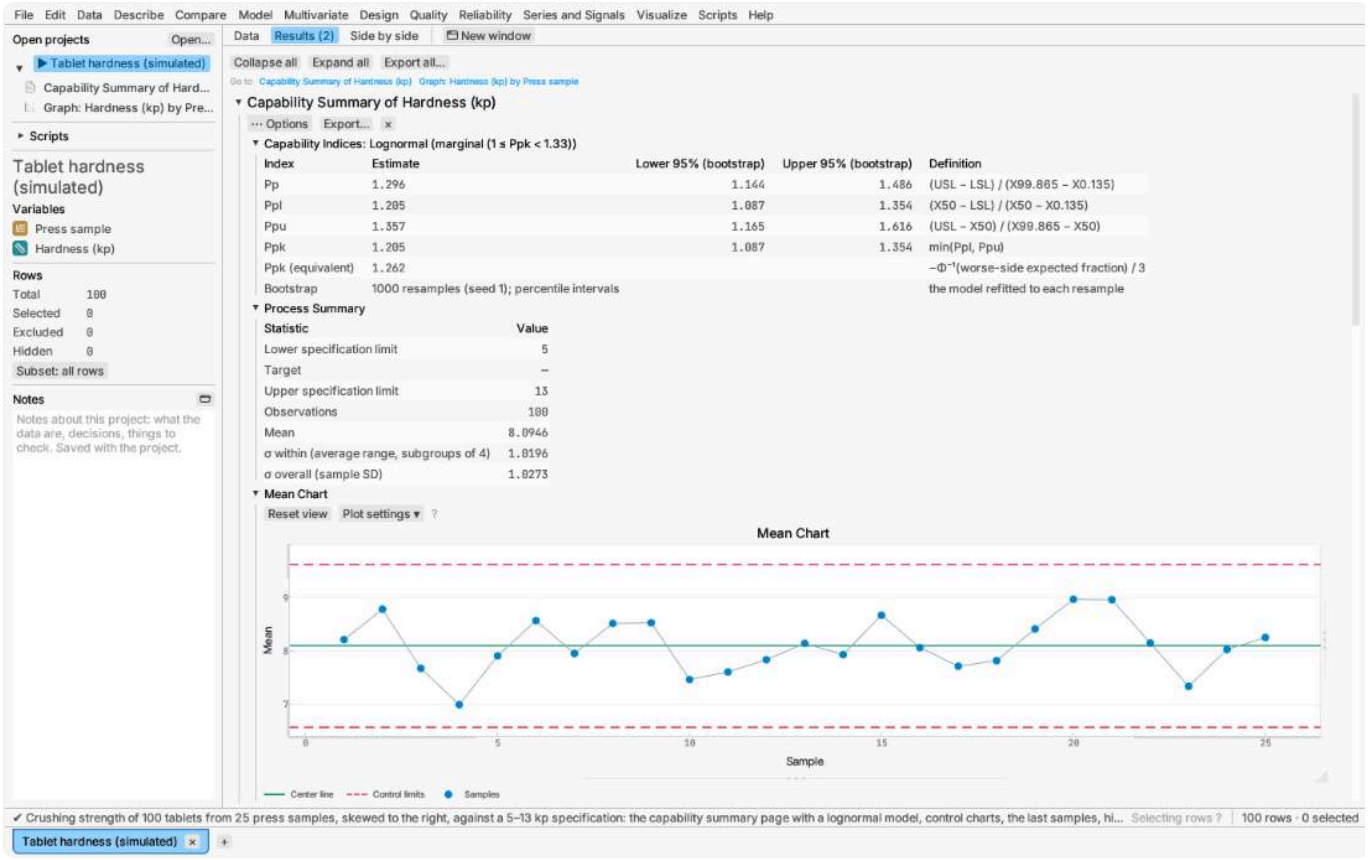


Capability summary page

Quality ▸ Capability summary page… opens the same dialog with **Summary page layout** ticked (it can also be switched on in any capability report's Options menu). The report then gathers on one page:

- the capability indices and the process summary;
- a **location chart** and a **dispersion chart** of the data, subgrouped as for the within σ (individuals and moving range, or $\bar{X}$ with R or S for consecutive subgroups), with all run rules on the location chart and rule 1 on the dispersion chart;
- the **last 25 subgroups** (or values) against the specification limits and target: every value, with a line through the subgroup means;
- the **histogram** with the limits and the fitted density;
- a **probability plot** against the chosen model (normal, or the fitted distribution).

Try it: Tablet hardness (a lognormal model, summary page and bootstrap intervals).



Gauge R&R

Quality ▸ Gauge R&R… runs a measurement systems study and reports the variance components: repeatability, reproducibility and part-to-part variation, with %contribution, %study variation, %tolerance (when you enter a tolerance) and the number of distinct categories. The data are one row per measurement, with a part column, an operator column and a numeric measurement column. Choose the **study type** at the top of the dialog:

- **Crossed** — every operator measures every part the same number of times. Analysed by the ANOVA method with a part × operator interaction, which can be pooled into repeatability when it is not significant.
- **Nested** — for destructive tests, where a part cannot go to a second operator: each operator measures their own parts (in practice, pieces cut from one batch stand in for repeat measurements). The ANOVA has operators, parts within operators and repeatability; there is no interaction, and reproducibility is the operator component. Parts may be numbered 1, 2, 3… within each operator: in a nested study a part is its label together with its operator. Every operator must test the same number of parts, each equally often.
- **Expanded** — for studies with more factors than parts and operators, or with readings missing. Add extra factor columns (a second gauge, a fixture, a location, a day) with **Add factor**, and give each one a role — *random* (its levels are a sample, such as gauges drawn from the plant's stock, and it gets a variance component) or *fixed* (the levels are the ones of interest, and it is tested for a shift in the mean) — and a place: crossed with parts and operators, nested within parts, or nested within operators. Tick *Parts nested within operators* for a destructive test, and *Two-factor interactions* to include every interaction of factors that are not nested in one another (part × operator, part × gauge, operator × gauge…). The model is fitted by restricted maximum likelihood (REML), so unbalanced data and empty cells are fine. Gauge R&R totals are built from the random components: the part term is part-to-part variation, the residual is repeatability, and every other random component counts as reproducibility — except a random factor nested within parts (a location or position on the part): that *within-part* variation is a property of the product, not of the measurement system (AIAG *Measurement Systems Analysis*; Burdick, Borror & Montgomery 2005), so the Variance Components table lists it under *Part-to-part* (as *Between parts* and *Within part*), it counts in part-to-part variation, its %study variation and the number of distinct categories, and stays out of gauge R&R. Its interactions with operators or gauges are still reproducibility. Fixed factors do not enter the totals; a *Tests of Fixed Factors* table gives their F tests. When missing cells leave only some of a fixed term's effects estimable, the study summary notes that its F test covers only

those. *Variance Component Intervals* lists each component and total with its standard error and 95% confidence limits (Satterthwaite's χ^2 limits, which stay positive). Terms the data cannot separate from repeatability or from another term are left out and listed in the study summary. For balanced data the expanded study gives the same estimates as the crossed or nested ANOVA whenever those are all positive.

Run order (optional): a numeric column (run number or time stamp) giving the order in which the measurements were taken, used by the gauge run chart; without one the row order is used.

The **Gauge Run Chart** shows each part in its own pane, with that part's measurements in run order, coloured and shaped by operator, and a thin line at the part's mean. Scatter within a pane is repeatability, operators sitting apart is reproducibility, and a trend across panes or within a pane suggests drift.

Options: the ANOVA table (crossed and nested), measurements by part and by operator, the gauge run chart, pooling the part × operator interaction when it is not significant (crossed), and two-factor interactions (expanded). In the by-operator chart of a nested study each operator's pane holds only that operator's parts.

Try it: Gauge study (crossed), Weld pull test (nested), Bore gauges (expanded, with seven readings lost).

File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help

Open projects Open...

▶ Weld pull test (simulated)

Nested gauge R&R: Strength

▸ Scripts

Weld pull test (simulated)

Variables

- Run
- Operator
- Part
- Coupon
- Strength

Rows

Total	45
Selected	0
Excluded	0
Hidden	0

Subset: all rows

Notes

Notes about this project: what the data are, decisions, things to check. Saved with the project.

Data | Results (1) | Side by side | New window

Collapse all | Expand all | Export all...

Nested gauge R&R: Strength

⋯ Options | Export... | ×

Variance Components

Source	Variance	% Contribution	Std Dev	Study Var (6σ)	% Study Var	% Tolerance
Total gauge R&R	93.7145	42.62	9.68063	58.0838	65.28	72.60
Repeatability	20.5662	9.35	4.535	27.21	30.58	34.01
Reproducibility	73.1483	33.26	8.55268	51.3161	57.68	64.15
Operator	73.1483	33.26	8.55268	51.3161	57.68	64.15
Part-to-part (within operator)	126.187	57.38	11.2333	67.3999	75.75	84.25
Total variation	219.982	100.00	14.8291	88.9745	100.00	111.22

Study Summary

Item	Value
Design	nested: each part measured by one operator
Parts (Part)	15
Operators (Operator)	3
Parts per operator	5
Measurements per part	3
Number of distinct categories	1
Tolerance (USL − LSL)	80

Analysis of Variance

Source	DF	Sum of Squares	Mean Square	F	p-value
Operator (Operator)	2	2992.71	1496.35	3.749	0.0543
Part (Part) within operator	12	4789.54	399.128	19.41	★ 6.4e-11
Repeatability	30	616.987	20.5662		
Total	44	8399.23			

Gauge Run Chart

Reset view | Plot settings ▾ | ?

✔ A destructive test: three operators each pull three coupons from five batches of their own (batches numbered 1–5 within each operator). Nested gauge R&R with parts within operators, the gauge run c... Selecting rows ? | 45 rows · 0 selected

Weld pull test (simulated) × +

File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help

Open projects Open...

▶ Bore gauges (simulated)

Expanded gauge R&R: Diame...

▸ Scripts

Bore gauges (simulated)

Variables

- Run
- Part
- Operator
- Gauge
- Diameter

Rows

Total	89
Selected	0
Excluded	0
Hidden	0

Subset: all rows

Notes

Notes about this project: what the data are, decisions, things to check. Saved with the project.

Data | Results (1) | Side by side | New window

Collapse all | Expand all | Export all...

Expanded gauge R&R: Diameter

⋯ Options | Export... | ×

Variance Components

Source	Variance	% Contribution	Std Dev	Study Var (6σ)	% Study Var	% Tolerance
Total gauge R&R	6.34947e-5	28.74	0.00796836	0.0478101	53.61	47.81
Repeatability	8.26341e-6	3.74	0.00287462	0.0172477	19.34	17.25
Reproducibility	5.52313e-5	25.00	0.00743178	0.0445907	50.00	44.59
Operator	3.02439e-5	13.69	0.00549944	0.0329967	37.00	33.00
Gauge	2.42889e-5	10.99	0.00492757	0.0295654	33.15	29.57
Part × Operator	5.03563e-7	0.23	0.000709622	0.00425773	4.77	4.26
Part × Gauge	1.94147e-7	0.09	0.000440621	0.00264373	2.96	2.64
Operator × Gauge	8.77372e-9	0.00	9.36681e-5	0.000562009	0.63	0.56
Part-to-part	0.000157434	71.26	0.0125473	0.0752835	84.42	75.28
Total variation	0.000220928	100.00	0.0148637	0.0891819	100.00	89.18

Variance Component Intervals

Source	Levels	Variance	Std Error	Lower 95%	Upper 95%
Part	8	0.000157434	8.46981e-5	6.85359e-5	0.000660193
Operator	3	3.02439e-5	3.05888e-5	8.11902e-6	0.00128436
Gauge	2	2.42889e-5	3.46363e-5	4.79521e-6	0.0277676
Part × Operator	24	5.03563e-7	1.10222e-6	6.78566e-8	7.58526
Part × Gauge	16	1.94147e-7	9.49519e-7	1.69557e-8	2.92073e30
Operator × Gauge	6	8.77372e-9	5.61462e-7	–	–
Repeatability	89	8.26341e-6	1.57055e-6	5.87949e-6	1.24677e-5
Total gauge R&R	–	6.34947e-5	4.62429e-5	2.22865e-5	0.000576809
Reproducibility	–	5.52313e-5	4.62582e-5	1.73668e-5	0.000855175
Part-to-part	–	0.000157434	8.46981e-5	6.85359e-5	0.000660193
Total variation	–	0.000220928	9.65036e-5	0.000109346	0.000658159

Study Summary

Item	Value
Design	expanded, fitted by REML
Parts (Part)	8
Operators (Operator)	3
Parts nested within operators	no
Factor Gauge	2 levels, random, crossed
Two-factor interactions	included

✔ Eight parts measured twice by three operators on each of two bore gauges, with seven readings lost. Expanded gauge R&R by REML with the gauge as a random factor and all two-factor interactions, ... Selecting rows ? | 89 rows · 0 selected

Bore gauges (simulated) × +

To plan a study, Quality ▸ Gauge study worksheet… creates a randomized run sheet as a new table; see Design.

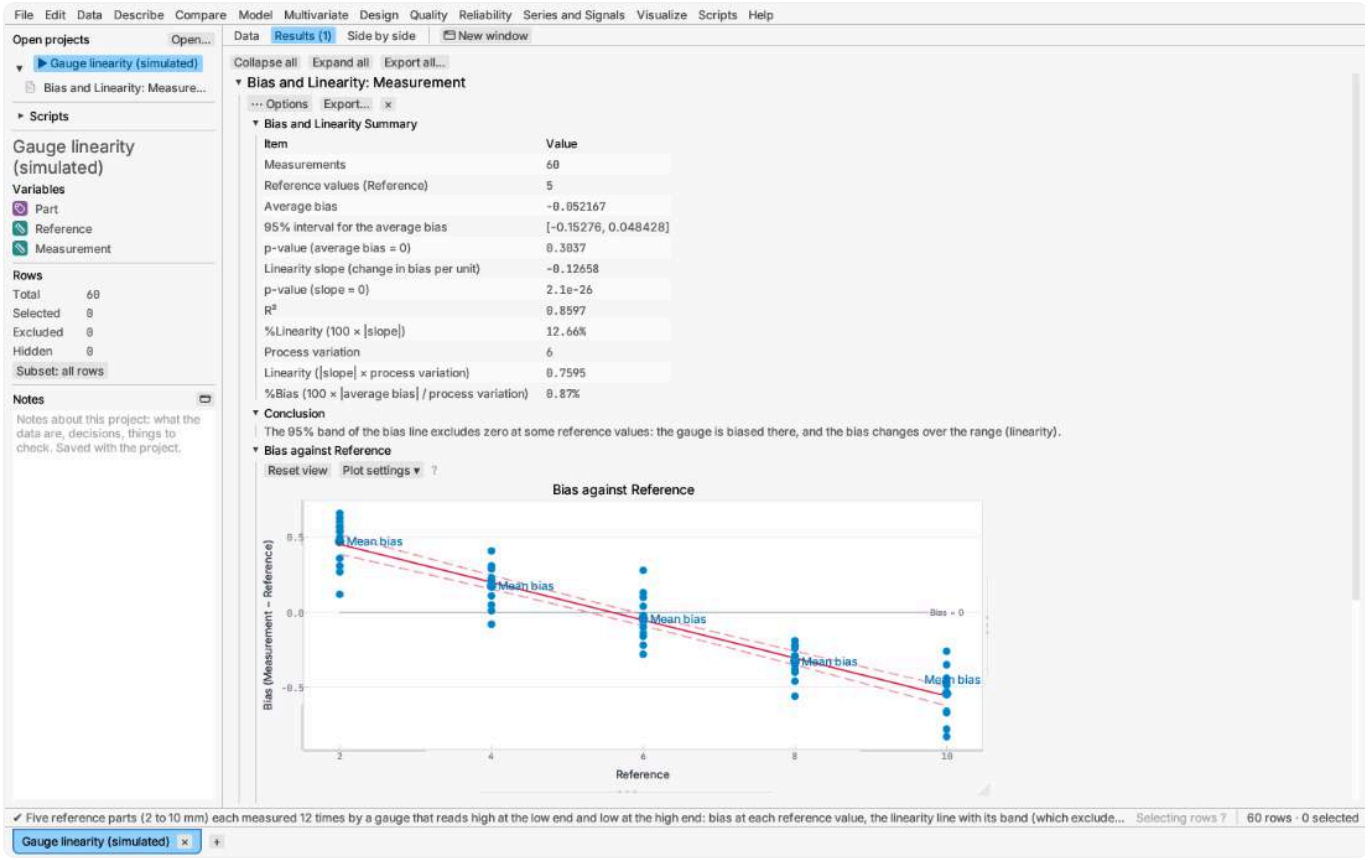
Bias and linearity

Quality ▸ Bias and linearity… checks a gauge's accuracy over its range. Measure several reference parts of known value repeatedly, then choose the reference-value column and the measurement column. Following the AIAG measurement systems analysis method, each measurement's bias (measured minus reference) is regressed on the reference value. The report shows:

- **Summary:** the average bias with its confidence interval and p-value, the slope of the bias line (the change in bias per unit of reference) with its p-value, R^2, and %linearity = 100 × |slope|. With a process variation (for example 6σ of the process) entered, also linearity = |slope| × process variation and %bias = 100 × |average bias| / process variation.
- **Conclusion:** whether zero bias lies inside the confidence band across the whole range (the gauge is acceptably linear and unbiased), or where it does not.
- **Bias plot:** each bias and the mean bias at each reference value, the fitted line with its confidence band, and the zero-bias line.
- **Linearity line:** slope and intercept with standard errors, t tests and confidence intervals.
- **Bias by reference value:** for each reference value, the number of readings, mean bias, standard deviation, t and p.

The confidence level and the process variation can be changed in place.

Try it: Gauge linearity.



Gauge repeatability (Cg, Cgk)

Quality ▸ Gauge repeatability (Cg, Cgk)… evaluates one gauge by measuring one master part of known reference value many times (Dietrich & Schulze, *Measurement Process Qualification*). Choose the measurement column and enter the reference value and the tolerance T of the feature the gauge will check. With s the standard deviation of the readings and bias their mean minus the reference:

- Cg = (K/100 · T) / (L · s)
- Cgk = (K/200 · T − |bias|) / (L/2 · s)

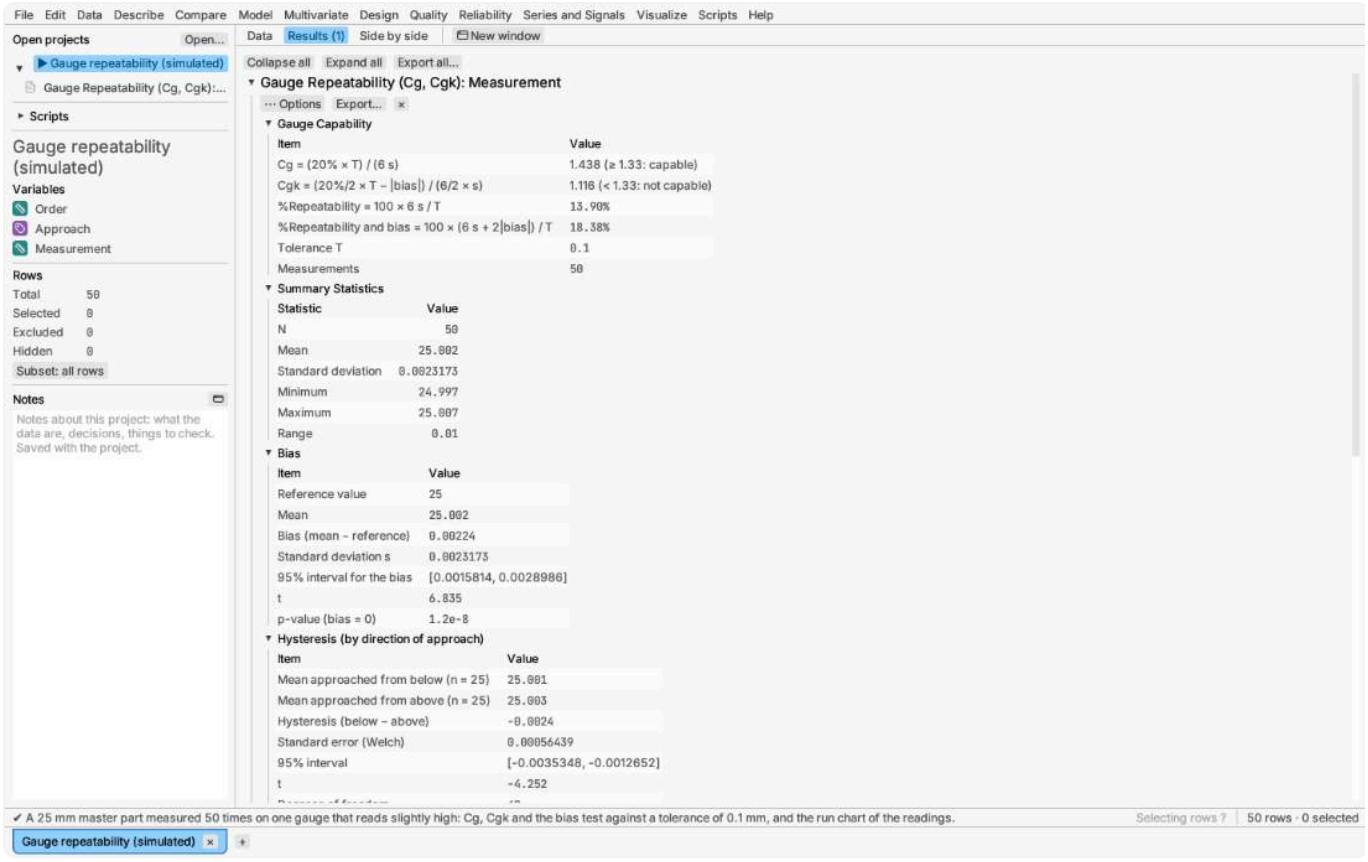
K, the share of the tolerance the gauge may use, is 20% and L, the spread of the readings, is 6 s by default; both can be changed. A gauge is usually called capable when Cg and Cgk are both at least 1.33; the report marks each index against that value with a word as well as a colour. It also gives %repeatability (100 L s / T) and %repeatability with bias (100 (L s + 2 |bias|) / T), a t test of the bias with its confidence interval, and a run chart of the readings with the reference value and the reference ± K/200 · T limits.

Two optional columns add sections:

- **Direction of approach** (a column with two values, such as "below" and "above"): a **Hysteresis** section with each direction's N, mean and standard deviation and the hysteresis h = $\bar{y}_1 - \bar{y}_2$ (the first value in row order minus the second), with a Welch t test of h = 0 and its confidence interval. Readings that approach the reference from opposite sides should agree; a gauge with backlash or friction gives h ≠ 0.
- **Time or order** (numeric; by default the row order 1, 2, …): a **Drift** section fits reading = a + c · time by least squares and reports the slope c with its standard error, t test of c = 0 and interval, and the total change c × (last − first time) over the study. Uncheck **Drift over time or order** under More options to leave it out.

The **Summary Statistics** section lists N, mean, standard deviation, minimum, maximum and range by default; under More options (or the report's options) choose any of N, mean, standard deviation, standard error of the mean, minimum, median, maximum, range, 6 × standard deviation and the coefficient of variation.

Try it: Gauge repeatability. Its Approach column alternates readings from below and above, which differ by about 0.0015 mm, and the readings drift slowly upward over the run.



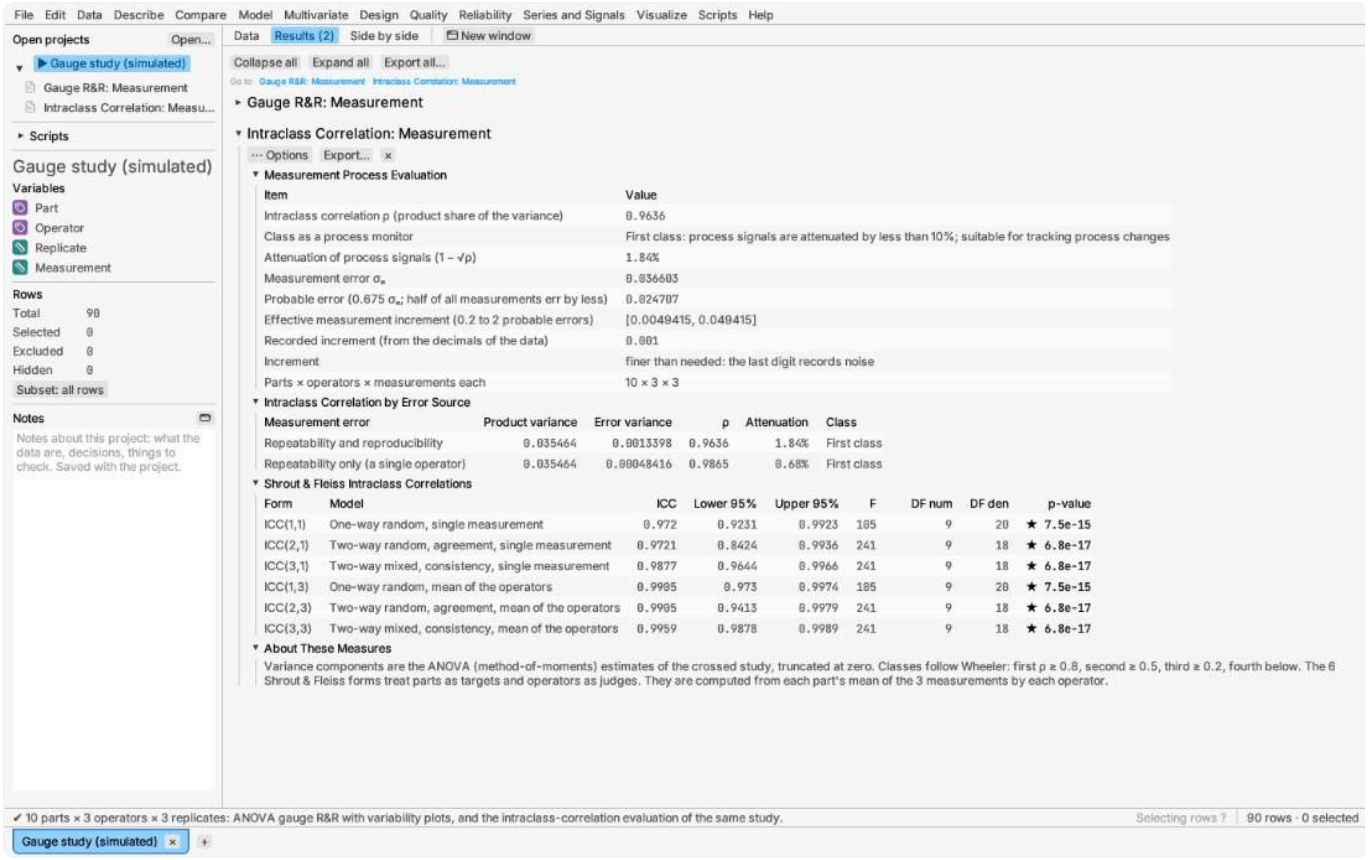
Intraclass correlation

Quality ▸ Intraclass correlation… asks how well a measurement separates the parts, using a crossed study (every operator measures every part, optionally with replicates). Choose the part, operator and measurement columns. The report shows:

- **Evaluation (Wheeler & Lyday, *Evaluating the Measurement Process*):** the intraclass correlation ρ — the share of the measurements' variance that comes from the parts — and the monitor class it puts the measurement in: first class (ρ ≥ 0.8; signals are attenuated by less than 10%), second (≥ 0.5), third (≥ 0.2) or fourth (below 0.2). With attenuation $1 - \sqrt{\rho}$, the probable error $0.675\ \sigma_e$ (half of all measurements are within this of the part's value) and the smallest and largest useful measurement increments (0.2 and 2 probable errors). The increment of the recorded data, read from its decimals or entered, is compared with them.
- **By source of error:** the same evaluation with repeatability alone and with repeatability and reproducibility combined (from the gauge R&R variance components when there are replicates).
- **Shrout & Fleiss forms:** the six forms ICC(1,1), ICC(2,1), ICC(3,1) and their averages over k operators, ICC(1,k), ICC(2,k), ICC(3,k), each with a confidence interval (Shrout & Fleiss 1979, computed on the operator–part cell means).

The confidence level and the measurement increment can be changed in place.

Try it: Gauge study (it opens a gauge R&R and an intraclass correlation report).



Attribute agreement

Quality ▸ Attribute agreement… studies a measurement that sorts parts into categories (pass/fail, a defect type, a 1–5 grade) rather than measuring them. Several appraisers rate the same parts, usually more than once, and ideally the right category of every part (the standard) is known.

The data have **one row per rating**: a part column, an appraiser column and the rating. Two columns are optional:

- **Trial** — which repeat a rating is. Without it, an appraiser's ratings of a part are taken as trials in the order of the rows.
- **Standard** — the part's known category, repeated on each of its rows (empty cells are ignored, but a part may not have two different values).

Every appraiser must rate every part the same number of times; excluded rows count against this. Tick **Ordered categories** when the categories have an order (grades, severity levels); a numeric or ordinal rating column starts ticked. The order of the categories is shown lowest first and can be changed with the arrow buttons. The categories analysed are

those of the ratings and standards in the rows used, so a category found only in excluded rows is left out (it would otherwise shift the weighted kappas).

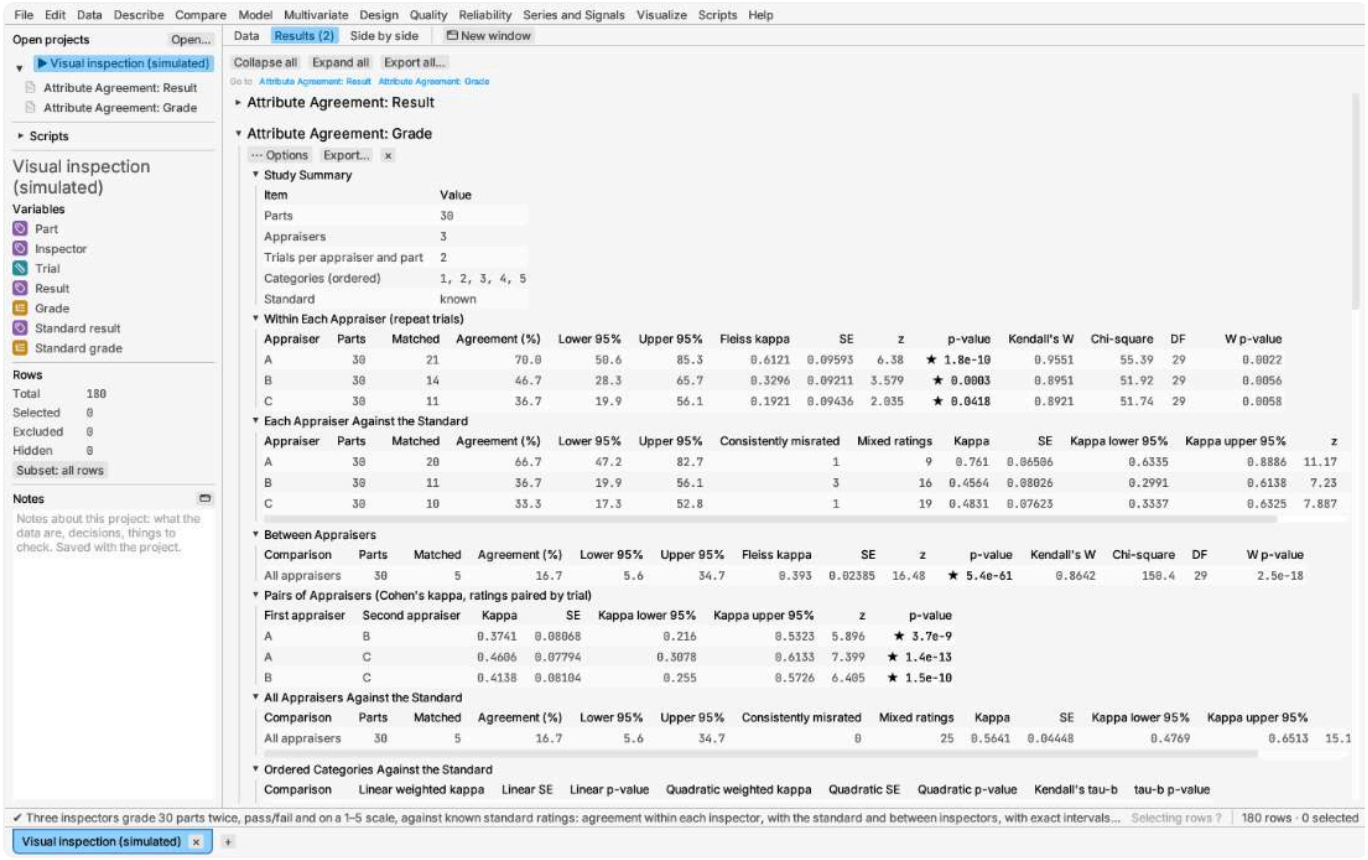
The report answers four questions, each with the percentage of parts on which all the ratings compared agree, and its exact (Clopper–Pearson) confidence interval:

- **Within each appraiser:** does the appraiser give a part the same rating every trial? With Fleiss' kappa (the trials as the ratings), and for ordered categories Kendall's coefficient of concordance W across trials.
- **Each appraiser against the standard:** are all the appraiser's ratings of a part right? The table also counts parts *consistently misrated* (the same wrong rating every trial) and parts with *mixed ratings*, and gives Cohen's kappa of the ratings against the standard (every trial a pair) with its standard error, interval and z test.
- **Between appraisers:** do all appraisers give a part the same rating on every trial? With Fleiss' kappa over all the ratings of a part, Kendall's W for ordered categories, and Cohen's kappa for every pair of appraisers (ratings paired by trial).
- **All appraisers against the standard:** is every rating of a part right?

For ordered categories, **Ordered Categories Against the Standard** adds linear and quadratic weighted kappas, which give partial credit to near misses, and Kendall's τ_b between the ratings and the standard. **Kappa by Category** gives each category's kappa in every comparison (a category against all the others), showing which categories are confused. **Ratings Against the Standard** counts every rating by the standard category. The **agreement plot** shows each appraiser's percentage of parts matched with its interval — within the appraiser and against the standard, each in its own colour and marker.

Kappa is 1 for perfect agreement and 0 for agreement no better than chance; it is shown as — when every rating falls in one category (chance agreement is then complete). The confidence level and the ordered-categories choice can be changed in place.

Try it: Visual inspection (it opens the pass/fail study and the 1–5 grade study, both against the standard).



Attribute gauge bias and repeatability

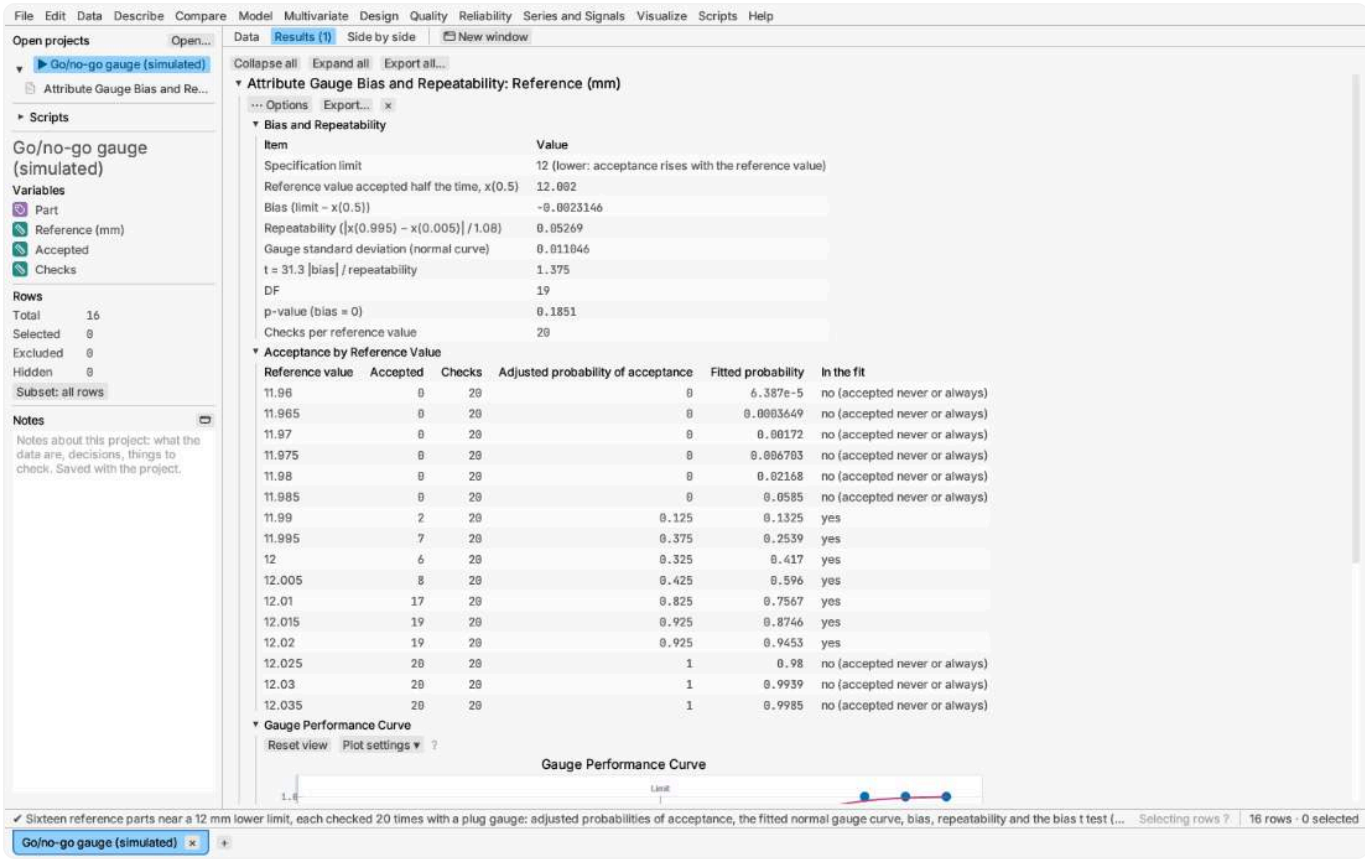
Quality ▸ Attribute gauge bias and repeatability… evaluates a go/no-go gauge by the analytic method of the AIAG *Measurement Systems Analysis* manual. Parts of known reference value spanning the region where the gauge changes from rejecting to accepting are each checked the same number of times (20 in the AIAG design). The data can hold the counts — a reference value, the times accepted and the times checked on each row — or one row per check with a result column; then choose the result that means accepted. Enter the specification limit the gauge decides at.

The probability of acceptance at each reference value is adjusted half a check away from 0 and 1, and the reference values accepted some but not all of the time are fitted with a normal curve (a straight line on the probit scale). From the curve:

- **Bias** — the limit minus the reference value accepted half the time;
- **Repeatability** — the distance between the reference values accepted 0.5% and 99.5% of the time, divided by AIAG's adjustment factor 1.08;
- **t test of bias = 0** — t = 31.3 · |bias| / repeatability on (checks − 1) degrees of freedom.

Acceptance that rises with the reference value means a lower limit; falling acceptance an upper limit. The report notes where the study departs from the recommended design (at least eight parts, six of them accepted some of the time, the extremes always and never accepted, 20 checks each). The limit can be changed in place.

Try it: Go/no-go gauge.



Pareto charts

Quality ▸ Pareto chart... ranks the categories of a column (defect types, complaint reasons, causes) by how often they occur, largest first, with a line of cumulative percentages (Juran's "vital few and useful many"). Each row counts once, or choose a **frequency** column whose values are added up (counts, costs or minutes lost). Pick a **grouping** column to draw one chart per level side by side (for example per line or shift), each ranked on its own, so you can see whether the same causes lead everywhere.

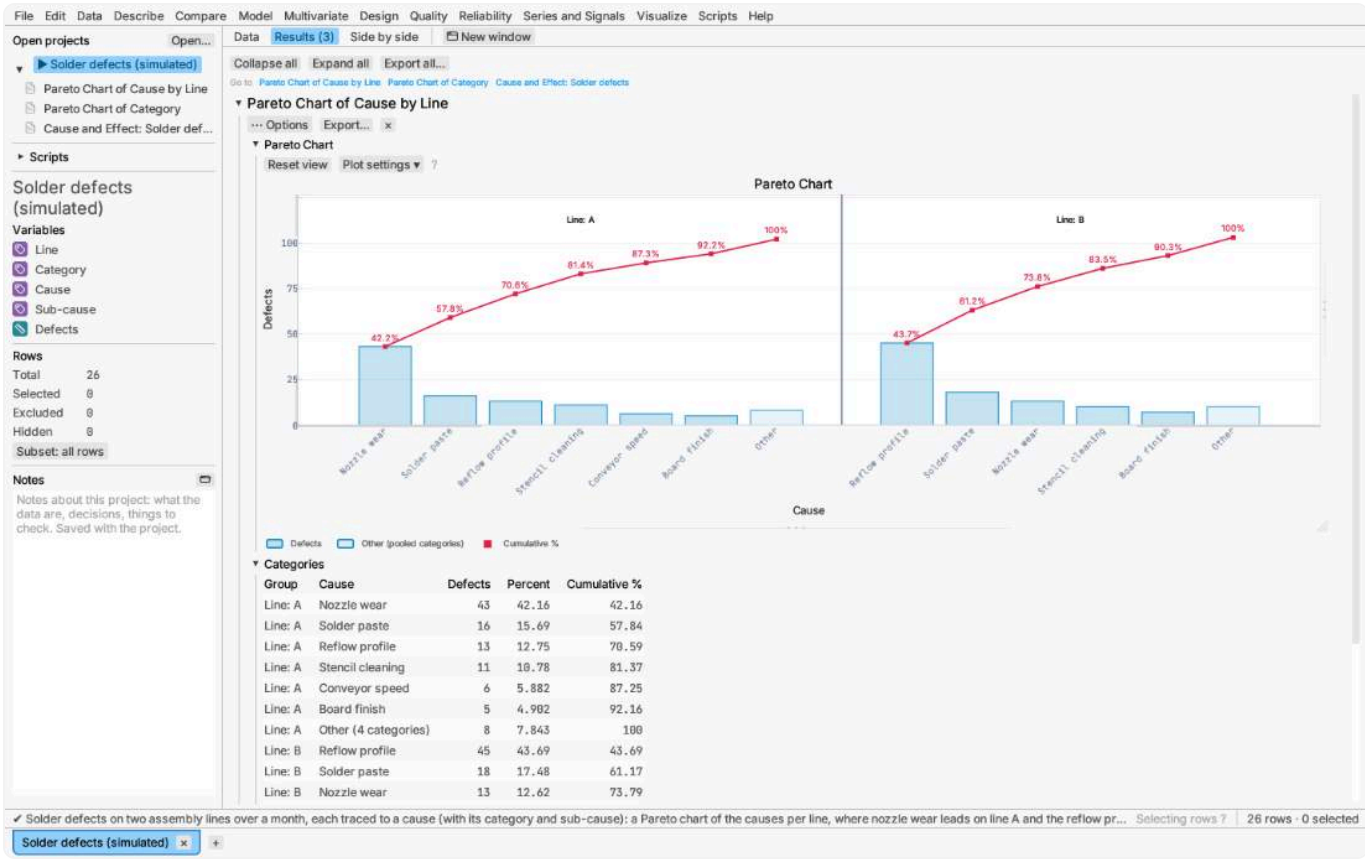
Each point of the cumulative line is labelled with its percentage. The report adds a table of counts, percentages and cumulative percentages, and a summary naming the first categories that together reach 80%. When that point falls in the **Other** bar, the summary

names it with the number of categories it pools (for example "Other (8 categories)") and counts each of them. Ties rank in natural order of the category names.

Every bar carries its rows: click a bar, or drag a box over several, to select their rows in the data and every other view. Selected rows show as a highlighted part of each bar (in proportion to their frequencies).

Options: pool the categories whose share is below a percentage you choose into one **Other** bar (drawn lighter, always last; at least two categories must qualify), and show bar heights as **percentages** of the total instead of counts. Both are also in the dialog.

Try it: Solder defects. The first report ranks causes by defects for each line: nozzle wear leads line A, the reflow profile line B, and the rarest causes pool into Other.



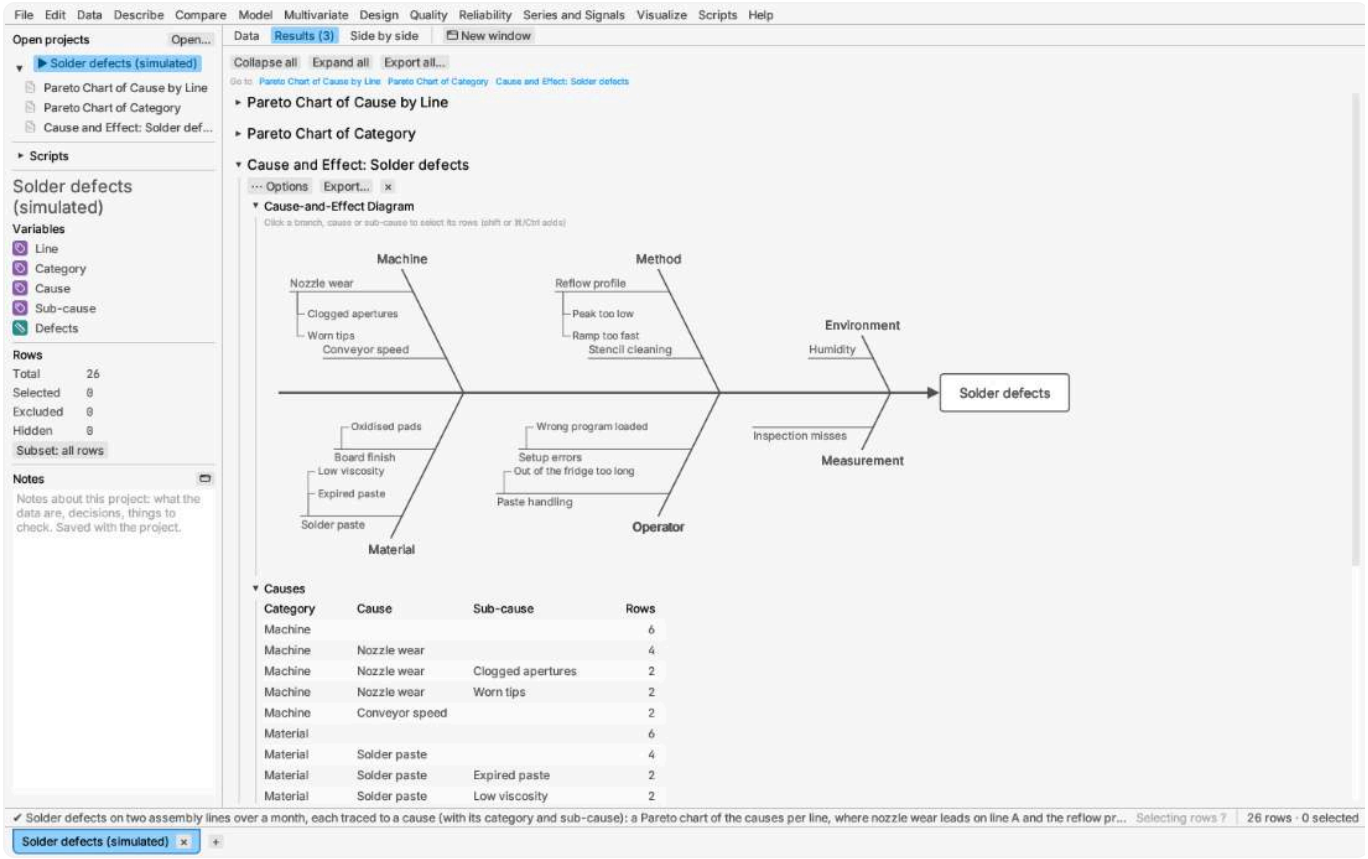
Cause-and-effect diagrams

Quality ▸ Cause-and-effect diagram... draws a fishbone diagram (Ishikawa) from columns of your table: the first column names the **main branches** (categories of causes, such as machine, method, material), the second the **causes** on each branch and the third,

optionally, their **sub-causes**. Each row is one path; repeated paths are merged and counted, and an empty cell ends the path. Branches and causes keep the order in which they first appear, so arrange the table the way you want the diagram read. Type the **effect** (the problem) for the head of the diagram.

Branches slant off the spine alternately above and below it, each cause on a short rib, its sub-causes indented under it. Click any branch, cause or sub-cause to select the rows behind it (shift or ⌘/Ctrl adds to the selection); labels with selected rows are tinted and marked with a dot. The **Causes** table lists every branch, cause and sub-cause with its number of rows. Right-click the diagram to save it as PNG, SVG, PDF or EPS.

Try it: Solder defects, the third report: six branches from the category, cause and sub-cause columns.



Multi-vari charts

Quality ▸ Multi-vari chart… shows how a response varies across up to four factors, outermost first (Seder 1950) — for example batch (time), shaft within batch (piece to piece)

and position on the shaft (within piece). Factors can be nested (shaft 1 of one batch is not shaft 1 of another) or crossed; only the combinations present in the data are drawn.

The outermost factor's levels become side-by-side panes. Each innermost combination (cell) gets a place along the horizontal axis, labelled with its levels. The chart shows:

- each **individual value** at its cell: by default a cell's values are spread a little to the left and right of the cell's place in their row order and joined by a thin line, so each cell reads as a short run chart and a drift or a jump within the cell shows (the classic form of the chart, Seder 1950; Snee 2001);
- the **means** of every factor's levels, each factor with its own colour and marker shape (named in the key);
- lines joining each factor's means within each level of the factor above it, and the outermost factor's means joined across the panes.

The **Variation by factor** table puts numbers on what the chart shows: for each factor, the range of its level means within each level of the factor above (averaged, and the largest), then the average range within the cells and the pooled within-cell standard deviation. The largest of these is the family of variation to chase first.

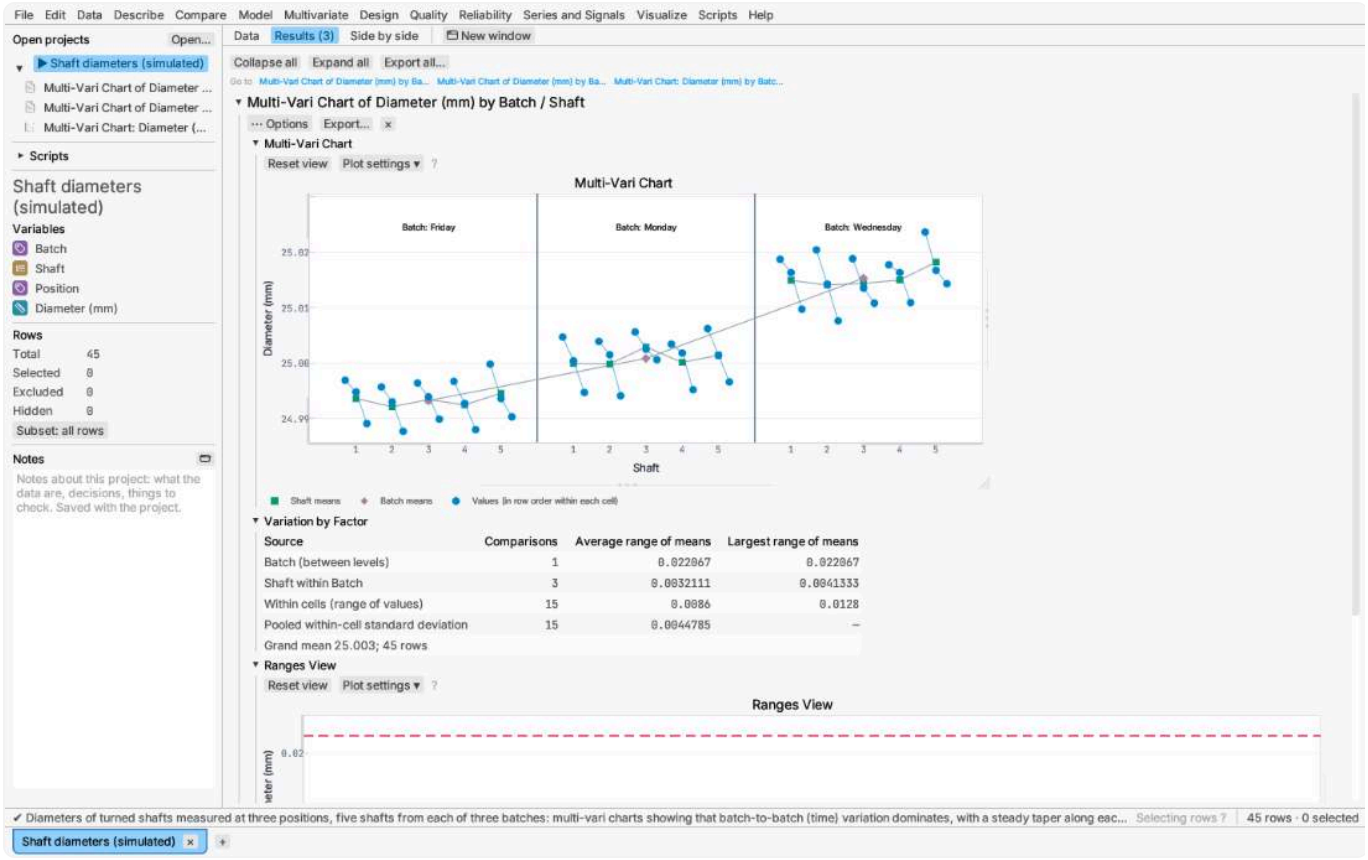
The **Ranges view** charts the range of the values within each cell, with 3-sigma limits from the cell sizes ($\bar{R}$, $D_3\bar{R}$ and $D_4\bar{R}$ when the cells are the same size); a cell beyond the upper limit is flagged. Cells with a single value have no range and are left out.

Values in the chart and cells in the ranges view are linked to their rows: box-select or click to select them everywhere.

Options: individual values, the ranges view, a cell summary table (count, mean, minimum, maximum, range and standard deviation per cell), and **Replicates**: *In order (side by side)*, the default, or *Stacked*, which draws a cell's values on one vertical line at the cell. The layout is the same in exported figures and report exports; pane headings sit above the highest value inside the frame, as on screen. With the individual values off, the headings stand above the highest mean.

The graph canvas has the same chart as a chart form (Visualize ▸ Graph canvas, Chart: Multi-vari chart): factors on the horizontal-axis shelf, the response on the vertical one.

Try it: Shaft diameters. The first report (batch and shaft) shows that batches differ far more than shafts within a batch; the second adds the position and shows each shaft's taper; the third is the graph-canvas form.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Shaft diameters (simulated)
Multi-Vari Chart of Diameter ...
Multi-Vari Chart of Diameter ...
Multi-Vari Chart: Diameter (...
Scripts
Shaft diameters (simulated)
Variables
Batch
Shaft
Position
Diameter (mm)
Rows
Total 45
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (3)
Side by side
New window
Collapse all
Expand all
Export all...
Multi-Vari Chart of Diameter (mm) by Batch / Shaft
Options
Export...
Multi-Vari Chart
Reset view
Plot settings
Multi-Vari Chart
Batch: Friday
Batch: Monday
Batch: Wednesday
Diameter (mm)
25.02
25.01
25.00
24.99
1 2 3 4 5
Shaft
Shaft means
Batch means
Values (in row order within each cell)
Variation by Factor
Source
Comparisons
Average range of means
Largest range of means
Batch (between levels) 1 0.022867 0.022867
Shaft within Batch 3 0.0032111 0.0041333
Within cells (range of values) 15 0.0086 0.0128
Pooled within-cell standard deviation 15 0.0044785 –
Grand mean 25.003; 45 rows
Ranges View
Reset view
Plot settings
Ranges View
0.02
✔ Diameters of turned shafts measured at three positions, five shafts from each of three batches: multi-vari charts showing that batch-to-batch (time) variation dominates, with a steady taper along eac...
Selecting rows ?
45 rows · 0 selected
Shaft diameters (simulated)

Reliability

The Reliability menu is in four groups, each under a small heading: **Life data** (survival curves, life distributions, competing risks), **Life data with predictors** (proportional hazards, parametric survival regression, life–stress models, degradation), **Field and repair data** (warranty returns, repairable systems, reliability growth) and **Planning** (test plans).

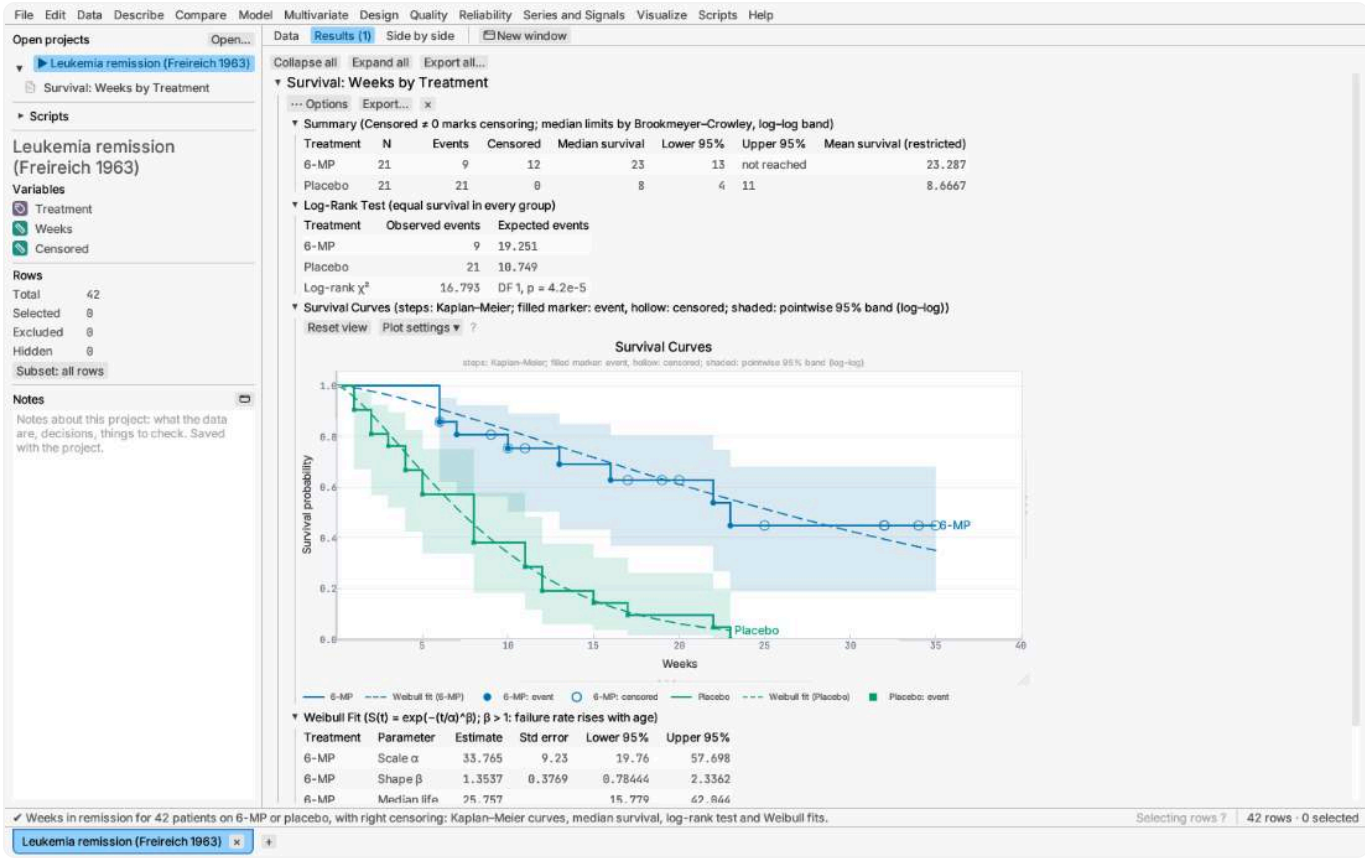
Survival curves

Reliability ▸ Survival curves… analyses times to an event, some of which may be censored (the event had not happened when observation stopped).

- Kaplan–Meier survival curves, with confidence bands on the plain, log or log-log scale;
- the product-limit table;
- a log-rank test comparing groups, and (tick **Log-rank test for trend** in the report's options) Peto's test for a trend across ordered groups: χ^2 on one degree of freedom from the weighted observed-minus-expected events, with the group labels as scores when they are numbers (doses, stages) and 1, 2, 3, … in the groups' order otherwise;
- a **number-at-risk table** under the plot (tick **Number at risk under the curves**): for each group, the units still at risk at each labelled time tick, aligned with the ticks. **Also count censored rows** adds, in brackets, the units censored since the previous tick. The same counts appear as a table in the report;
- quick Weibull and lognormal maximum-likelihood fits drawn over the curves, with AIC and BIC to compare them. For other distributions, left or interval censoring, and intervals for percentiles, use Life distributions.

Try it: Leukemia remission; Bearing life test.

Verified: against published Kaplan–Meier and maximum-likelihood results.



Life distributions

Reliability ▸ Life distributions… fits parametric distributions to failure times by maximum likelihood when some times are only partly known:

- **right-censored** — the unit was still running when last seen;
- **left-censored** — it had already failed at the first look;
- **interval-censored** — it failed between two inspections;
- **exact** failures.

Choose one of two layouts in the dialog:

- **Time and censor columns:** one time per row and, optionally, a censor column. Choose whether a non-zero censor value means "still running" (the default) or "failed".
- **Start and end of the failure interval:** the last time the unit was seen working and the time it was found failed. Leave the start empty for a unit that had failed by its first inspection, leave the end empty for one still running at the start time, and give equal values for an exact failure.

A **frequency** column gives the number of units a row stands for, so inspection results can be entered as counts. A **group** column fits each group separately.

Tick the distributions to fit: exponential, Weibull, lognormal, log-logistic, gamma, normal, logistic, smallest and largest extreme value, and the forms with a threshold (a failure-free period). One of them is shown in detail — the best by AIC unless you choose one. You can list the percentiles (in % failing) and the times you want estimates for, and set the confidence level.

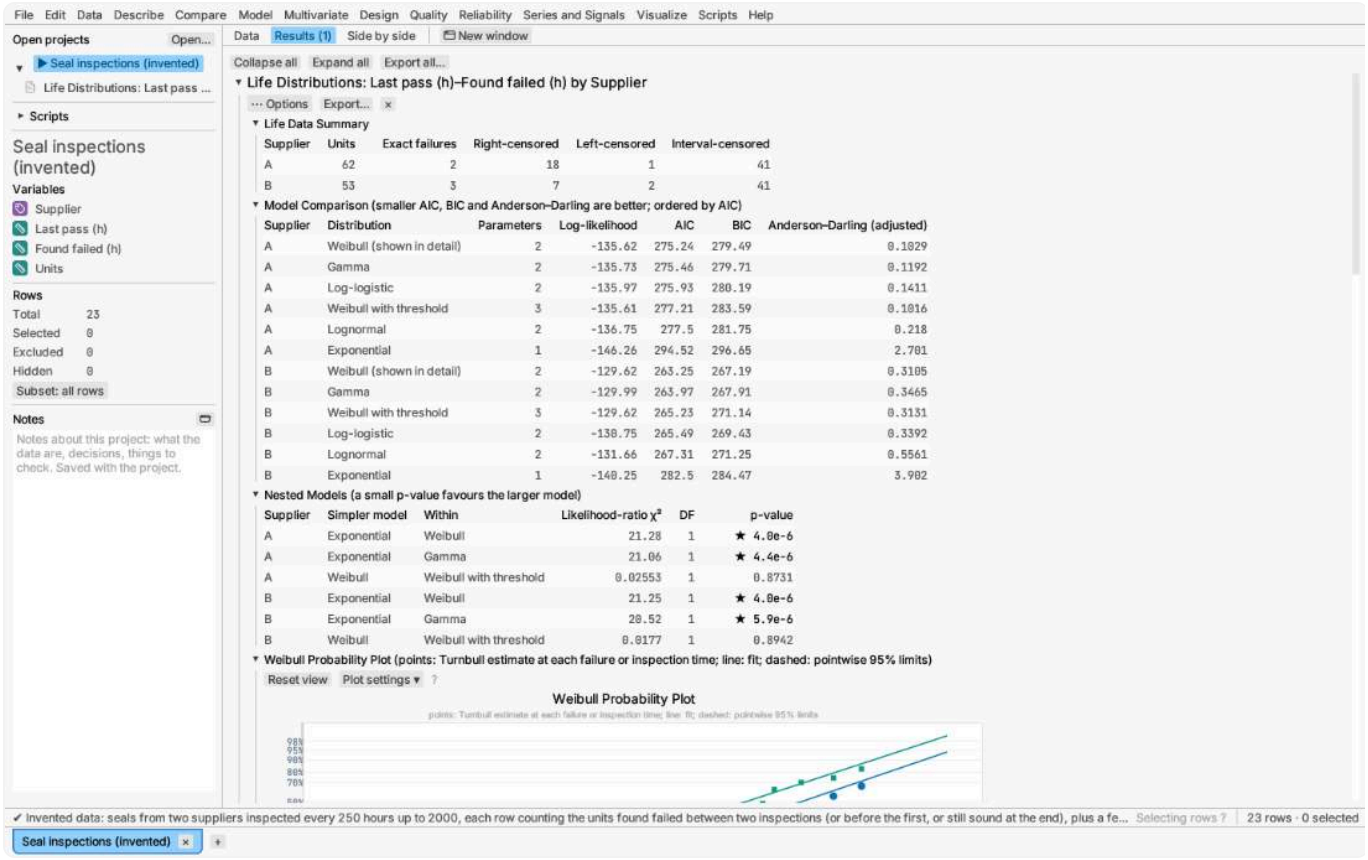
The report has:

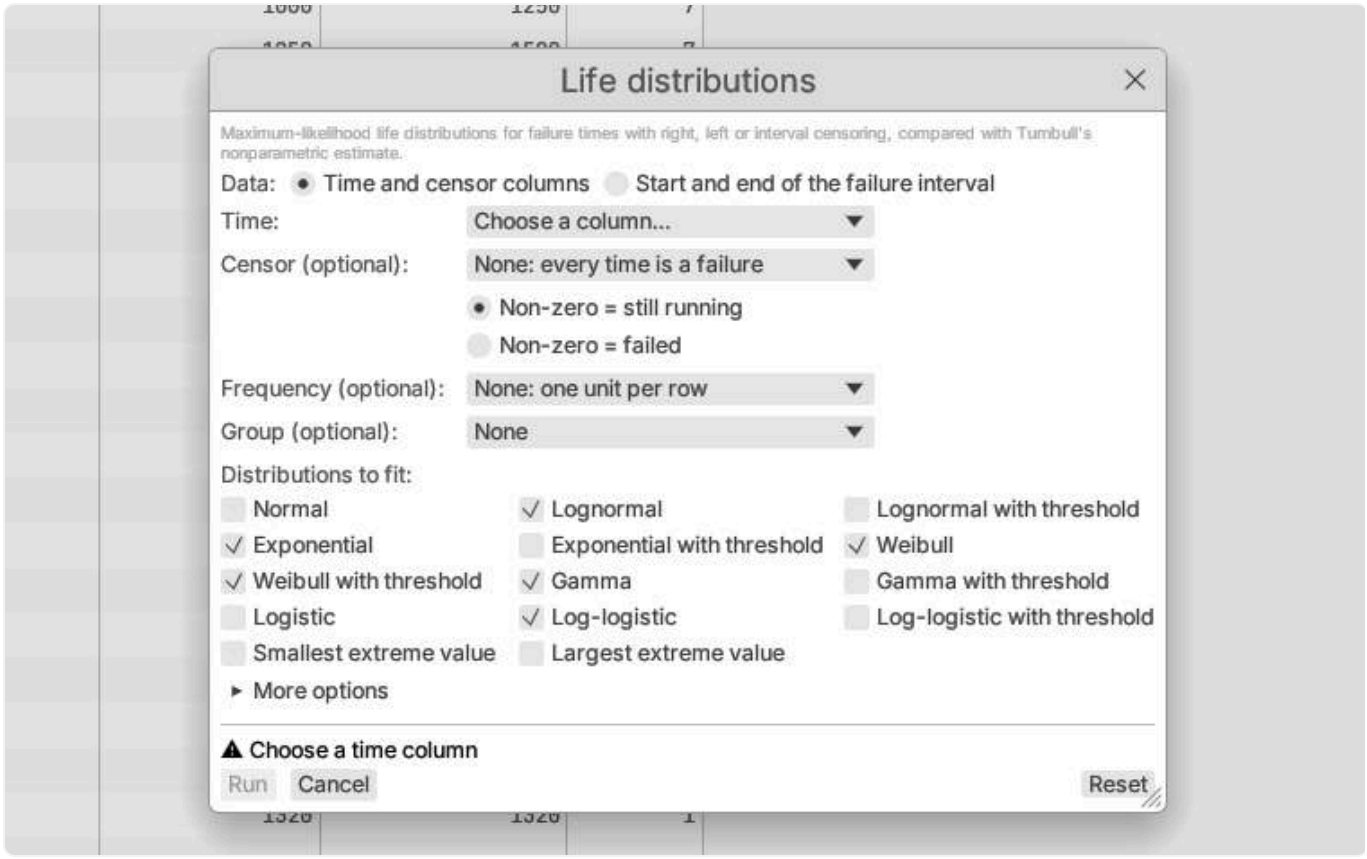
- **Life Data Summary:** units and how many of each kind of observation;
- **Model Comparison:** each distribution's log-likelihood, AIC, BIC and an Anderson–Darling distance adjusted for censoring (the distance between the nonparametric estimate and the fitted distribution; smaller is better), ordered by AIC;
- **Nested Models:** likelihood-ratio tests of the exponential within the Weibull and gamma, and of each two-parameter form within its threshold form;
- a **probability plot** of the chosen distribution: the nonparametric estimate at each failure or inspection time on the distribution's own scale (straight when the distribution fits), the fitted line and pointwise confidence limits; the vertical axis is labelled in % failing;
- **parameter estimates** with standard errors, Wald limits and likelihood-ratio (profile) limits;
- **percentiles** (the time by which a given fraction fails) and **failure probabilities** at the times you gave, each with Wald and likelihood-ratio limits;
- **survival** (the nonparametric estimate with the fitted curve; failures are filled markers, censored units hollow ones) and **hazard** plots;
- the **Turnbull nonparametric estimate**: the probability of failing in each interval it can resolve. With only exact and right-censored times it is the Kaplan–Meier estimate.

Few or no failures. Tick *Weibull with known shape* and give the shape β (from experience with similar parts). With failures and right-censored times only, the report adds the scale estimate with chi-square limits and one-sided lower bounds for the scale, the percentiles and the reliability at your times — usable even with no failures at all (Nelson 1985).

Try it: Seal inspections — interval-censored counts from two suppliers; Bearing endurance test — two failures in twelve units, with the known-shape Weibull.

Verified: against an independent SciPy implementation of the censored likelihood, profile intervals, Turnbull's estimate and the adjusted Anderson–Darling distance (see the validation record).


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Seal inspections (invented)
Life Distributions: Last pass ...
Scripts
Seal inspections (invented)
Variables
Supplier
Last pass (h)
Found failed (h)
Units
Rows
Total 23
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (1)
Side by side
New window
Collapse all
Expand all
Export all...
Life Distributions: Last pass (h)–Found failed (h) by Supplier
Options
Export...
Life Data Summary
Supplier Units Exact failures Right-censored Left-censored Interval-censored
A 62 2 18 1 41
B 53 3 7 2 41
Model Comparison (smaller AIC, BIC and Anderson–Darling are better; ordered by AIC)
Supplier Distribution Parameters Log-likelihood AIC BIC Anderson–Darling (adjusted)
A Weibull (shown in detail) 2 -135.62 275.24 279.49 0.1029
A Gamma 2 -135.73 275.46 279.71 0.1192
A Log-logistic 2 -135.97 275.93 280.19 0.1411
A Weibull with threshold 3 -135.61 277.21 283.59 0.1016
A Lognormal 2 -136.75 277.5 281.75 0.218
A Exponential 1 -146.26 294.52 296.65 2.701
B Weibull (shown in detail) 2 -129.62 263.25 267.19 0.3105
B Gamma 2 -129.99 263.97 267.91 0.3465
B Weibull with threshold 3 -129.62 265.23 271.14 0.3131
B Log-logistic 2 -130.75 265.49 269.43 0.3392
B Lognormal 2 -131.66 267.31 271.25 0.5561
B Exponential 1 -140.25 282.5 284.47 3.902
Nested Models (a small p-value favours the larger model)
Supplier Simpler model Within Likelihood-ratio χ² DF p-value
A Exponential Weibull 21.28 1 ★ 4.0e-6
A Exponential Gamma 21.06 1 ★ 4.4e-6
A Weibull Weibull with threshold 0.02553 1 0.8731
B Exponential Weibull 21.25 1 ★ 4.0e-6
B Exponential Gamma 28.52 1 ★ 5.9e-6
B Weibull Weibull with threshold 0.0177 1 0.8942
Weibull Probability Plot (points: Turnbull estimate at each failure or inspection time; line: fit; dashed: pointwise 95% limits)
Reset view
Plot settings
Weibull Probability Plot
✓ Invented data: seals from two suppliers inspected every 250 hours up to 2000, each row counting the units found failed between two inspections (or before the first, or still sound at the end), plus a fe...
Selecting rows ?
23 rows · 0 selected
Seal inspections (invented)


Life distributions
Maximum-likelihood life distributions for failure times with right, left or interval censoring, compared with Turnbull's nonparametric estimate.
Data: Time and censor columns Start and end of the failure interval
Time: Choose a column...
Censor (optional): None: every time is a failure
Non-zero = still running
Non-zero = failed
Frequency (optional): None: one unit per row
Group (optional): None
Distributions to fit:
Normal
Lognormal
Lognormal with threshold
Exponential
Exponential with threshold
Weibull
Weibull with threshold
Gamma
Gamma with threshold
Logistic
Log-logistic
Log-logistic with threshold
Smallest extreme value
Largest extreme value
More options
Choose a time column
Run
Cancel
Reset

Life–stress models

Reliability ▸ Life–stress models… analyses accelerated life tests: units run at raised temperature, voltage, humidity or another stress so that they fail sooner, and the results are extrapolated to the conditions of use. The log of life follows a location–scale distribution whose location μ depends on the stresses (Nelson 1990; Meeker & Escobar 1998, ch. 19–20).

Data use the layouts of Life distributions: a time column with an optional censor column, or the start and end of the interval in which each unit failed, so right, left and interval censoring are all allowed; a **frequency** column counts units per row. Then choose:

- the **stress** column, its **relationship** and its **use condition**:
  - **Arrhenius** (temperature, in °C or K): $\mu = \beta_0 + \beta_1 \cdot 11604.518/T$, so β_1 is the activation energy E_a in electron-volts;
  - **Eyring** (temperature): as Arrhenius with a further $-\ln T$ term;
  - **Inverse power** (voltage, load): $\mu = \beta_0 + \beta_1 \ln s$; life $\propto s^{-n}$ with $n = -\beta_1$;
  - **Exponential** (linear): $\mu = \beta_0 + \beta_1 s$;
- optionally a **second stress** with its own relationship and use condition (Arrhenius with an inverse-power humidity or voltage term is the Peck model), and their **interaction** (the generalized Eyring form);
- the **life distribution**: Weibull, lognormal, log-logistic or exponential.

The scale (the Weibull shape is $1/\sigma$) is the same at every stress unless you tick *Scale depends on the stresses* under More options, which makes $\ln \sigma$ linear in the stresses and adds a likelihood-ratio test of a constant scale. More options also take the percentiles and times reported at use conditions, and the confidence level.

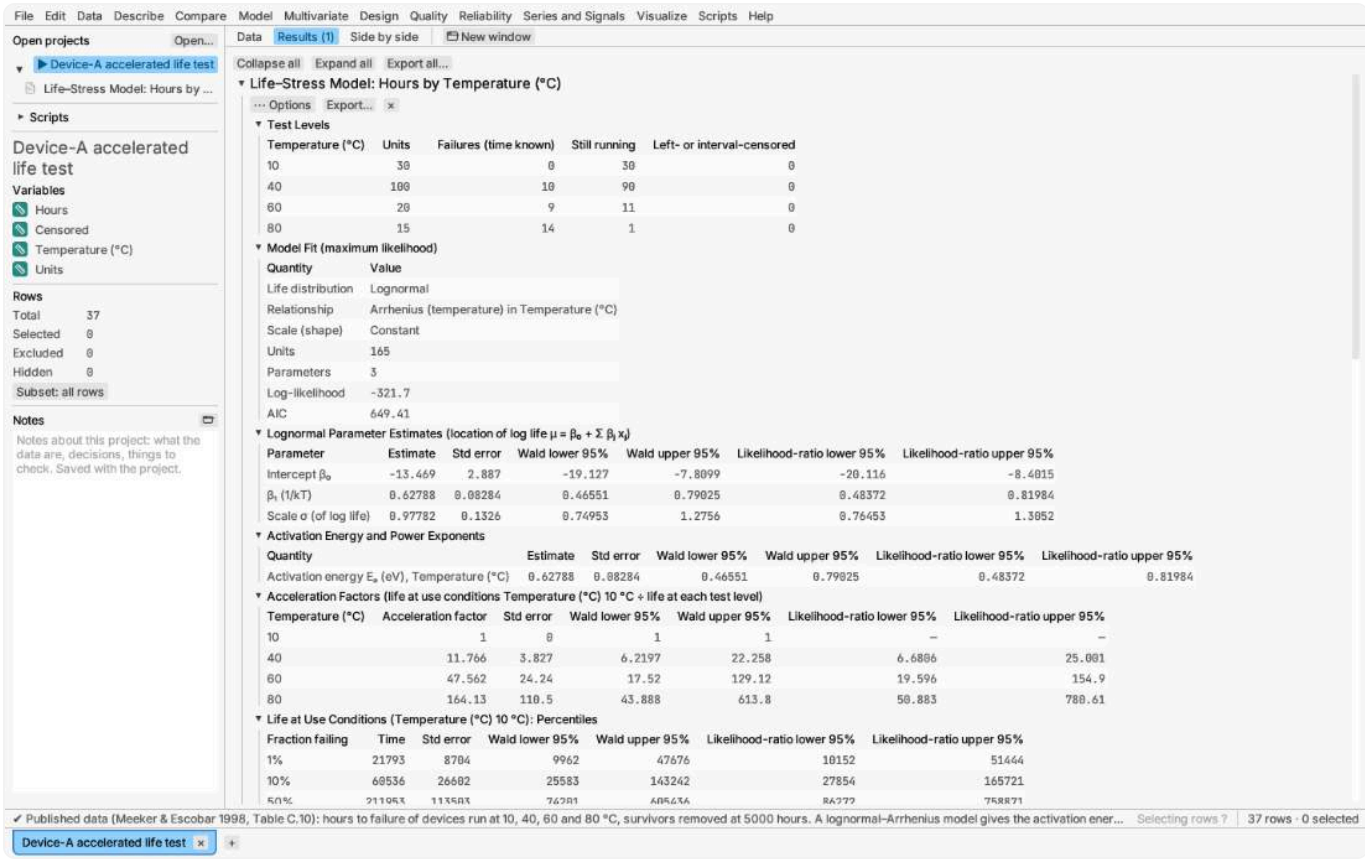
The report has:

- **Test Levels:** units, exact failures, survivors and left- or interval-censored units at each stress combination;
- **Model Fit:** the distribution and relationships, units, parameters, log-likelihood and AIC;
- **Parameter Estimates** with standard errors, Wald limits and likelihood-ratio (profile) limits;
- **Activation Energy and Power Exponents** with their limits;
- **Acceleration Factors:** life at use conditions divided by life at each test level, with limits;
- **Life at Use Conditions:** percentiles, and failure probabilities at your times, each with Wald and likelihood-ratio limits;

- the **life–stress plot:** failure and censoring times against the stress (on the 1/T scale for temperature, log scale for an inverse power), each kind of observation with its own marker, and percentile lines of the fitted model from the use condition to the highest test level, with the median life at use marked;
- the **probability plot by test level:** Turnbull's estimate at each level on the distribution's own scale, each level with its own color and marker, and the fitted lines (parallel when the scale is constant);
- **Model Checks:** likelihood-ratio tests of equal scales across levels, of the life–stress relationship against a separate location per level, and of the model against separate fits at every level (and of a constant scale when the scale depends on the stresses);
- **Individual Fits by Level:** each level fitted on its own.

Try it: Device-A accelerated life test — a lognormal–Arrhenius model at 10 °C, with an activation energy of about 0.63 eV.

Verified: against an independent SciPy implementation of the censored likelihood and profile limits for every family, one and two stresses, interval censoring and a stress-dependent scale; against lifelines' accelerated failure time fitters; and against Meeker & Escobar's published Device-A estimates and Nelson's Class-B insulation fit (see the validation record).



Fatigue (stress–life) curves

Reliability ▸ Fatigue (stress–life) curves… fits an S–N curve to constant-amplitude fatigue tests: specimens cycled at a stress amplitude S until they break after N cycles, or are stopped unbroken (**run-outs**, right-censored at their cycles). Choose the **cycles**, an optional **run-out** column (non-zero = did not fail), the **stress** and an optional **frequency** column. The life follows a log-location–scale distribution (**lognormal**, **Weibull** or **log-logistic**) whose location μ = E[ln N] (for the Weibull, ln of the 63.2% life) depends on the stress through the **curve** (ASTM E739-10; Meeker & Escobar 1998, ch. 17; Pascual & Meeker 1999):

- **Basquin** (log–log line): $\mu = \beta_0 + \beta_1 \ln S$, equivalently $S = A \cdot N^{b}$ with $A = \exp(-\beta_0/\beta_1)$ and $b = 1/\beta_1$ (at $N = e^{\mu}$);
- **Semi-log**: $\mu = \beta_0 + \beta_1 S$;
- **Quadratic log–log**: $\mu = \beta_0 + \beta_1 \ln S + \beta_2 (\ln S)^2$ (three or more stress levels);
- **Endurance limit** (Stromeyer form): $\mu = \beta_0 + \beta_1 \ln(S - E)$ for $S > E$, and no failure at or below the endurance limit E. E is found by profile likelihood over (0, min S), with a likelihood-ratio interval; standard errors of the other parameters are conditional on $\hat{E}$.

The scale σ is the same at every stress unless *Scatter depends on stress* is ticked under More options ($\ln \sigma = \gamma_0 + \gamma_1 \ln S$). All parameters are fitted by maximum likelihood with the run-outs contributing their survival probability. More options also set the quantile curves (% failing; 10 and 50 by default), the lives for the fatigue strength (10^6 and 10^7 cycles), the stresses for life quantiles (the tested levels by default) and the confidence level.

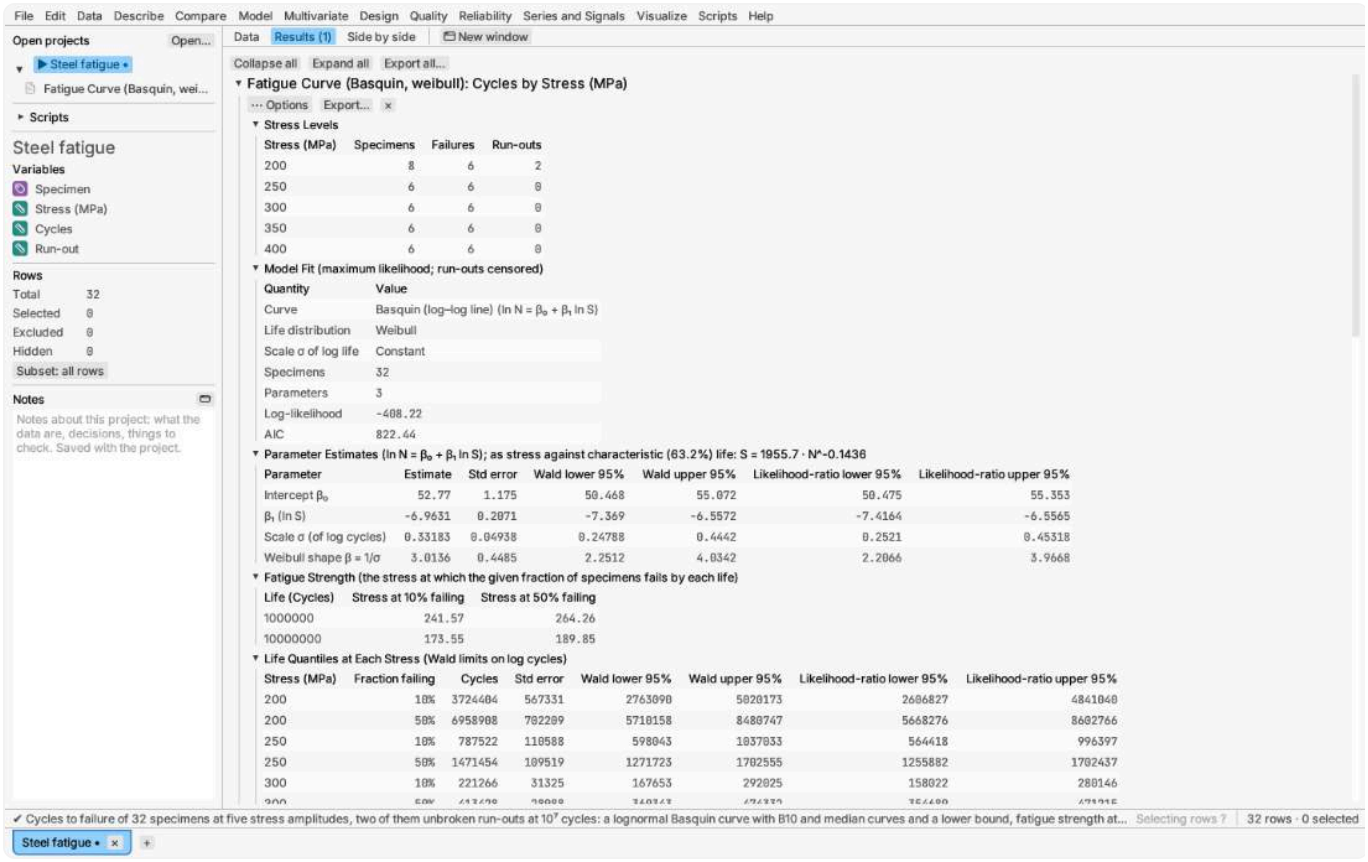
Report:

- *Stress Levels* — specimens, failures and run-outs at each stress.
- *Model Fit* — curve, distribution, parameters, log-likelihood, AIC, and for the endurance form $\hat{E}$ with its interval.
- *Parameter Estimates* — coefficients and σ (or the Weibull shape) with Wald and likelihood-ratio limits, and Basquin's A and b.
- *Fatigue Strength* — the stress at which each quantile life equals each chosen number of cycles.
- *Life Quantiles* — the cycles by which each percentage fails at each stress, with limits.
- *S–N Curve* — failures and run-outs (own markers) on log axes (log cycles and log stress; linear stress for the semi-log curve) with the quantile curves, for example B10 and the median, and the lower confidence bound of the lowest one.
- *Standardized Residuals* — $(\ln N - \mu)/\sigma$ against stress, run-outs marked.

- *Probability Plot of Standardized Residuals* — Turnbull's estimate of the standardized residuals of all levels together (run-outs as censored) on the distribution's own scale against the standard distribution's line: points far from the line question the distribution or the curve.

Verified: an independent SciPy maximum-likelihood implementation (every curve and distribution, stress-dependent scale, the endurance limit and its interval) and lifelines' accelerated failure time fitters for the Basquin fits.

Try it: Steel fatigue (32 specimens at 200–400 MPa, two run-outs at 10^7 cycles): a lognormal Basquin curve, and an endurance-limit curve with an estimated limit of about 144 MPa.



Degradation

Reliability ▸ Degradation… draws life information from a measured characteristic — light output, current, crack length, strength — that drifts towards a failure threshold, which is useful when few units fail during the test (Meeker & Escobar 1998, ch. 13, 21).

Choose one of two layouts:

- **Repeated measurements of each unit:** unit, time and response columns, several measurements per unit;
- **Destructive:** time and response columns, one measurement per unit (measuring it destroys it, as when a bond is pulled apart).

Give the **failure threshold** and whether failure is when the response **rises** or **falls** to it.

For repeated measurements, choose the **path model** fitted to each unit by least squares: **linear** on transformed scales (the response and time can each be left as they are, logged or square-rooted), **asymptotic** D∞(1 − e^(−kt)) or **power** a·t^b. Then choose how life information is obtained:

- **Pseudo failure times, then a life distribution:** each unit's path is solved for the time it reaches the threshold. A unit whose path never reaches it is right-censored at its last measurement, and one already past it at its first measurement is left-censored. The chosen life distribution is fitted to these times; with a stress column (constant within each unit), a life–stress model is fitted instead.
- **Random-effects paths** (linear paths only): each unit has its own intercept and slope, drawn from a bivariate normal distribution, fitted by REML. The distribution of the time to reach the threshold follows from the model; its pointwise limits come from a parametric bootstrap (the number of replicates is under More options; the bootstrap is seeded, so the report is reproducible).

For destructive data the transformed response follows a normal, logistic or extreme-value distribution whose location falls (or rises) linearly in transformed time, $\mu = \beta_0 + \beta_1 \cdot h(t)$. With an **accelerating stress** the rate is $\beta_1 \cdot \exp(\beta_2(x - x_{use}))$, where x is the stress on its relationship's scale (1/kT for temperature), so β_1 is the rate at use conditions and, for temperature, $-\beta_2$ is the activation energy of the degradation rate.

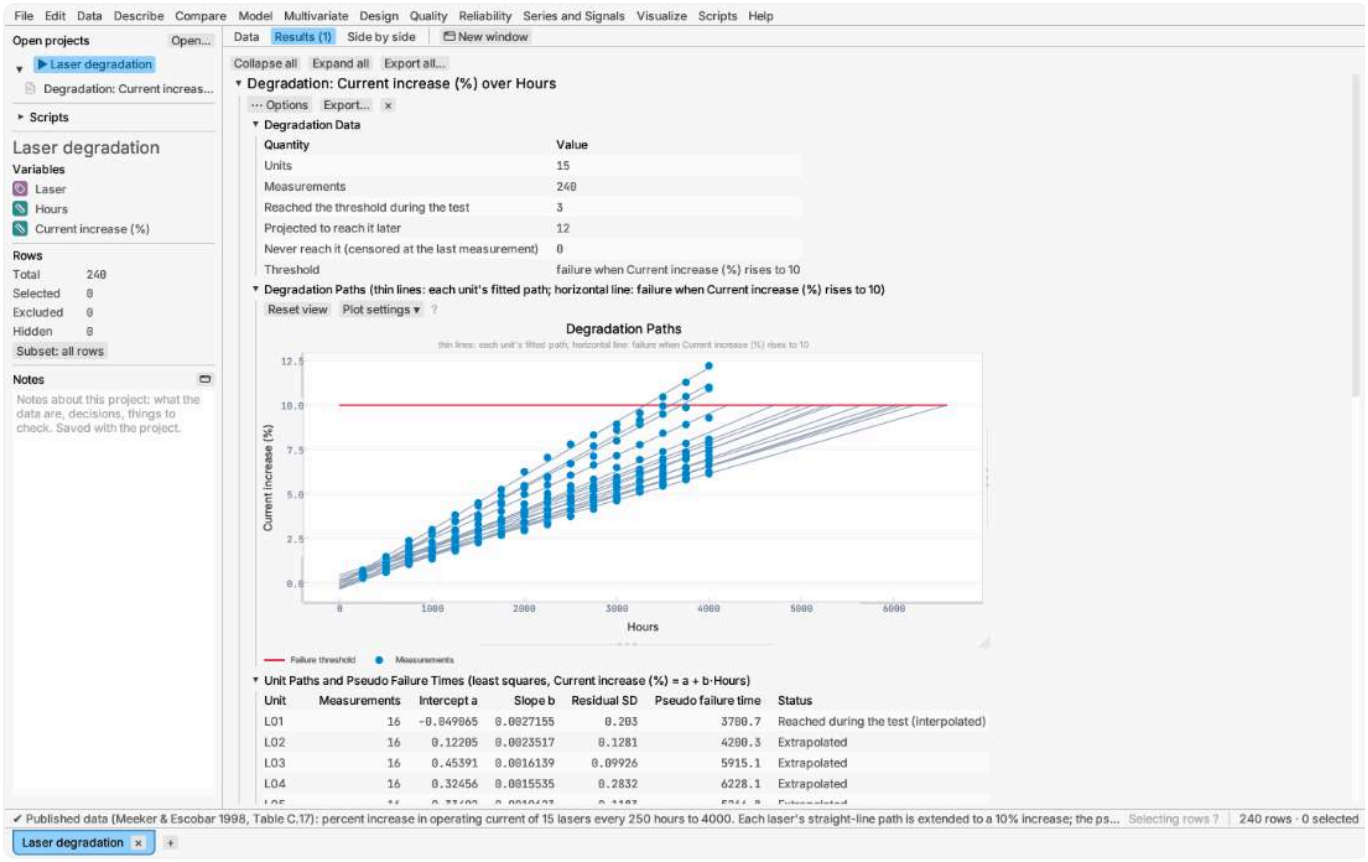
The report has:

- a **summary** of units, measurements and how many reached the threshold;
- the **path plot:** measurements, each unit's fitted path (for destructive data, the fitted median response at each stress level, each with its own color and marker) and the threshold as a horizontal line;
- **Unit Paths and Pseudo Failure Times:** each unit's parameters, residual SD, pseudo failure time and whether it was reached, extrapolated or censored;
- the **life distribution** parameters with limits, a **probability plot** of the pseudo failure times with the fitted line, and **percentiles** and **failure probabilities** with limits; or, for random effects, the **variance components** and the **time-to-threshold distribution** curve with its bootstrap band; or, for destructive data, the **regression** parameters

(with the activation energy of the rate) and the time-to-threshold distribution at use conditions.

Try it: Laser degradation — 15 lasers whose operating current rises; three reach a 10% increase within 4000 hours. Adhesive bond strength — destructive tests at three temperatures with a 50 N threshold at 40 °C.

Verified: per-unit fits, pseudo failure times and life fits against NumPy/SciPy; the random-effects model against statsmodels' MixedLM; the destructive model against an independent SciPy maximum-likelihood fit with delta-method limits; and the laser data's three threshold crossings as reported by Meeker & Escobar (see the validation record).



Proportional hazards

Reliability ▸ Proportional hazards… fits Cox's regression model: each predictor multiplies the hazard of failure by a constant factor, the hazard ratio, while the shape of the hazard over time is left unspecified (Cox 1972). It suits clinical follow-up and field data where the question is "which factors shorten life, and by how much?" rather than "what is the life distribution?".

Lay the data out with one row per unit: its **time** of failure or censoring and, optionally, a **censor** column (choose whether non-zero means censored or failed; with no censor column every time is a failure). Then tick the **predictors**. Continuous columns enter as slopes; categorical ones as indicator columns against their first level. Under *Model effects* add two-factor interactions or squares and cubes of continuous predictors; with *Keep the model hierarchical* ticked (the default) an interaction brings in its main effects, and the report notes any violation when it is unticked. Optionally:

- a **strata** column gives each of its levels its own baseline hazard while the predictors' effects are shared — the way to adjust for a factor whose effect is not proportional;
- **Survival curves for** one of the predictors draws a curve per level (or at its 10th, 50th and 90th percentiles when continuous), the others held at their mean or most frequent level.

More options sets how **tied failure times** are handled (Efron, the default, or Breslow), the time scale of the proportional-hazards test (Kaplan–Meier, the default; rank; identity; log), times at which to tabulate predicted survival, what the Response Explorer shows (the hazard ratio against the average unit, or the probability of surviving past a time) and the confidence level.

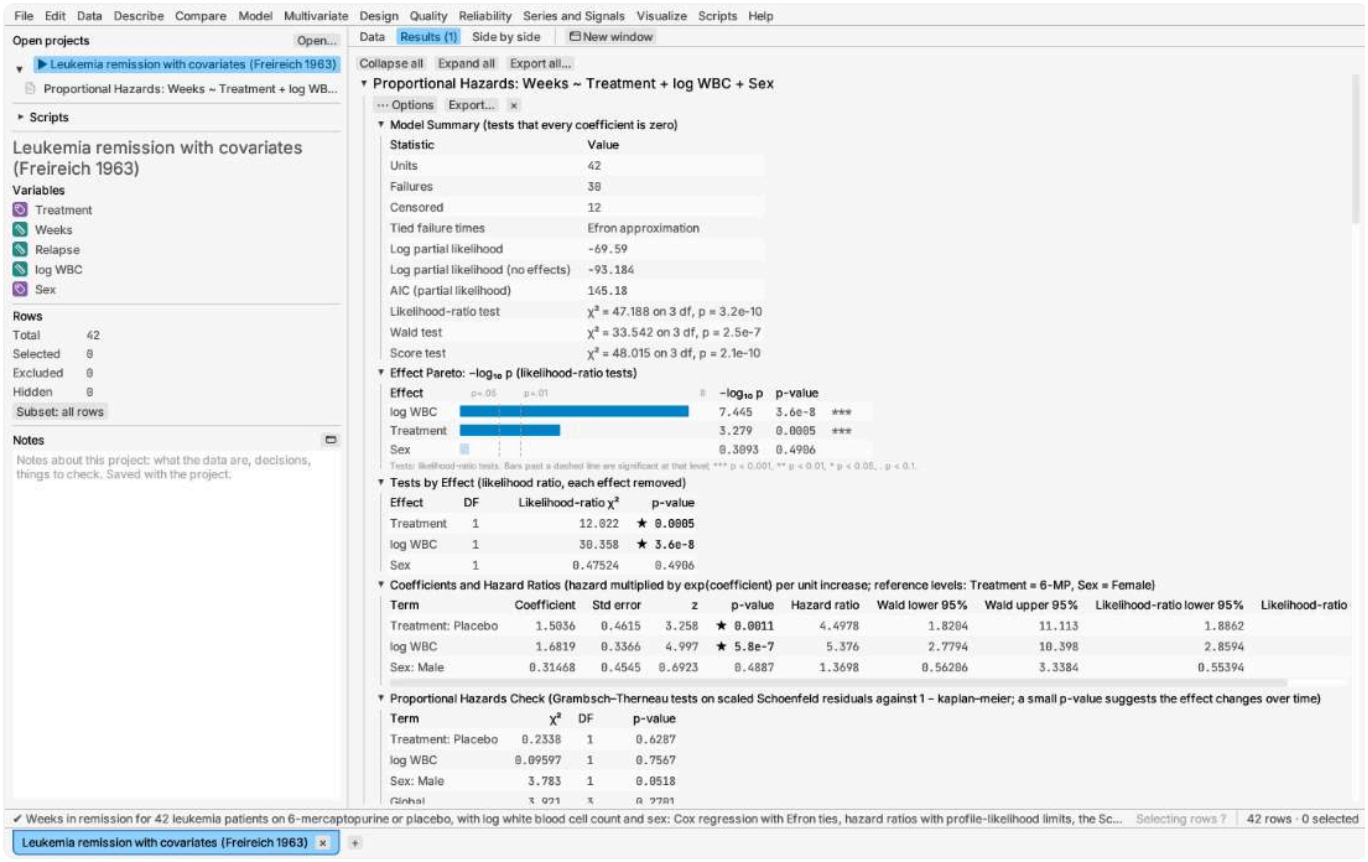
The report has:

- a **Model Summary**: log partial likelihood, AIC, and the likelihood-ratio, Wald and score tests that every coefficient is zero;
- the **Effect Pareto** and **Tests by Effect**: a likelihood-ratio test of each effect, refitting without it;
- **Coefficients and Hazard Ratios**: each coefficient with its standard error and Wald z test, and the hazard ratio exp(β) with Wald and likelihood-ratio (profile) limits;
- the **Proportional Hazards Check**: Grambsch & Therneau's (1994) test of each coefficient and the global test, on the scaled Schoenfeld residuals against transformed time — a small p-value says that effect changes over time;
- a **scaled Schoenfeld residual plot** for each coefficient, one point per failure, with the least-squares trend the test measures and a reference line at the estimate; a flat trend supports proportional hazards;
- **Predicted Survival** curves with pointwise bands (from the log cumulative hazard), and a table at your times;
- **Deviance residuals** against the linear predictor, to find units that failed much earlier or later than the model expects;
- the **Baseline Survival**: Breslow's estimate with every model column at its mean, per stratum.

The ⋯ menu shows the Response Explorer and 3-D surface profiler and saves the linear predictor, hazard ratio, predicted survival, and martingale and deviance residuals as new columns.

Try it: Leukemia with covariates — remission times of 42 patients by treatment, log white cell count and sex; curves by treatment.

Verified: coefficients, standard errors, test statistics and profile-likelihood limits against statsmodels `PHReg` (Efron and Breslow ties, with and without strata), and baseline hazard, residuals and proportional-hazards tests against lifelines `CoxPHFitter` and a direct NumPy implementation of the published formulas (see the validation record).



Parametric survival regression

Reliability ▸ Parametric survival regression… fits an accelerated failure time model: the log of the failure time is a linear function of the predictors plus a scaled error with a Weibull (smallest extreme value), lognormal (normal), log-logistic (logistic) or exponential distribution. A predictor's coefficient stretches or shrinks every percentile of life by the same factor — the time ratio — which makes it the natural model for accelerated life tests

and for life data with a few explanatory variables (Meeker & Escobar 1998, ch. 17; Kalbfleisch & Prentice 2002).

Lay out the failure times exactly as for Life distributions: a time and censor column, or the start and end of each failure interval (empty start = failed before the first look; empty end = still running; equal = exact), with an optional frequency column. Choose the distribution, tick the predictors and their effects (with the same hierarchy rule as above), and optionally a predictor whose levels or percentiles get their own curve and predictions. *More options* sets the percentiles (% failing) and times to predict, the fraction failing the Response Explorer shows the time for, and the confidence level.

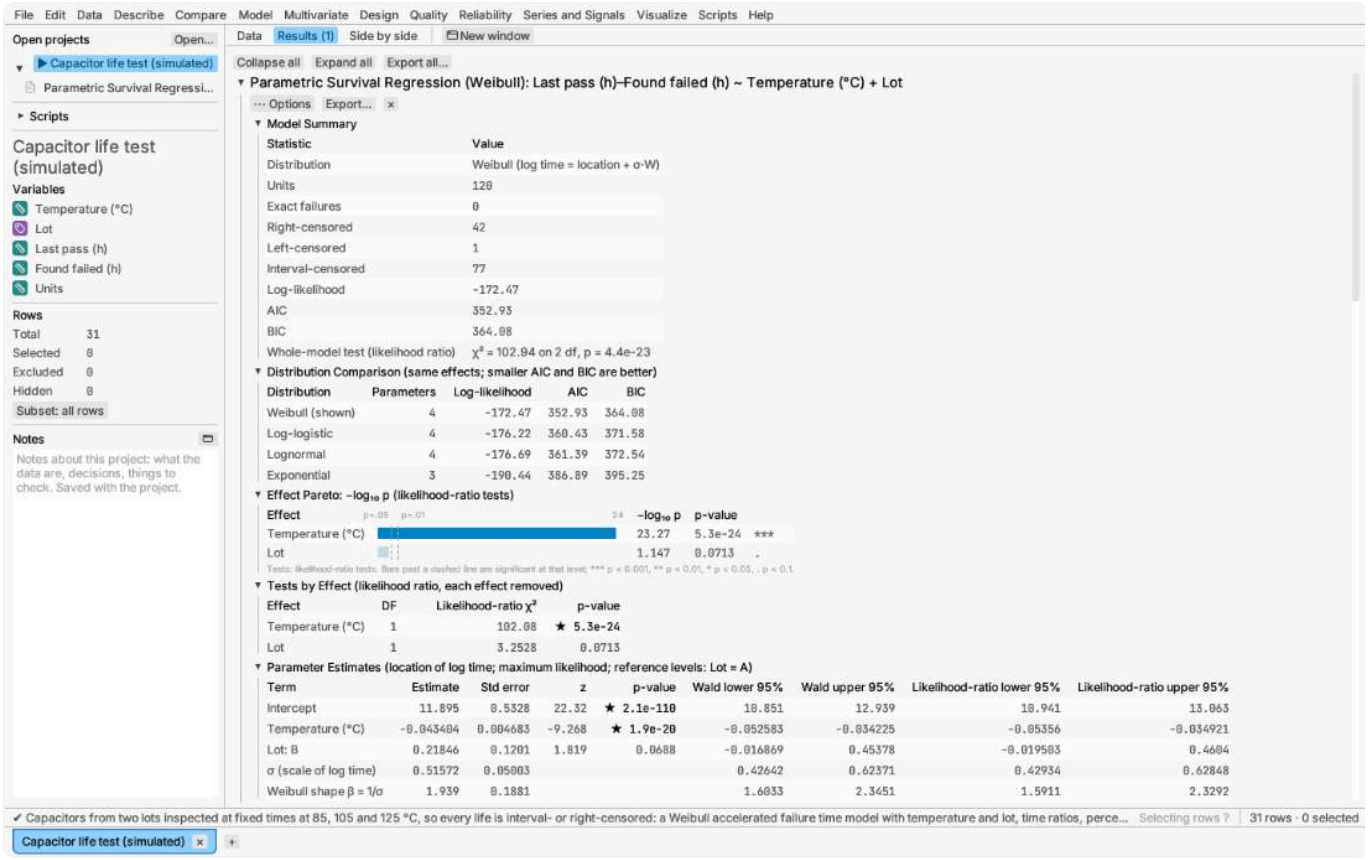
The report has:

- a **Model Summary**: log-likelihood, AIC and BIC, and the likelihood-ratio test of all effects;
- a **Distribution Comparison** of the four distributions with the same effects;
- the **Effect Pareto** and likelihood-ratio **Tests by Effect**;
- **Parameter Estimates** of the location of log time and the scale σ (and the Weibull shape 1/σ), with standard errors from the observed information, Wald limits and likelihood-ratio limits;
- **Time Ratios**: exp(coefficient), the factor applied to every percentile per unit increase, with Wald and profile limits;
- **Predicted Quantiles** and **Predicted Failure Probabilities** at each setting, with Wald limits (delta method, on log time for quantiles and on the standardized scale for probabilities);
- **Fitted Survival**: the nonparametric (Turnbull) estimate for each level of a categorical curve predictor with the model's dashed curve and band;
- a **Cox–Snell Residual Check**: the residuals' cumulative hazard against the unit exponential line — points near the line when the distribution and effects fit.

The ⋯ menu shows the Response Explorer (predicted time to the chosen fraction failing) and saves the predicted quantile and the Cox–Snell residuals as new columns.

Try it: Capacitor life test — inspections at 85, 105 and 125 °C, every life interval- or right-censored; a Weibull model with temperature and lot.

Verified: against lifelines' Weibull, lognormal and log-logistic AFT fitters (right- and interval-censored) and an independent SciPy implementation of the censored likelihood, profile limits and predictions (see the validation record).



Reliability growth

Reliability ▸ Reliability growth… follows a system under development whose failures are fixed as they are found, and asks whether — and how fast — its reliability is improving. It fits the power-law (Crow–AMSAA) process, whose failure intensity is λβt^(β−1): a shape β below 1 means failures are becoming rarer.

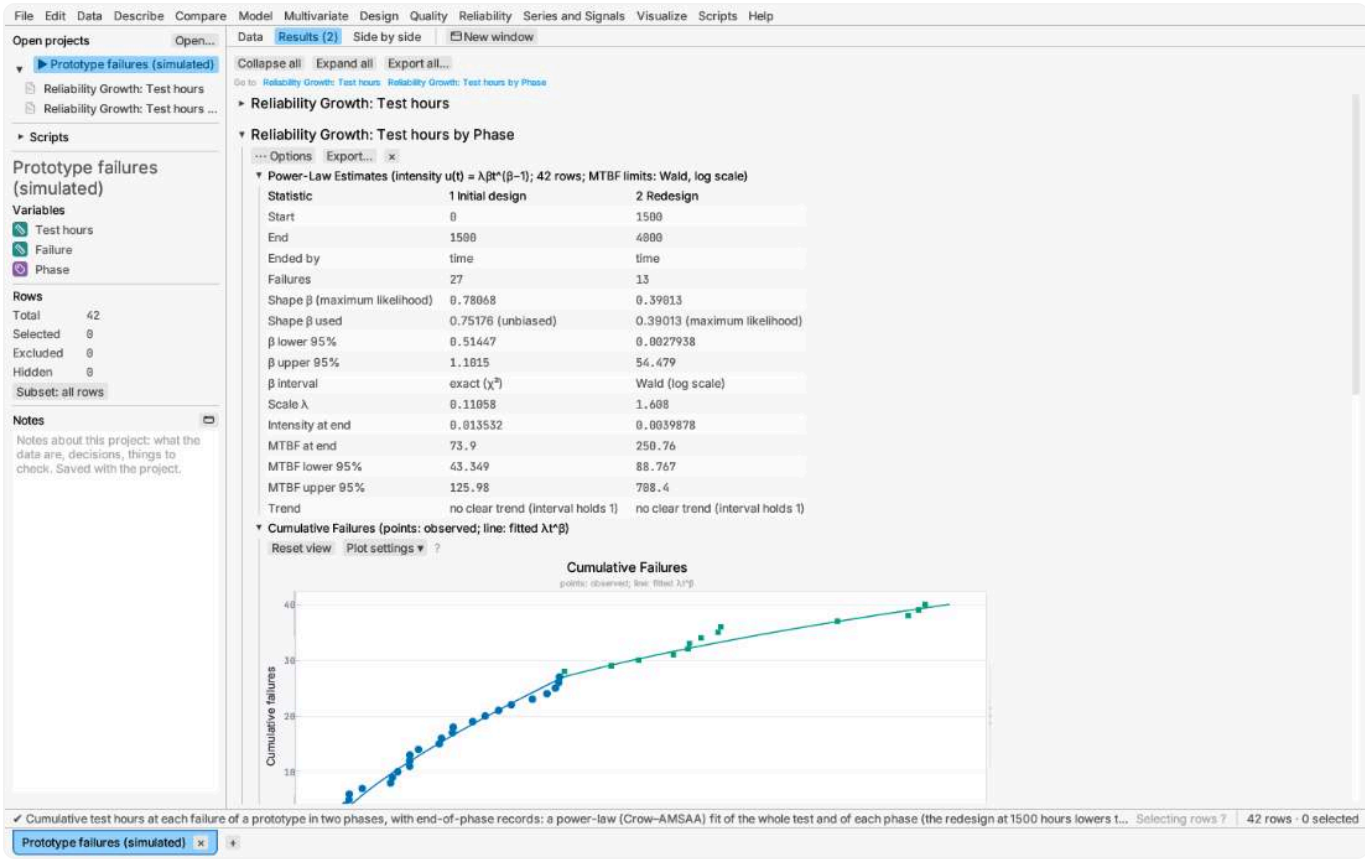
Lay the data out with one row per failure and its **cumulative** test time. Optionally:

- an **event** column, where 0 marks a row that is not a failure but the end of the test (or of a phase) — the test is then *time-terminated* there; without one it is *failure-terminated* at the last failure, unless you give the end time in the dialog;
- a **phase** column, splitting the test where the design changed. Phases follow each other on the same clock; each is fitted separately, starting where the one before ended.

The report has:

- an estimates table per phase: failures, shape β with exact chi-square limits, scale λ, and the intensity and MTBF (mean time between failures) at the end of the phase with confidence limits. The shape is the unbiased estimate by default ((n−1)/n or (n−2)/n times the maximum-likelihood one, for time- or failure-terminated tests); the dialog can show the maximum-likelihood estimate instead;
- cumulative failures against time, with the fitted mean;
- the intensity and the MTBF over time, each with a confidence band and a curve per phase;
- the Cramér–von Mises goodness-of-fit test of the power-law model, with a simulated p-value (for phases starting at time 0).

Try it: Prototype failures — two design phases; the redesign's MTBF is higher and the fit is adequate.



Repairable systems

Reliability ▸ Repairable systems… describes how repairs accumulate over the age of systems that are repaired rather than replaced (pumps, vehicles, servers). Lay the data out with one row per repair: the **system** identifier and its **age** at the repair. An optional **event**

column with 0 marks the row that gives a system's age at the end of observation; a system without one is taken to be observed up to its last repair. An optional **population** column splits the systems into groups.

The report has:

- the **mean cumulative function** (Nelson's estimate of the average number of repairs per system by each age) with confidence limits, plotted as a step curve with repairs as filled markers and ends of observation as hollow ones, and as a table. The variance is the robust one (no assumption that repairs form a Poisson process) unless you choose Poisson. The limits are formed on the log scale by default, so they never go below zero; normal limits (the estimate ± z standard errors) are an option in the dialog;
- with exactly two populations, the difference between their mean cumulative functions with limits: where the band excludes zero, the populations differ;
- fits of a constant-rate (homogeneous Poisson) process and of a power-law process, with a likelihood-ratio test of a constant rate;
- the Laplace and military handbook trend tests per population: a small p-value means the repair rate changes with age (improving or deteriorating, as the report says).

Try it: Pump repairs — old-design pumps need repairs ever more often; new-design pumps show no trend.

Competing risks

Reliability ▸ Competing risks… is for units that can fail in several ways — failure modes of a component, causes of death, reasons for leaving a study — where the first failure ends the unit's life and hides the others. Lay the data out with one row per unit: the **time** of failure or censoring and the **cause** (failure mode). Choose the cause value that means the unit was still running (censored); the dialog proposes one such as `0`, `Censored` or `Running` when the column has it. Optional columns:

- a **group** column, compared cause by cause;
- a **frequency** column, the number of units a row stands for;
- **covariates** (in *More options*) for a regression: continuous columns, and nominal ones coded by indicators of all levels but the first. The group column may also be a covariate.

The report has:

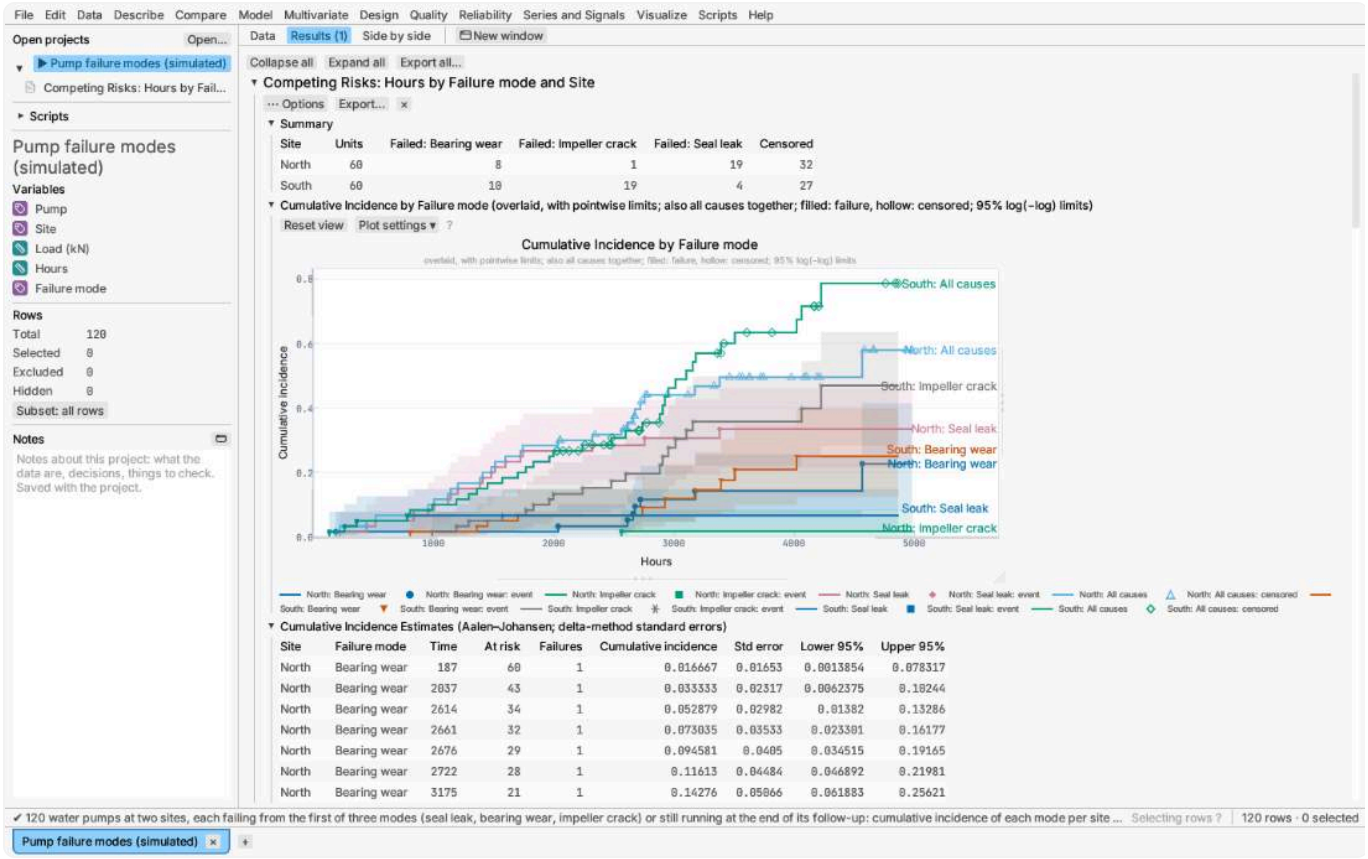
- a **summary** of units, failures from each cause and censored units per group;

- the **cumulative incidence** of each cause — the probability of having failed from that cause by each time, while the other causes compete — by the Aalen–Johansen estimator, drawn as step curves. *Overlaid* (the default), each cause has its pointwise confidence limits (delta-method standard errors, formed on the log(−log) scale) and an *All causes* curve shows 1 − Kaplan–Meier; *stacked*, each curve adds the next cause, so each band between curves is one cause and the top curve is all causes. Failures are filled markers on their cause's curve, censored units hollow ones on the all-causes curve. A table lists the estimates at each failure time;
- with a group column, **Gray's test** for each cause: whether its cumulative incidence is the same in every group (a χ^2 test with one degree of freedom fewer than groups). Unlike a log-rank test on that cause alone, it compares the incidence actually seen, which also depends on the other causes;
- with covariates, **subdistribution hazards regression** (Fine–Gray) for one cause — the first unless you choose another: coefficients, robust standard errors (which allow for the weights below), and subdistribution hazard ratios with limits. A ratio above 1 means the covariate raises that cause's cumulative incidence. Units that failed from another cause stay in the risk set, weighted by the probability that they would still have been observed. A fit table gives the log partial likelihoods and the robust Wald test of all coefficients, and a plot shows the **predicted cumulative incidence** at covariate settings: by default each level of the first covariate (or its 10th, 50th and 90th percentiles when it is continuous) with the others at their mean or most frequent level; in *Predict at* you can give your own, one per line with values in covariate order (e.g. `10, North`);
- with *Fit a life distribution to each failure mode* (Weibull by default), the engineering view (Meeker & Escobar 1998, ch. 15): each mode's distribution is fitted with the other modes' failures treated as censored, and the modes are combined into the **series system** — a unit fails at its first mode failure, so its failure probability is 1 − Π(1 − F_j(t)). A table gives each mode's parameters; a plot shows each mode's Kaplan–Meier estimate with its fit and the system with pointwise limits (delta method on the logit scale); a table gives the system's failure probability with limits and each mode's at chosen times (by default near the system's 1%, 10% and 50% points), and the times by which 1%, 10% and 50% of systems fail. This assumes the modes act independently.

Try it: Pump failure modes — three modes at two sites: impeller cracks come sooner at the harsher site (Gray's test), a higher load hastens bearing wear (the regression), and the Weibull fits per mode combine into the pump's life.

Verified: cumulative incidence, variances and limits against lifelines' Aalen–Johansen estimator; Gray's test and Fine–Gray regression (estimates, robust and model-based standard errors, baseline) against independent NumPy implementations, and with no

censoring against lifelines' Cox regression on the subdistribution risk set; the series system against an independent SciPy fit (see the validation record).



Warranty returns

Reliability ▸ Warranty returns… turns shipment and return counts into life data and forecasts the returns still to come. Ages are counted in periods (months, weeks): units enter service at the start of the period they were shipped in, so a return in a unit's first period failed at an age between 0 and 1, in its second between 1 and 2, and so on (interval censoring); a unit not returned has survived to its current age (right censoring), or to the warranty length when you give one and it has passed it.

Choose one of two layouts:

- **One row per shipment period** (a triangular table): the units shipped, an optional period label, and one column per calendar period of return, in order — the first is the first row's period, and the cells before a row's own period are empty. There are as many periods observed as return columns.

- **One row per count:** whole-number shipment and return periods, the units returned, and the units shipped in the row's shipment period (repeated on its rows). A row with an empty return period gives only the shipment.

Tick the distributions to compare (Weibull, lognormal, exponential and log-logistic by default) and choose the one to forecast with — Weibull by default, or the best by AIC. Give the number of periods to forecast and, optionally, **planned shipments** for the coming periods (units shipped in future periods 1, 2, …). *More options* has the warranty length and the confidence level.

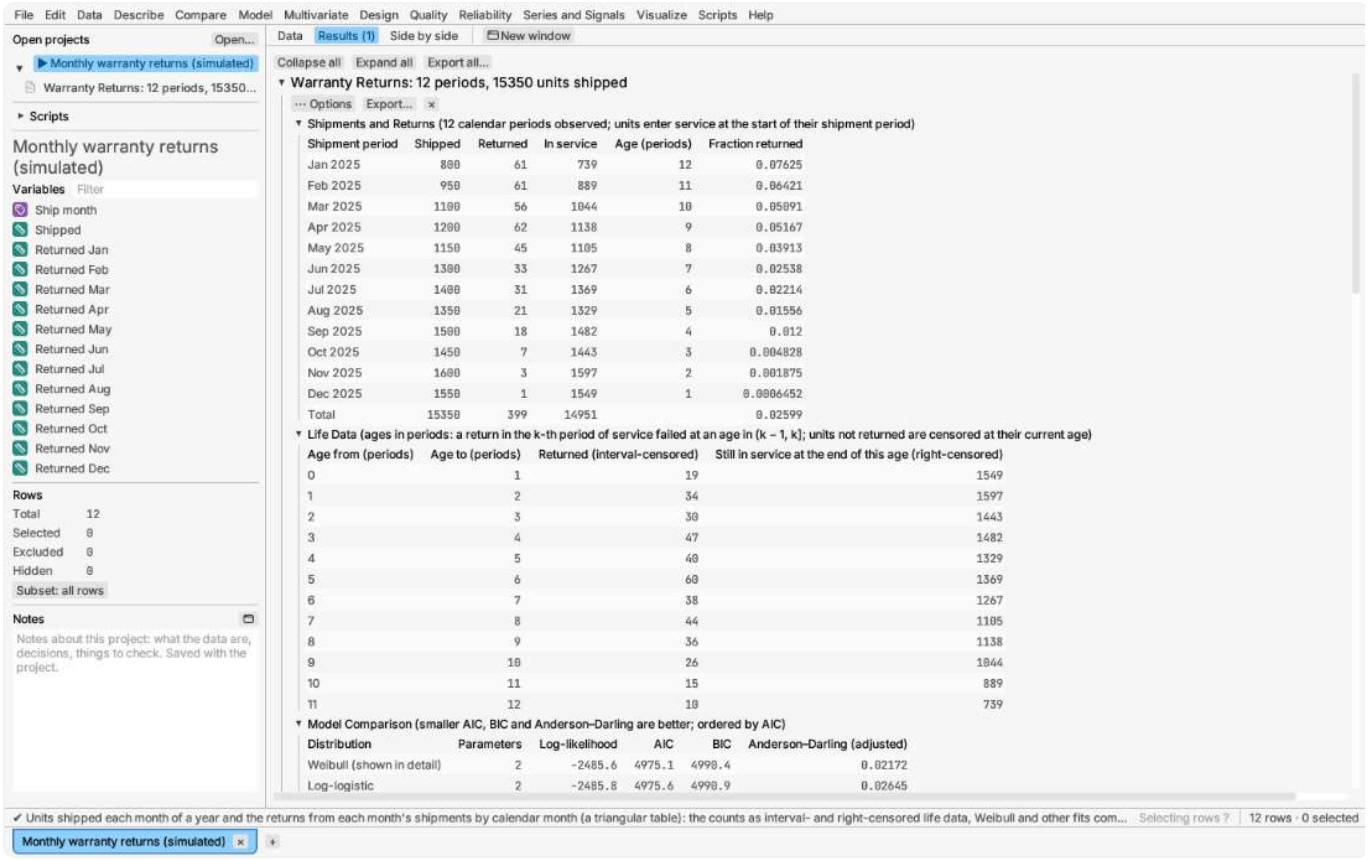
The report has:

- **Shipments and Returns:** per shipment period, units shipped, returned, still in service, their age and the fraction returned;
- the **life data:** returns per age interval and units still in service at each age;
- **Model Comparison**, the **probability plot** and the **parameter estimates** of the chosen distribution, as in Life distributions, with the times by which 1%, 5% and 10% of units fail;
- **Returns per Period:** observed returns per calendar period (markers), the returns expected under the fit, and the forecast after the current period with prediction limits;
- **Forecast Returns per Period:** expected returns in each future period, split into those from units already in service — each cohort's units still in service times its probability of failing in that period given that it has survived to its current age — and those from planned shipments, with prediction limits;
- **Cumulative Forecast Returns:** the total from the next period through each future period, with limits.

The prediction limits are a normal approximation: the variance adds the binomial variation of the returns (for known parameters) to the uncertainty of the expected count from estimating the distribution (delta method), and the lower limit is never below zero.

Try it: Monthly warranty returns — a year of shipments with returns by month and six months of planned shipments.

Verified: the fits and forecasts against an independent SciPy implementation of the likelihood built from the counts, and the forecast by hand for the exponential distribution (see the validation record).

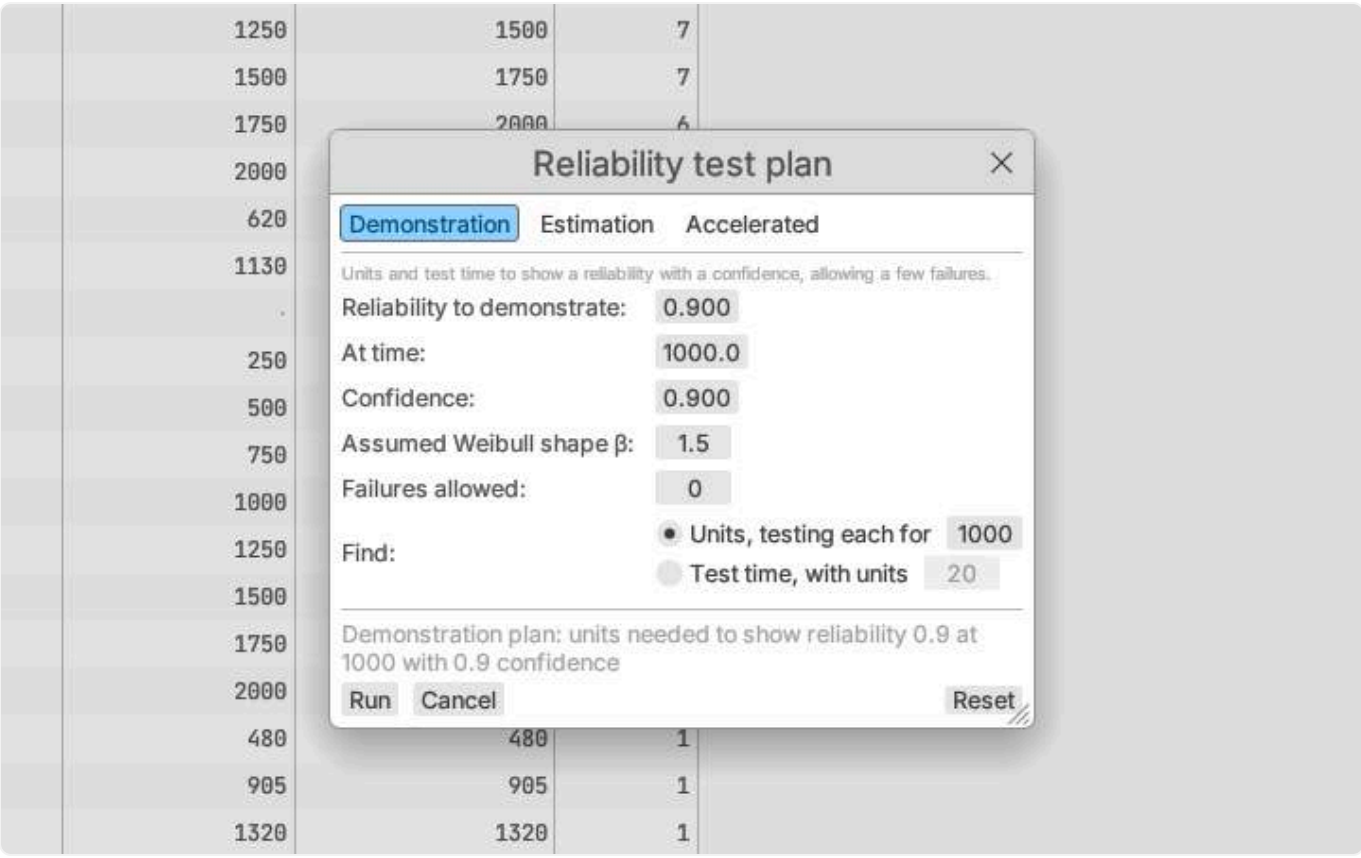


Test plans

Reliability ▸ Test plans… plans a life test before you run it; it does not read the data. Three kinds of plan:

- **Demonstration:** how many units to test for a given time (or how long to test a given number of units) to show, with a confidence, that the reliability at a time is at least a target, allowing 0, 1, … failures, for a Weibull life of known shape. The report also lists the plans for other numbers of allowed failures and plots the chance of passing against the true reliability.
- **Estimation:** how many units are needed to estimate a percentile (say the B10 life) of a Weibull or lognormal life within a precision factor R (an interval from $\hat{t}/R$ to $\hat{t}\cdot R$) when the test stops at a time by which a given fraction will have failed.
- **Accelerated:** for a test at raised stress (temperature with the Arrhenius relationship, voltage with inverse power, or a linear stress), the best two-level plan and the 4:2:1 compromise plan (three levels, more units at the lower stresses), and optionally your own levels: for each, the stresses, the share of units at each, the expected fraction failing and the units needed for the precision you ask for at use conditions.

Planning values (the Weibull shape or lognormal σ, the failure probabilities) are guesses from experience or earlier tests; trying a few values shows how much the plan depends on them. Open any table first — the menu works on the open document.



Verified: the growth, repairable-system and test-plan calculations against MIL-HDBK-189 and Crow's formulas, Nelson's (2003) and Meeker & Escobar's (1998) published formulas, and independent computations.

Series and Signals: time series

The **Series and Signals** menu analyses series of values recorded at equal intervals: monthly sales, daily temperatures, hourly loads. Its **Explore** items describe a series and its dependence on the past, split it into trend and seasonal parts, fit trend curves, and relate two series at a delay. Its **Forecasting models** items (exponential smoothing and ARIMA) fit a model and forecast the series ahead with prediction intervals. Its **Signals and spectra** items (Fourier spectra, digital filters, convolution and correlation, wavelets) are described in Series and Signals: signal processing.

The series and its time axis

Every time series dialog asks for:

- **Series columns** — numeric columns, one report each. The rows are taken in table order, so sort the table by time first.
- **Time column** (optional) — a date-time or numeric column that labels the time axis (dates are ticked on the calendar) and the time stamps of forecasts. It is not used in the computations: the methods assume equal spacing. A date-time column in the table is chosen for you; choose *None* to number the observations 1, 2, …. Its values must run in time order. When they run backwards down the whole table (newest first, as many exports are), the rows are taken from the bottom up and the report's summary says so. Any other order, or a repeated time, is an error naming the first row out of order (*the time column … is not in time order at row N*): sort the table by the time column first.
- **Observations per seasonal cycle** — the period of the seasonal pattern. By default it follows the time column's spacing: monthly dates give 12, quarterly 4, weekly 52, daily 7 and hourly 24 (a numeric column in steps of 1/12 or 1/4 gives 12 or 4); anything else gives 1, no seasonality. Set it yourself for other cycles, such as 5 for working days.

Rows that are excluded or missing at the start or end of the series are left out (the report's summary says how many). An excluded or missing row in the middle is a gap: the report says *the series has a gap at row N* and shows nothing else, because these methods need equally spaced, complete data. Fill the gap (Data ▸ Impute missing values…, or by hand) or include the row again.

Series description

Series and Signals ▸ Series description… is the place to start with a new series.

Under the series choices you can transform it first:

- **Box–Cox transformation** — `(y^λ − 1)/λ`, the logarithm at λ = 0, for a series whose swings grow with its level (positive values only). Give λ, or let **Guerrero's method** choose the λ in [−1, 2] that makes each seasonal cycle's spread most nearly proportional to its level. The cycles are counted back from the last observation, so when the length is not a whole number of cycles the incomplete cycle at the start is left out.
- **Differences** — replace each value by its change from the previous one (d times), and **seasonal differences** — by its change from the same season a cycle earlier (D times). Differencing removes a trend or a seasonal pattern that wanders, which the unit-root tests below help judge.

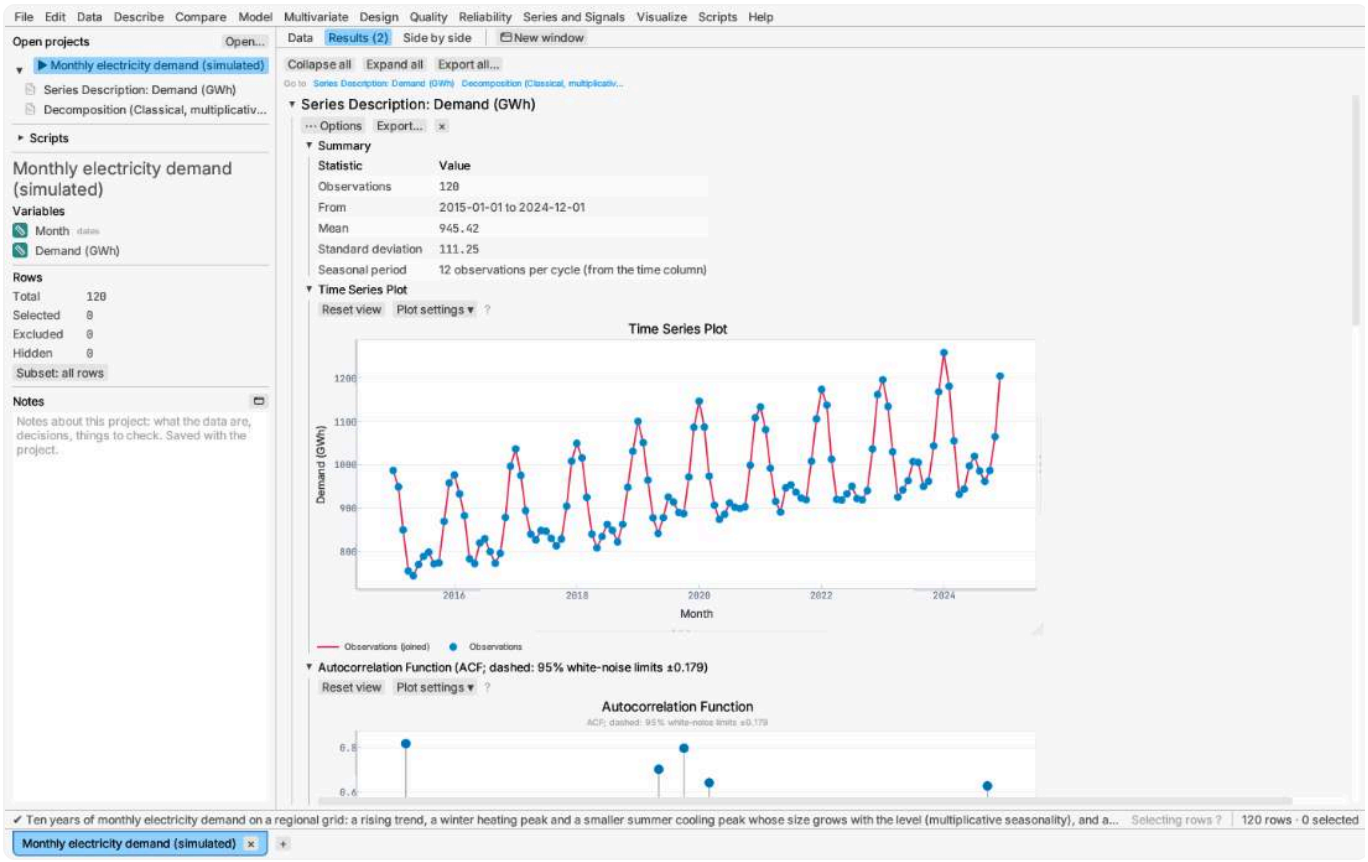
The report has:

- **Summary** — observations, time span, mean and standard deviation, the period, and what the transformation did;
- **Time Series Plot** — the values against time, joined in order. The points are the table's rows, so selecting them selects rows everywhere;
- **Transformed Series** (when transformed or differenced);
- **Autocorrelation Function (ACF)** and **Partial Autocorrelation Function (PACF)** — a needle for each lag with dashed limits. A needle beyond the limits is a correlation unlikely for white noise (values unrelated to earlier ones). The limits are ±z/√n; ticking **Bartlett limits** widens the ACF's limits with the lag, as appropriate for judging whether a moving average of lower order explains the correlations. A slowly decaying ACF suggests a trend; spikes at the period and its multiples suggest seasonality. By default lags run to $10 \cdot \log_{10} n$, and at least two seasonal cycles;
- **Autocorrelations** — the values in a table;
- **Ljung–Box Tests** — of no autocorrelation up to lags of one to four seasonal cycles (or 5, 10, 15, 20 without a period). A small p-value means the values depend on earlier ones;
- **Unit-Root and Stationarity Tests** — the augmented Dickey–Fuller test (null: a unit root, the series wanders and needs differencing; lagged differences chosen by AIC; deterministic terms none, a constant, or a constant and trend) and the KPSS tests (null: stationary around a level, or around a trend). Each row gives the statistic, the lag, the p-value and a conclusion at the 5% level in words. The two kinds of test have

opposite null hypotheses, so a series is most clearly stationary when ADF rejects and KPSS does not. KPSS p-values come from a table and are shown as ≤ 0.01 or ≥ 0.10 beyond it.

The ⋯ **Options** menu turns the unit-root tests on or off, picks the Dickey–Fuller terms, switches Bartlett limits, and adds the **Spectral Density**: the periodogram (markers) and a smoothed spectrum (line; smoothing by modified Daniell kernels, from none to very strong) with its confidence limits on a log scale, against frequency (cycles per observation) or period (observations per cycle). Thin rules mark the seasonal frequency and its multiples, and a table lists the largest peaks: a peak at period 12 in monthly data is a yearly cycle. The series is detrended (least-squares line) and tapered (10% at each end) first.

Try it: Monthly electricity demand — ten years of monthly demand with a rising trend and winter and summer peaks.



Decomposition

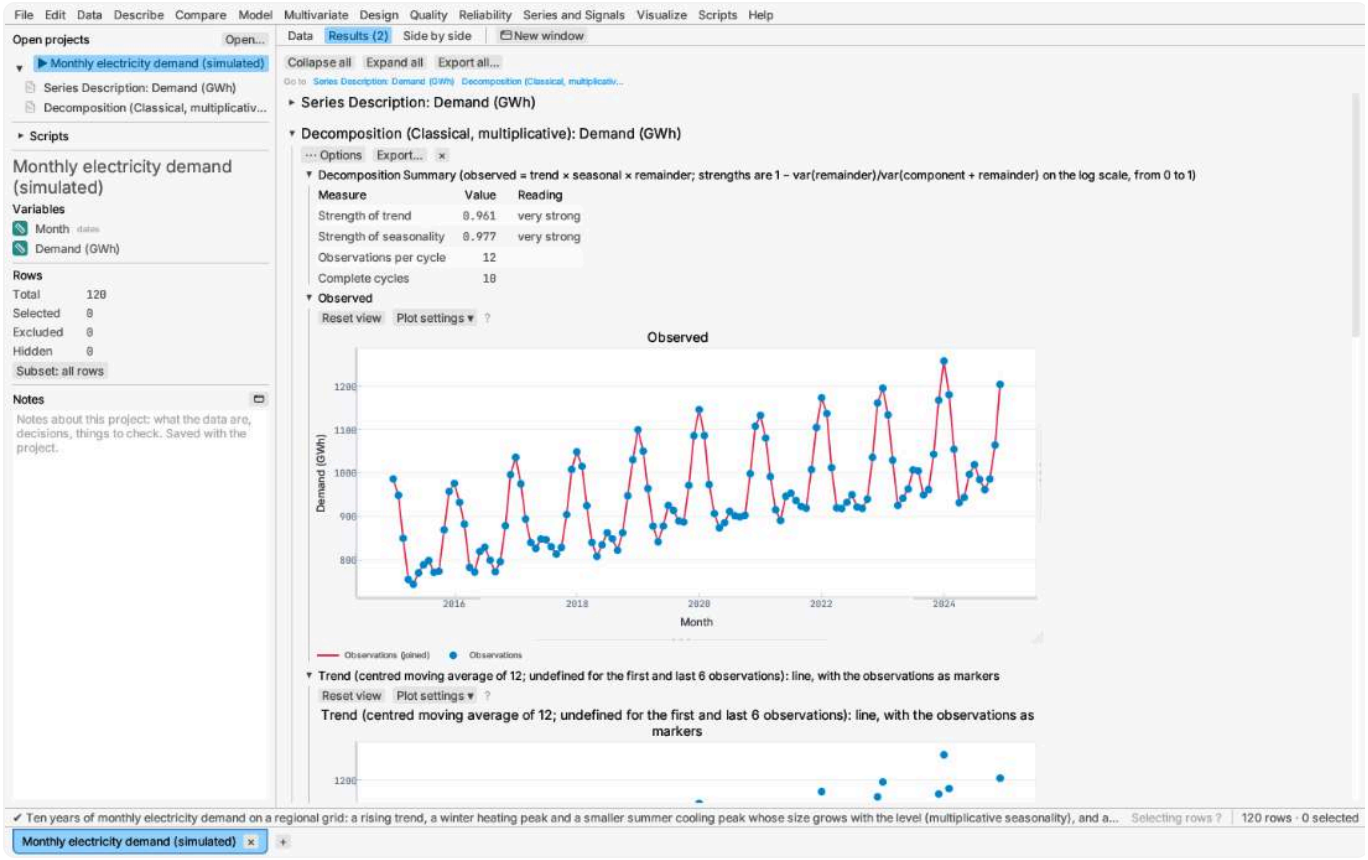
Series and Signals ▸ Decomposition… splits a seasonal series (at least two observations per cycle and two full cycles) into a **trend**, a **seasonal** pattern and a **remainder**:

- **Classical, additive** — the trend is a centred moving average over one cycle; the seasonal effect of each position in the cycle is the average of the detrended values there (the effects sum to zero); `observed = trend + seasonal + remainder`. The moving average is undefined for half a cycle at each end.
- **Classical, multiplicative** — the same with ratios: seasonal factors that average 1, for swings that grow with the level (positive values).
- **STL** — seasonal-trend decomposition by LOESS (Cleveland et al. 1990): the trend reaches the ends, and with a seasonal **span** (odd, 7 or more) instead of *the same every cycle* the seasonal pattern may change slowly. **Robust** downweights outliers so they stay in the remainder.

The report shows the **Decomposition Summary** (the strengths of trend and of seasonality, `1 − var(remainder)/var(component + remainder)`, from 0 for none to 1 for a component that dominates the noise; computed on the log scale for a multiplicative decomposition), then panels of the **Observed** series, the **Trend** (with the observations as markers), the **Seasonal** component and the **Remainder**, all against the same time axis, and for a classical decomposition the **Seasonal Indices** by month or quarter. Look at the remainder for outliers and leftover pattern.

The **⋯ Options** menu saves the trend, seasonal component, remainder or seasonally adjusted series (observed minus, or divided by, the seasonal component) as new columns.

Try it: Monthly electricity demand — a multiplicative decomposition shows January about 18% above the trend.



Trend models

Series and Signals ▸ Trend models… fits one curve against time, with t = 1 at the first observation:

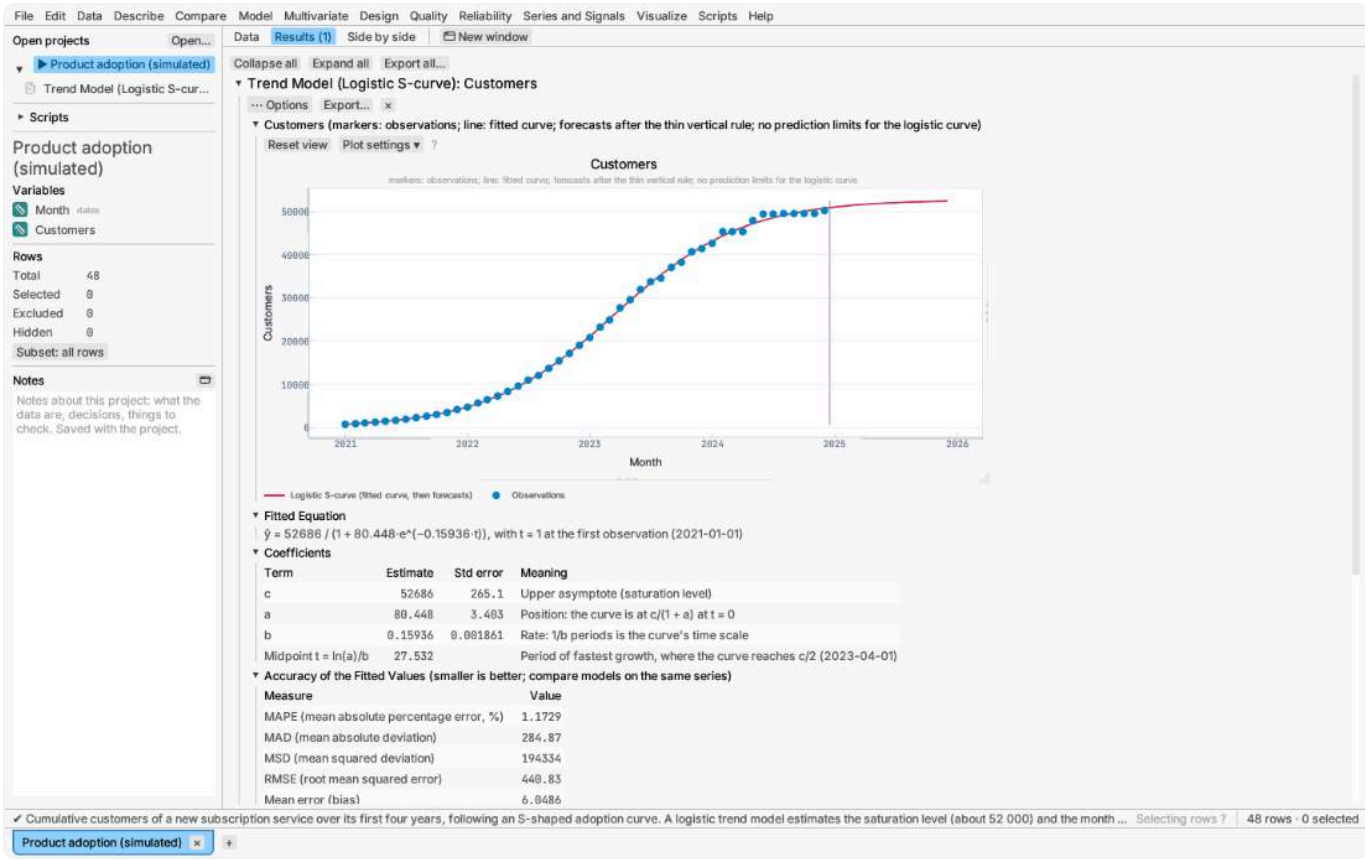
- **Linear** $y = \beta_0 + \beta_1 t$ and **quadratic** $y = \beta_0 + \beta_1 t + \beta_2 t^2$, by least squares;
- **Exponential growth** $y = b_0 \cdot b_1^{\,t}$ — a steady percentage growth of $(b_1 - 1)\cdot 100\%$ per period, fitted by least squares on the log scale (positive values);
- **Logistic S-curve** $y = c/(1 + a \cdot e^{(-bt)})$ — growth that slows towards a ceiling c (adoption, saturation), by nonlinear least squares (positive values); the report adds the midpoint, the period of fastest growth;
- **Moving average** — forecasts every period by the mean of the last k observations; its one-step forecasts within the data are scored.

The report plots the data (markers) with the fitted curve and the forecasts after a thin vertical rule, with prediction limits (dashed) for the linear, quadratic and exponential models; then the **Fitted Equation**, the **Coefficients** with standard errors and what each means, the **Accuracy** of the fitted values (MAPE, the mean absolute percentage error;

MAD, the mean absolute deviation; MSD, the mean squared deviation; RMSE and the mean error), and the **Forecasts** with time stamps that continue the time column (month by month for monthly dates). Compare models on the same series by their accuracy: smaller is better.

The **⋯ Options** menu changes the number of periods forecast and saves the fitted values or residuals as columns. Right-click the forecast table to save it as a data table.

Try it: Product adoption — four years of cumulative customers following an S-curve; the logistic model finds the ceiling and the month of fastest growth.



Cross-correlation

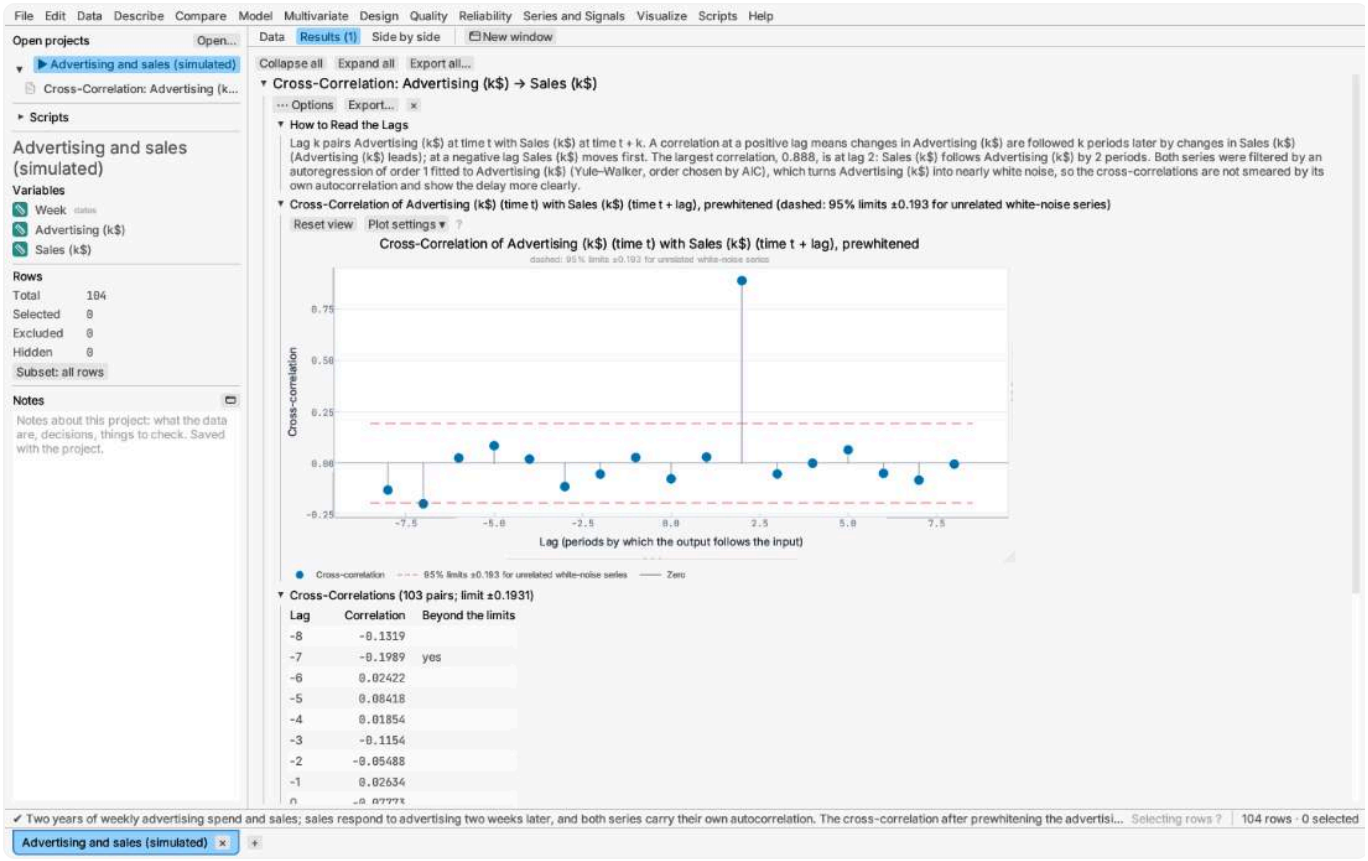
Series and Signals ▸ Cross-correlation… relates an **input** series (one that may lead, such as advertising) to one or more **output** series (such as sales; one report each). The two series use the same rows.

The correlation at **lag k** pairs the input at time t with the output at time t + k. A needle beyond the dashed limits at a **positive lag** means changes in the input are followed k

periods later by changes in the output; at a **negative lag**, the output moves first. The report's first section says this in words for its own columns and names the largest correlation.

When the input is itself autocorrelated, its correlations with the output are smeared over neighbouring lags and the limits are too narrow. **Prewhitening** fits an autoregression to the input (order chosen by AIC, or fixed), filters both series with it, and correlates the filtered series; the report then lists the filter. Turn it on in the dialog or in the report's ⋯ **Options** menu.

Try it: Advertising and sales — two years of weekly data in which sales respond to advertising two weeks later: the prewhitened cross-correlation has its peak at lag 2.



Forecasting models

Both forecasting reports share these settings and sections:

- **Forecast steps ahead** (12 by default) and **Prediction intervals** as percentages (80 and 95 by default).

- **Forecast:** the observations (circles, which you can select and brush like any plot's points), the one-step fitted values as a solid line that continues as the forecast (diamond markers) after a thin vertical rule, the widest prediction limits dashed and narrower ones as thin lines. With a date-time column the axis shows dates, and forecasts continue the calendar (whole months for monthly data).
- **Forecasts:** each step ahead with its time, the forecast, its standard error and the limits at each level.
- **Residuals over Time**, **Residual Autocorrelation** with dashed 95% white-noise limits (±1.96/√n), and the **Ljung–Box Test** pooling 2m lags for a seasonal series (10 otherwise, at most n/5), with the fitted model parameters taken off its degrees of freedom. A small p-value means the model has left structure in the residuals.
- **In-Sample Accuracy:** RMSE, MAE and MAPE of the one-step fitted values.

Saving forecasts. ⋯ *Options* ▸ *Save forecasts as data table* adds a table to the project with the time, the actual and one-step fitted values for each observation, then one row per forecast with its standard error and limits. The table is computed again from the report's settings when you save it; it does not update when the data change — save it again.

Exponential smoothing

Series and Signals ▸ Exponential smoothing… forecasts from a level, an optional trend and optional seasonal effects, each updated as observations arrive. Choose the model:

- **Automatic (smallest AICc):** every model in the error–trend–seasonal family is fitted and the one with the smallest AICc kept. Multiplicative errors and seasonality are considered only when every value is above zero (untick *Consider multiplicative errors and seasonality* to leave them out); untick *Consider damped trends* to leave those out. Seasonal models need a period of at least 2 and two full cycles of data. Additive errors with multiplicative seasonality are not considered, because their forecast variance can be infinite.
- **Simple exponential smoothing** (a level only), **Holt linear trend**, **Damped trend** (a trend that flattens out), **Holt–Winters, additive seasonality** and **Holt–Winters, multiplicative seasonality** (seasonal swings that grow with the level; multiplicative errors).
- **Custom combination:** additive or multiplicative errors; no, additive or damped trend; no, additive or multiplicative seasonality.

For a chosen model, *More options* can hold α, β, γ or φ at a value instead of estimating it.

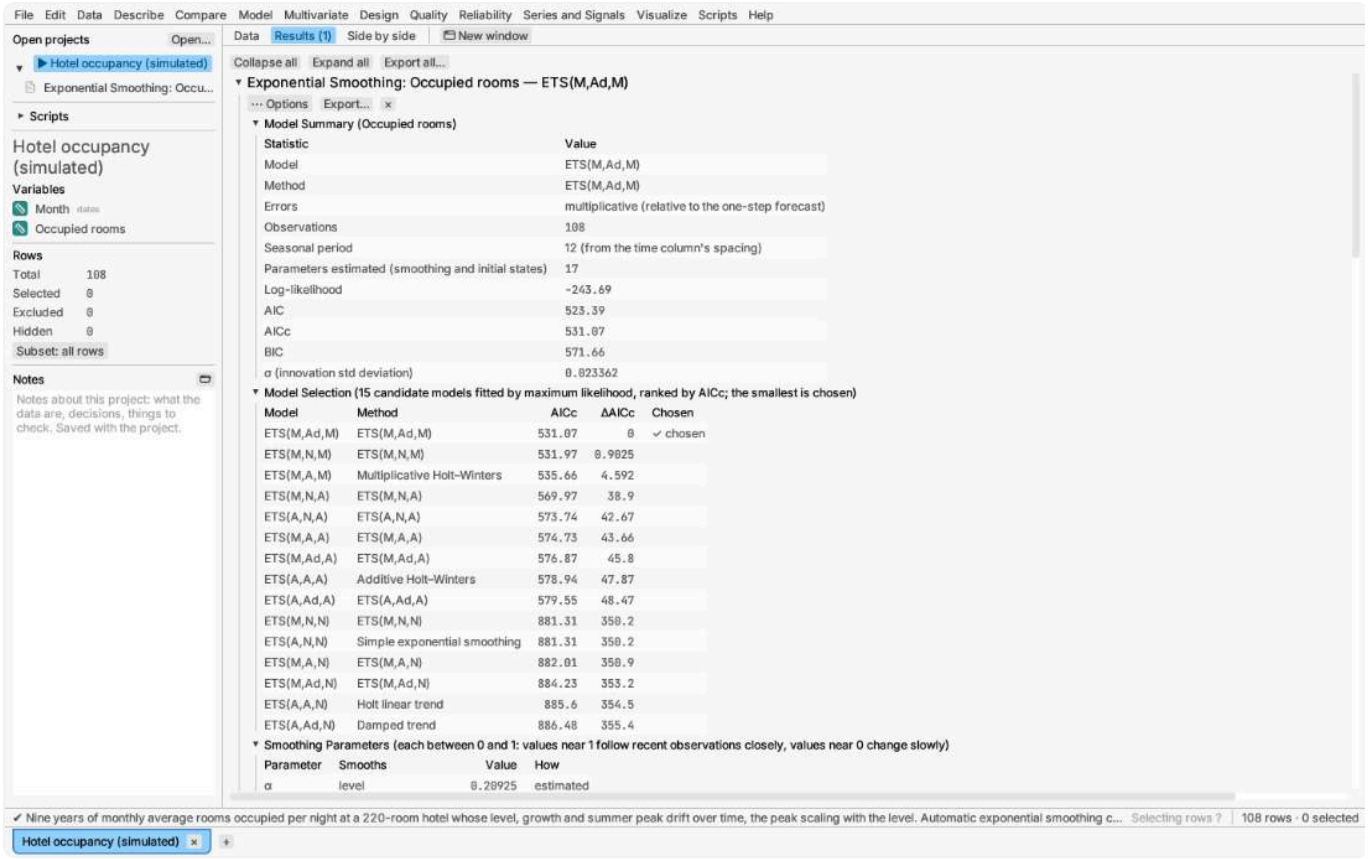
The report has, besides the shared sections:

- **Model Summary:** the model's label (such as ETS(M,A,M)) and plain name, the seasonal period and how it was chosen, the log-likelihood, AIC, AICc, BIC and the innovation standard deviation σ;
- **Model Selection** (automatic only): every candidate with its AICc and its distance from the best, the chosen one marked "✓ chosen";
- **Smoothing Parameters:** α (level), β (slope), γ (seasonal) and φ (damping), each marked estimated or fixed;
- **Initial States:** the starting level, slope and the seasonal effects (summing to 0) or factors (averaging 1) of the first cycle.

With multiplicative seasonality there is no formula for the forecast variance, so the limits are quantiles of 5000 simulated sample paths (with a fixed seed, so they are the same every time) and the forecast table adds the mean of the simulated paths.

Try it: Hotel occupancy — nine years of monthly rooms occupied, whose level, growth and summer peak drift over time, the peak scaling with the level; the automatic choice is multiplicative seasonality.

Verified: the engine against published and independently computed exponential smoothing fits (see the validation record).



ARIMA

Series and Signals ▸ ARIMA… fits a seasonal ARIMA model by exact maximum likelihood: the series is differenced d times (and D times at the seasonal lag) and modelled with p autoregressive and q moving-average terms (P and Q at the seasonal lag).

- **Automatic search:** the differencing is chosen by KPSS tests (d) and the strength of seasonality (D), then the stepwise search of Hyndman and Khandakar tries neighbouring orders, keeping the smallest AICc. Untick *Stepwise search* to fit every model up to a total order of 5, or *Consider a mean or drift term* to leave out the constant.
- **Choose:** the regular orders (p, d, q) and seasonal orders (P, D, Q), which use the seasonal period, and whether to include a mean (no differencing) or a drift (one difference in all). With two or more differences a constant is not allowed.

More options offers a **Box–Cox transformation** before fitting (positive data only): a λ you give (0 is the logarithm) or Guerrero's choice. The forecasts and limits are transformed back, so the point forecast is then the median of the forecast distribution, and the standard errors are on the transformed scale. A Box–Cox transform with $\lambda > 0$ cannot go below $-1/\lambda$, but a wide prediction interval can; such a lower limit is shown as 0 (and with $\lambda < 0$ an upper limit beyond the transform's range as ∞, drawn at the top of the plot), and the table's title says so.

The report has, besides the shared sections:

- **Model Summary:** the model label (such as ARIMA(1,1,1) or ARIMA(0,1,1)(0,1,1)[12] with drift), observations used after differencing, the seasonal period, the Box–Cox λ, σ^2, σ, log-likelihood, AIC, AICc and BIC, and the smallest root modulus of the AR and MA parts (values near 1 warn of a near unit root);
- **Model Search** (automatic only): every model tried with its AICc, the order tried and the chosen one marked;
- **Coefficients:** each term (ar1, ma1, sar1, sma1, intercept or drift) with its estimate, standard error, t ratio and p-value.

The residuals are the one-step innovations scaled to a common variance; the first d + mD observations have none.

Input series (transfer functions)

When another column drives the series (fuel feed and a kiln's temperature, advertising and sales), open **Input series (transfer functions)** and tick it. Each input acts on the series

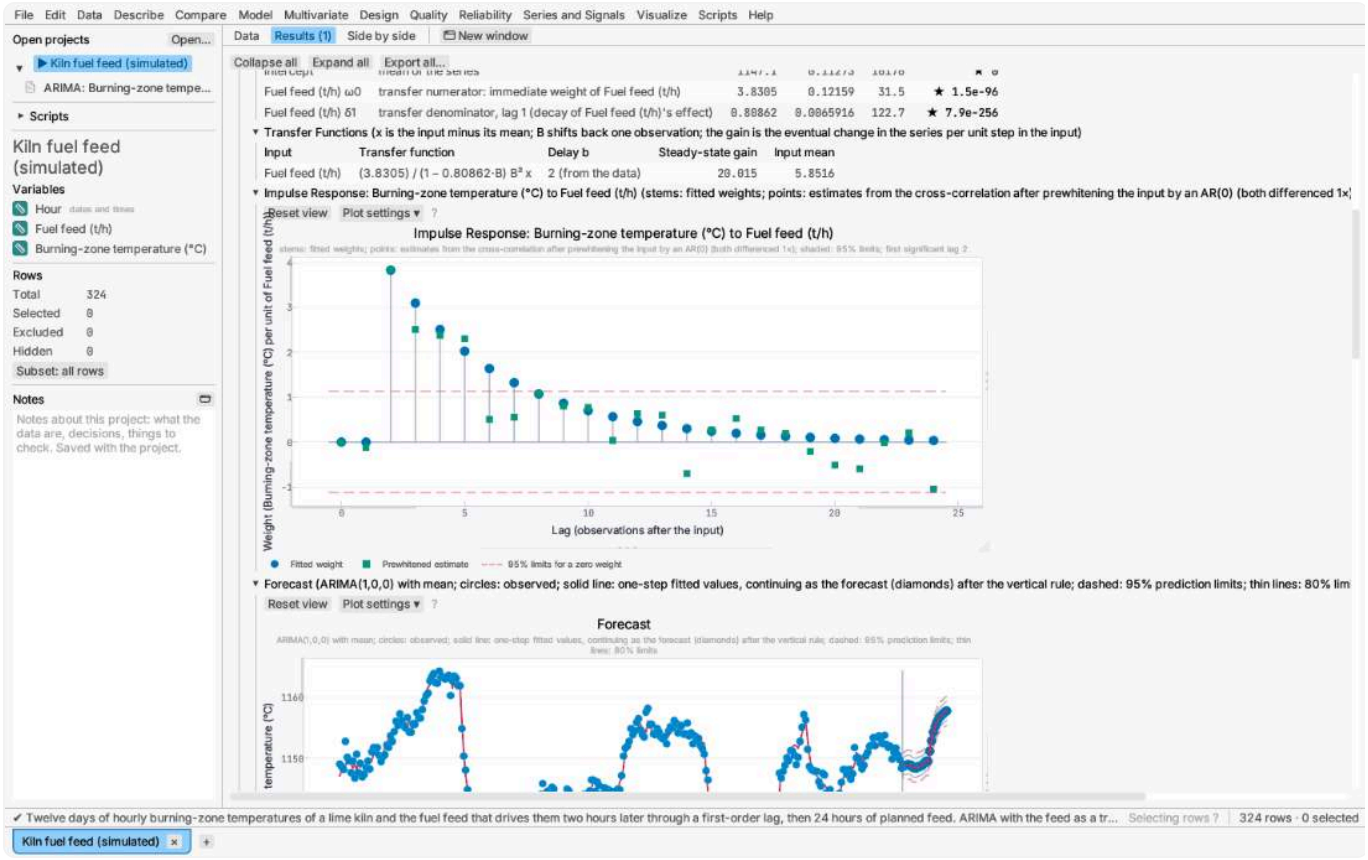
through a *transfer function*: after a **delay** of b observations, a **numerator** of s + 1 weights on the input's recent values and a **denominator** of order r that lets the effect build up or die away geometrically (Box, Jenkins, Reinsel and Ljung, chapters 11–12). The defaults, delay *From the data*, numerator 0 and denominator 1, describe an effect that starts after the delay and settles to a new level. The ARIMA orders (chosen or searched) then model what the inputs leave unexplained.

- **Delay from the data:** the input is prewhitened (an autoregression turns it into near white noise, and the series is filtered the same way), and the delay is the first lag at which the two correlate significantly. Both are differenced as the model differences the series first.
- **Rows used:** the rows where the series and every input have values. Rows after the series with the inputs filled in (and the series empty) are the inputs' planned future values; forecasts use them, so a forecast can reach as far ahead as those rows plus the delay. With fewer rows the forecasts stop early and the Model Summary says so.
- The input is used as deviations from its mean, and its values before the first row count as that mean, so the fitted effect has no start-up jump.

With inputs the report adds:

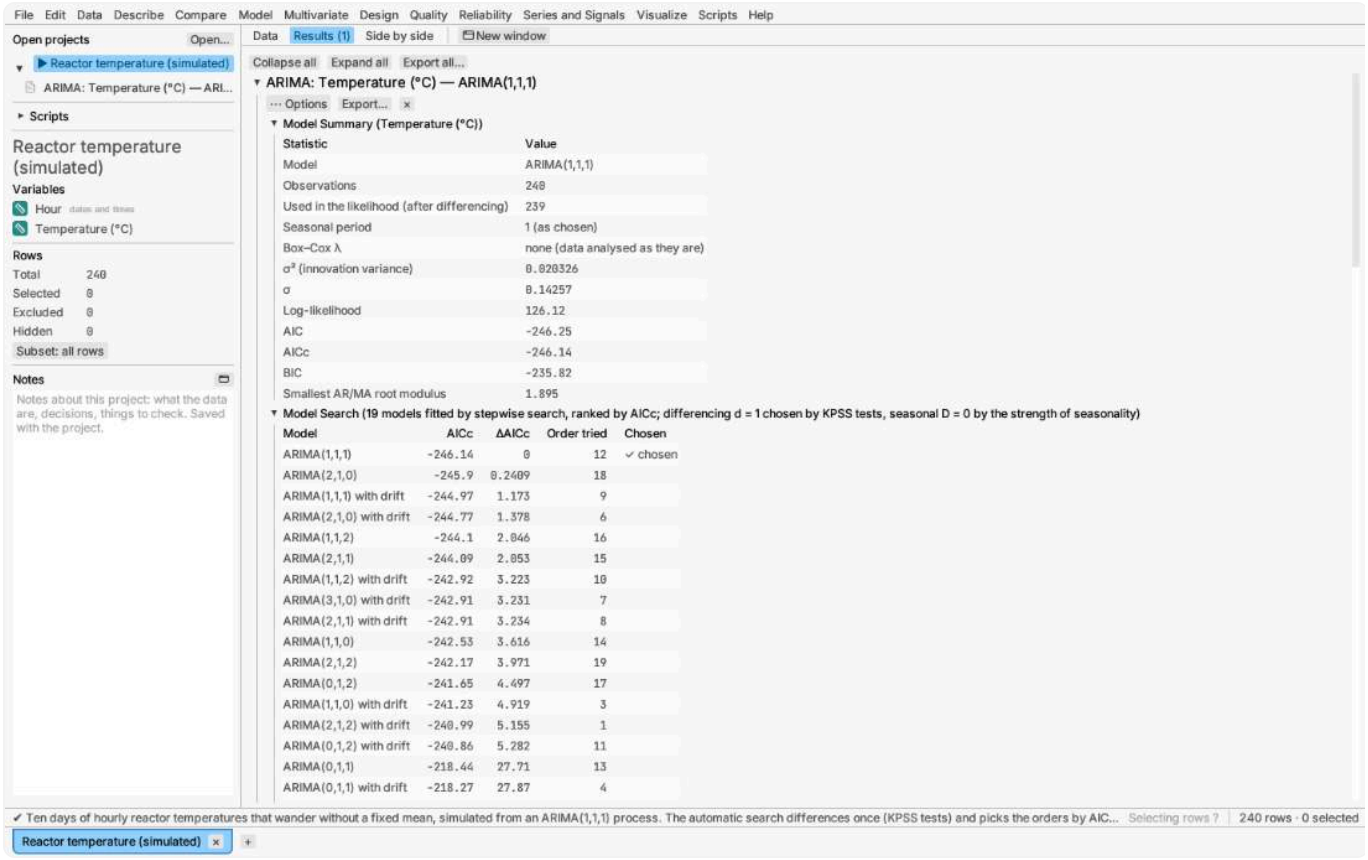
- **Transfer Functions:** each input's fitted transfer function written out, its delay, the **steady-state gain** (the eventual change in the series per unit step in the input) and the input's mean; the weights (ω0, ω1, … and δ1, …) are also in the Coefficients table with standard errors;
- **Impulse Response** for each input: the fitted response of the series to a one-off unit pulse in the input (stems), the same weights estimated from the prewhitened cross-correlation (points) and 95% limits for a zero weight, so you can see whether the chosen delay and orders follow the data;
- **Residual Cross-Correlation with Prewhitened** input: the residuals should not correlate with the input's past; the title gives the summary test S with its p-value, and a small p-value means the transfer function misses part of the input's effect (try another delay or more weights).

Try it: Kiln fuel feed — twelve days of hourly burning-zone temperatures and the fuel feed that drives them two hours later, followed by a day of planned feed. The report finds the delay of 2 hours, a gain of about 20 °C per t/h, and forecasts the planned day.



Try it: Reactor temperature — ten days of hourly temperatures that wander without a fixed mean; the automatic search differences once and picks ARIMA(1,1,1), forecasting the next 24 hours.

Verified: the engine against reference fits (see the validation record).



Scripts

Each Explore and forecasting report can be made from a script with `analyze`, for example

```
let t = current_table();
t.analyze(#{analysis: "decomposition", column: "Demand (GWh)", time: "Month",
method: "stl", robust: true});
```

Forecasting reports use the keys `exponential-smoothing` and `arima`. The settings of each analysis are listed in the scripting reference.

Series and Signals: signals and spectra

The **Signals and spectra** items at the foot of the **Series and Signals** menu work on curves measured point by point: an accelerometer or sensor trace, a chromatogram, a Raman, infrared or X-ray spectrum, a titration curve. They smooth a noisy signal, take its derivative and running integral, resample it onto even spacing, remove a background, find peaks and separate overlapping peaks by fitting a sum of peak shapes.

Menu item	What it does
Smoothing…	Savitzky–Golay, moving-average, exponential, running median or percentile, and Gaussian kernel smoothing, with the raw and smoothed signals overlaid
Derivatives and integrals…	Derivative (central differences or Savitzky–Golay), running integral (trapezoid or Simpson's rule), and resampling onto an even grid (linear, cubic spline or shape-preserving interpolation)
Baseline correction…	The spectrum report with the baseline methods in front: asymmetric least squares, arPLS, SNIP or a known baseline column
Peak finding…	The spectrum report: peaks above the noise by height, prominence and separation
Peak fitting…	The spectrum report with a peak model fitted: centres, heights, widths and areas with standard errors, a comparison of peak counts, and one parameter table for many curves

Baseline correction, peak finding and peak fitting are one report opened with different defaults; any of them can do all three.

Columns and curves

Every dialog asks for:

- **y** — the measured values (signal, intensity, absorbance);
- **x** — the position (time, wavenumber, wavelength, retention time). For smoothing it is optional: without it the rows are taken in table order;

- **One signal (curve, spectrum) per** — an optional column whose levels each hold one curve, for data stacked in long form. Every curve is analysed on its own, and the tables list them all; plots are drawn for the first eight.

The points of each curve are sorted by x. Rows with the same x in one curve are averaged into a single point (the report's notes say so). Excluded rows and rows with a missing x or y are left out.

The smoothing filters and the Savitzky–Golay derivative assume evenly spaced points, as recorded by an instrument; central differences, the integrals and interpolation accept any spacing.

Smoothing

Series and Signals ▸ Smoothing… offers:

- **Savitzky–Golay** (Savitzky & Golay 1964): at each point a polynomial of the chosen *degree* is fitted by least squares to the *window* of points centred on it, and its value there is the smoothed value. It keeps peak heights and widths far better than a moving average of the same length. The window is odd and larger than the degree; near the ends, a single polynomial is fitted to the first (last) window. A *derivative* of 1 or 2 gives the fitted polynomial's slope or curvature per x unit instead (using the x spacing).
- **Moving average** — centred (the window around each point, odd) or trailing (each point and the ones before it; fewer at the start).
- **Exponential** — $s_i = \alpha \cdot y_i + (1 - \alpha) \cdot s_{i-1}$, starting from the first value.
- **Running median** (Tukey 1977) — the median of the window; it ignores isolated spikes that a mean would smear. **Running percentile** takes a chosen percentile of the window instead (the value of rank $\lfloor w \cdot p/100 \rfloor$ in the sorted window, from 0).
- **Gaussian kernel** — a weighted mean with weights $\exp(-j^2/2\sigma^2)$ for offsets j up to $\lfloor 4\sigma + ½ \rfloor$ points, σ in points.

Centred filters see the signal mirrored beyond its ends (… $y_2\ y_1\ y_0$ | $y_0\ y_1\ y_2$ …).

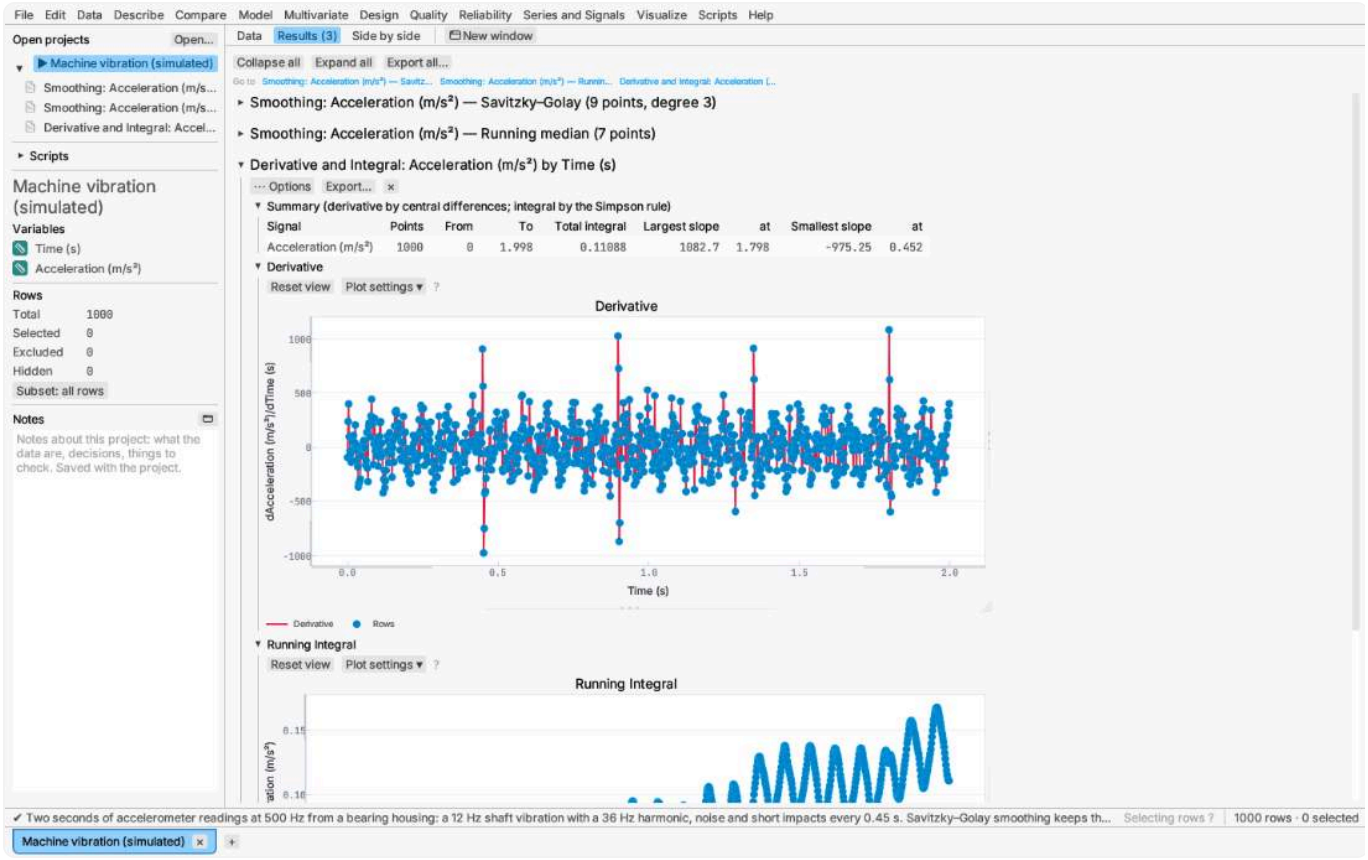
The report has a **Summary** (points, spacing and the standard deviation of the residual raw − smoothed, or of the derivative), a **Raw and Smoothed** plot for each curve (points are the table's rows, so selecting them selects rows everywhere) and, in its options, **Save smoothed values** as a new column. The window, degree and other settings can be changed in the report's options; it recomputes at once.

Derivatives and integrals

Series and Signals ▸ Derivatives and integrals… computes, for each curve:

- the **derivative** dy/dx — by *central differences*, $(y_{i+1} - y_{i-1})/(x_{i+1} - x_{i-1})$ generalized to unequal spacing (second-order accurate, exact for parabolas) with one-sided differences at the ends, or by a *Savitzky–Golay* derivative, which smooths at the same time;
- the **running integral** from the first point — by the *trapezoid rule* or by *Simpson's rule* (a parabola through each interval and a neighbour, exact for cubics on even spacing);
- optionally, the curve **resampled** onto evenly spaced x from its first to its last point, by *linear* interpolation, a *cubic spline* (not-a-knot ends), a *natural cubic spline* (zero curvature at the ends) or the *shape-preserving cubic* (PCHIP, Fritsch & Carlson 1980), which never overshoots the data between points.

The report has a **Summary** (the total integral and the largest and smallest slopes with their x), plots of the **Derivative** and the **Running Integral**, and the **Resampled Values** table (save it as a data table from the table's menu). The derivative and integral can be saved as columns from the options.



Baseline correction

A spectrum often sits on a slowly varying background (fluorescence, scatter, drift). The spectrum report removes it before finding and fitting peaks:

- **arPLS** (Baek et al. 2015, the default) — a smooth curve z minimizing $\Sigma w_i(y_i - z_i)^2 + \lambda\Sigma(\Delta^2 z_i)^2$, where the weights come from a logistic function of the residuals scaled by the noise of the points below the curve, so points on peaks get almost no weight. One setting: the smoothness $\log_{10}\lambda$ (larger is stiffer; 5–7 suits a few hundred to a few thousand points).
- **Asymmetric least squares** (Eilers & Boelens 2005) — the same penalty, with weight p for points above the curve and 1 − p below.
- **SNIP** (Ryan et al. 1988) — statistics-sensitive non-linear iterative peak clipping: each point is replaced by the smaller of itself and the mean of its neighbours k points away, for k up to the *half-window* (decreasing windows by default). Set the half-window a little above half the widest peak's width in points. The *log–log–square-root* option works on $v = \ln(\ln(\sqrt{(y+1)}+1)+1)$ for counts spanning orders of magnitude.
- **Known** — subtract a baseline column measured or computed elsewhere.

- **None** — the data already sit on zero.

The **Spectrum and Baseline** plot shows the spectrum and the estimated baseline; the corrected spectrum and the baseline can be saved as columns.

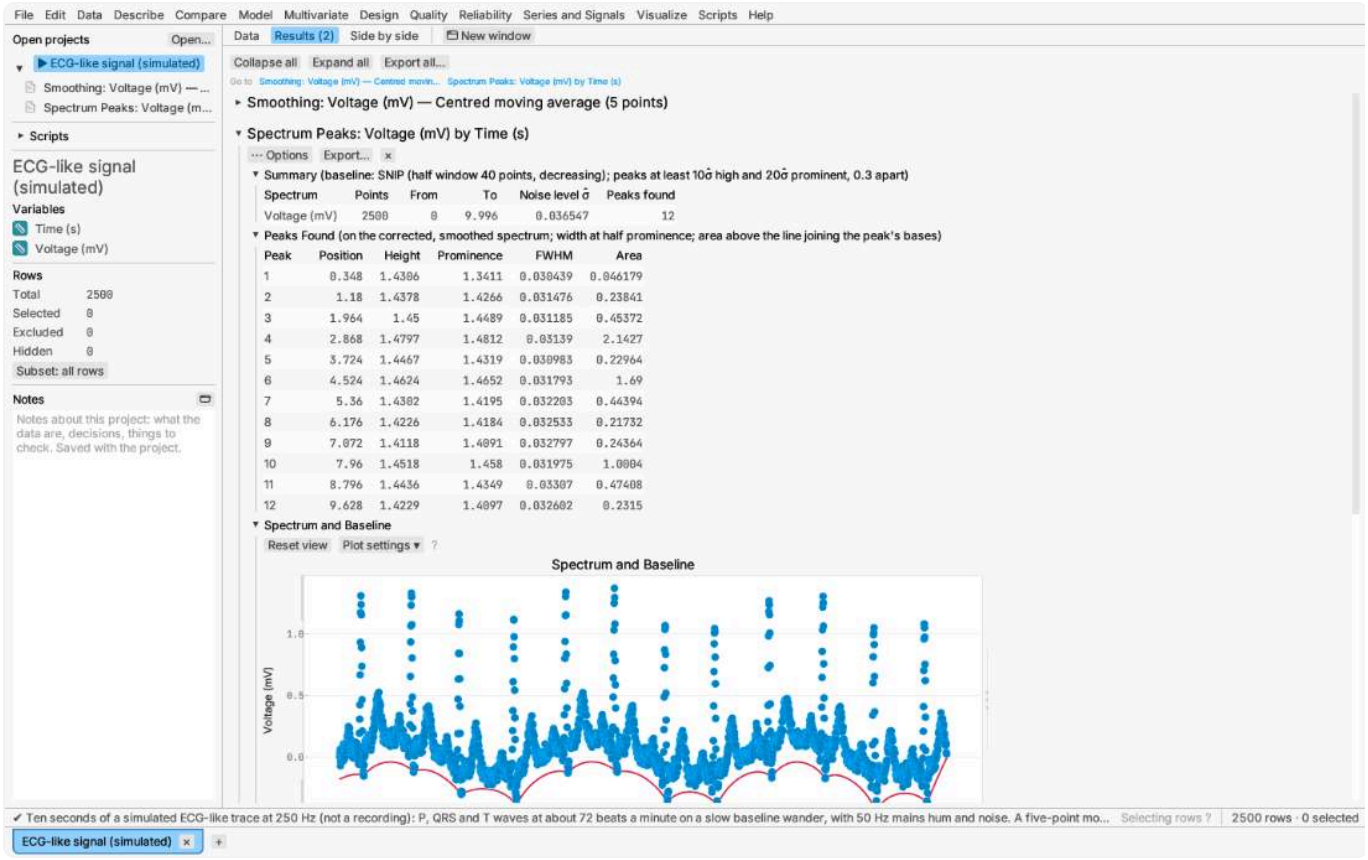
Peak finding

Peaks are local maxima of the corrected spectrum after optional Savitzky–Golay smoothing (11 points, cubic by default), kept when they are:

- at least so many **noise levels high** (3 by default), and
- at least so many noise levels **prominent** (5 by default): prominence is how far a peak rises above the higher of the lowest points between it and a higher peak (or the end) on either side;
- at least the **separation** apart in x units (lower peaks give way).

The noise level $\hat{\sigma}$ is estimated robustly from the successive differences of the corrected spectrum, $\hat{\sigma} = 1.4826 \cdot \mathrm{MAD}(\Delta y)/\sqrt{2}$, but never below 0.1% of its range, so a smooth or noiseless curve does not turn every ripple into a peak. These rules follow SciPy's `find_peaks`.

The **Peaks Found** table gives each peak's position, height, prominence, width at half prominence (interpolated between points) and area above the line joining its bases; the **Peaks** plot marks them on the corrected spectrum.



Peak fitting

With **Fit a peak model** ticked (the default for Peak fitting…), the corrected spectrum is fitted by a sum of peaks plus a baseline term by Levenberg–Marquardt least squares:

Shape	Profile (height h, centre c)	Width parameters
Gaussian	$h \cdot \exp(-4 \ln 2\,(x - c)^2/w^2)$	FWHM w
Lorentzian	$h / (1 + 4(x - c)^2/w^2)$	FWHM w
Pseudo-Voigt	weighted sum of a Gaussian and a Lorentzian of the same FWHM and area; η is the Lorentzian share of the area	FWHM w, share η (0–1)
Voigt	the convolution of a Gaussian and a Lorentzian, computed from the Faddeeva function (Weideman 1994)	Gaussian FWHM, Lorentzian FWHM

Shape	Profile (height h, centre c)	Width parameters
Split Gaussian, split Lorentzian	different widths left and right of the centre (asymmetric peaks)	left and right FWHM

- **Baseline under the peaks** — none, a constant or a straight line, fitted with the peaks (on top of the baseline already removed).
- **Number of peaks** — every peak found (at most 12) or the most prominent few; or type **starting centres**. The starts (centre, height, width at half prominence) come from the first curve; each other curve starts from the same centres and widths with its own heights.
- **One width for every peak** — the peaks share their width parameters.
- **Centres** — free, **equally spaced** (a first centre and a spacing are estimated) or **spaced by** a fixed distance.

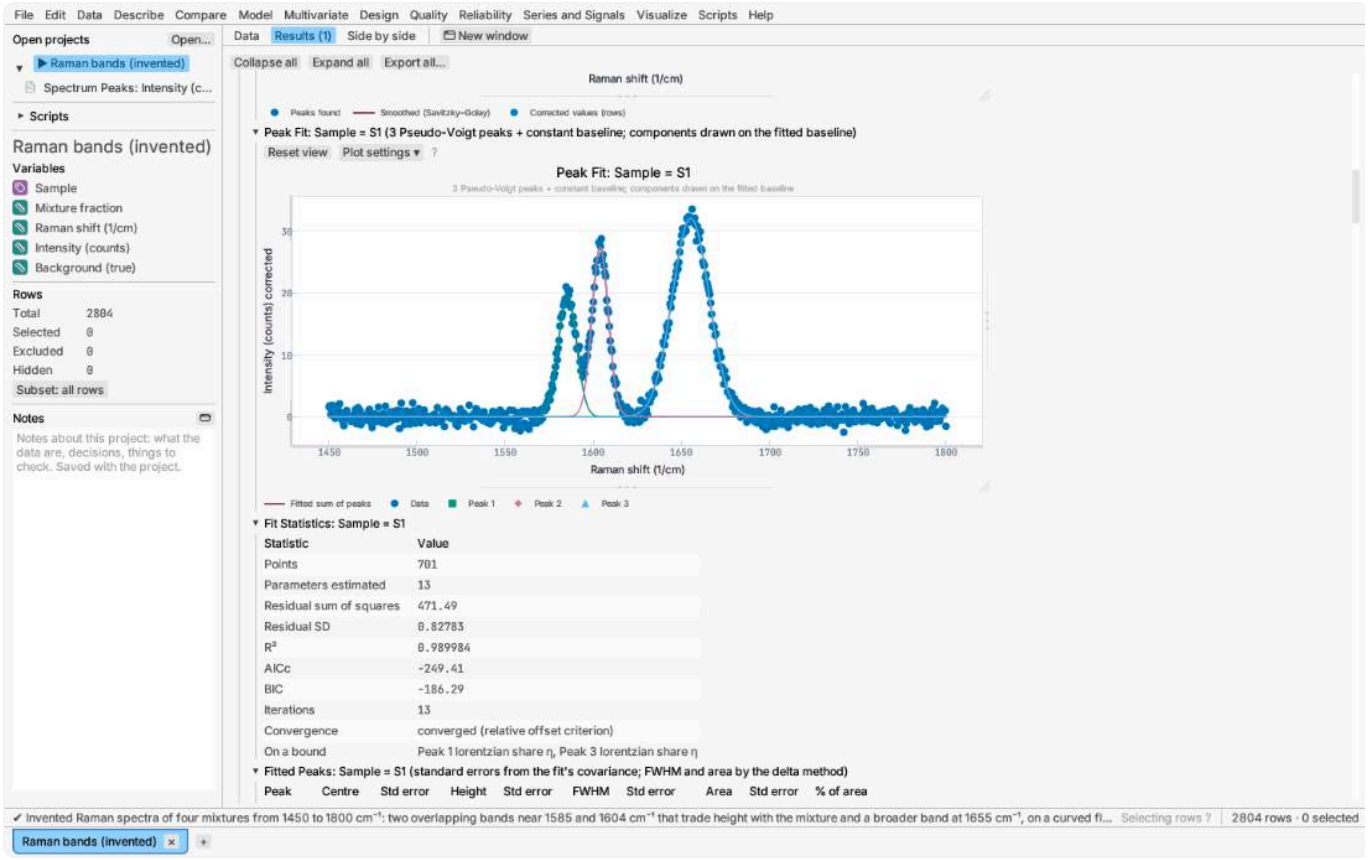
Heights stay positive, widths positive and below twice the x range, centres within the x range and η within 0–1. A parameter that ends on a bound is listed under *On a bound* in the fit statistics and has no standard error.

The report has, for each curve:

- **Peak Fit** — the corrected points, the fitted curve and each component drawn on the fitted baseline;
- **Fit Statistics** — points, parameters, residual sum of squares and SD, R^2, AICc and BIC, and how the iterations ended;
- **Fitted Peaks** — centre, height, FWHM and area of each peak with standard errors (from the fit's covariance, $s^2(J^TJ)^{-1}$; FWHM and area by the delta method for shapes where they combine parameters), and each peak's share of the total area;
- **Parameter Estimates** — every parameter with its standard error;
- **Residuals** against x;
- **Numbers of Peaks** (unless switched off) — the fits with the 1, 2, … most prominent peaks, each with its AICc ($2k - 2 \ln L$ plus the small-sample correction, counting σ as a parameter), ΔAICc from the best, and the extra-sum-of-squares F test against one peak fewer.

With a curve column, **Peak Parameters by Curve** lists every curve's centres, heights, FWHMs and areas side by side: save it as a data table from the table's menu to analyse the parameters further. The fitted values and residuals can be saved as columns.

The shape, baseline term, comparison and common width can be changed in the report's options.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Raman bands (invented)
Spectrum Peaks: Intensity (c...
Scripts
Raman bands (invented)
Variables
Sample
Mixture fraction
Raman shift (1/cm)
Intensity (counts)
Background (true)
Rows
Total 2804
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (1)
Side by side
New window
Collapse all
Expand all
Export all...
Raman shift (1/cm)
Peaks found
Smoothed (Savitzky–Golay)
Corrected values (rows)
Peak Fit: Sample = S1 (3 Pseudo-Voigt peaks + constant baseline; components drawn on the fitted baseline)
Reset view
Plot settings
Peak Fit: Sample = S1
3 Pseudo-Voigt peaks + constant baseline; components drawn on the fitted baseline
Intensity (counts) corrected
Raman shift (1/cm)
Fitted sum of peaks
Data
Peak 1
Peak 2
Peak 3
Fit Statistics: Sample = S1
Statistic Value
Points 701
Parameters estimated 13
Residual sum of squares 471.49
Residual SD 0.82783
R^2 0.989984
AICc -249.41
BIC -186.29
Iterations 13
Convergence converged (relative offset criterion)
On a bound Peak 1 lorentzian share η, Peak 3 lorentzian share η
Fitted Peaks: Sample = S1 (standard errors from the fit's covariance; FWHM and area by the delta method)
Peak Centre Std error Height Std error FWHM Std error Area Std error % of area
✓ Invented Raman spectra of four mixtures from 1450 to 1800 cm⁻¹: two overlapping bands near 1585 and 1604 cm⁻¹ that trade height with the mixture and a broader band at 1655 cm⁻¹, on a curved fl...
Selecting rows ?
2804 rows · 0 selected
Raman bands (invented)

Peak fitting
Separate overlapping peaks by fitting a sum of peak shapes, and compare models with different numbers of peaks.
y (intensity): Choose a column...
x (position): Choose a column...
One spectrum per: None: one signal
Baseline:
arPLS Asymmetric least squares SNIP Known (a column) None
Smoothness log10 λ: 5.0
Peak finding (thresholds in multiples of the noise level σ̂):
Height at least 3.0 Prominence at least 5.0
Peaks at least 0.0 x units apart
Smooth before searching (Savitzky–Golay), window 11 degree 3
Fit a peak model
Peak shape: Gaussian
Baseline under the peaks: None Constant Straight line
Number of peaks: The peaks found The most prominent 2
Or start at centres: e.g. 1200, 1240
Compare fits with 1, 2, ... peaks (AICc and F tests)
One width for every peak
Centres: Free Equally spaced Spaced by 10.0
Choose the y column
Run
Cancel
Reset

Formulas and scripts

The smoothers, derivative, running integral and interpolation are also formula functions (`savgol`, `moving_average`, `running_median`, `exp_smooth`, `gaussian_smooth`, `derivative`, `cum_integral`, `interpolate`, `interpolate_spline`, `interpolate_pchip`; see Formula columns), and every routine, baseline and peak fitting included, is a script function (`smooth`, `derivative`, `cumulative_integral`, `interpolate`, `baseline`, `noise_level`, `find_peaks`, `fit_peaks`; see Scripting). The reports run from scripts as `"signal-smoothing"`, `"curve-calculus"` and `"spectrum-peaks"`.

Try it

- *Raman bands (invented)* — peak fitting of two overlapping bands and a broader one in four mixtures, with the true background included as a column for comparison with the estimated one.
- *Machine vibration (simulated)* — Savitzky–Golay and running-median smoothing, and acceleration integrated to velocity.
- *ECG-like signal (simulated)* — moving-average smoothing of mains hum and peak finding of the R waves.

Validation

The filters, calculus, baselines and peak finding match SciPy and pybaselines, the Faddeeva function matches SciPy's `wofz`, and the peak fits match lmfit, on the cases in `crates/stat-signal/tests/data` (the validation record, `docs/validation.md` in the repository, lists each check and its tolerance).

Series and Signals: signal processing

The Fourier, filter, correlation and wavelet items under **Signals and spectra** at the bottom of the **Series and Signals** menu analyse signals: values recorded at a steady rate, such as sound, vibration, voltage or a sensor read many times a second. They find the frequencies a signal is made of, filter frequencies out, compare signals at a delay, and take signals apart scale by scale with wavelets.

Item	What it does
Fourier spectrum…	Amplitude, power and phase at each frequency; the strongest frequencies; Welch's averaged spectrum; a spectrogram
Digital filter…	Low-pass, high-pass, band-pass, band-stop and notch filters, with their frequency and phase responses
Convolution and correlation…	Cross- and autocorrelation with the lag of best match, convolution and deconvolution with a kernel, the envelope
Wavelets…	Multilevel wavelet decomposition, wavelet denoising, a scalogram

The signal and its sampling

Every one of these dialogs asks for:

- **Signal columns** — numeric columns, one report each.
- **X column** (optional) — the time or position of each sample, numeric or date-time. The rows are sorted by it, so the table need not be in order. Without it, the rows are read in table order.
- **Samples per unit of time** (only without an x column) — the sampling rate, for frequencies in cycles per second (or per your unit of time). Left unticked, frequencies are in cycles per sample (per row).
- **Group column** (optional) — for several signals stacked in one column (long format), such as several runs or channels: each level is analysed as its own signal, plots overlay the groups (up to 12) and tables list them all.

Under the choices the dialog shows the **sampling rate** it found and the **Nyquist frequency**, half the rate: the highest frequency the samples can show. A tone above it

appears at a lower, wrong frequency (aliasing), so record fast enough.

With an x column, the sampling step is the median spacing of the x values (per second for date-times). When the spacing is uneven — a missing sample, a jittery clock — the signal is **resampled** onto an even grid at that step by linear interpolation, and the report's summary says so. Excluded rows and rows with a missing value are left out (and, being uneven gaps, are interpolated over); the summary counts them. Two samples at the same x in one signal are an error naming the rows.

Frequencies are reported in *cycles per unit of x*, *cycles per second* for a date-time x, *cycles per unit of time* with a sampling rate, or *cycles per sample* otherwise. Each report starts with a **Summary**: the signal, the number of groups and samples, the sampling rate, the Nyquist frequency, the unit, and what was resampled or left out.

Every plot can be exported from its right-click menu, and every table can be copied or made into a new data table.

Try it: the sample **Test bench signals** — three 2-second runs of a vibration test at 500 samples per second, stacked by run — opens with one report of each kind.

Fourier spectrum

Series and Signals ▸ Fourier spectrum… breaks each signal into sine waves by the fast Fourier transform (of any length: lengths with only small prime factors directly, others by Bluestein's algorithm), and shows how strong each frequency is.

Options:

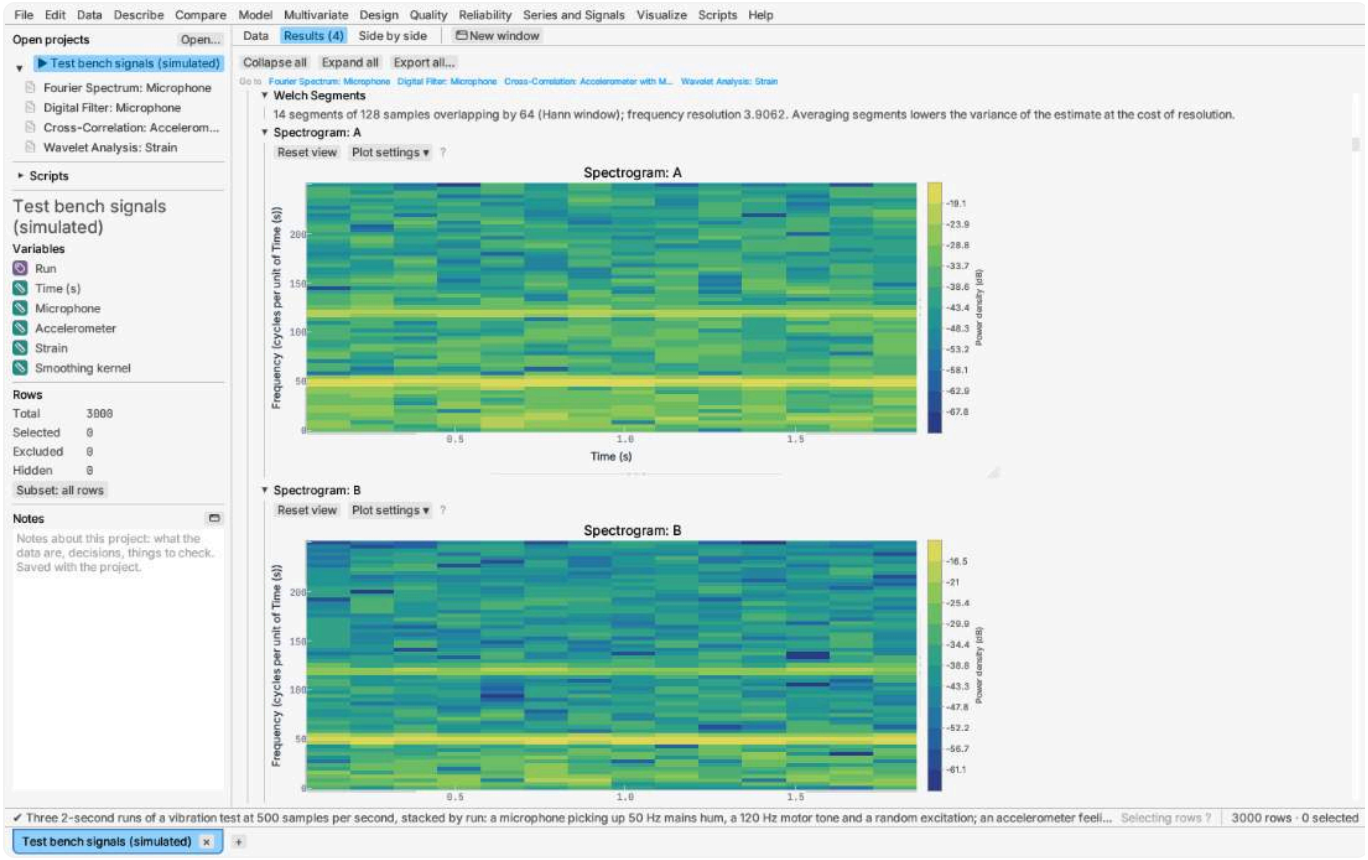
- **Window** — the signal is multiplied by a taper before the transform to reduce *leakage*, the spreading of a strong tone into neighbouring frequencies: Rectangular (none: sharpest peaks, most leakage), **Hann** (the default, a good general choice), Hamming, Blackman, Flat-top (accurate peak heights, broad peaks), Kaiser (β = 8.6) and Tukey (α = 0.25). Amplitudes are corrected for the window's average, so a tone's peak keeps its height.
- **Remove first** — nothing, **the mean** (the default; otherwise a large spike sits at frequency zero), or a straight-line trend.
- **Zero padding** — transform 2, 4 or 8 times the signal length (zeros added), and optionally round up to a power of two: the spectrum is drawn at finer frequency

steps. Padding interpolates; it does not separate tones closer than the resolution 1/(signal duration).
- **Power in decibels** ($10 \cdot \log_{10}$, on by default), **Phase spectrum**, and the number of **dominant frequencies** listed (5).
- **Welch's averaged spectrum** — the signal is cut into overlapping segments (length 256 and 50% overlap by default), each windowed and transformed, and their power averaged: a much less noisy estimate, at the cost of coarser frequency steps.
- **Spectrogram** — the spectrum of each segment drawn against time, to see frequencies come and go.
- Under **More options**: one-sided spectra (the default; power at negative frequencies is folded onto positive ones) or two-sided, and power as a **density** per unit of frequency (the default; it does not depend on the signal length) or as the **power at each frequency**.

The report has:

- **Signal** — the samples (points, which you can select) with a line through the even grid; one colour per group.
- **Amplitude Spectrum** — the amplitude of the sine wave at each frequency: a tone `A·sin(2πft)` shows a peak of height A at f.
- **Power Spectral Density** (or **Power Spectrum**) — power at each frequency, in dB by default.
- **Phase Spectrum** (optional) — the phase of each frequency in degrees.
- **Dominant Frequencies** — the largest local peaks of the power spectrum, with their period, amplitude, power and share of the total power.
- **Welch Spectrum** (optional) and a note giving the segments used.
- **Spectrogram** (optional, for up to four groups) — a time–frequency map: time across, frequency up, power as colour on a sequential scale binned into 12 steps, with a colour bar beside the map whose values mark the steps. Hovering a cell gives its time, frequency and power. In dB the map shows the top 100 dB.
- **Spectrum Values** — frequency, period, amplitude, phase and power, a table you can make into a new data table.

Try it: Test bench signals — the microphone's spectrum shows the 50 Hz mains hum and a 120 Hz motor tone above a broad hump from the shaker.



Digital filter

Series and Signals ▸ Digital filter… keeps or removes a range of frequencies.

- **Filter**:
  - **Butterworth** — the flattest pass band (no ripple).
  - **Chebyshev type I** — a sharper cut-off with a set ripple in the pass band (dB).
  - **Chebyshev type II** — a sharper cut-off with a set attenuation in the stop band (dB), flat in the pass band.
  - **Elliptic** — the sharpest cut-off for its order, with ripple in both bands.
  - **FIR, windowed sinc** — a finite impulse response from the ideal filter's impulse response tapered by a window (Hamming by default), with a set number of **taps** (an odd number for high-pass and band-stop); its phase is linear, so it delays every frequency equally.
  - **FFT, ideal cut-off** or **smooth cut-off** — the signal's Fourier transform is multiplied by the gain (1 in the band, 0 outside, or a Butterworth-shaped gain of the set order) and transformed back. It treats the signal as periodic, so its ends can show wrap-around.

    - **Notch** — removes one frequency (such as mains hum) with a second-order filter; the **quality factor** is the frequency divided by the width of the notch.
- **Band** — low pass, high pass (one cut-off), band pass or band stop (two edges). Frequencies are in the signal's unit and must lie between 0 and the Nyquist frequency; the cut-off starts at a fifth of it.
- **Order** (1–30) of the recursive filters: higher orders cut off more sharply. Band-pass and band-stop filters have twice as many poles.
- **Zero phase** (on by default) — the filter is run forwards and then backwards, so the output is not delayed and the gain is squared (the response plot shows the gain as applied). Turn it off for a causal filter, as a real-time instrument would apply it; the output then lags the input by the filter's phase. Zero-phase filtering pads each end with a mirror image of the signal (three times the filter length), so a signal shorter than that is refused.

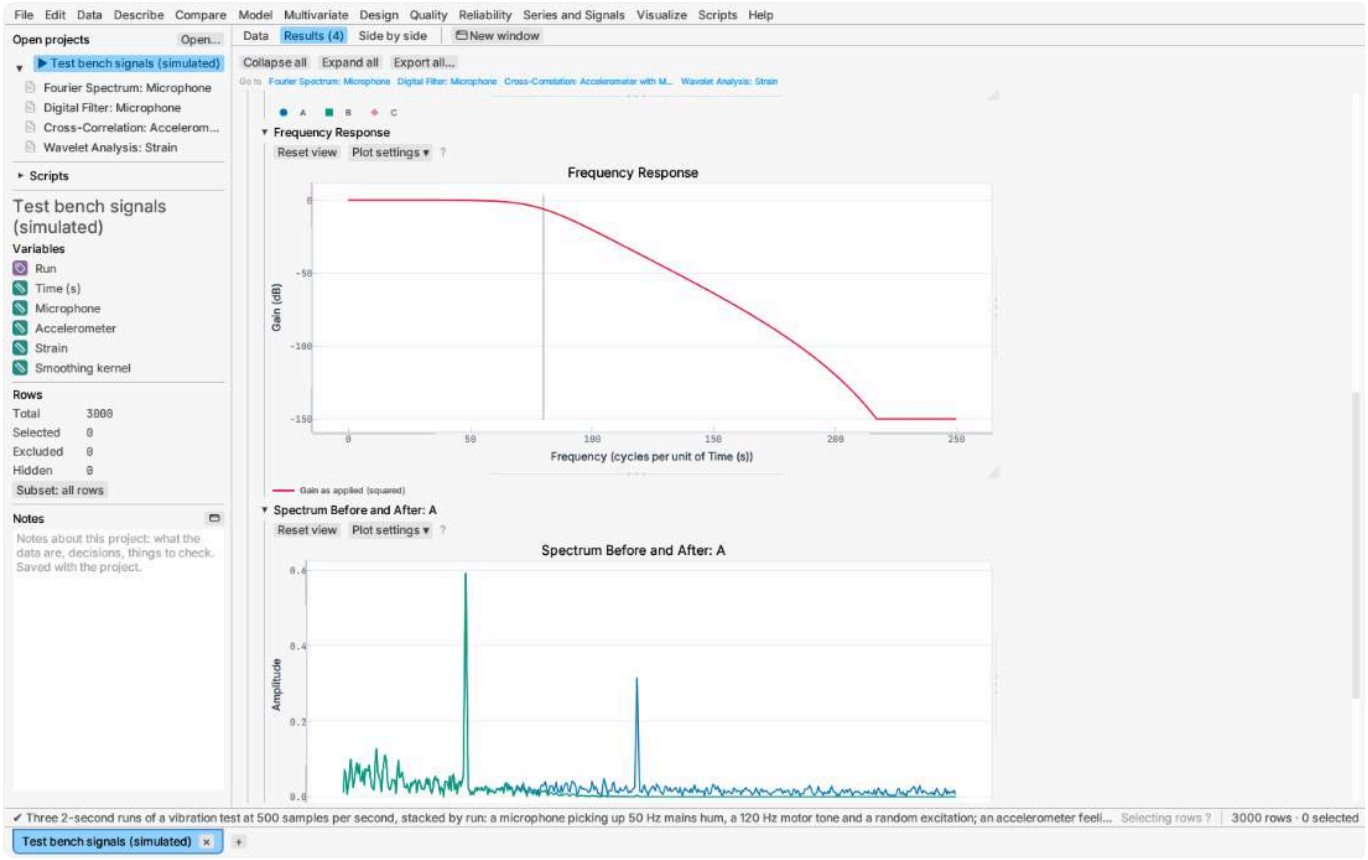
The recursive filters are designed from the analog prototype (its poles, zeros and gain), transformed to the band by frequency mapping with prewarping, then to digital form by the bilinear transform, and applied as **second-order sections**, which stay accurate at high orders where a single polynomial would not.

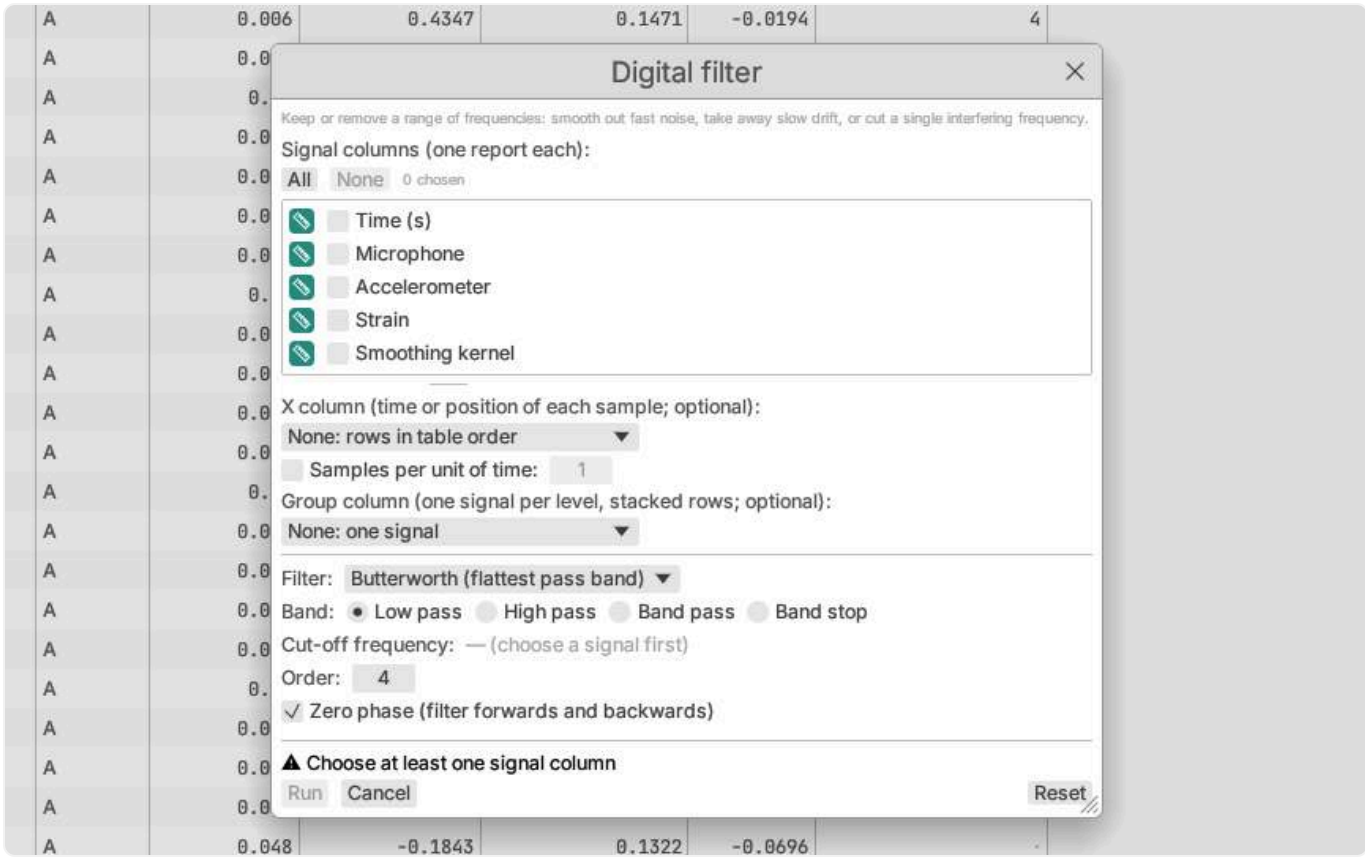
The report has:

- **Filtered Signal** — the samples and the filtered signal as a line.
- **Frequency Response** — the gain in dB against frequency (deep stop bands clipped at −150 dB) with the band edges marked.
- **Phase Response** — phase in degrees (only for causal filters).
- **Spectrum Before and After** — the amplitude spectrum of the first signal before and after filtering.
- **Second-Order Sections** (b_0, b_1, b_2, a_1, a_2 of each section, $a_0 = 1$) or **FIR Coefficients**. The summary says whether all poles are inside the unit circle (the filter is stable).

The **⋯ Options** menu switches zero phase on or off and saves the **filtered signal** and the **removed part** (signal − filtered) as new columns, on the rows the signal came from (interpolated back to the rows' own x values when the signal was resampled).

Try it: Test bench signals — a fourth-order Butterworth low-pass filter at 80 Hz removes the 120 Hz motor tone from the microphone.


File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Open...
Test bench signals (simulated)
Fourier Spectrum: Microphone
Digital Filter: Microphone
Cross-Correlation: Accelerom...
Wavelet Analysis: Strain
Scripts
Test bench signals (simulated)
Variables
Run
Time (s)
Microphone
Accelerometer
Strain
Smoothing kernel
Rows
Total 3000
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data
Results (4)
Side by side
New window
Collapse all
Expand all
Export all...
Frequency Response
Reset view
Plot settings
Frequency Response
Gain (dB)
Frequency (cycles per unit of Time (s))
Spectrum Before and After: A
Reset view
Plot settings
Spectrum Before and After: A
Amplitude
✓ Three 2-second runs of a vibration test at 500 samples per second, stacked by run: a microphone picking up 50 Hz mains hum, a 120 Hz motor tone and a random excitation; an accelerometer feeli...
Selecting rows ?
3000 rows · 0 selected
Test bench signals (simulated)


A 0.006 0.4347 0.1471 -0.0194 4
A 0.048 -0.1843 0.1322 -0.0696
Digital filter
Keep or remove a range of frequencies: smooth out fast noise, take away slow drift, or cut a single interfering frequency.
Signal columns (one report each):
All
None
0 chosen
Time (s)
Microphone
Accelerometer
Strain
Smoothing kernel
X column (time or position of each sample; optional):
None: rows in table order
Samples per unit of time: 1
Group column (one signal per level, stacked rows; optional):
None: one signal
Filter: Butterworth (flattest pass band)
Band: Low pass High pass Band pass Band stop
Cut-off frequency: — (choose a signal first)
Order: 4
Zero phase (filter forwards and backwards)
Choose at least one signal column
Run
Cancel
Reset

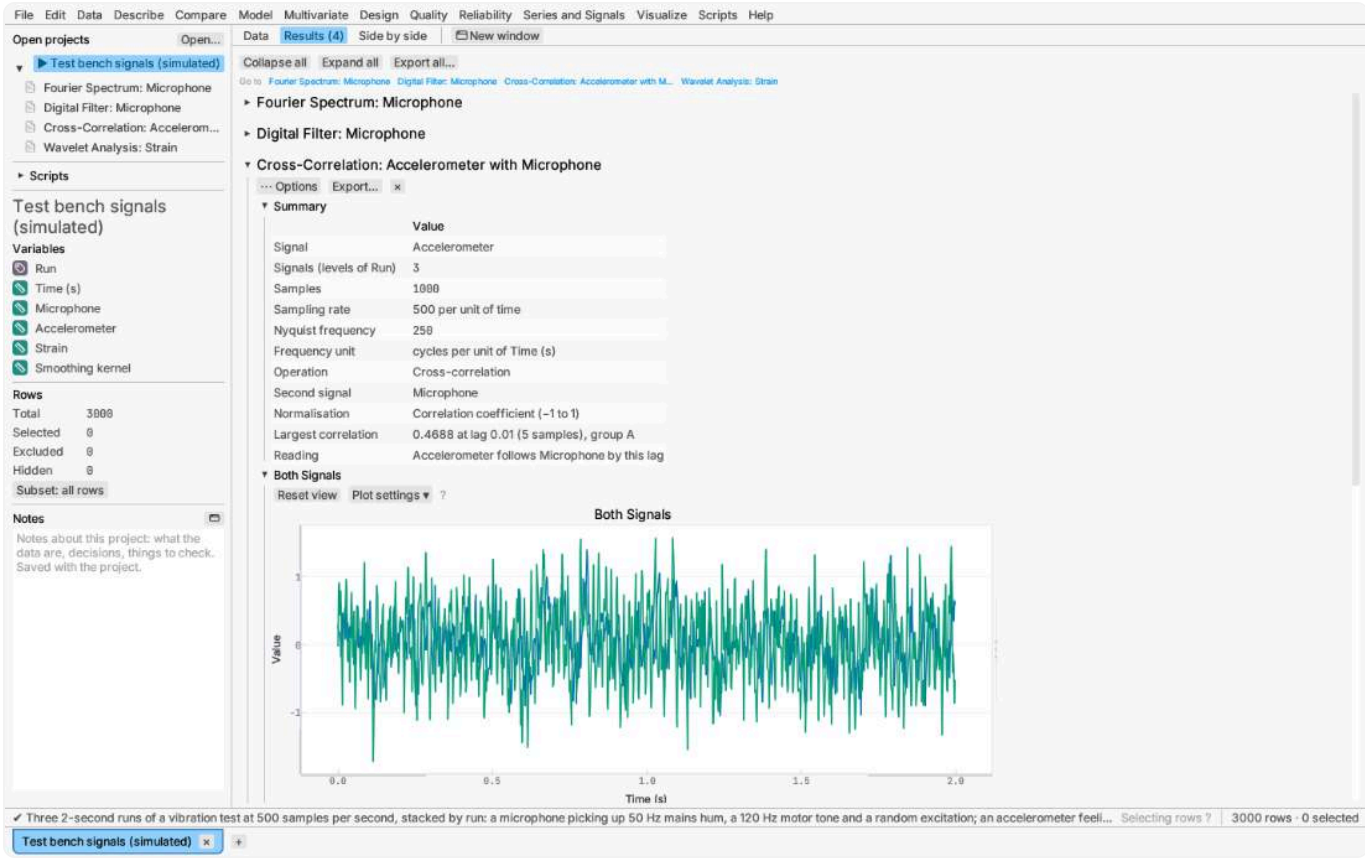
Convolution and correlation

Series and Signals ▸ Convolution and correlation… offers five operations:

- **Cross-correlation with a second signal** — the second column is read with the same x and group columns. The correlation at lag ℓ pairs the first signal at time t + ℓ with the second at time t, so a peak at a **positive lag** means the first signal follows the second by ℓ samples; the summary says this in words and gives the lag in time units. **Normalisation**: the correlation coefficient (−1 to 1, after removing the means; the default), divided by n (biased), divided by the number of overlapping samples (unbiased), or raw sums. **Lags** up to a quarter of the length by default, or up to a number you set.
- **Autocorrelation** — the signal with itself; the table lists the peaks after lag 0, which reveal repeating patterns and their periods.
- **Convolution with a kernel** — the kernel is a column's values in row order (its non-missing, included rows), such as weights 1, 4, 6, 4, 1; **Divide the kernel by its sum** makes it a weighted moving average. The result keeps the signal's length, centred on the kernel.
- **Deconvolution by a known kernel** — undoes a convolution (an instrument's blur): **Wiener** division of the transforms with a noise-to-signal ratio λ, `Y·conj(H)/(|H|² + λ)` (larger λ is smoother), or **Richardson–Lucy** iteration for non-negative data such as counts and intensities (the kernel is normalised to sum 1).
- **Envelope and instantaneous phase** — from the analytic signal `x + i·H[x]` (H the Hilbert transform, computed with the FFT): the envelope is its magnitude (the signal's slowly changing amplitude), the phase its unwrapped angle in radians, and the instantaneous frequency the phase's rate of change in cycles per unit of time.

The reports plot both signals, the correlation against lag (with the lag of maximum marked), the convolved or deconvolved signal with the kernel, or the signal with its envelope and the phase and frequency. The ⋯ **Options** menu saves the convolved or deconvolved signal, or the envelope, phase and frequency, as columns.

Try it: Test bench signals — the accelerometer correlates best with the microphone at lag +5 samples (10 ms) in every run: the vibration reaches the accelerometer 10 ms after the microphone hears it.



Wavelets

Series and Signals ▸ Wavelets… splits a signal into an *approximation* (the smooth part) and *details* at successively coarser scales by the discrete wavelet transform: at each level the current approximation is filtered into a low-pass half (the next approximation) and a high-pass half (that level's detail), each kept at every second sample.

- **Wavelet** — **haar**, Daubechies **db1–db10**, symlets **sym2–sym10** and coiflets **coif1–coif5** (db4 by default). Longer wavelets are smoother and separate frequencies better; haar suits step-like signals.
- **At the ends** — the signal is extended by its mirror image (**symmetric**, the default), **periodically** (each level then has exactly half as many coefficients and the transform is orthogonal), or by zeros.
- **Levels** — as many as the length allows (`⌊log₂(n / (L − 1))⌋` for a wavelet of length L), or a number you set.
- **Denoise** — shrink the detail coefficients towards zero and reconstruct: the **universal** threshold `σ̂√(2 ln n)` (Donoho and Johnstone), which removes nearly all the noise, or **SURE** (Stein's unbiased risk estimate, per level, with the universal threshold where

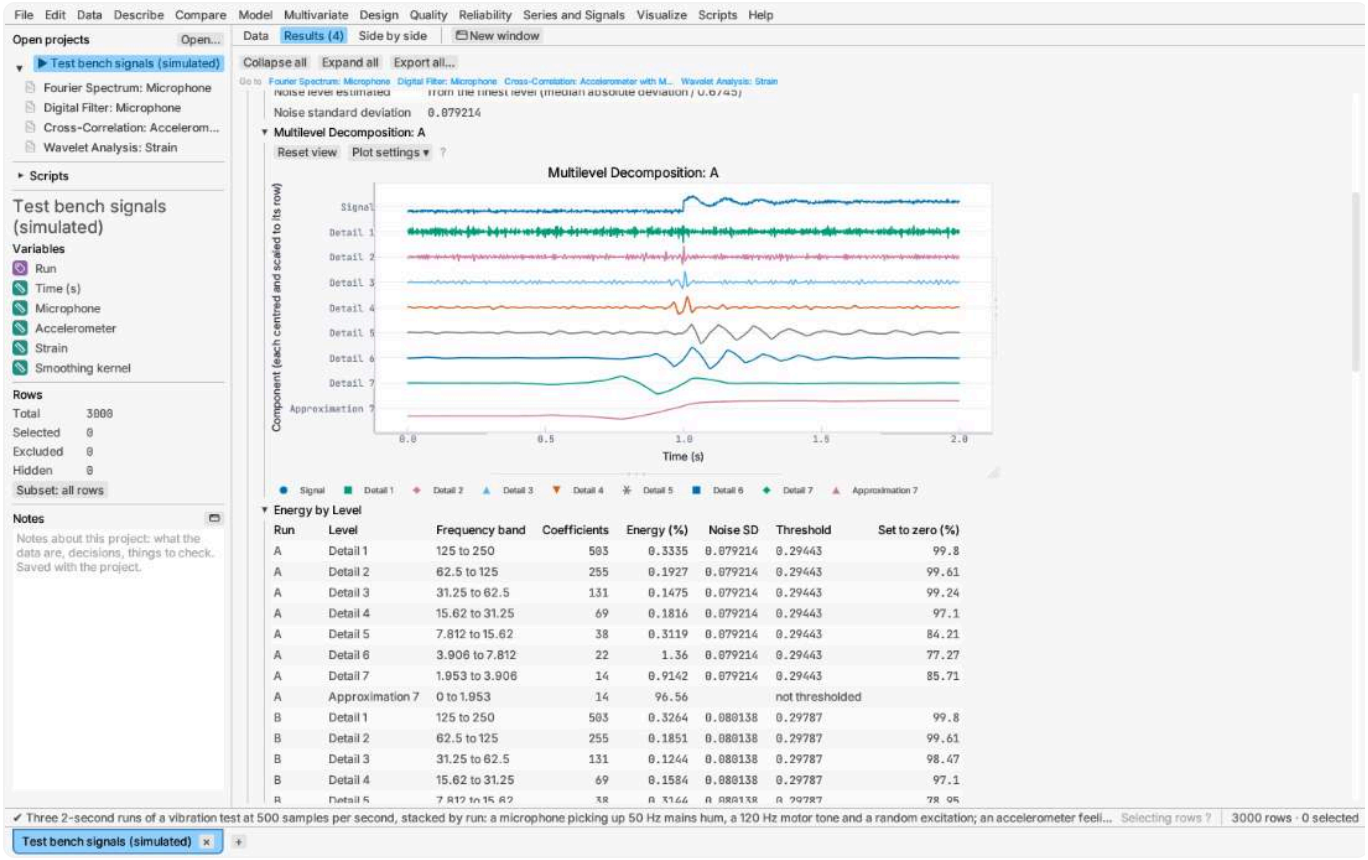
the signal is sparse), which keeps more detail, or a **fixed** value. **Soft** shrinkage moves every coefficient towards zero by the threshold; **hard** keeps those above it unchanged and zeroes the rest. The noise level $\hat{\sigma}$ is the median absolute deviation of the finest details divided by 0.6745, or estimated at **each level** for coloured noise. The approximation is never thresholded.
- **Scalogram** — the continuous wavelet transform with a **Morlet** or **Mexican hat** wavelet at 48 scales (4–200), spaced evenly on a log scale from the Nyquist frequency down to two cycles over the signal.

The report has:

- **Multilevel Decomposition** (first signal) — the signal, each detail level and the approximation, each reconstructed alone at the signal's length and stacked in rows (each centred and scaled to its row, so compare sizes in the table below).
- **Energy by Level** — for each detail level (finest first) and the approximation: the frequency band it covers (detail j covers fs/2^(j+1) to fs/2^j), the coefficients, the share of the energy, and when denoising the noise SD, the threshold and the share set to zero.
- **Denoised Signal** — the samples with the denoised signal.
- **Scalogram** (up to four groups) — time across, frequency up on a log scale, the magnitude of the coefficients as colour with a colour bar; long signals are averaged into 400 time steps. Hovering a cell gives its time, frequency and value.

The ⋯ **Options** menu switches denoising, the scalogram and soft or hard shrinkage, and saves the **denoised signal**, the **removed noise** and the **smooth part** (the approximation alone) as columns.

Try it: Test bench signals — the strain gauge's step and ringing come through the sym4 denoising sharply, and the scalogram shows the 8 Hz ringing dying away after the step.



In formulas and scripts

Formula columns can filter and transform a column down the rows (in row order, optionally within groups): `lowpass_filter`, `highpass_filter`, `bandpass_filter`, `bandstop_filter` (fourth-order Butterworth, zero phase, frequencies in cycles per row), `envelope`, `instant_phase` and `wavelet_denoise`. See Formula columns.

Scripts have `fft`, `ifft`, `spectrum`, `welch_spectrum`, `spectrogram`, `filter_signal`, `design_filter`, `convolve`, `circular_convolve`, `cross_correlation`, `autocorrelation`, `deconvolve`, `hilbert`, `wavelet_decompose`, `wavelet_reconstruct`, `wavelet_denoise` and `cwt` on arrays, and run the reports with `analyze` (analysis names `fourier-spectrum`, `digital-filter`, `signal-correlation` and `wavelet-analysis`). See Scripting.

Accuracy

The transforms, windows, spectra, filter designs, convolutions, Hilbert transform and wavelets are checked against NumPy, SciPy, scikit-image and PyWavelets (`docs/validation.md` in the source lists each check and its tolerance).

Series and Signals: image processing

The three items under **Images** at the bottom of the **Series and Signals** menu analyse pictures in the frequency domain, by the two-dimensional fast Fourier transform: where a signal report asks which frequencies a trace is made of, these ask which *spatial* frequencies (repeating patterns across the picture) an image is made of, and keep or remove them.

Item	What it does
Image spectrum…	Log-magnitude and phase spectra, the radially averaged power spectrum with a power-law fit, radial profiles along chosen directions, power by direction, the strongest spatial frequencies
Image frequency filter…	Low-, high-, band-pass and band-reject filters and notch filters, applied by inverse FFT; filtered image and removed part saved as columns
Image registration and deblurring…	Wiener, constrained least-squares and Richardson–Lucy deblurring of a Gaussian blur; the shift between two images by phase correlation; the 2-D autocorrelation

The image

An image is an ordinary table with **one row per pixel**: an **x** column (across, increasing to the right), a **y** column (down, increasing downwards, as image rows are counted) and one or more **value** columns. Opening a PNG, JPEG, TIFF or BMP file with File ▸ Open (see Images as tables) gives exactly this table (columns `x`, `y` and `intensity`, or `red`, `green`, `blue` and `luminance`), but any table of that shape works, such as a height map exported by a profilometer. The dialogs pick the columns named x and y, and an `intensity` or `luminance` column, when the table has them.

- **Value columns** — numeric, one report each.
- **x** and **y** — the pixel positions. The distinct x values must be evenly spaced, and so must the distinct y values; those spacings are the pixel sizes, and spatial frequencies are reported in *cycles per unit of x*. Rows may be in any order.
- **Group column** (optional) — several images stacked in one table: each level is its own image. The first four are drawn; tables list them all.

The dialog shows the image size and the **Nyquist frequency**, half a cycle per pixel: the finest pattern the pixels can show.

A pixel whose row is excluded, or whose value is missing, leaves a hole, filled with the image's mean (the summary counts them), so the image keeps its shape. Two rows for one pixel are an error naming the rows. Images up to 4096 × 4096 pixels are accepted.

Every image section is drawn as a picture with a colour bar, and hovering a cell reports its position and value, so colour is never the only way to read it. Dragging a box on an image (or clicking a pixel) selects those pixels' rows in the data grid and in every other plot; rows selected elsewhere are shaded in the selection colour on the image. Each picture can be exported from its right-click menu as PNG, SVG, PDF or EPS (the vector formats embed the picture as a bitmap).

Image spectrum

Series and Signals ▸ Images ▸ Image spectrum… computes the 2-D discrete Fourier transform of the image (after optionally removing its mean and applying a window) and draws it **centred**, with zero frequency in the middle:

```
F(u, v) = Σ_r Σ_c w(r, c) · (f(r, c) − f̄) · e^{−2πi (u c/C + v r/R)}
```

for an image of `R` rows and `C` columns, with frequencies `f_x = u / (C Δx)` and `f_y = v / (R Δy)` (Gonzalez & Woods 2018, *Digital Image Processing*, 4th ed., ch. 4; Bracewell 2000, *The Fourier Transform and Its Applications*, 3rd ed.). A pattern that repeats every `p` units in direction θ shows as a pair of bright spots at distance `1/p` from the centre, on the line at angle θ, mirrored through the centre.

Options:

- **Window** — none, Hann across and down (the outer product of two periodic Hann windows; the default), or radial Hann (one Hann window swept around the centre, which favours no direction). Without a window the jump between opposite edges of the image shows as a bright cross through the centre of the spectrum.
- **Padding** — none (the image is treated as repeating), zeros to the next size with small prime factors, or mirroring (doubles each side; no edge jumps).
- **Remove the mean first** (on by default), so the zero-frequency spike does not swamp the scale.

- **Phase spectrum**, **Radial spectrum and power law**, **Angular spectrum** (with the number of direction sectors over 0–180°), the number of **dominant frequencies** listed, and the **lowest frequency** considered for the peaks, fit and angles, as a percentage of the Nyquist frequency (default 2 %, which skips slow shading).
- **Radial profiles at directions (°)** (on by default, at 0 and 90): the directions to cut the spectrum along, typed as degrees separated by commas. **Average a wedge of ±** a number of degrees around each direction instead of a straight cut (default 0), and **Also along each dominant frequency's direction** adds a profile through every peak listed.

Report sections:

- **Summary** — values column, size, pixel spacing, Nyquist frequency, unit, holes filled, window and padding.
- **Image** — the picture analysed.
- **Log-Magnitude Spectrum** — `10 log₁₀ |F|²` (decibels), with the dominant frequencies ringed and numbered. Its axes are the horizontal and vertical frequency; the vertical axis increases downwards, as y does, so a spot at `(f_x, f_y)` is the wave vector in the image's own coordinates.
- **Phase Spectrum** (optional) — `arg F`, in radians.
- **Radially Averaged Power Spectrum** — the mean power over rings of equal radial frequency `ρ = √(f_x² + f_y²)`, on log–log axes (van der Schaaf & van Hateren 1996, *Vision Research* 36). Natural textures and rough surfaces fall off as a power law `P ∝ ρ^−β`.
- **Power-Law Fit of the Radial Spectrum** — least-squares slope of `log₁₀ P` on `log₁₀ ρ` between the lowest frequency and the Nyquist frequency, its standard error, `β = −slope`, R^2 and the number of rings. β near 2 is typical of photographs of natural scenes; a flat spectrum (β ≈ 0) is white noise.
- **Radial Profiles** — the power spectrum along each chosen direction, from zero frequency outwards, one line per direction, with frequency on a linear axis and power on a log axis. Where the radially averaged spectrum averages every direction together, a profile keeps them apart, so it shows directional structure (anisotropy): a lay of tool marks gives sharp peaks in the profile across the marks and none along them, and grid artefacts show as peaks along the axes. A straight cut steps one column or one row at a time, whichever the direction crosses faster (`Δ = 1 / max(|cos θ| / s_x, sin θ / s_y)` with `s_x`, `s_y` the frequency steps), reading the power by bilinear interpolation between cells, so a cut at 0° or 90° reads the centre row or column exactly. A real image's spectrum is symmetric through the centre, so each point is read on whichever side lies inside the spectrum, and the cut runs out to its edge: the Nyquist frequency along the axes, further along a diagonal. A wedge of

±w° instead averages the cells within w° of the direction whose radial frequency rounds to each step; ±90° averages every direction and gives the radially averaged spectrum.

- **Radial Profile Values** — the profiles as numbers: direction, frequency, period (`1/f`), power, and the cells averaged for a wedge. It starts collapsed; make it a data table from its menu to plot or model the profiles further.
- **Angular Spectrum** — mean power in each direction sector (0° is a wave running horizontally, 90° vertically, counter-clockwise on screen), for frequencies above the lowest one. A peak shows the dominant orientation of the texture (machining lay, fibre direction, grain).
- **Dominant Frequencies** — the strongest local maxima of the power spectrum: horizontal and vertical frequency (refined between cells), the radial frequency, the **period** (pattern spacing, `1/ρ`), the **direction**, the amplitude (twice the cell's magnitude: a cosine's amplitude when it falls on a frequency of the grid) and the share of the total power (both spots of the pair). Copy it or make it a data table from its menu.

The report's options menu turns the phase, radial, radial profile and angular sections on and off and changes the window in place.

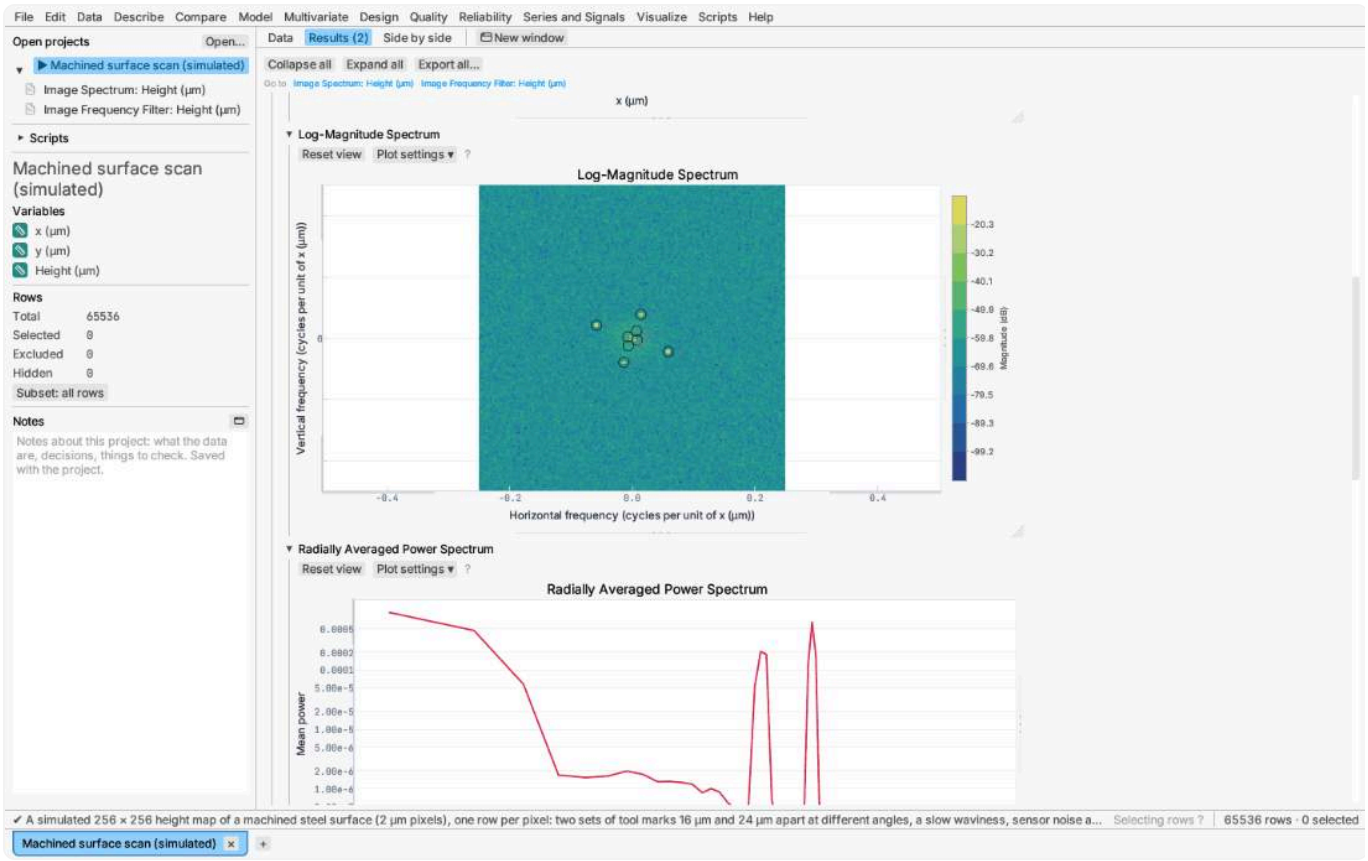


Image frequency filter

Series and Signals ▸ Images ▸ Image frequency filter… multiplies the image's transform by a **mask** `H(u, v)` and transforms back. The masks are real and symmetric through the origin, so the result is real and nothing is shifted. With `D` the radial frequency of a cell, a cut-off `D₀`, and for a band `[f₁, f₂]` its centre `C₀ = (f₁ + f₂)/2` and width `W = f₂ − f₁` (Gonzalez & Woods 2018, §4.8–4.10):

Shape	Low-pass `H_LP(D)`	Band-reject `H_BR(D)`
Ideal	1 if `D ≤ D₀`, else 0	0 inside the band, else 1
Butterworth, order `n`	`1 / (1 + (D/D₀)^{2n})`	`1 / (1 + (DW / (D² − C₀²))^{2n})`
Gaussian	`exp(−D² / (2D₀²))`	`1 − exp(−((D² − C₀²) / (DW))²)`

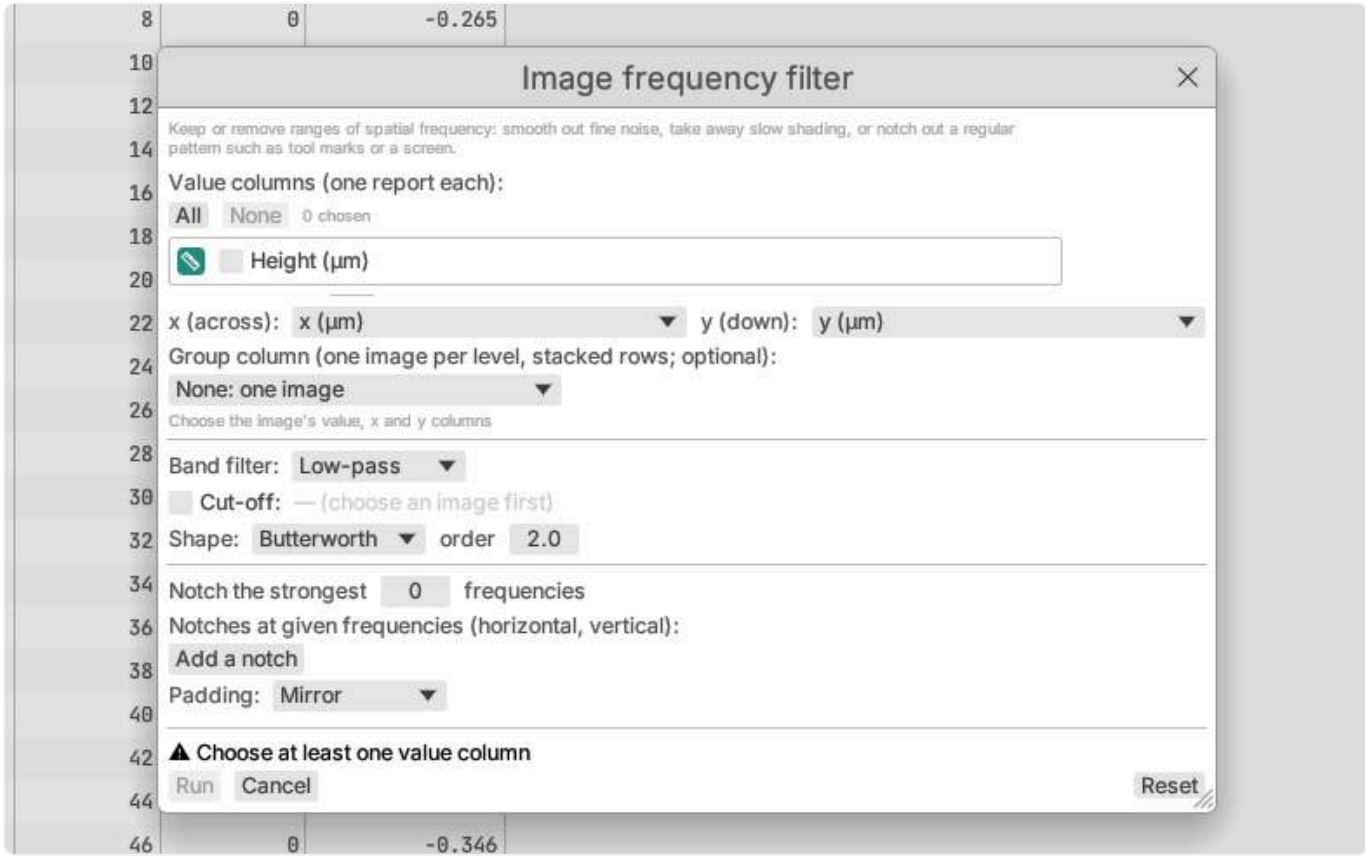
High-pass is `1 − H_LP` and band-pass `1 − H_BR`. An ideal mask cuts sharply and rings around edges in the image; Gaussian masks never ring; Butterworth lies between, sharper

as the order rises. A Gaussian low-pass with $D_0 = 1/(2\pi\sigma)$ is a Gaussian blur of standard deviation σ.

Notches remove a regular pattern, which shows in the spectrum as a pair of spikes at `(u, v)` and `(-u, -v)`. The notch-reject mask is the product over notches of high-pass masks centred on both spikes of each pair, with the notch radius as cut-off; **Keep only the notched frequencies** uses `1 -` that product instead, which isolates the pattern itself. Notches can be typed as (horizontal, vertical) frequencies, read from an Image spectrum report's dominant-frequency table, and the **Notch the strongest … frequencies** option finds them itself (the strongest local maxima of a Hann-windowed spectrum above 2 % of the Nyquist frequency). The default radius is two frequency steps of the image (the coarser of across and down), with a Gaussian shape.

Options: the band (none, low-pass, high-pass, band-pass, band-reject), its cut-off or edges (default a quarter of the Nyquist frequency; bands a quarter to a half), the shape and Butterworth order (default 2), the notches, their radius and shape, and **padding** — mirroring by default, which stops the filter wrapping one edge of the image into the opposite one.

Report sections: **Summary**, **Original Image**, **Filtered Image**, **Removed by the Filter** (original minus filtered), **Spectrum with Notches** (the power spectrum in decibels with each notch marked by a numbered cross), **Filter Mask** (the gain, 0 to 1, centred like the spectrum) and **Notches** (each notch's frequencies, period and whether it was given or found). The options menu saves **Filtered** and **Removed** columns (one value per pixel row; NaN on rows the image does not use) and switches notch-pass and padding in place.


8 0 -0.265
Image frequency filter
Keep or remove ranges of spatial frequency: smooth out fine noise, take away slow shading, or notch out a regular pattern such as tool marks or a screen.
Value columns (one report each):
All None 0 chosen
Height (µm)
x (across): x (µm) y (down): y (µm)
Group column (one image per level, stacked rows; optional):
None: one image
Choose the image's value, x and y columns
Band filter: Low-pass
Cut-off: — (choose an image first)
Shape: Butterworth order 2.0
Notch the strongest 0 frequencies
Notches at given frequencies (horizontal, vertical):
Add a notch
Padding: Mirror
Choose at least one value column
Run Cancel Reset
46 0 -0.346

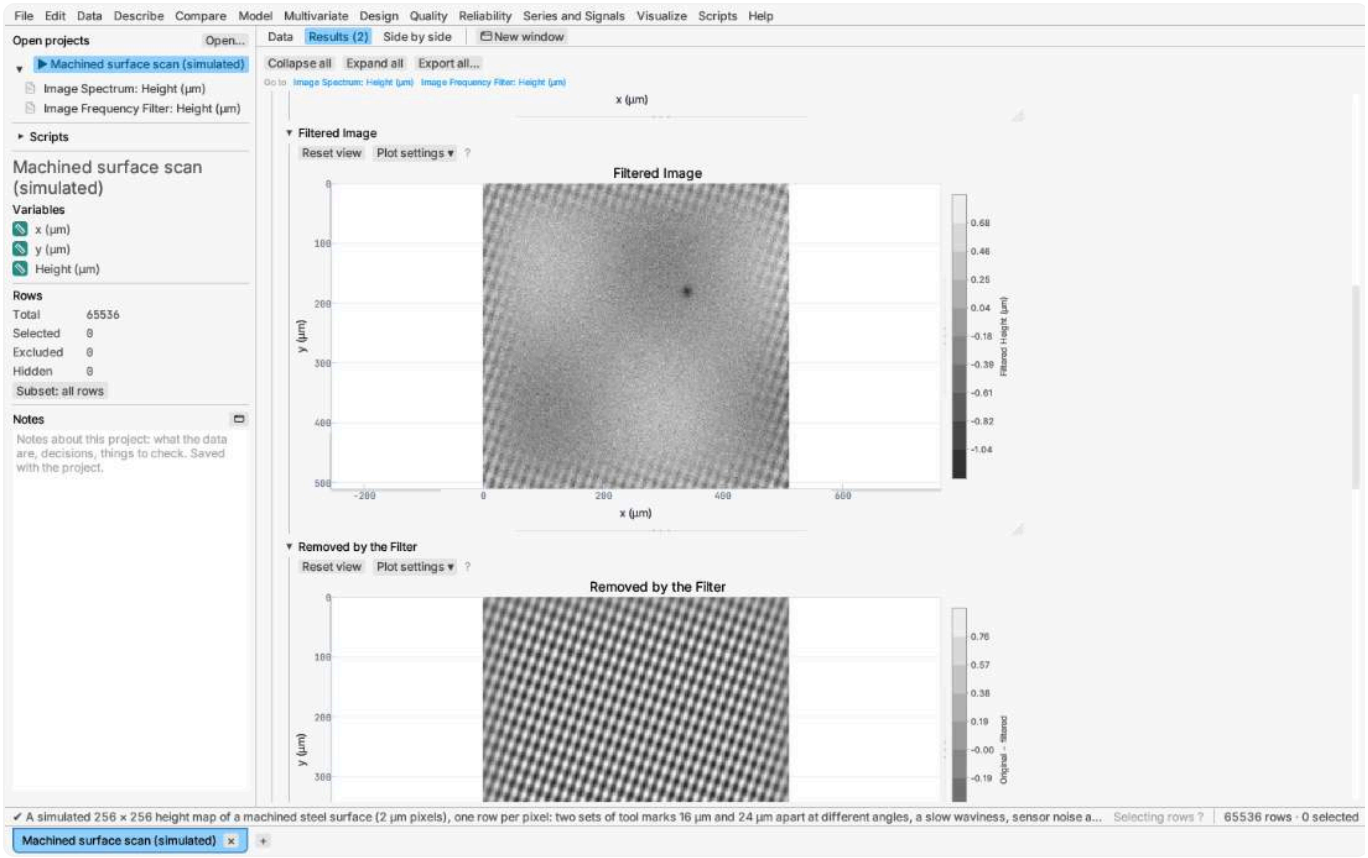

File Edit Data Describe Compare Model Multivariate Design Quality Reliability Series and Signals Visualize Scripts Help
Open projects
Machined surface scan (simulated)
Image Spectrum: Height (µm)
Image Frequency Filter: Height (µm)
Scripts
Machined surface scan (simulated)
Variables
x (µm)
y (µm)
Height (µm)
Rows
Total 65536
Selected 0
Excluded 0
Hidden 0
Subset: all rows
Notes
Notes about this project: what the data are, decisions, things to check. Saved with the project.
Data Results (2) Side by side New window
Collapse all Expand all Export all...
Filtered Image
Reset view Plot settings
Removed by the Filter
A simulated 256 × 256 height map of a machined steel surface (2 µm pixels), one row per pixel: two sets of tool marks 16 µm and 24 µm apart at different angles, a slow waviness, sensor noise a...
Selecting rows ?
65536 rows · 0 selected
Machined surface scan (simulated)

Image registration and deblurring

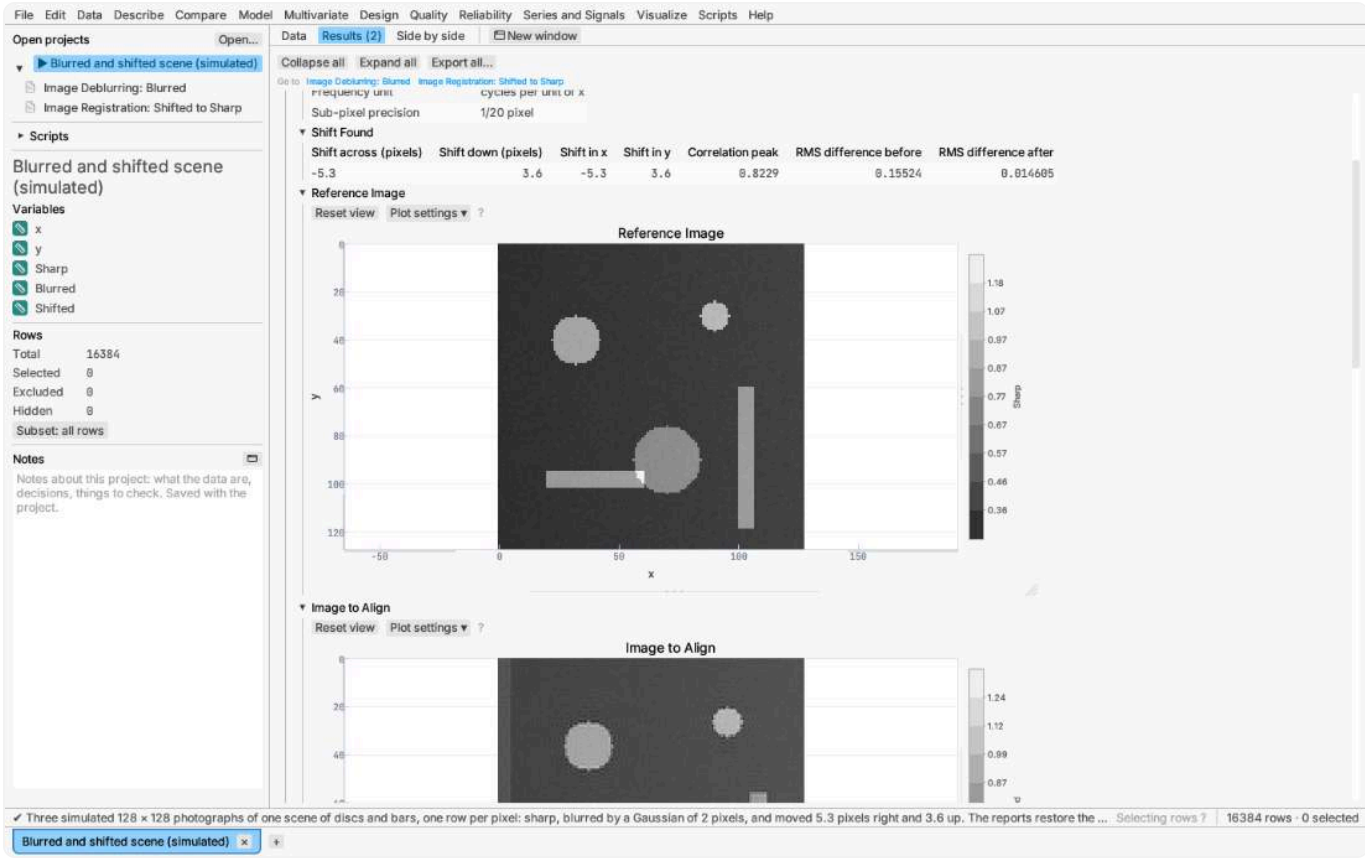
Series and Signals ▸ Images ▸ Image registration and deblurring… offers three operations.

Deblur undoes a Gaussian blur whose standard deviations across and down (in pixels) you give; the point-spread function is truncated at four standard deviations and sums to 1.

- **Wiener filter** — `F̂ = H̄ G / (|H|² + λ)`, with `G` the blurred image's transform and `H` the blur's (Gonzalez & Woods 2018, §5.8). λ = 0 divides by the blur outright, which amplifies noise wherever the blur leaves little signal; larger λ gives a smoother, less sharp result.
- **Constrained least squares** — `F̂ = H̄ G / (|H|² + λ|L|²)` with `L` the transform of the discrete Laplacian (§5.9): the roughness of the result is penalised, which suppresses speckle.
- **Richardson–Lucy** — the multiplicative iteration `f_{k+1} = f_k · (h̃ ⋆ (g / (h ⋆ f_k)))`, with `h̃` the flipped PSF (Richardson 1972, *JOSA* 62; Lucy 1974, *AJ* 79). It keeps values non-negative (so the image must be), and more iterations sharpen more and amplify noise. A 1024 × 1024 image takes about a fifth of a second per iteration.

The restoration treats the image as repeating, so a few pixels along the border can ring. Sections: **Summary**, **Blurred Image**, **Restored Image**; the options menu saves a **Restored** column.

Align a second image finds the shift between this image (the reference) and another value column on the same pixels by phase correlation: the inverse transform of `F₁ F̄₂ / |F₁ F̄₂|` peaks at the shift (Kuglin & Hines 1975), refined to a fraction of a pixel by evaluating the transform on an upsampled grid around the peak (Guizar-Sicairos, Thurman & Fienup 2008, *Optics Letters* 33(2)). The **precision** sets the fraction (default 1/20 pixel). The shift reported is the one that moves the second image onto the reference (the convention of scikit-image), applied as a Fourier shift. Sections: **Summary**, **Shift Found** (across and down in pixels and in x and y units, the height of the correlation peak, 1 for an exact circular shift, and the RMS difference before and after aligning), **Reference Image**, **Image to Align**, **Aligned Image**, and the **Difference Before Aligning** and **After**. The options menu saves an **Aligned** column.



2-D autocorrelation correlates the image with itself at every shift (mean removed first by default), normalised to 1 at zero shift. A texture that repeats every `p` units shows as a lattice of peaks `p` apart; elongated central peaks show the direction of the grain.

Samples

- **Machined surface scan (simulated)** — a 256 × 256 height map of a machined steel surface (2 µm pixels) with two sets of tool marks 16 µm and 24 µm apart, a slow waviness, noise and a pit as deep as the marks are high. It opens with an Image spectrum report, whose dominant-frequency table finds both sets of marks with their spacing and direction, and an Image frequency filter report that notches out the two strongest frequencies, leaving the pit plainly visible in the filtered image.
- **Blurred and shifted scene (simulated)** — three 128 × 128 pictures of one scene: sharp, blurred by a Gaussian of 2 pixels, and moved 5.3 pixels right and 3.6 up. It opens with a Wiener deblurring of the blurred picture and the registration of the moved picture, which recovers the shift to within a tenth of a pixel.

Scripting

The reports are available to scripts by their analysis names `image-spectrum`, `image-filter` and `image-restoration` (see the scripting reference), with the columns `column`, `x`, `y` and optionally `group`, and the settings as in a saved project.

Coming soon

These features are planned but not yet released. Each is described here when it ships.

- **Your own curve templates:** save a curve-template dialog's model, bounds, shared parameters and weighting under a name, and pick it beside the built-in templates (see Curve templates).

Further plans are listed in the repository's `ROADMAP.md`.

Scripting reference

AnovaCraft scripts are written in Rhai, a small embedded language with JavaScript-like syntax (`let`, `fn`, `if`, `for … in`, closures `|x| x + 1`, arrays `[1, 2]`, object maps `#{ a: 1 }`, template strings `` `R² = ${r2}` ``). Script files end in `.rhai`.

Scripts call the application's own operations: an analysis run from a script is the same request a dialog makes and gives the same report, section for section.

```
let t = open_table("data/plants.csv");
t.set_level("Greenhouse", "nominal");
let r = t.analyze("linear_model", #{
    response: "Height",
    predictors: ["Fertilizer", "Light hours", "Greenhouse"],
});
print(`R² = ${r.value("Fit Statistics", "R²")}`);
r.save("plants-model.md");
```

Worked examples are in `examples/scripts`.

Running scripts

```
anovacraft-run [--max-operations N] script.rhai [ARGUMENTS…]
anovacraft-run --licences
```

`print` output goes to standard output, `debug` output to standard error. Arguments after the script are in the `args` array. Relative paths are resolved against the script's own directory. The exit status is 0 on success, 1 if the script fails (the message names the line and column) and 2 for a usage error. Build it with `cargo build --release -p stat-script`. `anovacraft-run --licences` prints the copyright notices and licence texts of the open-source software built into it (`THIRD-PARTY-NOTICES.txt`).

To run a script over a folder of data files, see Batch runs.

The Script window

Scripts ▸ Script window opens an editor for one script at a time (**New**, **Open…**, **Save**, **Save As…**; **Scripts ▸ Open script…**, or opening or dropping a `.rhai` file, loads one). Rhai is highlighted as you type. An unsaved script is asked about before the app quits.

- **Run** (Ctrl/⌘ + Enter) runs the whole script; **Run selection** (Ctrl/⌘ + Shift + Enter) runs only the selected text, keeping its line numbers. **Stop** ends a running script between operations (an analysis already under way finishes first).
- The **Output** pane shows `print` and `debug` output, then what the run changed. An error names its line: the line is marked in the editor (a `▸` in the margin and a tinted background), and clicking the message moves the cursor there.
- Scripts work on the **open tables**: `current_table()` is the table in the active tab and `open_tables()` lists every open table, in tab order. Changes to them appear when the script ends: all of a table's data edits become **one step that Edit ▸ Undo reverses**, reports follow their columns as they do for edits made by hand, the selection and exclusions are applied, and new reports join the table's report list.
- **Other tables** (opened from files, samples, `new_table`, or made by table operations) stay inside the script unless it calls `t.show()`, which opens the table as a new, unsaved tab when the script ends; the tab you were on stays in front, and the Output pane names each new tab.
- The script runs on copies, so the application stays usable while it runs. If you edit or close a table while a script that uses it is running, your edits are kept and the script's version opens as a new tab instead.
- If the script fails, the changes it made before the error are still applied (and can be undone).
- Relative paths are resolved against the script file's folder (the current folder for an unsaved script; the project file's folder for a script kept in a project).
- **Keep in project** stores the script in the current table's project, either to run on request or to run when the project opens. Save the project to keep it; the table's right-click menu (and **File**) then lists it under **Scripts** (Run, Open in Script window, Run when the project opens, Remove from project). Opening a project whose scripts are set to run on open always asks first (**Run**, **Don't run**, or **Show in Script window** to read it before deciding), since scripts can change data and write files.

`examples/scripts/flag_outliers.rhai` is written for the window: open a table (for example the Michelson sample), open the script and press Run.

Editing aids

- **Brackets:** with the cursor beside a `(`, `[` or `{`, it and its partner are tinted and underlined.
- **Folding:** a line that opens a `{ … }` block spanning several lines has ⊟ in the margin; click it to show the block as `{…}`, and ⊞ to open it again. **Fold ▸ Fold all blocks** folds every outermost block, **Unfold all** opens them. Folding changes only what is shown: Run, Save and Find use the whole script, typing in a folded script keeps the hidden lines, and an error opens every fold so its line can be marked.
- **Completion:** after two letters of a name, a list offers matching script functions (with their arguments) and names the script declares (`let`, `const`, `fn`); inside quotes it offers the active table's column names. **Ctrl + Space** opens the list at any point. ↑ ↓ choose, **Tab** or **Enter** inserts (a function gets its opening bracket), **Esc** closes; clicking an entry inserts it too.
- **Find and replace** (**Find…**, ⌘/Ctrl + F in the editor): type the text, then **Next** or **Previous** (Enter steps on) to select each match; **Replace** changes the selected match and moves on, **Replace all** changes every one and says how many. **Match case** and **Whole names** (only where the text is a complete name, not part of a longer one) narrow the search. A recorded action list opens in this window, so find and replace works on it too, for example to point a recording at another column or file.

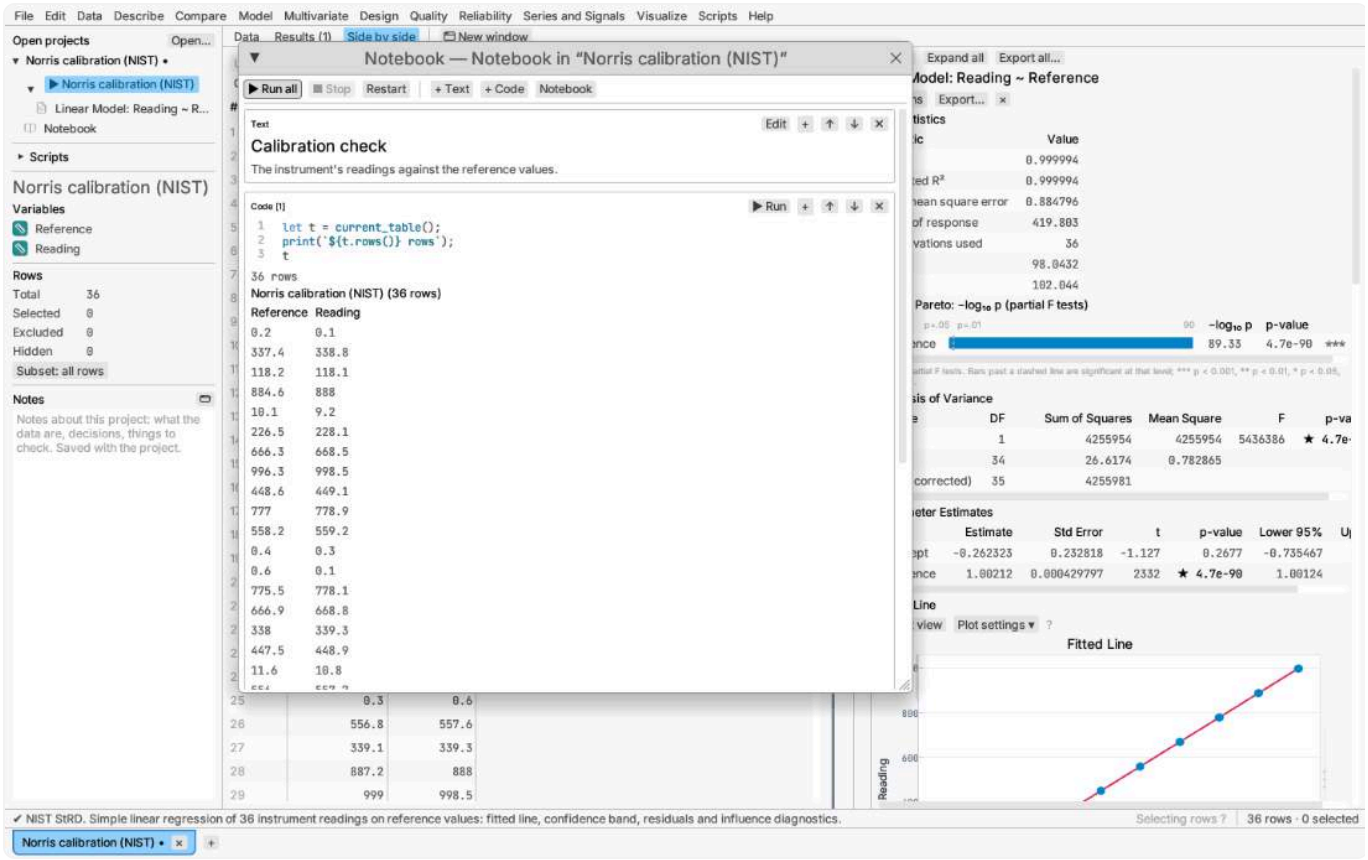
Notebooks

Scripts ▸ New notebook adds a notebook to the current table's project and opens it; **Scripts ▸ Notebooks** lists the project's notebooks to open again. A notebook is a list of **text cells** (notes, shown as written) and **code cells** (Rhai) with their **outputs** under them.

- ▶ **Run** on a code cell (or ⌘/Ctrl + Enter in it) runs that cell; Shift + Enter runs it and moves to the next cell. ▶ **Run all** runs every code cell from the top, stopping at an error. ■ **Stop** stops the running cell.
- Cells share one session: variables (`let`) and functions (`fn`) defined in one cell are there in the cells run after it, until **Restart**. Tables keep up with the application: a variable holding an open table sees edits made in the grid between cells.
- Each run cell shows its run number (`[3]`) and its outputs: lines printed with `print`; an error with its line (click it to go there); the value of its last line — a table as its first 50 rows, an array of maps as a table, a map as name and value pairs, a report as its tables, anything else as text; and every report the cell opened, as a link that shows it.

Changes to the open tables merge as a Script window run's do (one undoable step for the data edits).

- Cells can be added (**+ Text**, **+ Code**, or a cell's **+** menu for one below it), deleted (**×**) and moved (↑ ↓). Text cells switch between **Edit** and **Done**.
- **Notebook ▸ Clear outputs**, **Copy as one script** (the code cells in order, text cells as `//` comments, opened in the Script window), **Rename…** and **Delete notebook**.
- Notebooks, with their outputs, are saved in the project file (key `"notebooks"`), so a reopened project shows the results of the last run without running anything.

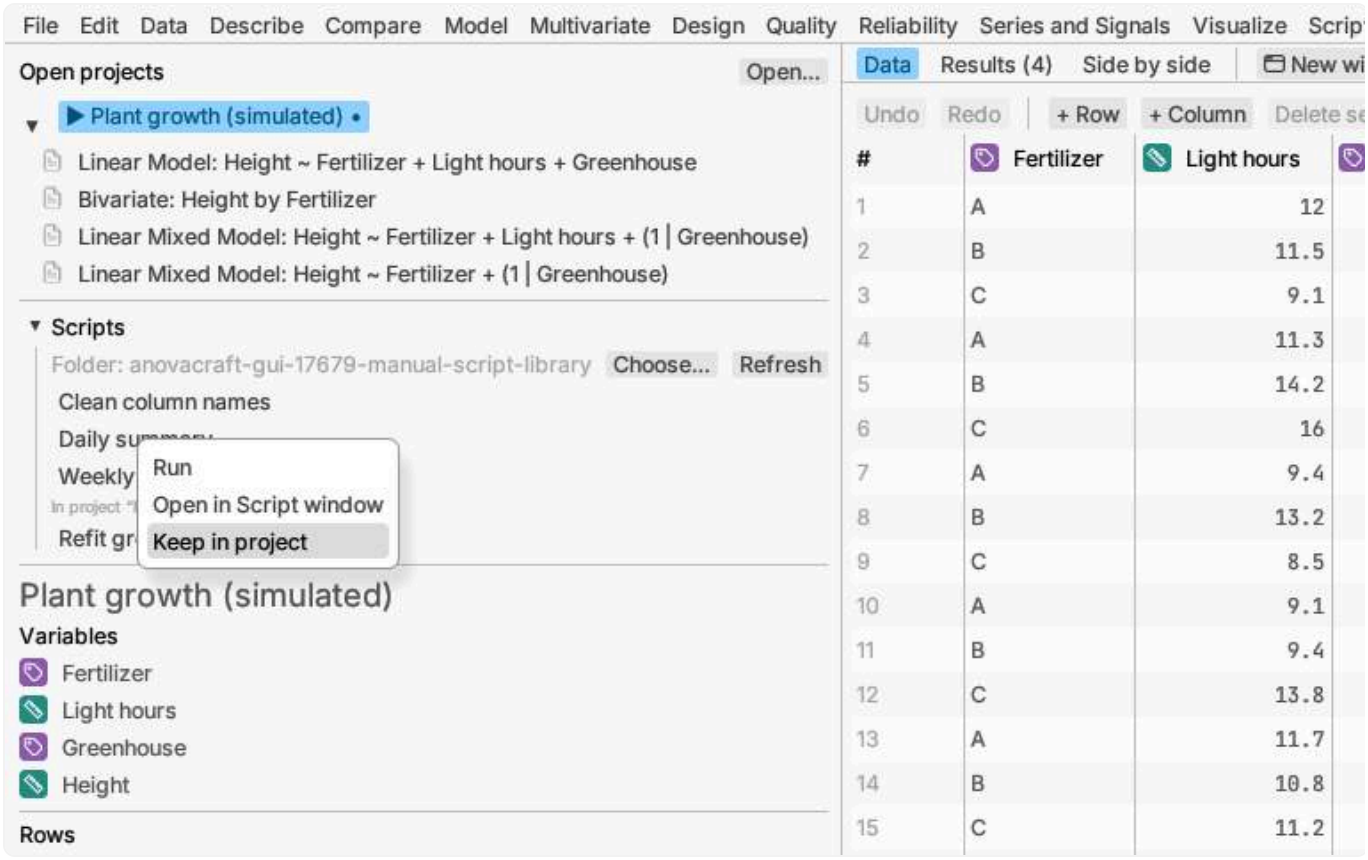


Script library

The **Scripts** section of the left sidebar (click its heading to open it) lists the scripts you keep at hand:

- the `.rhai` files in your **scripts folder** — choose it once with **Choose…** (the application remembers it), and **Refresh** after adding files outside the application (the list also refreshes every few seconds while open);
- the scripts kept in the **active table's project**, marked when they run on opening.

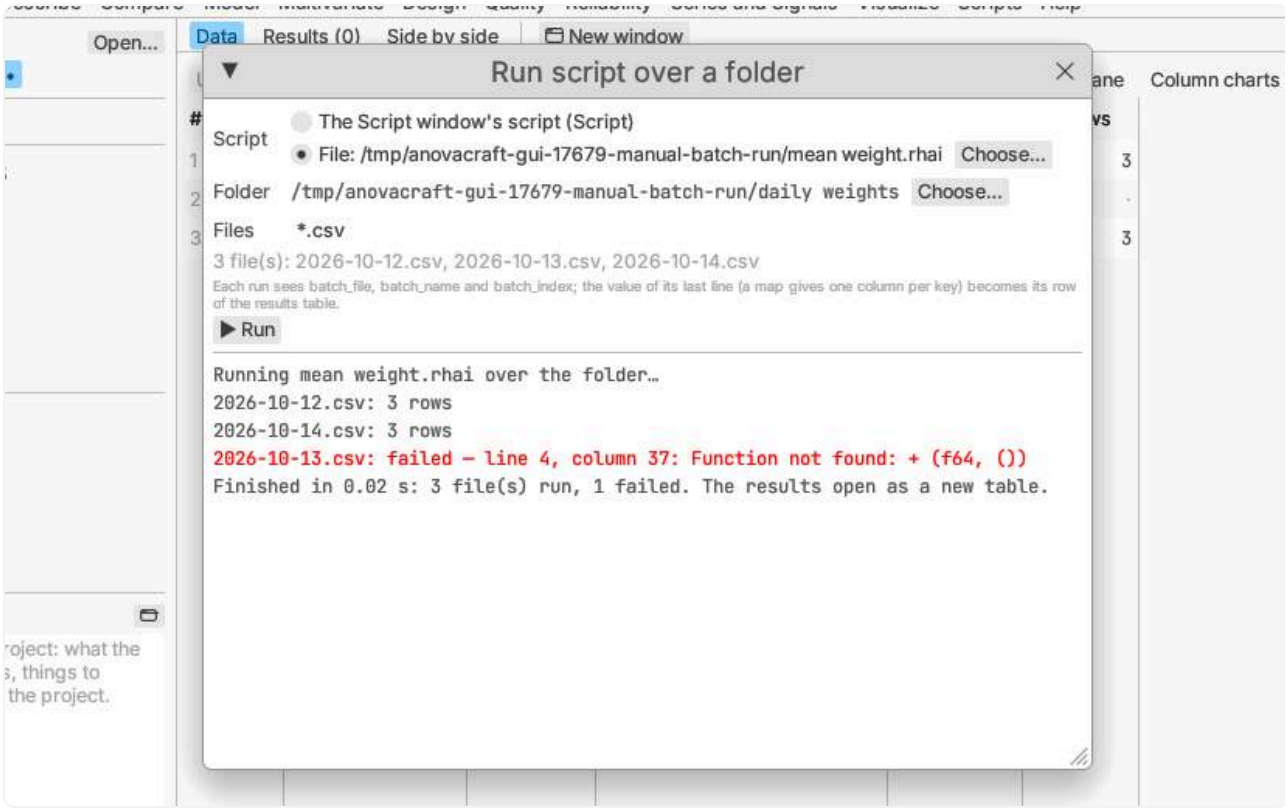
Click a script to open it in the Script window. Right-click a folder script to **Run** it, open it, or **Keep in project** (copy it into the active table's project; save the project to store it); right-click a project script to run, open or **Remove from project** it.



Batch runs

A batch run applies one script to every data file in a folder — the same summary for each day's export, say — and collects what each run returns in a table.

- In the application: **Scripts ▸ Run script over a folder...**. Choose the script (the Script window's, or a file), the folder, and optionally which files (`*.csv`; `*` stands for any characters, `?` for one; every file the application opens when empty — `.csv`, `.tsv`, `.tab`, `.txt`, `.dat`, `.json`, workbooks and `.acraft` projects — the window shows how many match). **Run** works in the background, showing each run's output with its file name; **Stop** ends the batch after the current file. The results open as a new table.



- From the command line:

```
anovacraft-run --batch FOLDER [--pattern GLOB] [--results FILE]
script.rhai [ARGUMENTS…]
```

  `--results` writes the table as CSV, TSV, JSON or `.xlsx` (by its extension: `.csv`, `.tsv`, `.json` or `.xlsx`, in any case; any other extension is a usage error, exit status 2, before anything runs); without it the table is printed to standard output as CSV and the runs' own output goes to standard error. The exit status is 1 when any run failed.

Each run starts afresh (nothing carries over between files) with these constants besides `args`:

Constant	Value
`batch_file`	the file's full path, for `open_table` or `open_project`
`batch_name`	the file's name
`batch_index`	its position in the batch, from 0

The value of the script's last statement is its result: a map gives one results column per key (numbers stay numeric; a blank key is headed `Result`), any other value a `Result` column, and `()` nothing. The results table has `File`, `Status` (`ok` or `error`), `Error` (when a run

failed: its message — the batch carries on with the next file), then the returned columns (a map's keys in alphabetical order, as Rhai keeps them). A syntax error stops the batch before any file is run.

```
// Rows and mean of the "Weight" column of each file.
let t = open_table(batch_file);
let total = 0.0;
for w in t.column("Weight") { total += w; }
#{ rows: t.rows(), mean: total / t.rows() }
```

Recording

Scripts ▸ Record actions starts writing a script of what you do; **Scripts ▸ Stop recording** ends it and opens the script in the Script window (copied to the clipboard instead if the window holds unsaved work). The status bar shows ● Recording meanwhile.

The tables open when recording starts are found as `current_table()`, by name with `table(name)`, or by tab position. Tables opened or made while recording are opened the same way in the script (`open_table`, `open_sample`, `open_project`, `t.derive(…)` for table operations, and `new_table` with each column written out for typed-in, pasted or designed data) and shown as new tabs. Then each step is written as the call that does it: data edits (undo and redo are written as the edit they made), selection, exclusion, hiding and colours, new reports, changed display options (`t.update_report`), closed reports, and saving. While a mouse button is held (dragging a selection box, say) only the result is recorded.

The application and scripts carry out these steps through the same code (the action model in `app_model::action`), so running the recording with the same tables open reaches the same data, row states and reports.

Tables

A `Table` is one dataset with its row states and reports, like a document tab in the application. Copies of a table value refer to the same table.

Function	Result
`open_table(path)`	Open a `.csv`, `.txt` or `.dat` (delimiter detected), `.tsv` or `.tab` (tab-separated), `.json`, workbook (`.xlsx`, `.xlsm`, `.xlsb`, `.xls`, `.ods`: the first sheet with data), image (`.png`, `.jpg`, `.jpeg`, `.tif`, `.tiff`, `.bmp`: a long table of pixels `x`, `y`, `intensity`, colour images as their luminance, reduced by block averages when larger than 1024 pixels on a side) or `.acraft` file. Text and sheets are read with the settings the Open Data dialog would detect (title rows above the table skipped, names from the table's first row). A project brings its row states and reports.
`open_table(path, sheet)`	One sheet of a workbook, by name; the table is named `"file - sheet"`.
`open_table(path, #{ … })`	A text file or workbook read with the Open Data dialog's choices; members left out are detected. `header_row` (0 for no names; giving only this puts the values right below the names), `header_rows` (rows of names, joined with a space), `data_start`, `data_end` (0: to the end), `first_column`, `last_column` (0: to the end); for text `delimiter` (one character, `"tab"`, or `""` to detect), `decimal` (`"."` or `","`) and `missing` (a list of extra texts read as missing); for a workbook `sheet` (a name), or `sheets` (a list of names) with `stack: true` to stack them into one table with a column of sheet names called `sheet_column` (`"Sheet"`). Example: `open_table("report.xlsx", #{ sheet: "Line 1", header_row: 4, first_column: 3 })`. Recorded scripts write this map when the dialog's choices differ from the detected ones.
`open_workbook(path)`	Every sheet with data, as an array of tables (named `"file - sheet"` when there are several).

Function	Result
`open_project(path)`	Open a `.acraft` project.
`new_table(name)`	An empty table; the first column added sets the number of rows.
`open_sample(name)`	A built-in sample dataset (with its starting reports), found by the start of its menu name, e.g. `"Norris"`.
`split_plot_design(#{ … })`	A new D-optimal split-plot design table, as Design ▸ Split-plot creates it (with the design declared and the project script that fits its mixed model). Settings: `factors` (`[#{ name, low, high, changes }]`, `changes` `"easy"` (default), `"hard"` or `"very-hard"`), `runs`, `whole_plots`, `subplots` (when some factors are very hard and others hard), `model` (formula over the factor names, or `"main effects"`, `"interactions"` (default), `"quadratic"`), `ratio` (η, whole-plot ÷ run variance; 1), `subplot_ratio` (1), `starts` (20), `seed` (1), `randomize` (a seed, default 1, or `false`), `units` (`"actual"` or `"coded"`), `response` (`"Y"`). Analyse it with `t.analyze("mixed-model", …)`: the table's script `"Fit split-plot model"` holds that call.
`definitive_screening_design(#{ … })`	A new definitive screening design table. Settings: `factors` (a count, or `[#{ name, low, high }]`), `extra_runs` (at least this many runs beyond the smallest design, added in pairs from a larger conference matrix; 0), `center_runs` (center runs in all; 1), `randomize` (a seed, default 1, or `false`), `units` (`"actual"` or `"coded"`), `response` (`"Y"`).

Function	Result
`compare_definitive_screening(#{ … })`	A table comparing definitive screening designs for the same factors across nearby extra runs and center runs (settings as for `definitive_screening_design`): runs, error df after the main effects and after the main and quadratic effects, main-effect D-efficiency, largest |r| among quadratic effects and between two-factor interactions, mean standard error of a quadratic effect (σ units).
`screening_design(#{ … })`	A new D-optimal screening design with continuous and categorical factors. Settings: `factors` (`[#{ name, low, high }]` for a two-level continuous factor, `#{ name, levels }` for a categorical one with 2 to 12 levels, which becomes a text column `L1`, `L2`, …), `runs` (default: terms + 4, rounded up to balance the levels), `model` (`"main effects"` (default) or `"interactions"`), `starts` (20), `seed` (1), `randomize`, `units`, `response`.
`read_database(#{ kind, … })`	Run an SQL query into a new table. `kind`: `"sqlite"` (with `path`, the database file), `"postgresql"` or `"mysql"` (with `host`, `port`, `database`, `user`, `password`, and `tls`: `"disable"`, `"prefer"` (default) or `"require"`), or `"odbc"` (with `path`, the connection string, and `password`; only in builds with the `odbc` feature). Give `query` (SQL) or `table` (read the whole table). The table remembers its source (without the password), so a saved project can refresh it in the application.
`database_tables(#{ kind, … })`	The database's table names, with the same connection settings.

Function	Result
`read_url(url)`, `read_url(url, #{ headers, secret_headers, format, json_path, table })`	Fetch JSON, delimited text or a table on a web page from a web address into a new table. `headers` and `secret_headers` map names to values (secret ones are never saved; headers such as `Authorization`, `Cookie` or names containing `key`, `token` or `secret` always count as secret); `format`: `"auto"` (default; from the content type), `"json"`, `"delimited"` or `"html"`; `json_path`: dotted path to the array of rows inside the document (e.g. `"data.items"`); `table`: which table of a web page, by number from 1 or by id (default: the one the address's `#section` points to, else the largest). Nested objects become dotted column names. Query parameters of the address whose names mark a credential (the same rule) are saved in projects with their values blanked.
`current_table()`	In the Script window: the table in the active tab (an error in `anovacraft-run`, which has no open tables).
`open_tables()`	In the Script window: every open table, in tab order (empty in `anovacraft-run`).
`table(name)`	In the Script window: the open table with that name (an error if none or several have it).
`t.show()`	Open `t` as a new tab when the script ends (in the Script window; does nothing in `anovacraft-run`). Open tables are already shown.
`t.name`, `t.rename_table(new)`	Table name; give the table another name (undoable like any edit).
`t.rows()`, `t.columns()`	Number of rows; column names.
`t.has_column(name)`, `t.level(name)`	Whether a column exists; its measurement level (`"continuous"`, `"nominal"`, `"ordinal"`).

Function	Result
`t.column(name)`	The column's values as an array. Missing values are `()`. Dates and times are seconds since 1970-01-01 UTC.
`t.row(i)`, `t.cell(i, name)`	Row `i` as a map of column name → value; one value. Rows are numbered from 0.

Changing data

Each change is the same undoable edit the data grid makes; reports stored with the table keep following their columns.

Function	Effect
`t.add_column(name, values)`, `t.add_column(name, values, level)`	Append a column. All numbers (or `()`, `true` / `false`) make a numeric column; anything else makes a text column.
`t.set_column(name, values)`	Replace a column's values.
`t.simulate_responses(#{ … })`	Add simulated responses for the design in `t` (one edit) and return a new table of their true coefficients (one row per response: its name, the number of active terms, the active terms, and every term's coefficient). Settings: `factors` (column names, or `#{ column, low, high }`), `model` (formula over the factor names, or `"main effects"`, `"interactions"` (default), `"quadratic"`), `count` (1, at most 1000), `seed` (1), `name` (`"Simulated Y"`, numbered when several), and the prior: `main` (probability a main effect is active; 0.25), `all_parents`, `some_parents`, `no_parents` (probability a higher-order term is active when all, some or none of its factors are; 0.25, 0.1, 0.01), `effect_sd` (SD of an active coefficient in coded units; 2), `inactive_sd` (0), `noise_sd` (1), `intercept` (0).
`t.set_cell(i, name, value)`	Set one value.
`t.rename_column(old, new)`, `t.remove_column(name)`	Rename; delete (refused while a report uses the column).

Function	Effect
`t.set_level(name, level)`	Change a measurement level.
`t.set_value_labels(name, #{ "1": "Female", "2": "Male" })`, `t.clear_value_labels(name)`, `t.value_labels(name)`	Name a column's stored values (analyses and plots then use the names); remove the names; read them as a map.
`t.set_hover(name, true)`, `t.hover_columns()`	Show a column's value in point tooltips; list such columns.
`t.save_labelled(path)`	Save the table like `t.save`, writing value labels in place of the stored values.
`t.add_rows(n)`	Append `n` empty rows.
`t.insert_rows(at, n)`	Insert `n` empty rows before row `at` (`at = t.rows()` appends).
`t.insert_rows([[at, [values…]], …])`	Insert rows with values (one per column), each ending up at its position; positions increase.
`t.remove_rows([i, …])`	Delete rows.
`t.insert_column(i, column)`	Insert a column at position `i`. `column` is a map in the project-file form: `#{ name, type: "numeric" | "text" | "datetime", level, values }` (a date-time column also gives `format`, `fraction_digits` and `offset_minutes`). A formula column gives `formula` and may leave out `values`.
`t.set_formula(name, formula)`	Give a column a formula (its values are computed and kept up to date), or add a formula column at the end when there is no such column.
`t.formula(name)`	A column's formula, or `()` for a column of entered values.
`t.clear_formula(name)`	Keep a formula column's current values as entered data.
`t.replace_column(name, column)`	Replace a column wholesale (new type, name or values), `column` as for `insert_column`.
`t.sort(keys)`	Sort in place. A key is a column name (ascending) or `#{ column: "x", descending: true }`.

Function	Effect
`t.reorder_rows(order)`	Put rows in a new order: row `i` becomes the row that was at `order[i]`.

Row states

Function	Effect
`t.select(|row| test)`	Select the rows for which the function returns `true` (replacing the selection); returns how many. `row` maps column names to values, e.g. `|row| row.Weight > 3 && row["Plant type"] == "B"`.
`t.select_rows([i, …])`, `t.clear_selection()`, `t.selected_rows()`	Select listed rows; clear; list.
`t.exclude(|row| test)`, `t.exclude_rows([i, …])`	Exclude rows from analyses (adding to those already excluded); `exclude` returns how many matched.

The row test given to `select` and `exclude` may read the table but must not add or remove rows: the script stops with an error if the number of rows changes while rows are being tested.

| `t.set_excluded([i, …])` | Exclude exactly these rows. | | `t.include_all()`, `t.excluded_rows()` | Include every row again; list excluded rows. | | `t.hide_rows([i, …])`, `t.set_hidden([i, …])`, `t.unhide_all()`, `t.hidden_rows()` | Hide rows in plots (adding to those hidden); hide exactly these; show all; list. | | `t.color_rows([i, …], "#rrggbb")`, `t.uncolor_rows([i, …])`, `t.row_color(i)` | Give rows a display colour; remove it; read it (`()` if none). | | `t.mark_rows([i, …], "shape")`, `t.unmark_rows([i, …])`, `t.row_marker(i)` | Give rows a marker shape (`"circle"`, `"square"`, `"triangle"`, `"diamond"`, `"down triangle"`, `"plus"`, `"cross"`, `"asterisk"`); remove it; read it (`()` if none). | | `t.color_by_column("name")`, `t.clear_row_colors()` | Colour and mark every row by a column's value, as Data ▸ Row states ▸ Colour and marker by column… does; remove every row colour and marker. | | `t.set_style_column("name")`, `t.set_style_column(())`, `t.style_column()` | Name (or forget) the column the row colours came from, which plot keys use. |

| `t.save_subset("name", [i, …])`, `t.save_subset("name", \|row\| test)` | Save rows as a named row subset (replacing one of that name). | | `t.use_subset("name")`, `t.use_all_rows()`, `t.active_subset()` | Make a subset active (analyses use only its rows); use every row again; the active subset's name (`()` if none). | | `t.remove_subset("name")`, `t.subset_names()`, `t.subset_rows("name")` | Delete a subset; list the names; list a subset's rows. |

Excluded rows leave every analysis run afterwards, as in the application. So do rows outside the active subset: `t.excluded_rows()` lists them too, while `t.exclude_rows`, `t.include_all` and `t.set_excluded` change only the rows you excluded yourself.

Table operations

These make a new table and leave the original unchanged, like the Data menu's Rows, Reshape, Merge and Summarize commands. Settings not given take the dialog's defaults. A new table is named like the application names it (`"Wide, stacked"`) unless the settings include `name`.

Function	Settings
`t.sorted(keys)`	Sort keys as for `sort`.
`t.remove_duplicates(#{ columns })`	Columns that define a duplicate (all when empty).
`t.subset(#{ rows, columns, sample })`	`rows`: `"all"`, `"selected"`, `"included"` or `#{ matching: [#{ column, levels: […] }, #{ column, min, max }] }`; `sample`: `#{ size: #{ count: n } or #{ fraction: f }, strata: […], seed: n }`.
`t.stack(#{ columns, label, value })`	Wide to long.
`t.unstack(#{ label, values, ids })`	Long to wide.
`t.transpose(#{ columns, label })`	Rows to columns.
`t.summarize(#{ groups, columns, statistics, quantile_type, include_excluded })`	Statistics: `"n"`, `"missing"`, `"mean"`, `"sd"`, `"min"`, `"max"`, `"median"`, `"sum"`, `#{ quantile: p }` (default `["n", "mean", "sd"]`).

Function	Settings
`t.multiple_imputation(#{ columns, engine, imputations, iterations, donors, column_methods, predictors, visit_order, seed })`	The stacked table of a multiple imputation: an `Imputation` column (0 = the original rows), `Row`, then the table's columns with the imputed ones filled in each copy. `engine` `"chained-equations"` (default) or `"normal-model"`; the other settings as for the `imputation` analysis below. Analyses on it can pool across the imputations (`pool_imputations`).
`t.augment_whole_plots(#{ factors, whole_plot, responses, model, whole_plots, runs, ratio, starts, seed, randomize })`	The split-plot design in `t` with new whole plots, as Design ▸ Augment design ▸ New whole plots makes it: `factors` (column names, or `#{ column, low, high }`), `whole_plot` (the column naming each run's whole plot), `model` (formula, or `"main effects"`, `"interactions"` (default), `"quadratic"`), `whole_plots` (how many to add), `runs` (in each), `ratio` (η; 1). Factors constant within every existing whole plot stay fixed within each new one.
`t.join(other, #{ keys, kind, relationship })`	Keys: column names shared by both tables, or `[["left name", "right name"], …]`; kind `"inner"` (default), `"left"`, `"right"`, `"full"`; relationship `"any"`, `"many-to-one"`, `"one-to-one"`.
`t.combine_rows([others…])`, `t.combine_rows([others…], #{ source_column })`	This table's rows followed by the others'.
`t.derive(#{ operation, … })`, `t.derive(#{ … }, [others…])`	Any operation by its project-file name (`"sort"`, `"remove-duplicates"`, `"subset"`, `"combine-rows"`, `"join"`, `"stack"`, `"unstack"`, `"transpose"`, `"summarize"`, `"multiple-imputation"`, `"distance-matrix"`).

Function	Settings
`t.derive(#{ operation: "distance-matrix", columns, metric, standardize, label })`	The distances between the included rows: an `Object` column, then one column per row. `metric` (required): `"euclidean"`, `"manhattan"`, `"gower"`, `"jaccard"`, `"bray-curtis"`, `"hamming"`, `"simple-matching"`; `standardize` (Euclidean and Manhattan; `false`); `label` (a column naming the rows, or `()` for row numbers).

Saving

Function	Effect
`t.save(path)`	Write `.csv`, `.tsv` (or `.tab`), `.json`, `.xlsx` (one sheet, dates as spreadsheet dates), or `.acraft` (the whole project).
`t.save_project(path)`	Write a `.acraft` project: data, measurement levels, row states and every report run on the table. Opening it in the application re-runs the reports.
`t.save_reports(path)`, `t.save_reports(path, #{ paper: "a4" })`	Write every report of the table, in display order, to one document whose format the extension names: `.pdf`, `.docx` (Word), `.pptx` (PowerPoint), `.html` (or `.htm`), `.md` (plots as PNG files in a `<name>_files` folder beside it) or `.xlsx` (a sheet per report); see Saving plots and results in the manual. `paper` (`"letter"`, the default, or `"a4"`) sizes PDF and Word pages. A report that cannot be computed appears with its error.

Analyses

```
let r = t.analyze(kind, #{ settings });
let r = t.analyze(#{ analysis: kind, settings… });
```

`kind` is the analysis identifier used in project files; underscores and hyphens are interchangeable (`"linear_model"` = `"linear-model"`). The settings are exactly a project

file's report entry, with columns by name, so the quickest way to find them for an analysis is to run it once from its dialog and look at `r.request()` (or the `"reports"` array of a saved project). Display options go in `options` and default to the dialog's.

Any analysis also takes a **row filter** of its own: `row_filter`, a list of conditions, each `#{ column: "Region", levels: ["East", "West"] }` (the row's level is one of these) or `#{ column: "Age", min: 20, max: 40 }` (the value lies in the interval, ends included). Only rows passing every condition enter the report; the table's row states are not changed, so its other reports still use every row. `column_switcher` (a list of column names) gives a report with one response or process column the column picker its ⋯ Options menu adds; `switcher_limits` (`[#{ column, lower, upper, target }]`) holds a capability study's limits for the other columns in that list. On a stacked imputation table, `pool_imputations: "Imputation"` (the column numbering the imputations) pools a linear model, generalized linear or mixed model, one-sample, two-group or paired comparison, or correlation report across the imputations by Rubin's rules: its pooled sections come first, and its usual sections use the original rows (imputation 0) only. All of these come back in `r.request()`.

A report with contour plots (`response-surface`, `mixture-model`, `nonlinear-regression` with a surface, `data-surface`) also takes `contour_style`, how its maps are drawn, as Plot settings sets it: `#{ fill: true, lines: true, labels: true, scale: "sequential", reverse: false, opacity: 1.0, levels: 8, min: 0.0, max: 100.0, centre: 50.0 }`. `scale` is `"sequential"`, `"heat"`, `"spectrum"`, `"diverging"` or `"grey"`; `levels` (1–50), `min` with `max` (a fixed range) and `centre` (the diverging scale's middle) are optional, and members left out keep their defaults. `fill: false` draws level lines and values only. In the report's data a contour plot's `contours` holds `levels` and `lines` (one `#{ level, lines }` per level, each line a list of `[x, y]` points).

Any report also takes `plot_styles`, the Plot settings of plots changed from their defaults: a list of `#{ plot: "Residuals#0", x: #{…}, y: #{…}, y2: #{…}, equal_scale, scale_ratio, print_scale, category_labels, hover_coordinates, show_key, title, show_title }` (`scale_ratio`: one x unit as long as that many y units at equal scale; `print_scale`: millimetres per x unit in saved figures, or `()`), where `plot` names the section kind and which plot of that kind (from 0), and each axis holds `major` (`"auto"` or an interval), `minor` (`"none"`, `"auto"`, `#{ subdivisions: n }` or `#{ every: d }`), `origin`, `grid` (`"major"`, `"major-and-minor"`, `"none"`), `tick_marks`, `tick_labels`, `format` (`"auto"`, `#{ decimals: n }`, `#{ scientific: n }` or `#{ percent: n }`), `title`, `show_title`, `side` (`"standard"`, `"opposite"`), `break` (`[from, to]` or `()`), `tick_direction` (`"inside"`, `"outside"`, `"both"`), `major_mark` and `minor_mark` (`#{ length, width }` in points), `offset` (points between the data area and the axis line),

`extend_to_ticks`, `custom_ticks` (a list of `#{ value, label }`, labels in math markup) and `range` (`[min, max]` typed in Plot settings, or `()`). Members left out keep their defaults. The simplest way to get one is to set the plot up in the app and copy the report's script call.

Kind	Main settings
`linear-model`	`response`, `predictors`, `effects` (optional: `[[[predictor position, power], …], …]`; main effects when absent), `options`; `splines` (one entry per predictor: `()` for a straight line, or `#{ kind: "natural"`
`stepwise`	`response`, `predictors`, `effects`, `direction`, `criterion` (`"p-value"` with `enter` / `remove`, `"aic"`, `"bic"`), `heredity`
`distribution-identification`	`column`, `distributions` (keys such as `"normal"`, `"weibull"`, `"weibull-threshold"`, `"gamma"`, `"log-logistic-threshold"`, `"smallest-extreme-value"`; all fourteen when absent), `transformations` (`true`), `plots` (4)
`best-subsets`	`response`, `predictors`, `effects`, `forced` (effects in every model, written like `effects`), `heredity` (`"strong"`, `"weak"`, `"none"`), `models_per_size` (3), `max_size` (`()` for every candidate), `criterion` (`"r-squared"`, `"adjusted-r-squared"`, `"cp"`, `"aicc"`, `"bic"`; picks the best overall)
`generalized-linear-model`	`response`, `predictors`, `family` (`"binomial"`, `"poisson"`, `"normal"`, `"gamma"`), `link` (`"logit"`, `"probit"`, `"cloglog"`, `"log"`, `"identity"`, `"inverse"`; `()` for the family's default), `event` (binomial: the response value modelled; `()` for the second of its two values in natural order), `effects` (as for `linear-model`; main effects and two-factor interactions), `splines` (one entry per predictor: `()` for a straight line, or `#{ kind: "natural"`
`ordinal-logistic`	`response`, `predictors`, `effects` (as for `linear-model`), `order` (the categories, lowest first; sorted order when absent)
`nominal-logistic`	`response`, `predictors`, `effects`, `reference` (the reference category; the first category when absent)
`dose-response`	`dose`, `response` (number responding, or 0/1), `trials` (number of subjects; `()` for one per row), `link` (`"probit"`, `"logit"`, `"cloglog"`), `scale` (`"log10"`, `"ln"`, `"linear"`), `probabilities` (`[0.1, 0.5, 0.9]`), `level` (0.95), `heterogeneity_alpha` (0.05)

Kind	Main settings
`nonlinear-regression`	`response`; either `form` (`"exponential"`, `"exponential_asymptote"`, `"logistic3"`, `"logistic4"`, `"gompertz"`, `"michaelis_menten"`, `"power"`, `"rational"`, `"line"`, `"dose_response_log"`, `"dose_response"`, `"logistic5"`, `"weibull_growth"`, `"beta_growth"`, `"segmented"`, `"two_exponentials"`, `"gaussian_peak"`, `"lorentzian_peak"`) with `predictor`, or `formula` (columns and parameters as variables), or `rate` (dy/dx) with `predictor`, `initial` (the parameter holding y at the start) and optional `start_x`, or `equation` (zero on an implicit curve in the response and one other column); `parameters` (`[#{ name, start, lower, upper, shared }, …]`, one per parameter in model order; a library form's `start` may be `()` for the automatic value, a formula's may not; `shared` (`true`) applies with `group`), `group` (data-set column), `weighting` (`"equal"`, `"inverse_y2"`, `"inverse_y"` or `#{ column: "W" }`), `method` (`"least_squares"`, `"robust"`, `"outliers"`) with `q` (0.01), `compare` (another form), `level` (0.95), `profile` (`true`), `max_iterations` (200). An assay curve template is `template: #{ key, x, second, dilution, constants: #{ … }, cv_limit, tolerance, end_tolerance, min_fraction, min_levels, standard }` together with the `formula` and `parameters` it produced (Model ▸ Curve templates… records them; `key` is one of the template keys such as `"standard_4pl"`, `"mixed_inhibition"`, `"gaddum_schild"`, `"relative_potency"`; column members are column names). Saves `predicted`, `prediction_formula`, `residual` and `inverse_predicted`.

Kind	Main settings
`treatment-effects`	`treatment` (a column with two values), `treated_level` (the treated value; `()` for the second in natural order), `outcome` (numeric), `outcome_kind` (`"continuous"`, `"binary"` (0/1), `"count"`), `covariates` (columns measured before treatment; categorical ones enter as level indicators), `method` (`"weighting"`, the default, `"doubly-robust"`, `"matching"`), `estimand` (`"ate"`, `"att"`), `ratio` (matches per row; 1), `replace` (`false`; the ATE needs `true`), `caliper` (SD of the logit of the score; 0.2, `()` for none), `stabilized` (`true`), `trim` (leave out scores outside [trim, 1 − trim]; 0), `intervals` (`"sandwich"`, `"bootstrap"` with `replicates` (500) and `seed` (1)), `level` (0.95), `subgroups` (columns whose levels get their own estimates). Saves `propensity`, `weight` and, for matching without replacement, `matched_set`.
`method-comparison`	`reference` (X), `test` (Y), `deming` and `passing_bablok` (both `true`), `variance_ratio` (Var(error of Y)/Var(error of X); 1), `level` (0.95)
`shelf-life`	`time`, `response`, `batch` (`()` for one batch), `lower` and/or `upper` (acceptance limits), `pooling_alpha` (0.25), `level` (0.95), `horizon` (longest time searched; twice the longest time observed when absent), `time_unit` (`"months"`, `"weeks"`, `"days"`, `"years"` or `"none"`; for ICH Q1E's limit of 12 months beyond the data. Absent means `"none"`, which applies only the twice-the-data limit), `batches` (`"random"` fits batch as a random effect instead of the fixed-batch pooling analysis; needs 3 or more batches), `bound` (with random batches: `"future-batch"` (default) or `"mean"`)

Kind	Main settings
`mixed-model`	`response`, `predictors`, `effects`, `random`, `family` (`"normal"`, `"binomial"`, `"poisson"`, `"gamma"`, `"negative_binomial"`, `"ordinal"`), `reml`, `trials`, `offset` (column added to the linear predictor; GLMMs), `quadrature_points`, `frequency` (whole-number count column; each row stands for that many observations), `weight` (prior weight column; not binomial), `order` (ordinal: the categories, lowest first; natural order when absent), `robust` (`false`; normal: CR2 cluster-robust standard errors on the outermost grouping), `residual_groups` (normal: column whose levels get their own residual variance, with a likelihood-ratio test), `df_method` (`"satterthwaite"` or `"between-within"`), `compare_slices`, `compare_slopes` (as for `generalized-linear-model`), …
`bayesian-estimation`	`parameter` (`"mean"`, `"proportion"`, `"rate"`), `response`, `group` (two groups) or `second` (paired means), `denominator` (trials or exposure per row), `frequency`, `event` (labelled proportion; `()` for the second value in natural order), `prior` (`"default"` (Jeffreys), `"flat"`, `"informative"` with `prior_mean` and `prior_spread`: the prior SD for a proportion or rate, the prior's weight in observations for a mean), `equal_variances` (`false`), `contrast` (two proportions: `"difference"`, `"ratio"`, `"odds-ratio"`), `reference` (one group; `()` for 0, ½ or 1), `rope` (`[low, high]` or `()`), `level` (0.95)
`bayes-factor`	`test` (`"one-sample"`, `"paired"`, `"two-sample"`, `"correlation"`, `"contingency"`), `first`, `second` (the group column for `"two-sample"`, the column variable for `"contingency"`), `null` (one-sample mean; 0), `sampling` (contingency: `"poisson"`, `"joint-multinomial"`, `"fixed-rows"`, `"fixed-columns"`), `frequency`, `scale` (prior r, κ or a; √2⁄2 for t tests, 1 otherwise), `direction` (`"two-sided"`, `"greater"`, `"less"`), `level`
`bayesian-linear-model`	`response`, `predictors`, `effects` (as for `linear-model`; heredity is added), `continuous_prior` (`"zellner-siow"` or `"hyper-g"`) with `continuous_scale` (r, √2⁄4; or a, 3), `categorical_scale` (½), `model_prior` (`"uniform"`, `"beta-binomial"`), `full_model_only` (`false`), `level`, `draws` (10000), `seed`

Kind	Main settings
`bayesian-model`	`response`, `predictors`, `effects`, `random` (`[#{ group, within, intercept, slopes }]`), `family` (`"normal"`, `"binomial"`, `"poisson"`, `"negative-binomial"`), `trials`, `offset`, `known_sd`, `coefficient_prior` (`"normal"`, `"student-t"` with `coefficient_df`, `"cauchy"`), `coefficient_scale` (2.5), `intercept_location` (`()`: the response's mean), `intercept_scale` (2.5), `sd_prior` (`"exponential"`, `"half-normal"`, `"half-t3"`, `"half-cauchy"`), `sd_scale` (1), `autoscale` (`true`), `chains` (4), `warmup` (1000), `draws` (1000), `seed` (1), `target_accept` (0.8), `level`, `plot_parameter` (the parameter in the trace, rank and density plots)
`penalized-regression`	`response`, `predictors`, `alpha`, `folds`, `seed`
`decision-tree`	`response`, `predictors`, `validation` (`#{ method: "none" }`; `#{ method: "holdout", validation, test, seed, stratify, group }`; `#{ method: "k-fold", folds, seed }`; or `#{ method: "column", column }`), `criterion` (`"gini"` or `"entropy"`), `min_split`, `min_leaf`, `max_depth`, `pruning` (`"one-se"`, `"minimum"` or `"none"`). Its tree section's value is `#{ kind: "tree", classes, nodes: [#{ rule, summary, rows, children, shares, leaf }, …] }`.
`random-forest`	`response`, `predictors`, `validation` (as for `decision-tree`), `trees`, `features_per_split` (0: √p for a categorical response, p/3 for a continuous one), `sample_fraction` (bootstrap size as a share of the training rows, in (0, 1]), `min_split`, `min_leaf`, `max_depth`, `criterion`, `seed`. Saves the same columns as `decision-tree` (values, not formulas).
`boosted-trees`	`response`, `predictors`, `validation` (as for `decision-tree`; validation rows or k-fold choose the number of rounds), `rounds`, `learning_rate` (in (0, 1]), `max_depth`, `min_leaf`, `lambda` (L2 penalty on leaf values), `subsample` (share of training rows per round, in (0, 1]), `early_stopping` (rounds without improvement; 0 never stops), `loss` (`"squared"` or `"absolute"`, continuous responses), `seed`. Saves the same columns as `random-forest`.

Kind	Main settings
`partial-least-squares`	`response`, `predictors` (numeric), `validation` (as for `decision-tree`; validation rows or k-fold choose the number of factors), `max_factors`, `factors` (a fixed number; omit it to choose the number), `algorithm` (`"simpls"` or `"nipals"`), `scale` (standardize predictors; `true` by default). Saves `predicted` with its prediction formula (continuous response), or the same columns as `random-forest`.
`model-search`	`response`, `predictors`, `validation` (as for `decision-tree`; with none, 5-fold cross-validation), `models` (list of `"decision-tree"`, `"random-forest"`, `"boosted-trees"`, `"neural-network"`, `"support-vector-machine"`, `"adaptive-splines"`, `"partial-least-squares"`, `"discriminant"`; all by default), `seed`, `repeats` (1 to 50: repeated k-fold cross-validation with seeds `seed`, `seed` + 1, ...), `nested` (`true`: each model tuned within the outer training folds only). Saves no columns (open a model's own report to save its predictions).
`neural-network`	`response`, `predictors`, `validation` (as for `decision-tree`; validation rows or k-fold tune the penalty, 5 folds when every row trains), `hidden` (one or two node counts, e.g. `[5, 3]`; `[3]` by default), `activation` (`"tanh"`, `"logistic"`, `"relu"` or `"identity"`), `penalty` (a fixed α; omit to tune it), `solver` (`"lbfgs"`, `"adam"` or `"auto"`), `starts` (random starts; 3), `max_iter` (2000), `seed`. Saves `predicted` (or class probabilities and `most_likely`), `prediction_formula` (continuous response) and `validation`.
`gaussian-process`	`response`, `predictors`, `validation` (as for `decision-tree`; with none, each row's held-out prediction is its exact leave-one-out prediction), `kernel` (`"matern52"`, the default, or `"squared-exponential"`), `nugget` (a fixed noise variance as a fraction of the response variance; omit or `null` to estimate it), `starts` (likelihood starts; 10), `seed`; also `show_profiler`, `show_surface_profiler`, `show_observed`. Saves `predicted`, `std_error` and `validation`.

Kind	Main settings
`support-vector-machine`	`response`, `predictors`, `validation` (as for `decision-tree`; validation rows or k-fold tune C and γ, 5 folds when every row trains), `kernel` (`"radial"`, `"polynomial"` or `"linear"`), `degree` (polynomial kernel; 3 by default), `cost` and `gamma` (fixed values; omit either to tune it), `epsilon` (regression, in response standard deviations; 0.1 by default), `seed`. Saves the same columns as `random-forest`.
`adaptive-splines` (also `mars`)	`response` (continuous), `predictors`, `linear` (continuous predictors that enter only linearly, no knots), `validation` (as for `decision-tree`; validation rows or k-fold choose the number of terms; with every row training GCV chooses it), `degree` (greatest interaction degree; 1 by default), `max_terms` (forward-pass terms including the intercept; 0 for min(200, max(20, 2p)) + 1), `penalty` (GCV cost per knot; 2 additive, 3 with interactions by default), `threshold` (smallest gain in R^2 for a forward step; 0.001), `min_span`, `end_span` (cases between knots and outside the end knots; 0 for Friedman's rules). Saves `predicted`, `prediction_formula` (a formula column of `max` hinges) and `validation`.
`model-comparison`	`response`, `models` (prediction columns, one per model: predicted values, or one class's probability), `event` (the event class of a categorical response; `()` for the last class), `validation` (a column of roles, or `()`).
`response-surface`	`response`, `factors`, `options`
`mixture-model`	`response`, `components`, `model`

Kind	Main settings
`several-responses`	`responses`, `predictors`, `model` (`"main-effects"`, `"interactions"`, `"quadratic"`), `goals` (one per response: `()` for the default, or `#{ goal: "maximize" | "minimize" | "target" | "none", low, target, high, shape, shape_above, importance }`), `settings` (one per predictor: a number or a level), `simulation` (`#{ runs, seed, factors: [#{ distribution: "fixed" | "normal" | "uniform" | "triangular", spread }, …], noise, lower_spec, upper_spec, region }`, where `region` is one `[low, high]` per predictor: the box whose corners are checked for the probability of meeting every specification), `constraints` (linear constraints and `exclude` lines in the factors' units, one per line as for optimal designs, which the best settings keep to; `""` for none), `selection` (each response's terms chosen by its own stepwise search: `#{ direction: "forward" | "backward" | "mixed", criterion: "p-value" | "aic" | "bic", enter, remove, heredity: "strong" | "weak" | "none", candidates }`, where `candidates` lists effects as `[[predictor position, power], …]` and is left out for the `model` terms; `()` or absent fits every term). Its sections include the best settings found, the simulation at the given settings and, with a region, its corners and contour plot.
`distribution`	`column`, `quantile_type` (1–9; the application's default is 8), `options`. Besides the display switches, `options` takes `tolerance: #{ proportion, confidence, sides: "two-sided" | "lower" | "upper", method: "normal" | "distribution-free" }`, `outliers: #{ method: "grubbs" | "dixon" | "generalized-esd", side: "two-sided" | "greater" | "less", alpha, dixon_ratio: "r10" … "r22", max_outliers }` and `runs: #{ reference, alternative }` (each `()` or absent when off). Censored values: `censor` (a flag column, non-zero = censored) with `censor_direction` (`"left"`, the default: below the value; or `"right"`), or `upper` (a column with each value's upper end; the analysed column is the lower end, an empty end is open).

Kind	Main settings
`bivariate`	`x`, `y`, `pairwise` (for a continuous and a categorical column with three or more groups: `"tukey-kramer"`, the default, `"games-howell"`, `"holm-t"`, `"bonferroni"`, `"sidak"`, `"holm-sidak"`, `"fisher-lsd"`, `"newman-keuls"`, `"tamhane-t2"`, `"dunnett-t3"` or `"none"`)
`one-sample`, `compare-groups`, `paired`	columns, `alternative`, `level`, ...; `compare-groups` also takes `method` (all-pairs comparisons: the `pairwise` keys of `bivariate` other than `"none"`, or `"dunn"`), `linear_trend` (`true`: a linear-trend contrast across the groups in order), `control` (a group label: comparisons with that control by Dunnett's method), `best` (`"largest"` or `"smallest"`: Hsu's comparisons with the best) `ratio_of_means` (`true`: two groups, the ratio of means with Fieller's interval) and `median_ties` (`"below"`, `"above"`, `"ignore"`: how the median test counts values at the median); `paired` takes `equivalence` (`[low, high]`) and `ratio_equivalence` (`true`: test the ratio of geometric means, with the bounds as ratios). A `linear-model`'s `options` also take `best`, `pairwise` (the keys above) and `linear_trend` for one-way models, and `show_effect_sizes` (η^2, partial η^2, ω^2, Cohen's f with intervals) for any model.
`many-tests`	`columns` (numeric), `group` (a column with two values), `test` (`"welch"`, the default, `"pooled"` or `"rank-sum"`), `adjust` (`"bh"`, the default, `"bonferroni"`, `"sidak"`, `"holm"`, `"hochberg"` or `"by"`), `alpha` (0.05), `order` (`"columns"`, `"p-value"` or `"difference"`: the order of the table's rows)
`screening`	`responses` (numeric columns), `factors`, `model` (`"linear"`, the default, `"poisson"`, `"negative-binomial"`, `"kruskal-wallis"` with one categorical factor), `subgroup` (a column: one screen per level; `()` for none), `fdr` (`"bh"`, the default, `"by"`, `"storey"`), `alpha` (0.05), `order` (`"p-value"`, the default, `"effect"`, `"columns"`). Saves no columns.
`area-under-curve`	`x`, `y` (numeric columns), `group` (one curve per level, or `()`), `baseline` (a value; 0) or `baseline_points` (the mean of the first k points is the baseline), `level` (0.95), `min_peak_share` (0: list every peak). Saves `baseline_subtracted`, `percent_of_range`, `fraction_of_total`.

Kind	Main settings
`multiple-response`	Either `column` (a text column of lists) with `delimiter` (`";"`; `""` treats each cell as one item), or `indicators` (one column per item; a non-zero number, yes, true, x or ✓ is ticked); `group` (cross-tab and tests by a column, or `()`), `order` (`"frequency"`, most ticked first, or `"as-found"`).
`pharmacokinetics`	`subject`, `time`, `concentration`, `treatment` (a column or absent), `dose` (a number) or `dose_column` (absent: no dose, so no CL or Vz), `auc_method` (`"linear-up-log-down"`, the default, or `"linear"`), `administration` (`"extravascular"`, `"iv-bolus"`, `"iv-infusion"` with `infusion_duration`), `terminal` (`"best-fit"`, `"last-points"` with `terminal_points`, or `"time-range"` with `terminal_range: [start, end]`)
`adjust-p-values`	`p_values` (a column), `label` (a column naming the tests, or `()`), `weight` (Stouffer weights, or `()` for equal weights), `alpha` (0.05), `lambda` (Storey's λ, 0.5). Saves `bonferroni`, `sidak`, `holm`, `hochberg`, `bh`, `by` and `q`.
`proportions`, `rates`, `power`	see `r.request()` of a dialog-made report. `power` takes `design` and its `parameters`: `"one-sample-t"`, `"one-sample-z"`, `"paired-t"` (`difference`, `std_dev`, `correlation`), `"two-sample-t"`, `"one-way-anova"`, `"one-proportion"`, `"two-proportions"`, `"one-rate"` (`null`, `actual`, `exposure`), `"two-rates"` (`first`, `second`, `exposure`, `ratio`), `"one-std-dev"`, `"two-variances"`, `"equivalence-one-sample"`, `"equivalence-two-sample"`, `"margin-mean"` (`margin`, `std_dev`), `"margin-proportion"` (`margin`, `proportion`), `"margin-std-dev"`, `"margin-rate"` (`margin`, `rate`, `exposure`), `"tolerance-interval"` (`proportion`, `confidence`, `max_proportion`, `two_sided`: 1 or 0), `"tolerance-distribution-free"` (`proportion`, `removed`: order statistics outside the interval, `two_sided`: 1 or 0; solve for the confidence as the power), `"two-level-factorial"` (`factors`, `fraction`, `center_points`, `interactions`: 1 or 0, `effect`, `std_dev`), `"screening"` (`runs`, `factors`, `center_points`, `effect`, `std_dev`), `"general-factorial"` (`levels`, `other_cells`, `max_difference`, `std_dev`); and `solve` (`"power"` with `n`, `"sample-size"` with `target`, `"effect"` with both)

Kind	Main settings
`goodness-of-fit`	`column`, `frequency` (optional column), `model` (`"proportions"` with `proportions: [#{ level, proportion }, …]`, empty for equal proportions; or `"poisson"` with `min_expected`, 5 by default)
`imputation`	`columns` (numeric; any but date-time columns for chained equations), `method` (`"normal-em"`, `"low-rank-svd"`, `"mean"`, `"median"`, `"multiple-chained-equations"`, `"multiple-normal-model"`); for the two multiple imputations `imputations` (20), `iterations` (10 for chained equations, 50 for the normal model), `donors` (5, predictive mean matching), `column_methods` (one per column: `"pmm"`, `"bayesian-regression"`, `"logistic"`, `"multinomial"`, `"none"` or `()` for the default), `predictors` (a matrix of `true` / `false`, row = column imputed) and `visit_order` (column positions), `seed` (of the first chain; chain *i* uses seed + *i* − 1); `rank` (low-rank SVD; `()` chooses it on held-out values), `shrinkage` (0; above 0 replaces the regularization), `regularize` (`true`: regularized iterative PCA), `standardize` (`true`), `max_iterations` (1000), `tolerance` (1e-8), `seed`. Saves `imputed1`, `imputed2`, … (the completed columns, in order) and `count` (values imputed per row).
`multiway-table`	`rows`, `columns` (classification columns, outermost first; at least one in all), `frequencies` (`"count"`, `"row-percent"`, `"column-percent"`, `"total-percent"`; `["count"]` when absent), `measures` (continuous columns), `statistics` (`"n"`, `"mean"`, `"sd"`, `"median"`, `"min"`, `"max"`, `"sum"`; `["mean"]`), `frequency` (optional count column), `margins` (`true`), `subtotals` (`false`)
`stratified-tables`	`row`, `column`, `strata` (stratum columns; `[]` for one table), `frequency` (optional count column), `confidence` (0.95, of the common odds ratio), `exact` (`true`: Fisher's exact test of each table), `monte_carlo_tables` (10000, used when a table is too large for the exact computation), `seed` (1), `fdr` (`"bh"`, `"by"`, `"storey"`, or `()` for off: each stratum's p-values adjusted across the strata)
`crossover`	`response`, `subject`, `period`, `treatment`, `reference` (a treatment value; the first in order when absent), `scale` (`"ratio"` or `"difference"`), `bounds` (`[low, high]`; 0.8 and 1.25 on the ratio scale by default), `alpha` (0.05)

Kind	Main settings
`factorial-repeated-measures`	`response`, `subject` (long data: one row per subject and within-subject cell; replicates are averaged, incomplete subjects dropped), `within` (one to three within-subject factors), `between` (between-subject factors; four factors at most in all), `simple_effects` (optional `#{ factor, by }`: the test of `factor` at each level of `by`)
`nested-comparison`	`response`, `group`, `subject` (subjects nested in groups; each subject's label is read within its group), `level` (0.95)
`distribution-calculator`	`family` (a key such as `"normal"`, `"weibull"`, `"noncentral-t"`, `"binomial"`), `parameters` (`#{ key: value }`, defaults for any left out), `region` (`"below"`, `"above"`, `"between"`, `"outside"`), `a`, `b`, `probability`; uses no columns
`principal-components`	`columns`, `standardize`
`clustering`	`columns`, `standardize`, `method` (`"k-means"`, `"hierarchical"`), `k`, `seed`, `linkage`, `choose_k` (k-means: `()` or `#{ max_k, references, reference }` with `reference` `"data-range"` or `"principal-components"`: the gap statistic and average silhouettes for k = 1 … `max_k`), `distance` (hierarchical: `"euclidean"`, the default, a metric of the `distance-matrix` table operation such as `"gower"`, or `"matrix"` when the columns are a distance matrix)
`correspondence`	`rows`, `columns`
`correlations`	`columns` (two or more), `missing` (`"pairwise"`, `"listwise"`), `level` (0.95), `spearman`, `partial`, `covariance` (true/false)
`outlier-distances`	`columns`, `distance` (the one plotted: `"mahalanobis"`, `"jackknife"`, `"t2"`), `alpha` (upper-tail probability of the limit, 0.05)
`manova`	`responses` (two or more continuous columns), `predictors`, `effects` (as for `linear-model`; `[]` for main effects), `statistic` (ranking the Effect Pareto: `"pillai"`, the default, `"wilks"`, `"hotelling-lawley"`, `"roy"`), `contrasts` (`[#{ name, factor, weights: #{ level: weight, … } }, …]`, weights adding to zero), `plot_factor` (a categorical predictor for the canonical plot; `()` for the first)

Kind	Main settings
`latent-class`	`items` (nominal or ordinal columns), `min_classes` (1), `max_classes` (5), `starts` (10), `smoothing` (0), `seed` (1). Saved columns: `class`, `probability_1`, `probability_2`, …
`text-exploration`	`column` (a text column), `case_fold` (true), `remove_stop_words` (true), `extra_stop_words` (`[]`), `stem` (false), `min_length` (2), `keep_numbers` (false), `min_document_frequency` (2), `max_terms` (0: all), `weighting` (`"tf-idf"`, `"log-tf-idf"`, `"count"`, `"log-count"`, `"binary"`), `table_terms` (100), `chart_terms` (20), `cloud_words` (80), `phrase_length` (3; below 2 for no phrases), `min_phrase_count` (2), `dimensions` (2; 0 for none), `clusters` (0), `seed` (1). The word cloud section's value is `#{ kind: "word-cloud", words: [#{ text, count, size, rows }] }`. Saved columns: `dimension_1`, `dimension_2`, …, `cluster`
`topic-model`	the term settings of `text-exploration` (`column` … `max_terms`), `topics` (5), `alpha` (`()` for 50 ÷ topics), `beta` (0.1), `iterations` (1000), `burn_in` (200), `seed` (1), `top_terms` (8). Saved columns: `topic` (most likely), `topic_1`, `topic_2`, … (proportions)
`sentiment`	`column`, `negation` (true), `negation_window` (3), `lexicon` (`#{ word: valence, … }` on −4…4, added to or replacing the bundled lexicon), `threshold` (0.05, the neutral band). Saved columns: `compound` (score), `sum`, `sentiment` (category), `positive_words`, `negative_words`
`gaussian-mixture`	`columns`, `structures` (any of `"EII"`, `"VII"`, `"EEI"`, `"VVI"`, `"EEE"`, `"VVV"`; all six by default), `min_components` (1), `max_components` (6), `standardize` (false), `start` (`"k-means"` or `"hierarchical"`), `starts` (5), `seed` (1). Saved columns: `probability_1`, `probability_2`, …, `cluster`, `uncertainty`
`embedding`	`method` (`"classical-mds"`, `"metric-mds"`, `"t-sne"`, `"umap"`), `columns`, `distances` (true: the columns are a distance matrix, one per row), `standardize` (true), `dimensions` (2), `perplexity` (30, t-SNE), `neighbours` (15, UMAP), `min_dist` (0.1, UMAP), `iterations` (`()` for the method's default), `seed` (1), `color` (a column name or `()`). Saved columns: `dim1`, `dim2`, …

Kind	Main settings
`principal-coordinates`	`columns`, `distances` (true: the columns are a distance matrix), `metric` (as for the `distance-matrix` table operation; `"gower"`), `standardize` (`false`), `correction` (`"none"`, `"lingoes"`, `"cailliez"`), `color` (a column name or `()`). Saved columns: `pco1`, `pco2`, `pco3`.
`item-analysis`	`items` (two or more columns), `reversed` (item columns scored low + high − x), `scale` (`[low, high]`, or `()` for the smallest and largest answer), `level` (of alpha's interval, 0.95)
`factor-analysis`	`columns` (three or more numeric), `factors` (a number; omit it for the eigenvalues above 1), `extraction` (`"maximum-likelihood"` or `"principal-axis"`), `rotation` (`"varimax"`, `"promax"` or `"none"`), `matrix` (`"correlation"` or `"covariance"`), `scores` (saved scores: `"regression"` or `"bartlett"`), `promax_power` (4). Saves `factor1`, `factor2`, ….
`discriminant-analysis`	`response` (categorical), `predictors` (numeric), `rule` (`"linear"` or `"quadratic"`), `priors` (`"proportional"` or `"equal"`), `validation` (as for `decision-tree`; validation and test rows are only classified, k-fold adds held-out predictions). Saves the same columns as `random-forest` plus `canonical1`, `canonical2`, … (canonical scores).
`functional-data`	`curve`, `time`, `value` (long data, one row per reading), `factors` (curve-level columns for the score model; `[]` for none), `basis` (`"b-spline"`, the default, or `"fourier"`), `order` (4, B-splines), `functions` (omit for automatic), `period` (Fourier; omit for the time range), `smoothing` (`"gcv"`, `"fixed"` with `lambda`, or `"none"`), `landmarks` (`[#{ feature: "maximum" or "minimum", from, to }, …]`; leave out `from` / `to` to search the whole range), `continuous_registration` (false), `warp_functions` (6), `warp_penalty` (0.0001), `components` (omit for 90% of the variation, at most six), `interactions` (false), `confidence` (0.95). Saved columns: `fpc1`, `fpc2`, …, `smoothed`, `registered_time` (when registered).

Kind	Main settings
`control-chart`	`column`, `chart` (`"mean-range"`, `"mean-std-dev"`, `"individuals-moving-range"`, `"moving-average"`, `"three-way-range"`, `"three-way-std-dev"`, `"zone"`, `"short-run"`), `subgroup_size` (used when there is no `subgroup` column; 1 plots individual values on a moving-average or zone chart), `subgroup`, `phase`, `label`, `part` (optional columns; `part` is required for `short-run`), `span` (moving-average span, default 5), `sigma_multiple` (k of the kσ limits, 3; individuals, X̄–R and X̄–S), `range_span` (values per moving range of an individuals chart, 2), `standard` (`#{ center, sigma }`: limits from standard values, or `()`), `last` (`#{ count, limits_from: "all" or "last" }`: show only the last subgroups), `options` (display switches, `show_run_chart` and `run_center` (`"median"` or `"mean"`), and `location_rules` / `dispersion_rules` / `within_rules`: the run-rule numbers 1–8 applied to each chart, `within_rules` for a three-way chart's within-subgroup chart). Zone-chart points carry `note` (the cumulative score) and `signal`. Its chart sections' values carry `points` (with `label`), `limits` (`#{ from, to, center, lower, upper }`, one per stretch of equal limits) and `phases` (`#{ at, label }`).
`ewma-chart`, `cusum-chart`	`column`, `lambda` / `width` or `k` / `h`
`attribute-chart`	`chart`, `count`, `size`
`pareto-chart`	`category`, `frequency` and `group` (optional columns), `options` (`other_percent`: pool categories below this percentage into "Other", 0 for none; `percent_scale`). The chart section's value has `panels`, each with `label`, `total` and `bars` (`label`, `value`, `share`, `cumulative`, `other`, `rows`).
`cause-and-effect`	`causes` (one to three columns: branches, causes, sub-causes), `effect`. The diagram section's value has `effect` and `branches` (each `label`, `rows`, `children`).
`multi-vari`	`response`, `factors` (one to four columns, outermost first), `options` (`show_values`, `show_ranges`, `show_cells`, `replicates`: `"in-order"` (default) or `"stacked"`)

Kind	Main settings
`multivariate-chart`	`chart` (`"hotelling-t2"`, `"generalized-variance"`, `"mewma"`), `columns` (two or more), `subgroup` (an ID column) or `subgroup_size` (1: individual observations), `label`, `reference` (points in the reference sample, from the start; `()` for all), `alpha`, `lambda`, and `mewma_limit` (a value of H) or `mewma_arl` (in-control average run length; 200 by default)
`pca-chart`	`columns` (two or more), `label`, `reference` (rows in the reference period, from the start; `()` for all), `standardize` (true: correlations), `components` (a number, or `()` to choose by cross-validation), `folds` (7), `alpha` (0.0027), `spe_limit` (`"jackson-mudholkar"`, `"scaled-chi-square"`)
`monitoring-limits`	`value`, `readings` (`"counts"`, `"positives"`, `"continuous"`), `samples` (positives: a column or a number), `location`, `order` (optional columns), `limits_by` (`"all"`, `"location"`, or a column name), `family` (a family key such as `"negative-binomial"`, `"zero-inflated-poisson"`, `"beta-binomial"` or `"lognormal"`; `()` for the best by AIC), `levels` (`"quantiles"` with `alert` / `action` probabilities, 0.975/0.9985; `"sigma"` with `alert` / `action` multiples, 2/3; `"typed"` with `typed: [#{ group, alert, action }]`, an empty group for every other; `"columns"` with `alert` / `action` column names), `alerts_in_window` (`[n, m]`, `()` to skip; `[2, 5]`), `rising_run` (increases in a row, `()` to skip; 3), `signals_only` (false)
`analysis-of-means`	`data` (`"means"`, `"proportions"`, `"rates"`), `response`, `group`, `second_group` (means only), `size` (trials or exposure: a column name or a number), `alpha`
`process-capability`	`column`, `lower`, `upper`, `target`, `within`, `subgroup_size`, `box_cox`, `distribution` (`normal`, a family key such as `weibull` or `lognormal-threshold`, `best-fit`, `johnson`, `nonparametric`), `by`, `group_order`, `probability_plot` (as below)
`between-within-capability`	`column`, `subgroup` (a column) or `subgroup_size`, `lower`, `upper`, `target`, `within` (`"pooled-sd"`, `"average-range"`), `between` (`"moving-range"`, `"variance-components"`)
`attribute-capability`	`model` (`"binomial"`, `"poisson"`), `count`, `size` (a column, a number, or `()` for one unit per row, Poisson only), `level` (0.95)

Kind	Main settings
`acceptance-sampling`	`design` (`"single"`, `"double"` with `second_ratio`, `"variables"` with `sigma`: `"known"` / `"unknown"`, `"none"`), `model` (`"binomial"`, `"hypergeometric"`, `"poisson"`), `aql`, `alpha`, `ltpd`, `beta`, `lot_size`, `plans` (`[#{ type: "single", n, c }, #{ type: "double", n1, c1, n2, c2 }, #{ type: "variables", n, k, sigma }]`); uses no columns
`attribute-chart`	`chart` (`"p"`, `"np"`, `"c"`, `"u"`, `"laney-p"`, `"laney-u"`, `"g"`, `"t"`), `count`, `size` (not used by c, g and t), `phase` (optional column), `rules` (run-rule numbers 1–8; default `[1]`)
`process-capability`	`column`, `lower`, `upper`, `target`, `within`, `subgroup_size`, `box_cox`, `distribution` (`normal`, a family key such as `weibull` or `lognormal-threshold`, `best-fit`, `johnson`, `nonparametric`), `layout` (`"standard"` or `"summary"` for the summary page), `bootstrap` (`#{ resamples, seed }`: bootstrap intervals for the percentile indices of a non-normal model), `by` (a column whose groups are also studied one by one), `group_order` (`"labels"`, `"ascending"` or `"descending"` by Ppk, for the Ppk by group chart), `probability_plot` (`true` adds the probability plot with the limits to the standard layout)
`gauge-study`	`part`, `operator`, `measurement`, `study_multiplier`, `tolerance`, `pool_interaction` (crossed), `design` (`"crossed"`, the default, `"nested"` or `"expanded"`), `run_order` (a numeric column, or `()` for row order), and for an expanded study `factors` (`[#{ column, random: true | false, nesting: "crossed" | "within-part" | "within-operator" }, …]`), `parts_within_operators` (false) and `interactions` (true); `options` takes `show_anova`, `show_part_plot`, `show_operator_plot`, `show_run_chart`
`gauge-linearity`	`reference`, `measurement`, `confidence` (0.95), `process_variation` (a number, or `()` for none)

Kind	Main settings
`gauge-repeatability`	`measurement`, `reference`, `tolerance` (both required), `percent_of_tolerance` (K, 20), `spread` (L, 6), `confidence` (0.95), `direction` (a column with two values: the direction each reading approached from; adds hysteresis), `order` (a numeric time or order column for the drift; `()` for the row order), `show_drift` (`true`), `summary_statistics` (keys among `"n"`, `"mean"`, `"sd"`, `"se"`, `"min"`, `"median"`, `"max"`, `"range"`, `"six_sd"`, `"cv"`; default n, mean, sd, min, max, range)
`intraclass-correlation`	`part`, `operator`, `measurement`, `confidence` (0.95), `increment` (the recorded measurement increment, or `()` to read it from the data's decimals)
`attribute-agreement`	`part`, `appraiser`, `rating`, `trial` and `standard` (optional columns, `()` for none), `confidence` (0.95), `ordinal` (default: true for a numeric or ordinal rating column), `order` (category labels lowest first; others follow in natural order)
`attribute-gauge`	`reference`, `acceptance` (times accepted, or each check's result), `checks` (times checked; `()` for one row per check), `accept_label` (the result meaning accepted, one row per check), `limit`
`signal-to-noise`	`factors`, `responses` (one column per outer-array run or repeat), `ratio` (`"smaller"`, `"larger"`, `"nominal"`, `"nominal-variance"`)
`declared-design`	`factors` (column names, or `#{ column, low, high }` with `()` for the data's range), `responses`, and for a split-plot design `whole_plot` and `subplot` (columns) and `model` (the formula the design was built for)
`design-evaluation`	`factors` (as for `declared-design`), `model` (formula over the factor names such as `"A + B + A*B + A^2"`, or `"main effects"`, `"interactions"` (default), `"quadratic"`), `alias` (formula of alias terms; `""` for the default), `coefficient` (anticipated coefficient in units of σ, one number or a list with one per non-intercept term; 1), `alpha` (0.05), `compare` (column whose two levels are two designs to compare; `()` for none), `constraints` (constraints in the factors' actual units, one per line as for optimal designs; `""` (default) for the whole cube)

Kind	Main settings
`factorial-analysis`	`factors` (as for `declared-design`), `response`, `terms` (`"all"`, the default: every estimable interaction; a number: interactions of up to that many factors; or a formula such as `"A + B + A*B"`), `response-type` (`"continuous"`, `"binary"`, `"count"`), `alpha` (0.05), `means` (`"data"` or `"model"`), `cube` (up to three factor columns; empty for the first three)
`sequential-optimization`	`responses`, `factors` (continuous or categorical), `goals` (one object per response: `goal` `"maximize"`, `"minimize"` or `"target"` with `target`, optional desirability `low` and `high`, and `importance`; maximize by default), `bounds` (one `[low, high]` or `null` per factor; `null`: the data's range), `constraints` (text, one per line, as for optimal designs), `round` (column numbering the rounds), `criterion` (`"expected-improvement"`, the default, or `"upper-confidence-bound"` with `kappa`), `batch` (4), `kernel`, `nugget`, `starts` (5), `stop_improvement` (fraction of the observed range; 0.01), `stop_value`, `candidates` (2000), `seed`. Saves no columns; add the proposed runs from the report's menu, or read them from its Proposed Runs section.
`survival`	`time`, `status`, `status_coding`, `group`, `options` (`options` take, among others, `trend` (log-rank test for trend), `risk_table` (number at risk under the plot) and `risk_censored` (censored counts too))
`life-distributions`	`layout` (`"times"`: `time`, `censor` (optional; a censored row is still running), `censor_coding` (`"nonzero-censored"`, the default, or `"nonzero-event"`); `"intervals"`: `start` and `end`, empty start = failed by the end, empty end = still running, equal = exact), `frequency`, `group` (optional columns), `families` (keys as for `process-capability`: `"exponential"`, `"weibull"`, `"lognormal"`, `"log-logistic"`, `"gamma"`, `"normal"`, `"logistic"`, `"smallest-extreme-value"`, `"largest-extreme-value"`, and `"…-threshold"` forms), `detail` (the family shown in detail; absent for the best by AIC), `confidence` (0.95), `probabilities` (fractions failing whose times are estimated; `[0.01, 0.1, 0.5]`), `times` (times whose failure probabilities are estimated), `known_shape` (Weibull β for the known-shape fit; absent for none)

Kind	Main settings
`life-stress`	the `life-distributions` layout keys (`layout`, `time`, `censor`, `censor_coding`, or `start` and `end`) and `frequency`; `stresses` (one or two objects: `column`, `relationship` — `"arrhenius"` (the default), `"eyring"`, `"inverse-power"` or `"exponential"` (also `"linear"`) — and, for temperatures, `unit`: `"celsius"` (the default) or `"kelvin"`), `use` (the use condition of each stress, e.g. `[10]`), `interaction` (`false`), `distribution` (`"weibull"`, the default, `"lognormal"`, `"log-logistic"`, `"exponential"`), `stress_dependent_scale` (`false`), `confidence` (0.95), `probabilities` (fractions failing whose times at use are estimated; `[0.01, 0.1, 0.5]`), `times` (times whose failure probabilities at use are estimated)
`fatigue`	`cycles`, `stress`, `runout` (non-zero = did not fail; absent: all failed), `frequency`, `form` (`"basquin"`, `"semi-log"`, `"quadratic-log"`, `"endurance-limit"`), `distribution` (`"lognormal"`, `"weibull"`, `"log-logistic"`), `stress_dependent_scale` (`false`), `confidence` (0.95), `probabilities` (fractions failing for the quantile curves; `[0.1, 0.5]`), `lives` (cycles for the fatigue strength; `[1e6, 1e7]`), `stresses` (for life quantiles; empty: the tested levels)
`degradation`	`layout` (`"repeated"`, the default: `unit`, `time`, `response`; `"destructive"`: `time`, `response`), `path` (`"linear"`, the default, with `response_transform` and `time_transform` each `"none"`, `"log"` or `"sqrt"`; `"asymptotic"`; `"power"`), `threshold` (required), `direction` (`"rising"`, the default, or `"falling"`), `method` (`"pseudo-failure-times"`, the default, or `"random-effects"`), `distribution` (life distribution of the pseudo failure times, as for `life-stress`; `"weibull"`), `response_distribution` (destructive: `"normal"`, the default, `"logistic"`, `"smallest-extreme-value"`, `"largest-extreme-value"`), `stress` (optional, an object as in `life-stress stresses`) with `use` (its use condition), `confidence` (0.95), `probabilities` (`[0.1, 0.5, 0.9]`), `times`, `bootstrap` (random-effects bootstrap replicates; 200, 0 for none)

Kind	Main settings
`proportional-hazards`	`time`, `censor` (optional), `censor_coding` (`"nonzero-censored"`, the default, or `"nonzero-event"`), `predictors`, `effects` (as for `linear-model`), `strata` (optional column), `ties` (`"efron"` or `"breslow"`), `confidence` (0.95), `ph_transform` (time scale of the proportional-hazards test: `"km"`, `"rank"`, `"identity"`, `"log"`), `curve_by` (a predictor whose levels or percentiles get their own survival curve; optional), `times` (times at which survival is tabulated), `profile` (`"hazard-ratio"` or `"survival"`), `profile_time` (for the survival profile; the median time when absent), `show_profiler`, `show_surface_profiler`. Saves `linear_predictor`, `hazard_ratio`, `survival`, `martingale` and `deviance`.
`survival-regression`	`layout`, `time`, `censor`, `censor_coding`, `start`, `end`, `frequency` (as for `life-distributions`), `predictors`, `effects` (as for `linear-model`), `distribution` (`"weibull"`, `"lognormal"`, `"loglogistic"`, `"exponential"`), `confidence` (0.95), `curve_by` (optional predictor), `probabilities` (fractions failing whose times are predicted; `[0.1, 0.5]`), `times` (times whose failure probabilities are predicted), `profile_probability` (fraction failing the Response Explorer shows; 0.5), `show_profiler`, `show_surface_profiler`. Saves `predicted` (the time to `profile_probability` failing) and `cox_snell`.
`reliability-growth`	`time` (cumulative test time), `event` (optional; 0 marks an end-of-test row), `phase` (optional), `end_time` (end of a time-terminated test without an end row), `beta` (`"unbiased"` or `"maximum-likelihood"`), `confidence` (0.95)
`repairable-systems`	`system`, `time` (age at repair), `event` (optional; 0 marks the end of observation), `group` (optional), `variance` (`"robust"` or `"poisson"`), `limits` (`"log"`, the default, or `"normal"`), `confidence`

Kind	Main settings
`competing-risks`	`time`, `cause` (the cause or failure-mode column), `censor_value` (the cause value meaning still running; absent when every row failed), `group`, `frequency` (optional columns), `covariates` (columns for subdistribution hazards regression; nominal ones are reference-coded), `regression_cause` (the cause modelled; absent for the first), `profiles` (covariate settings to predict at, each an array of values in covariate order), `stacked` (`false`), `mode_family` (family key, as for `life-distributions`, fitted to each failure mode; absent for none), `times` (times whose system failure probability is estimated), `confidence` (0.95)
`warranty-returns`	`layout` (`"triangular"`, the default: `shipped`, `returns` (one column per calendar period, in order), `label` (optional period label column); `"long"`: `ship_period`, `return_period`, `returns` (units returned) and `shipped` columns), `families` (keys as for `life-distributions`; Weibull, lognormal, exponential and log-logistic by default), `detail` (the family used for the forecast; absent for the best by AIC), `horizon` (periods to forecast; 12), `planned` (units shipped in future periods), `warranty` (periods of service in which returns are accepted; absent for no limit), `confidence` (0.95)
`series-description`	Every time series analysis takes `column` (the numeric series, rows in table order; an excluded or missing row inside it is an error), `time` (optional date-time or numeric column labelling the axis and forecast stamps) and `period` (observations per cycle; absent: from the time column's spacing, 12 for monthly dates). This one also takes `box_cox` (`"none"`, the default, `"guerrero"`, or λ), `differences` (0), `seasonal_differences` (0), `max_lag` (`()`: $10 \cdot \log_{10} n$ and at least two cycles), `bartlett` (`false`), `confidence` (0.95), `adf` (Dickey–Fuller terms: `"none"`, `"constant"`, the default, or `"trend"`), `unit_root` (`true`), `spectrum` (`false`), `spans` (Daniell spans smoothing the periodogram; `[3, 3]`, `[]` for none), `spectrum_axis` (`"frequency"` or `"period"`)

Kind	Main settings
`decomposition`	`column`, `time`, `period` (at least 2), `method` (`"classical-multiplicative"`, the default, `"classical-additive"` or `"stl"`), `seasonal_window` (STL: an odd span of at least 7, or `"periodic"`, the default), `robust` (STL; `false`). Saves `trend`, `seasonal`, `remainder` and `adjusted` (seasonally adjusted).
`trend-models`	`column`, `time`, `period`, `model` (`"linear"`, the default, `"quadratic"`, `"exponential"`, `"logistic"` or `"moving-average"`), `span` (moving average; 3), `horizon` (periods forecast; 12), `confidence` (0.95, of the prediction limits). Saves `fitted` and `residual`.
`cross-correlation`	`column` (the output series), `input` (the input series), `time`, `period`, `max_lag` (`()`: $10 \cdot \log_{10}(n/2)$), `prewhiten` (`"none"`, the default, `"aic"`, or an autoregressive order), `confidence` (0.95). Lag k pairs the input at t with the output at t + k.
`fourier-spectrum`	Every signal analysis takes `column` (the signal), `x` (optional time column: rows are sorted by it, the sampling rate is read from its spacing, and uneven x–y pairs are resampled by linear interpolation onto an even grid), `group` (optional; one signal per level), `sampling_rate` (without `x`; 1). This one adds `window` (`"hann"`, the default, `"rectangular"`, `"hamming"`, `"blackman"`, `"flat-top"`, `"kaiser"` with `window_beta`, `"tukey"` with `window_alpha`), `detrend` (`"mean"`, the default, `"none"`, `"linear"`), `padding` (zero padding factor; 1), `power_of_two` (`false`), `sides` (`"one"`, `"two"`), `scaling` (`"density"`, `"spectrum"`), `decibels` (`true`), `phase` (`false`), `peaks` (5), `welch` (`false`), `spectrogram` (`false`), `segment_length` (256), `overlap_percent` (50)
`digital-filter`	the signal keys, `design` (`"butterworth"`, the default, `"chebyshev1"` with `ripple_db` (1), `"chebyshev2"` with `attenuation_db` (40), `"elliptic"` with both, `"fir"` with `taps` (61) and `window` (Hamming), `"fft"` with `cutoff_shape` (`"ideal"`, `"smooth"`), `"notch"` with `quality` (30)), `order` (4), `band` (`"lowpass"`, `"highpass"`, `"bandpass"`, `"bandstop"`), `low` and `high` (edge frequencies in the units of the sampling rate), `zero_phase` (`true`)

Kind	Main settings
`signal-correlation`	the signal keys, `operation` (`"cross-correlation"`, the default, with `other`; `"autocorrelation"`; both with `scaling` (`"coefficient"`, `"biased"`, `"unbiased"`, `"none"`) and `max_lag`; `"convolution"` with `kernel` and `normalize` (`true`); `"deconvolution"` with `kernel` and `method` (`"wiener"` with `noise_ratio` (0.01), or `"richardson-lucy"` with `iterations` (30)); `"envelope"`)
`wavelet-analysis`	the signal keys, `wavelet` (`"db4"`; `haar`, `db1`–`db10`, `sym2`–`sym10`, `coif1`–`coif5`), `extension` (`"symmetric"`, `"periodic"`, `"zero"`), `levels` (as many as the length allows), `denoise` (`true`), `threshold_rule` (`"universal"`, `"sure"`, `"fixed"` with `threshold`), `shrinkage` (`"soft"`, `"hard"`), `noise` (`"finest"`, `"each-level"`), `scalogram` (`false`), `continuous_wavelet` (`"morlet"`, `"mexican-hat"`), `scales` (48)
`image-spectrum`	Every image analysis takes `column` (the pixel values), `x` and `y` (pixel positions, evenly spaced; y increases downwards) and `group` (optional; one image per level). This one adds `window` (`"separable-hann"`, the default, `"radial-hann"`, `"none"`), `padding` (`"none"`, the default, `"zero"`, `"mirror"`), `remove_mean` (`true`), `phase` (`false`), `radial` (`true`), `angular` (`true`), `profiles` (`true`), `profile_directions` (degrees, `[0, 90]`), `profile_half_width` (wedge half-width in degrees, 0 for a straight cut), `profile_peaks` (`false`; also profile each dominant frequency's direction), `sectors` (36), `peaks` (6), `min_frequency_fraction` (0.02 of the Nyquist frequency)
`image-filter`	the image keys, `band` (`"lowpass"`, the default, `"highpass"`, `"bandpass"`, `"bandreject"`, `"none"`), `low` and `high` (cut-off or band edges in cycles per unit of x; a quarter and a half of the Nyquist frequency), `shape` (`"butterworth"`, `"ideal"`, `"gaussian"`), `order` (2), `notches` (array of `[fx, fy]` pairs), `auto_notches` (0: notch this many of the strongest frequencies), `notch_radius` (two frequency steps), `notch_shape` (`"gaussian"`), `notch_pass` (`false`), `padding` (`"mirror"`)

Kind	Main settings
`image-restoration`	the image keys and `operation`: `"deblur"` (the default) with `method` (`"wiener"`, `"least-squares"`, `"richardson-lucy"`), `lambda` (0.01), `iterations` (30), `sigma_x` and `sigma_y` (Gaussian blur in pixels; 2); `"register"` with `other` (the image to align) and `upsample` (20); or `"autocorrelation"` with `remove_mean` (`true`)
`exponential-smoothing`	`column`, `time`, `period` (as for `series-description`), `model` (`"automatic"`, the default, `"simple"`, `"holt-linear"`, `"damped-trend"`, `"holt-winters-additive"`, `"holt-winters-multiplicative"`, `"custom"` with `error` (`"additive"` / `"multiplicative"`), `trend` (`"none"` / `"additive"` / `"damped"`) and `seasonal` (`"none"` / `"additive"` / `"multiplicative"`)), `allow_multiplicative` and `allow_damped` (automatic only; `true`), `alpha`, `beta`, `gamma`, `phi` (values held fixed; a chosen model only), `horizon` (12), `levels` (`[0.8, 0.95]`)
`arima`	`column`, `time`, `period` (as for `series-description`), `orders` (`"automatic"`, the default, with `stepwise` (`true`) and `allow_constant` (`true`); or `[p, d, q]` with `seasonal_orders` (`[P, D, Q]`, using `period`) and `constant` (`false`; a mean when d + D = 0, a drift when 1)), `box_cox` (`"none"`, the default, `"guerrero"`, or λ; forecasts are back-transformed, so the point forecast is the median), `horizon` (12), `levels` (`[0.8, 0.95]`), `inputs` (input series with transfer functions: a list of maps with `column`, `delay` (a number, or `"automatic"`, the default: the first lag of significant prewhitened cross-correlation), `numerator` (0) and `denominator` (0); rows after the series with the inputs filled in supply their future values)
`signal-smoothing`	`y` (the signal), `x` (optional: orders the points; absent: row order), `group` (optional: one signal per level), `method` (`"savitzky-golay"`, the default, with `window` (11, odd), `order` (3) and `derivative` (0); `"moving-average"` or `"trailing-average"` with `window`; `"exponential"` with `alpha` (0.3); `"running-median"` with `window`; `"running-percentile"` with `window` and `percentile` (50); `"gaussian"` with `sigma` (2, in points))

Kind	Main settings
`curve-calculus`	`x`, `y`, `group` (optional), `derivative` (`"central"`, the default, or `"savitzky-golay"` with `window` (11) and `order` (3)), `integral` (`"trapezoid"`, the default, or `"simpson"`), `resample` (absent for none, or `"linear"`, `"spline"`, `"natural-spline"` or `"pchip"`, with `points` (101))
`spectrum-peaks`	`x`, `y`, `curve` (optional: one spectrum per level), `baseline` (`"arpls"`, the default, with `lambda` (10^5); `"asls"` with `lambda` (10^5) and `p` (0.01); `"snip"` with `half_window` (20), `decreasing` (`true`) and `lls` (`false`); `"column"` with `baseline_column`; `"none"`), `smooth_window` and `smooth_order` (Savitzky–Golay smoothing before the search; 11 and 3, `()` for none), `min_height` (3) and `min_prominence` (5), in noise levels, `min_separation` (0, in x units), `fit` (absent: peaks only; or a map with `shape` (`"gaussian"`, the default, `"lorentzian"`, `"pseudo-voigt"`, `"voigt"`, `"split-gaussian"`, `"split-lorentzian"`), `baseline` (`"constant"`, the default, `"none"` or `"linear"`), `peaks` (how many of the most prominent; absent: every peak found, at most 12), `centres` (starting centres instead), `compare` (`true`), `common_width` (`false`), `spacing` (`"free"`, `"equal"` or a number))
`reliability-test-plan`	`plan`: `"demonstration"` with `reliability`, `time`, `confidence`, `shape` (Weibull β), `failures` and either `test_time` (find the units) or `units` (find the test time); `"estimation"` with `distribution` (`"weibull"`, `"lognormal"`), `spread` (β or σ), `fraction_failing`, `percentile`, `confidence`, `precision`; `"accelerated"` with `distribution`, `spread`, `relationship` (`"linear"`, `"arrhenius"`, `"inverse-power"`), `use_stress`, `high_stress`, `p_use`, `p_high`, `percentile`, `confidence`, `precision`, `levels` (`[[stress, share], …]`, optional). Uses no columns.
`permutation-test`	`design` (`"two-groups"` with `response`, `group` and `statistic` `"mean"` or `"median"`; `"paired"` with `first`, `second`; `"correlation"` with `x`, `y` and `statistic` `"pearson"` or `"spearman"`), `alternative` (`"two-sided"`, `"less"`, `"greater"`), `exact_limit` (enumerate every arrangement up to this many; 100000), `resamples` (random arrangements otherwise; 9999), `seed` (1)

Kind	Main settings
`bootstrap`	`report` (the bootstrapped report's own settings, with its `analysis`), `section` (title of one of its tables), `columns` (headings of the cells to collect), `rows` (labels in the table's first column; empty or absent for every row), `replicates` (1000), `seed` (1), `level` (0.95), `stratify` (a column resampled within its levels, or `()`)
`scatter-3d`	`x`, `y`, `z` (numeric columns), `color`, `size` (numeric), `shape` (optional columns; `()` for none), `drop_lines` (`false`), `camera` (`#{ turn, tilt }` in degrees). Its sections are the plotted-columns table and a note of rows left out; the 3-D plot itself is drawn only in the application.
`data-surface`	`x`, `y`, `z` (the response), `method` (`"triangulation"`, `"spline"` for the interpolating thin-plate spline, `"smoothing-spline"` with λ by generalized cross-validation), `grid` (divisions per axis, 8–120; 40), `levels` (contour levels; 10), `contour_plot`, `surface`, `filled`, `surface_contours`, `floor_map`, `points` (display switches), `camera` (`#{ turn, tilt }`). Its sections are the surface-fit table and the contour plot (its `contours`: the levels and each level's lines, and the observations).
`image-plot`	`x`, `y` (numeric columns placing each row on an evenly spaced grid), `value` (the column coloured), `scale` (`"sequential"`, the default, `"heat"`, `"spectrum"`, `"diverging"`, `"grey"`), `limits` (`[low, high]` for a fixed colour range, or `()` for the data range), `reverse` (`true`: the colours run from the scale's high end to its low end, so the grey scale goes from black for low values to white for high ones), `y_down` (`true`: y grows downwards as in image files), `equal_aspect` (`true`). Its picture section's value is `#{ kind: "image", x_label, y_label, value_label, x_range, y_range, y_down, values }`, where `x_range` and `y_range` are the outer edges of the grid (low, high) and `values` lists the grid row by row, the first row at the top of the picture (NaN for empty cells); its table is the Grid summary.

Kind	Main settings
`graph`	the graph canvas's JSON (`encoding`, layers…), as saved in projects and pipeline files. `encoding.y2` lists right-axis columns; `axes` (optional) holds `title`, `x_title`, `y_title`, `y2_title` (strings, read as math markup: `x_1`, `x^2`, `\alpha`, `\frac{a}{b}`; see the Reference Manual, "Math in labels"), `x_break` and `y_break` (`[from, to]`, in data units: the stretch of the axis left out) and `markup_labels` (`true` reads column names, levels and labels as markup too). Files without these members draw as before. Its `form` object's `kind` is `"panels"` (default), `"treemap"` (`size_by_sum`, `labels`), `"parallel"`, `"multi-vari"`, `"scatter-matrix"`, `"pie"` (`donut`, `size_by_sum`, `labels`), `"radar"` (`group_means`, `fill`), `"polar"` (`angle`: `"compass"`, `"degrees"` or `"radians"`; `connect`), `"wind-rose"` (`sectors`, 4–72, 16 by default; `percent`), `"ternary"`, `"waterfall"` (`total`, `labels`), `"ridgeline"` (`height`, percent of the gap between ridges, 20–400, 150 by default; `rug`) or `"vector-field"` (`scale`, percent of the automatic arrow length, 10–1000); missing settings take their defaults and numbers out of range are clamped. The bars layer's `layout` is `"side_by_side"`, `"stacked"`, `"percent"` or `"lollipop"`.

An analysis that cannot be computed stops the script with the analysis's own message (catch it with `try { … } catch (e) { … }` to carry on). Each successful analysis is stored with the table, so `t.save_project` keeps it.

`t.reports()` lists the table's reports, newest first as the application lists them (those run in the script come before those it was opened with), re-run on the current data.

Function	Effect
`t.update_report(i, #{ settings })`	Change report `i` (numbered as in `t.reports()`) to new settings and display options, in the form `t.analyze` takes; returns the new report.
`t.close_report(i)`	Remove report `i`.

A report's ⋯ **Options ▸ Copy as script** in the application copies the `t.analyze(…)` call that recreates it, every setting and display option included.

Reports

Function	Result
`r.title`, `r.analysis`	Report heading; analysis identifier.
`r.sections()`	Section headings, in order.
`r.has_section(title)`	Whether a section exists.
`r.section(title)`	A section's content as data (see below).
`r.value(title, row)`, `r.value(title, row, column)`	One cell of a table section: the row whose first cell is `row`, in the second column or the named one, e.g. `r.value("Fit Statistics", "R²")`.
`r.request()`	The analysis settings, as a map (the project-file form).
`r.text()` or `print(r)`	The report as Markdown text.
`r.save(path)`, `r.save(path, #{ paper: "a4" })`	`.md` or `.txt`: the report as text. `.html` (or `.htm`): the report as one self-contained web page, every section included (tables as HTML tables, plots as inline SVG, light and dark appearance, fonts embedded; nothing is loaded from elsewhere). `.pdf`, `.docx`, `.pptx`, `.xlsx`: the report as a PDF, Word, PowerPoint or Excel document, every section included, as File ▸ Export reports… writes it (`paper`: `"letter"` or `"a4"`). `.png`, `.svg`, `.pdf`, `.eps`: a graph canvas as a figure (PNG is a 300 ppi picture; a graph canvas's `.pdf` is its figure).
`r.save_section(title, path)`	One section: its data as `.csv`, or tab-separated as `.tsv`, `.tab` or `.txt` (a table as shown; a plot's points with a `Row` column of 1-based row numbers, as in the data view, unlike `t.row(i)`), or a plot as a `.png` (300 ppi) picture or an `.svg`, `.pdf` or `.eps` figure (titled with the section title; excluded rows drawn hollow and gray).
`r.saveable_columns()`	The columns the report can add to its table, as `#{ key, name }` maps (empty if none).
`r.save_columns()`, `r.save_columns([keys…])`	Add all of those columns, or the named ones, to the table the report was run on. Returns the new column names (a number is added to a name already in use).

Columns reports offer: linear models `predicted`, `prediction_formula` (every row with its predictors present, rows without a response too), `residual` (every row with complete data, excluded rows too); generalized linear and penalized models `predicted`, `prediction_formula`; nonlinear regression `predicted`, `prediction_formula`, `residual`; `prediction_formula` adds a formula column of the fitted equation; mixed models `conditional`, `marginal`, `residual`; principal components `pc1`, `pc2`, ...; clustering `cluster`; outlier distances `mahalanobis`, `jackknife`, `t2`; item analysis `total` (the scale score, reversed items flipped); factor analysis `factor1`, `factor2`, ... (factor scores); discriminant analysis `probability_1`, ... `most_likely`, `validation`, `canonical1`, ...; functional data `fpc1`, `fpc2`, ..., `smoothed`, `registered_time`; distribution of positive data `box_cox`; p-value adjustment `bonferroni`, `sidak`, `holm`, `hochberg`, `bh`, `by`, `q`; area under a curve `baseline_subtracted`, `percent_of_range`, `fraction_of_total`. A report saves columns only while its columns still exist under the names it was run with.

Section titles match exactly, then without regard to case, then by the name before any details in parentheses (`"Influence Diagnostics"` finds `"Influence Diagnostics (2 of 60 rows flagged; …)"`).

`r.section(title)` returns:

- **tables:** an array of row maps keyed by column heading. Cells that read as numbers are numbers (as shown in the report, so rounded to its displayed digits); empty cells are `()`; others (such as `"<.0001"`) are text.
- **plots:** a map with `kind: "plot"`, `x_label`, `y_label`, `points` (`#{ row, x, y }`, rows numbered from 0), `line` and `band` (`[x, y]` polylines, or `()`).
- **histograms and bar charts:** `kind: "bars"` and `bars` (`#{ center, width, count, rows }`).
- **box plots, control charts, survival plots:** their statistics and points (`kind` says which).
- **heat maps** (such as "Correlation Heat Map"): `kind: "heat-map"`, `measure`, `rows`, `columns` (labels) and `values` (an array of rows of numbers).
- **notes and formulas** (such as a model's "Prediction Formula"): the text.

Functions that use a table inside a chain

Rhai keeps a variable in use for the whole of a chain of calls on it, so a function inside the chain cannot use the same variable: `t.columns().filter(|c| t.level(c) == "nominal")` fails with “Data race detected”. Give the first call its own statement:

```
let columns = t.columns();
let nominal = columns.filter(|c| t.level(c) == "nominal");
```

Statistical functions

These work on arrays of numbers (such as `t.column("Weight")`) and return a map of results. Missing values (`()`) are left out. The same methods run as reports through `t.analyze` (tolerance intervals, outlier and runs tests as `distribution` options; Dunnett and the median test in `compare-groups`; `best-subsets`; the p-value functions as `adjust-p-values`).

Function	Options (all optional)	Result
`tolerance_interval(values, #{…})`	`proportion` (0.90), `confidence` (0.95), `sides` (`"two-sided"`, `"lower"`, `"upper"`), `method` (`"normal"`: exact normal-theory factor; `"distribution-free"`: order statistics)	`lower`, `upper` (±∞ for an open side), `factor`, `mean`, `std_dev`, `n`, `achieved_confidence` (distribution-free intervals reach at least the confidence asked for, or say how much less a small sample gives)
`grubbs_test(values, #{…})`	`side` (`"two-sided"`, `"greater"` for the largest value, `"less"` for the smallest), `alpha` (0.05)	`statistic` (G), `p_value`, `critical_value`, `value`, `index` (position in `values`, from 0), `mean`, `std_dev`
`dixon_test(values, #{…})`	`ratio` (`"r10"`, `"r11"`, `"r21"`, `"r22"`; by sample size when left out), `side`, `alpha`	`ratio`, `statistic`, `p_value`, `critical_value`, `value`, `index`

Function	Options (all optional)	Result
`esd_test(values, #{…})`	`max_outliers` (⌊n/10⌋, at least 1), `alpha`	`outliers` (how many), `indices`, `steps` (`#{ index, value, statistic, critical_value }` per step)
`best_test([group, …], #{…})`	`best` (`"largest"` or `"smallest"`), `level` (0.95)	`df`, `mse`, `comparisons` (`#{ group, mean, difference, critical_value, lower, upper, conclusion }`; conclusion `"the best"`, `"not the best"` or `"may be the best"`)
`dist_density(family, [parameters], x)`, `dist_cdf(…)`, `dist_quantile(family, [parameters], p)`	— (`x` and `p` may be a number or an array)	the density (probability mass for counts), P(X ≤ x), or the quantile; families by key, parameters in the calculator's order (e.g. `dist_cdf("weibull", [2, 10], 5)`)
`random_values(family, [parameters], n, seed)`	—	an array of `n` seeded draws (the same seed gives the same values)
`dunnett_test(control, [group, …], #{…})`	`level` (0.95), `alternative` (`"two-sided"`, `"greater"`: above the control, `"less"`)	`critical_value`, `df`, `mse`, `comparisons` (`#{ group, difference, std_error, t, p_value, lower, upper }`, groups numbered from 0)

Function	Options (all optional)	Result
`median_test([group, …], #{…})`	`ties` (values equal to the grand median count `"below"`, `"above"`, or `"ignore"`)	`median`, `above`, `below` (counts per group), `chi_square`, `df`, `p_value`, `min_expected`
`runs_test(values, #{…})`	`reference` (the median when left out), `alternative` (`"less"`: too few runs, `"greater"`: too many, `"two-sided"`)	`runs`, `n`, `above`, `below`, `reference`, `expected`, `std_dev`, `p_fewer`, `p_more`, `p_value`, `exact`
`up_down_runs_test(values, #{…})`	`alternative`	as `runs_test`, for runs of increases and decreases (exact up to 400 values)
`p_adjust(p_values, method)`	— (`method`: `"bonferroni"`, `"sidak"`, `"holm"`, `"hochberg"`, `"bh"`, `"by"`, or `"q"` for Storey's q-values with λ = 0.5)	an array of adjusted p-values in the same positions; missing values stay `()` and are not counted
`ranks(values)`, `ranks(values, ties)`	`ties`: `"average"` (default), `"min"`, `"max"`, `"dense"` or `"first"` (row order)	an array of ranks from 1 in the same positions; missing values stay `()` and are not ranked
`q_values(p_values, #{…})`	`lambda` (0.5)	`pi0` (Storey's estimate of the share of true nulls), `lambda`, `q_values` (in the same positions)
`combine_p_values(p_values, #{…})`	`weights` (an array, one per p-value, for Stouffer's weighted Z)	`fisher` and `stouffer`, each `#{ statistic, df, p_value, k }` (`df` is `()` for Stouffer)

Function	Options (all optional)	Result
`best_subsets(y, #{ name: values, … }, #{…})`	`models_per_size` (3), `max_size`, `forced` (names in every model)	an array of models, by size and then best first: `#{ predictors, size, rss, r_squared, adj_r_squared, cp, aicc, bic }`; rows with a missing value are left out
`smooth(values, #{…})`	`method` and its settings as for the `signal-smoothing` analysis; `spacing` (1: the x step, for Savitzky–Golay derivatives)	an array of smoothed values in the same positions (missing values are skipped and stay `()`)
`derivative(x, y)`	—	dy/dx by central differences (any spacing), in the same positions
`cumulative_integral(x, y, #{…})`	`rule` (`"trapezoid"` or `"simpson"`)	the running integral from the smallest x, in the same positions
`interpolate(x, y, at, #{…})`	`method` (`"linear"`, `"spline"`, `"natural-spline"`, `"pchip"`)	y at `at` (a number or an array); `()` outside the range of x
`baseline(values, #{…})`	`method` (`"arpls"`, the default, `"asls"`, `"snip"`), `lambda` (10^5), `p` (0.01), `half_window` (20), `decreasing` (`true`), `lls` (`false`)	the estimated baseline, in the same positions

Function	Options (all optional)	Result
`noise_level(values)`	—	σ̂ = 1.4826·MAD of the successive differences / √2
`find_peaks(values, #{…})`	`min_height`, `min_prominence` (0), `min_distance` (1, in points) — in the values' own units	an array of `#{ index, height, prominence, left_base, right_base, width }` (positions from 0; width at half prominence, in points)
`fit_peaks(x, y, #{…})`	`shape` (`"gaussian"` …, as for `spectrum-peaks`), `baseline` (`"linear"`, the default, `"constant"`, `"none"`), `centres` (starting centres; else the `peaks` most prominent maxima, at least 5 noise levels prominent), `fwhm` (starting width with `centres`), `common_width`, `spacing`, `compare` (`false`)	`peaks` (`#{ centre, centre_se, height, height_se, fwhm, fwhm_se, area, area_se, parameters, parameter_se }`), `parameter_names`, `baseline`, `baseline_se`, `n`, `parameters`, `sse`, `sigma`, `r_squared`, `aicc`, `bic`, `converged`, `termination`, `fitted`, and with `compare` a `comparison` array (`#{ peaks, aicc, delta_aicc, f, p_value }`)

```
let t = open_sample("Hald cement");
let x = #{};
for name in t.columns() {
    if name != "Heat (cal/g)" { x[name] = t.column(name); }
}
for m in best_subsets(t.column("Heat (cal/g)"), x, #{ models_per_size: 2 }) {
    print(`${m.predictors}: Cp = ${m.cp}`);
}
```

Signal processing functions

These work on arrays of evenly spaced samples (such as `t.column("Microphone")`), which must be complete (no missing values), and return arrays or maps of arrays. Options use the same names as the signal reports' settings below; `sampling_rate` (1) sets the unit of frequency.

Function	Options (all optional)	Result
`fft(x)`, `fft(re, im)`, `ifft(re, im)`	—	`re`, `im`, `amplitude`, `phase` of the discrete Fourier transform (any length; the inverse is scaled by 1/n)
`spectrum(x, #{…})`	`sampling_rate`, `window` (`"rectangular"`, the default here, `"hann"`, `"hamming"`, `"blackman"`, `"flat-top"`, `"kaiser"` with `window_beta`, `"tukey"` with `window_alpha`), `detrend` (`"none"`, `"mean"`, `"linear"`), `fft_length`, `sides` (`"one"`, `"two"`), `scaling` (`"density"`, `"spectrum"`)	`frequency`, `amplitude`, `phase`, `power`, `resolution`

Function	Options (all optional)	Result
`welch_spectrum(x, #{…})`, `spectrogram(x, #{…})`	as `spectrum`, plus `segment_length` (256), `overlap` (samples; half a segment for Welch, an eighth for the spectrogram); Hann and Tukey (α = 0.25) windows by default	Welch: `frequency`, `power`, `segments`; spectrogram: `frequency`, `time` (segment centres), `power` (one array per segment)

Function	Options (all optional)	Result
`filter_signal(x, #{…})`	`design` (`"butterworth"`, `"chebyshev1"`, `"chebyshev2"`, `"elliptic"`, `"fir"`, `"fft"`, `"notch"`), `band` (`"lowpass"`, `"highpass"`, `"bandpass"`, `"bandstop"`), `low` (or `cutoff`) and `high`, `order` (4), `ripple_db` (1), `attenuation_db` (40), `taps` (61), `window` (Hamming), `cutoff_shape` (`"ideal"`, `"smooth"`), `quality` (30), `zero_phase` (`true`), `sampling_rate`	the filtered array
`design_filter(#{…})`	as `filter_signal`, plus `points` (512)	`frequency`, `gain`, `gain_db`, `phase` (one forward pass), and `sections` (`[b₀, b₁, b₂, a₀, a₁, a₂]` each) or `taps`

Function	Options (all optional)	Result
`convolve(a, b)`, `convolve(a, b, mode)`, `circular_convolve(a, b)`	`mode`: `"full"` (the default), `"same"` (centred, the length of `a`), `"valid"`	the convolution
`cross_correlation(x, y, #{…})`, `autocorrelation(x, #{…})`	`scaling` (`"coefficient"`, the default, `"biased"`, `"unbiased"`, `"none"`), `max_lag`	`lag`, `value`, `lag_of_max`, `max_value`; r(ℓ) = Σ x(t+ℓ)·y(t), so a positive lag means `x` follows `y`
`deconvolve(y, kernel, #{…})`	`method` (`"wiener"`, the default, with `noise_ratio` (0.01); `"richardson-lucy"` with `iterations` (30), for non-negative data)	the deconvolved arra
`hilbert(x, #{…})`	`sampling_rate`	`envelope`, `phase` (unwrapped, radians) `frequency` (instantaneous, one fewer value), `imaginary` (the Hilbert transform)

Function	Options (all optional)	Result
`wavelet_decompose(x, #{…})`, `wavelet_reconstruct(decomposition)`	`wavelet` (`"db4"`), `extension` (`"symmetric"`, `"periodic"`, `"zero"`), `levels` (as many as the length allows)	a map with `approximation`, `details` (coarsest first), `wavelet`, `extension`, `length`, which `wavelet_reconstruct` turns back into the signal
`wavelet_denoise(x, #{…})`	`wavelet`, `extension`, `levels`, `threshold_rule` (`"universal"`, `"sure"`, `"fixed"` with `threshold`), `shrinkage` (`"soft"`, `"hard"`), `noise` (`"finest"`, `"each-level"`)	`signal`, `thresholds` and `noise_sd` (per detail level, coarsest first), `share_zeroed`
`cwt(x, #{…})`	`wavelet` (`"morlet"`, `"mexican-hat"`), `scales` (an array of scales in samples, or a count (32) spaced on a log scale from 1 to n/4), `sampling_rate`	`scales`, `frequency`, `coefficients` (one array per scale)

```
let t = open_sample("Test bench signals");
let mic = t.column("Microphone").extract(0, 1000);   // run A
let s = spectrum(mic, #{ sampling_rate: 500.0, window: "hann" });
let low = filter_signal(mic, #{ band: "lowpass", cutoff: 80.0, sampling_rate:
500.0 });
```

Other functions

Function	Result
`parse_json(text)`	JSON text as script values (`null` is `()`).
`to_json(value)`	A value as JSON text.
`args`	Command-line arguments after the script path.

Limits and safety

Each run is limited to 1 000 000 000 operations (time inside an analysis counts as one) and 64 nested calls. Programs embedding the engine can also cap string, array and map sizes (`stat_script::Limits`); these are off by default because Rhai enforces them by re-measuring every array after each change, which makes building a large array very slow. Scripts read and write files only through the functions above: `import` and `eval` are not available.

Performance

`examples/scripts/benchmark.rhai` times each kind of script work and prints one line per step in milliseconds:

```
cargo build --release -p stat-script
target/release/anovacraft-run examples/scripts/benchmark.rhai 100000
target/release/anovacraft-run examples/scripts/benchmark.rhai 200000
```

Doubling the row count should roughly double every line; a step whose time quadruples does not scale. Use a release build: a debug build is many times slower. Reference times from a cloud Linux container, release build:

Step	100 000 rows	200 000 rows
Script arithmetic loop	17 ms	33 ms
Build two arrays element by element	178 ms	313 ms
Make a three-column table	51 ms	114 ms
Select rows with a test function	101 ms	279 ms

Step	100 000 rows	200 000 rows
Sort by two keys	71 ms	221 ms
Summarize by group	20 ms	52 ms
Linear model, continuous and 5-level factor	265 ms	533 ms
100 small model fits (36 rows each)	7 ms	7 ms

Analyses run the same compiled code as the application, so a script adds only the cost of turning its settings into a request (microseconds). Time goes into script-level loops over rows: a test function in `select` or `exclude` builds a map of the row's values for every row, so prefer table operations (`subset` with `matching` conditions, `summarize`, `sort`) on large tables. `tests/scripts.rs` checks that building a 100 000-element array stays fast, so a quadratic slowdown is caught.

Settings

File ▸ Settings… opens the Settings window.

- **Colour scheme:** follow the system, light or dark. Plots use matching light and dark palettes.
- **Text size:** standard, large or extra large. The whole interface scales, including report tables and plot labels.
- **Tooltip delay:** short (½ second), medium (1 second, the default) or long (2 seconds). This is how long the pointer must rest on a control before its explanation pops up. Tooltips appear only once the pointer is still, so they stay out of the way while you move across the window, and a click hides them. Moving from one control to the next waits the full delay again before the next explanation appears. The value readouts that follow the pointer over plot points and bars are not delayed.
- **Mark p-values below 0.05 with ★** (on by default): report tables show significant p-values with a ★ in strong text.
- **Separate thousands with commas** (off by default): report tables write numbers with five or more digits before the decimal point as 184,150,146.
- **Line up decimal points in each column** (off by default): numbers in a report table column share one decimal-point position.
- **Open files directly when the table starts in the top-left cell** (off by default): text files and workbooks whose table has its names in row 1 and its values right below open without the Open Data dialog, and grey images within 1024 × 1024 pixels open without the Open image dialog; other files still show them.
- **Show getting-started tips:** the short tips shown above the data the first time you open a dataset, and the line on how to work plots above the reports; "Got it" hides them, and this brings them back.
- **Show frame time** (off by default, under *Troubleshooting*): the status bar shows how long the window took to draw last time and, when a report took noticeable time, which one was slowest (for example `Frame 14 ms · slowest: Distribution of Height 9 ms`). Use it to find the report or plot behind a sluggish window.

Copying a table or saving it as a data table always uses the plain numbers.

The scripts folder listed in the sidebar's *Scripts* section is chosen there (**Choose…**) rather than in this window.

Settings are remembered between sessions.

Keeping the manuals current

AnovaCraft has two manuals, both mdBooks in the repository that change in the same pull request as the code they describe, so they never fall behind a release:

- this **Reference Manual** (`docs/manual`), the detailed one for power users: every option, report section, method and script function;
- the **Quick Guide** (`docs/guide`), for people new to the app or to statistics: one page per menu describing every item in plain language, plus a first walk-through, a "which analysis do I need?" chooser and a glossary. Each entry links to the Reference Manual page with the details. Its pictures (named `guide-*`) are in `docs/guide/src/images` .

The rule

Any change a user can see updates the manual page that describes it, in the same pull request. That includes:

- a new analysis, report section, option, dialog setting, menu item or file format: add or extend its page;
- a change to something already finished, such as a renamed menu item, a new default, a different layout or a removed option: correct every page that mentions it, and replace or re-mark any screenshot it makes wrong;
- a feature leaving the Coming soon page: move it to its own page;
- a menu item added, renamed or moved: its entry on the Quick Guide's page for that menu (`docs/guide/src/menus/<menu>.md`), in **bold**, in a sentence or two of plain language.

`FEATURES.md` (the terse list of what the app does) changes in the same pull request too; the manual is the explanation, `FEATURES.md` the inventory.

How it is enforced

- **CLAUDE.md** states the rule, so every coding session follows it.
- The **pull request template** has a manual checkbox.

- **CI** builds both manuals on every pull request and checks them: a missing page or include, a link to a page that does not exist, an image that is not in `images/`, a picture whose proportions are not a window's (5:4 to 2:1) or whose size is forced, or a screenshot marker missing from the checklist fails the build. It also warns when a pull request changes user-facing code (`crates/app-gui`, `crates/app-model`, `crates/stat-script`) or `FEATURES.md` without touching either manual, `docs/formulas.md` or `docs/scripting.md`. A warning is a prompt to check, not a failure: internal refactors need no manual change.
- The test `embedded_manuals_match_their_sources` fails when either book changed without its PDF being rebuilt (see below).
- The GUI test `quick_guide_names_every_menu_item` (run with the other tests) opens every menu of the running app and fails if the Quick Guide does not name one of its items.

Where things go

What	Where
Pages	`docs/manual/src/<part>/<page>.md`
Table of contents	`docs/manual/src/SUMMARY.md`
Quick Guide pages	`docs/guide/src/menus/<menu>.md` and the pages beside it
Screenshots	`docs/manual/src/images/<id>.png`, rendered by `scripts/capture-manual-shots.sh` (see Screenshots)
Formula function reference	`docs/formulas.md` (included here, not copied)
Script API reference	`docs/scripting.md` (included here, not copied)

The PDFs in the app

Help ▸ Quick guide and Help ▸ Reference manual open PDFs of the two books that are built into the app (`crates/app-gui/assets/manuals`, embedded with `include_bytes!`, so every build carries them). After changing either book, `docs/formulas.md` or `docs/scripting.md`, rebuild them:

```
./scripts/build-manual-pdfs.sh
```

It builds both books with mdBook, prints each to a PDF with a title page, linked contents, in-document chapter links and a bookmark outline (`scripts/book-pdf.mjs` , which needs Node with Playwright's Chromium), and re-encodes the screenshots as JPEG at most 1100 pixels wide (quality 0.65; `MANUAL_IMAGE_WIDTH` and `MANUAL_JPEG_QUALITY` change that). It also writes `docs-stamp.txt` , a hash of every source file of the books (`scripts/manual-stamp.mjs`). Commit the three files it writes.

The test `embedded_manuals_match_their_sources` (run with the other tests) computes the same hash and fails when the Markdown changed but the PDFs were not rebuilt. When two branches both rebuilt the PDFs, resolve the merge by running the script again on the merged sources.

Previewing

Install mdBook once, then serve the book with live reload:

```
cargo install mdbook
mdbook serve docs/manual --open   # the Reference Manual
mdbook serve docs/guide --open    # the Quick Guide
```

`scripts/check-manual.sh` runs the same checks as CI.

Style

- Write for someone using the app, not building it: what to click, what they get, what it means.
- Use the app's own labels in **bold**, and menu paths as Menu ▸ Item.
- In the Quick Guide, write for someone who has never taken a statistics course: what the item is for and when to reach for it, in everyday words; leave options and formulas to the Reference Manual.
- Name the published method and, where useful, the sample dataset to try.
- Follow the project's clean-room naming: standard statistical terms only.

Knowledge pack for chat assistants

`knowledge-pack/` holds Markdown files that a user can give any web chat assistant so it can answer questions about AnovaCraft: an `llms.txt` index, a short `00-essentials.md`, and seven files covering the overview and glossary, menus and features, this manual, the validation record, design conventions, a code map built from doc comments, and the development guide, plus `anovacraft-knowledge-pack.md`, which holds all seven in one self-contained file for assistants that take a single upload. The files are generated from FEATURES.md, both manuals, `docs/validation.md`, CLAUDE.md, ROADMAP.md and the crates' doc comments; never edit them by hand.

```
python3 scripts/build-knowledge-pack.py          # regenerate
python3 scripts/build-knowledge-pack.py --check  # is it current?
python3 scripts/build-knowledge-pack.py --zip knowledge-pack.zip  # also zip
it
```

Regenerate the pack when a release is prepared, or whenever the documentation has changed a good deal.

Licence notices

`THIRD-PARTY-NOTICES.txt` at the repository root holds the copyright notices and licence texts of everything the shipped programs contain that AnovaCraft's authors did not write: the bundled fonts, icons and lexicon, and every Rust crate compiled into the app, `anovacraft-run` or the web viewer (on macOS, Windows and WebAssembly). The app and `anovacraft-run` embed it, the macOS and Windows build scripts copy it beside the program, and Publish as web app... writes it beside the viewer in the published folder. It is generated from Cargo.lock and the crates' own licence files (crates that ship none get the standard text from `scripts/licences/`); never edit it by hand.

```
python3 scripts/build-notices.py          # regenerate after changing
dependencies
python3 scripts/build-notices.py --check  # is it current?
```

The test `third_party_notices_match_cargo_lock` fails when Cargo.lock changes without the file being rebuilt. A new bundled font, icon set or data file gets an entry in the script's `ASSETS` list, with its licence file in the repository.

Screenshots

The pictures in both manuals (this Reference Manual and the Quick Guide) are rendered from the running app by one command, so they can be refreshed whenever a screen changes:

```
./scripts/capture-manual-shots.sh                       # every picture
./scripts/capture-manual-shots.sh pareto-chart arima    # only these
```

It runs the ignored test `manual_screenshots` (`crates/app-gui/src/app/gui_tests/manual.rs`), which opens each sample, report, dialog or menu in a window of 1440 × 900 points with the light colour scheme, and saves `docs/manual/src/images/<id>.png`. Reports are shown whole-window; menus and dialogs are cropped to a 16:10 close-up, so every picture keeps a normal window's proportions. If Pillow is installed, the script then shrinks each picture to a 256-colour palette. It needs a graphics driver; on a Linux server without one, install Mesa's software driver first (`apt-get install mesa-vulkan-drivers`).

`scripts/check-manual.sh` fails on a picture whose width-to-height ratio is outside 5:4 to 2:1, or whose size is forced in the page, so nothing is shown stretched or squashed. The Quick Guide's pictures (`guide-*`) are moved to `docs/guide/src/images`.

Adding one

1. Add a line for it to `REPORT_SHOTS`, `DIALOG_SHOTS` or `custom_shots` in `manual.rs`, and run the script with its id. When an ordinary GUI test already reaches the view and calls `gui.shot(…)` there, add it to `TEST_SHOTS` instead (picture, the test's shot name, the test): the script runs that test with `MANUAL_TEST_SHOTS` set, and the shot is taken at 1440 × 900 in light colours under the picture's name.
2. Look at the picture.
3. In the page, put an image line where the picture belongs, for example `![The main window with the Longley sample open](../images/window-overview.png)`.

The test draws a view's own window (a data grid or report moved out of the main window) inside the main window, as an ordinary inner window, through the same code as a native

window. Pictures with several pieces (four chart forms, say) are drawn one by one and tiled into one 1440 × 900 picture.

A picture the test cannot take is marked in the text like this until someone captures it by hand at about 1440 × 900 (on macOS, ⌘⇧4 then Space, and click the window):

> **Screenshot to add** (`example-id`): what the picture should show.

`scripts/check-manual.sh` checks that every marker is listed below and that every image the manual uses exists.

To capture

Nothing at present.

Done

Rendered by `scripts/capture-manual-shots.sh`:

ID	Page	What to show
`formula-typeset`	Data: formula columns	Longley, New formula column with `sqrt(GNP) / Unemployed ** 2`: the Reads as line under the text with the radical sign and fraction bar, and the preview of the first rows
`distribution-censored-report`	Describe	Well water arsenic, Distribution of Arsenic with Non-detect as left censoring: censored-values table, Kaplan–Meier percentiles, fits by AIC and the distribution function plot
`pk-report`	Describe	PK profiles, Pharmacokinetic parameters by Subject and Formulation with the dose column: parameter table, summary by formulation and semi-log profiles

ID	Page	What to show
`pcoa-report`	Multivariate	Shrub traits, Principal coordinates of Gower distances coloured by Growth form: eigenvalues with broken stick and the ordination plot
`report-filter-panel`	Reading a report	Plant growth, Distribution of Height with the filter panel open on Fertilizer, stepping at "Level 2 of 3: B"
`graph-drop-zones`	Graph canvas	Longley graph canvas of Employed by GNP while a column is dragged over it, the five zones drawn and the centre one highlighted
`save-plot-dialog`	Saving plots and results	Longley, the Residual Normal Quantile Plot's Save plot dialog with a size on paper of 89 mm wide, Serif font at 125% and the preview
`duplicate-on-another-table`	Documents, files and projects	Tensile tests sample, Duplicate on another table from batch A's graph onto batch C, with Test piece and Elongation (%) chosen
`plot-axis-breaks`	Working with plots	Longley graph canvas of Employed by GNP with two vertical breaks, Plot settings open at Segments and marks with the lowest piece on Log (base 10) and Zigzag marks, and the note naming the scales
`plot-axis-look`	Working with plots	Longley linear model, Plot settings with the Ticks and Spine groups open, beside the plot with outside ticks, an offset vertical axis and a π tick label
`graph-annotations`	Graph canvas	Longley, Employed by GNP with a fit line: a text annotation, an arrow with text at its tail and a rectangle, one selected (dashed outline), and the Edit key window open

ID	Page	What to show
`fatigue-report`	Reliability	Steel fatigue, Basquin lognormal fit: S–N plot with B10 and median curves and the lower bound, run-outs marked, and the residual probability plot
`reference-lines-window`	Working with plots	A control chart with a line at the mean and a shaded specification band, its Reference lines window open beside it
`row-colours-in-plot`	Row states	Clinical visits, rows coloured and marked by Arm, the bivariate plot with its "Arm = Active" and "Arm = Placebo" key
`dialog-value-labels`	Editing the data grid	Clinical visits, the Value labels dialog for Arm with 0 = Placebo and 1 = Active
`heading-charts`	Editing the data grid	Clinical visits grid with Column charts on: level bars, a histogram and the date line
`project-folders`	Documents, files and projects	Plant growth with an empty table added to its project: the sidebar tree with a folder "Week 1" holding the empty table and a report, and Plant growth's other reports listed under it
`window-overview`	A tour of the window	Main window, Longley sample, Side by side with one report
`welcome-page`	A tour of the window	Window with nothing open: start page with recent projects, the task cards and a tip
`analysis-dialog`	Using analysis dialogs	Linear model dialog, Longley: response and two predictors chosen, the summary line and Run, Cancel and Reset at its foot
`first-analysis-brushing`	Your first analysis	Norris linear model, one residual selected and ringed in both plots and the grid

ID	Page	What to show
`data-grid`	Editing the data grid	Cell being edited, one column selected, a column's right-click menu open
`import-database`	Data from databases and the web	File ▸ Import from database..., an SQLite file with its tables listed and one picked
`import-web`	Data from databases and the web	File ▸ Import from the web..., an address, an Authorization header marked secret, the summary line
`imputation-report`	Missing values and imputation	Air quality sample: EM imputation report with the pattern map and the scatter plot of imputed values
`report-options-menu`	Reading a report	Linear model report with ⋯ Options open
`response-explorer`	Response Explorer and 3-D profiler	Catalyst factorial explorer, one reference line mid-drag
`distribution-report`	Describe	Michelson: histogram, box plot and normal quantile plot
`distribution-identification`	Describe	Seal strength: goodness-of-fit table and the Weibull-with-threshold quantile plot
`capability-non-normal`	Quality	Seal strength capability with a Weibull-with-threshold fit: indices, histogram with fitted density, quantile plot
`pareto-chart`	Quality	Solder defects: Pareto chart of causes by line, cumulative line and Other bars
`cause-effect-diagram`	Quality	Solder defects fishbone diagram, one cause clicked and its rows selected
`space-filling-dialog`	Design	Design ▸ Latin hypercube (maximin)...: three factors, 25 runs, the criteria in the preview
`multi-vari-chart`	Quality	Shaft diameters multi-vari chart by batch and shaft, ranges view below

ID	Page	What to show
`linear-model-report`	Model: linear models	Longley report, Effect Pareto and estimates expanded
`best-subsets`	Model: linear models	Hald cement best subsets: models table and the BIC-by-size plot
`screening-volcano`	Model: linear models	Transcript counts screen: volcano plot of Treatment with the discoveries named
`dunnett-comparisons`	Compare	Blood counts: comparisons with the control and the interval plot
`best-comparisons`	Compare	Paint drying times: comparisons with the best and the interval plot
`crossover-equivalence`	Compare	Bioequivalence crossover: effects, equivalence table and subject profiles
`distribution-calculator`	Describe	Distribution calculator: Weibull density with the region between a and b hatched
`decision-tree-diagram`	Model: decision trees	Loan applications tree, one node clicked and its rows selected
`forest-oob-error`	Model: random forests and boosted trees	Loan applications forest: out-of-bag error by trees, importance below
`pls-report`	Model: partial least squares	Fuel spectra: factors table, PRESS plot and score plot
`svm-tuning`	Model: support vector machines	Pump sensors radial SVM: tuning table and error-by-cost plot
`mars-report`	Model: adaptive regression splines	Fuel consumption degree-2 adaptive splines: basis functions, GCV by number of terms, Response Explorer
`neural-network-report`	Model: neural networks	Polymer batches strength network: network diagram, penalty tuning plot, Response Explorer
`gaussian-process-report`	Model: Gaussian processes	Reaction yield Gaussian process: length scales, actual by predicted, contour maps
`sequential-optimization`	Model: Gaussian processes	Reaction yield proposals: best run, proposed runs, progress by round

ID	Page	What to show
`model-search`	Model: model search	Polymer batches search of tensile strength: ranking table and held-out RMSE by model
`model-comparison`	Model: comparing models	Loan applications comparison: ROC curves and calibration plot
`several-responses`	Model: several responses	Coating formulation: explorer, goals and simulation
`several-responses-stepwise`	Model: several responses	Coating formulation, terms selected stepwise: the Term Selection table
`dialog-several-responses-stepwise`	Model: several responses	Several responses dialog with Select terms stepwise for each response
`dialog-stepwise-several`	Linear models	Stepwise dialog with Several responses ticked
`correlations-heat-map`	Multivariate	Flower measurements correlations: Pearson matrix and heat map with its key
`scatterplot-matrix`	Multivariate	Flower measurements scatterplot matrix by variety: ellipses, diagonal histograms, a box selection ringed everywhere
`outlier-distances`	Multivariate	Flower measurements jackknifed distances by row: limit line, rows above it marked
`manova`	Multivariate	Flower measurements MANOVA: multivariate tests and the canonical plot coloured by variety
`item-analysis`	Multivariate	Course evaluation item analysis: reliability, item statistics with reversed items marked, inter-item heat map
`factor-analysis`	Multivariate	Ability tests promax solution: rotated pattern loadings, factor correlations and the loading plot

ID	Page	What to show
`discriminant-canonical-plot`	Multivariate	Honey origins linear discriminant analysis: canonical plot with the origins and their means, confusion matrices by role
`gaussian-mixture`	Multivariate	Cell populations Gaussian mixture: BIC plot by structure and the cluster plot with component ellipses
`latent-class`	Multivariate	Symptom checklist latent class analysis: model-selection table and the response-probability profile plot
`dimension-reduction`	Multivariate	Expression profiles t-SNE map coloured by cell type, with a box selection ringed
`shepard-diagram`	Multivariate	Town distances metric scaling: map and Shepard diagram
`functional-data`	Multivariate	Exotherm profiles functional data: summary table and smoothed curves
`design-dialog`	Design	Central composite dialog with its live preview
`extreme-vertices-dialog`	Design	Mixture dialog with Extreme vertices chosen: four components with bounds and the preview
`robust-design-dialog`	Design	Robust design dialog: L8 inner array of four control factors, L4 outer array of two noise factors
`signal-to-noise`	Design	Moulding shrinkage signal-to-noise report: summary, effects plot and ranking
`split-plot-dialog`	Design	D-optimal design dialog with one factor marked Hard in the Changes column, whole plots and the variance ratio set, and the split-plot notes
`design-constraints`	Design	D-optimal design dialog with two constraints in the Constraints box and the notes listing them

ID	Page	What to show
`design-augment`	Design	Augment design dialog folding over a screening design, with its notes
`design-evaluation`	Design	Design comparison sample: fraction-of-design-space curves of both designs with the key, and the comparison table
`factorial-analysis`	Design	Filtration rate factorial analysis: half-normal plot of effects with ME and SME, and the cube plot
`bayes-estimation`	Bayesian analysis	Response rates by site: posterior of each arm's response proportion and their difference, probabilities with the region of practical equivalence
`bayes-factor-test`	Bayesian analysis	Stroop reaction times two-sample Bayes factor: data, Bayes factor table, prior and posterior of δ
`bayes-anova`	Bayesian analysis	Stroop reaction times Bayesian ANOVA: models table and inclusion probabilities
`bayes-model`	Bayesian analysis	Eight schools hierarchical model: model, priors and posterior summary with $\hat{R}$ and ESS
`bayes-model-trace`	Bayesian analysis	Eight schools: trace plot of the between-school SD, one pane per chain, and the rank histograms
`bayes-model-dialog`	Bayesian analysis	Model ▸ Bayesian ▸ Bayesian regression or hierarchical model..., opened on the Eight schools sample
`gauge-nested`	Quality	Weld pull test nested gauge R&R: variance components, the nested ANOVA and the gauge run chart
`gauge-expanded`	Quality	Bore gauges expanded study: variance components, intervals and the gauge run chart
`gauge-linearity`	Quality	Gauge linearity report: bias plot with the fitted line, band and zero line, and the summary

ID	Page	What to show
`gauge-repeatability`	Quality	Gauge repeatability report: Cg and Cgk table and the run chart
`intraclass-correlation`	Quality	Gauge study intraclass correlation report: evaluation table and Shrout & Fleiss forms
`attribute-agreement`	Quality	Visual inspection grade report: within-appraiser and against-standard tables and the agreement plot
`attribute-gauge`	Quality	Go/no-go gauge report: bias and repeatability table and the gauge performance curve
`control-chart-phases`	Quality	Fill weights $\bar{X}$ chart with Filler setting as the phase: two sets of limits, boundary and phase names
`between-within-capability`	Quality	Coating batches between/within report: indices, variance components, values by batch
`attribute-capability`	Quality	Supplier lots binomial capability: summary, p chart, cumulative percent defective
`acceptance-sampling`	Quality	Supplier lots acceptance sampling: plan table and OC curves with the risk points
`control-chart-three-way`	Quality	Coil thickness three-way chart: subgroup means as individuals, their moving range, within-subgroup range chart
`control-chart-zone`	Quality	Coil thickness zone chart of Line speed: cumulative scores above the points, signals as diamonds
`attribute-laney`	Quality	Ward safety p chart and p' chart of readmissions: narrow limits flagging most weeks, widened p' limits
`capability-summary`	Quality	Tablet hardness capability summary page: indices, control charts, last subgroups, histogram, probability plot

ID	Page	What to show
`multivariate-chart`	Quality	Bottle filling T^2 chart of subgroups: reference and monitoring limits, subgroup 23 flagged, contribution chart
`analysis-of-means`	Quality	Heat treatment hardness by furnace and shift: furnace chart with furnace 3 flagged, interaction chart
`categorical-logistic-report`	Model: generalized, mixed and penalized models	Service ratings ordinal report: odds ratios, parallel-lines test and the predicted-probability curves against Wait
`dose-response`	Model: dose–response, method comparison and shelf life	Rotenone probit analysis: curve with the ED50 drop line and fiducial interval, effective-dose table
`method-comparison`	Model: dose–response, method comparison and shelf life	Glucose methods scatter plot with identity, Deming and Passing–Bablok lines, and the difference plot
`shelf-life`	Model: dose–response, method comparison and shelf life	Tablet stability plot: three batches with parallel lines, the lower confidence bound of batch C meeting 95%
`nonlinear-regression`	Model: nonlinear regression	Seedling growth logistic fit: estimates with Wald and profile intervals, fitted curve with confidence band and prediction limits
`nonlinear-global-fit`	Model: nonlinear regression	Compound potency global fit: shared parameters, LogEC50 per compound, model comparison
`survival-curves`	Reliability	Leukemia remission Kaplan–Meier curves with bands

ID	Page	What to show
`life-distributions`	Reliability	Seal inspections: model comparison by supplier and the Weibull probability plot with percentage axis
`life-distributions-dialog`	Reliability	Life distributions dialog with the start/end layout, frequency column and families chosen
`life-stress`	Reliability	Device-A: the life–stress plot with percentile lines down to 10 °C, and the probability plot by temperature
`degradation`	Reliability	Laser degradation: the path plot with fitted lines and the 10% threshold, and the Weibull probability plot of the pseudo failure times
`proportional-hazards`	Reliability	Leukemia with covariates: hazard ratios, the proportional-hazards check and survival curves by treatment
`survival-regression`	Reliability	Capacitor life test: distribution comparison, time ratios and fitted survival by temperature
`reliability-growth`	Reliability	Prototype failures growth report: estimates by phase, MTBF plot with a band per phase
`repairable-systems`	Reliability	Pump repairs: mean cumulative functions of the two designs with limits, difference plot
`competing-risks`	Reliability	Pump failure modes: overlaid cumulative incidence curves by site with limits, and the series-system plot with each mode's fit
`warranty-returns`	Reliability	Monthly warranty returns: observed and forecast returns per month with the prediction band
`text-exploration`	Multivariate: text analysis	Espresso machine reviews text exploration: the word cloud with one word clicked and its documents selected, and the document map coloured by cluster

ID	Page	What to show
`topic-model`	Multivariate: text analysis	Espresso machine reviews topic model: the top terms by topic table and the documents-by-topic bars
`sentiment-scores`	Multivariate: text analysis	Espresso machine reviews sentiment: the score distribution with the neutral band and the sentiment words table
`time-series-description`	Series and Signals	Monthly electricity demand series description: time series plot, ACF with spikes at 12 and 24, and the unit-root tests table
`time-series-decomposition`	Series and Signals	Monthly electricity demand multiplicative decomposition: the four panels and the seasonal indices table
`time-series-trend`	Series and Signals	Product adoption logistic trend: data, fitted S-curve and forecasts, and the coefficients table
`time-series-cross-correlation`	Series and Signals	Advertising and sales prewhitened cross-correlation: needle plot with the peak at lag 2, and the note on reading the lags
`exponential-smoothing`	Series and Signals	Hotel occupancy: the forecast plot with 80% and 95% limits under the model selection table
`arima`	Series and Signals	Reactor temperature: model search, coefficients and the forecast plot for the next 24 hours
`arima-inputs`	Series and Signals	Kiln fuel feed: Transfer Functions table, impulse response plot and the start of the forecast
`signal-spectrum`	Series and Signals: signal processing	Test bench signals Fourier spectrum: Welch summary and the spectrograms of runs A and B with the 50 and 120 Hz bands and the colour bar
`signal-filter`	Series and Signals: signal processing	Test bench signals 80 Hz low-pass filter: frequency response and the spectrum before and after

ID	Page	What to show
`digital-filter-dialog`	Series and Signals: signal processing	Series and Signals ▸ Digital filter…: signal, x and group columns, filter design, band, cut-off and order
`signal-correlation`	Series and Signals: signal processing	Accelerometer against microphone cross-correlation: peaks at a lag of 5 samples in each run
`signal-wavelets`	Series and Signals: signal processing	Strain wavelet analysis: multilevel decomposition of run A and the energy-by-level table
`image-spectrum`	Series and Signals: image processing	Machined surface scan image spectrum: the log-magnitude spectrum with the two tool-mark pairs ringed and numbered
`image-radial-profiles`	Series and Signals: image processing	Machined surface scan radial profiles at 0°, 90° and through each dominant frequency
`image-filter`	Series and Signals: image processing	Machined surface scan notch filter: the filtered image with the pit visible, and the part removed
`image-filter-dialog`	Series and Signals: image processing	Series and Signals ▸ Image frequency filter…: value, x, y and group columns, the image size, band filter and notches
`image-registration`	Series and Signals: image processing	Blurred and shifted scene registration: the shift found and the reference image
`reliability-test-plan`	Reliability	Test plans dialog (Demonstration tab) and the report with plans table and passing-probability curve
`multiway-table`	Describe	Multi-centre trial multi-way table: centre and treatment by outcome with subtotals, counts, row % and mean score change
`stratified-tables`	Compare	Multi-centre trial stratified analysis: tests by stratum, CMH tests, common odds ratio

ID	Page	What to show
`bootstrap-report`	Resampling	Repair times bootstrap report: estimates table and the bootstrap distribution of the mean with its interval
`permutation-test-report`	Resampling	Repair times permutation test of median repair time by crew, with the permutation distribution
`scatter-3d`	3-D scatter, contour and surface plots	Hillside survey 3-D scatter of elevation by easting and northing: soil type colours and shapes, size by soil depth, drop lines
`data-surface`	3-D scatter, contour and surface plots	Hillside survey filled contour plot of elevation with its colour bar and observations, and the filled 3-D surface with the floor contour map
`contour-filled`	Working with plots	Reaction yield response surface: filled contour plot with its colour bar, level lines and values, and the stationary point
`mixture-ternary`	Model: linear models	Yarn elongation ternary contour plot of the fitted mixture model, filled, with its colour bar and the blends as points
`graph-canvas`	Graph canvas	Columns on X, Y, colour and panel shelves; points plus a smoother
`categorical-logistic-dialog`	Model: generalized, mixed and penalized models	Ordinal or nominal logistic regression dialog on Service ratings: Rating as the response, the category order list, Wait and Channel chosen
`factorial-repeated-measures`	Compare	The Reaction times by task and cue report: univariate tests with GG/HF columns, the sphericity table and the simple effects of Task within each age group
`nested-comparison`	Compare	The Dendrite lengths per mouse report: group means, tests, variance components and nested ANOVA

ID	Page	What to show
`export-reports-dialog`	Saving plots and results	The Export reports dialog opened from a Plant growth report's Export… button, with Word document chosen and Letter paper
`view-windows`	Documents, files and projects	Longley in the main window, its data grid and a bivariate report each in a window of its own (drawn inside the main window, as the tests draw them), a box selection highlighted in both
`project-tree`	Documents, files and projects	Plant growth project with "Summary by Fertilizer" and "Selected rows" under the source in the sidebar, its tabs grouped under the project's name, and the Source changed banner above the summary's grid
`keep-in-project`	Documents, files and projects	Summarize by group on Plant growth, Fertilizer as the group, Height summarized, Keep in this project ticked and the summary line naming the new table and project
`view-window-placeholder`	Documents, files and projects	A dataset whose data grid is in its own window: the placeholder panel with Show window and Bring back here, and the window symbol in the sidebar
`dashboard-overview`	Dashboards	Greenhouse trial sample on its dashboard: toolbar, Greenhouse and Fertilizer filters, heading, note and four report tiles, the climate tiles saying "not filtered by Fertilizer"
`dashboard-move-tile`	Dashboards	A report tile dragged by its strip: column guides and the outline where it will land
`dashboard-filter`	Dashboards	The Greenhouse filter's menu open with its level check boxes, tile strips showing the rows kept

ID	Page	What to show
`dashboard-switcher`	Dashboards	Greenhouse trial dashboard with the Humidity picker on Temperature, the distribution tile following it and the scatter-plot tile saying "not switched: Humidity"
`dashboard-web-viewer`	Dashboards	The Greenhouse dashboard in the viewer the published web app runs: top bar (title, "changes are not saved"), filters and tiles, no editing controls
`publish-build-viewer`	Dashboards	Publish as web app… before the viewer is built: the Build the web viewer window with the command, Copy and Build it now
`join-dialog`	Table operations	Data ▸ Merge ▸ Join… with keys chosen and the preview
`area-under-curve-report`	Describe	Plasma concentration sample: the Area under the Curve table by formulation, the peaks table and formulation A's curve with the baseline line
`many-tests-report`	Compare	Gene screen sample: the Two groups, many variables report with the Tests by Variable table (smallest p-value first) and the volcano plot, significant genes named
`adjust-p-values-report`	Compare	Trial endpoints sample: the Multiple Testing Summary, Adjusted p-values table and ordered p-values against the Benjamini–Hochberg line
`guide-menu-compare`	Quick Guide: Compare	The Compare menu open
`report-filter-switcher`	Reading a report	Distribution report with a row filter chip and the column picker line

ID	Page	What to show
`graph-distribution-layers`	Graph canvas	Seal strength sample: peel strength histogram, dot plot, normal probability plot, and a Weibull probability plot on a log horizontal scale
`graph-brackets`	Graph canvas	Reading trial sample: scores by programme as a swarm with Tukey–Kramer significance brackets
`graph-estimation`	Graph canvas	Reading trial sample: Cumming estimation plot of the four programmes against Control
`graph-flows`	Graph canvas	Parcel flows sample: Sankey diagram of origin, hub and destination sized by parcels, Hub A clicked
`graph-cluster-map`	Graph canvas	Regional climate sample: clustered heat map with row and column dendrograms
`graph-round-forms`	Graph canvas	Harbour weather wind rose and donut chart (one slice selected), Soil texture ternary plot, Household budget waterfall chart
`script-library`	Scripting and recording	Sidebar Scripts section: folder scripts, a project script, right-click menu with Keep in project
`batch-run`	Scripting and recording	Run script over a folder window after a run: folder, pattern, output with one failed file

Automated GUI tests

The application's screens are tested automatically, without a window or a graphics card, on every pull request. Each test starts the whole app, clicks menus, buttons and drop-down lists, types into fields and drags axes the way a person would, then checks what is on screen and what the app did. A change that breaks a dialog or a plot interaction fails CI before it reaches anyone.

Running them

The GUI tests run with the rest of the test suite:

```
cargo test --workspace --no-fail-fast
```

To run only the GUI tests:

```
cargo test -p app-gui gui_tests
```

Pictures of the screens

Set `ANOVACRAFT_GUI_SHOTS` to a folder to also save a picture of the window at each point a test marks:

```
ANOVACRAFT_GUI_SHOTS=gui-shots cargo test -p app-gui gui_tests
```

Rendering the pictures needs a graphics driver: a Mac uses its own (Metal), and on a Linux machine without a graphics card, Mesa's software Vulkan driver (`mesa-vulkan-drivers` on Debian and Ubuntu) works. The pictures are a record for reviewers, not a comparison: a test fails only on what it checks, never on a pixel difference, so fonts or graphics drivers that draw slightly differently on another computer do not fail it.

For design reviews, `feedback_tour` (in `gui_tests/tour.rs` , run on demand) saves 60 numbered pictures of the most used screens: the welcome page and data grid, every top

menu open, the common dialogs, a report from each main sample, the graph canvas, scripts and settings, and a few dark-theme views:

```
ANOVACRAFT_GUI_SHOTS=feedback-shots cargo test -p app-gui feedback_tour -- --
ignored
```

Where they live

The tests are in `crates/app-gui/src/app/gui_tests/`, grouped by area (dialogs, documents, plots, scripts). They start the app with the default settings, so the settings saved on your computer never change what they see, and they never write to it.

Each test finds controls by the text a screen reader would read: a button's label, a window's title, a check box's text. Controls drawn without text, such as the axis rulers, are given an accessible name so tests (and screen readers) can find them.

Adding tests

A pull request that adds or changes a dialog, a report interaction or a plot gesture adds a GUI test for it, and a fix for a GUI bug adds a test that fails without the fix.

Web build

AnovaCraft runs in a web browser through WebAssembly as the **read-only dashboard viewer** that Publish as web app… puts in its folder. This page explains how that build is made and checked, and what differs from the desktop app at run time.

Building the viewer

```
./scripts/build-web-viewer.sh            # writes target/web-viewer
./scripts/build-web-viewer.sh --serve    # also publishes the Greenhouse sample
                                         # and serves it on
http://localhost:8000
```

The script needs the `wasm32-unknown-unknown` target (it adds it with rustup if missing) and `wasm-bindgen-cli` at exactly the version of the `wasm-bindgen` crate in `Cargo.lock` (it installs that with `cargo install` when it is missing). It builds `app-gui` in release mode for that target, runs `wasm-bindgen --target web`, and shrinks the result with `wasm-opt` when binaryen is installed. The two files it writes, `anovacraft_viewer.js` and `anovacraft_viewer_bg.wasm`, are what Publish as web app… copies. The app looks for them in, in order:

1. the folder named by the `ANOVACRAFT_WEB_VIEWER` environment variable;
2. a `web-viewer` folder beside the program, one level up (so a Cargo build in `target/release` finds `target/web-viewer`), or in a macOS app bundle's `Resources`.

Without them it writes nothing and opens the "Build the web viewer" window (`app/viewer_build.rs`), which shows this command and, in a source checkout, runs it in the background and then carries on with the publish.

`app-gui --viewer project.acraft` shows a project the way the viewer does, in a desktop window; `app-gui --publish-sample folder` writes the published Greenhouse sample to a folder (what `--serve` serves).

Checking it

```
./scripts/check-web-build.sh
```

runs `cargo check` for `wasm32-unknown-unknown` on the engine crates, `app-model` and `stat-script` without their default features, `stat-viz` with its `egui` feature, and `app-gui`. It is not part of CI; run it after changing dependencies or adding platform-specific code. The viewer's logic is tested natively (`gui_tests/viewer.rs` drives it with `egui_kittest`); the only browser-only check is loading a published folder in a browser.

How the app builds for the web

Crate	For `wasm32-unknown-unknown`
All engine crates (`stat-core` … `stat-timeseries`, `stat-strd`)	Compile unchanged.
`stat-viz`, with and without its `egui` feature	Compiles unchanged.
`app-model`	Without its default features: no database drivers or web access (its `sqlite`, `postgres`, `mysql`, `web` and `odbc` features).
`stat-script`	Without its `external-data` feature (which passes app-model's default features on).
`app-gui`	Compiles; on this target it starts as the viewer.

What `app-gui` does differently on this target:

- **Dependencies:** the database drivers and web import, the native file dialogs (`rfd`) and the clipboard (`arboard`) are native-only dependencies. `web_shims` stands in for the dialogs (always cancelled) and the clipboard (always empty); the viewer offers no command that uses them.
- **Entry point:** `main` calls `web_main::start`, which fetches the project file next to the page (`project.acraft`, or the one a `?project=` query parameter names), reads it with `load_project_file` and starts `eframe::WebRunner` on the page's `anovacraft_canvas`, drawing with WebGL. Errors and panics are shown in the page's `anovacraft_status` line.

- **Viewer mode:** `StatsApp::new_viewer` opens the project and shows its dashboards full window (`app/viewer.rs`): no menus, sidebar, tabs or windows of their own. Its dashboards are read-only (`OpenDashboard::read_only`): the toolbar keeps the name and Undo/Redo; tile strips have no move handle or buttons; there is no resize grip; and only filter, picker and tile display edits are applied (`DashboardOutcome::keep_view_changes`).
- **Settings:** the viewer uses the default settings and saves none.

At run time

- **Threads:** the engine's `rayon` loops (random forests, bootstrap and permutation loops) run on one thread, which rayon does by itself when threads are unavailable. Results are the same (each tree and replicate has its own seed); they take longer. The script window, batch runs and external data, which start threads, are not part of the viewer.
- **Clock:** `std::time::SystemTime::now` and `Instant::now` panic on this target. Nothing the viewer draws calls them (the engine uses them only in tests); `html_export::export_date` must not be called there.
- **Files:** `std::fs` compiles but every call fails; the project comes from the page.
- **Graphics:** the web build uses eframe's WebGL (glow) renderer, which every current browser has; the desktop app keeps wgpu.

Size

The build uses the `wasm-release` profile (optimized for size, with link-time optimization, stripped): about 20 MB of WebAssembly before `wasm-opt` , 7 MB gzipped, which web servers usually apply on their own. It links every analysis, since `run_report` dispatches to all of them, plus egui and the bundled fonts. Leaving out analyses a dashboard does not use would need `run_report` split by feature; that is not worth doing for now.